

React: Building User Interfaces

Notes and exercises for all 41 chapters

Made on October 5, 2026

Contents

About this course

Level 1: The Basics

- 01 Getting Started
- 02 JSX
- 03 Components
- 04 Props
- 05 Conditional Rendering
- 06 Rendering Lists
- 07 Events
- 08 State
- 09 Forms
- 10 Project: To-Do App, the React Way

Level 2: Thinking in React

- 11 Updating Objects and Arrays in State
- 12 How Rendering Works
- 13 Lifting State Up
- 14 Thinking in React
- 15 Styling
- 16 Tailwind CSS
- 17 Effects
- 18 Fetching Data
- 19 Refs
- 20 Custom Hooks
- 21 Project: Recipe Finder

Level 3: Real Apps

- 22 Context
- 23 useReducer
- 24 React Router
- 25 TypeScript Patterns for React
- 26 Error Boundaries and Suspense
- 27 Performance
- 28 Testing
- 29 Project: Online Bookstore

Level 4: The React Toolbox

- 30 Axios

- 31 TanStack Query
- 32 Zod
- 33 React Hook Form
- 34 Zustand
- 35 Redux Toolkit
- 36 TanStack Router
- 37 Project: Task Board

Level 5: Mastery

- 38 Accessibility
- 39 Next.js and Server Components
- 40 Deploying
- 41 Final Project

Hints

About this course

React is a JavaScript **library** (code someone else wrote that you can use in your own projects) for building user interfaces. You build a page from small, reusable pieces called **components**, like snapping LEGO bricks together. When your data changes, React updates the page for you.

This course takes you from your first component to building and deploying full apps with the tools real teams use. There are 41 chapters in five levels. They go in order, and each one builds on the ones before it.

Before you start: finish at least Levels 1 to 3 of the JavaScript course, and the whole TypeScript course. React uses a lot of modern JavaScript, like array methods, destructuring, modules, and `fetch`. And this course is written in TypeScript from the very first chapter.

The hints for the exercises are in the [Hints section](#) at the end. Try each exercise on your own first, and read one hint at a time.

Level 1: The Basics

Goal: build small, interactive pages out of components.

01 Getting Started

What is it?

React is a JavaScript library for building **user interfaces**, or **UI**: the part of an app you see and click, like buttons, lists, and forms.

You describe what the screen should look like for your data. React makes the real page match that description, and keeps it matching whenever the data changes.

In this course, you'll write React with **TypeScript**, just like most professional React teams do. Everything you learned in the TypeScript course works here.

Why does it matter?

Remember the stadium scoreboard from your to-do app in JavaScript chapter 24? You changed the `tasks` array, then called `render()` to redraw the page. That `render()` function was a lot of work. You emptied the list, built every `<li>` with `createElement`, and had to remember to call it after every change.

Here's a tiny version of that problem: a button that counts its own clicks, written the plain DOM way (JavaScript chapters 20 and 21):

```
let count = 0;
const button = document.querySelector("#counter");

button.addEventListener("click", () => {
  count = count + 1; // job 1: change the data
  button.textContent = `Clicked ${count} times`; // job 2: update the page yourself
});
```

You have two jobs: change the data, *and* update the page. Forget job 2, and the page shows the wrong number. In one button, that's easy to remember. In a real app, with dozens of things on screen depending on the same data, it's where most bugs come from.

Here's the same button in React:

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Don't worry about the details yet. The HTML-like part is [chapter 02](#), `onClick` is chapter 07, and `useState` is chapter 08. Just notice what's missing: there's no `querySelector` and no `textContent`. You only describe what the button should look like for any `count`. When `count` changes, React updates the page for you.

(And there are no type annotations, either. TypeScript infers that `count` is a number from the `0`, just like in TypeScript chapter 02.)

That's what React gives you:

- **The page always matches your data.** You change the data. React does the redrawing, so you can't forget.
- **Reusable pieces.** You build a piece of UI once, like a product card, and use it as many times as you want. These pieces are called **components**.
- **A huge community.** React is the most popular tool for building web apps. It's used by a lot of companies, and there's a library for almost every job (Level 4 of this course covers the popular ones).

Real-world example

Think about getting a taxi.

Getting a taxi	React
You tell the driver the address	You describe what the screen should show
The driver picks the turns	React works out which parts of the page to change
You change your mind, and the driver re-routes	Your data changes, and React updates the page

Getting a taxi	React
Giving every turn yourself: "left here, now right..."	Plain DOM code, like <code>createElement</code> and <code>textContent</code>

Saying *what* you want, and letting a tool work out *how*, is called **declarative** code. Giving every step yourself is called **imperative** code. React is declarative. That's the big shift in this course.

How it works

What you need

- **The TypeScript course**, finished. This course builds on it from the first line.
- **Node.js**, version 20.19 or newer. (If you have version 22, it needs to be 22.12 or newer.) Check yours in a terminal:

```
node --version
```

If it prints something older, like `v18.20.4`, install the latest **LTS** version from nodejs.org. **LTS** means "long-term support": the stable version most people use.

If you installed Node with **nvm** (a tool that keeps several Node versions side by side), `nvm list` shows the versions you have, and `nvm use` followed by a version number switches to it.

- **VS Code** and **Chrome or Edge**, the same as in the JavaScript and TypeScript courses.

Create your practice app

You'll write all your React practice in one app. The tool that creates it is **Vite** (say "veet"), which you used in TypeScript chapter 12. Vite sets up a project and runs a small web server on your computer while you work.

Open a terminal inside this `React` folder and run:

```
npm create vite@latest
```

Vite asks you a few questions. Use the arrow keys and Enter to answer:

Question	Your answer
Project name	<code>playground</code>
Select a framework	React

Question	Your answer
Select a variant	TypeScript (the first choice)
Which linter to use?	Oxlint (the first choice)
Install with npm and start now?	Yes

A few notes on those choices:

- Pick plain **TypeScript**, not "TypeScript + React Compiler". The React Compiler is a tool that makes apps faster automatically, and chapter 27 covers it.
- A **linter** is a spell-checker for bugs. You used one called ESLint in JavaScript chapter 50. **Oxlint** does the same job, just faster. Either one is fine.

(Vite's questions change a little between versions. If yours look different, pick React and TypeScript, and the default for anything else.)

Vite creates a `playground` folder, installs React and TypeScript, and starts the server. You'll see something like:

```
VITE v8.3.1 ready in 350 ms

→ Local: http://localhost:5173/
```

Ctrl + click the `Local` address to open it. You'll see Vite's welcome page with a **Count is 0** button. Click it. That's a React app running on your computer.

`localhost` means "this computer". `5173` is the **port**, like an apartment number for programs on your computer. If `5173` is busy, Vite picks the next number, like `5174`.

Stopping and starting again

The server keeps running as long as the terminal stays open.

- **To stop it:** click in the terminal and press **Ctrl + C**.
- **To start it again later:** open a terminal *inside the `playground` folder* and run:

```
npm run dev
```

If you answered "No" to "Install with npm and start now?", run these instead: `cd playground`, then `npm install`, then `npm run dev`.

A tour of your project

Open the `playground` folder in VS Code. These are the files that matter for now:

```
playground/
├─ index.html      the only HTML page. It has an empty <div id="root">
├─ package.json    your project's settings and packages (JavaScript chapter 50)
├─ tsconfig.json   TypeScript's settings, split across three files (more below)
├─ tsconfig.app.json settings for your app's code, in src/
├─ tsconfig.node.json settings for vite.config.ts
├─ vite.config.ts  Vite's settings. It turns on React support
├─ public/        files the browser gets exactly as they are, like the tab icon
└─ src/          your code lives here
    ├─ main.tsx   the starting point: puts your app on the page
    ├─ App.tsx    what your page shows
    ├─ App.css    styles for App
    ├─ index.css  styles for the whole page
    └─ assets/    images your code uses
```

You'll also see `node_modules` (the installed packages, never edit them), `package-lock.json`, `.gitignore`, `.oxlintrc.json` and a `README.md`. You can ignore those for now.

What's `.tsx`? You know `.ts` files from the TypeScript course. A `.tsx` file is a TypeScript file that can also contain HTML-like tags. [Chapter 02](#) is all about those tags. Any file with tags in it must end in `.tsx`.

How your app gets onto the page

When you open the page, three files work together, one after the other.

1. `index.html` has almost nothing in it:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
```

An empty `<div>`, and a script. The empty `<div>` is where React will draw everything.

Browsers can't run `.tsx` files. Vite translates `main.tsx` into plain JavaScript on the fly, the moment the browser asks for it.

2. `src/main.tsx` is that script. Here it is, exactly as Vite made it:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Line by line:

Code	What it means
<code>import { createRoot } from 'react-dom/client'</code>	Load <code>react-dom</code> , the part of React that draws on web pages
<code>import './index.css'</code>	Load the page's styles. Vite lets you import CSS files like this
<code>import App from './App.tsx'</code>	Load your app (a default export, from JavaScript chapter 29)
<code>document.getElementById('root')!</code>	Find the empty <code><div></code> . The <code>!</code> is explained below
<code>createRoot(...)</code>	Tell React: "this <code><div></code> is yours now"
<code>.render(<App />)</code>	Draw <code>App</code> inside it
<code><StrictMode></code>	A helper that warns you about common mistakes while you work. It doesn't show anything on screen

What's the `!`? You met it in TypeScript chapter 12. `getElementById` might find nothing and return `null`, so its type is `HTMLElement | null`. `createRoot` needs a real element, so TypeScript would complain. The `!` (the **non-null assertion**) tells TypeScript "trust me, it's there". Here, that's a safe promise, because the `<div id="root">` is right there in `index.html`. (In the exercises, you'll break that promise on purpose.)

Vite's files leave out semicolons and use single quotes. Both styles work. This course uses double quotes and semicolons, like the JavaScript and TypeScript courses.

3. `src/App.tsx` is what the page actually shows. Vite's version is long, because it draws the whole welcome page. But its shape is simple:

```
function App() {
  // ...some setup...
  return (
    // ...what to show on the page...
  )
}

export default App
```

`App` is a **function that returns what should appear on the screen**. That's a **component**, and every React app is built out of them. You'll write your own in chapter 03.

So the whole path is: the browser opens `index.html`, which loads `main.tsx`, which draws `App` inside `<div id="root">`. That one `<div>` holds your entire app.

Your first change

Replace *everything* in `src/App.tsx` with this:

```
function App() {
  return <h1>Hello, React!</h1>;
}

export default App;
```

Save the file and look at the browser. The page changes by itself, and you didn't have to refresh. This is called **hot module replacement**, or **HMR**: when you save, Vite swaps only the file you changed into the running page.

The page probably still has the starter's colors and layout, and a dark background if your computer uses dark mode. Those styles come from `src/index.css`. To start with a clean, plain page, delete everything *inside* `src/index.css` and save. Keep the empty file, because `main.tsx` still imports it.

`App.css` and the `assets` folder aren't used anymore, since your new `App.tsx` doesn't import them. You can delete them, or leave them. They do no harm.

Checking your types

Here's something from TypeScript chapter 01 that matters a lot now: **Vite doesn't check your types**. Just like `node file.ts`, Vite removes the types and runs what's left. A page with type errors can still look fine in the browser.

So you check types in two other ways:

- **In VS Code**, all the time. Type errors get a red squiggly underline. Hover over it to read the message.

- **In the terminal**, for the whole project at once. Open a second terminal inside `playground` (leave the first one running the server) and run:

```
npx tsc -b
```

No output means no problems.

Why `-b`? Vite splits TypeScript's settings into three files. The main `tsconfig.json` doesn't list any code itself. It just points to the other two files (these pointers are called **references**). `-b`, short for "build mode", tells `tsc` to follow those pointers and check everything. TypeScript chapter 11 explains the settings inside.

`npm run build`, which makes the final version of your app for putting online (chapter 40), runs `tsc -b` first. So a type error stops you from publishing a broken app.

Two settings in this project are worth knowing now:

- **Strict mode is on.** You won't see `"strict": true` in the files, because TypeScript 6 turns it on by default. You get all the checks you learned in the TypeScript course.
- **Unused variables and imports are errors.** If you import something and never use it, `tsc -b` reports:
`'Ex2' is declared but its value is never read.` Delete what you don't use.

React Developer Tools

Install the free **React Developer Tools** extension for Chrome or Edge. It adds a **Components** tab to the DevTools panel (press `F12`), where you can see each component on the page. It'll be really useful from chapter 03 on.

React is a library, not a framework

A **framework** is a complete kit that decides how your whole app is organized: pages, data, the server, and more. React only handles one job: what's on the screen.

For everything else, like moving between pages, loading data, and checking forms, you pick extra libraries. That's why this course has a whole level (Level 4) on the popular ones. There are also frameworks built *on top of* React, like Next.js (chapter 39).

Common mistakes

1. Opening `index.html` by double-clicking it

You get a blank white page. Browsers can't read `.tsx` files or find npm packages by themselves. Vite translates your code while `npm run dev` is running. Always open the `localhost` address instead.

2. Running commands in the wrong folder

If you run `npm run dev` in the `React` folder instead of `React/playground`, npm complains that it can't find `package.json` (you'll see `ENOENT`), or says `Missing script: "dev"`. Open the terminal inside `playground`. In VS Code, you can right-click the folder and choose **Open in Integrated Terminal**.

3. Skipping `npm install`

On Windows:

```
'vite' is not recognized as an internal or external command,  
operable program or batch file.
```

(On a Mac, it says `vite: command not found`.) The packages aren't installed yet. Run `npm install` once, then `npm run dev`.

4. Using an old version of Node.js

Vite needs a recent Node.js. With an old one, `npm create vite` may crash with strange errors, or warn you about your Node version. Check with `node --version` and update if needed.

5. Following an old tutorial that uses `create-react-app`

Lots of older tutorials start with `npx create-react-app my-app`. That tool is **deprecated** (officially retired) since 2025, and React's own docs no longer recommend it. Use Vite, like you did here.

6. Closing the terminal while you work

The browser shows **"This site can't be reached"** or **"localhost refused to connect"**. Closing the terminal stops the server. Start it again with `npm run dev`.

7. Thinking "the page works, so my types are fine"

Vite shows your page even when your code has type errors. Keep an eye on the red squiggles in VS Code, and run `npx tsc -b` before you call something done.

8. Running `npx tsc` without `-b`

In a Vite project, plain `npx tsc` checks *nothing* and prints nothing, because the main `tsconfig.json` has no code in it. That silence looks like "no errors", even when there are some. Always add `-b`.

Quick recap

- React builds user interfaces. You describe what the screen should show, and React keeps the page matching your data.

- Create a React + TypeScript app with `npm create vite@latest` (React, then TypeScript), and run it with `npm run dev` from inside the project folder.
 - The path is `index.html` → `main.tsx` → `App.tsx`. React draws your whole app inside one `<div id="root">`. Files with tags in them end in `.tsx`.
 - A component is a function that returns what should appear on the screen. `App` is your first one.
 - Vite doesn't check types. Watch the red squiggles in VS Code, and check the whole project with `npx tsc -b`.
-

Next: try the [exercises](#), then move on to [02 JSX](#).

Exercises

How to do these:

- Do these in your practice app, `React/playground`, with `npm run dev` running.
- Make a folder for each chapter inside `src`, like `src/ch01/`, and one file per exercise: `Ex2.tsx`, `Ex3.tsx`, and so on. Start each name with a capital letter (chapter 03 explains why).
- Each exercise file looks like this:

```
function Ex2() {  
  return <h1>Your answer goes here</h1>;  
}  
  
export default Ex2;
```

- To see an exercise in the browser, make `src/App.tsx` show it:

```
import Ex2 from "../ch01/Ex2.tsx";  
  
function App() {  
  return <Ex2 />;  
}  
  
export default App;
```

`<Ex2 />` means "show whatever `Ex2` returns, right here". Chapter 03 explains how that works. To switch to another exercise, *change* the `import` line (both names) instead of adding a second one. An import you don't use is a type error in this project.

- An exercise is done when the page looks right, VS Code shows no red squiggles, and `npx tsc -b` prints nothing.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Your first React app

Create your practice app exactly as the notes describe, and open it in the browser.

1. Click the **Count is 0** button a few times.
2. In `src/App.tsx`, find the text `Get started` and change it to `My first React app`. Save. Did you have to refresh the page?
3. Open a second terminal inside `playground` and run `npx tsc -b`. What does it print?
4. Stop the server with `Ctrl + C`, then start it again.

Write down the address the terminal shows under `Local`.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): The café sign

Time for a clean slate:

1. Delete everything inside `src/index.css` (keep the empty file).
2. Create `src/ch01/Ex2.tsx` that shows a sign for a café:

```
Café Luna is open!
```

It should be a big heading.

3. Replace everything in `src/App.tsx` with the version from "How to do these", so it shows `Ex2`.

You should see a plain white page with your heading and nothing else, and `npx tsc -b` should print nothing.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Follow the trail

Read `index.html` and `src/main.tsx`. Then create `src/ch01/ex3-answers.md` and answer these in your own words:

1. Which file does the browser open first?
2. What's the `id` of the `<div>` where React draws your app?
3. Which line in `main.tsx` finds that `<div>`? What does the `!` on that line promise TypeScript?
4. Now an experiment. In `index.html`, change `id="root"` to `id="app"`, but *don't* change `main.tsx`. Save. What does the page show? Open the Console (`F12`) and write down the red error message.
5. Run `npx tsc -b`. Did TypeScript warn you about this problem? Explain why, or why not.
6. In one sentence, explain *why* the error in step 4 happens. Then change the `id` back and check that your café sign is back.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Break it on purpose

Programmers see error messages every day. The skill is reading them calmly. Copy your `Ex2.tsx` to `Ex4.tsx`, rename the function inside it to `Ex4` (in both places), change the `App.tsx` import so it shows `Ex4`, and then:

1. Delete the closing `</h1>` tag and save. What do you see in the browser? What does VS Code show on that line? Write down the file name and line number from the error. Then put the tag back.
2. Above the `return`, add `const openHour: number = "9am";` and show it in the heading, like `Open from {openHour}`. Save. Does the page work? What do VS Code and `npx tsc -b` say? Then fix the line so it has the right type.
3. Change `export default Ex4;` to `export default Ex4x;` and save. Where do you see an error this time: the browser, the Console, VS Code, `tsc`? Write down each message, then fix it.
4. Stop the server with `Ctrl + C`, then refresh the browser. What happens? Start the server again.

Save your notes in `src/ch01/ex4-answers.md`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Plain DOM code vs React

Build the same coffee counter twice, then compare.

Part 1: the plain DOM way. Make a second, React-free project, like in TypeScript chapter 12. Run `npm create vite@latest` in the `React` folder, name the project `playground-plain`, and choose **Vanilla** and **TypeScript**. It

ends up next to `playground` .

Then, using what you learned in JavaScript chapters 20 and 21, build a button that counts cups of coffee:

```
Cups of coffee today: 0
```

Every click adds one.

Part 2: React. Create `src/ch01/Ex5.tsx` in your practice app and type in the `Counter` example from the "Why does it matter?" section of the notes. Then:

- Rename the function from `Counter` to `Ex5` , and add `export default Ex5;` at the bottom, like your other exercise files.
- Put this line at the very top of the file, so React's `useState` is available:

```
import { useState } from "react";
```

- Change the button so it shows the same coffee text.

Part 3: compare. In `src/ch01/ex5-answers.md` , answer:

1. How many lines is each version?
2. In the plain version, which line updates the page? Which line does that job in the React version?
3. In the plain version, what did you have to do about `null` ? Did you need that in React? Why not?
4. Name one thing that would go wrong in the plain version if you forgot a line, but can't go wrong in React.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

02 JSX

What is it?

JSX lets you write HTML-like tags right inside your code:

```
const heading = <h1>Hello, React!</h1>;
```

There are no quotes around it, so it's not a string. And it's not real HTML either. Before your code reaches the browser, Vite translates the JSX into plain JavaScript.

(JSX is short for "JavaScript XML". XML is a stricter cousin of HTML, which explains some of the rules below. When you use JSX in TypeScript, the files end in `.tsx`, and people sometimes call it TSX. It's the same thing.)

Why does it matter?

In JavaScript chapter 20, you built a product card like this:

```
const card = document.createElement("div");
card.classList.add("card");

const title = document.createElement("h2");
title.textContent = "Notebook";

const price = document.createElement("p");
price.textContent = "Price: $3.50";

card.append(title, price);
```

It works, but you have to read every line carefully to picture what's on the page. Here's the same card in JSX:

```
<div className="card">
  <h2>Notebook</h2>
  <p>Price: $3.50</p>
</div>
```

It looks like the result. You can see the card's shape at a glance, which makes JSX quick to write and easy to read.

With TypeScript, you get a bonus: **TypeScript knows every HTML tag and attribute**. Misspell one, and VS Code underlines it before you even look at the browser. And as you'll see at the end of this chapter, JSX is also safer than building HTML by hand.

Real-world example

Think of a birthday card from a shop. The design and most of the words are already printed. There are just a few blanks for you to fill in:

Happy birthday, ____! You're __ today!

JSX works the same way:

Birthday card	JSX
The printed design and words	The tags and plain text
A blank space	Curly braces: <code>{ }</code>
What you write in the blank	A JavaScript value, like <code>{name}</code>
The same card, for a different friend	The same JSX, with different data

How it works

Where to try the examples

All the examples in this chapter go in `src/App.tsx`, inside the `App` function. The simplest one looks like this:

```
function App() {  
  return <h1>Hello, React!</h1>;  
}  
  
export default App;
```

`App` returns some JSX, and React shows it on the page. Whatever you put after `return` is what you'll see.

Keep `npm run tsc -b` handy in a second terminal, like in [chapter 01](#). Many of the mistakes in this chapter show up there, and as red squiggles in VS Code, but not in the browser.

JSX is JavaScript in disguise

When Vite translates your JSX, this:

```
<h1 className="title">Hello</h1>
```

becomes something like this:

```
jsx("h1", { className: "title", children: "Hello" });
```

`jsx()` is a React function. It makes a small object that *describes* the element: "an `h1`, with this class, containing this text." React reads these descriptions and builds the real page from them.

You'll never write `jsx()` yourself. (Older tutorials show `React.createElement()` instead. It's the same idea, from older versions of React.) But knowing the secret helps in three ways:

- **JSX is a value**, like a number or a string. You can store it in a variable, or return it from a function.
- **TypeScript can check it**. The attributes become an object, `{ className: "title", ... }`, and React comes with types that describe which properties each tag's object may have. So TypeScript checks your tags like any other object (TypeScript chapter 03).
- **The rules below make sense**. They exist because, underneath, JSX is JavaScript.

Rule 1: Return one parent element

This breaks:

```
function App() {  
  return (  
    <h1>Corner Café</h1>  
    <p>Fresh coffee every morning.</p>  
  );  
}  
// ❌ JSX expressions must have one parent element.
```

Tip: in this course, `// ❌` in a code example shows the error TypeScript reports, like in the TypeScript course. You'll see it when you hover over the red squiggly line in VS Code, and when you run `npx tsc -b`. Broken *syntax*, like this example, also shows up as an error box in the browser, because Vite can't translate the file at all. Type errors don't show up in the browser.

Why does this break? Each tag turns into a `jsx()` call, and a function can only return *one* value. You can't `return 1 2;` in JavaScript either.

The fix is to wrap them in one parent:

```
function App() {
  return (
    <div>
      <h1>Corner Café</h1>
      <p>Fresh coffee every morning.</p>
    </div>
  );
}
```

That works, but it adds an extra `<div>` to your page. If you don't need it, use a **fragment**, written `<>` and `</>`:

```
function App() {
  return (
    <>
      <h1>Corner Café</h1>
      <p>Fresh coffee every morning.</p>
    </>
  );
}
```

A fragment is an invisible wrapper. It groups the elements for React, but adds nothing to the page. Vite's starter `App.tsx` used one, too.

Rule 2: Close every tag

In HTML, some tags don't need closing, like `<img>`, `<br>`, and `<input>`. In JSX, **every** tag must be closed. For tags with nothing inside, add a slash before the `>`:

```

<br />
<input type="text" />
```

This is called a **self-closing** tag. Forget the slash, and you get:

```

// ❌ JSX element 'img' has no corresponding closing tag.
```

(`/favicon.svg` is the tab icon in your `public` folder. Files in `public` are available at `/` plus their name.)

Rule 3: Attributes use camelCase, and `class` becomes `className`

JSX attributes are really JavaScript property names. So they follow JavaScript's rules, and use camelCase (from JavaScript chapter 02):

HTML	JSX	Why
<code>class="card"</code>	<code>className="card"</code>	<code>class</code> is a JavaScript keyword (JavaScript chapter 27)
<code>for="email"</code>	<code>htmlFor="email"</code>	<code>for</code> is a JavaScript keyword too (loops)
<code>tabindex="0"</code>	<code>tabIndex={0}</code>	camelCase, and it's a number (more on that below)
<code>maxLength="20"</code>	<code>maxLength={20}</code>	camelCase, and it's a number
<code>onclick="..."</code>	<code>onClick={...}</code>	camelCase (events are chapter 07)

Two kinds of attributes keep their dashes: the ones that start with `aria-` and `data-`. You used both in your JavaScript to-do app:

```
<button aria-label="Delete task" data-id="3">x</button>
```

If you write `class` by mistake, TypeScript catches it straight away:

```
<h1 class="title">Hi</h1>
// ✖ Property 'class' does not exist on type
'DetailedHTMLProps<HTMLAttributes<HTMLHeadingElement>, HTMLHeadingElement>'. Did you mean
'className'?
```

That error looks scary, so here's how to read it. **Read the last sentence first:** Did you mean 'className'? That's usually all you need. `DetailedHTMLProps<HTMLAttributes<HTMLHeadingElement>, ...>` is just TypeScript's long name for "the list of attributes a heading can have". (The full message starts with one more line above this one. It mentions `children`, which means whatever is between the opening and closing tags.)

If you ignore the squiggle, the page may even look right. But React also warns you in the Console (F12):
`Invalid DOM property `class`. Did you mean `className`?` Always fix both kinds of warnings.

Curly braces: a window back into JavaScript

This is the most important part of JSX. Inside JSX, **curly braces { }** let you drop in any JavaScript value, like the blanks on the birthday card:

```
function App() {
  const shopName = "Corner Café";
  const coffeePrice = 3.5;
  const cups = 2;

  return (
    <div>
      <h1>Welcome to {shopName}!</h1>
      <p>{cups} coffees cost ${coffeePrice * cups}.toFixed(2)}</p>
      <p>Shout it: {shopName.toUpperCase()}</p>
    </div>
  );
}
```

The page shows:

```
Welcome to Corner Café!
2 coffees cost $7.00
Shout it: CORNER CAFÉ
```

A few things to notice:

- `{shopName}` shows what's in the variable, just like `${shopName}` in a template literal (JavaScript chapter 06).
- You can do math, and call functions and methods, like `toFixed` from JavaScript chapter 05.
- The `$` in `$7.00` is just a dollar sign. In JSX, the braces do all the work by themselves, so a `$` in front of them is plain text.
- There are no type annotations. TypeScript infers the types from the values, and checks them everywhere they're used. Hover over `shopName` in VS Code: because it's a `const`, you'll see the exact value as its type, `"Corner Café"` (a literal type, from TypeScript chapter 02).
- Do your setup (the `const` lines) *above* the `return`. Keep the JSX for showing things.

Only expressions go inside the braces

Braces take an **expression**: a piece of code that produces a value. `cups`, `cups * 2`, and `shopName.toUpperCase()` are all expressions.

A **statement** *does* something instead of producing a value, like `if`, `for`, or `const`. Statements don't work in braces:

```
<p>{if (isOpen) { "Open" }}</p>
// ❌ Expression expected.
```

When you need to choose between two values, use the ternary operator from JavaScript chapter 07. It's an expression, so it's allowed:

```
const isOpen = true;

<p>We're {isOpen ? "open" : "closed"}.</p>
// shows: We're open.
```

[Chapter 05](#) shows all the ways to show things only sometimes.

Curly braces in attributes

Braces work in attributes too. Use them instead of quotes when the value comes from JavaScript:

```
const logoUrl = "/favicon.svg";
const shopName = "Corner Café";

<img src={logoUrl} alt={shopName + " logo"} />
```

The rule is: **quotes for fixed text, braces for JavaScript. Never both.** Writing `src="{logoUrl}"` gives the image the text `{logoUrl}` as its address, so you get a broken image.

Numbers go in braces, too. Quotes always make a string, even when there's a number inside them. TypeScript knows which attributes want a number:

```
<input type="email" maxLength="50" />
// ❌ Type 'string' is not assignable to type 'number'.

<input type="email" maxLength={50} /> // ✅ a real number
```

Plain HTML doesn't care about this difference, but TypeScript does. It's the same rule as in TypeScript chapter 02: `"50"` and `50` are different types.

TypeScript knows every tag and attribute

Because React comes with types for every HTML tag, TypeScript spots typos in your JSX just like typos in your objects. And it usually suggests the fix:


```

<button onclick={() => {}}>Go</button>
// ❌ Property 'onclick' does not exist on type '...'. Did you mean 'onClick'?

<input maxLength={20} />
// ❌ Property 'maxLength' does not exist on type '...'. Did you mean 'maxLength'?

```

(The `'...'` is a shortcut for the long type name you saw in Rule 3. It's there in the real message.)

As you type an attribute in VS Code, a list pops up with the attributes that tag allows. That's TypeScript too. Use it: it's faster than remembering, and you can't misspell what you pick from a list.

What shows up on the page

Not every value appears on the page the way you might expect:

You write	The page shows
<code>{"hello"}</code>	hello
<code>{42}</code>	42
<code>{0}</code>	0 (yes, zero shows up)
<code>{true}</code> or <code>{false}</code>	nothing
<code>{null}</code> or <code>{undefined}</code>	nothing
<code>["a", "b"]</code>	ab (each item; chapter 06 does this properly)
<code>{ {name: "Pen"} }</code> (an object)	a type error (and a crash if you run it anyway)

Everything React can show (text, numbers, JSX, `true`, `false`, `null`, `undefined`, and arrays of these) has one type name: `ReactNode`. An object isn't on that list, so TypeScript stops you. You'll see `ReactNode` again in chapter 04.

Showing nothing for `true`, `false`, `null` and `undefined` sounds odd, but it's very useful. Chapter 05 uses it to hide things. Keep in mind that `0` *does* show, because it trips people up in chapter 05.

The `style` attribute takes an object

In HTML, `style` is a string. In JSX, it's a JavaScript object:

```

<h1 style={{ color: "tomato", fontSize: 32 }}>Sale!</h1>

```

The **double braces** look odd, but they're two different things:

- The outer `{ }` means "here comes JavaScript".
- The inner `{ }` is the object itself, like in JavaScript chapter 11.

The CSS property names are camelCase too: `font-size` becomes `fontSize`, and `background-color` becomes `backgroundColor`. A plain number means pixels for sizes, so `fontSize: 32` becomes `32px`.

TypeScript knows every CSS property name, so it catches typos here too:

```
<h1 style={{ fontsize: 32 }}>Sale!</h1>
// ❌ Object literal may only specify known properties, but 'fontsize' does not exist in type
'Properties<string | number, string & {}>'. Did you mean to write 'fontSize'?
```

Use `style` for quick experiments. For real styling, `className` with a CSS file is usually better. [Chapter 15](#) covers that.

Comments inside JSX

Between tags, a comment needs braces too, because it's JavaScript:

```
<div>
  {/* The menu goes here later */}
  <h1>Corner Café</h1>
</div>
```

A normal `// comment` there doesn't work. It shows up on the page as text!

Long JSX: wrap it in parentheses

When your JSX takes more than one line, put it in parentheses after `return`, like all the examples above. Without them, this happens:

```
function App() {
  return
    <h1>Corner Café</h1>;
}
```

JavaScript sees `return` alone on its line and stops right there, so `App` returns nothing and the page is blank. VS Code shows the `<h1>` line faded out, which means "this code never runs". The `(` right after `return` tells JavaScript "keep going, there's more".

JSX is a value you can store

Since JSX is a value, you can keep a piece of it in a variable and use it later:

```
function App() {
  const badge = <span>★ Bestseller</span>;

  return (
    <div>
      <h2>The Hobbit</h2>
      {badge}
    </div>
  );
}
```

Hover over `badge` in VS Code. TypeScript says its type is `React.JSX.Element`: a piece of JSX.

This feels small now, but it's the start of something big. In chapter 03, you'll put JSX in functions and reuse it anywhere.

JSX keeps you safe

In JavaScript chapter 51, you learned that putting user text into `innerHTML` can let an attacker run code (XSS). JSX protects you automatically:

```
const comment = "<img src=x onerror=alert('hacked')>";

<p>{comment}</p>
```

The page shows the comment as plain text, angle brackets and all. It never becomes a real `<img>`, so nothing runs. React treats everything in braces as text, like `textContent` does.

(React does have a way to insert raw HTML. It's called `dangerouslySetInnerHTML`, and the name is the warning. You won't need it in this course.)

From HTML to JSX: a checklist

You'll often copy HTML from somewhere and turn it into JSX. Go through this list:

1. Wrap everything in **one parent** (or a fragment).
2. **Close every tag**: `<br>` becomes `<br />`.
3. Change `class` to `className` and `for` to `htmlFor`.
4. Change other attributes to **camelCase**: `tabindex` becomes `tabIndex`.
5. Put **numbers in braces**: `maxLength="50"` becomes `maxLength={50}`.

6. Turn `style` strings into **objects**: `style="color: red"` becomes `style={{ color: "red" }}`.
7. Turn HTML comments `<!-- -->` into `{/* */}`.

Then let TypeScript check your work. Any red squiggle left over is something you missed.

Common mistakes

1. Two elements side by side

```
return (  
  <h2>Menu</h2>  
  <p>Coffee, tea, cake</p>  
)  
// ❌ JSX expressions must have one parent element.
```

Wrap them in a `<div>` or a fragment (`<>` and `</>`).

2. Forgetting to close a tag

```
<input type="email">  
// ❌ JSX element 'input' has no corresponding closing tag.
```

Fix: `<input type="email" />`.

3. Writing `class` instead of `className`

```
<h1 class="title">Hi</h1>  
// ❌ Property 'class' does not exist on type '...'. Did you mean 'className'?
```

It's the same for `for`: TypeScript asks `Did you mean 'htmlFor'?`

4. Putting a number in quotes

```
<input maxLength="50" />  
// ❌ Type 'string' is not assignable to type 'number'.
```

Quotes make a string. Fix: `maxLength={50}`.

5. Putting quotes around braces

```
const photo = "/favicon.svg";  

```

The image is broken, because its address is the text `{photo}`. TypeScript *can't* catch this one, because `"{photo}"` is a perfectly good string. Fix: `src={photo}`.

6. Writing `${ }` out of habit

After template literals, it's easy to type this:

```
const name = "Maya";
<p>Hello, ${name}!</p>
// shows: Hello, $Maya!
```

No error, just a stray `$` on the page. TypeScript can't catch this one either, because a `$` is allowed in text. In JSX, the braces alone are enough: `<p>Hello, {name}!</p>`.

7. Putting a whole object in braces

```
const product = { name: "Pen", price: 2 };
<p>{product}</p>
// ❌ Type '{ name: string; price: number; }' is not assignable to type 'ReactNode'.
```

React doesn't know how to show a whole object, so pick the parts you want: `{product.name}` costs `{product.price}`. (If you run it anyway, the page crashes with `Objects are not valid as a React child`.)

8. Writing `style` as a string

```
<h1 style="color: red">Sale!</h1>
// ❌ Type 'string' has no properties in common with type 'Properties<string | number, string & {}>'.
```

`Properties<...>` is TypeScript's name for "an object of CSS properties". Fix: `style={{ color: "red" }}`.

Quick recap

- JSX lets you write HTML-like tags in your code. In TypeScript, those files end in `.tsx`. Vite translates JSX into plain JavaScript function calls.
- Three rules: return **one parent** (a fragment `<>...</>` works), **close every tag**, and use **camelCase** attributes, with `className` instead of `class`.
- Curly braces `{ }` drop any JavaScript **expression** into JSX, in text or in attributes. Statements like `if` don't work there. Use the ternary instead.
- TypeScript knows every tag, attribute and CSS property, so it catches typos. Numbers go in braces: `maxLength={50}`. Read long errors from the last sentence.

- `true`, `false`, `null` and `undefined` show nothing. `0` does show. Objects aren't a `ReactNode`, so they're an error.
 - `style` takes an object with camelCase names: `style={{ fontSize: 24 }}`. JSX shows text in braces as text, which protects you from XSS.
-

Next: try the [exercises](#), then move on to [03 Components](#).

Exercises

How to do these:

- Work in your practice app, the same way as in [chapter 01](#): one file per exercise in `src/ch02/` (`Ex1.tsx`, `Ex2.tsx`, ...), and change the `import` line in `src/App.tsx` to show the one you're working on.
 - Keep the Console open (`F12`) while you work.
 - An exercise is done when the page looks right, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no errors or React warnings.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Fix the library card

A friend wrote this for their library's website, but it's broken. Copy it into `src/ch02/Ex1.tsx`:

```
function Ex1() {
  return (
    <h1 class="title">City Library</h1>
    
    <label for="card">Card number</label>
    <input id="card" type="text">
  );
}

export default Ex1;
```

There are **five** problems. Find and fix them all, so the page shows the heading, the logo, the label, and the text box.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Movie ticket

Create `src/ch02/Ex2.tsx`. Above the `return`, make three `const` variables: the movie title, the seat number, and the price as a number (`12.5`). Don't write any type annotations. Use the variables in your JSX so the page shows:

```
Inside Out 2
Seat: 14
Price: $12.50
```

The title should be a heading, and the other two lines paragraphs.

Then hover over each variable in VS Code, and write the type TypeScript shows as a comment next to it. Why isn't the title's type just `string`?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Product card

You're building an online shop. In `src/ch02/Ex3.tsx`, start with this object above the function:

```
const product = {
  name: "Wireless Mouse",
  price: 24.99,
  stock: 3,
  onSale: true,
  image: "/favicon.svg",
};
```

Part 1: a type. Describe the product with a `type Product` (TypeScript chapter 04), and use it to annotate `product`. Test it: misspell one property, like `prise`, and check that TypeScript complains. Then fix it.

Part 2: experiments. Try these one at a time, inside your function's JSX, and write what happened as a comment in your file:

1. Put `<p>{product.onSale}</p>` in your JSX. Does TypeScript complain? What shows up on the page? Why?
2. Put `<p>{product}</p>` in your JSX. What does VS Code say? Save anyway, and check the Console too. Then remove it.

Part 3: the card. Build a card that shows:

```
(the image)
Wireless Mouse
$24.99
Only 3 left!
On sale!
```

- The image's `src` comes from `product.image`, and its `alt` is the product's name.
- The name is an `<h2>`.
- The price is green and `20px`, using the `style` attribute.
- The "Only 3 left!" line uses `product.stock`, so it would still be right if the stock changed to 7.
- The last line says `On sale!` when `onSale` is `true`, and `Full price` when it's `false`. Change `onSale` to `false` to test it.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): From HTML to JSX

A gym gave you this HTML for their newsletter sign-up. Turn it into JSX in `src/ch02/Ex4.tsx`:

```
<!-- Newsletter sign-up -->
<h2 class="form-title">Join the Gym Newsletter</h2>
<p style="color: gray; font-size: 14px">One email a week. No spam.</p>
<label for="email">Email</label>
<input id="email" type="email" placeholder="you@example.com" maxlength="50">
<br>
<button class="btn" type="submit">Sign up</button>
```

It's done when the form shows up on the page, with the paragraph in small gray text, and TypeScript and the Console have nothing to say. Count how many changes you made.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Shop receipt

This one combines JSX with array methods from JavaScript chapter 13 and types from the TypeScript course. In `src/ch02/Ex5.tsx`, start with:


```
const cart = [
  { name: "Notebook", price: 3.5, quantity: 5 },
  { name: "Backpack", price: 34.99, quantity: 1 },
  { name: "Sticky notes", price: 1.99, quantity: 2 },
];
const taxRate = 0.08;
```

First, write a `type CartItem` for one item, and annotate `cart` as an array of them (TypeScript chapter 03).

Then make the page show exactly this:

```
Your Receipt
Items: 8
Subtotal: $56.47
Tax (8%): $4.52
Total: $60.99
Most expensive item: Backpack
```

Rules:

- Calculate everything from `cart` and `taxRate`, in your function *above* the `return`. Don't type in any of the numbers yourself. If someone changes the cart, the receipt should still be right.
- The JSX should only *show* your results.
- Don't show each item in a list yet. That's chapter 06.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

03 Components

What is it?

A **component** is a function that returns JSX. It's one piece of your page, with a name:

```
function Greeting() {  
  return <h1>Hello, React!</h1>;  
}
```

Once it has a name, you can use it like an HTML tag:

```
<Greeting />
```

You've already met one. `App`, in `src/App.tsx`, is a component. So is every exercise file you've written so far.

Why does it matter?

Here's a café menu, written the way you know from [chapter 02](#):

```
function App() {  
  return (  
    <div>  
      <div className="card">  
        <h2>Espresso</h2>  
        <p>$2.50</p>  
      </div>  
      <div className="card">  
        <h2>Latte</h2>  
        <p>$3.50</p>  
      </div>  
      <div className="card">  
        <h2>Flat White</h2>  
        <p>$3.50</p>  
      </div>  
    </div>  
  );  
}
```

It works, but look at the problems:

- **The same shape is written three times.** Add an "Add to cart" button, and you edit three places. Miss one, and your page is inconsistent.
- **It's hard to read.** You have to look carefully at all those `<div>` s to see that the page is really just three cards.
- **A real menu has twenty items**, not three. This file would be 150 lines of almost-identical JSX.

This is the same problem that functions solve in plain JavaScript (JavaScript chapter 09): when you write the same thing twice, give it a name and write it once. A component *is* a function. It just happens to return a piece of a page.

Components also give your page a vocabulary. Instead of nested `<div>` s, you write:

```
<Header />
<Menu />
<Footer />
```

Anyone can read that, including you in six months.

Real-world example

A component is a **rubber stamp**.

Rubber stamp	React component
You carve the stamp once, carefully	You write the function once
The name on the handle: "PAID"	The component's name: <code>Greeting</code>
Pressing it onto the paper	Writing <code><Greeting /></code>
Pressing it in five places	Using it five times
Re-carving the stamp	Editing the function: every stamp on every page changes

And because a component can contain other components, you can build big things out of small ones, like snapping LEGO bricks together.

How it works

Your first component

Make a file `src/ch03/Greeting.tsx` :

```
function Greeting() {
  return <h1>Hello, React!</h1>;
}

export default Greeting;
```

That's the whole recipe, and it has three parts:

1. **A function** with a name that starts with a **capital letter**.
2. It **returns JSX**.
3. It's **exported**, so other files can use it (JavaScript chapter 29).

Now use it in `src/App.tsx`:

```
import Greeting from "../ch03/Greeting.tsx";

function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
    </div>
  );
}

export default App;
```

The page shows "Hello, React!" twice. One function, two stamps.

The capital letter is a rule, not a style

This one trips up everybody once, so it's worth understanding properly.

When Vite translates your JSX, it looks at the **first letter of the tag**:

You write	What it becomes	What it means
<code><h1></code>	<code>jsx("h1", ...)</code>	the <i>string</i> <code>"h1"</code> : a built-in HTML tag
<code><Greeting /></code>	<code>jsx(Greeting, ...)</code>	the <i>variable</i> <code>Greeting</code> : your component

Lowercase means "an HTML tag". Capital means "the thing in scope with this name". So a lowercase component name goes looking for an HTML tag that doesn't exist:

```
function greeting() {
  return <h1>Hi</h1>;
}

// somewhere else
<greeting />
// ❌ Property 'greeting' does not exist on type 'JSX.IntrinsicElements'.
```

`JSX.IntrinsicElements` is React's list of built-in HTML tags. **Intrinsic** means "built in". So the error says: "there is no HTML tag called `greeting`."

The nasty version of this mistake is when your component's name happens to *be* a real HTML tag:

```
function header() {
  return <h1>Corner Café</h1>;
}

<header />
```

No error at all. React quietly makes an empty `<header>` element, and your heading never appears. Nothing on the page, nothing in the Console, no red squiggle.

So: **component names start with a capital letter**. When the name is several words, capitalise each one: `MenuItem`, `SignUpForm`. That style is called **PascalCase**. Name the file the same way: `MenuItem.tsx`.

Self-closing, or a pair of tags

These two are exactly the same:

```
<Greeting />
<Greeting></Greeting>
```

Use the short one. (Putting something *between* the tags does have a meaning, and it's the `children` prop in [chapter 04](#).)

Forget to close it at all, and you get the same error as any other tag:

```
<Greeting>
// ❌ JSX element 'Greeting' has no corresponding closing tag.
```

A component is a normal function

Everything you know about functions still applies. You can do work above the `return`:

```
function TodayBanner() {
  const today = new Date();
  const weekday = today.toLocaleDateString("en-GB", { weekday: "long" });

  return <p>Happy {weekday}! Coffee is half price today.</p>;
}
```

(`toLocaleDateString` is from JavaScript chapter 19.)

The pattern is always the same: **work out your values above the `return`, then show them in the JSX.**

Returning nothing on purpose

A component can return `null`, which means "put nothing on the page":

```
function Nothing() {
  return null;
}
```

That's useful, and [chapter 05](#) uses it a lot.

Returning nothing **by accident** is a different story. This happens with arrow functions:

```
const Greeting = () => {
  <h1>Hello!</h1>;    // no return!
};
```

The function body is in `{ }`, so JavaScript needs a `return`. Without one, the function gives back `undefined`, React shows nothing, and there's **no error message** anywhere: not in VS Code, not in the terminal, not in the Console. If a component of yours shows nothing at all, check for a missing `return` first.

Components inside components

A component can use other components. That's how you build a page:

```
function Header() {
  return <h1>Corner Café</h1>;
}

function Footer() {
  return <p>Open 7am to 5pm, every day</p>;
}

function Page() {
  return (
    <>
      <Header />
      <p>Fresh coffee every morning.</p>
      <Footer />
    </>
  );
}
```

`Page` is the **parent**. `Header` and `Footer` are its **children**. Notice that `Page` mixes your components and HTML tags freely. React doesn't care which is which.

Line those relationships up and you get the **component tree**:

```
App
├── Page
│   ├── Header
│   ├── p
│   └── Footer
```

Every React app is one tree like this, growing from a single **root** component. In your app, that root is `App`, which `main.tsx` renders into `<div id="root">`.

This is where the **React Developer Tools** extension from [chapter 01](#) starts to pay off. Press `F12`, open the **Components** tab, and you'll see exactly this tree, live. Click any component to highlight it on the page. Get in the habit of looking here when something is missing: if a component isn't in the tree, it never ran.

Back to the café menu

So can you fix that repeated menu from the start of the chapter? Partly:

```
function MenuItem() {
  return (
    <div className="card">
      <h2>Espresso</h2>
      <p>$2.50</p>
    </div>
  );
}
```

Now `<MenuItem />` gives you the card's *shape* in one line. But every single one says "Espresso", which isn't much of a menu. To fix that, a component needs to accept information from outside:

```
<MenuItem name="Latte" price={3.5} />
```

Those are **props**, and they're the whole of [chapter 04](#). This chapter and the next are really two halves of one idea, so don't worry that your components feel a bit rigid right now.

Plenty of components genuinely never change, though: a header, a footer, a logo, an "about us" paragraph, a divider. Those are useful on their own today.

One component per file

The usual convention:

- **One component per file**, and the file is named after it: `MenuItem.tsx`.
- **Export it as the default** export.
- If a small helper component is only ever used by one parent, it can live in the same file as that parent.

```
src/
├─ App.tsx
├─ main.tsx
└─ ch03/
   ├─ Header.tsx
   ├─ Footer.tsx
   └─ MenuItem.tsx
```

([Chapter 14](#) talks about organising bigger apps.)

Both kinds of export from JavaScript chapter 29 work:


```
// default export: the importer picks the name
export default Greeting;
import Greeting from "../ch03/Greeting.tsx";

// named export: the name must match
export function Greeting() { /* ... */ }
import { Greeting } from "../ch03/Greeting.tsx";
```

Vite's starter and React's own docs use default exports, so this course does too. Some teams prefer named exports, because then everyone spells the component the same way. Either is fine. Just don't mix them up, or you'll hit mistake 3 below.

The file must end in `.tsx`. A `.ts` file can't contain tags.

What TypeScript thinks a component is

Hover over `Greeting` in VS Code and you'll see:

```
function Greeting(): React.JSX.Element
```

TypeScript worked out the return type by itself, from the JSX you returned. You don't have to write it, and this course doesn't.

You'll see older tutorials write components like this:

```
const Greeting: React.FC = () => {
  return <h1>Hi</h1>;
};
```

`React.FC` is short for `FunctionComponent`, a type React provides. It isn't wrong, but it isn't needed either, and React's own documentation uses plain functions. Plain functions are also easier to type when props get complicated ([chapter 25](#)). If you join a team that uses `React.FC`, use `React.FC`. Otherwise, keep it simple.

Declaration or arrow function?

All three of these are the same component as far as React is concerned:

```
function Greeting() {
  return <h1>Hi</h1>;
}

const Greeting = () => {
  return <h1>Hi</h1>;
};

const Greeting = () => <h1>Hi</h1>; // returns without the word "return"
```

This course uses `function`, because it's hoisted (JavaScript chapter 14), so the order of components in a file never matters. Use whichever your team uses. Just remember the missing- `return` trap above.

Each use is its own thing

When you write `<Greeting />` twice, React puts two separate copies on the page. They look the same, but they are not connected. Change one later and the other doesn't follow.

That sounds obvious for a heading. It matters enormously from [chapter 08](#), when components start remembering things: two `<Counter />` s on one page count separately, each with its own number.

Don't define a component inside another component

```
function App() {
  function Menu() { // ❌ don't do this
    return <p>Today's menu</p>;
  }

  return <Menu />;
}
```

This looks tidy and it even works at first. But React runs `App` again every time something changes, and each run creates a **brand new** `Menu` function. React compares functions to decide whether it's looking at the same component as before, so a new function every time means "this is a different component": it throws the old one away and builds a fresh one from scratch. Anything the component was remembering is lost, and the page does far more work than it needs to.

The fix is easy: define components at the **top level** of a file, side by side.

A component should be predictable

Given the same input, a component should always return the same JSX, and it shouldn't reach out and change things outside itself while it's running. This is called being **pure** (JavaScript chapter 43 covers pure functions).

Here's the impure version:

```
let count = 0;

function Counter() {
  count = count + 1;      // ✗ changing something outside the component
  return <p>Rendered {count} times</p>;
}
```

Run it and you'll see **2**, not 1. That's `<StrictMode>` in `main.tsx` doing its job: during development it calls each component twice on purpose, precisely so that this kind of bug is loud instead of silent. React reserves the right to call your component whenever it likes, so a component that counts its own runs will give you different answers on different days.

Keep components to reading their input and returning JSX. When something genuinely needs to change, that's state ([chapter 08](#)) or an effect ([chapter 17](#)).

How small should a component be?

There's no rule, but these signals are reliable:

- **You've written the same JSX twice.** Make a component.
- **A chunk has a name you'd say out loud** ("the search bar", "the price tag"). Make a component.
- **The function is too long to see on one screen.** Split it.
- **It's used once, it's three lines, and it has no name in your head.** Leave it alone.

Beginners usually split too little rather than too much, but don't go hunting for components either. Split when it makes the file easier to read.

Common mistakes

1. A lowercase component name

```
<greeting />
// ✗ Property 'greeting' does not exist on type 'JSX.IntrinsicElements'.
```

And if the name is a real HTML tag, like `header` or `section`, there's no error at all: it just renders empty. Capital letters, always.

2. Calling a component like a function

```
<div>{Greeting()}</div>      // works, but don't
<div><Greeting /></div>      // ✓
```

Calling it drops the JSX straight in, so the page looks right. But React never learns that a component was involved: it won't appear in the Components tab, and from chapter 08 it can't keep its own state. Always use the tag.

3. Export and import don't match

```
export default Header;          // default export
import { Header } from "../ch03/Header.tsx"; // named import
// ✗ Module "../ch03/Header.tsx" has no exported member 'Header'.
```

And if you forget `export` entirely, the import is `undefined`, and the page crashes with `Element type is invalid: expected a string or a class/function but got: undefined`.

4. An arrow function with no `return`

```
const Footer = () => {
  <p>Corner Café 2026</p>;
};
```

Blank page, no error message at all. Add `return`, or drop the braces: `const Footer = () => <p>Corner Café 2026</p>;`

5. Returning two elements side by side

```
return (
  <Header />
  <Footer />
);
// ✗ JSX expressions must have one parent element.
```

Same rule as chapter 02. Wrap them in a fragment: `<>` and `</>`.

6. Putting JSX in a `.ts` file

You get strange errors that don't mention JSX at all, like `Cannot find name 'h1'`, because in a plain `.ts` file `<h1>` looks like a type assertion, not a tag. Rename the file to `.tsx`.

7. Defining a component inside another component

Covered above. Move it out to the top level of the file.

8. Getting the case wrong in an import path

```
import Header from "./ch03/header.tsx"; // the file is really Header.tsx
```

Windows and macOS don't care about the case of file names, so this works fine on your computer. Most servers run Linux, which *does* care, and your app breaks the moment you put it online. Type import paths carefully, or let VS Code's autocomplete write them for you.

Quick recap

- A component is a function that returns JSX. Use it like a tag: `<Greeting />`.
- **Names must start with a capital letter.** Lowercase means "HTML tag", so `<greeting />` is an error and `<header />` silently renders nothing.
- Components nest inside each other and form a tree, growing from `App`. The React DevTools **Components** tab shows you that tree.
- One component per file, named after the component, with a default export. Files with tags end in `.tsx`.
- Do your work above the `return`. Define components at the top level, never inside another component, and don't change anything outside the component while it renders.
- A component on its own is a fixed stamp. Props, in chapter 04, are what let you press the same stamp with different words.

Next: try the [exercises](#), then move on to [04 Props](#).

Exercises

How to do these:

- Work in your practice app. This chapter needs more than one file per exercise, so give each exercise its own folder inside `src/ch03/`: `src/ch03/ex1/`, `src/ch03/ex2/`, and so on.
- Each exercise folder has one file per component, named after the component (`Header.tsx`), plus one top-level file named after the exercise (`Ex1.tsx`) that puts them together.
- To see an exercise, change the `import` line in `src/App.tsx`, the same as before:

```
import Ex1 from "../ch03/ex1/Ex1.tsx";

function App() {
  return <Ex1 />;
}

export default App;
```

- Keep the Console open (`F12`), and open the **Components** tab too. You'll need it in exercises 3 and 4.
- An exercise is done when the page looks right, VS Code shows no red squiggles, `npm tsc -b` prints nothing, and the Console has no errors or React warnings.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Fix the games shop

Copy this into `src/ch03/ex1/Ex1.tsx`, exactly as it is:

```
function header() {
  return <h1>Pixel Games</h1>;
}

function Tagline() {
  <p>New and retro games, since 2019.</p>;
}

function Ex1() {
  return (
    <div>
      <header />
      <Tagline />
      <Footer />
    </div>
  );
}

function Footer() {
  return <p>Open Tuesday to Sunday</p>;
}
```

Nothing works: even the `import` in `App.tsx` is broken. There are **three** problems. Find and fix them, so all three lines appear on the page.

Then answer two questions in a comment at the bottom of your file:

1. `Footer` is written *after* `Ex1` uses it. Why does that work?
2. Which of the three problems did TypeScript warn you about, and which did you have to spot yourself?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Split the bakery page

Here's a whole page in one lump. Put it in `src/ch03/ex2/Ex2.tsx` first, and check it works:

```
function Ex2() {
  return (
    <div>
      <h1>Flour & Salt Bakery</h1>
      <p>Baked fresh every morning on Mill Lane.</p>
      <hr />
      <h2>Today</h2>
      <p>Sourdough, cinnamon buns, and a very good brownie.</p>
      <hr />
      <p>Open 7am to 2pm, or until we sell out.</p>
    </div>
  );
}

export default Ex2;
```

Now split it into **four** components, each in its own file inside `src/ch03/ex2/`:

- `Header.tsx`: the `<h1>` and the paragraph under it.
- `Today.tsx`: the `<h2>` and its paragraph.
- `Footer.tsx`: the opening hours.
- `Divider.tsx`: just an `<hr />`.

Then rewrite `Ex2.tsx` so it imports all four and shows them in the same order. Use `<Divider />` twice.

The page should look **exactly the same** as before you started. That's how you know the split worked.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Build a film night poster

Build this from scratch in `src/ch03/ex3/`, with no starter code. The page should show:

```
FILM NIGHT
Thursdays at 7pm, Room 12

The Princess Bride
1987 · 98 minutes

Free popcorn for everyone
```

Build it out of **five** components, in five files, nested two levels deep:

- `Ex3` uses `<Poster />` and nothing else.
- `Poster` uses `<PosterHeader />`, `<FilmDetails />` and `<Note />`.
- `PosterHeader` shows the first two lines. Make "FILM NIGHT" an `<h1>` and give it a colour with the `style` attribute ([chapter 02](#)).
- `FilmDetails` shows the film's title as an `<h2>` and the year and length under it.
- `Note` shows the last line.

When it works:

1. Open the **Components** tab in DevTools (`F12`).
2. Draw the component tree in a comment at the bottom of `Ex3.tsx`, the same way the notes draw it.
3. Click `FilmDetails` in the tree. What happens on the page?

Stuck? *Hint 1*, *Hint 2* and *Hint 3* are in the [Hints section](#) at the end.

Exercise 4 (Medium): Three bad habits

This file works, sort of, but it breaks three of the rules from the notes. Copy it into `src/ch03/ex4/Ex4.tsx`:


```

let renders = 0;

function Ex4() {
  renders = renders + 1;

  function Sidebar() {
    return <p>Links go here</p>;
  }

  return (
    <div>
      <h1>Dashboard</h1>
      {Sidebar()}
      <p>This component has rendered {renders} times.</p>
    </div>
  );
}

export default Ex4;

```

Part 1: look before you fix.

1. What number does the page show? Was that what you expected? Open `src/main.tsx` and look at what wraps `<App />`. Now explain the number.
2. Open the **Components** tab. Can you find `Sidebar` in the tree? Why not?

Part 2: fix it. Rewrite the file so it follows the rules:

- `Sidebar` is a top-level component in its own file, `Sidebar.tsx`, and it's used as a tag.
- Nothing outside the component is changed while it renders. (You'll have to give up on counting renders. That's the right answer: counting your own renders is a job for [chapter 19](#), not for a variable at the top of a file.)

Write one sentence next to each fix saying what could go wrong if you left it as it was.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Bike shop dashboard

This one pulls in JSX from [chapter 02](#) and array methods from JavaScript chapter 13.

In `src/ch03/ex5/`, build a small dashboard for a bike repair shop. Start from this data, in a file called `data.ts` (a plain `.ts` file, because there are no tags in it):

```

export type Repair = {
  bike: string;
  job: string;
  price: number;
  done: boolean;
};

export const repairs: Repair[] = [
  { bike: "Blue Raleigh", job: "New brake pads", price: 24, done: true },
  { bike: "Red BMX", job: "Puncture", price: 12, done: true },
  { bike: "Green tourer", job: "Full service", price: 85, done: false },
  { bike: "Silver hybrid", job: "New chain", price: 30, done: false },
];

```

The page should show:

```

Mill Lane Cycles
Monday

Jobs in: 4
Finished: 2
Still to do: 2
Money earned: $36

⚠ 2 jobs still waiting

```

Build it from these components, each in its own file:

- `Ex5` puts everything together.
- `ShopHeader` shows the shop name and **today's weekday**, worked out with `Date` (JavaScript chapter 19). It won't say "Monday" unless you're doing this on a Monday, and that's fine.
- `Divider`, used **twice**, exactly like exercise 2.
- `JobStats` imports `repairs` and works out all four numbers itself. **Don't type any of the numbers in by hand.** "Money earned" only counts repairs where `done` is `true`.
- `PendingAlert` imports `repairs`, counts the unfinished ones, and shows the warning line in orange. If every job is finished, it must show **nothing at all**.

Then test it properly:

1. Change `done` to `true` on the green tourer. Every number should follow, and the warning should still appear (one job left).
2. Change all four to `true`. The warning should disappear completely. Check the **Elements** tab in DevTools: there should be no empty paragraph left behind.
3. Add a fifth repair to the array. Every number should update by itself.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

04 Props

What is it?

Props are the information you pass into a component. You write them like HTML attributes:

```
<Greeting name="Maya" />
```

Inside the component, they arrive as the function's argument:

```
function Greeting({ name }: GreetingProps) {  
  return <h1>Hello, {name}!</h1>;  
}
```

"Props" is short for **properties**. If a component is a function that returns a piece of a page, props are that function's parameters.

Why does it matter?

[Chapter 03](#) left the café menu half-finished. You had a `MenuItem` component, but it could only ever say "Espresso":

```
function MenuItem() {  
  return (  
    <div className="card">  
      <h2>Espresso</h2>  
      <p>$2.50</p>  
    </div>  
  );  
}
```

To show three drinks you'd need three nearly identical components, which is exactly the copy-and-paste you were trying to escape.

With props, you write the card **once** and fill in the blanks each time you use it:

```
<MenuItem name="Espresso" price={2.5} />  
<MenuItem name="Latte" price={3.5} />  
<MenuItem name="Flat White" price={3.5} />
```

Three lines instead of eighteen. Change the card's design once, and all three change. Add a fourth drink, and it's one more line.

That's the whole point of components, and props are what unlock it. Without props, components just give names to fixed lumps of HTML. With props, they become reusable tools.

There's a second benefit, and it's a TypeScript one. Because you *describe* what a `MenuItem` needs, TypeScript can check every use of it:

```
<MenuItem name="Latte" />
// ❌ Property 'price' is missing in type '{ name: string; }' but required in type
'MenuItemProps'.
```

You can't forget a piece of information, misspell one, or pass a number where text belongs. That check happens as you type, in every file that uses the component.

Real-world example

Props are a **coffee order**.

Ordering a coffee	Props
The barista, who can make any drink	The component
"A large oat latte, extra hot"	<code><Coffee size="large" milk="oat" extraHot /></code>
The order slip they write it on	The props object
The barista makes what the slip says	The component shows what the props say
The barista can't change your order for you	Props are read-only
Same barista all day, different drinks	Same component, different props

One more detail that matters later: orders only travel **one way**, from you to the barista. If the barista needs to tell you something ("we're out of oat milk"), that's a different conversation. Props work the same way, and you'll see that in the "Data flows down" section.

How it works

Passing props and receiving them

Passing them looks exactly like HTML attributes:

```
<Greeting name="Maya" />
```

React collects every attribute you wrote into **one object** and hands it to your function as its first argument:

```
function Greeting(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

So `<Greeting name="Maya" age={30} />` sends `{ name: "Maya", age: 30 }`. One object, always, no matter how many props there are. If you pass none, you get an empty object `{}`, never `undefined`.

That code has one problem, though. In TypeScript, an unannotated parameter is an error:

```
function Greeting(props) {  
  // ✖ Parameter 'props' implicitly has an 'any' type.
```

You met this in TypeScript chapter 05: TypeScript can't guess a parameter's type, and strict mode won't let it silently become `any`. So props always come with a type.

Typing props

Describe the object with a **type alias** (TypeScript chapter 04), right above the component:

```
type GreetingProps = {  
  name: string;  
};  
  
function Greeting(props: GreetingProps) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
export default Greeting;
```

The name `GreetingProps` is a convention: the component's name plus `Props`. Nothing enforces it, but everyone does it, so do it too. Keep the type in the same file as the component, and `export` it as well if a parent needs it.

Now TypeScript checks every use of `<Greeting />` for you:

```

<Greeting name="Maya" />           // ✓

<Greeting />
// ✗ Property 'name' is missing in type '{}' but required in type 'GreetingProps'.

<Greeting name={42} />
// ✗ Type 'number' is not assignable to type 'string'.

<Greeting nmae="Maya" />
// ✗ Property 'nmae' does not exist on type 'IntrinsicAttributes & GreetingProps'. Did you mean 'name'?

```

(`IntrinsicAttributes` is React's own small list of built-in props, like `key`, which you'll meet in [chapter 06](#). It's joined to your type with `&`, from TypeScript chapter 04. You can read straight past it.)

Best of all: type `<Greeting` and press **Ctrl + Space** in VS Code. TypeScript lists the props this component takes, and what type each one is. You never have to open the component file to remember what it needs.

`interface GreetingProps { name: string }` works just as well. The differences are in TypeScript chapter 04; this course uses `type` throughout.

Destructuring props

Writing `props.` over and over gets noisy. Since props are just an object, you can destructure them (JavaScript chapter 15) right in the parameter list:

```

function Greeting({ name }: GreetingProps) {
  return <h1>Hello, {name}!</h1>;
}

```

Same thing, less typing, and the first line tells you exactly what this component uses. This is what most React code looks like, and it's what the rest of this course does.

With several props:

```

type MenuItemProps = {
  name: string;
  price: number;
};

function MenuItem({ name, price }: MenuItemProps) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>${price.toFixed(2)}</p>
    </div>
  );
}

```

Hover over `price` inside the function: TypeScript says `number`, so `toFixed` is offered in autocomplete.

Keep `props.something` when there are a lot of props, or when you want the extra clarity of seeing where a value came from. Both styles are fine, and you'll meet both in real code.

Quotes for text, braces for everything else

This is the rule from [chapter 02](#), and it matters even more now:

```

<MenuItem
  name="Latte"           // string: quotes
  price={3.5}            // number: braces
  vegan={true}           // boolean: braces
  tags=["hot", "milky"]} // array: braces
  shop={{ city: "Leeds" }} // object: braces (double, like style)
/>

```

Only strings get quotes. Everything else goes in braces. Get it wrong and TypeScript tells you straight away:

```

<MenuItem name="Latte" price="3.5" />
// ❌ Type 'string' is not assignable to type 'number'.

```

You can put a string in braces too, and you'll need to when it comes from a variable: `name={drinkName}`.

Booleans have a shortcut

For a prop typed `boolean`, writing just the name means `true`:


```
<MenuItem name="Latte" price={3.5} vegan />
// exactly the same as vegan={true}
```

Leaving it out does **not** mean `false`, though. If `vegan: boolean` is required, leaving it out is a missing-prop error. To make "left out" mean "no", make the prop optional with a default, which is next.

Optional props and default values

Put a `?` after the name to make a prop optional (TypeScript chapter 05):

```
type BadgeProps = {
  label: string;
  colour?: string;
};
```

Now `<Badge label="New" />` is fine. But inside the component, `colour` has the type `string | undefined`, so you have to handle the missing case (TypeScript chapter 06).

The tidy way is a **default value** in the destructuring:

```
function Badge({ label, colour = "steelblue" }: BadgeProps) {
  return <span style={{ color: colour }}>{label}</span>;
}
```

Two nice things happen:

- `<Badge label="New" />` gets `"steelblue"`, and `<Badge label="Sale" colour="tomato" />` gets `"tomato"`.
- Hover over `colour` inside the function: TypeScript now says plain `string`, not `string | undefined`. The default removed the `undefined` case, so there's nothing left to check.

One catch: **a default only fills in undefined**. Pass `colour={null}` or `colour=""` and you get `null` or an empty string, not `"steelblue"`. (And `colour={null}` is a type error anyway, unless you asked for it.)

Make a prop optional when there's a sensible default, or when the information genuinely might not exist (a user with no photo). Otherwise keep it required and let TypeScript make sure it's there.

The `children` prop

There's one prop you don't write as an attribute. Whatever you put **between** a component's tags arrives as a prop called `children`:

```
<Card>
  <h2>Espresso</h2>
  <p>$2.50</p>
</Card>
```

```
import type { ReactNode } from "react";

type CardProps = {
  children?: ReactNode;
};

function Card({ children }: CardProps) {
  return <div className="card">{children}</div>;
}
```

The type is `ReactNode`, the "anything React can show" type from [chapter 02](#): text, numbers, JSX, arrays of those, `null`, `undefined`. It comes from React, and `import type` brings in a type without importing any actual code (TypeScript chapter 11).

The `?` makes an empty `<Card />` allowed. Drop it if a card with nothing inside makes no sense.

This is a big idea, and it's called **composition**. `Card` owns the frame: the border, the padding, the shadow. It has no idea what's inside, and doesn't need to. The parent decides that:

```
<Card>
  <h2>Espresso</h2>
  <p>$2.50</p>
</Card>

<Card>
  
</Card>
```

One `Card` component, any content. You'll use this shape constantly for layouts, panels, and dialogs.

Data flows down

Props travel in one direction: from parent to child. Here's a two-level example:

```
function PriceTag({ price }: { price: number }) {
  return <p>${price.toFixed(2)}</p>;
}

function MenuItem({ name, price }: MenuItemProps) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <PriceTag price={price} />
    </div>
  );
}

function Menu() {
  return <MenuItem name="Latte" price={3.5} />;
}
```

`Menu` gives the price to `MenuItem`, which passes it on to `PriceTag`:

```
Menu
├─ MenuItem    name, price
│   └─ PriceTag price
```

React calls this **one-way data flow**. It makes bugs much easier to find: when a wrong price shows up, you only ever look *upwards* to see where it came from. Nothing else on the page could have changed it.

(That `{ price: number }` written straight in the parameter list is fine for a tiny one-prop component. For anything bigger, a named type is easier to read.)

A child can never reach up and change its parent's data. When a child needs to tell its parent something ("the user clicked me"), the parent passes down a **function** as a prop, and the child calls it. That's [chapter 07](#), and [chapter 13](#) makes it a proper pattern.

Props are read-only

Your component must not change what it was given. This is the rule that keeps one-way data flow honest.

Reassigning the parameter is pointless but harmless, since it only affects your local copy:

```
function Greeting({ name }: GreetingProps) {
  name = name.toUpperCase(); // confusing; make a new variable instead
```

Changing an object or array you were given is the real problem, because that object belongs to the parent (JavaScript chapter 16):

```
function Cart({ items }: CartProps) {  
  items.push("Pen");           // ❌ you just changed the parent's array
```

React doesn't watch for this, so nothing on the page updates, but the parent's data is now different. These bugs are horrible to track down. Always make a **new** value instead:

```
const withPen = [...items, "Pen"];
```

TypeScript can enforce this for you with `readonly` (TypeScript chapter 03):

```
type CartProps = {  
  items: readonly string[];  
};  
  
// items.push("Pen");  
// ❌ Property 'push' does not exist on type 'readonly string[]'.
```

A `readonly string[]` still accepts a normal `string[]` from the parent. It just stops *you* from changing it. It's a cheap, honest way to say "I only read this". [Chapter 11](#) turns copying-instead-of-changing into a habit.

Many small props, or one object?

Both of these work:

```
<MenuItem name="Latte" price={3.5} vegan />  
  
<MenuItem drink={latte} />
```

Use **separate props** when there are only a few values, or when they come from different places. It reads well and each one is checked on its own.

Use **one object prop** when the thing is a real item in your data, like a product, a user, or a repair job. Then you already have a type for it, and you just reuse it:

```

type Drink = {
  name: string;
  price: number;
  vegan: boolean;
};

type MenuItemProps = {
  drink: Drink;
};

function MenuItem({ drink }: MenuItemProps) {
  return (
    <div className="card">
      <h2>{drink.name}</h2>
      <p>${drink.price.toFixed(2)}</p>
    </div>
  );
}

```

The object style becomes the obvious choice in [chapter 06](#), when you turn an array of drinks into a list of cards.

Spreading props

If you already have an object whose properties match the props, you can spread it (JavaScript chapter 15):

```

const latte = { name: "Latte", price: 3.5 };

<MenuItem {...latte} />
// the same as: <MenuItem name={latte.name} price={latte.price} />

```

TypeScript still checks it properly: a missing or wrong property is still an error.

It's tempting, but use it sparingly. Reading `<MenuItem {...latte} />` tells you nothing about what `MenuItem` is actually receiving, and if `latte` picks up an extra property later you may not notice. It earns its place in one situation, which [chapter 25](#) covers: wrapper components that pass a pile of standard HTML props straight through.

Props can hold anything, including JSX

A prop's type can be anything TypeScript can describe. Arrays, objects, functions (chapter 07), and even JSX:

```

type PanelProps = {
  title: ReactNode;      // text, or a whole piece of JSX
  children?: ReactNode;
};

<Panel title={<em>Today only</em>}>
  <p>Half price pastries.</p>
</Panel>

```

`children` is just the one that gets special treatment from JSX. Everything else is an ordinary prop.

Seeing props in DevTools

Open the **Components** tab (`F12`), and click any component in the tree. The panel on the right lists its props and their current values. When something on screen is wrong, this tells you immediately whether the component got bad data or displayed good data badly.

Common mistakes

1. A number in quotes

```

<MenuItem name="Latte" price="3.5" />
// ❌ Type 'string' is not assignable to type 'number'.

```

Quotes always make a string. Braces: `price={3.5}`.

2. Using a prop name you didn't destructure

```

function Greeting(props: GreetingProps) {
  return <h1>Hello, {name}!</h1>;
}
// ❌ Cannot find name 'name'.

```

Either write `props.name`, or destructure: `function Greeting({ name }: GreetingProps)`.

3. Forgetting a required prop

```

<MenuItem name="Latte" />
// ❌ Property 'price' is missing in type '{ name: string; }' but required in type 'MenuItemProps'.

```

Pass it, or mark it optional with `?` and give it a default.

4. The name in the type and the name you pass don't match

```
type MenuItemProps = { name: string; price: number };

<MenuItem title="Latte" price={3.5} />
// ✖ Property 'title' does not exist on type 'IntrinsicAttributes & MenuItemProps'.
// ✖ Property 'name' is missing in type ...
```

One typo gives you two errors: an unexpected prop and a missing one. That pair is a reliable sign of a misspelt prop name.

5. Changing an object or array prop

```
items.push("Pen");    // ✖ that array belongs to the parent
```

Make a new one: `[...items, "Pen"]`. Type the prop as `readonly string[]` so TypeScript stops you.

6. Expecting props to update on their own

```
let price = 3.5;

function App() {
  price = 4;                // nothing happens on screen
  return <MenuItem name="Latte" price={price} />;
}
```

Changing a plain variable doesn't tell React anything, so the page keeps showing the old value. Data that changes over time has to live in **state** ([chapter 08](#)). Props are how state gets handed down.

7. Reaching for `any` when the type gets awkward

```
function MenuItem(props: any) {    // ✖ every check you just built, switched off
```

`any` makes the red squiggle go away and takes your autocomplete, your typo checking and your safety with it (TypeScript chapter 02). If you're stuck on a type, write the shape out longhand, or ask. Don't reach for `any`.

8. Forgetting that `children` is spelled exactly that way

```

type CardProps = { content?: ReactNode };

<Card>
  <p>Hi</p>
</Card>
// ✖ Property 'children' does not exist on type 'IntrinsicAttributes & CardProps'.

```

React always calls it `children`. `content`, `body` and `inner` are ordinary props that must be passed as attributes.

Quick recap

- Props pass information into a component, like arguments to a function: `<Greeting name="Maya" />`. React gathers them into one object.
- Describe them with a type alias called `<Component>Props`, then destructure in the parameter list: `function Greeting({ name }: GreetingProps)`.
- TypeScript then checks every use: missing props, wrong types, and misspelt names are all errors, and VS Code autocompletes the props for you.
- **Strings get quotes. Everything else gets braces.** A `boolean` prop on its own means `true`.
- `?` makes a prop optional, and a default in the destructuring (`colour = "steelblue"`) fills in the gap and removes `undefined` from its type.
- Whatever sits between a component's tags arrives as the `children` prop, typed `ReactNode`. That's how you build reusable frames like `Card`.
- Data flows **down** only, and props are **read-only**: never change an object or array you were given, and use `readonly` to prove it.

Next: try the [exercises](#), then move on to [05 Conditional Rendering](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch04/ex1/`, `src/ch04/ex2/`, and so on. One file per component, plus an `Ex1.tsx` that puts them together.
- Change the `import` line in `src/App.tsx` to see the exercise you're working on, the same as in [chapter 03](#).
- Keep the Console open (`F12`), and use the **Components** tab to check what props each component actually received.
- An exercise is done when the page looks right, VS Code shows no red squiggles, `npm tsc -b` prints nothing, and the Console has no errors or React warnings.
- Try on your own first. Only open a hint if you've been stuck for a while.

- When you're done, ask Claude to check your code.

Exercise 1 (Easy): The name badge

Create `src/ch04/ex1/Badge.tsx`. It should take two props, `name` (text) and `role` (text), and show:

```
Maya
Volunteer
```

Write a `type BadgeProps` above the component, destructure the props in the parameter list, and give the name an `<h2>`.

Then in `Ex1.tsx`, show **three** badges with different names and roles.

Now break it on purpose, one at a time, and write down the error message each time before you fix it:

1. Remove `role` from one of the badges.
2. Pass `name={5}` on one of them.
3. Rename one badge's `name` to `naem`.
4. Add `age={30}` to one of them, without changing `BadgeProps`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): The menu, finished at last

Finish the café menu the notes keep talking about. In `src/ch04/ex2/MenuItem.tsx`, write a `MenuItem` component that takes:

- `name`: text
- `price`: a number
- `vegan`: a boolean

It should show the name as an `<h2>`, the price as `$3.50` (always two decimal places), and the word `Vegan` underneath **only when** `vegan` **is** `true`. When it's `false`, that line should not be there at all.

In `Ex2.tsx`, show four drinks:

Name	Price	Vegan
Espresso	2.5	no

Name	Price	Vegan
Latte	3.5	no
Oat Flat White	3.75	yes
Mint Tea	2	yes

Use the boolean shorthand for at least one of them. Check that Mint Tea shows `$2.00`, not `$2`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): A button you can reuse

Real apps have one button component used everywhere, with a few knobs to turn. Build one in `src/ch04/ex3/Button.tsx`.

ButtonProps :

Prop	Type	Required?	Default
<code>label</code>	text	yes	
<code>colour</code>	text	no	<code>"steelblue"</code>
<code>size</code>	<code>"small"</code> or <code>"large"</code>	no	<code>"small"</code>
<code>disabled</code>	boolean	no	<code>false</code>

The component returns a real `<button>` with:

- the `label` as its text,
- `backgroundColor` set to `colour` and white text, using the `style` attribute,
- a font size of `14` when `size` is `"small"` and `20` when it's `"large"`,
- the real HTML `disabled` attribute set from the `disabled` prop.

In `Ex3.tsx`, show five buttons that between them use every prop, including one with **only** a label.

Then try `<Button label="Go" size="medium" />` and write down what TypeScript says. Why is that a much better error than finding out in the browser?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A card that holds anything

Build a reusable frame in `src/ch04/ex4/Card.tsx`. `Card` takes:

- `title`: text, required. Shown as an `<h3>` at the top.
- `children`: optional, anything React can show. Shown underneath the title.

Give the card a visible frame with the `style` attribute: a `1px solid grey` border, `12px` of padding, and `8px` of margin at the bottom.

In `Ex4.tsx`, use `<Card>` three times with completely different contents:

1. A card with a paragraph of text inside.
2. A card with a `<ul>` of three shopping items inside.
3. A card with **two** of your `Badge` components from exercise 1 inside. (Import them from `../ex1/Badge.tsx`.)

Then answer in a comment: `Card` doesn't know anything about badges or shopping lists. Why is that a good thing?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Library checkout page

This one puts everything together: object props, a `readonly` array, two levels of passing props down, and array methods from JavaScript chapter 13.

Start with `src/ch04/ex5/data.ts`:

```

export type Book = {
  title: string;
  author: string;
  year: number;
  daysLate: number;
};

export const alice: Book = {
  title: "The Hobbit",
  author: "J. R. R. Tolkien",
  year: 1937,
  daysLate: 0,
};

export const bob: Book = {
  title: "Dune",
  author: "Frank Herbert",
  year: 1965,
  daysLate: 3,
};

export const carla: Book = {
  title: "Small Gods",
  author: "Terry Pratchett",
  year: 1992,
  daysLate: 12,
};

```

Part 1: BookRow . Takes **one** prop, `book` , typed with the `Book` type you just imported. It shows:

```

The Hobbit
J. R. R. Tolkien, 1937
On time

```

The third line comes from a second component, `LateNotice` , which `BookRow` renders. `LateNotice` takes only `daysLate` (a number), and shows:

- On time in green when `daysLate` is 0 ,
- 3 days late – \$0.75 fine in red otherwise, where the fine is **25 cents per day late**, shown with two decimal places.

Notice what's happening: `BookRow` receives a whole `Book` , but passes only the one number `LateNotice` needs. That's normal and good. A component should be given the least it can do its job with.

Part 2: CheckoutSummary . Takes one prop, `books` , a **readonly array** of `Book` . It works out and shows:

```
3 books out
2 overdue
Total fines: $3.75
```

Every number must be calculated from `books`. Don't type any of them in.

Part 3: Ex5. Puts the page together: a heading, a `<CheckoutSummary />`, and three `<BookRow />`s. Build the array for the summary from your three books.

Part 4: prove the rules. Do these and write down what happens:

1. Inside `CheckoutSummary`, try `books.push(alice)`. What does TypeScript say, and why is that good?
2. Inside `CheckoutSummary`, try `books.sort((a, b) => a.year - b.year)`. What does TypeScript say? (`sort` changes the array it's called on, so this is the same rule.) How would you sort them without breaking it?
3. Change `carla.daysLate` to `0`. Which numbers on the page change by themselves, and which would you have had to fix by hand if you'd typed them in?

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

05 Conditional Rendering

What is it?

Conditional rendering means showing part of your page only when it should be there.

```
{isLate && <p>This book is overdue.</p>}
```

There's no special React feature here. You use `if`, the ternary `? : ,` and `&&`, exactly as you learned them in JavaScript chapter 07. What's new is *where* you can put them, and a couple of traps.

Why does it matter?

Almost nothing on a real page is there all the time:

- An error message appears only when something went wrong.
- A "Loading..." spinner appears only while you're waiting.
- "Log in" or "Log out" depends on who's looking.
- An empty shopping cart shows "Your cart is empty", not an empty box.
- A delete button appears only for things you're allowed to delete.

In [chapter 04](#), the last exercise needed a "Vegan" line that only showed for some drinks, and the exercise before that needed a warning that disappeared entirely when there was nothing to warn about. You reached for the ternary and `null` because that was all you had. This chapter gives you the full set, and tells you which one to reach for when.

Real-world example

Think about a **departure board** at a train station.

Departure board	Conditional rendering
Every train shows a time and a platform	The parts that are always there
"DELAYED 10 MIN" appears only for late trains	<code>{isLate && <p>Delayed</p>}</code>
A train shows either "On time" or "Cancelled"	<code>{isCancelled ? "Cancelled" : "On time"}</code>
A platform that's blank until it's decided	<code>{platform ?? "TBC"}</code>
The board is empty, so it says "No departures"	An empty state

The board isn't hiding the delay notice behind a curtain. For an on-time train, that notice **doesn't exist**. That's the important part, and it's how React works too.

How it works

The setup for this chapter

Most examples use this component. Put it in `src/ch05/Notice.tsx` and change the props as you read:

```
type NoticeProps = {
  daysLate: number;
};

function Notice({ daysLate }: NoticeProps) {
  return (
    <div>
      <h2>Your loan</h2>
      {/* the interesting bit goes here */}
    </div>
  );
}

export default Notice;
```

Option 1: `if` above the `return`

A component is a normal function, so you can use a normal `if` before you return anything:

```
function Notice({ daysLate }: NoticeProps) {
  if (daysLate > 0) {
    return <p>This book is {daysLate} days overdue.</p>;
  }

  return <p>Nothing is overdue.</p>;
}
```

This is the plainest option, and it's often the best one. Two completely different-looking results, two plain returns, no cleverness.

When the first branch returns and the rest of the function carries on, it's called an **early return**. It keeps your code flat: no `else`, no extra indentation.

Coming later: from [chapter 08](#), your components will call functions like `useState` at the top. Those must run on **every** render, so all your early returns have to come *after* them. There's nothing to do about that yet, but tuck it away.

Option 2: return `null` for nothing at all

From [chapter 03](#): returning `null` means "put nothing on the page".

```
function Notice({ daysLate }: NoticeProps) {  
  if (daysLate === 0) {  
    return null;  
  }  
  
  return <p>This book is {daysLate} days overdue.</p>;  
}
```

Look in the **Elements** tab of DevTools: there is no empty `<p>`, no empty `<div>`, nothing. The element genuinely isn't in the page.

`null` is the honest way to say "nothing here". `<></>` would look the same on screen, but `null` says what you mean.

There's a design question hiding in here. Should the component decide for itself that it has nothing to say (return `null`), or should the parent decide not to render it at all?

```
// the component decides  
<Notice daysLate={0} />  
  
// the parent decides  
{daysLate > 0 && <Notice daysLate={daysLate} />}
```

Both are fine. Let the component decide when the rule is really about that component ("an overdue notice with no overdue days is nothing"). Let the parent decide when it's about the page's layout.

Option 3: the ternary, inside JSX

You can't use `if` inside curly braces, because braces take an **expression** ([chapter 02](#)), and `if` is a statement. The ternary is the expression version of `if`:


```
function Notice({ daysLate }: NoticeProps) {
  return (
    <div>
      <h2>Your loan</h2>
      <p>{daysLate > 0 ? "Overdue" : "On time"}</p>
    </div>
  );
}
```

It works with whole chunks of JSX too, not just text:

```
<div>
  <h2>Your loan</h2>
  {daysLate > 0 ? (
    <p style={{ color: "crimson" }}>Overdue by {daysLate} days</p>
  ) : (
    <p style={{ color: "green" }}>On time</p>
  )}
</div>
```

Wrapping each branch in `( )` isn't required, but it keeps multi-line JSX readable. Your code formatter will do it for you.

Use a ternary when you want **one thing or the other thing**, in the middle of some JSX that stays the same.

Option 4: `&&`, for "show this, or show nothing"

Very often there's no "other thing". You want a line that's either there or absent:

```
<div>
  <h2>Your loan</h2>
  {daysLate > 0 && <p>Overdue by {daysLate} days</p>}
</div>
```

To understand why this works, remember what `&&` really does in JavaScript (chapter 04). It is **not** a yes/no question. It looks at the left side:

- If the left side is **falsy**, the whole expression is that falsy value, and the right side never runs.
- If the left side is **truthy**, the whole expression is the **right side**.

So `daysLate > 0 && <p>...</p>` is either `false` (when there's nothing to say) or the `<p>` itself. And from the table in [chapter 02](#): React shows nothing for `false`. That's the whole trick.

This is the most common pattern in React, and it's the one with the famous trap.

The 0 trap

Here's a bug you will write one day, so let's write it now:

```
type CartProps = {
  itemCount: number;
};

function Cart({ itemCount }: CartProps) {
  return (
    <div>
      {itemCount && <p>You have {itemCount} items.</p>}
    </div>
  );
}
```

With `itemCount={3}`, you get "You have 3 items." Good.

With `itemCount={0}`, you get a bare **0** sitting on your page.

Why? `0` is falsy, so `&&` gives back the left side, which is `0`. And `0` is one of the few falsy values React **does** show. (`false`, `null` and `undefined` show nothing; `0` shows `0`.) That warning in chapter 02 was pointing straight at this.

The fix is to make the left side a real true-or-false:

```
{itemCount > 0 && <p>You have {itemCount} items.</p>}
```

Rule of thumb: put a real comparison on the left of `&&`, never a bare number. `list.length` is the same trap, because an empty array has length `0`:

```
{books.length && <BookList books={books} />} // ❌ shows "0" when empty
{books.length > 0 && <BookList books={books} />} // ✅
```

Strings are safer, because an empty string renders as nothing. But a comparison still reads better: `{name !== "" && <p>{name}</p>}`.

Option 5: a variable that holds JSX

When the logic gets long, lift it out of the JSX entirely. JSX is a value, so you can build it up above the `return`:

```
function Notice({ daysLate }: NoticeProps) {
  let message = <p style={{ color: "green" }}>On time</p>;

  if (daysLate > 7) {
    message = <p style={{ color: "crimson" }}>Very overdue. Please return it.</p>;
  } else if (daysLate > 0) {
    message = <p style={{ color: "orange" }}>Overdue by {daysLate} days.</p>;
  }

  return (
    <div>
      <h2>Your loan</h2>
      {message}
    </div>
  );
}
```

Now the JSX at the bottom is easy to read, and the decision is written as a plain `if / else if` chain, which is easy to read too. This is the answer whenever you catch yourself nesting ternaries.

(Hover over `message` in VS Code: TypeScript infers `React.JSX.Element` from the first assignment, which is exactly what you want.)

Option 6: pick with an object

For "one of several fixed choices", a lookup object is often tidiest (JavaScript chapter 11):

```
type Status = "on-time" | "late" | "lost";

type BadgeProps = {
  status: Status;
};

const labels: Record<Status, string> = {
  "on-time": "On time",
  late: "Overdue",
  lost: "Reported lost",
};

function Badge({ status }: BadgeProps) {
  return <span>{labels[status]}</span>;
}
```

`Record<Status, string>` is a utility type from TypeScript chapter 09: "an object with one key for every `Status`, each holding a string". The payoff is real: add `"damaged"` to `Status` and forget to add a label, and TypeScript tells you which one is missing. A chain of ternaries would just quietly show nothing.

The values can be JSX, not just text.

Choosing between them

Situation	Use
Show something, or show nothing	<code>&&</code> with a real comparison
Show A or B, inside surrounding JSX	ternary <code>? :</code>
Two completely different pages	<code>if</code> with early returns
Nothing at all to show	<code>return null</code>
Three or more cases	<code>if / else if</code> into a variable
One of a fixed set of options	a lookup object

If you're ever unsure, use `if` above the `return`. Nobody has ever been confused by an `if`.

Conditional attributes and classes

The same tools work inside attributes:

```
<p style={{ color: daysLate > 0 ? "crimson" : "green" }}>
  {daysLate > 0 ? "Overdue" : "On time"}
</p>

<button disabled={itemCount === 0}>Check out</button>

<li className={task.completed ? "task done" : "task"}>{task.text}</li>
```

Note `disabled={itemCount === 0}`. Attributes take real values, so you don't need `&&` or a ternary. Just give it the boolean.

Careful with `&&` in an attribute, though. It doesn't disappear the way it does in text:

```
<div className={isActive && "active"}>
// ❌ Type 'false | "active"' is not assignable to type 'string | undefined'.
```

There, `false` would end up as the class name. TypeScript catches it. Use a ternary with an empty string:

```
className={isActive ? "active" : ""}. (Chapter 15 shows a tidier way for class names.)
```

The empty state

When a list is empty, showing an empty box is a bad experience. Say something:

```
type BookListProps = {
  books: readonly string[];
};

function BookList({ books }: BookListProps) {
  if (books.length === 0) {
    return <p>You have no books on loan.</p>;
  }

  return <p>You have {books.length} books on loan.</p>;
}
```

Getting the empty state right is one of the clearest signs of a thoughtful app. Ask yourself for every list: what does this look like on someone's very first day, when they have nothing? ([Chapter 06](#) actually draws the list.)

Hiding with CSS is not the same thing

You could hide something with CSS instead:

```
<p style={{ display: daysLate > 0 ? "block" : "none" }}>Overdue</p>
```

The page looks the same, but the paragraph **is still there**, in the HTML. That matters:

- Screen readers may still announce it ([chapter 38](#)).
- Anyone can see it in DevTools, so it's no good for anything private.
- With hundreds of rows, you're building elements nobody sees.

Not rendering is the default choice. CSS hiding is for things that must keep their place on screen, or need to animate in and out.

TypeScript and narrowing

Conditions do double duty in TypeScript: they also **narrow** types (TypeScript chapter 06). This matters most with optional props:

```

type UserCardProps = {
  name: string;
  photoUrl?: string;
};

function UserCard({ name, photoUrl }: UserCardProps) {
  return (
    <div>
      <h2>{name}</h2>
      {photoUrl && <img src={photoUrl} alt={name} />}
    </div>
  );
}

```

`photoUrl` is `string | undefined`. Inside the `&&`, TypeScript knows it's a `string`, so `src={photoUrl}` is accepted. Take the check away and you get:

```

<img src={photoUrl} alt={name} />
// ✖ Type 'string | undefined' is not assignable to type 'string'.

```

The error isn't in your way. It's telling you there's a case you haven't thought about.

This works with early returns too:

```

function UserCard({ name, photoUrl }: UserCardProps) {
  if (photoUrl === undefined) {
    return <h2>{name}</h2>;
  }

  // from here down, TypeScript knows photoUrl is a string
  return (
    <div>
      <h2>{name}</h2>
      <img src={photoUrl} alt={name} />
    </div>
  );
}

```

Nullish coalescing for defaults

When you want a fallback *value* rather than a different element, `??` is neat (JavaScript chapter 38):

```

<p>Platform {platform ?? "TBC"}</p>

```

?? only steps in for `null` and `undefined`, so a legitimate `0` or `""` is kept. `||` would replace those too, which is usually a bug.

Common mistakes

1. A bare number on the left of `&&`

```
{books.length && <BookList />} // ❌ shows "0" when the list is empty  
{books.length > 0 && <BookList />} // ✅
```

The single most common React bug. If a stray `0` ever appears on your page, look for an `&&`.

2. Trying to use `if` inside braces

```
<p>{if (isLate) { "Overdue" }}</p>  
// ❌ Expression expected.
```

Braces take expressions. Use a ternary, or move the `if` above the `return`.

3. Nested ternaries

```
{daysLate > 7 ? <VeryLate /> : daysLate > 0 ? <Late /> : <OnTime />}
```

This works, and you'll meet it in real code. Two levels is already hard to read, and three is impossible. Move it into a variable with `if / else if`.

4. Hiding with CSS when you meant to not render

`display: none` still puts the element in the page. Use a condition unless you specifically need the element to stay.

5. `&&` in a `className`

```
<div className={isActive && "active"}>  
// ❌ Type 'false | "active"' is not assignable to type 'string | undefined'.
```

Use a ternary with an empty string for the false case.

6. Expecting the condition to re-check by itself

```
let isLoggedIn = false;

function App() {
  isLoggedIn = true; // the page does not change
  return <div>{isLoggedIn && <p>Welcome back!</p>}</div>;
}
```

React only re-runs your component when **state** changes. That's [chapter 08](#). A condition is checked when the component renders, and not a moment later.

7. Using `||` where you meant `??`

```
<p>You have {count || "no"} items.</p> // count of 0 becomes "no"... by accident
```

Fine here by luck, but `{price || "Free"}` turns a price of `0` into "Free" and a price of `0.0` into "Free" too. `??` only replaces `null` and `undefined`.

8. Forgetting the empty case entirely

The app looks perfect with your three test items and looks broken for a brand-new user with none. Always try it with nothing.

Quick recap

- Conditional rendering is plain JavaScript conditions, used to decide what JSX to return.
- `if` above the `return` (with early returns) is the plainest tool. `return null` means "nothing at all", and leaves nothing in the page.
- Inside JSX, use a **ternary** for "A or B", and `&&` for "this or nothing".
- **Always put a real comparison on the left of `&&`**. `{count && ...}` prints a stray `0`. `{count > 0 && ...}` doesn't.
- For three or more cases, build the JSX into a variable with `if / else if`, or look it up in an object. Don't nest ternaries.
- Conditions narrow types, so checking an optional prop is what makes TypeScript let you use it.
- Not rendering is better than hiding with CSS, and every list needs an empty state.

Next: try the [exercises](#), then move on to [06 Rendering Lists](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch05/ex1/`, and so on. Change the `import` line in `src/App.tsx` to see the one you're working on.
 - Nothing changes on its own yet, so to test a condition you change a prop by hand, save, and look. That feels clumsy. It's fixed in [chapter 08](#).
 - Use the **Elements** tab in DevTools as well as the page. "Not there" and "there but empty" look identical on screen and completely different in the HTML.
 - An exercise is done when the page looks right in **every** case you're asked to test, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no errors or warnings.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): The stray zero

Copy this into `src/ch05/ex1/Ex1.tsx`:

```
type BasketProps = {
  itemCount: number;
  couponCode?: string;
};

function Basket({ itemCount, couponCode }: BasketProps) {
  return (
    <div>
      <h2>Your basket</h2>
      {itemCount && <p>{itemCount} items ready to check out.</p>}}
      {couponCode && <p>Coupon applied: {couponCode}</p>}}
    </div>
  );
}

function Ex1() {
  return (
    <div>
      <Basket itemCount={3} couponCode="SAVE10" />
      <Basket itemCount={0} />
    </div>
  );
}

export default Ex1;
```

1. Look at the page. The second basket shows something it shouldn't. What, and why?
2. Fix it, so an empty basket shows `Your basket is empty.` instead.
3. The coupon line has no bug. Explain in a comment why `couponCode && ...` is safe when `itemCount && ...` isn't.
4. Now try `<Basket itemCount={0} couponCode="" />`. What shows up? Is that a bug, or just luck?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Train departure board

Build `src/ch05/ex2/Departure.tsx`. It takes:

Prop	Type	Meaning
<code>destination</code>	text	where the train goes
<code>time</code>	text	e.g. <code>"14:32"</code>
<code>platform</code>	text, optional	not decided yet if missing
<code>minutesLate</code>	number	<code>0</code> means on time
<code>cancelled</code>	boolean	

It shows, in this order:

1. The time and destination: `14:32 to Leeds`.
2. `Platform 4`, or `Platform TBC` when there's no platform yet.
3. A status line:
 - `Cancelled` in red when `cancelled` is `true`. When a train is cancelled, **nothing** about lateness is shown.
 - `On time` in green when it isn't late.
 - `10 minutes late` in orange when it is.

In `Ex2.tsx`, show four departures that between them cover every case: on time with a platform, late with a platform, on time with no platform, and cancelled.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): One card, four states

Real components usually have to handle "we're still loading", "it broke", "there's nothing", and "here it is". Build `src/ch05/ex3/WeatherCard.tsx` with these props:

```
type Weather = {
  city: string;
  temperature: number;
  description: string;
};

type WeatherCardProps = {
  loading: boolean;
  error?: string;
  weather?: Weather;
};
```

The rules, in order:

1. While `loading` is `true`, show only `Loading the forecast...`. Nothing else.
2. Otherwise, if there's an `error`, show `Something went wrong:` plus the message, in red.
3. Otherwise, if there's no `weather`, show `No forecast for this city yet.`
4. Otherwise, show the city as an `<h2>`, the temperature, and the description. Add `Wrap up warm!` **only** when the temperature is below 5.

In `Ex3.tsx`, show five cards covering all four states, plus one that triggers the "wrap up warm" line.

Then, in a comment, answer this: in case 4, TypeScript lets you write `weather.city` with no complaint, even though `weather` is optional. Why?

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): Order status badge

An online shop shows an order's status as a coloured badge. Build `src/ch05/ex4/StatusBadge.tsx`:

```
type OrderStatus = "pending" | "shipped" | "delivered" | "cancelled";
```

Each status has a label and a colour:

Status	Label	Colour
pending	Waiting to be packed	grey

Status	Label	Colour
shipped	On its way	steelblue
delivered	Delivered	green
cancelled	Cancelled	crimson

Part 1. Write it with a chain of `if / else if` above the `return`. Show all four in `Ex4.tsx`.

Part 2. Now rewrite it with **one lookup object** instead, typed so that TypeScript forces you to cover every status. The component's body should end up being about two lines.

Part 3. Add `"refunded"` to `OrderStatus`, but *don't* add it to your lookup object yet. Where does TypeScript put the red squiggle? Now go back to your Part 1 version and add `"refunded"` there too. Does it warn you? Write down which version you'd rather maintain, and why.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Library account page

This pulls in props, array methods, and everything from this chapter. In `src/ch05/ex5/`, start with `data.ts`:

```
export type Loan = {
  title: string;
  daysLate: number;
  renewable: boolean;
};

export type Account = {
  name: string;
  membershipEnds: string;
  loans: readonly Loan[];
};
```

Build these components, each in its own file:

LoanRow takes one `loan`. It shows the title, then:

- Due back soon when `daysLate` is `0`.
- `X days late – $Y fine` in red when it's late, at 25 cents a day, to two decimal places.
- A **Renew** button (it doesn't have to do anything yet) **only** when `renewable` is `true` **and** the loan isn't late. A late book can't be renewed.

`AccountPage` takes one `account` . It shows:

1. Welcome back, Maya as an `<h1>` .
2. A warning line, `⚠ Your membership ends on 2026-03-01` , shown **only** when `membershipEnds` is set. Make `membershipEnds` optional on the `Account` type so you can test both.
3. If there are no loans: You have nothing on loan right now. and **nothing else at all** — no summary, no list heading, no empty list.
4. Otherwise: a You have 3 books on loan line, a red 2 of them are overdue line shown only when some are, and a `<LoanRow />` for each book. You don't know how to loop yet, so just write out three `<LoanRow />` s by hand with three different loans. ([Chapter 06](#) fixes that in about one line.)

Test all of these and check the **Elements** tab each time, not just the page:

1. An account with three loans, one of them 4 days late.
2. The same account with no `membershipEnds` .
3. An account with an empty `loans` array. There should be no stray `␣` , no empty `<p>` , and no leftover heading anywhere in the HTML.
4. A loan that is both `renewable: true` and 3 days late. No Renew button.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

06 Rendering Lists

What is it?

To turn an array into a list on the page, you use `map` (JavaScript chapter 13):

```
const drinks = ["Espresso", "Latte", "Mint Tea"];

<ul>
  {drinks.map((drink) => (
    <li key={drink}>{drink}</li>
  ))}
</ul>
```

`map` turns each item into a piece of JSX, and React shows them all. The odd-looking `key` is the important part of this chapter, and it gets a long section of its own.

Why does it matter?

At the end of [chapter 05](#), you wrote out three `<LoanRow />`s by hand. That's fine for exactly three loans that never change. Real data isn't like that:

- You don't know how many items there are until the app is running.
- The number changes: someone adds a task, deletes a book, filters the list.
- The data arrives from a server, as an array.

Writing a component per item by hand isn't just tedious, it's impossible. You need a way to say "one of these for every item in that array", and `map` is it.

This is also where React starts to feel like a big win over plain JavaScript. In JavaScript chapter 24, drawing the to-do list took a `render()` function that emptied the `<ul>` and rebuilt every `<li>` with `createElement`. Here it's three lines, and React works out what to change.

Real-world example

Think about **mail merge**, or printing address labels.

Address labels	Rendering a list
A spreadsheet of names and addresses	Your array

Address labels	Rendering a list
One blank label design	Your component
The printer runs the design once per row	<code>map</code>
A hundred rows means a hundred labels	React makes one element per item
Each label needs the <i>right</i> address on it	This is what <code>key</code> is for

That last row is the one to remember. A stack of labels is useless if they're stuck to the wrong envelopes.

How it works

`map` inside braces

Curly braces accept any expression, and an **array** of JSX is one of the things React knows how to show (from the table in [chapter 02](#)). So you can drop a `map` straight in:

```
function DrinkList() {
  const drinks = ["Espresso", "Latte", "Mint Tea"];

  return (
    <ul>
      {drinks.map((drink) => (
        <li key={drink}>{drink}</li>
      ))}
    </ul>
  );
}
```

`map` gives back `[<li>Espresso</li>, <li>Latte</li>, <li>Mint Tea</li>]`, and React renders each one in order.

It must be `map`, not `forEach`. `forEach` returns `undefined`, so you'd get a blank page and no error at all.

Arrays of objects, with a component

Real lists hold objects, and each item usually deserves its own component:

```
// Drink.ts
export type Drink = {
  id: number;
  name: string;
  price: number;
};

export const drinks: Drink[] = [
  { id: 1, name: "Espresso", price: 2.5 },
  { id: 2, name: "Latte", price: 3.5 },
  { id: 3, name: "Mint Tea", price: 2 },
];
```

```
// MenuItem.tsx
type MenuItemProps = {
  drink: Drink;
};

function MenuItem({ drink }: MenuItemProps) {
  return (
    <li>
      {drink.name} — ${drink.price.toFixed(2)}
    </li>
  );
}
```

```
// Menu.tsx
function Menu() {
  return (
    <ul>
      {drinks.map((drink) => (
        <MenuItem key={drink.id} drink={drink} />
      ))}
    </ul>
  );
}
```

That's the standard shape you'll write hundreds of times: **a list component that maps, and an item component that renders one thing.**

Two things to notice:

- **The key goes on the element the map returns** — here, on `<MenuItem />` — not on the `<li>` inside `MenuItem`. More on that below.
- **TypeScript needs no help.** `drinks` is `Drink[]`, so `drink` is a `Drink`. Hover over it and see. Misspell `drink.nmae` and you get an error, as always.

The two shapes of an arrow function

This trips people up, so it's worth being explicit. These are the same:

```
{drinks.map((drink) => (  
  <MenuItem key={drink.id} drink={drink} />  
))}  
  
{drinks.map((drink) => {  
  return <MenuItem key={drink.id} drink={drink} />;  
})}
```

Round brackets `( )` after the arrow mean "here comes the value to return". Curly braces `{ }` mean "here comes a block of code", and a block needs the word `return`. Forget it and you get an empty list, silently.

Use the `( )` form when you're just returning JSX. Use `{ }` when you need a line or two of work first:

```
{drinks.map((drink) => {  
  const label = drink.price > 3 ? "Premium" : "Everyday";  
  return <MenuItem key={drink.id} drink={drink} label={label} />;  
})}
```

Keys: why React needs them

Leave the `key` out and React complains in the Console:

```
Each child in a list should have a unique "key" prop.
```

It's worth understanding what React is actually asking for, because this is one of the few places where doing it thoughtlessly causes real bugs.

When your list changes, React doesn't rebuild the whole thing. It compares the new list with the old one and changes as little as possible. To do that, it has to answer one question for each item: **"is this the same item I had before, or a different one?"**

Without a key, all React has to go on is **position**. Item 1 matches item 1, item 2 matches item 2. That's the address-label problem: if you insert a new name at the top of the spreadsheet, every label after it is now on the wrong envelope.

A `key` is a name tag. It lets React say "this is the same item, it just moved" instead of "everything from position 2 onwards changed".

What goes wrong without a good key

Here's the bug, concretely. Imagine a list where each row has a tick box:

```
[ ] Buy milk
[x] Book the dentist    <- you ticked this one
[ ] Call grandma
```

Now you add "Walk the dog" at the **top** of the array. With position-based matching, React thinks:

- Position 1 used to be "Buy milk", now it's "Walk the dog" → change the text.
- Position 2 used to be "Book the dentist", now it's "Buy milk" → change the text.
- ...and so on.

It only changes the *text*, because that's all that differs. The tick box at position 2 is left alone. So you get:

```
[ ] Walk the dog
[x] Buy milk          <- the tick stayed behind!
[ ] Book the dentist
[ ] Call grandma
```

The tick didn't follow its task. With `key={task.id}`, React knows "Book the dentist" is the same item that just moved down one, and moves the whole row, tick and all.

The same thing happens to anything React is keeping for that row: what you typed into a box, whether a section is open, an animation halfway through, and (from [chapter 08](#)) the row's own state.

The rules for keys

A key must be:

1. **Unique among its siblings.** Two lists on the same page can both use key `1`. Two items *in the same list* cannot.
2. **Stable.** The same item must get the same key every time. Keys that change on every render are as bad as no keys, and slower.
3. **From your data.** An `id` from your database, a filename, an email address. Something that genuinely identifies the item.

```
<MenuItem key={drink.id} drink={drink} />    // ✓ an id
<li key={filename}>{filename}</li>            // ✓ unique, stable text
<li key={index}>{name}</li>                   // ⚠ see below
<li key={Math.random()}>{name}</li>          // ✗ never
```

`key={Math.random()}` is the worst of all. Every render gives every item a brand-new key, so React throws away every row and rebuilds it from scratch, every single time.

What about the index?

`map` hands you the index as a second argument, and it's tempting:

```
{drinks.map((drink, index) => (  
  <MenuItem key={index} drink={drink} />  
))}
```

The index is the item's **position**, which is exactly the thing that isn't stable. Using it as a key is the same as having no key at all, just without the warning. That's why it's worse than it looks: it silences the message that was trying to help you.

The index is fine **only when all three of these are true**:

- the list never gets reordered or sorted,
- items are never added or removed except at the very end,
- the items have no state of their own and no inputs.

A fixed list of table headings? Fine. Anything a user can touch? No.

If your data has no id, give it one. Where the data comes from decides how:

- **From a server:** it almost always has an `id` already.
- **Made in your app:** generate one when you create the item, not when you render it. In the browser, `crypto.randomUUID()` gives you a unique string:

```
const newTask = { id: crypto.randomUUID(), text: "Buy milk", done: false };
```

The key part is *when*: the id is created once, when the task is born, and stored with it. That's what makes it stable.

key is not a prop

This surprises everyone once:

```
function MenuItem({ key, drink }: MenuItemProps) {  
  console.log(key);  // undefined
```

React uses `key` itself and doesn't pass it through. If the component needs the id, pass it separately:

```
<MenuItem key={drink.id} id={drink.id} drink={drink} />
```

(Usually it's already inside `drink`, so you don't need to.)

`key` is one of those `IntrinsicAttributes` that TypeScript mentioned back in [chapter 04](#): React's own built-in props, allowed on every component.

Where exactly to put the key

On the **outermost element returned by the map callback**. Not inside the component:

```
// ❌ the warning doesn't go away
function MenuItem({ drink }: MenuItemProps) {
  return <li key={drink.id}>{drink.name}</li>;
}

{drinks.map((drink) => <MenuItem drink={drink} />)}

// ✅
function MenuItem({ drink }: MenuItemProps) {
  return <li>{drink.name}</li>;
}

{drinks.map((drink) => <MenuItem key={drink.id} drink={drink} />)}
```

React needs the key at the point where it's comparing a list of things. That's the `map`.

If each item needs two elements with no wrapper, use a **fragment with a key**, which needs its long form:

```
import { Fragment } from "react";

{drinks.map((drink) => (
  <Fragment key={drink.id}>
    <dt>{drink.name}</dt>
    <dd>${drink.price.toFixed(2)}</dd>
  </Fragment>
))}
```

The short `<>` can't take a key.

Filtering, sorting and mapping together

`map` chains with everything else you know:

```
{drinks
  .filter((drink) => drink.price < 3)
  .map((drink) => <MenuItem key={drink.id} drink={drink} />)}
```

Reading it out loud gets you the meaning: take the drinks, keep the cheap ones, make a card for each.

When the chain gets long, pull it above the `return` and give it a name:

```
function Menu() {
  const cheapDrinks = drinks.filter((drink) => drink.price < 3);

  return (
    <ul>
      {cheapDrinks.map((drink) => (
        <MenuItem key={drink.id} drink={drink} />
      ))}
    </ul>
  );
}
```

Same rule as always: work above the `return`, show in the JSX.

Careful with `sort`. It changes the array it's called on (JavaScript chapter 16), which would be changing your data during a render — exactly the impurity [chapter 03](#) warned about. Sort a copy:

```
const byPrice = [...drinks].sort((a, b) => a.price - b.price);
const byPrice = drinks.toSorted((a, b) => a.price - b.price); // newer, same idea
```

Don't forget the empty list

`map` over an empty array gives an empty array, which renders nothing at all. Usually you want to say something ([chapter 05](#)):

```
function Menu() {
  if (drinks.length === 0) {
    return <p>Nothing on the menu today.</p>;
  }

  return (
    <ul>
      {drinks.map((drink) => (
        <MenuItem key={drink.id} drink={drink} />
      ))}
    </ul>
  );
}
```

This matters even more with a filter, because the *source* list can be full while the filtered list is empty. "No drinks under \$3" is a different message from "nothing on the menu today".

Lists inside lists

Nesting works exactly as you'd hope. Each `map` needs its own keys, unique among its own siblings:

```
{categories.map((category) => (
  <section key={category.id}>
    <h2>{category.name}</h2>
    <ul>
      {category.drinks.map((drink) => (
        <MenuItem key={drink.id} drink={drink} />
      ))}
    </ul>
  </section>
))}
```

The inner keys only need to be unique inside their own `<ul>`, not across the whole page.

Typing a list prop

A component that takes a list takes an array prop. Use `readonly` so you can't accidentally change the parent's data ([chapter 04](#)):

```

type MenuProps = {
  drinks: readonly Drink[];
};

function Menu({ drinks }: MenuProps) {
  return (
    <ul>
      {drinks.map((drink) => (
        <MenuItem key={drink.id} drink={drink} />
      ))}
    </ul>
  );
}

```

`map`, `filter`, `toSorted` and friends all work on a `readonly` array, because none of them change it. `push` and `sort` don't, which is exactly what you want.

Common mistakes

1. No key

Each child in a list should have a unique "key" prop.

Never ignore this. It's telling you the address labels might end up on the wrong envelopes.

2. Using the index as the key in a list that changes

```
{tasks.map((task, index) => <TaskRow key={index} task={task} />)}
```

No warning, and it works until someone deletes a row or sorts the list. Then ticks, typing and highlights stay behind on the wrong rows. Use a real id.

3. `key={Math.random()}`

Silences the warning and makes everything worse: every item is "new" on every render, so React rebuilds the entire list each time.

4. Duplicate keys

Encountered two children with the same key, `3`.

Two items sharing a key confuses React exactly as much as no key. Usually it means your ids aren't as unique as you thought.

5. `forEach` instead of `map`

```
{drinks.forEach((drink) => <MenuItem key={drink.id} drink={drink} />)}
```

Blank list, no error. `forEach` returns `undefined`, and React shows nothing for `undefined`.

6. Braces without `return`

```
{drinks.map((drink) => {  
  <MenuItem key={drink.id} drink={drink} />;    // no return  
})}
```

Blank list again, and again no error. Either add `return`, or swap the `{ }` for `( )`.

7. The key inside the item component

The warning stays. Put it on what the `map` returns.

8. Sorting your props

```
drinks.sort((a, b) => a.price - b.price);    // ❌ changes the parent's array
```

Copy first: `[...drinks].sort(...)` or `drinks.toSorted(...)`. Typing the prop `readonly Drink[]` makes TypeScript stop you.

Quick recap

- Turn an array into JSX with `map`. It goes straight inside curly braces.
- The usual shape is a **list component** that maps and an **item component** that renders one thing.
- Every item in a list needs a `key`: unique among siblings, stable, and taken from your data — normally an `id`.
- Keys let React tell "the same item moved" from "this item changed". Without them, ticks, typed text and state stay behind on the wrong rows.
- **The index is a position, not an identity.** Only use it for lists that never change order or length.
- `key` goes on what the `map` returns, and React never passes it to your component.
- Chain `filter` and `map` freely, but sort a **copy**, and always handle the empty list.

Next: try the [exercises](#), then move on to [07 Events](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch06/ex1/`, and so on.
 - **Keep the Console open (F12) for every single one of these.** Key problems show up there and nowhere else.
 - An exercise is done when the page looks right, the Console is completely empty, VS Code shows no red squiggles, and `npm tsc -b` prints nothing.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Your first list

In `src/ch06/ex1/Ex1.tsx`, start with this array:

```
const languages = ["JavaScript", "TypeScript", "Python", "SQL", "Go"];
```

1. Show them as a `<ul>` with one `<li>` each. Don't write any `<li>` by hand.
2. Above the list, show `You are learning 5 languages.` — with the number worked out, not typed in.
3. Below it, show a second list of only the languages whose name contains the letter `s`. Give that list its own heading.

Then break it on purpose and write down what happens each time:

4. Change `map` to `forEach`. What's on the page? What's in the Console?
5. Put it back, then delete the `key`. What's on the page? What's in the Console?
6. Change one array item so the array is `["Go", "Go", "Python"]`. With `key={language}`, what does the Console say?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): The café menu, for real

Finally do the menu properly. In `src/ch06/ex2/`, start with `data.ts`:

```
export type Drink = {
  id: number;
  name: string;
  price: number;
  vegan: boolean;
};

export const drinks: Drink[] = [
  { id: 1, name: "Espresso", price: 2.5, vegan: false },
  { id: 2, name: "Latte", price: 3.5, vegan: false },
  { id: 3, name: "Oat Flat White", price: 3.75, vegan: true },
  { id: 4, name: "Mint Tea", price: 2, vegan: true },
  { id: 5, name: "Hot Chocolate", price: 3.2, vegan: false },
];
```

Build two components:

- `MenuItem` takes one `drink` and shows its name, its price to two decimal places, and the word `Vegan` only when it should be. (You can reuse your chapter 04 version.)
- `Menu` takes a `drinks` prop — a **readonly array** of `Drink` — and renders a `<MenuItem />` for each one, plus a heading saying how many drinks there are.

`Ex2.tsx` imports the data and passes it in: `<Menu drinks={drinks} />`.

Then:

1. Add a `<Menu />` showing only the vegan drinks, with its own heading.
2. Add a `<Menu />` showing only drinks over \$10. It should say `No drinks match.` instead of an empty box. Make sure this doesn't break the other two menus.
3. Inside `Menu`, try `drinks.sort((a, b) => a.price - b.price)`. What does TypeScript say, and why is it right to stop you?
4. Fix question 3 so the menu really is sorted by price, cheapest first, without upsetting TypeScript.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Prove the key bug

This exercise makes the address-label problem happen on your own screen. It's the most useful thing in this chapter, so don't skip it.

In `src/ch06/ex3/Ex3.tsx`:

```

const tasks = [
  { id: 1, text: "Buy milk" },
  { id: 2, text: "Book the dentist" },
  { id: 3, text: "Call grandma" },
];

function Ex3() {
  return (
    <ul>
      {tasks.map((task, index) => (
        <li key={index}>
          <input type="checkbox" /> {task.text}
        </li>
      ))}
    </ul>
  );
}

export default Ex3;

```

Part 1: see it break.

1. Run it and tick the box next to **Book the dentist**. Leave the other two alone.
2. Now, *without refreshing the page*, edit the array in your code: add `{ id: 4, text: "Walk the dog" }` at the **front**. Save. Vite swaps the file in without reloading, so the tick boxes keep whatever you set.
3. Which task is ticked now? Write it down. Is it the one you ticked?

Part 2: fix it. Change the key to `task.id`. Refresh, tick "Book the dentist" again, and add a fifth task at the front. Does the tick follow its task now?

Part 3: explain it. In a comment, write two or three sentences on why the tick moved in Part 1 and not in Part 2. Say what React compared in each case.

Part 4: when is the index fine? Write a small `<Legend />` component that maps over `["Red", "Amber", "Green"]` with `key={index}` and explain in a comment why that's genuinely fine here.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Grouped shopping list

Lists inside lists. In `src/ch06/ex4/`, start with:

```

export type Item = {
  id: number;
  name: string;
  quantity: number;
};

export type Aisle = {
  id: number;
  name: string;
  items: Item[];
};

export const aisles: Aisle[] = [
  {
    id: 1,
    name: "Fruit and veg",
    items: [
      { id: 11, name: "Apples", quantity: 6 },
      { id: 12, name: "Carrots", quantity: 3 },
    ],
  },
  {
    id: 2,
    name: "Dairy",
    items: [{ id: 21, name: "Milk", quantity: 2 }],
  },
  { id: 3, name: "Bakery", items: [] },
];

```

Build a page that shows:

Shopping list – 11 things to buy

Fruit and veg (2 items)

Apples × 6
Carrots × 3

Dairy (1 item)

Milk × 2

Bakery

Nothing needed from here.

Rules:

- Two components: `AisleSection` (takes one aisle) and `ItemRow` (takes one item).
- The 11 at the top is the total **quantity** across every aisle, worked out, not typed.

- (2 items) must say (1 item) when there's only one, and must not appear at all for an empty aisle.
- An empty aisle shows `Nothing needed from here.` instead of an empty list.
- The Console must be completely empty.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Leaderboard

In `src/ch06/ex5/`, build a leaderboard for a running club. Start with:

```
export type Runner = {
  id: string;
  name: string;
  club: string;
  timeSeconds: number;
  finished: boolean;
};

export const runners: Runner[] = [
  { id: "a1", name: "Maya", club: "Mill Lane", timeSeconds: 1284, finished: true },
  { id: "b2", name: "Tom", club: "Riverside", timeSeconds: 1190, finished: true },
  { id: "c3", name: "Priya", club: "Mill Lane", timeSeconds: 1352, finished: true },
  { id: "d4", name: "Sam", club: "Riverside", timeSeconds: 0, finished: false },
  { id: "e5", name: "Ana", club: "Mill Lane", timeSeconds: 1190, finished: true },
];
```

The page should show:

Race results

1. Tom (Riverside) – 19:50 🏆
1. Ana (Mill Lane) – 19:50 🏆
3. Maya (Mill Lane) – 21:24
4. Priya (Mill Lane) – 22:32

Did not finish: Sam

Requirements:

1. Only finished runners are in the numbered list, sorted fastest first. **Don't change the `runners` array** while you do it.
2. Times show as `minutes:seconds` with the seconds always two digits (`19:50` , not `19:5`).

3. Runners on the same time share a position, and the next position skips accordingly (see how 3 follows two runners at 1). Give every runner on the fastest time a 🏆.
4. The "Did not finish" line lists the unfinished runners by name, separated by commas. If everyone finished, that line doesn't appear at all.
5. A `ClubTally` component underneath shows how many finishers each club had, one line per club, worked out from the data. Don't type the club names in anywhere.

Test it by adding a sixth runner, by making everyone finish, and by making nobody finish. The page should hold up every time, and the Console should stay empty.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

07 Events

What is it?

An **event** is something that happens on the page: a click, a key press, a form being sent. You react to one by handing React a function:

```
function Ex() {  
  function handleClick() {  
    console.log("Clicked!");  
  }  
  
  return <button onClick={handleClick}>Click me</button>;  
}
```

Same idea as `addEventListener` from JavaScript chapter 21, written as an attribute.

Why does it matter?

Everything you've built so far just sits there. Events are where your app starts listening.

Compare the two ways. In plain JavaScript you had to find the element first, then attach a listener to it:

```
const button = document.querySelector("#save");  
button.addEventListener("click", handleClick);
```

Two steps, and they're in different places: the `id` in the HTML, the `querySelector` in the JS. Rename the id and nothing tells you the listener has stopped working.

In React, the handler sits on the element it belongs to:

```
<button onClick={handleClick}>Save</button>
```

One place, nothing to look up, nothing to keep in sync. And when React removes the button from the page, the handler goes with it. There's no cleaning up.

There's a bigger reason too. Events are how a child component talks **back** to its parent. [Chapter 04](#) said props only flow down and a child can never change its parent's data. Passing a **function** down as a prop is how the child says "the user did something", and lets the parent decide what that means. That pattern runs through the whole rest of this course.

Real-world example

A **doorbell**.

Doorbell	React event
The button by the door	<code><button></code>
Wiring it to a bell	<code>onClick={handleClick}</code>
Someone presses it	The event happens
The bell rings	Your function runs
A note saying who called and when	The event object
Wiring it to a bell that's <i>already ringing</i>	<code>onClick={handleClick()}</code> — the classic bug

That last row is the mistake everyone makes once. You want to give React the bell to ring later, not ring it yourself while you're installing it.

How it works

Attach a handler

Event attributes are camelCase, and the value goes in braces:

```
function Toolbar() {
  function handleSave() {
    console.log("Saving...");
  }

  return <button onClick={handleSave}>Save</button>;
}
```

Three things to get right:

- **onClick**, not **onclick**. TypeScript catches this: `Property 'onclick' does not exist on type '...'`. Did you mean `'onClick'`?
- **Braces, not quotes**. `onClick="handleSave"` gives React the *text* "handleSave".
- **handleSave**, not **handleSave()**. This is the big one, and it gets its own section.

Define handlers inside the component, above the `return`. They need to see the component's props and (soon) its state.

Pass the function, don't call it

```
<button onClick={handleSave}>Save</button>    // ✅ here's a function, ring it later  
<button onClick={handleSave()}>Save</button>    // ❌ calls it right now
```

The second one runs `handleSave` while React is *building* the page, and gives React whatever it returned — usually `undefined`. So the message appears immediately, on every render, and clicking does nothing.

It's worth being able to spot the shape rather than remembering the rule: `()` **means "do it now"**. In an `onClick`, you almost never want "now".

This bug can also be spectacular. From [chapter 08](#), a handler that sets state, called during render, sets state during render, which re-renders, which calls it again:

Too many re-renders. React limits the number of renders to prevent an infinite loop.

If you ever see that, look for a stray `()` in an event attribute.

Passing arguments to a handler

So what if your handler needs an argument?

```
<button onClick={deleteTask(task.id)}>Delete</button>    // ❌ deletes it immediately
```

Wrap it in a new function. That function is the one React holds on to, and *it* does the calling:

```
<button onClick={() => deleteTask(task.id)}>Delete</button>    // ✅
```

Read it as: "when clicked, run `deleteTask(task.id)`". The arrow is the "when clicked" part.

This is the standard way to pass arguments, and you'll write it constantly, especially inside a `map`:

```
{tasks.map((task) => (  
  <li key={task.id}>  
    {task.text}  
    <button onClick={() => deleteTask(task.id)}>Delete</button>  
  </li>  
))}
```

Each button gets its own little function that remembers its own `task.id`. That works because of closures (JavaScript chapter 25).

Inline or named?

```
<button onClick={() => console.log("Hi")}>Hi</button>    // inline

function handleHi() {                                    // named
  console.log("Hi");
}
<button onClick={handleHi}>Hi</button>
```

Both are normal React. Use **inline** for one-liners and for passing arguments. Use a **named function** as soon as there's more than a line or two, because the JSX stays readable and the function gets a name that explains it.

Don't worry that a new arrow function is created on every render. It is, and it doesn't matter. [Chapter 27](#) covers the rare cases where it does.

Naming: `handleX` and `onX`

A convention worth following from day one:

- `handleSomething` for the function that does the work: `handleClick`, `handleSubmit`, `handleDeleteTask`.
- `onSomething` for a prop that carries a function: `onClick`, `onDelete`, `onTaskAdded`.

So the two meet like this:

```
<TaskRow task={task} onDelete={handleDeleteTask} />
```

The parent *handles* it; the prop describes *when*. Every React codebase you ever open will use these names.

The event object

Your handler gets one argument: an object describing what happened.

```
function handleClick(event: React.MouseEvent<HTMLButtonElement>) {
  console.log(event.type);           // "click"
  console.log(event.currentTarget); // the button
}
```

Most of the time you don't need it. When you do, it's usually for one of these:

Property or method	What it's for
<code>event.currentTarget</code>	the element the handler is on

Property or method	What it's for
<code>event.target</code>	the element actually clicked, which may be inside it
<code>event.preventDefault()</code>	stop the browser's default behaviour
<code>event.stopPropagation()</code>	stop the event travelling up to parent elements
<code>event.key</code>	which key was pressed, on keyboard events

`event.currentTarget` is almost always the one you want, and it's the one TypeScript types precisely. `event.target` could be any element, so TypeScript types it loosely and you'll have to prove what it is.

Typing the event

Here's a wrinkle that catches every TypeScript beginner. Inline, TypeScript works the type out for you:

```
<button onClick={(event) => console.log(event.currentTarget)}>Go</button>
```

Hover over `event`: `React.MouseEvent<HTMLButtonElement>`. TypeScript knows the handler is going on a `<button>`, so it knows what the event must be. This is **contextual typing** (TypeScript chapter 05).

Write the handler separately and that context is gone:

```
function handleClick(event) {
  // ❌ Parameter 'event' implicitly has an 'any' type.
```

So you annotate it yourself:

```
function handleClick(event: React.MouseEvent<HTMLButtonElement>) {
  console.log(event.currentTarget.textContent);
}
```

The ones you'll actually use:

Event	Type
Click, on a button	<code>React.MouseEvent<HTMLButtonElement></code>
Typing in a text box	<code>React.ChangeEvent<HTMLInputElement></code>
A <code><textarea></code>	<code>React.ChangeEvent<HTMLTextAreaElement></code>
A <code><select></code>	<code>React.ChangeEvent<HTMLSelectElement></code>

Event	Type
Sending a form	<code>React.FormEvent<HTMLFormElement></code>
A key press	<code>React.KeyboardEvent<HTMLInputElement></code>

The bit in angle brackets is the element the handler is attached to (TypeScript chapter 08 explains that syntax). Getting it right is what makes `event.currentTarget.value` work without complaint.

Two shortcuts:

- If you don't need the event at all, leave the parameter off: `function handleClick() { ... }`. React passes it anyway; you're just ignoring it. This is by far the most common case.
- To get the type without looking it up: write it inline first, hover over `event`, and copy what VS Code tells you.

You can also `import type { MouseEvent } from "react"` and drop the `React.` prefix. Both are common. This course keeps `React.` so it's obvious the type comes from React and not from the browser's own `MouseEvent`, which is a different (but related) thing.

`preventDefault`

Some elements do something by default. A `<form>` reloads the page when you submit it. A link navigates. `preventDefault()` stops that (JavaScript chapter 22):

```
function ContactForm() {
  function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    console.log("Sending...");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" />
      <button type="submit">Send</button>
    </form>
  );
}
```

Leave `preventDefault` out and the page reloads, wiping everything your app was holding. The symptom is unmistakable: you click, the screen flashes, and everything resets.

Put the handler on the `<form>`, not the button. Then Enter in a text box works too, which people expect.

React doesn't have HTML's old `return false` trick. Call `preventDefault()`.

Which events can you use?

Anything the browser offers, in camelCase:

Attribute	When it fires
<code>onClick</code>	a click
<code>onDoubleClick</code>	a double click
<code>onChange</code>	the value of an input changed (see the warning below)
<code>onSubmit</code>	a form is sent
<code>onKeyDown</code>	a key goes down
<code>onFocus</code> / <code>onBlur</code>	an element gains or loses focus
<code>onmouseenter</code> / <code>onmouseleave</code>	the pointer arrives or leaves

One important difference from plain HTML. In the browser, `change` on a text box only fires when you click away. React's `onChange` fires on **every keystroke**, like the browser's `input` event. That's almost always what you want, and it's what [chapter 09](#) is built on. But it does mean your instincts from plain JavaScript are slightly off here.

VS Code will list every available event if you type `on` inside a tag and press Ctrl + Space.

Events travel upwards

Just like in plain JavaScript, an event **bubbles**: it fires on the element you clicked, then its parent, then its parent's parent.

```
<div onClick={() => console.log("card")}>
  <button onClick={() => console.log("button")}>Delete</button>
</div>
```

Click the button and you get **both** messages, button first.

Usually that's handy. When it isn't — a delete button inside a card that also opens the card — stop it:

```

<button
  onClick={(event) => {
    event.stopPropagation();
    deleteTask(task.id);
  }}
>
  Delete
</button>

```

Use `stopPropagation` when you have a real conflict, not by default. Sprinkling it everywhere makes events mysteriously stop working elsewhere.

You don't need **event delegation** any more, by the way. In JavaScript chapter 21 you put one listener on the `<ul>` because elements created later had no listeners of their own. In React, the handler is part of the JSX, so every row gets one automatically, however many rows appear later. That whole class of bug is gone.

Passing handlers down as props

Here's the pattern that matters most. A child component takes a function as a prop and calls it:

```

type TaskRowProps = {
  task: Task;
  onDelete: (id: number) => void;
};

function TaskRow({ task, onDelete }: TaskRowProps) {
  return (
    <li>
      {task.text}
      <button onClick={() => onDelete(task.id)}>Delete</button>
    </li>
  );
}

```

```
function TaskList({ tasks }: TaskListProps) {
  function handleDelete(id: number) {
    console.log("Parent should delete task", id);
  }

  return (
    <ul>
      {tasks.map((task) => (
        <TaskRow key={task.id} task={task} onDelete={handleDelete} />
      ))}
    </ul>
  );
}
```

Read the type out loud: `onDelete: (id: number) => void` is "a function that takes a number and returns nothing" (TypeScript chapter 05). `void` means the parent doesn't promise to give anything back.

The split is the point:

- `TaskRow` knows **when** something happened. It has the button.
- `TaskList` knows **what to do** about it. It has the data.

`TaskRow` can be dropped into any list, and the type tells you exactly what it expects. This is how information travels *up* in React: not by the child reaching into the parent, but by the parent handing down a function and the child calling it.

Your own components don't have real events

This surprises people:

```
<TaskRow onClick={handleClick} />
```

That does **nothing** unless `TaskRow` takes an `onClick` prop and puts it on something. `onClick` is only magic on real HTML tags. On your own component it's an ordinary prop with an ordinary name, and TypeScript will tell you it doesn't exist if you haven't declared it.

And now, the cliffhanger

Try this:

```
function Counter() {
  let count = 0;

  function handleClick() {
    count = count + 1;
    console.log(count);
  }

  return <button onClick={handleClick}>Clicked {count} times</button>;
}
```

Click it. The Console counts up: 1, 2, 3. The button still says **0**.

Two things are wrong, and they're the same thing twice:

1. Changing a plain variable doesn't tell React anything, so the component never re-renders.
2. Even if it did, `let count = 0` runs again on every render, resetting it.

A component needs a memory that survives re-renders, and changing it has to tell React to redraw. That's `useState`, and it's the next chapter. Events and state are the two halves of making a page interactive; you've now got the first half.

Common mistakes

1. Calling the handler instead of passing it

```
<button onClick={handleSave()}>Save</button>
```

Runs during render, does nothing on click. Drop the `()`, or wrap it: `onClick={() => handleSave()}.`

2. Lowercase event names

```
<button onclick={handleSave}>Save</button>
// ❌ Property 'onclick' does not exist on type '...'. Did you mean 'onClick'?
```

3. Quotes instead of braces

```
<button onClick="handleSave">Save</button>
// ❌ Type 'string' is not assignable to type 'EventHandler<HTMLElement>'.
```

4. Forgetting `preventDefault` on a form

The page reloads and everything resets. Put `onSubmit` on the `<form>` and call `event.preventDefault()` first thing.

5. An untyped event parameter

```
function handleChange(event) {  
  // ❌ Parameter 'event' implicitly has an 'any' type.
```

Annotate it, or if you don't need it, remove the parameter entirely.

6. The wrong element in the event type

```
function handleChange(event: React.ChangeEvent<HTMLElement>) {  
  console.log(event.currentTarget.value);  
  // ❌ Property 'value' does not exist on type 'HTMLElement'.  
}
```

Not every element has a `value`. Say which one it is: `HTMLInputElement`.

7. Expecting the page to change

```
function handleClick() {  
  count = count + 1; // the page doesn't move  
}
```

Only state changes cause a re-render. Chapter 08.

8. `onClick` on your own component

```
<MenuItem onClick={handleClick} />  
// ❌ Property 'onClick' does not exist on type 'IntrinsicAttributes & MenuItemProps'.
```

Add it to the props type, and put it on a real element inside.

Quick recap

- Handle events with camelCase attributes and a function in braces: `onClick={handleClick}`.
- **Pass the function, don't call it.** `onClick={handleClick()}` runs during render. To pass arguments, wrap it: `onClick={() => deleteTask(task.id)}`.
- Name the worker `handleX` and the prop `onX`.
- The event object is the handler's one argument. Inline handlers get their type for free; separate ones need an annotation like `React.ChangeEvent<HTMLInputElement>`. If you don't need it, leave the parameter off.

- Put `onSubmit` on the `<form>` and call `event.preventDefault()`, or the page reloads.
 - React's `onChange` fires on every keystroke, unlike the browser's `change`.
 - Events bubble; `stopPropagation()` when there's a real conflict. You never need event delegation in React.
 - A **function passed down as a prop** is how a child tells its parent something happened. That's how data gets back up.
 - Changing a plain variable in a handler does nothing to the page. That's what state is for.
-

Next: try the [exercises](#), then move on to [08 State](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch07/ex1/`, and so on.
 - **Nothing on the page can change yet.** That arrives in [chapter 08](#). So these exercises prove their work in the Console (`F12`), which is exactly how you'd debug a handler in real life anyway.
 - Keep the Console open the whole time, and clear it () between tries so you can see what a single click produced.
 - An exercise is done when the right messages appear at the right moments, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and there are no React warnings.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Four buttons, one bug

Copy this into `src/ch07/ex1/Ex1.tsx`:

```
function Ex1() {
  function handleHello() {
    console.log("Hello!");
  }

  function greet(name: string) {
    console.log("Hello, " + name + "!");
  }

  return (
    <div>
      <button onClick={handleHello}>A</button>
      <button onClick={handleHello()}>B</button>
      <button onClick={() => greet("Maya")}>C</button>
      <button onClick={greet("Tom")}>D</button>
    </div>
  );
}

export default Ex1;
```

1. Load the page **without clicking anything**. What's already in the Console, and which buttons put it there?
2. Click each button in turn. Which ones do nothing?
3. Fix B and D so they only log when clicked.
4. `onClick={greet("Tom")}` has a red squiggle but `onClick={handleHello()} doesn't. Read both, and explain in a comment why TypeScript catches one and not the other.`

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): The vending machine

Build `src/ch07/ex2/` with two components.

`ProductButton` takes:

Prop	Type
<code>name</code>	text
<code>price</code>	number
<code>onBuy</code>	a function taking the name and the price, returning nothing

It renders a `<button>` showing `Crisps – $1.20`, and calls `onBuy` with that product's name and price when clicked.

Ex2 holds an array of four products and renders a `ProductButton` for each with `map`. Its `handleBuy` logs `You bought Crisps for $1.20`.

Then:

1. Add a `<button>Refund</button>` at the bottom that logs `Refunding...`, written as an **inline** arrow function.
2. Add one that logs a two-line message, written as a **named** function. Which did you prefer writing, and why?
3. Write out the full type of `onBuy` in a comment, and read it aloud in plain English.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Event detective

This one is about the event object. In `src/ch07/ex3/Ex3.tsx`:

```
function Ex3() {
  return (
    <div
      style={{ border: "2px solid grey", padding: 20 }}
      onClick={() => console.log("OUTER div")}
    >
      <p>Click the button, then click the grey area around it.</p>
      <button type="button">Delete</button>
    </div>
  );
}

export default Ex3;
```

Part 1: bubbling. Add an `onClick` to the button that logs `BUTTON`. Click the button. How many messages do you get, and in what order? Explain it in a comment.

Part 2: stop it. Change the button's handler so clicking it logs **only** `BUTTON`. Clicking the grey area must still log `OUTER div`.

Part 3: target vs currentTarget. Write the button's handler as a separate, properly typed function that logs both `event.target` and `event.currentTarget`. Put an `<span>Delete</span>` **inside** the button and click exactly on the word. Are the two the same? Explain why in a comment.

Part 4: the keyboard. Add an `<input type="text" />`. Log `Enter pressed` when someone presses Enter in it, and nothing for any other key. Type a normal sentence to confirm you're not logging on every keystroke.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A form that doesn't reload the page

In `src/ch07/ex4/Ex4.tsx`, build a "Join the newsletter" form: a labelled email box, a labelled name box, and a **Subscribe** button.

1. Give the form an `onSubmit` handler that logs `Form submitted`. Click **Subscribe**. **What happens to the page?** Watch the browser tab and the Console carefully, then write down what you saw.
2. Fix it so the page stays put.
3. Now click into the email box and press Enter instead of clicking the button. Does it still work? Why?
4. Move your handler off the `<form>` and onto the button's `onClick` instead. Does Enter still work now? Put it back on the form, and write one sentence on why `onSubmit` on the `<form>` is the right place.
5. Add an `onFocus` to the email box that logs `Editing email` and an `onBlur` that logs `Left email`. Click between the two boxes and watch the order of the messages.
6. Use `<label htmlFor="...">` properly for both boxes, and check it works by clicking the label text: the cursor should jump into the box.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A to-do list that talks back

This is a dry run for the chapter 10 project. The list won't actually change yet (no state until chapter 08), but every single interaction will report itself correctly, and the components will be shaped exactly the way the real app needs.

In `src/ch07/ex5/`, start with `types.ts`:

```
export type Task = {
  id: string;
  text: string;
  done: boolean;
};
```

Build three components, each in its own file:

`TaskRow` takes one `task`, plus three function props: `onToggle`, `onDelete`, and `onEdit`. It renders an `<li>` containing:

- a tick box whose `checked` matches `task.done`, with an `onChange` that calls `onToggle` with the task's id,
- the task text, crossed out with `textDecoration: "line-through"` when it's done,
- a `✎` button calling `onEdit` with the id,
- a `✕` button calling `onDelete` with the id.

`TaskList` takes a `tasks` array (readonly) and passes the three functions straight through to each row. It shows `Nothing to do!` when the list is empty.

`Ex5` holds the array, defines the three handler functions (each just logs what it would do, with the task's **text**, not just the id), and renders the list plus an `Add task` form with a text box and a button.

Requirements:

1. Clicking `✕` on a row must log **only** that row's delete message. Wrap the whole `<li>` in an `onClick` that logs `Row clicked` and make sure the buttons don't trigger it.
2. The Add form must not reload the page, and pressing Enter in the box must work the same as clicking the button.
3. Every handler function that takes an event is separately defined and properly typed. No `any` anywhere.
4. In a comment at the bottom of `Ex5.tsx`, answer: `TaskRow` knows the id but not what deleting means, and `Ex5` knows what deleting means but doesn't have a button. Why is it a good thing that those two facts live in different files?
5. In the same comment, list everything that's still missing before this is a real to-do app. (There are two big things, and they're chapters 08 and 09.)

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

08 State

What is it?

State is a component's memory: a value it keeps between renders, and that tells React to redraw the page when it changes.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

That's the whole of React's interactivity, in six lines. This is the most important chapter in Level 1.

Why does it matter?

[Chapter 07](#) ended with a broken counter:

```
function Counter() {
  let count = 0;

  function handleClick() {
    count = count + 1;
    console.log(count); // 1, 2, 3...
  }

  return <button onClick={handleClick}>Clicked {count} times</button>;
}
```

The Console counts up. The button says **0** forever. Two separate problems:

1. **The variable doesn't survive.** Every time React renders `Counter`, it runs the whole function again, and `let count = 0` starts over. Local variables die when the function ends.
2. **Nothing tells React to redraw.** Changing a variable is invisible to React. It has no idea anything happened.

You need something that survives re-renders **and** tells React when it changes. Neither a `let` inside the component nor a variable outside it can do both. `useState` does both.

This is also the thing that makes React worth using at all. Back in JavaScript chapter 24, you had to remember to call `render()` after every change to your `tasks` array. Miss one call, and the page silently showed stale data. With state, redrawing isn't something you remember to do. It's automatic.

Real-world example

Think about a **café's chalkboard** showing how many cakes are left.

The café	React
The number written on the board	The state value, <code>count</code>
Selling a cake	An event
The one person allowed to change the board	The setter, <code>setCount</code>
Everyone reads the board to know the number	Your JSX reads <code>count</code>
The board is rewritten, so everyone sees the new number	React re-renders
Muttering "that's 4 now" to yourself	Changing a local variable: nobody else finds out

The rule that makes it work: **you never scribble on the board yourself**. You tell the one person whose job it is. In React, that's the setter, and going around it is the single most common source of "why isn't my page updating?".

How it works

`useState`

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  // ...
}
```

Line by line:

- `useState(0)` says "I need a piece of memory, starting at `0`".

- It gives back an **array of two things**: the current value, and a function that changes it.
- `const [count, setCount] = ...` is array destructuring (JavaScript chapter 15). You pick both names.
- The convention is `thing` and `setThing`. Follow it; every React codebase does.
- **Import it from `react`**. Forget, and you get `Cannot find name 'useState'`.

Why an array and not an object? Purely so you can name them whatever you like: `useState` doesn't know if you're storing a count, a name, or a list of tasks.

What actually happens on a click

This sequence is worth learning properly, because almost every React question comes back to it:

```
function Counter() {
  const [count, setCount] = useState(0);
  console.log("rendering with", count);

  return <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>;
}
```

1. **First render.** React runs `Counter`. `useState(0)` sets the value to `0` and returns it. The button reads "Clicked 0 times". The Console says `rendering with 0`.
2. **You click.** The handler runs `setCount(1)`.
3. **React makes a note.** "This component's state should be 1 now. I'll re-render it."
4. **React runs `Counter` again.** This time `useState(0)` returns `1`, not `0`. The `0` was only ever the *starting* value; React ignores it from then on.
5. The button now reads "Clicked 1 times". The Console says `rendering with 1`.

Two things to take from that:

- `count` is a `const` that changes between renders, not during one. Inside a single render it never changes, which is why `const` is correct.
- **React calls your function again.** Your component runs from the top every time. That's why a `let` inside it resets, and why state has to live somewhere else — inside React.

Add that `console.log` to something you're building and watch it. It makes the whole model click.

Typing state

Most of the time, TypeScript works it out from the starting value:

```
const [count, setCount] = useState(0);           // number
const [name, setName] = useState("");           // string
const [isOpen, setIsOpen] = useState(false);    // boolean
```

Hover over `setCount: (value: number | ((prev: number) => number)) => void`. It only accepts numbers. `setCount("hello")` is an error, in this file and everywhere else.

Sometimes inference isn't enough, and there are two big cases.

An empty array. TypeScript sees `[]` and, with nothing to go on, infers `never[]` — "an array that can never contain anything":

```
const [tasks, setTasks] = useState([]);
setTasks([{ id: "1", text: "Buy milk", done: false }]);
// ✖ Type '{ id: string; ... }' is not assignable to type 'never'.
```

Say what you mean, using the angle-bracket syntax from TypeScript chapter 08:

```
const [tasks, setTasks] = useState<Task[]>([]);
```

Something that starts empty and fills in later. A value that's `null` now and an object later:

```
const [user, setUser] = useState<User | null>(null);
```

Without the annotation, TypeScript infers plain `null`, and you could never put a user in it. With it, you get a union (TypeScript chapter 06), and TypeScript makes you check before you use it:

```
if (user === null) {
  return <p>Not signed in.</p>;
}
return <p>Hello, {user.name}</p>;
```

That check isn't an annoyance. It's the "not signed in" case, which your app needs anyway.

Rule of thumb: let TypeScript infer when the starting value is a real example of the data. Annotate when it's a placeholder — `[]`, `null`, or `undefined`.

Setting state replaces, it doesn't merge

```
const [user, setUser] = useState({ name: "Maya", age: 30 });

setUser({ name: "Tom" });
// ✖ Property 'age' is missing in type '{ name: string; }' ...
```

The setter swaps the whole value. It doesn't merge in the way you might expect from other tools. To change one field, spread the rest (JavaScript chapter 15):

```
setUser({ ...user, name: "Tom" });
```

TypeScript catches the missing field here, which is a nice safety net. [Chapter 11](#) is entirely about doing this well for objects and arrays.

State updates are not instant

This one genuinely surprises everybody:

```
function handleClick() {  
  setCount(count + 1);  
  console.log(count);    // still the OLD number!  
}
```

`setCount` doesn't reach into the variable and change it. It tells React "re-render this component with a new value". The current render's `count` is a `const`; it was fixed the moment this render started, and nothing can change it now. The new number shows up in the **next** render.

React also **batches**: several setter calls in one handler cause one re-render, not several. That's why this famous bug happens:

```
function handleClick() {  
  setCount(count + 1);  
  setCount(count + 1);  
  setCount(count + 1);  
}
```

Click it and the count goes up by **one**, not three. With `count` at `0`, all three lines say "set it to 1".

The updater function

When the new value depends on the old one, pass a **function** instead of a value. React runs it with the latest value, in order:

```
function handleClick() {  
  setCount((prev) => prev + 1);  
  setCount((prev) => prev + 1);  
  setCount((prev) => prev + 1);  
}
```

Now the count goes up by three: `0 → 1 → 2 → 3`.

When to use which:

- New value doesn't depend on the old one? Pass the value: `setName("Tom")`, `setTasks([])`.
- New value is built from the old one? Pass a function: `setCount((prev) => prev + 1)`, `setTasks((prev) => [...prev, newTask])`.

The updater form is never wrong, so when in doubt, use it. It becomes essential from [chapter 17](#), where the value you captured could genuinely be stale.

(`prev` is just a parameter name. `c`, `current` and `oldCount` are all fine.)

Each component has its own state

```
function Ex() {
  return (
    <>
      <Counter />
      <Counter />
    </>
  );
}
```

Two counters, counting separately. That's what [chapter 03](#) meant by "each use is its own thing". React keeps state per **place in the tree**, not per component function.

Which leads to the natural question: what if two components need the *same* number? Then the state has to live in a component above both of them, and come down as props. That's **lifting state up**, and it's [chapter 13](#). For now, when you're deciding where a piece of state goes, put it in the **closest component that contains everything that needs it**.

State goes down as props

State and props aren't rivals. They work together:

```
function TaskApp() {
  const [tasks, setTasks] = useState<Task[]>([]);

  function handleDelete(id: string) {
    setTasks(tasks.filter((task) => task.id !== id));
  }

  return <TaskList tasks={tasks} onDelete={handleDelete} />;
}
```

- **State** lives in one component, and only that component can change it.
- **Props** carry it down to whoever needs to show it.
- **Functions passed down as props** ([chapter 07](#)) let children ask for changes.

That triangle — state down as props, events back up as functions — is the shape of essentially every React app. Read `TaskApp` above a few times; the chapter 10 project is that component with more features.

Don't change state directly

```
const [tasks, setTasks] = useState<Task[]>([]);

tasks.push(newTask);           // ❌ nothing happens on screen
setTasks([...tasks, newTask]); // ✅
```

`push` changes the array in place. React compares the old value to the new one to decide whether to re-render, and it's the *same array object*, so React concludes nothing changed. Your data is now different from what's on screen, which is the worst kind of bug.

Make a new value every time:

```
setTasks([...tasks, newTask]);           // add
setTasks(tasks.filter((task) => task.id !== id)); // remove
setTasks(tasks.map((task) =>             // change one
  task.id === id ? { ...task, done: !task.done } : task
));
```

Those three lines are the bulk of the chapter 10 project. [Chapter 11](#) drills them properly.

Don't store what you can calculate

A very common beginner move:

```
const [tasks, setTasks] = useState<Task[]>([]);
const [taskCount, setTaskCount] = useState(0); // ❌
```

Now every change has to update two things, and the day you forget one, the count is wrong. Just work it out during the render:

```
const [tasks, setTasks] = useState<Task[]>([]);
const taskCount = tasks.length;
const remaining = tasks.filter((task) => !task.done).length;
```

These are **derived values**. They can't go stale, because they're recalculated every render. (Worried about the cost? Don't be. [Chapter 27](#) deals with the rare case where it matters.)

Ask of every new piece of state: can I work this out from something I already have? If yes, don't store it.

Hooks, and their two rules

`useState` is a **hook**: a function that hooks your component into a React feature. You can spot one by its name — hooks always start with `use`. You'll meet `useEffect`, `useRef`, `useContext` and others later, and they all follow the same two rules.

Rule 1: call hooks at the top level of your component. Never inside an `if`, a loop, or a nested function.

```
function Bad({ isOpen }: Props) {  
  if (isOpen) {  
    const [count, setCount] = useState(0);    // ✖  
  }  
}
```

React doesn't know your state's *name*. It keeps state in a list, in the order the hooks were called, and matches them up by position on every render. Skip a hook one time round and everything after it shifts by one, which is exactly as bad as it sounds. React usually catches it:

React has detected a change in the order of Hooks called by Bad.

Your linter should catch it too, before you even save.

This is why early returns go *after* your hooks — the thing [chapter 05](#) told you to tuck away:

```
function TaskList({ tasks }: Props) {  
  const [filter, setFilter] = useState("all");    // hooks first  
  
  if (tasks.length === 0) {                      // then early returns  
    return <p>Nothing to do!</p>;  
  }  
  // ...  
}
```

Rule 2: only call hooks from React components or from other hooks. Not from plain functions, not from event handlers.

Two more things worth knowing now

Lazy initial state. The argument to `useState` is only used on the first render, but if it's a function *call*, that call still happens every render:

```
const [tasks, setTasks] = useState(loadFromStorage()); // runs every render
const [tasks, setTasks] = useState(() => loadFromStorage()); // runs once
```

Pass a function and React only calls it the first time. It matters for anything slow, like reading `localStorage` or parsing JSON, and you'll want it in [chapter 10](#).

Never set state during render.

```
function Bad() {
  const [count, setCount] = useState(0);
  setCount(count + 1); // ✗
  return <p>{count}</p>;
}
```

Setting state causes a re-render, which sets state, which causes a re-render:

Too many re-renders. React limits the number of renders to prevent an infinite loop.

Set state in **event handlers**, or in effects ([chapter 17](#)). Never in the body of a component.

Seeing state in DevTools

Open the **Components** tab, click a component, and its hooks are listed on the right under **hooks** — `State: 0`, and so on. You can even edit the value there and watch the page update. When something's wrong, this tells you in one glance whether the state is wrong or only the display is.

Common mistakes

1. Changing the variable instead of calling the setter

```
count = count + 1; // ✗ and TypeScript stops you: count is a const
tasks.push(newTask); // ✗ TypeScript allows this one, and it silently does nothing
```

The second is the dangerous one. Always make a new value and pass it to the setter.

2. Expecting the new value immediately

```
setCount(count + 1);  
console.log(count); // the old number
```

The new value arrives on the next render. To log what you're about to set, log `count + 1`.

3. Setting from a stale value repeatedly

```
setCount(count + 1);  
setCount(count + 1); // goes up by one, not two
```

Use the updater: `setCount((prev) => prev + 1)`.

4. `useState([])` with no type

```
const [tasks, setTasks] = useState([]);  
// ❌ Type 'Task' is not assignable to type 'never'.
```

Write `useState<Task[]>([])`.

5. Calling a hook inside a condition

```
if (isOpen) {  
  const [x, setX] = useState(0); // ❌  
}
```

Hooks go at the top level, before any early return.

6. Setting state during render

Too many re-renders.

Look for a setter called in the component body, or a stray `()` in an event attribute: `onClick={handleClick()}`.

7. Storing what you could calculate

```
const [remaining, setRemaining] = useState(0); // ❌ goes stale  
const remaining = tasks.filter((t) => !t.done).length; // ✅ can't go stale
```

8. Forgetting to import `useState`

Cannot find name 'useState'.

`import { useState } from "react";` at the top. VS Code will add it if you accept its suggestion.

9. One giant piece of state for unrelated things

```
const [stuff, setStuff] = useState({ tasks: [], filter: "all", isDark: false });
```

Now every change means spreading the whole object. Use separate `useState` calls for things that change separately. Group them only when they really change together.

Quick recap

- `const [value, setValue] = useState(initial)` gives a component a memory that survives re-renders and tells React to redraw when it changes.
- React runs your component function again on every render. The starting value is only used the first time.
- **State updates aren't instant.** After `setCount(count + 1)`, `count` is still the old value until the next render.
- When the new value depends on the old one, pass a function: `setCount((prev) => prev + 1)`.
- **Never change state in place.** Make a new array or object and pass it to the setter.
- Don't store what you can calculate. Derived values can't go stale.
- Hooks are called at the **top level** of a component, before any early return, and never inside an `if` or a loop.
- State lives in one component and flows **down** as props; children ask for changes through **functions** passed down as props.

Next: try the [exercises](#), then move on to [09 Forms](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch08/ex1/`, and so on.
- **Your pages finally do things.** Click everything, repeatedly, in the wrong order, and see if it holds up.
- Keep the **Components** tab open next to the page. Click your component and watch its state change as you click. It's the single best way to learn this chapter.
- An exercise is done when it behaves correctly however you click it, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console is empty.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): The counter, three ways

In `src/ch08/ex1/Ex1.tsx`, build a counter with four buttons: `+1`, `-1`, `+10`, and `Reset`.

1. Get it working with `useState`.
2. Add `console.log("rendering with", count);` above the `return`. Click a few times and read the Console. How many times does the component run for one click?
3. Add a fifth button, `+3`, that calls `setCount(count + 1)` **three times in a row**. How much does the count go up by? Explain it in a comment.
4. Fix that button using the updater form so it really adds 3.
5. Add a button whose handler is:

```
function handleShow() {  
  setCount(count + 1);  
  console.log("count is now", count);  
}
```

What does it log? Is the message right or wrong? Rewrite the log line so it prints the number the page is about to show.

6. Stop the count going below zero. `-1` at zero should do nothing.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Light switch and secret message

Three small pieces of state, in `src/ch08/ex2/Ex2.tsx`.

1. **A light switch.** A button that toggles between  `on` and  `off`. The page's background colour changes with it (use the `style` attribute on a wrapping `<div>`).
2. **A show/hide.** A button labelled `Show answer` / `Hide answer` that reveals a paragraph underneath. When it's hidden, the paragraph must **not be in the page at all** — check the Elements tab, not just your eyes.
3. **A step counter with a limit.** A `Next` button that walks through the strings `["Wake up", "Coffee", "Emails", "Lunch"]` one at a time, showing `Step 2 of 4: Coffee`. At the last step, the button is disabled and the page says `All done!`.

Then answer in a comment: you now have three `useState` calls in one component. Why three, rather than one object holding all three values?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Shopping cart

In `src/ch08/ex3/`, build a cart. Start with:

```
export type Product = {
  id: string;
  name: string;
  price: number;
};

export const products: Product[] = [
  { id: "p1", name: "Notebook", price: 3.5 },
  { id: "p2", name: "Backpack", price: 34.99 },
  { id: "p3", name: "Sticky notes", price: 1.99 },
];
```

Build:

- A product list. Each product has an **Add to cart** button.
- A cart below it, listing what's been added, with a `×` button on each line to remove it.
- A line showing `3 items – $40.48`, with both numbers worked out.
- A **Clear cart** button.
- `Your cart is empty.` when it is, and no summary line or Clear button in that case.

Rules:

1. The cart's state is an array of items. Give each **cart item** its own id when it's added, so adding the same notebook twice gives you two removable lines. Use `crypto.randomUUID()`.
2. **No `useState` for the count or the total.** Both are derived.
3. Nothing in your code may use `push`, `splice`, or `sort` on the state.
4. Test it hard: add the same product three times, remove the middle one, add another, clear, add again. It should never get confused.

Then in a comment: which line of your code would break if you'd used the array index as the React `key`? ([Chapter 06](#), if you need reminding.)

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Quiz

In `src/ch08/ex4/`, build a three-question quiz.

```
export type Question = {
  id: string;
  prompt: string;
  options: string[];
  correctIndex: number;
};

export const questions: Question[] = [
  {
    id: "q1",
    prompt: "What does JSX turn into?",
    options: ["HTML", "JavaScript function calls", "CSS"],
    correctIndex: 1,
  },
  {
    id: "q2",
    prompt: "What must every item in a list have?",
    options: ["A key", "An index", "A class"],
    correctIndex: 0,
  },
  {
    id: "q3",
    prompt: "What does useState give back?",
    options: ["A value", "A value and a setter", "A component"],
    correctIndex: 1,
  },
];
```

How it works:

1. One question on screen at a time, with its options as buttons.
2. Clicking an option locks the answer in: the buttons stop responding, the chosen one turns green or red, and the correct one is always shown in green.
3. A **Next question** button appears only after an answer is given. On the last question it says **See results**.
4. The results screen shows `You scored 2 out of 3`, a different message for a perfect score, and a **Try again** button that puts everything back to the start.

Keep it to three pieces of state, and make sure none of them is something you could derive.

Stuck? [Hint 1](#), [Hint 2](#), [Hint 3](#) and [Hint 4](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Seat booking

In `src/ch08/ex5/`, build a cinema seat picker. This one is about getting the state *shape* right, which is the real skill this chapter teaches.

The cinema has 4 rows (A to D) of 6 seats. Some are already taken:

```
export const takenSeats = ["A3", "B1", "B2", "D6"];
export const seatPrice = 9.5;
```

The page shows:

Pick your seats

```
A  [1][2][X][4][5][6]
B  [X][X][3][4][5][6]
C  [1][2][3][4][5][6]
D  [1][2][3][4][5][X]
```

Selected: B3, C4

2 seats – \$19.00

[Book now] [Clear]

Requirements:

1. Build the grid with **nested map s** over the rows and the seat numbers. Don't write out 24 buttons.
2. A taken seat shows `x`, is disabled, and is grey.
3. Clicking a free seat selects it (green). Clicking it again deselects it. This must be a **toggle**, and you must be able to select and deselect in any order without the list getting duplicates.
4. Maximum **4 seats**. At 4, every unselected seat becomes disabled and a message says `You can book at most 4 seats.`
5. `Selected: ...` lists the seats **in row order**, not the order you clicked them. `Book now` is disabled with nothing selected.
6. `Book now` swaps the whole page for `Booked! Enjoy the film.` and a `Book more seats` button that resets everything (the seats you booked stay taken for the rest of the session — a nice extra, if you want it).
7. Only two pieces of state. If you find yourself adding a third, ask whether it's derivable.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

09 Forms

What is it?

A form in React is state plus events, wired together. You put what someone typed into state, and you show it back from state:

```
const [email, setEmail] = useState("");

<input
  type="email"
  value={email}
  onChange={(event) => setEmail(event.target.value)}
/>
```

An input wired up like that is called a **controlled input**: React controls what's in it. Those two lines — `value` and `onChange` — are 90% of this chapter.

Why does it matter?

In plain HTML, a text box keeps its own value. You type, the box remembers, and when you want to know what's in it you go and ask (JavaScript chapter 22):

```
const email = document.querySelector("#email").value;
```

That's fine when you only look once, at the end. But real forms need more:

- Disable the Submit button until the form is valid.
- Show "That doesn't look like an email address" as someone types.
- Count characters left: `47 / 280`.
- Have one field change another: pick a country, and the list of cities changes.
- Fill the form in from data you loaded.

Each of those means knowing what's in the box *right now*. If the box holds the truth, you're constantly running off to ask it, and nothing on the page knows to update when the answer changes.

Controlled inputs turn it round. **Your state holds the truth, and the box just displays it.** Everything else on the page can read that state and react to it, because that's what state does.

Real-world example

Think about ordering at a **café counter** in two different ways.

	Uncontrolled (plain HTML)	Controlled (React)
Where the order lives	Written on the customer's own bit of paper	Written in the café's ledger
Changing it	The customer scribbles on their paper	You tell the clerk, who writes it in the ledger
To find out the order	Ask to see their paper	Read the ledger
Who else can see it	Nobody, until they hand it over	Anyone in the café, all the time
Showing the running total	You can't, until the end	Trivially, from the ledger

With the ledger, everything the café does — totals, offers, "we're out of that" — works from one source. There's a bit more ceremony to write an order in, but one thing is always true, everywhere.

How it works

One controlled input

```
import { useState } from "react";

function NameForm() {
  const [name, setName] = useState("");

  return (
    <div>
      <label htmlFor="name">Your name</label>
      <input
        id="name"
        type="text"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      <p>Hello, {name || "stranger"}!</p>
    </div>
  );
}
```

Type in the box and the greeting updates on every keystroke. Follow the loop:

1. You press a key.
2. `onChange` fires. `event.target.value` is what's in the box now.
3. `setName` puts it in state.
4. React re-renders. `value={name}` puts the new text in the box.

So the letter you see isn't the one the browser put there. It's the one that came back from state. That round trip is the whole idea.

Three pieces, every time:

- `value={name}` — the box shows the state.
- `onChange={...}` — typing updates the state.
- `useState("")` — somewhere to keep it. Start with an empty string, not `undefined`.

And `<label htmlFor="name">` with a matching `id` is not optional politeness: it lets people click the label to focus the box, and it's what a screen reader announces ([chapter 38](#)).

value without onChange

Miss the `onChange` and React tells you:

```
You provided a `value` prop to a form field without an `onChange` handler.  
This will render a read-only field.
```

You'll see this as a box you literally cannot type in. It's the same thing you saw in the chapter 07 exercises with the stuck tick box: React is putting the value back the way the state says, every time. The state just never changes.

If you genuinely want a read-only box, say so: add `readOnly`.

Typing the change handler

Inline, TypeScript works it out ([chapter 07](#)):

```
onChange={(event) => setName(event.target.value)}
```

Hover over `event`: `React.ChangeEvent<HTMLInputElement>`. Separately, you annotate:


```
function handleChange(event: React.ChangeEvent<HTMLInputElement>) {
  setName(event.target.value);
}
```

For change events, `event.target` is typed properly (unusually — it's `event.currentTarget` you normally trust). Both work here.

Several fields

The simple way is one `useState` per field:

```
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [message, setMessage] = useState("");
```

Slightly repetitive, completely clear, and each field is typed on its own. For up to about four fields, this is the right answer and you shouldn't feel clever about avoiding it.

For bigger forms, one object with one handler:

```
type ContactForm = {
  name: string;
  email: string;
  message: string;
};

const [form, setForm] = useState<ContactForm>({
  name: "",
  email: "",
  message: "",
});

function handleChange(event: React.ChangeEvent<HTMLInputElement>) {
  const { name, value } = event.target;
  setForm({ ...form, [name]: value });
}
```

```
<input name="name" value={form.name} onChange={handleChange} />
<input name="email" value={form.email} onChange={handleChange} />
```

The `name` attribute on each input tells the handler which field to change, and `[name]: value` is a **computed property name** (JavaScript chapter 11). The `...form` spread is essential: without it you'd replace the whole object and lose the other fields.

Be aware of the trade: TypeScript can't check that `name="email"` matches a real key here. That's one reason the separate-`useState` version is nicer while forms are small, and why [chapter 33](#) exists for when they're big.

Submitting

Put `onSubmit` on the `<form>` and stop the reload:

```
function ContactForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    console.log("Sending", { name, email });
    setName("");
    setEmail("");
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input id="name" value={name} onChange={(e) => setName(e.target.value)} />

      <label htmlFor="email">Email</label>
      <input id="email" type="email" value={email} onChange={(e) => setEmail(e.target.value)} />

      <button type="submit">Send</button>
    </form>
  );
}
```

Points worth noticing:

- `onSubmit` on the `<form>`, not `onClick` on the button. Then pressing Enter in a text box submits too, which people expect.
- `event.preventDefault()` first. Without it the page reloads and every piece of state is wiped ([chapter 07](#)).
- `type="submit"` on the button. Inside a form, that's the default, but saying it is clearer.
- **Clearing the form is just setting the state back.** No DOM poking.

If a button inside a form shouldn't submit — a "Clear" button, say — give it `type="button"`, or it will submit the form.

The other input types

Most inputs work like the text box. A few don't.

Checkbox: `checked`, `not value`.

```
const [agreed, setAgreed] = useState(false);

<label>
  <input
    type="checkbox"
    checked={agreed}
    onChange={(event) => setAgreed(event.target.checked)}
  />
  I agree to the terms
</label>
```

State is a boolean, and you read `event.target.checked`. Putting the input inside the `<label>` is a neat alternative to `htmlFor`.

Radio buttons: one piece of state for the whole group.

```
const [size, setSize] = useState("medium");

[["small", "medium", "large"].map((option) => (
  <label key={option}>
    <input
      type="radio"
      name="size"
      value={option}
      checked={size === option}
      onChange={(event) => setSize(event.target.value)}
    />
    {option}
  </label>
))]}
```

`checked={size === option}` is what makes exactly one of them selected. They all need the same `name`.

Select: `value` on the `<select>`.

```
const [country, setCountry] = useState("uk");

<select value={country} onChange={(event) => setCountry(event.target.value)}>
  <option value="uk">United Kingdom</option>
  <option value="np">Nepal</option>
</select>
```

This is different from HTML, where you'd put `selected` on an `<option>`. In React the `<select>` owns the value. Its event type is `React.ChangeEvent<HTMLSelectElement>`.

Textarea: `value` , not text between the tags.

```
<textarea value={message} onChange={(event) => setMessage(event.target.value)} />
```

Again different from HTML, and again more consistent: everything is `value` . Its event type is `React.ChangeEvent<HTMLTextAreaElement>` .

Number inputs still give you a string.

```
<input type="number" value={age} onChange={(event) => setAge(event.target.value)} />
```

`event.target.value` is **always** a string, even here. So either keep the state as a string and convert when you use it, or convert as you set it:

```
const [age, setAge] = useState(0);

onChange={(event) => setAge(Number(event.target.value))}
```

Careful with the second one: clearing the box gives `""` , and `Number("")` is `0` , so the box refills itself with a `0` you can't delete. For inputs people edit freely, **keep the state as a string** and convert at the point of use. It's the pragmatic choice, and the one most real forms make.

Validation, and when to show it

The simplest version is derived, calculated on every render:

```
const [email, setEmail] = useState("");
const emailIsValid = email.includes("@") && email.includes(".");

<button type="submit" disabled={!emailIsValid}>Send</button>
{!emailIsValid && email !== "" && <p style={{ color: "crimson" }}>That doesn't look like an
email.</p>}
```

Two things worth copying there:

- The error is a **derived value**, not state. It can't go stale.
- `email !== ""` stops the error shouting at someone who hasn't typed anything yet. Nobody likes a form that's angry before you start.

A common, kinder pattern is to show errors only after a field has been left, using `onBlur` :

```
const [emailTouched, setEmailTouched] = useState(false);

<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
  onBlur={() => setEmailTouched(true)}
/>
{emailTouched && !emailIsValid && <p>That doesn't look like an email.</p>}
```

Track "have they finished with this field yet?" and you've got a form that feels polite.

This is fine for small forms. It also clearly doesn't scale: three fields with three rules each is a lot of booleans. That's what [Zod](#) and [React Hook Form](#) are for in Level 4. Learn the plain version first so you know what they're doing for you.

(The browser's own `required`, `minLength` and `type="email"` are still worth adding. They cost nothing and they work before your JavaScript loads.)

Uncontrolled inputs

You don't *have* to control every input. Left alone, an input keeps its own value, and you can give it a starting one with `defaultValue`:

```
<input type="text" defaultValue="Maya" />
```

Then you read it when the form is submitted, using a ref ([chapter 19](#)) — or, for a whole form at once, `FormData`.

Use `defaultValue`, never `value`, for these. `value` with no `onChange` gives you the read-only field from earlier.

It's genuinely useful for big forms you only read once, and for file inputs, which *must* be uncontrolled (the browser won't let you set a file input's value — imagine if a page could). But controlled is the default, and it's what the rest of this course uses.

Don't switch between the two by accident:

A component is changing an uncontrolled input to be controlled.

That warning means a `value` was `undefined` on the first render and a real string later. Start your state at `""`, not `undefined`, and it can't happen.

What React 19 adds

React 19 lets you pass a function straight to a form's `action`, and adds hooks like `useActionState` for handling the result. They're built for React frameworks that run code on the server, so this course meets them in [chapter 39](#). Everything here works exactly as described in React 19, and `onSubmit` isn't going anywhere.

Common mistakes

1. `value` with no `onChange`

You provided a `value` prop to a form field without an `onChange` handler.

The box won't accept typing. Add the handler, or add `readOnly`.

2. Starting state as `undefined`

```
const [name, setName] = useState<string>(); // undefined at first
```

Gives you the "changing an uncontrolled input to be controlled" warning as soon as someone types. Start at `""`.

3. Forgetting `preventDefault`

The page reloads, all state is lost, and the Console clears before you can read it. Turn on **Preserve log** if you're not sure that's what happened.

4. `onClick` on the submit button instead of `onSubmit` on the form

Works when clicked, does nothing on Enter. Half your users will think the form is broken.

5. `value` on a checkbox

```
<input type="checkbox" value={agreed} /> // ❌  
<input type="checkbox" checked={agreed} /> // ✅
```

Checkboxes use `checked`, and you read `event.target.checked`.

6. Expecting a number from a number input

```
const total = price * quantity; // "3" * 2 happens to work; "3" + 2 gives "32"
```

`event.target.value` is always a string. Convert deliberately, where you use it.

7. Replacing the whole object instead of spreading it

```
setForm({ [name]: value });  
// ❌ Type '{ [x: string]: string; }' is missing the following properties...
```

Spread the rest: `setForm({ ...form, [name]: value })`. TypeScript catches this one, which is a good reason to type your form object.

8. A Clear button that submits the form

```
<button onClick={handleClear}>Clear</button>    // inside a <form>: submits!  
<button type="button" onClick={handleClear}>Clear</button>    // ✅
```

9. Inputs with no label

`<input placeholder="Email" />` on its own is not accessible: the placeholder vanishes the moment someone types, and screen readers may not announce it. Use a real `<label>`.

Quick recap

- A **controlled input** takes `value` from state and updates state in `onChange`. Your state is the truth; the box only displays it.
- The pattern is always `value={x} + onChange={(e) => setX(e.target.value)} + useState("")`.
- `value` with no `onChange` gives a read-only field and a warning. Start state at `""`, never `undefined`.
- `onSubmit` goes on the `<form>`, and `event.preventDefault()` is the first line of the handler. Clearing a form means resetting its state.
- Checkboxes use `checked` and `event.target.checked`. Radios share one piece of state and one `name`. `<select>` and `<textarea>` both use `value`, unlike in HTML.
- Number inputs still give strings. Keep the state as a string and convert where you use it.
- Validation can just be a derived value. Use `onBlur` to hold errors back until someone has finished with a field.
- Every input gets a real `<label>`.

Next: try the [exercises](#), then build the [10 Project: To-Do App, the React Way](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch09/ex1/`, and so on.

- Keep the **Components** tab open and watch the state change as you type. Watching a string grow one letter at a time is the fastest way to believe how controlled inputs work.
- Test every form by **pressing Enter** as well as clicking the button. Half of all form bugs only show up that way.
- An exercise is done when it works however you use it, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no errors or warnings.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Feel the loop

In `src/ch09/ex1/Ex1.tsx`:

```
import { useState } from "react";

function Ex1() {
  const [name, setName] = useState("");

  return (
    <div>
      <label htmlFor="name">Your name</label>
      <input id="name" type="text" value={name} />
      <p>Hello, {name}!</p>
    </div>
  );
}

export default Ex1;
```

1. Try typing in the box. What happens, and what does the Console say?
2. Fix it with an `onChange`.
3. Change the handler to `setName(event.target.value.toUpperCase())`. Type `may`. Explain in a comment why the box itself shows capitals, even though you typed lowercase.
4. Put it back, and add:
 - a live count: `4 / 20 characters`,
 - the box refusing anything over 20 characters (do it in the handler, not with `maxLength`),
 - `Hello, stranger!` when the box is empty,
 - a **Clear** button that empties it.
5. Now delete `value={name}` but keep the `onChange`. Type something. Does the greeting still work? Does the Clear button still work? Explain what you learned about what `value` is actually for.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Sign-up form

In `src/ch09/ex2/Ex2.tsx`, build a sign-up form with:

- **Email** (a text box)
- **Password** (`type="password"`)
- **I agree to the terms** (a checkbox)
- A **Create account** button

Rules:

1. One `useState` per field. Every field has a real `<label>`.
2. The button is disabled unless the email contains an `@`, the password is at least 8 characters, and the box is ticked.
3. Under the password, show `Password must be at least 8 characters` in red — but only once someone has started typing in it.
4. Submitting logs the email and the password length (never the password itself), then clears the whole form, tick box included.
5. Pressing Enter in the email box submits.
6. Add a **Clear** button next to Submit that empties everything without submitting. Get this right: it's the one in mistake 8.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Pizza order

One form, every input type. In `src/ch09/ex3/Ex3.tsx`:

Field	Control	Options
Your name	text	
Size	radio	Small (\$8), Medium (\$11), Large (\$14)
Crust	select	Thin, Thick, Stuffed (+\$2)
Toppings	checkboxes	Mushroom, Olives, Pepper, Ham (each +\$1.50)
How many	number	1 to 10
Notes for the kitchen	textarea	max 200 characters

Requirements:

1. A live **order summary** underneath that updates as you change anything, ending with a total price. Nothing in that summary is stored in state.
2. Toppings are one piece of state — an array of the chosen topping names — not four booleans.
3. The quantity box must survive being cleared. Empty it completely: no stray `0` should appear, and the total should handle it sensibly.
4. The notes box shows `12 / 200` and won't go over.
5. Submitting logs the whole order as one object and shows `Thanks, Maya! Your order is in.` in place of the form, with a **Start another order** button that resets everything.

Stuck? [Hint 1](#), [Hint 2](#), [Hint 3](#) and [Hint 4](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): Searchable, filterable list

Forms aren't only for sending data. Here they drive what's on screen. In `src/ch09/ex4/`, start with:

```
export type Book = {
  id: string;
  title: string;
  author: string;
  year: number;
  genre: "fiction" | "science" | "history";
  available: boolean;
};
```

Write eight books of your own across all three genres, some available and some not.

Build controls above the list:

- A **search box** matching the title *or* the author, ignoring capitals.
- A **genre select**, including an `All genres` option.
- An **Available only** checkbox.
- A **sort select**: `Title A-Z`, `Newest first`, `Oldest first`.
- A **Reset filters** button.

And below them:

1. The filtered, sorted list.
2. `Showing 3 of 8 books`.
3. `No books match your search.` when nothing does — with the controls still on screen, so people can undo what they did.

Rules:

- The book list itself is **not** state. Neither is the filtered list. Only the four control values are.
- Sorting must not change the original array.
- Type the genre select's state so only the three genres plus "all" are possible.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Multi-step booking form

The final rehearsal before the project. In `src/ch09/ex5/`, build a three-step restaurant booking form. One step on screen at a time, with `Back` and `Next` buttons and a `Step 2 of 3` indicator.

Step 1 — Who: name (required), email (must contain `@` and `.`), phone (optional).

Step 2 — When: date (`type="date"`), time (a select of `18:00` to `21:00` in half hours), number of guests (1 to 12).

Step 3 — Extras: a textarea for dietary requirements, a checkbox for `It's a birthday`, and a radio group for `Inside` / `Outside` / `No preference`.

Then a **review screen** listing everything, with an **Edit** link next to each section that jumps back to that step with the answers still filled in, and a **Confirm booking** button.

Requirements:

1. `Next` is disabled until the current step is valid. Steps 2 and 3 have their own rules; step 3 is always valid.
2. Errors appear **only after a field has been left**, using `onBlur`. A blank form must not be covered in red before you've touched it.
3. Going `Back` and `Next` again keeps everything you typed. Nothing is ever lost.
4. Confirming logs the whole booking as one object and shows a confirmation screen with a **Make another booking** button that resets everything, including the step.
5. At most **three** pieces of state for the whole form's data. The step number and the touched-fields tracking are separate and don't count.
6. Every field has a real label, and pressing Enter never reloads the page on any step.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

10 Project: To-Do App, the React Way

What you'll build

The same to-do app you built in JavaScript chapter 24 — but in React, and in TypeScript. When you're done you'll be able to:

- add tasks,
- tick them off,
- delete them,
- see how many are left,
- filter to All, Active, or Completed,
- clear away all the finished ones at once,
- close the browser, come back tomorrow, and find your list as you left it.

```
My To-Do List

[ e.g. Buy milk ] [ Add ]

( All ) ( Active ) ( Completed )

[x] Buy milk      <- crossed out    x
[ ] Book the dentist      x
[ ] Call grandma      x

2 tasks left      Clear completed
```

The features are identical on purpose. That's what makes the comparison at the end of this chapter worth something: same app, same you, two very different ways of getting there.

What you'll practise:

Skill	Chapter
Components and props	03 and 04
Conditional rendering	05
Lists and keys	06
Events and callback props	07
State with <code>useState</code>	08

Skill	Chapter
Controlled forms	09
Types, unions, and typed props	The whole TypeScript course

Eight milestones. After each one the app works a bit better, and you can check it in the browser before moving on.

Getting started

This project gets its own app, separate from your `playground`.

1. Open a terminal **in this React folder** and run `npm create vite@latest`, answering exactly as you did in [chapter 01](#), but calling the project `todo-app`.
2. Copy the two files from this chapter's `starter/src/` folder over the ones in `todo-app/src/`:
 - `App.tsx` — a stub with the milestone list in a comment.
 - `index.css` — all the styling, already written.
3. Delete `todo-app/src/App.css` and `todo-app/src/assets/`. Nothing imports them now.
4. `cd todo-app`, then `npm run dev`, and open the address it prints.

You'll see the heading and nothing else. That's your job.

Keep a second terminal open in `todo-app` running `npm run tsc -b` whenever you want to check the whole project, and keep DevTools (`F12`) open on the **Components** tab. You'll use it constantly.

The CSS is done for you so you can concentrate on React. It styles a fixed set of class names, listed in `starter/README.md` and mentioned as you go. Use those names and everything will look right.

The big idea: state is the truth

In the JavaScript version, the key idea was the stadium scoreboard: the `tasks` array was the truth, and a `render()` function redrew the page from it. Every feature followed three steps:

1. Something happens.
2. You change the array.
3. **You call `render()`**.

React keeps steps 1 and 2 and does step 3 for you. That's the whole difference:

JavaScript version	React version
<code>let tasks = [...]</code>	<code>const [tasks, setTasks] = useState<Task[]>([])</code>
Change the array, then call <code>render()</code>	Call <code>setTasks(...)</code> — React redraws
<code>render()</code> empties the <code></code> and rebuilds it	<code>tasks.map(...)</code> in your JSX
Forget <code>render()</code> and the page goes stale	Can't forget
<code>textContent</code> to stay safe from XSS	JSX is safe by default

There's one price for this, and it's the rule you'll break at least once today: **you must never change the array in place**. `push`, `splice` and `sort` all edit the array you already have, and React decides nothing has changed. Every change makes a **new** array.

Milestone 1: Types, and a list on the page

Goal: show three hard-coded tasks, built from components.

Make `src/types.ts`:

```
export type Task = {
  id: string;
  text: string;
  done: boolean;
};
```

`id` is a string because you'll generate ids with `crypto.randomUUID()` later.

Now three components, each in its own file in `src/`:

`TaskItem.tsx` takes one `task` and renders:

```
<li className="todo-item">
  <input className="todo-checkbox" type="checkbox" checked={task.done} readOnly />
  <span className="todo-text">{task.text}</span>
  <button className="delete-button" type="button">x</button>
</li>
```

Add `className="todo-item completed"` when the task is done — the CSS crosses it out. (`readOnly` is temporary: it stops React's "value without onChange" warning until milestone 5. Remember to take it out then.)

`TaskList.tsx` takes a `tasks` prop, typed `readonly Task[]`, and maps over it inside a `<ul className="todo-list">`.

`App.tsx` holds three sample tasks in a plain `const` and renders `<TaskList tasks={tasks} />`.

Check it: three tasks on screen, one crossed out, and an **empty Console**. If you see a message about keys, fix it now ([chapter 06](#)).

Why split it three ways? `TaskItem` knows how to draw one task. `TaskList` knows how to draw many. `App` will know what the tasks *are*. That separation is what makes the next seven milestones small.

Milestone 2: The counter and the empty state

Goal: nothing hard-coded that could be worked out.

Under the list, add a footer:

```
<footer className="todo-footer">
  <span className="todo-count">2 tasks left</span>
</footer>
```

but with the number calculated:

```
const remaining = tasks.filter((task) => !task.done).length;
```

Get the wording right: `1 task left`, not `1 tasks left`.

In `TaskList`, handle the empty case:

```
if (tasks.length === 0) {
  return <p className="empty-message">Nothing to do. Enjoy your day!</p>;
}
```

Check it: delete the sample tasks from `App.tsx` and save. You should see the message, not an empty box. Put them back.

`remaining` is a **derived value**. Never store it in state — it would only go stale.

Milestone 3: Make it live

Goal: move the tasks into state, and prove to yourself why that matters.

First, the experiment. In `App.tsx`, add a temporary button:

```
<button onClick={() => tasks.push({ id: "x", text: "Test", done: false })}>
  Add test task
</button>
```

Click it a few times. **Nothing happens.** Now open the Components tab and check — nothing there either. The array really is growing; the page just has no idea. This is the bug that React exists to prevent, and it's worth seeing once with your own eyes.

Now do it properly:

```
import { useState } from "react";
import type { Task } from "./types.ts";

function App() {
  const [tasks, setTasks] = useState<Task[]>([
    { id: "1", text: "Buy milk", done: true },
    { id: "2", text: "Book the dentist", done: false },
    { id: "3", text: "Call grandma", done: false },
  ]);
  // ...
}
```

The `<Task[]>` matters. Without it, an empty starting array would be inferred as `never[]` ([chapter 08](#)).

Change the test button to:

```
<button onClick={() => setTasks([...tasks, { id: crypto.randomUUID(), text: "Test", done: false }])}>
  Add test task
</button>
```

Check it: clicking now adds a task, and the counter moves by itself. Watch `tasks` grow in the Components tab. Then delete the test button — milestone 4 replaces it.

Milestone 4: Add a task

Goal: a real form.

Make `src/AddTaskForm.tsx`. It owns the text box's state (nobody else needs it), and tells its parent when a task should be added:


```
type AddTaskFormProps = {
  onAdd: (text: string) => void;
};
```

Inside:

```
const [text, setText] = useState("");

function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();

  const trimmed = text.trim();
  if (trimmed === "") {
    return;
  }

  onAdd(trimmed);
  setText("");
}
```

The JSX:

```
<form className="todo-form" onSubmit={handleSubmit}>
  <label className="visually-hidden" htmlFor="new-task">New task</label>
  <input
    id="new-task"
    className="todo-input"
    type="text"
    placeholder="e.g. Buy milk"
    value={text}
    onChange={(event) => setText(event.target.value)}
  />
  <button className="add-button" type="submit">Add</button>
</form>
```

(`visually-hidden` is a class in the CSS that hides the label on screen but keeps it for screen readers. A placeholder is not a label — [chapter 38](#).)

In `App.tsx`:

```
function handleAddTask(text: string) {
  const newTask: Task = { id: crypto.randomUUID(), text, done: false };
  setTasks([...tasks, newTask]);
}
```

Check it: type and press **Enter**, and type and click **Add** — both must work. Empty and spaces-only submissions do nothing. The box clears after each add. The counter keeps up.

Why does the text live in `AddTaskForm` but the tasks live in `App`? State goes in the closest component that contains everything needing it. Only the form cares what's half-typed. Everything cares about the tasks.

Why generate the id in `App` and not in the form? The form's job is "someone wants to add this text". Ids are part of what a task *is*, so they belong where tasks are made.

Milestone 5: Tick a task off

Goal: the tick box works, and survives everything.

`TaskItem` gets a second prop:

```
type TaskItemProps = {
  task: Task;
  onToggle: (id: string) => void;
};
```

Remove `readOnly` and wire the box up:

```
<input
  className="todo-checkbox"
  type="checkbox"
  checked={task.done}
  onChange={() => onToggle(task.id)}
/>
```

`TaskList` takes `onToggle` too and passes it straight through to every row.

In `App.tsx`, this is the important one:

```
function handleToggleTask(id: string) {
  setTasks(
    tasks.map((task) =>
      task.id === id ? { ...task, done: !task.done } : task
    )
  );
}
```

Read it slowly, because this line is the heart of React state:

- `map` builds a **new array**.
- For the task that matches, it makes a **new object** with `done` flipped.
- Every other task is passed through **unchanged** — the same object, not a copy.

Compare it with what you'd write in plain JavaScript:

```
const task = tasks.find((t) => t.id === id);
task.done = !task.done;    // ❌ in React: nothing happens on screen
```

That edits the object React already has, so React sees no change and doesn't redraw.

Check it: ticking crosses the task out and the counter drops. Tick a task, then add a new one — the tick stays with the right task. (That's your keys doing their job, [chapter 06](#).)

Milestone 6: Delete a task

Goal: the `×` button works.

Same shape as milestone 5. `TaskItem` takes `onDelete: (id: string) => void`, and the button calls it:

```
<button className="delete-button" type="button" aria-label={`Delete ${task.text}`} onClick={() => onDelete(task.id)}>
  ×
</button>
```

(`aria-label` gives the button a real name. On its own, `×` tells a screen-reader user nothing.)

In `App.tsx`:

```
function handleDeleteTask(id: string) {
  setTasks(tasks.filter((task) => task.id !== id));
}
```

`filter` returns a new array without that task. One line.

Check it: delete the middle task, then the last, then all of them — you should land on the empty message. Delete a task while another is ticked, and make sure the tick doesn't jump rows.

Notice that `TaskList` is now just passing `onToggle` and `onDelete` through without using them. That's called **prop drilling**, and with one layer it's completely fine. [Chapter 22](#) covers what to do when it's five layers.

Milestone 7: Filters and Clear completed

Goal: All / Active / Completed, and a button to tidy up.

A second piece of state in `App.tsx`, with a type that makes a typo impossible:

```
type Filter = "all" | "active" | "completed";

const [filter, setFilter] = useState<Filter>("all");
```

Make `src/FilterButtons.tsx`:

```
type FilterButtonsProps = {
  filter: Filter;
  onChange: (filter: Filter) => void;
};
```

Map over `["all", "active", "completed"]` to make three buttons. The selected one gets `className="filter-button active"`, the others `"filter-button"`.

Back in `App.tsx`, the filtered list is **derived**, not state:

```
const visibleTasks = tasks.filter((task) => {
  if (filter === "active") return !task.done;
  if (filter === "completed") return task.done;
  return true;
});
```

Pass `visibleTasks` to `TaskList`. Keep using the **full** `tasks` list for the counter — "2 tasks left" shouldn't change just because you're looking at a filter.

Then **Clear completed** in the footer:

```
function handleClearCompleted() {
  setTasks(tasks.filter((task) => !task.done));
}
```

Show it only when there's something to clear.

Check it: each filter shows the right tasks. Tick a task while on **Active** — it should vanish from view but the counter should still be right. Switch to **Completed** and it's there. Clear completed on the Completed tab leaves you with an empty list and the right message.

Careful with the empty state now. `TaskList` says "Nothing to do. Enjoy your day!" whenever its list is empty — including when you're on **Completed** with nothing finished, which reads oddly. Fixing that is stretch goal 3.

Milestone 8: Remember the list

Goal: close the tab, come back, and it's all still there.

React doesn't have a special way to do this — it's the plain `localStorage` and JSON you know from JavaScript chapter 23. What's new is *where* it goes.

Add to `App.tsx`, above the component:

```
const STORAGE_KEY = "react-todo-tasks";

function loadTasks(): Task[] {
  const saved = localStorage.getItem(STORAGE_KEY);
  if (saved === null) {
    return [];
  }

  try {
    const parsed: unknown = JSON.parse(saved);
    if (!Array.isArray(parsed)) {
      return [];
    }
    return parsed as Task[];
  } catch {
    return [];
  }
}
```

The `try / catch` matters: `JSON.parse` throws on damaged data, and one bad character in `localStorage` would otherwise leave your app permanently broken with a white screen.

Loading is the state's starting value:

```
const [tasks, setTasks] = useState<Task[]>(loadTasks);
```

Note: `loadTasks`, **not** `loadTasks()`. Passing the function is the **lazy initial state** form from [chapter 08](#), so React only calls it on the first render instead of every one.

For saving, add one small function and route every change through it:

```
function updateTasks(next: Task[]) {
  setTasks(next);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(next));
}
```

Now replace `setTasks(...)` with `updateTasks(...)` in all four handlers. There's nowhere left to forget.

Check it: add tasks, tick some, refresh. Everything's there. Open DevTools → **Application** → **Local Storage** and watch the JSON change as you click. Then break it deliberately: edit the stored value to `not json at all`, refresh, and confirm the app starts empty instead of crashing.

Two honest notes. First, as `Task[]` is a promise TypeScript can't check (TypeScript chapter 02). If old data has a different shape, TypeScript believes you and your app breaks at runtime. [Zod](#) in Level 4 is how you check for real. Second, `updateTasks` works, but React has a purpose-built tool for "keep this in sync with something outside React", and that's `useEffect` in [chapter 17](#). Doing it the long way first means you'll know exactly what `useEffect` is buying you.

Now compare

You've now built the same app twice. Open `JavaScript/24-project-todo-app` next to your new `App.tsx` and look at them side by side. It's worth ten minutes.

What disappeared:

In the JavaScript version	In React
<code>render()</code> : 25 lines emptying the <code></code> and rebuilding it	<code>tasks.map(...)</code>
<code>document.querySelector</code> for six elements	Nothing
<code>createElement</code> , <code>append</code> , <code>classList.add</code>	JSX
Event delegation on the <code></code>	An <code>onClick</code> on each button
<code>dataset.id</code> and <code>Number(...)</code> to find what was clicked	<code>task.id</code> , already the right type
Remembering to call <code>render()</code> after every change	Automatic
<code>textContent</code> to stay safe from XSS	Safe by default

What's new or harder:

- **You can't change anything in place.** `push` and `task.done = true` are gone. Every change builds a new array or object. This is the tax React charges, and it takes a while to stop feeling like a chore.

- **There's more ceremony.** A form that adds a task now involves a prop type, a callback, and a controlled input. The JavaScript version was one `addEventListener`.
- **You have to decide where state lives.** That question didn't exist before, because everything was in one file.

What got better, and why it's worth it:

- **The page can't disagree with the data.** In the JavaScript version, a forgotten `render()` was a bug you only found later. Here it's impossible.
- **Adding a feature is local.** Stretch goal 1 is dark mode: one piece of state and one class name. In the JavaScript version it meant touching `render()` too.
- **The pieces have names.** `TaskItem` is a thing you can point at, test, and reuse. `render()` was one function that did everything.
- **TypeScript can see the joins.** `onToggle: (id: string) => void` is checked at both ends. `dataset.id` was a string you had to remember to convert, with nothing to catch you.

That last group is why the extra ceremony pays off. It's barely worth it for 150 lines. At 15,000 lines, with four people editing, it's the difference between an app you can change and one you're afraid of.

Common mistakes

1. Changing the array in place

```
tasks.push(newTask);           // ✗ nothing happens
const task = tasks.find((t) => t.id === id);
task.done = !task.done;       // ✗ nothing happens
```

The screen doesn't move and there's no error, which makes it the worst one. New array, every time.

2. A missing or index-based `key`

```
{tasks.map((task, index) => <TaskItem key={index} task={task} ... />)}
```

The list is reordered and filtered constantly here, so this will put ticks on the wrong rows. Use `task.id`.

3. Forgetting `event.preventDefault()`

Add a task and the whole page reloads, wiping your state. Once you reach milestone 8 it's worse: the reload *looks* like it worked, because the data comes back from storage.

4. Filtering the counter

```
const remaining = visibleTasks.filter((task) => !task.done).length; // ✖
```

"2 tasks left" would change every time you switched tab. Count `tasks`, not `visibleTasks`.

5. Storing what you can derive

```
const [visibleTasks, setVisibleTasks] = useState<Task[]>([]); // ✖  
const [remaining, setRemaining] = useState(0); // ✖
```

Both go stale. Calculate them during the render.

6. `value` on a checkbox

`checked={task.done}`, and `onChange`, not `onClick`. Otherwise you get the read-only warning from [chapter 09](#).

7. Generating the id during render

```
<TaskItem key={crypto.randomUUID()} ... /> // ✖
```

A brand-new key every render means React rebuilds every row every time. The id is created once, when the task is born.

8. Calling `loadTasks()` instead of passing `loadTasks`

```
useState<Task[]>(loadTasks()) // reads localStorage on every single render  
useState<Task[]>(loadTasks) // ✔ once
```

Both *work*, which is why it's easy to miss.

Quick recap

- State is the truth. React redraws from it, so the page and the data can't disagree.
- Every change makes a **new** array or object: `[...tasks, task]` to add, `filter` to remove, `map` with a spread to change one.
- State lives in the closest component that contains everything needing it. Here that's `App` for the tasks, and `AddTaskForm` for the half-typed text.
- Children report events upwards through **callback props** (`onAdd`, `onToggle`, `onDelete`), typed at both ends.
- Anything you can calculate — the counter, the filtered list — is derived during the render, never stored.
- The React version has more ceremony and far fewer ways to go wrong. That trade gets better the bigger the app gets.

Next: try the [stretch goals](#), then move on to [11 Updating Objects and Arrays in State](#), which turns the copying rules you've just been following into a proper toolkit.

Exercises

These are **stretch goals**: extra features for your finished app. Do them in any order, and do as many as you like.

How to do these:

- Finish all 8 milestones first. Then commit your work, or copy the `todo-app` folder, so you've always got a working version to go back to.
- `index.css` already has styles for these goals. Look for "Styles for the stretch goals" near the bottom.
- Keep the Console and the **Components** tab open the whole time.
- After every goal, test the whole app again, not just the new bit. Ticking, filtering, and refreshing must all still work.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Dark mode

Planning tomorrow's tasks late at night is hard on the eyes. Add a button that switches the app between light and dark, and remember the choice.

1. A button in the top right saying  Dark or  Light — always what clicking it will *do*.
2. In dark mode, the wrapping `<div>` gets `className="app dark"`. The CSS does the rest.
3. The choice survives a refresh, saved under its own key, `"react-todo-theme"`.

What you should see: click  Dark, and the app goes dark. Refresh — still dark. Your tasks are untouched.

Then answer in a comment: in the JavaScript version this needed a `classList.toggle` and a line in `render()`. What did it need here?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Tick everything at once

Add a **Toggle all** control to the left of the input box.

1. If **any** task is unfinished, clicking it marks them all done.
2. If **all** tasks are already done, clicking it marks them all undone.
3. Its label says what it will do: `Mark all done` or `Mark all undone`.
4. It doesn't appear at all when the list is empty.
5. It respects the current filter in one specific way: it always acts on **every** task, not just the visible ones. (Think about why that's the less surprising choice, and write a sentence on it.)

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Honest empty states

Right now `TaskList` says "Nothing to do. Enjoy your day!" whenever its list is empty — even on the **Completed** tab when you simply haven't finished anything. That's not just untidy; it's wrong.

Make the message fit the situation:

Situation	Message
No tasks at all	Nothing to do. Enjoy your day!
Tasks exist, Active tab, none active	All done. Nice work!
Tasks exist, Completed tab, none completed	Nothing finished yet.

Rules:

1. `TaskList` must not import the filter or reach for the full task list itself. It shows what it's given.
2. So the message has to arrive as a **prop**. Decide its type, and work the message out in `App`.
3. When you're done, ask yourself whether `TaskList` is better or worse for this change. There's a real argument both ways, and either answer is fine as long as you can say why.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Edit a task

Let people fix a typo without deleting and retyping.

1. Double-clicking a task's text turns it into a text box holding the current text, with the cursor in it.
2. **Enter** or clicking away saves it. **Escape** cancels and puts the original text back.
3. Saving an empty task deletes it instead.
4. Only one task can be in edit mode at a time.

5. Ticking and deleting still work normally; editing must not fire when you click the tick box or the `×`.

This is the hardest of these goals, and it's the one that will teach you the most about where state belongs.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Undo

Everyone deletes the wrong task eventually. Give them a way back.

1. After any destructive action — deleting a task, or Clear completed — a bar appears at the bottom: `Task deleted.` with an **Undo** button.
2. Undo puts things back exactly as they were, including the task's position in the list and whether it was ticked.
3. The bar disappears when you undo, and when you do anything else.
4. Undo works for **Clear completed** too, bringing every cleared task back in its original place.
5. Only the most recent action can be undone.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

When you're done

You've finished Level 1. Before moving on, it's worth a few minutes writing down:

- One thing that was much easier in React than in plain JavaScript.
- One thing that was harder or more fiddly.
- One thing you still don't feel sure about.

That last one is the useful one. Bring it to Claude and ask.

Level 2: Thinking in React

Goal: organize your data well, talk to the outside world, and reuse your logic.

11 Updating Objects and Arrays in State

What is it?

This chapter is a toolkit for one rule you've been following since [chapter 08](#): **never change an object or array that's in state. Always make a new one.**

```
// ❌  
task.done = true;  
tasks.push(newTask);  
  
// ✅  
setTask({ ...task, done: true });  
setTasks([...tasks, newTask]);
```

You've seen the shallow version of this a dozen times. This chapter covers what happens when the object has an object inside it, or the array holds objects you need to change one of.

Why does it matter?

You've built a to-do app with flat data: a task has a string and a boolean. Real data nests. A user has an address, which has a city. An order has a list of items, each with its own quantity. The moment your state has a shape inside a shape, the simple `{ ...obj, field: newValue }` you know isn't automatically enough, and it's easy to write something that *looks* right and quietly doesn't work.

Here's the failure mode, concretely:

```

type Task = {
  id: string;
  text: string;
  assignee: { name: string; avatar: string };
};

function handleReassign(id: string, name: string) {
  const task = tasks.find((t) => t.id === id);
  if (task) {
    task.assignee.name = name;      // ✗ still mutation, just one level deeper
    setTasks([...tasks]);           // a new array... of the same objects
  }
}

```

This one is sneaky, because it *looks* like you did the right thing — there's a spread, right there. But `[...tasks]` copies the array; it doesn't copy the objects **inside** it. Every task in the new array is the exact same object as before, `assignee` included, and you just changed one of those objects directly. React might not even notice something changed, and even if it does, you've corrupted the state you were trying to protect. This chapter is about spotting that trap and knowing exactly how deep a copy needs to go.

Real-world example

Think about **photocopying a filing cabinet**.

Filing cabinet	Nested state
The cabinet	The array of objects
A folder inside it	One object in the array
A page inside the folder	A field inside that object
Photocopying just the cabinet's index card	Spreading the array, <code>[...tasks]</code>
The folders inside are still the original folders	The objects inside are still the same references
Writing on a page inside an "original" folder	Mutating a nested field
To really get a fresh copy: photocopy the cabinet, and the folder, and the page	Spread at every level you're changing

You only need to photocopy as deep as the level you're writing on. Folders you're not touching can stay as they are.

How it works

The rule, precisely

Copy every level between the top of your state and the value you're changing. Levels you're not touching can be reused as-is.

That's it. It sounds abstract, so the rest of this chapter is worked examples.

One nested object

```
type Profile = {
  name: string;
  address: {
    city: string;
    postcode: string;
  };
};

const [profile, setProfile] = useState<Profile>({
  name: "Maya",
  address: { city: "Leeds", postcode: "LS1 1AA" },
});
```

To change just the city, you copy two levels: the profile, and the address inside it.

```
setProfile({
  ...profile,
  address: { ...profile.address, city: "York" },
});
```

Read it inside-out: `{ ...profile.address, city: "York" }` makes a **new address object**, same postcode, new city. Then `{ ...profile, address: ... }` makes a **new profile object**, same name, with that new address slotted in.

What goes wrong if you skip the inner spread:

```
setProfile({ ...profile, address: { city: "York" } });
// ❌ Property 'postcode' is missing in type '{ city: string; }' but required in type '{ city: string; postcode: string; }'.
```

TypeScript catches this one for you, because the address type has two required fields and you only gave it one. That's not a coincidence — it's one of the best reasons to give nested state a real type instead of leaving it as an

inferred object shape.

A helper function for one field

Typing that out for every field gets old. A small helper keeps the pattern in one place:

```
function updateAddress(field: keyof Profile["address"], value: string) {
  setProfile({
    ...profile,
    address: { ...profile.address, [field]: value },
  });
}

updateAddress("city", "York");
```

`Profile["address"]` reaches inside the `Profile` type to grab the address's type (TypeScript chapter 10), and `keyof` turns that into the union `"city" | "postcode"`. Pass anything else and TypeScript stops you.

Arrays of objects: change one item

This is the one you'll write constantly, and you already met it in the to-do app: `map`, replacing the matching item with a new object and leaving the rest alone.

```
type Task = {
  id: string;
  text: string;
  done: boolean;
};

function handleToggle(id: string) {
  setTasks(
    tasks.map((task) =>
      task.id === id ? { ...task, done: !task.done } : task
    )
  );
}
```

Two different things happen in that `map`, and it's worth naming them:

- The matching task gets a **new object**: `{ ...task, done: !task.done }`.
- Every other task is returned **unchanged** — literally the same object reference as before, not a copy.

That second point isn't laziness, it's correct. Copying objects you didn't change achieves nothing and [chapter 27](#) explains a case where it actively costs you: some optimisations rely on unrelated items staying the *same*

reference so React can skip re-rendering them.

Arrays of objects, one level deeper

Now put a nested object inside the array item, and both rules apply at once:

```
type Task = {
  id: string;
  text: string;
  assignee: { name: string; avatar: string };
};

function handleReassign(id: string, name: string) {
  setTasks(
    tasks.map((task) =>
      task.id === id
        ? { ...task, assignee: { ...task.assignee, name } }
        : task
    )
  );
}
```

This is the fixed version of the bug at the start of the chapter. Copy the array (`map` does that for you), copy the item you're changing, copy the nested object inside *that* item. Untouched tasks, and the untouched `avatar` inside the touched task, are reused as-is.

Adding, removing, and reordering

These don't need deep copies — they only touch the array itself — but they're worth having side by side, because you'll reach for all of them constantly:


```

// add to the end
setTasks([...tasks, newTask]);

// add to the start
setTasks([newTask, ...tasks]);

// insert at a position
setTasks([...tasks.slice(0, index), newTask, ...tasks.slice(index)]);

// remove
setTasks(tasks.filter((task) => task.id !== id));

// replace one
setTasks(tasks.map((task) => (task.id === id ? updated : task)));

// reorder (move index i to the end, say)
setTasks([...tasks.slice(0, i), ...tasks.slice(i + 1), tasks[i]]);

// sort, without touching the original
setTasks([...tasks].sort((a, b) => a.text.localeCompare(b.text)));
setTasks(tasks.toSorted((a, b) => a.text.localeCompare(b.text))); // same idea, newer method

```

`slice` (copies part of an array) is not `splice` (changes the array in place and returns what it removed). They're one letter apart and do opposite things. If you only remember one thing from this section, remember that `splice` doesn't belong anywhere near state.

Updating an array of numbers or strings

Primitives are simpler, because there's no inner object to worry about — but the array itself still needs to be new:

```

const [tags, setTags] = useState<string[]>(["urgent"]);

setTags([...tags, "today"]); // add
setTags(tags.filter((tag) => tag !== "urgent")); // remove
setTags(tags.map((tag) => (tag === "urgent" ? "high-priority" : tag))); // rename

```

Why React needs a new reference

It's worth knowing what React is actually checking, because it explains every rule above.

When state changes, React compares the **new value to the old one** to decide whether to re-render. For objects and arrays, that comparison is `Object.is`, which — like `===` — checks whether two variables point at the **same object in memory**, not whether their contents look the same (JavaScript chapter 16).

```
const a = { name: "Maya" };
const b = { name: "Maya" };
console.log(a === b); // false – same content, different objects

const c = a;
c.name = "Tom";
console.log(a === c); // true – same object, even after the "change"
```

That's the whole story. `task.done = true` doesn't change *what* `task` refers to, so `oldTask === newTask` is still `true`, and React sees nothing to redraw. `{ ...task, done: true }` makes a genuinely different object, so the comparison correctly says "this changed."

This also explains why an **unrelated** field changing doesn't need a new reference. If you change `address.city`, the `name` field's *value* didn't change — but you still had to make a new `profile` object, because `profile` itself is a different object now, and `profile !== oldProfile` is exactly what tells React to re-render.

`structuredClone`, and why it's not the answer here

JavaScript has a built-in function for a **deep copy** — a copy all the way down, however nested:

```
const original = { address: { city: "Leeds" } };
const copy = structuredClone(original);
copy.address.city = "York";
console.log(original.address.city); // "Leeds" – untouched
```

It's tempting to reach for this everywhere and sidestep the spreading rules entirely. Don't. Two reasons:

1. **It's wasteful.** It copies parts of your state you weren't even changing, every single time.
2. **It throws away the reference sharing that makes React fast.** Remember the untouched tasks from earlier, reused as the same object on purpose? `structuredClone` would silently replace every one of them with a new, equal-but-different copy, which defeats optimisations `React.memo` relies on ([chapter 27](#)).

Spread only as deep as you're actually changing. It's more typing up front, and it's the version that scales.

When nesting gets too deep

If you find yourself spreading three or four levels to change one field, that's a signal, not just an inconvenience:

```
setState({
  ...state,
  a: { ...state.a, b: { ...state.a.b, c: { ...state.a.b.c, value: 5 } } },
});
```

Nobody enjoys reading that, let alone writing it. Two honest fixes:

- **Flatten your state.** Often nesting was arbitrary, not necessary. A `taskAssigneeName` field next to `taskText`, instead of `task.assignee.name`, might genuinely be simpler for what your app does.
- **Reach for a library.** [Immer](#) is a popular one: you write code that *looks* like mutation, and it produces the correctly-copied result behind the scenes. You'll meet it inside Redux Toolkit in [chapter 35](#). Until then, it's worth knowing the name for when your state gets genuinely gnarly. Most apps never need it on its own.

Try flattening first. It's usually the better fix, and it costs nothing to learn.

Objects with array fields

The two ideas combine however they're nested. Here's an object with an array inside it:

```
type Board = {
  title: string;
  tasks: Task[];
};

function handleAddTask(board: Board, newTask: Task): Board {
  return { ...board, tasks: [...board.tasks, newTask] };
}

setBoard(handleAddTask(board, newTask));
```

Same rule, applied twice: copy the object because you're changing its `tasks` field, and build a new `tasks` array because you're adding to it.

`readonly` keeps you honest

[Chapter 04](#) introduced `readonly` on props. It's just as useful on state you pass around, because it turns a whole class of these mistakes into a compile error instead of a silent bug:

```
type Task = {
  readonly id: string;
  readonly text: string;
  readonly assignee: {
    readonly name: string;
    readonly avatar: string;
  };
};

task.assignee.name = "Tom";
// ✖ Cannot assign to 'name' because it is a read-only property.
```

It won't write the copying code for you, but it will stop you from taking the shortcut that seems to work until it doesn't. Adding `readonly` to state types you know will be nested is a habit worth building now.

Common mistakes

1. Spreading the array but not the object inside it

```
setTasks([...tasks.map((t) => (t.id === id ? t : t))]); // nothing actually changed
task.done = true;
setTasks([...tasks]); // ❌ still the same task objects
```

A new array of the same objects is still the same objects. Change the matching item with `map` and a spread, not by editing it first.

2. Only copying one level of a deeply nested update

```
setProfile({ ...profile, address: profile.address }); // address object unchanged
```

If you're about to change something *inside* `address`, `address` needs its own spread too.

3. Using `splice` because it "sounds like" `slice`

```
tasks.splice(index, 1); // ❌ mutates in place, and returns the removed items, not the array
```

Use `filter` to remove, or `slice` (no `p`) to copy a range.

4. Sorting or reversing state directly

```
tasks.sort((a, b) => a.text.localeCompare(b.text)); // ❌ mutates
setTasks(tasks); // same reference – may not even re-render
```

Copy first: `[...tasks].sort(...)` or `tasks.toSorted(...)`.

5. Reaching for `structuredClone` out of habit

It works, but it copies more than you need and breaks reference sharing for everything you didn't touch. Spread exactly as deep as the change.

6. Forgetting that objects from `map`'s "no change" branch must be the *same* object

```
tasks.map((task) => (task.id === id ? { ...task, done: true } : { ...task })))
```

That final `{ ...task }` looks harmless, but it creates a **new** object for every task you didn't even mean to change. Just return `task` as-is in the untouched branch.

7. Trusting a shallow spread to protect nested data

```
const copy = { ...profile };
copy.address.city = "York";    // ❌ this still changes profile.address.city too!
```

`{ ...profile }` copies the top-level fields, but `address` is still the *same* nested object in both `profile` and `copy`. Changing it through one changes it through the other. This is the single most common surprise in this whole chapter — say it out loud once: a spread only copies one level deep.

Quick recap

- The rule is precise, not vague: **copy every level between the top of your state and the field you're changing**. Untouched levels can be reused as-is.
- A shallow spread, `{ ...obj }`, only copies the top level. Anything nested inside is still the *same* object in the copy — change it and you've mutated the original too.
- For arrays of objects, `map` combined with a spread on the matching item is the standard shape:
`tasks.map((t) => (t.id === id ? { ...t, done: true } : t)).`
- Add, remove, and reorder with `[...arr, x]`, `filter`, and `slice` — never `push`, `splice`, or `sort` directly on state.
- React compares state by **reference**, with `Object.is`. That's why a genuinely new object or array is required, and why reusing unchanged references is not just allowed but good.
- `structuredClone` deep-copies everything, which is usually more than you want. Prefer spreading only as deep as needed.
- Deeply nested state that needs three or four levels of spreading is a sign to flatten your data, or to learn a tool like Immer.
- `readonly` on nested state types turns silent mutation bugs into compile errors.

Next: try the [exercises](#), then move on to [12 How Rendering Works](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch11/ex1/`, and so on.
- Watch the **Components** tab as you click. For several of these, the bug you're asked to find won't show up on the page at all — only in whether the state actually changed underneath it.

- An exercise is done when the page behaves correctly, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console is empty.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Fix the settings form

Copy this into `src/ch11/ex1/Ex1.tsx`:

```
import { useState } from "react";

type Settings = {
  displayName: string;
  notifications: {
    email: boolean;
    sms: boolean;
  };
};

function Ex1() {
  const [settings, setSettings] = useState<Settings>({
    displayName: "Maya",
    notifications: { email: true, sms: false },
  });

  function toggleEmail() {
    settings.notifications.email = !settings.notifications.email;
    setSettings(settings);
  }

  return (
    <div>
      <p>Email notifications: {settings.notifications.email ? "On" : "Off"}</p>
      <button onClick={toggleEmail}>Toggle email</button>
    </div>
  );
}

export default Ex1;
```

1. Click the button. What do you expect to happen, and what actually happens?
2. Open the Components tab and watch `settings` while you click. What do you see there, compared to the page?
3. Fix `toggleEmail` so it works, copying only as deep as you need to.

4. Add a second toggle for `sms`, using the same pattern.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Shopping cart quantities

In `src/ch11/ex2/Ex2.tsx`, start from:

```
type CartItem = {
  id: string;
  name: string;
  price: number;
  quantity: number;
};

const initialCart: CartItem[] = [
  { id: "1", name: "Notebook", price: 3.5, quantity: 2 },
  { id: "2", name: "Backpack", price: 34.99, quantity: 1 },
];
```

Build a cart with three buttons per row: `+`, `-`, and `Remove`. Rules:

1. `+` and `-` change only that row's `quantity`. Quantity can't go below 1 — use `-` at 1 to test it stays at 1 (don't let it hit 0).
2. `Remove` takes the row out of the array entirely.
3. A total at the bottom, worked out from `price * quantity` across every item, never stored in state.
4. None of your code may use `push`, `splice`, or change an item's fields directly.

Then, in a comment: which lines of your `+` handler are "copy the array" and which are "copy the item"? Point at them.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Contact book with addresses

In `src/ch11/ex3/`, start with:

```

export type Contact = {
  id: string;
  name: string;
  favourite: boolean;
  address: {
    city: string;
    country: string;
  };
};

export const contacts: Contact[] = [
  { id: "1", name: "Maya", favourite: false, address: { city: "Leeds", country: "UK" } },
  { id: "2", name: "Priya", favourite: true, address: { city: "Pune", country: "India" } },
  { id: "3", name: "Tom", favourite: false, address: { city: "Bristol", country: "UK" } },
];

```

Build a page where each contact shows their name, city and country, and:

1. A ★ button toggles `favourite`. Favourited contacts sort to the top — **derive** the sorted order, don't store it.
2. Each contact has an **Edit city** button that turns the city into a text box (one contact editable at a time — reuse the "which one is being edited" idea from the [chapter 10 stretch goals](#) if you did them). Saving updates only that contact's `address.city`, leaving `country` and everything else about every other contact untouched.
3. A **Move all UK contacts to "United Kingdom"** button that rewrites the `country` field on every UK contact in one click, and leaves non-UK contacts completely alone — not even a new object for them.

For requirement 3, prove it in a comment: log each contact's `address` object before and after clicking, using `===`, and confirm which ones are the *same reference* afterwards and which aren't.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Kanban board

A classic nested shape: a board holds columns, and each column holds cards. In `src/ch11/ex4/`, start with:


```

export type Card = {
  id: string;
  text: string;
};

export type Column = {
  id: string;
  title: string;
  cards: Card[];
};

export const initialColumns: Column[] = [
  { id: "todo", title: "To Do", cards: [{ id: "c1", text: "Design the logo" }] },
  { id: "doing", title: "Doing", cards: [{ id: "c2", text: "Build the homepage" }] },
  { id: "done", title: "Done", cards: [] },
];

```

Keep **one** piece of state: `columns`. Build:

1. Each column shows its title, its cards, and a small form to add a new card **to that column** (give each card a `crypto.randomUUID()`).
2. Every card has a `×` that removes it from its column.
3. Every card has a `→` button that moves it to the **next** column (To Do → Doing → Done). On the Done column, there's no `→` at all.

Requirement 3 is the point of the exercise: one card has to leave one column's array and join another's, in a single state update, without ever mutating either array.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Undo, properly

This exercise makes you feel the payoff of never mutating state. In `src/ch11/ex5/`, take your Kanban board from Exercise 4 (or rebuild a simpler version if you skipped it) and add:

1. Every change — add a card, remove a card, move a card — pushes the **previous** `columns` value onto a history stack before applying the change.
2. An **Undo** button pops the most recent entry off that stack and makes it the current state.
3. **Redo** works too: undoing pushes the state you just left onto a separate redo stack, and any *new* change clears the redo stack.
4. Undo/Redo buttons are disabled when there's nothing to undo/redo.
5. Do this **without** `structuredClone` **anywhere**, and without any array or object in your history ever being mutated after being stored.

Then answer in a comment: why does storing a plain array of past `columns` values work correctly here, when it would *not* work if any of your update functions had ever mutated `columns` in place? Be specific about what would go wrong.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

12 How Rendering Works

What is it?

Rendering is React calling your component functions to work out what the page should look like. This chapter is about exactly when that happens, what your component sees while it's happening, and how React turns the result into real changes on the screen.

You've used state since [chapter 08](#) by following the rules without needing the machinery underneath. This chapter opens the machinery up. Nothing in it is new behaviour — it's the explanation for behaviour you've already seen.

Why does it matter?

A few things you've already run into only make sense once you see this picture:

- Why `count` inside a click handler is stuck at the value it had when the render started, even after you call `setCount`.
- Why calling `setCount` three times in a row only moves the count by one, unless you use the updater form.
- Why `<StrictMode>` runs your component twice in development.
- Why two `<Counter />`s on the same page never mix up their numbers.

Each of those was a "just trust me for now" moment. This chapter is where you stop trusting and start knowing. Once the model clicks, a lot of React stops feeling like magic and starts feeling like a fairly small, fairly logical machine.

Real-world example

Think about a **photographer taking a family portrait**.

Portrait session	React rendering
"Everyone, hold still!"	React starts a render
The photographer looks at everyone and takes one photo	React calls your component and gets back one description of the page
The photo shows exactly how everyone looked at that instant	That render's state values are fixed, for the whole photo

Portrait session	React rendering
Someone waves <i>after</i> the photo is taken — it's not in this photo	Calling <code>setState</code> inside the render doesn't change what's already being rendered
The photographer compares this photo to the last one, and only reprints the parts that changed	React compares the new result to the old one and updates only what's different
"Everyone, hold still again!" for the next photo	The next render, with fresh values

The photo is not video. It's a snapshot, frozen at one instant, and everything in your component during that render sees the same frozen instant.

How it works

Three steps, every time

Every update to the screen goes through the same three steps:

1. **Trigger.** Something asks for a render: your first call to `createRoot(...).render()`, or a `set` function being called.
2. **Render.** React calls your component function (and every component nested inside it that needs updating) and collects what they return.
3. **Commit.** React compares the result to what's currently on the screen, and changes only the real DOM elements that actually differ.

"Rendering" and "the page changing" are not the same event. Render is React figuring out *what should be true*. Commit is React actually *making it true*. You'll see why that distinction matters in a moment.

Trigger: what starts a render

Two things trigger a render:

- **The initial render.** `main.tsx` calls `createRoot(...).render(<App />)`, which renders `App` for the first time — and everything inside it.
- **A state update.** Calling a `set` function from `useState` schedules a re-render of that component (and its children, more on that below).

That's the complete list. A plain variable changing, a `console.log`, a function running — none of those are on it. Nothing else in React starts a render, which is exactly why changing a `let` in [chapter 08](#) never moved the page.

Render: React calls your function

When a render is triggered, React calls your component function from the top. For the first render, that means every component in the tree, starting from your root (`App` in your projects). For a re-render triggered by state, it means the component whose state changed, and — importantly — **every component nested inside it**, whether or not their own props changed.

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <Child />
    </div>
  );
}

function Child() {
  console.log("Child rendered");
  return <p>I'm a child, I take no props</p>;
}
```

Click the button, and "Child rendered" prints, even though `Child` takes nothing and nothing about it changed. That surprises people the first time. It's not a bug — by default, React just re-runs the whole subtree to be safe, and figures out the *actual* changes afterwards, in the commit step. [Chapter 27](#) covers `memo`, which lets you tell React "skip this subtree if its props are the same."

This is also why the rule from [chapter 03](#) — never define a component inside another component — matters so much. Every one of these renders creates a brand new function, so React treats each render's version of that "component" as a completely different one and throws away everything the previous one was doing.

The render must be fast and pure

Because React might call your component function often — every keystroke in a text box, every click — it needs to be cheap to run, and it needs to give the same result for the same inputs (**pure**, from [chapter 03](#)). No changing state during render, no changing things outside the component, no side effects like `fetch` calls. Those belong in event handlers or in [effects](#) — anywhere except the function body itself.

Commit: React updates the real DOM

Once React has the new description of the page, it compares it to what's currently shown and touches **only the parts that changed**. If a `<p>`'s text changed but its neighbouring `<button>` didn't, only the text node is touched. This comparison process is often called **reconciliation**, or informally "the virtual DOM" — you don't

need to manage it yourself, but it explains why React is fast even though it re-runs whole functions on every keystroke: re-running a function is cheap; touching the real DOM is the expensive part, and React does as little of that as it can.

Keys, from [chapter 06](#), are the input to exactly this process for lists: they're how React decides which item in the new list corresponds to which item in the old one, so it can update or move existing elements instead of throwing them all away and rebuilding from scratch.

On the **very first** render, there's nothing to compare against, so React builds the entire DOM structure from nothing. That's why your app's first paint can feel slightly heavier than an update later on.

The snapshot: state is fixed for the whole render

This is the idea that unlocks the rest of the chapter. **Every render has its own, fixed snapshot of state.** Inside one render, `count` genuinely never changes — it's a `const` that was set once, at the top of that particular call to your component.

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count);           // logs the OLD number, every time
    setTimeout(() => {
      console.log(count);        // still the OLD number, even a second later
    }, 3000);
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Click once, wait three seconds, and both logs show the number from *before* the click — even the delayed one. `handleClick` was created during a specific render, and it closed over that render's `count` (JavaScript chapter 25). Nothing that happens later — not React re-rendering, not three seconds passing — can reach back and change what that particular function remembers `count` to be.

This is not a flaw to work around most of the time; it's what makes React predictable. A handler always sees the world exactly as it was when it was created, no matter when it actually runs.

Why `setCount` three times only moves the count once

Now the classic bug from [chapter 08](#) has a real explanation:

```
function handleClick() {
  setCount(count + 1); // this render's count is 0, so: "set it to 1"
  setCount(count + 1); // still this render's count, still 0: "set it to 1"
  setCount(count + 1); // still 0: "set it to 1"
}
```

All three lines run inside the **same render's snapshot**, so `count` is `0` in every single one of them. They're not "add one, three times" — they're "set it to one," said three times.

The updater function fixes it precisely because it doesn't read the snapshot at all — it asks React for whatever the latest value is *when this update actually runs*:

```
function handleClick() {
  setCount((prev) => prev + 1); // "whatever it is, plus one"
  setCount((prev) => prev + 1); // "whatever THAT becomes, plus one"
  setCount((prev) => prev + 1); // "whatever THAT becomes, plus one"
}
```

React runs these in order, each one building on the result of the last, which is why the count genuinely goes up by three.

Batching

Inside one event handler, React doesn't re-render after every single `set` call. It waits until the handler finishes, then re-renders **once**, with everything applied.

```
function handleClick() {
  setCount(count + 1);
  setName("Tom");
  // React re-renders once here, not twice, with both changes
}
```

This is called **batching**, and it's why the snapshot idea holds together: if React re-rendered after every line, `count` in the second line would already be the *new* value, and the model in this chapter would be far more confusing to reason about. Batching is also just faster — one re-render instead of several.

Since React 18, batching happens almost everywhere — event handlers, timeouts, promises — so you rarely need to think about it directly. It's worth knowing the name, because you'll see it mentioned in library documentation and in older React discussions where it didn't always apply this broadly.

Each render gets its own everything

Not just state — every `const`, every function, every piece of JSX created during a render belongs to that render alone.

```
function Counter() {
  const [count, setCount] = useState(0);
  const message = `Count is ${count}`;    // a fresh string, this render

  function handleClick() {                // a fresh function, this render
    setCount(count + 1);
  }

  return <button onClick={handleClick}>{message}</button>;
}
```

Every time `Counter` runs, `message` and `handleClick` are brand new. The *old* `handleClick`, the one attached to the button before this render, still exists, still remembers the old `count`, and will keep working exactly as it did — until React swaps it for the new one during commit. There is never a shared, mutable `handleClick` that changes underneath you. Each render's version is separate, and separately correct for the moment it was created.

StrictMode, and why your component runs twice

You've seen this since [chapter 03](#): in development, wrapped in `<StrictMode>`, React calls your component function **twice** per render, and discards one set of results.

That's not a bug in your setup and it's not a performance measurement. It's a deliberate check: because render is supposed to be pure, calling it twice with the same state should always produce the same result and cause no visible side effects. If your component secretly changes something outside itself while rendering — a module-level variable, `console.log` used to count renders, anything at all — the double call makes that visible immediately, in development, on your machine, instead of as an intermittent bug in production months later.

It only happens in development, and only wraps render (and, from chapter 17, effects). It never runs twice in your built, deployed app.

Two components, two independent snapshots

```
function Ex() {  
  return (  
    <>  
      <Counter />  
      <Counter />  
    </>  
  );  
}
```

Each `<Counter />` is rendered separately, with its own state, its own snapshot, its own closures. Clicking one calls **that instance's** `handleClick`, built from **that instance's** render, touching **that instance's** state. This is the same fact from [chapter 08](#) — "each component has its own state" — but now you can see mechanically why it's true: there's no single global render happening, there's one render per component, each with everything scoped to it alone.

Common mistakes

1. Expecting a variable read right after `setState` to reflect the new value

```
setCount(count + 1);  
console.log(count); // still old – read it via useEffect (chapter 17) if you need to react to  
the new value
```

`count` won't update until the next render. If you need to *use* the new value immediately, compute it yourself:
`const next = count + 1; setCount(next); console.log(next);`

2. Calling `set` the same way multiple times in one handler, expecting each call to build on the last

```
setCount(count + 1);  
setCount(count + 1); // still reads the same old count
```

Use the updater form, `setCount((prev) => prev + 1)`, whenever the new value depends on the value before it.

3. Treating a `console.log count` as proof of how many times something "really" ran

In development, `StrictMode`'s intentional double-call can make a component or effect look like it ran twice when your built app would only run it once. Don't debug performance in a `StrictMode`-wrapped dev build; check the built app, or read the Console message carefully — React labels `StrictMode`'s extra calls.

4. Assuming a child that takes no props never re-renders

```
function Child() {
  console.log("rendered"); // fires on every parent re-render, by default
  return <p>Static</p>;
}
```

It's fine and normal. It only becomes a performance question at a scale where it's worth measuring — that's [chapter 27](#).

5. Thinking "render" means "the screen changed"

Render is React calling your functions and building a description of the page. Commit is what actually touches the DOM, and React only touches the parts that differ. A component can render and have nothing change on screen at all.

Quick recap

- Every update goes through three steps: **trigger** (initial render or a state update), **render** (React calls your component functions), **commit** (React updates only the real DOM elements that changed).
- Rendering the parent whose state changed re-renders every component nested inside it by default, whether or not their own props changed.
- **Every render has its own fixed snapshot of state and props.** A value read inside a render — including inside a handler defined during that render — never changes for the life of that render, however long the handler takes to actually run.
- That's why three `setCount(count + 1)` calls in a row only move the count by one: use the updater function, `setCount((prev) => prev + 1)`, when the new value depends on the old one.
- **Batching** means several `set` calls inside one event handler cause a single re-render, not one per call.
- **StrictMode** double-calls your component (and effects) in development only, specifically to surface impurity — code that shouldn't be there in the first place.
- Two instances of the same component render completely independently, each with its own snapshot, its own state, and its own closures.

Next: try the [exercises](#), then move on to [13 Lifting State Up](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch12/ex1/`, and so on.

- This chapter is about **predicting** what will happen before you run it, then checking. For every exercise, write your prediction in a comment **before** you click anything. Being wrong is the useful part — it tells you exactly which bit of the model hasn't clicked yet.
- Keep the Console open the whole time.
- An exercise is done when your code matches your (corrected) prediction, VS Code shows no red squiggles, and `npx tsc -b` prints nothing.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Predict the log

In `src/ch12/ex1/Ex1.tsx`:

```
import { useState } from "react";

function Ex1() {
  const [count, setCount] = useState(0);

  function handleClick() {
    console.log("before:", count);
    setCount(count + 1);
    console.log("after:", count);
  }

  return <button onClick={handleClick}>Count is {count}</button>;
}

export default Ex1;
```

1. **Before running it**, write down in a comment what you think the Console will show after one click.
2. Run it, click once, and compare. Were you right?
3. Click four more times in a row, quickly. Write down what the sequence of logs looks like, and explain why "after" is always one behind "the number now on the button."
4. Add a `setTimeout(() => console.log("later:", count), 2000);` inside `handleClick`. Click, wait, and write down what it logs. Why does waiting not change the answer?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Three clicks, one render

In `src/ch12/ex2/Ex2.tsx`, build a counter with a `+3` button.

1. Write it the "obvious" way first: three separate `setCount(count + 1)` calls in the handler. Predict what happens on one click, then test it.
2. Fix it with the updater form so it really adds 3.
3. Add `console.log("rendering, count is", count);` above the `return`. Click `+3` once. How many times does that line print? Was that a surprise?
4. Now wrap `<Ex2 />`'s render in `<StrictMode>` (check `main.tsx` — it may already be there) versus without it, if you want to compare. Which one doubles the render count, and why is that safe to ignore here specifically?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): The reluctant child

In `src/ch12/ex3/Ex3.tsx`:

```
function Timestamp() {
  console.log("Timestamp rendered");
  return <p>Rendered, but I take no props and have no state.</p>;
}

function Ex3() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>
      <Timestamp />
    </div>
  );
}
```

1. Click the button a few times. Does "Timestamp rendered" print every time? Write down what you expected before checking, and whether you were right.
2. In a comment, explain in your own words **why** this happens, using the words "render" and "commit" correctly and separately.
3. Open the **Elements** tab (not Components) and watch the `<p>` while you click. Does the actual DOM text node get touched on every click, or only sometimes? What does that tell you about the difference between a component re-rendering and the DOM actually changing?
4. Add a *second* completely unrelated counter, `<AnotherCounter />`, as a sibling of `Ex3`'s button — its own `useState`, its own button. Clicking the first counter's button: does the second counter's component function run? Write your prediction first.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The stale closure, on purpose

This exercise makes you build the classic React bug yourself, so you recognise it instantly later.

In `src/ch12/ex4/Ex4.tsx`, build a component with a number input and a button labelled **Log in 3 seconds**.

1. Clicking the button should, after a 3-second delay, log the number that was in the box **at the moment you clicked**, and nothing else.
2. Build the "broken" version first: read the input's value into a plain variable (not state) via a ref-free approach — just a `let` inside the component, set from an `onChange` — and log that variable inside the `setTimeout`. Type `5`, click, then type `9` before the 3 seconds are up. What gets logged?
3. Now build it with `useState` for the number, and log the state variable inside the same kind of `setTimeout`, started from a click handler. Repeat the same test: type `5`, click, type `9` before 3 seconds pass. What gets logged this time, and why is it different from step 2?
4. Explain the difference in a comment, using "snapshot" somewhere in your explanation.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Build a tiny render logger

Put everything in this chapter to use by building a small debugging tool — something real React developers actually reach for.

In `src/ch12/ex5/`, build `useRenderCount.ts`, a **custom hook** (don't worry about the official rules for writing one yet — [chapter 20](#) covers that properly; for now, just a function starting with `use` that calls `useRef`, which you also haven't met yet — see the hint) that returns how many times the calling component has rendered, **without causing any extra renders itself**.

```
function ProfileCard() {
  const renderCount = useRenderCount();
  // ...
  return <p>This component has rendered {renderCount} times.</p>;
}
```

Then build `Ex5.tsx`: a component using `useRenderCount`, with a text field, a counter button, and a **totally unrelated** sibling component also using `useRenderCount`. Use it to answer, with evidence from the numbers on screen (not just a guess):

1. Does typing in the text field bump the counter component's render count?
2. Does clicking the counter button bump the text field component's render count?
3. What does `<StrictMode>` do to these numbers on the very first render, compared to later ones?
4. If you wrap one of the two components in `React.memo` (skim ahead to [chapter 27](#) if you want the one-line version: `export default memo(ProfileCard)`), does its render count change when its sibling's state changes?

Write your answer to each as a comment, with the actual numbers you saw.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

13 Lifting State Up

What is it?

Lifting state up means moving a piece of state out of the component that uses it, and into the nearest component that's a parent of everyone who needs it.

```
// Before: two components, each with their own state, unable to talk to each other
function Celsius() {
  const [value, setValue] = useState(0);
  // ...
}
function Fahrenheit() {
  const [value, setValue] = useState(32);
  // ...
}

// After: one piece of state, in the parent, passed down to both
function TemperatureConverter() {
  const [celsius, setCelsius] = useState(0);
  return (
    <>
      <CelsiusInput value={celsius} onChange={setCelsius} />
      <FahrenheitInput value={celsius} onChange={(f) => setCelsius(toCelsius(f))} />
    </>
  );
}
```

You already know every individual piece of this — state, props, and callback props are all from earlier chapters. This chapter is about the *decision*: when two components need to agree on something, where does that something live?

Why does it matter?

[Chapter 08](#) said each component's state is its own, private and separate — two `<Counter />` s never mix up their numbers. That's usually exactly what you want. But sometimes it's the problem.

Picture a filter box and a results list, sitting next to each other:

```
function SearchBox() {
  const [query, setQuery] = useState("");
  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}

function ResultsList() {
  // needs `query` to know what to show... but it has no way to get it
}
```

`SearchBox` knows what was typed. `ResultsList` needs to know what was typed. They're siblings — neither is inside the other — so there's no prop that could carry it from one to the other. Props only flow **down** ([chapter 04](#)), and these two aren't on the same branch.

The fix isn't a new React feature. It's a design move: pull `query` out of `SearchBox` entirely, put it in whatever component contains both of them, and hand it down to each as props. Once it lives in one place, both children can read it, and one of them can change it. This is the single most common structural decision you'll make in a React app, and getting it right early saves you from a much messier fix later.

Real-world example

Think about a **shared calendar** at work versus everyone keeping their own diary.

Everyone keeps their own diary	A shared calendar
Each <code><Counter /></code> , private and separate	Each component with its own <code>useState</code>
You book a meeting in your diary	A component changes its own local state
Your colleague has no way to see it	A sibling component has no way to see it
Move the booking to the shared calendar	Lift the state up to the common parent
Everyone reads the same calendar	Every child reads the same prop
Whoever's allowed edits the shared calendar	Whoever's allowed calls the setter, passed down as a prop

The calendar has to live somewhere everyone involved can see it. Not on your desk, not on theirs — on the wall, above both of you.

How it works

Spot the symptom first

You don't lift state pre-emptively. You start with state wherever feels natural, usually in the component that first needs it, and you lift it only when you hit the actual symptom: **two components that aren't in a parent-child relationship both need to see or change the same value.**

```
function App() {  
  return (  
    <>  
      <SearchBox />  
      <ResultsList />  
    </>  
  );  
}
```

`SearchBox` and `ResultsList` are siblings. Neither can hand the other anything directly. The fix has three steps.

Step 1: find the closest common parent

Walk up the tree from both components until you find the first one that contains both. Here, that's `App`. It doesn't have to be the very top of your app — just the nearest point where both branches meet.

Step 2: move the state there

```
function App() {  
  const [query, setQuery] = useState("");  
  
  return (  
    <>  
      <SearchBox query={query} onQueryChange={setQuery} />  
      <ResultsList query={query} />  
    </>  
  );  
}
```

`App` now owns `query`. `SearchBox` no longer has a `useState` of its own for it at all — it takes the value and a way to change it, as props.

Step 3: make each child use the props instead of its own state

```
type SearchBoxProps = {
  query: string;
  onQueryChange: (query: string) => void;
};

function SearchBox({ query, onQueryChange }: SearchBoxProps) {
  return (
    <input
      value={query}
      onChange={(event) => onQueryChange(event.target.value)}
    />
  );
}

type ResultsListProps = {
  query: string;
};

function ResultsList({ query }: ResultsListProps) {
  const results = allItems.filter((item) => item.name.includes(query));
  return (
    <ul>
      {results.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

Notice `SearchBox`'s input is now a **controlled input** whose truth lives one level up ([chapter 09](#) controlled it from inside its own component; here it's controlled from outside). The pattern is identical — `value` reads from a single source of truth, `onChange` asks for a change — it's just that the source of truth has moved.

The general recipe

1. Identify the state that needs to be shared.
2. Find the closest common parent of every component that touches it.
3. Move the `useState` call there.
4. Pass the **value** down to whoever needs to display it.
5. Pass a **function** down to whoever needs to change it.

Steps 4 and 5 are exactly the "state down, events up" shape from [chapter 07](#) and [chapter 08](#). Lifting state up doesn't introduce a new pattern — it's an application of the one you already know, used to connect siblings instead of a single parent and child.

A worked example: the temperature converter

This is a classic example because both fields are genuinely the *same* value, just displayed two different ways, which makes the "who owns the truth" question unavoidable.

```

function celsiusToFahrenheit(c: number): number {
  return (c * 9) / 5 + 32;
}

function fahrenheitToCelsius(f: number): number {
  return ((f - 32) * 5) / 9;
}

type TemperatureInputProps = {
  label: string;
  value: number;
  onChange: (value: number) => void;
};

function TemperatureInput({ label, value, onChange }: TemperatureInputProps) {
  return (
    <label>
      {label}:
      <input
        type="number"
        value={value}
        onChange={(event) => onChange(Number(event.target.value))}
      />
    </label>
  );
}

function TemperatureConverter() {
  const [celsius, setCelsius] = useState(0);

  return (
    <>
      <TemperatureInput
        label="Celsius"
        value={celsius}
        onChange={setCelsius}
      />
      <TemperatureInput
        label="Fahrenheit"
        value={celsiusToFahrenheit(celsius)}
        onChange={(f) => setCelsius(fahrenheitToCelsius(f))}
      />
    </>
  );
}

```

There's exactly **one** number in state: `celsius`. The Fahrenheit box doesn't have its own state at all — its `value` is *derived* from `celsius` on every render, and typing in it converts back to Celsius before calling `setCelsius`.

This is worth sitting with, because the tempting-but-wrong version has **two** `useState` calls, one per box, kept "in sync" with a manual conversion every time either one changes. That version can drift — a rounding step missed on one side, an update applied to one box but not the other — and it's more code besides. One piece of state, with the other value derived, **cannot** drift, because there's nothing to keep in sync. It's the same "don't store what you can calculate" principle from [chapter 08](#), now applied across two components instead of inside one.

Sharing selection between a list and a detail view

Another extremely common shape: click an item in a list, see its details somewhere else on the page.

```
type Product = {
  id: string;
  name: string;
  price: number;
};

function ProductPage({ products }: { products: Product[] }) {
  const [selectedId, setSelectedId] = useState<string | null>(null);

  const selected = products.find((product) => product.id === selectedId) ?? null;

  return (
    <div className="layout">
      <ProductList
        products={products}
        selectedId={selectedId}
        onSelect={setSelectedId}
      />
      <ProductDetails product={selected} />
    </div>
  );
}
```

`selectedId` lives in `ProductPage`, above both the list and the detail view. `ProductList` highlights whichever id matches, and calls `onSelect` when you click one. `ProductDetails` gets handed the whole matching product, already looked up — it never needs to know about ids or the full list at all. That's worth noticing: `ProductPage` did the `find`, so `ProductDetails`'s job stays simple, which is the same idea from [chapter 04](#) about giving a component the least it needs to do its job.

You're not always lifting to the very top

A common overcorrection is dragging every piece of shared state all the way up to `App`, "just in case." Don't. Lift it only as far as the **nearest** component that contains everyone who needs it.

```
App
└─ Dashboard
    └─ Sidebar (doesn't care about the search)
        └─ SearchPanel
            └─ SearchBox    <- needs `query`
            └─ ResultsList <- needs `query`
```

Here, `query` belongs in `SearchPanel`, not `Dashboard` and definitely not `App`. `Sidebar` never touches it, and giving `App` a `query` it does nothing with just makes `App` bigger and forces every re-render of `query` to start further up the tree than it needs to. Put state as low as it can go while still being visible to everyone who needs it.

Lifting state you already have

Often you're not designing from scratch — you're noticing a component you already wrote needs a sibling to see its state. The mechanical steps are the same:

1. Cut the `useState` line out of the child.
2. Paste it into the parent.
3. Add props to the child for the value and, if the child changes it, a callback.
4. Replace every place the child used its own setter with a call to the callback prop.
5. Fix the types: the child's props type gains the value and the callback.

TypeScript is genuinely useful here — once you delete the `useState` line, every place the old setter was used lights up red, and that list is your checklist for step 4.

When *not* to lift

If only one component ever needs a piece of state, leave it exactly where it is. In [chapter 10](#)'s to-do app, the half-typed text in `AddTaskForm` stayed local — nothing else needed to know about a task before it was submitted. Lifting things that don't need to be shared just makes the parent bigger and forces it to re-render for changes it doesn't actually care about. "Shared state lives with the nearest common owner" cuts both ways: state with only one owner stays exactly where it is.

Common mistakes

1. Two `useState`s trying to stay "in sync"

```
const [celsius, setCelsius] = useState(0);
const [fahrenheit, setFahrenheit] = useState(32);

function handleCelsiusChange(c: number) {
  setCelsius(c);
  setFahrenheit(celsiusToFahrenheit(c)); // easy to forget, easy to get wrong
}
```

If two values are really one fact shown two ways, store the fact once and derive the rest. Two states that must be kept in sync by hand will eventually go out of sync.

2. Lifting state all the way to the root, out of habit

Every re-render of that state now starts higher up the tree than necessary, and components that don't care about it end up receiving it anyway. Lift only as far as the nearest shared parent.

3. Forgetting to remove the old `useState` from the child

```
function CelsiusInput({ value, onChange }: Props) {
  const [value, setValue] = useState(0); // ❌ shadows the prop, and does nothing useful
```

TypeScript will actually stop you here with a duplicate-name error, which is a helpful nudge — but the intent-level mistake is leaving orphaned local state around that no longer does anything, because the prop already provides the real value.

4. Passing the setter down directly when a conversion is needed

```
<TemperatureInput value={fahrenheit} onChange={setCelsius} /> // ❌ sets Celsius to a
Fahrenheit number!
```

If the child's value is a *transformed* view of the shared state, its `onChange` needs to transform back before calling the setter: `onChange={(f) => setCelsius(fahrenheitToCelsius(f))}`.

5. Reaching for Context before trying this

For two or three components close together in the tree, lifting state up is simpler than [Context](#) and doesn't need a new concept. Context earns its place when props would have to pass through many layers that don't care about the value — that's [chapter 22](#).

Quick recap

- When two components that aren't parent and child both need the same value, move the state to their **closest common parent** and pass it down as props.

- The recipe is always the same: state down as a **value**, a **function** passed down to request changes — the pattern from chapters 07 and 08, now connecting siblings.
 - If two displayed values are really the same underlying fact, store it **once** and derive the other from it. Don't keep two states in sync by hand.
 - Lift state only as **high** as it needs to go — to the nearest shared parent, not automatically to the root — and only when it's genuinely shared. State only one component needs stays local.
 - This is a design decision, not a new API. Every tool involved — `useState` , props, callback props — you already had.
-

Next: try the [exercises](#), then move on to [14 Thinking in React](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch13/ex1/` , and so on.
 - Several of these give you two components that **can't** talk to each other yet, on purpose. Get it running broken first, so you can see the actual symptom, before you fix it.
 - An exercise is done when both components genuinely agree, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console is empty.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Two clocks, one truth

In `src/ch13/ex1/Ex1.tsx` , copy this broken starting point:


```

import { useState } from "react";

function BrightnessSlider() {
  const [brightness, setBrightness] = useState(50);
  return (
    <input
      type="range"
      min={0}
      max={100}
      value={brightness}
      onChange={(e) => setBrightness(Number(e.target.value))}
    />
  );
}

function BrightnessPreview() {
  // needs to know the brightness... but from where?
  return <div style={{ width: 100, height: 100, background: `rgb(0,0,0)` }} />;
}

function Ex1() {
  return (
    <div>
      <BrightnessSlider />
      <BrightnessPreview />
    </div>
  );
}

export default Ex1;

```

1. Run it. The slider moves, but the preview box never changes shade. In a comment, explain in your own words *why* `BrightnessPreview` has no way to know the current value.
2. Lift `brightness` up into `Ex1`, following the three steps in the notes. `BrightnessSlider` should have **no `useState` left in it.**
3. Make `BrightnessPreview`'s box shade change with the value: `rgb(x, x, x)` where `x` is the brightness, 0 to 100 scaled to 0–255.
4. Add a third component, `BrightnessLabel`, showing `50%` in text, reading from the same lifted state.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Search box and results, properly

Build `src/ch13/ex2/` with:

```
export type Book = {
  id: string;
  title: string;
  author: string;
};

export const books: Book[] = [
  { id: "1", title: "The Hobbit", author: "J. R. R. Tolkien" },
  { id: "2", title: "Dune", author: "Frank Herbert" },
  { id: "3", title: "Small Gods", author: "Terry Pratchett" },
  { id: "4", title: "The Left Hand of Darkness", author: "Ursula K. Le Guin" },
];
```

Three components:

- **SearchBox**: a controlled text input, no local state — everything comes from props.
- **ResultCount**: shows 2 of 4 books match "dune", or 4 of 4 books when the search is empty.
- **ResultsList**: the filtered list, matching title **or** author, case-insensitive.

Ex2 owns the one piece of state, `query`, and renders all three.

Then, in a comment: which component would you have picked as the "natural" place to put `query` if you'd started without reading this chapter? Was it the right choice, and why or why not?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Tabs with a shared "unsaved changes" flag

This is a case where the thing you're sharing isn't the displayed data itself, but a flag *about* it.

In `src/ch13/ex3/`, build a two-tab settings page: **Profile** (a name field) and **Notifications** (two checkboxes). Only one tab's content shows at a time, switched by two tab buttons.

Requirements:

1. Typing in the name field, or toggling a checkbox, sets a shared `hasUnsavedChanges` flag to `true`.
2. Whenever the flag is `true`, a bar appears above the tabs: `You have unsaved changes.` with a **Save** button.
3. Clicking **Save** clears the flag (logging the current form values is enough — you don't need a real save).
4. Clicking a tab button while `hasUnsavedChanges` is `true` shows a confirmation via `window.confirm("You have unsaved changes. Switch tabs anyway?")`. Only actually switch if the person clicks OK.
5. Neither `ProfileTab` nor `NotificationsTab` should need to know the other exists.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Two sliders that must add up to 100

A workout app splits a session between **Cardio** and **Strength**, as two percentage sliders that must always add up to exactly 100%.

Build `src/ch13/ex4/Ex4.tsx` with two `<input type="range">`s, each 0–100, and their labels showing the live percentage. Moving one must instantly adjust the other so they always sum to 100 — no button, no lag.

1. There must be only **one** piece of state.
2. Dragging Cardio to 70 makes Strength show 30, live, and vice versa.
3. Add a **Reset to 50/50** button.
4. Add a written summary below: `70% Cardio, 30% Strength – a cardio-focused session` (swap the wording for `strength-focused` below 50, and `a balanced session` at exactly 50).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Master-detail with editing

The fullest version of the list-and-details pattern from the notes. In `src/ch13/ex5/`, start with:

```
export type Contact = {
  id: string;
  name: string;
  email: string;
  notes: string;
};

export const initialContacts: Contact[] = [
  { id: "1", name: "Maya", email: "maya@example.com", notes: "Prefers email." },
  { id: "2", name: "Tom", email: "tom@example.com", notes: "" },
  { id: "3", name: "Priya", email: "priya@example.com", notes: "Call after 5pm." },
];
```

Build:

- **ContactList**: shows names only, one per row, highlighting whichever is selected. Clicking a row selects it. Includes a + **New contact** button.
- **ContactDetails**: shows the selected contact's full details in an **editable form** (name, email, notes). Changes save as you type — no separate Save button. Shows `Select a contact` when nothing's selected.

- **ContactApp** : owns everything — the contacts array and which id is selected — and puts the two side by side.

Requirements:

1. Editing a field in `ContactDetails` updates that contact in the array immediately, and `ContactList` reflects a name change **live**, as you type it.
2. **+ New contact** adds a blank contact, selects it immediately, and puts the cursor in the name field. (`autoFocus` on the input is enough — [chapter 19](#) covers the more general tool.)
3. Deleting the selected contact (add a **Delete** button in `ContactDetails`) removes it from the list and clears the selection. If contacts remain, nothing needs to auto-select — `Select a contact` is a perfectly good result.
4. `ContactDetails` never reaches into the full `contacts` array itself — it only ever sees the one contact it's editing, and reports changes upward through a callback.
5. Selecting a contact that no longer exists (which can happen right after a delete) must not crash the app. Prove that it doesn't by deleting the currently selected contact and confirming the page still renders cleanly.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

14 Thinking in React

What is it?

A repeatable method for turning a design — a mockup, a sketch, a screenshot of an app you're copying — into working React code. Five steps:

- 1. Break the UI into a component hierarchy.
- 2. Build a static version, with no state at all.
- 3. Find the minimal set of state that represents everything that can change.
- 4. Decide where each piece of that state lives.
- 5. Add the events that flow data back up.

You already know every tool this method uses — components, props, state, lifting state up. This chapter is about the **order** to apply them in, so that a blank page and a design don't leave you staring, unsure where to start.

Why does it matter?

Every chapter so far started with the code already half-decided for you: "build a `MenuItem` that takes `name` and `price`." Real work doesn't arrive like that. It arrives as a Figma file, a screenshot, or "make it look like this other site," and *you* have to decide what the components are, what's state, and where that state should live.

Without a method, it's easy to start writing JSX for the whole page at once, discover three screens in that you need to share a value between two far-apart bits of it, and end up restructuring everything. This chapter's method front-loads exactly the decisions that are expensive to get wrong later — component boundaries and where state lives — before you've written much code to un-write.

Real-world example

Think about how an **architect** designs a house, not a builder swinging a hammer on day one.

Building a house	Thinking in React
Mark out the rooms on the floor plan	Break the UI into components
Build the shell, walls, doors, windows — no plumbing or wiring yet	Build a static version
Decide what actually needs power and water, and where	Find the minimal state
Decide which room has the fuse box and the stopcock	Decide where state lives
Wire the switches to the lights	Add the events that flow data back up

Nobody starts a house by wiring a light switch before the wall it's on exists. The order matters, and it matters for the same reason here: structure first, behaviour second.

How it works

The example for this chapter

A small inventory search tool for a hardware shop. Given this data:

```
type Product = {
  id: string;
  name: string;
  category: "Tools" | "Fixings" | "Paint";
  price: number;
  inStock: boolean;
};

const products: Product[] = [
  { id: "1", name: "Claw Hammer", category: "Tools", price: 12.5, inStock: true },
  { id: "2", name: "Wood Screws (100)", category: "Fixings", price: 4.25, inStock: true },
  { id: "3", name: "Masonry Drill Bit", category: "Tools", price: 6.0, inStock: false },
  { id: "4", name: "Matt White Paint 2.5L", category: "Paint", price: 18.99, inStock: true },
  { id: "5", name: "Wall Plugs (50)", category: "Fixings", price: 2.75, inStock: false },
];
```

it should show a search box, an "in stock only" checkbox, and a table of matching products grouped by category:

```
Search: [          ] [x] In stock only

Tools
  Claw Hammer          $12.50

Fixings
  Wood Screws (100)     $4.25

Paint
  Matt White Paint 2.5L  $18.99
```

Step 1: break the UI into a component hierarchy

Look at the design and draw boxes around things, the way you'd draw boxes around paragraphs in a document. The rule of thumb from [chapter 03](#) still applies: **one component per meaningful chunk**, and a chunk you'd give a name if you were describing the page out loud.

```
InventoryPage
├─ SearchControls    (the search box and the checkbox)
└─ ProductTable
   └─ CategorySection (one per category: Tools, Fixings, Paint)
      └─ ProductRow   (one per product)
```

A useful test: if you can point at a rectangle in the design and say its name in one breath — "that's the search bar," "that's a product row" — it's probably a component. Notice this step needed **no code and no state**. It's purely about shape.

Step 2: build a static version

Now write the JSX for that whole hierarchy, using **plain props, no state at all**. Everything is hard-coded or passed straight down from the top, exactly the way you built components in [chapters 03 and 04](#), before [chapter 08](#) existed.

```

function ProductRow({ product }: { product: Product }) {
  return (
    <tr>
      <td>{product.name}</td>
      <td>${product.price.toFixed(2)}</td>
    </tr>
  );
}

function CategorySection({ category, products }: { category: string; products: Product[] }) {
  return (
    <>
      <tr><th colspan={2}>{category}</th></tr>
      {products.map((product) => (
        <ProductRow key={product.id} product={product} />
      ))}
    </>
  );
}

function ProductTable({ products }: { products: Product[] }) {
  const categories = [...new Set(products.map((p) => p.category))];

  return (
    <table>
      <tbody>
        {categories.map((category) => (
          <CategorySection
            key={category}
            category={category}
            products={products.filter((p) => p.category === category)}
          />
        ))}
      </tbody>
    </table>
  );
}

function SearchControls() {
  return (
    <div>
      <input type="text" placeholder="Search..." />
      <label><input type="checkbox" /> In stock only</label>
    </div>
  );
}

function InventoryPage() {
  return (

```



```

    <div>
      <SearchControls />
      <ProductTable products={products} />
    </div>
  );
}

```

Data only flows **down** here — props, all the way — exactly one direction, same as every chapter so far. Nothing is interactive yet: typing in the search box does nothing, the checkbox does nothing. That's correct. This step is about proving the *shape* works and the data reaches everywhere it needs to, before a single line of state complicates it. It's more typing than jumping straight to the "real" version feels like it should need, and it's worth it: bugs in structure are far easier to see and fix when nothing is moving yet.

Step 3: find the minimal but complete state

This is the step most worth slowing down for. Go through **every piece of data** in your app and ask three questions about each:

1. **Does it stay the same over time?** If yes, it's not state — it's a constant, like your `products` array (well, until step 5 gives you a reason to change that).
2. **Can you calculate it from other state or props?** If yes, it's not state either — it's a [derived value](#).
3. **Does it change because of something the user does, and can you not work it out from anything else?** If yes to all of that, it's real state.

Walk through the candidates for this page:

Candidate	Does it change?	Can it be derived?	Real state?
The full <code>products</code> list	No (for now)	—	No — it's a constant
What's typed in the search box	Yes	No	Yes
Whether "in stock only" is ticked	Yes	No	Yes
The list of products matching the search and the filter	Yes	Yes — from <code>products</code> , the search text, and the checkbox	No — derive it
The categories present in the filtered results	Yes	Yes — from the filtered list	No — derive it

Two pieces of real state survive: the **search text** and the **stock filter**. Everything else on the page — the filtered products, the grouped categories — gets worked out fresh on every render, the same "don't store what you can

calculate" rule from [chapter 08](#), now applied at the scale of a whole feature instead of one component.

This is the step people skip, and it's the one that saves you the most trouble. Storing the filtered list *as well as* the search text means keeping them in sync by hand forever. Storing only the two real inputs means the filtered list literally cannot go stale, because it's recalculated from them every time.

Step 4: identify where each piece of state should live

For each piece of state, find **every component that reads it or writes it**, then find their closest common parent — exactly the process from [chapter 13](#).

- **Search text:** read and written by `SearchControls`. Also needed by `ProductTable`, to know what to show. `SearchControls` and `ProductTable` are siblings, so it belongs in their common parent: `InventoryPage`.
- **Stock filter:** same story, same components, same answer: `InventoryPage`.

Sometimes this step reveals that a component you drew in step 1 doesn't actually need to exist as its own state-holder — it's just a shape for the JSX. That's fine. Step 1 was about visual structure; this step is about data ownership, and they don't always draw the same lines.

Step 5: add inverse data flow

"Inverse" just means **up**, the direction props don't go on their own — the callback-prop pattern from [chapter 07](#). Now that state lives in `InventoryPage`, the components below it need a way to ask for changes.

```

type SearchControlsProps = {
  searchText: string;
  onSearchTextChange: (text: string) => void;
  inStockOnly: boolean;
  onInStockOnlyChange: (value: boolean) => void;
};

function SearchControls({
  searchText,
  onSearchTextChange,
  inStockOnly,
  onInStockOnlyChange,
}: SearchControlsProps) {
  return (
    <div>
      <input
        type="text"
        placeholder="Search..."
        value={searchText}
        onChange={(event) => onSearchTextChange(event.target.value)}
      />
      <label>
        <input
          type="checkbox"
          checked={inStockOnly}
          onChange={(event) => onInStockOnlyChange(event.target.checked)}
        />
        In stock only
      </label>
    </div>
  );
}

function InventoryPage() {
  const [searchText, setSearchText] = useState("");
  const [inStockOnly, setInStockOnly] = useState(false);

  const filtered = products.filter((product) => {
    const matchesSearch = product.name.toLowerCase().includes(searchText.toLowerCase());
    const matchesStock = !inStockOnly || product.inStock;
    return matchesSearch && matchesStock;
  });

  return (
    <div>
      <SearchControls
        searchText={searchText}
        onSearchTextChange={setSearchText}
        inStockOnly={inStockOnly}

```

```

        onInStockOnlyChange={setInStockOnly}
      />
      <ProductTable products={filtered} />
    </div>
  );
}

```

`ProductTable`, `CategorySection` and `ProductRow` don't change **at all** from step 2. They were already built to take a `products` array as a prop and display it — they have no idea whether that array is the full list or a filtered one, and they don't need to. All the new logic landed in exactly the two places step 4 said it should: the two controls that write the state, and the one component that owns it.

Why the order matters

Notice what each step protected you from:

- Doing step 3 **before** step 1 would mean guessing at state before you know what's actually on the page.
- Doing step 5 **before** step 3 would mean wiring up `onChange` handlers before you've decided what they're even supposed to set.
- Skipping step 2 tends to produce components tangled up with state from the very first line, which makes it much harder to later notice that half of what you stored could have been derived.

The method isn't a rulebook you must follow to the letter forever. Once this becomes second nature, you'll blend the steps without thinking about it. But when you're stuck looking at a design with no idea where to begin, coming back to these five steps in order is a reliable way to get moving again.

A second, smaller example: a tabbed panel

Worth doing once quickly with less data, to see the method isn't just for tables.

Design: three tabs (`Details`, `Reviews`, `Shipping`), one panel of content visible at a time.

1. **Hierarchy:** `ProductPage` → `TabBar` + `TabPanel`.
2. **Static version:** hard-code which tab shows, render `TabBar` (three buttons, none wired up) and `TabPanel` (always showing "Details," say).
3. **Minimal state:** the only thing that changes is *which* tab is active. That's one piece of state — a string, constrained to the three tab names with a union type, not three separate booleans (`"details"` | `"reviews"` | `"shipping"`, echoing the lookup-object pattern from [chapter 05](#)).
4. **Where it lives:** both `TabBar` (to highlight the active one and to change it) and `TabPanel` (to know what to show) need it, and they're siblings — so it lives in `ProductPage`.
5. **Inverse flow:** `TabBar` takes `activeTab` and `onTabChange`, and calls `onTabChange` when a button is clicked.

Same five steps, a tenth of the size. The method scales down as well as up.

Common mistakes

1. Writing state and interactivity from the very first line

Jumping straight to `useState` before the component shapes exist tends to produce state that doesn't match what the page actually needs — too much, too little, or in the wrong component. Build the static version first.

2. Storing a value that step 3 would have caught

```
const [filteredProducts, setFilteredProducts] = useState(products); // ❌
```

If you find yourself calling a setter every time *another* piece of state changes, just to "keep something updated," that something almost always shouldn't have been state at all. Derive it instead.

3. Putting state in the wrong component out of convenience

```
function SearchControls() {  
  const [searchText, setSearchText] = useState(""); // only SearchControls can see it  
  // ...  
}
```

It compiles, and the input works — right up until `ProductTable`, a sibling, also needs `searchText` and has no way to get it. This is the exact symptom [chapter 13](#) is about. Step 4 exists to catch this before you've written the component.

4. Treating the five steps as a one-way street

Real design work loops. Building the static version (step 2) often reveals a component boundary from step 1 that doesn't actually make sense once there's real JSX in it. Redraw the boxes and carry on — the method is a guide, not a contract.

5. Skipping the "can I derive it?" question for anything that isn't obviously a number or a list

Booleans slip through this check constantly. `isValid`, `hasResults`, `isEmpty` — all of these are almost always one line computed from other state, not state of their own.

Quick recap

- Five steps, in order: **break the UI into components, build a static version with no state, find the minimal state, decide where each piece lives, add the events that flow data back up.**

- For every candidate piece of state, ask: does it change, and can it be worked out from something else? If it can be derived, it isn't state.
- To find where a piece of state should live, find every component that reads or writes it and put it in their closest common parent — the same process as [chapter 13](#).
- The static version from step 2 usually needs **no changes at all** once state is wired up in step 5 — the new logic lands entirely in the components that own and write the state.
- The method isn't a strict pipeline. Real design loops back on itself; the value is having a reliable place to start and a reliable order to come back to when you're stuck.

Next: try the [exercises](#), then move on to [15 Styling](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch14/ex1/`, and so on.
- **Write out all five steps as comments before you write any real component code** — the component tree, which candidates you rejected as state and why, and where each surviving piece lives. The plan is most of the exercise. The code should follow quickly once the plan is right.
- An exercise is done when the page works, your five-step plan is in the file as comments, VS Code shows no red squiggles, and `npm tsc -b` prints nothing.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A rating widget

Design: five stars in a row. Clicking a star sets the rating to that many stars, filling every star up to and including it. Hovering shows a preview fill, without committing to it — moving the mouse away reverts to the actual rating.

★★★★☆ (rating: 2)

1. Write the five steps as comments in `src/ch14/ex1/Ex1.tsx`: the component hierarchy (there's a `StarRating` and a `Star`, at minimum), the state candidates and which survive, and where they live.
2. Build the static version first — five stars, all rendered, none of them doing anything yet.
3. Add the state and wire it up.

Pay particular attention to step 3 for the **hover preview**: is it the same piece of state as the actual rating, or a second one? Get this wrong and either hovering will overwrite your real rating, or the preview won't work at all.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): A shopping list with a running total

Design:

[item name	]	[qty]	[Add]
Milk	x 2	\$3.00	
Bread	x 1	\$2.50	

Total:		\$5.50	

- 1. Plan the five steps for `src/ch14/ex2/Ex2.tsx`. There are at least three components: the add-form, the list, and one row.
- 2. Give every item a fixed price by name from a small lookup object (`{ Milk: 1.5, Bread: 2.5, ... }`) rather than typing a price in — assume the form only offers items that exist in that lookup, via a `<select>`.
- 3. Build it.

In your state-candidates table (step 3), make sure "the total" appears as a rejected candidate, with a one-line reason.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): A filterable, sortable staff directory

Design: a table of employees, with a search box, a department dropdown, and clickable column headers that sort by that column (clicking the same header again reverses the order).

Search: [	]	Department: [All ▾]
Name ▲	Department	Start Date
Maya	Engineering	2022-01-10
Tom	Sales	2021-06-01
Priya	Engineering	2023-03-15

Start with:

```
export type Employee = {
  id: string;
  name: string;
  department: "Engineering" | "Sales" | "Support";
  startDate: string;
};

export const employees: Employee[] = [
  { id: "1", name: "Maya", department: "Engineering", startDate: "2022-01-10" },
  { id: "2", name: "Tom", department: "Sales", startDate: "2021-06-01" },
  { id: "3", name: "Priya", department: "Engineering", startDate: "2023-03-15" },
  { id: "4", name: "Sam", department: "Support", startDate: "2020-11-20" },
  { id: "5", name: "Ana", department: "Sales", startDate: "2023-07-02" },
];
```

1. Plan and write the five steps in `src/ch14/ex3/`. Be deliberate about step 3: search text, department filter, sort column, and sort direction are four separate, real pieces of state. The visible rows are not — they're derived from all four plus the constant array.
2. Build the static version: the table and controls, rendering the full unsorted, unfiltered list, with none of the controls wired up.
3. Wire it up. Clicking a header that's already the active sort column reverses direction instead of doing nothing.
4. Show ▲ or ▼ next to whichever column is currently sorted.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A multi-step design, given only the picture

This one starts you further from the code than usual, on purpose. Here's the whole design, described in words instead of a table of data:

A **flashcard quiz app**. One card is shown at a time, with a question. Clicking the card flips it to reveal the answer. Two buttons, **Got it right** and **Got it wrong**, appear only once the card is flipped, and move to the next card either way. A progress line reads `Card 3 of 8`. At the end, a results screen shows `You got 5 out of 8 right`, with a **Start over** button.

1. Sketch the component hierarchy in a comment — there's no single right answer, but you should be able to justify your boxes.

2. Write out every candidate for state you can think of, and for each one, say whether it survives and why, in a comment table like the ones in the notes.
3. Decide where each surviving piece lives — with this few components, that part may be quick, but write it down anyway.
4. Build the static version, then wire it up, in `src/ch14/ex4/`.

Use this data:

```
export type Card = {
  id: string;
  question: string;
  answer: string;
};

export const cards: Card[] = [
  { id: "1", question: "What does JSX compile to?", answer: "JavaScript function calls" },
  { id: "2", question: "What hook holds state?", answer: "useState" },
  { id: "3", question: "What must every list item have?", answer: "A unique, stable key" },
];
```

(Use just these three while you build; duplicate a couple to test with more once it works.)

Stuck? [Hint 1](#) and [Hint 2](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Redesign something you already built

Go back to the **library checkout page** from [chapter 04, exercise 5](#) or the **AccountPage** from [chapter 05, exercise 5](#) — pick whichever one you remember better. Neither had any state at all; they were built from fixed props.

In `src/ch14/ex5/`, redesign it as a genuinely interactive page using the five-step method:

1. **New design brief:** each loan row gets a **Renew** button (only for loans that are renewable and not late, per the original rules) that adds 14 days to a due date and resets `daysLate` to 0. Add a **Return** button that removes the loan from the account entirely. Add a text box that filters the visible loans by title as you type.
2. Write the five steps from scratch for this new version — don't just bolt state onto the old component. Some of your original components may need different props now that some of what they show is state instead of a fixed prop; say so in a comment where that happens.
3. Build it.
4. In a closing comment, answer honestly: what would have gone wrong if you'd added `useState` calls into the *old* components directly, without redoing the ownership step? Be specific about which component

would have ended up needing data it had no way to reach.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

15 Styling

What is it?

Four ways to put CSS on a React page, in the order this chapter covers them:

```
import "../App.css";           // 1. a plain, global CSS file
import styles from "../Card.module.css"; // 2. CSS Modules: scoped to one component

<div className="card">          // 1 & 2: fixed class names
<div style={{ color: "tomato" }}> // 3. inline styles, from chapter 02
<div className={isActive ? "tab active" : "tab"}> // 4. classes that depend on state
```

You've used `className` and `style` since [chapter 02](#). This chapter is about organising real stylesheets, and the one genuinely new tool: **CSS Modules**.

Why does it matter?

The [to-do app project](#) came with one finished `index.css`, written for you, and it worked because it was the *only* stylesheet in the app. Real apps have dozens of components, often built by different people, and plain CSS has one property that makes that dangerous: **every class name is global**, no matter which file it's written in.

```
/* Header.css */
.title { font-size: 24px; }

/* ProductCard.css */
.title { font-size: 14px; color: grey; }
```

Import both files into the same app, and whichever one loads second wins — for **every** `.title` on the page, in every component, everywhere. Nothing in the file names or the folder structure protects you. This is the exact problem JavaScript chapter 29 solved for *variables* with modules; CSS Modules solve the same problem for *class names*.

Real-world example

Think about **name tags at two different kinds of event**.

A small meetup	A big conference with several tracks
Plain CSS, one shared stylesheet	Global styles: fine when there's not much overlap

A small meetup	A big conference with several tracks
Everyone writes their own name on a blank tag	Everyone picks their own class names
Two "Alex"es show up, and it's genuinely confusing	Two <code>.title</code> classes collide, and the wrong one wins
The big conference prints tags as "Alex — Room 4B12"	CSS Modules rename <code>.title</code> to <code>.title_a8f3k</code> , unique to that file
Nobody has to <i>agree</i> on names in advance	Nobody has to <i>agree</i> on names in advance

The bigger the event, the more that automatic uniqueness is worth having.

How it works

Plain CSS files, imported

You've done this since [chapter 01](#): Vite lets you `import` a `.css` file directly in a `.tsx` file, and it gets included in the page.

```
// App.tsx
import "../App.css";

function App() {
  return <h1 className="title">Corner Café</h1>;
}
```

```
/* App.css */
.title {
  font-size: 28px;
  color: #2b2b2b;
}
```

The import has **no local name** — `import "../App.css"` rather than `import styles from "../App.css"` — because you're not asking for anything back. You're telling Vite "include this stylesheet," and then you refer to the classes by their plain string names in `className`, same as HTML always worked.

This is exactly what you did in the to-do app, and it's completely fine for **one shared stylesheet for the whole app**: variables, resets, typography, and anything meant to apply everywhere. It's the wrong tool the moment two different components' styles need to stay out of each other's way.

CSS Modules

Name a file `*.module.css`, and Vite treats it specially: instead of injecting the classes globally, it renames every class to something unique and gives you an object mapping your original names to the real ones.

```
/* Card.module.css */
.card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
}

.title {
  font-size: 18px;
  font-weight: 600;
}
```

```
// Card.tsx
import styles from "./Card.module.css";

function Card() {
  return (
    <div className={styles.card}>
      <h2 className={styles.title}>Espresso</h2>
    </div>
  );
}
```

`styles` is a plain object: `{ card: "Card_card_a8f3k", title: "Card_title_x921m" }` — Vite generates the real names for you, and they're guaranteed unique across your whole app, however many other files also have a class called `.title`. You never write the generated name yourself; you always go through `styles.something`.

The import needs a name, because this time you're genuinely using what comes back: `import styles from "./Card.module.css"`. Calling it `styles` is a strong convention, not a rule — you could call it anything, but everyone calls it `styles`, so do too.

TypeScript and CSS Modules

Hover over `styles` in VS Code and you'll usually see `any` or `{ [key: string]: string }`, depending on your project's setup — Vite's TypeScript template includes a declaration file (often `vite-env.d.ts`) that tells TypeScript "any `*.module.css` import is an object of strings." That's enough to let you write `styles.card`, but it's not enough to catch a typo:

```
<div className={styles.crad}> // no error! "crad" just doesn't exist in the real CSS either
```

TypeScript can't see inside the actual CSS file to know which class names really exist, so a misspelled property silently becomes `undefined`, and `className={undefined}` just means no class is applied — no crash, no red squiggle, just a plain `<div>` that isn't styled the way you expected. Check the Elements tab in DevTools if a CSS Modules class seems to be doing nothing; that's usually why.

When to use which

Use	For
A plain, global <code>.css</code> file	App-wide resets, typography, CSS variables/theme colours, one file imported once near the root
CSS Modules	Anything specific to one component — a card, a button, a modal
Inline <code>style</code>	A value computed at runtime: a colour from data, a width from a percentage (chapter 02)

Most real projects use the first two together: one small global stylesheet for the shared basics, and a `.module.css` file sitting right next to each component that needs its own styling. Put a component's module file next to its `.tsx` file with a matching name — `Card.tsx` and `Card.module.css` — so anyone opening the folder can see immediately which styles belong to which component.

Conditional class names

You've built these by hand since [chapter 05](#):

```
<li className={task.completed ? "task done" : "task"}>{task.text}</li>
```

For two classes it's fine. For three or four conditions, string-building gets messy fast:

```
<button
  className={
    (isPrimary ? "btn btn-primary " : "btn ") +
    (isDisabled ? "btn-disabled " : "") +
    (size === "large" ? "btn-large" : "btn-small")
  }
>
```

Stray spaces, missing spaces, and forgotten classes are all easy to introduce in code like that. A small helper function tidies it up — this is a pattern you'll see called `clsx` or `classnames` when it comes from a library, but

it's simple enough to write yourself:

```
function cx(...classes: (string | false | undefined][]): string {  
  return classes.filter(Boolean).join(" ");  
}
```

```
<button  
  className={cx(  
    "btn",  
    isPrimary && "btn-primary",  
    isDisabled && "btn-disabled",  
    size === "large" ? "btn-large" : "btn-small"  
  )}  
>
```

`isPrimary && "btn-primary"` gives you either the class name or `false` ([chapter 05](#) again), and `cx` filters the falsy ones out before joining what's left with spaces. `Boolean` used as a callback here is a handy trick — it's the same function you'd normally call as `Boolean(x)`, passed directly to `filter` so it keeps only truthy items.

With **CSS Modules**, the same idea, going through `styles`:

```
<button className={cx(styles.btn, isPrimary && styles.primary, isDisabled && styles.disabled)}>
```

Popular real projects usually reach for the `clsx` package instead of writing `cx` themselves — it's a few lines, well-tested, and saves everyone reinventing it. This course keeps it hand-written here so you understand exactly what it's doing; swapping in the real package later is a one-line change.

CSS variables for theming

You've already seen this pattern in the to-do app's stylesheet: CSS custom properties (variables), switched by a class or attribute on a parent element, work perfectly well with React and need nothing special:

```

:root {
  --accent: #3d6fd8;
  --bg: #ffffff;
}

.app.dark {
  --accent: #6b95ec;
  --bg: #1b1f24;
}

.button {
  background: var(--accent);
}

body {
  background: var(--bg);
}

```

```
<div className={isDark ? "app dark" : "app"}>
```

React doesn't need to know these variables exist. It's just toggling a class name, same as any other conditional class, and the browser's own CSS variable resolution does the rest. This is usually simpler than trying to manage a theme entirely through inline styles or JavaScript.

Where global styles belong

Keep exactly one, small global stylesheet, imported once, high up — usually in `main.tsx`:

```
import "./index.css";
```

Good candidates for it: a CSS reset (`* { box-sizing: border-box; }`), `body` defaults, your colour variables, and `@font-face` rules. Anything more specific than that belongs in a module next to the component it styles. A global stylesheet that grows into hundreds of unscoped class names is exactly the collision risk this chapter opened with, just moved one file later.

A note on other approaches

You'll come across other ways to style React apps in tutorials and job listings: **CSS-in-JS** libraries (writing CSS inside your `.tsx` files, as template literals or objects, so the styles live right next to the component that uses them) and **Sass** (a CSS preprocessor with variables and nesting, predating CSS's own). Both were extremely popular for a few years. CSS Modules plus plain CSS variables — what this chapter teaches — cover the same

ground with less machinery, and it's what this course uses throughout. [Chapter 16](#), next, is a different, and currently very popular, philosophy again: skip writing custom CSS almost entirely.

Common mistakes

1. Two class names colliding across files

```
/* Header.css */      /* ProductCard.css */  
.title { ... }        .title { ... }
```

Import both as plain CSS and only one wins, silently, for the whole app. Use CSS Modules for anything component-specific.

2. Forgetting the import needs a name for modules

```
import "./Card.module.css";      // ❌ nothing to reference the classes by  
import styles from "./Card.module.css"; // ✅
```

Without a name, you have no way to reach the generated class names at all.

3. Writing the class name as a plain string with a module import

```
import styles from "./Card.module.css";  
  
<div className="card"> // ❌ looks for a literal class called "card", which doesn't exist  
any more
```

The whole point of a module is that the real class name is something like `Card_card_a8f3k`. You always go through the object: `className={styles.card}`.

4. A typo in a module property, with no warning

```
<div className={styles.crad}> // undefined, className={undefined}, unstyled – no error  
anywhere
```

If a CSS Modules class isn't applying, check the property name against the actual CSS file. TypeScript's default setup for these files usually can't catch this for you.

5. String-concatenating several conditional classes by hand

```
className={"btn " + (isPrimary ? "btn-primary" : "") + " " + (isDisabled ? "btn-disabled" : "")}
```

Easy to get wrong (stray or missing spaces), and hard to read. Use a small `cx` helper, or the `clsx` package.

6. Putting component-specific styles in the one global stylesheet

It works, but it's the exact collision risk this chapter exists to avoid, and it makes `index.css` grow forever. Give a component its own `.module.css` file next to it instead.

Quick recap

- Plain CSS, imported with `import './file.css'`, is **global**: every class name can collide with any other file's class of the same name. Keep it for a small, shared, app-wide stylesheet.
- **CSS Modules** (`*.module.css`, imported as `import styles from './File.module.css'`) generate unique class names automatically, so component styles can never collide. Use them for anything specific to one component.
- Reach the real class names only through the imported object: `className={styles.card}`, never a plain string.
- TypeScript usually can't check that a `styles.something` property really exists in the CSS — a typo silently applies no class at all.
- Build conditional class names with `&&` inside a small `cx` helper (or the `clsx` package) rather than concatenating strings by hand.
- Inline `style`, from [chapter 02](#), is still the right tool for values computed at runtime. CSS variables, switched by a class name, are a clean way to handle themes without JavaScript doing the colour math.

Next: try the [exercises](#), then move on to [16 Tailwind CSS](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch15/ex1/`, and so on. Each `.tsx` file that needs styles gets a `.module.css` file right next to it, unless an exercise says otherwise.
- Open the **Elements** tab in DevTools and check the actual class names being applied — several of these exercises are about noticing when a class name *silently* isn't the one you expected.
- An exercise is done when the page looks right, VS Code shows no red squiggles, `npm run tsc` prints nothing, and the Console has no errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Make the collision happen, then fix it

In `src/ch15/ex1/`, create two plain (not module) CSS files:

`Banner.css`:

```
.title {  
  font-size: 28px;  
  color: darkred;  
}
```

`Sidebar.css`:

```
.title {  
  font-size: 12px;  
  color: grey;  
}
```

And two components, `Banner.tsx` and `Sidebar.tsx`, each importing its own CSS file and rendering `<h2 className="title">...</h2>` with different text.

1. Render both inside `Ex1.tsx`. Look at the page. Which styling won? Open the Elements tab and confirm it in the computed styles.
2. In a comment, explain why import order, not which file "looks more specific," decided the winner.
3. Now fix it: rename both files to `.module.css`, update the imports and the `classNames`, and confirm both headings now look correct and independent, in either import order.
4. Swap the order of the two imports in `Ex1.tsx`. Confirm nothing changes now — that's the proof the fix worked.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): A styled button component, three ways

Build one `Button.tsx` in `src/ch15/ex2/` that accepts `variant: "primary" | "secondary" | "danger"` and `disabled?: boolean`, and renders a `<button>` styled accordingly (background colour differs per variant; disabled is greyed out and has `cursor: not-allowed`).

1. First version: build it with **inline style** only, picking colours with a lookup object keyed by variant ([chapter 05](#)).
2. Second version, in a copy called `ButtonModule.tsx`: the same component, styled with a **CSS Module** instead — one class per variant (`.primary`, `.secondary`, `.danger`), combined with a small `cx` helper for the disabled state.

3. Render three of each variant, in both versions, side by side in `Ex2.tsx`.
4. In a comment: which version was easier to make hover states work for (`:hover` in CSS)? Try adding a hover effect to both and see for yourself before answering.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Find the bug in someone else's styles

Copy this into `src/ch15/ex3/ProfileCard.tsx` and `src/ch15/ex3/ProfileCard.module.css`, exactly as written — there are bugs on purpose:

```
// ProfileCard.tsx
import styles from './ProfileCard.module.css';

type ProfileCardProps = {
  name: string;
  role: string;
  isOnline: boolean;
};

function ProfileCard({ name, role, isOnline }: ProfileCardProps) {
  return (
    <div className="card">
      <h3 className={styles.name}><name></h3>
      <p className={styles.role}><role></p>
      <span className={isOnline ? styles.online : "offline"}>
        {isOnline ? "Online" : "Offline"}
      </span>
    </div>
  );
}

export default ProfileCard;
```

```

/* ProfileCard.module.css */
.card {
  border: 1px solid #ccc;
  padding: 16px;
  border-radius: 8px;
}

.name {
  font-size: 20px;
  font-weight: 600;
}

.role {
  color: grey;
}

.online {
  color: green;
  font-weight: 600;
}

.offline {
  color: grey;
}

```

1. Render two `<ProfileCard />` s in `Ex3.tsx` , one online and one offline. Look carefully: which styles are actually being applied, and which aren't? Use the Elements tab, not just your eyes — some of these bugs produce results that look almost right.
2. There are **three** separate mistakes, all from the "Common mistakes" list in the notes. Find and name each one before you fix it.
3. Fix all three.
4. In a comment, explain why TypeScript's red squiggles didn't catch any of them.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): Theming with CSS variables

In `src/ch15/ex4/` , build a small settings panel with a **light / dark / high-contrast** three-way toggle (radio buttons or three buttons — your choice), where the whole panel's colours change based on the selection.

1. Define your three themes as CSS custom properties under three different class names (or a `data-theme` attribute, if you'd rather — either is a legitimate real-world approach) in a plain, global `.css` file: `--bg` , `-text` , `--accent` , at minimum.

2. The panel itself, and everything inside it, uses `var(--bg)` etc. — no theme-specific logic in your `.tsx` file beyond choosing which class or attribute to apply.
3. React's only job is state (which theme is selected) and putting the right class name or attribute on the wrapping element.
4. Persist the chosen theme to `localStorage` so it survives a refresh, the same way you did with dark mode in [chapter 10](#).

In a comment: why does this approach mean your **component code** never needs an `if` or a lookup object for individual colours?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Build a reusable `cx` and a small design-system button

In `src/ch15/ex5/`, write `cx.ts` from scratch (don't copy it from the notes), typed properly:

```
export function cx(...classes: Array<string | false | null | undefined>): string {  
  // your implementation  
}
```

Then build `Button.tsx` and `Button.module.css`, a button component supporting **all** of these at once, combinable in any mix:

- `variant: "primary" | "secondary" | "outline"` — always exactly one.
- `size: "small" | "medium" | "large"` — always exactly one.
- `fullWidth?: boolean` — an extra class when true, nothing when false or omitted.
- `disabled?: boolean` — greys it out and disables real clicking.

Requirements:

1. Every combination should be reachable with a **single** `className={cx(...)}` call — no nested ternaries, no nested nested classNames.
2. Write a small test page in `Ex5.tsx` rendering at least six different combinations, including the "everything at once" case: `secondary`, `large`, `fullWidth`, `disabled`.
3. Prove your `cx` function works on its own: call it directly with a mix of strings, `false`, `undefined`, and `null`, log the result, and confirm falsy values are dropped and nothing extra (double spaces, leading/trailing spaces) sneaks in.
4. In a comment, explain what would happen — concretely, with an example — if `cx` used `.join(" ")` **without** filtering falsy values first.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

16 Tailwind CSS

What is it?

Tailwind CSS is a library that gives you hundreds of small, ready-made CSS classes, applied straight in your JSX, instead of writing your own CSS rules:

```
<button className="rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white hover:bg-blue-700">
  Save
</button>
```

Every class does one small job — `px-4` is padding on the left and right, `bg-blue-600` is a background colour. You build up a design by combining a lot of them, right there on the element, rather than switching to a `.css` file.

This course is written for **Tailwind CSS v4**.

Why does it matter?

In [chapter 15](#), styling a button meant naming a class, writing its rule in a `.module.css` file, and switching between two files every time you tweaked anything. That's completely reasonable, and plenty of real teams work exactly that way. Tailwind is popular because it removes a specific kind of friction from that loop:

- **No naming things.** `.btn`, `.btn-primary`, `.card-header-title` — naming CSS classes is a genuinely annoying part of writing CSS, and Tailwind mostly makes it unnecessary.
- **No switching files.** The styles live right next to the markup they style, so you can see and change both at once.
- **A design system for free.** Tailwind's spacing, colours, and font sizes come from a fixed, consistent scale (`p-4` is always the same amount of padding everywhere), which quietly stops a common real-world problem: five slightly different shades of blue creeping into an app because nobody remembered the exact one from last time.
- **Nothing unused ships.** Tailwind scans your actual code and only includes the CSS for classes you really used, so your final stylesheet stays small even though the *tool* offers thousands of classes.

The trade-off is real too, and worth saying plainly: your JSX gets visually busier, and there's a vocabulary of class names to learn. Whether that trade is worth it is a genuine, ongoing debate in the React community, and different teams land differently. This chapter teaches you Tailwind well enough to use it and to read it in other people's code, so you can make that call for yourself.

Real-world example

Think about **furnishing a room with flat-pack furniture** versus having a carpenter build everything from scratch.

Custom CSS (chapter 15)	Tailwind
A carpenter builds a bespoke shelf to your exact spec	You buy a shelf unit, already made, in one of the standard sizes
You describe what you want in words (a <code>.css</code> rule)	You pick from a catalogue of ready pieces (utility classes)
Every shelf can be a slightly different height	Every shelf is one of a fixed set of standard heights
Takes longer, fits your space exactly	Faster, and everything matches because it's all from the same catalogue
You need carpentry skill to make changes	You need to know the catalogue to make changes

Neither is "wrong." A showroom furnished entirely from one catalogue looks coherent fast. A house built entirely bespoke can look however you want, at the cost of more decisions and more time.

How it works

Setting up Tailwind in your Vite project

In your practice app (or a fresh `npm create vite@latest` project, same as [chapter 01](#)):

```
npm install tailwindcss @tailwindcss/vite
```

Add the plugin to `vite.config.ts`:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

Replace the contents of `src/index.css` with one line:


```
@import "tailwindcss";
```

That's the whole setup — no separate config file needed to get started, no PostCSS to wire up by hand. Save, and every Tailwind class is now available anywhere in your app. (Older tutorials describe a longer setup with a `tailwind.config.js` and three separate `@tailwind` directives — that was Tailwind v3. v4 is deliberately much shorter.)

Utility classes: one job each

A **utility class** does exactly one thing. Learning Tailwind is mostly learning the naming pattern, not memorising hundreds of individual names:

Category	Examples	What they do
Spacing	<code>p-4</code> , <code>px-2</code> , <code>py-6</code> , <code>m-4</code> , <code>mt-2</code> , <code>gap-4</code>	padding, margin, gap — the number is a step on Tailwind's spacing scale, not a raw pixel value
Sizing	<code>w-full</code> , <code>h-screen</code> , <code>max-w-md</code>	width, height
Colour	<code>bg-blue-600</code> , <code>text-white</code> , <code>border-grey-300</code>	background, text, border colour — each colour has shades 50 (lightest) to 950 (darkest)
Typography	<code>text-sm</code> , <code>text-xl</code> , <code>font-bold</code> , <code>italic</code>	font size, weight, style
Layout	<code>flex</code> , <code>grid</code> , <code>block</code> , <code>hidden</code>	<code>display</code>
Flexbox	<code>items-center</code> , <code>justify-between</code> , <code>flex-col</code>	flex alignment and direction
Borders	<code>rounded-lg</code> , <code>border</code> , <code>border-2</code>	border radius and width
Effects	<code>shadow-md</code> , <code>opacity-50</code>	box-shadow, opacity

`p-4` means "padding, size 4 on the spacing scale" — which happens to be `1rem` (16px), because Tailwind's scale goes up in fixed steps. You're not picking pixel values freely; you're picking a position on a scale designed so a handful of values look consistent together. That constraint is doing real work: it's much harder to accidentally end up with thirteen almost-identical spacing values scattered across an app.

A worked example

Here's the café menu card from [chapter 03](#), styled entirely with utilities:

```
function MenuItem({ name, price }: { name: string; price: number }) {
  return (
    <div className="rounded-lg border border-grey-200 bg-white p-4 shadow-sm">
      <h2 className="text-lg font-semibold text-grey-900">{name}</h2>
      <p className="mt-1 text-grey-600">${price.toFixed(2)}</p>
    </div>
  );
}
```

Read it left to right: rounded corners, a light grey border, white background, padding, a small shadow. Then the heading: medium-large text, semi-bold, near-black. Then the price: a small top margin, grey. No `.css` file, no class names to invent, no switching windows to see what `.card` looks like — it's all right there.

Hover, focus, and other states

Prefix a utility with a state name and a colon to apply it only in that state:

```
<button className="bg-blue-600 hover:bg-blue-700 focus:outline-2 focus:outline-blue-400
  active:bg-blue-800">
  Save
</button>
```

- `hover:bg-blue-700` — a darker blue, only while hovered.
- `focus:outline-2` — a visible focus ring, only while focused (keyboard users need this — never remove focus outlines without replacing them).
- `active:bg-blue-800` — darker still, only while being clicked.

This is one of Tailwind's real conveniences: in [chapter 15](#), a `:hover` effect needed a real CSS rule, because inline styles can't express it. Here it's just another class, written right next to the state it's for.

Responsive design

Prefix with a breakpoint name to apply a utility only above that screen width:

```
<div className="flex flex-col gap-4 sm:flex-row md:gap-8 lg:max-w-4xl">
```

- No prefix: applies at every width (this is "mobile-first" — you style for small screens by default, then override for bigger ones).
- `sm:` (640px and up), `md:` (768px), `lg:` (1024px), `xl:` (1280px) — each one means "apply this class from this width upward."

So `flex-col sm:flex-row` reads as: stack vertically on small screens, switch to a row from `sm` upward. You'll use this constantly for layouts that need to behave differently on a phone versus a laptop.

Dark mode

Tailwind v4 supports dark mode with a `dark:` prefix, matching the operating system's preference by default:

```
<div className="bg-white text-grey-900 dark:bg-grey-900 dark:text-white">
```

To toggle it with a button instead of following the system setting (the way the to-do app's stretch goal did), tell Tailwind to key off a class on a parent element, in your CSS:

```
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

Then your React code just toggles a `dark` class on a top-level element —

`document.documentElement.classList.toggle("dark")`, or by controlling `className` on your app's root `<div>`, whichever fits how your app is built — exactly the same state-driven class toggling from [chapter 15](#).

Combining conditional classes with Tailwind

State-driven classes work exactly like they did in chapter 15 — Tailwind classes are still just strings, so your `cx` helper (or the `clsx` package) works unchanged:

```
function cx(...classes: (string | false | undefined][]): string {
  return classes.filter(Boolean).join(" ");
}

function Tab({ label, isActive, onClick }: TabProps) {
  return (
    <button
      onClick={onClick}
      className={cx(
        "rounded-md px-3 py-1.5 text-sm font-medium",
        isActive ? "bg-blue-600 text-white" : "text-grey-600 hover:bg-grey-100"
      )}
    >
      {label}
    </button>
  );
}
```

Notice the shared classes (`rounded-md px-3 py-1.5 ...`) are written once, and only the classes that genuinely differ between states are conditional. That's worth being deliberate about — see mistake 4 below.

Extracting a component instead of extracting a CSS class

Tailwind's official answer to "won't I repeat these class strings everywhere?" is: **make a component**, the same tool you already have. If five buttons across your app all need the same look, that's `<Button>`, not a `.btn` CSS class:

```
type ButtonProps = {
  children: ReactNode;
  onClick?: () => void;
};

function Button({ children, onClick }: ButtonProps) {
  return (
    <button
      onClick={onClick}
      className="rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white hover:bg-blue-700"
    >
      {children}
    </button>
  );
}
```

Now every `<Button>Save</Button>` gets the styling automatically, and there's exactly one place to change it. This is [chapter 04](#)'s `children` pattern again — Tailwind doesn't replace component composition, it works alongside it. It's a genuinely good default: reach for a real component before reaching for a shared CSS class.

Arbitrary values, for the rare exception

When the scale genuinely doesn't have what you need, square brackets let you specify an exact value:

```
<div className="top-[117px] w-[calc(100%-2rem)]">
```

Use this rarely. Reaching for it often is usually a sign you're fighting the scale instead of using it — see mistake 5.

Tailwind and CSS Modules aren't rivals

You can use both in the same app. Some teams use Tailwind for almost everything and drop into a `.module.css` file only for something genuinely awkward to express with utilities — a complex `@keyframes` animation, an unusual gradient, a unique layout. There's no rule against mixing them; use whichever fits the specific thing you're styling.

Common mistakes

1. Fighting the missing `.css` file out of habit

Looking for "where's the CSS for this button" and expecting a stylesheet is the most common adjustment. There often isn't one — the styling lives in the `className` string, on the element itself.

2. Forgetting `hover:`, `focus:`, or a breakpoint prefix and wondering why nothing changed

```
<button className="bg-blue-700"> // always this colour, not just on hover
```

If you meant "only in that state," the prefix has to be there: `hover:bg-blue-700`.

3. Typing class names Tailwind doesn't recognise, and getting silence

```
<div className="padding-4"> // typo – nothing happens, no error, no warning
```

Because utilities are just strings, a misspelt one is simply an unused class name; nothing on the page changes and nothing tells you why. If a Tailwind class doesn't seem to be applying, double-check the spelling character by character. Editor extensions for Tailwind (worth installing) autocomplete class names and catch this for you.

4. Repeating a long, identical class string across many elements

```
<button className="rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white hover:bg-blue-700">Save</button>
<button className="rounded-lg bg-blue-600 px-4 py-2 font-semibold text-white hover:bg-blue-700">Cancel</button>
```

Copy-pasted styling drifts apart over time as one gets edited and the other doesn't. Extract a `<Button>` component the moment you're duplicating a class string like this.

5. Reaching for arbitrary values instead of the nearest scale step

```
<div className="p-[17px]"> // ❌ why 17? use p-4 (16px) unless there's a real, specific reason
```

Nearly every real design doesn't actually need that exact pixel value — it needs "roughly this much space, consistent with everywhere else." Use the scale, and reach for square brackets only for a value that's genuinely fixed by something outside your control (a third-party widget's exact height, say).

6. Building your whole design entirely from memory instead of checking the docs

Tailwind has a *lot* of utilities, and nobody has them all memorised. Keep the official docs open while you work, the same way you'd keep MDN open for a CSS property you don't use often.

Quick recap

- Tailwind gives you small, single-purpose utility classes — `p-4`, `bg-blue-600`, `flex` — applied directly in `className`, instead of writing your own CSS rules.
 - Set up with `npm install tailwindcss @tailwindcss/vite`, the plugin added to `vite.config.ts`, and one line, `@import "tailwindcss";`, in your CSS. Tailwind v4's setup is deliberately minimal.
 - State prefixes (`hover:`, `focus:`, `active:`) and breakpoint prefixes (`sm:`, `md:`, `lg:`) apply a class only under that condition. `dark:` handles dark mode.
 - Combine Tailwind classes with your `cx` helper exactly as you did with CSS Modules classes in [chapter 15](#).
 - Don't repeat long class strings — extract a **component**, the same tool you already have, rather than inventing a shared CSS class.
 - Prefer the spacing/colour **scale** over arbitrary bracketed values; the scale is what keeps a whole app looking consistent.
 - Tailwind and CSS Modules can live in the same app; use whichever fits the specific thing you're styling.
-

Next: try the [exercises](#), then move on to [17 Effects](#).

Exercises

How to do these:

- These need Tailwind installed. Either add it to your existing `playground` app (`npm install tailwindcss @tailwindcss/vite`, update `vite.config.ts`, and replace `src/index.css` 's contents with `@import "tailwindcss";`), or create a fresh app for this chapter, same as [chapter 01](#). Either way, one folder per exercise inside `src/ch16/`.
 - Keep the [Tailwind docs](#) open in a tab. Nobody does this chapter from memory, and neither should you.
 - An exercise is done when the page looks right, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and there's no leftover `.css` file with rules that duplicate what a utility class already does.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Convert a card, class by class

Here's a card styled with plain CSS, the [chapter 15](#) way:

```

.card {
  border: 1px solid #d1d5db;
  border-radius: 12px;
  padding: 20px;
  background: white;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
  max-width: 320px;
}
.title {
  font-size: 20px;
  font-weight: 700;
  color: #111827;
}
.subtitle {
  font-size: 14px;
  color: #6b7280;
  margin-top: 4px;
}
.button {
  margin-top: 12px;
  background: #2563eb;
  color: white;
  padding: 8px 16px;
  border-radius: 8px;
  font-weight: 600;
}

```

In `src/ch16/ex1/Ex1.tsx`, rebuild the exact same card using **only Tailwind utility classes**, no `.css` file at all. Match the colours and spacing as closely as the Tailwind scale allows — they won't be pixel-identical, and that's fine; note in a comment anywhere they genuinely can't match exactly.

Then add `hover:bg-blue-700` to the button, and confirm it does something a plain inline `style` could never have done on its own.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): A responsive nav bar

In `src/ch16/ex2/Ex2.tsx`, build a header with a logo on the left and three links on the right.

1. On small screens (below `sm`), stack the logo above the links, links centred, in a column.
2. From `sm` upward, they sit in a row: logo on the left, links on the right, vertically centred.
3. Give the links `hover:underline` and a colour change on hover.

4. Resize your browser window (or use DevTools' device toolbar) to actually see the breakpoint take effect, and write in a comment at what width it switches.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): A tab bar with conditional classes

In `src/ch16/ex3/Ex3.tsx`, build a three-tab bar (`Overview`, `Reviews`, `Shipping`) where the active tab is visually distinct (background colour, white text) and the others are plain text with a hover effect.

Requirements:

1. Use your `cx` helper from [chapter 15's exercise 5](#) (or write a fresh one) to combine shared classes with classes that depend on whether a tab is active.
2. Clicking a tab switches which one is active — this needs real state, from [chapter 08](#).
3. Show *something* below the tabs that changes with the selection, so it's clear the whole thing works, not just the tab styling.
4. In a comment, write the **one** class string your `cx` call produces for the active tab and for an inactive one, so you can see exactly what ends up in the DOM.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): Dark mode toggle

In `src/ch16/ex4/`, build a small page (a heading, two paragraphs, a card) that supports dark mode via a button, not just the system setting.

1. Set up the class-based dark mode variant, as described in the notes (`@custom-variant dark (&:where(.dark, .dark *));`).
2. A button toggles a `dark` class on a wrapping element, using state.
3. Every element that needs to look different in dark mode gets a `dark:` variant of its colour classes — background, text, and border, at minimum.
4. Persist the choice to `localStorage`, the same as the to-do app's stretch goal, so it survives a refresh.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A small design-system Button, the Tailwind way

Rebuild the [chapter 15, exercise 5](#) button component — but this time with Tailwind utilities instead of a CSS Module, and compare the two approaches honestly.

In `src/ch16/ex5/Button.tsx`, support the exact same API:

```
type ButtonProps = {
  children: ReactNode;
  variant: "primary" | "secondary" | "outline";
  size: "small" | "medium" | "large";
  fullWidth?: boolean;
  disabled?: boolean;
  onClick?: () => void;
};
```

Requirements:

1. Use **lookup objects** (from [chapter 05](#)) mapping `variant` to its class string, and `size` to its class string, rather than a long chain of ternaries inside the `className`.
2. Combine everything — the shared base classes, the variant classes, the size classes, and the conditional `fullWidth` / `disabled` classes — with your `cx` helper in one `className` call.
3. Render the same six-or-more combinations from the chapter 15 version in `Ex5.tsx`, including `disabled`, which should visibly grey the button out and use `disabled` Tailwind variants rather than a manually conditioned class.
4. Write a short, honest comparison in a comment: which version (the CSS Modules one from chapter 15, or this one) was faster to build? Which would be easier for someone else to change six months from now without breaking another variant by accident? There's no required answer — the point is forming and defending your own opinion, now that you've built the same thing both ways.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

17 Effects

What is it?

An **effect** is code that runs after React updates the screen, to synchronise your component with something **outside** React: the browser, a timer, `localStorage`, a server, a third-party widget.

```
import { useEffect } from "react";

useEffect(() => {
  document.title = `${count} unread messages`;
}, [count]);
```

`useEffect` is a hook, like `useState`. It takes a function to run, and a list of values that decide *when* to run it again.

Why does it matter?

Every hook so far has been about things fully inside React's world: a value your component remembers (`useState`), a description of the page (JSX). But components don't live in a vacuum. They need to talk to the tab title, to timers, to `localStorage`, to APIs — none of which React controls or knows about on its own.

You've already brushed up against this without the proper tool. In the [to-do app](#), milestone 8 saved to `localStorage` inside every event handler, one by one, with a note that this was "the long way round." That worked because every change to `tasks` happened to go through a handler. But what if something needed to react to `tasks` changing *no matter how* it changed — including a change that arrives from somewhere other than a click? Chasing every possible cause by hand doesn't scale. An effect says, once: "whenever `tasks` changes, for any reason, do this." That's the gap this chapter fills.

Real-world example

Think about a **smoke alarm**.

Smoke alarm	Effect
It doesn't care <i>why</i> there's smoke — a burnt toast, a candle, an actual fire	It doesn't care <i>why</i> a value changed — a click, a timer, data arriving
It reacts to the condition (smoke present), not to a specific event (someone pressing a "there's a fire" button)	It reacts to a dependency changing , not to a specific handler being called

Smoke alarm	Effect
Installed once, then works forever on its own	Set up once in your component; React re-runs it exactly when its dependencies say to
Needs its battery changed, or it stops working right — and needs disconnecting before you take it down	Needs a correct dependency list, and often a cleanup function before it's removed

You don't wire a smoke alarm to every possible fire-starting event by hand. You wire it to the condition, and let it fire whenever that condition becomes true. Effects work the same way.

How it works

The problem, without an effect

Say you want the browser tab's title to always show how many items are in a cart.

```
function CartPage({ items }: { items: CartItem[] }) {
  function handleAdd(item: CartItem) {
    setItems([...items, item]);
    document.title = `Cart (${items.length + 1})`; // 🤖 have to remember this everywhere
  }

  function handleRemove(id: string) {
    setItems(items.filter((item) => item.id !== id));
    document.title = `Cart (${items.length - 1})`; // and here
  }
  // ...and everywhere else items can change
}
```

Every place `items` can change needs its own copy of the same "update the title" logic, and it's easy to add a new way to change `items` later and forget the title update entirely. This is exactly the milestone-8 problem again, at a smaller scale.

The same problem, with `useEffect`

```
import { useEffect } from "react";

function CartPage({ items }: { items: CartItem[] }) {
  useEffect(() => {
    document.title = `Cart (${items.length})`;
  }, [items]);

  function handleAdd(item: CartItem) {
    setItems([...items, item]); // that's it – the effect handles the title
  }

  function handleRemove(id: string) {
    setItems(items.filter((item) => item.id !== id));
  }
}
```

Now `handleAdd` and `handleRemove` only do the one thing each is actually about: changing `items`. The effect runs automatically, after every render where `items` is different from last time, and nowhere else has to know or care that the title needs updating.

The shape of `useEffect`

```
useEffect(() => {
  // the effect: runs after the render commits
  return () => {
    // cleanup: runs before the next effect, and when the component is removed
  };
}, [dependency1, dependency2]);
```

- **The function** is your effect. It runs *after* React has updated the real DOM for this render — never during rendering itself, which keeps your component pure, exactly as [chapter 12](#) requires.
- **The dependency array** is the list of every reactive value — every prop, state variable, or value derived from them — that your effect's function actually reads. React compares this render's dependencies to last render's, and only re-runs the effect if at least one of them changed.
- **The optional returned function** is cleanup, which the next section covers properly.

The three shapes of the dependency array

This is the single most important thing to get right, so it's worth seeing all three side by side:

```

useEffect(() => {
  console.log("runs after every single render");
}); // no array at all

useEffect(() => {
  console.log("runs once, after the first render only");
}, []); // an empty array

useEffect(() => {
  console.log("runs after the first render, and again whenever `count` changes");
}, [count]); // an array with values in it

```

No array: runs after every render, no exceptions. Rare, and usually a sign you actually wanted one of the other two.

Empty array: runs exactly once, right after the component's first render — never again, however many times the component re-renders afterwards. Good for "set something up once": a one-time subscription, reading something from storage on load.

Array with values: runs after the first render, and then again on any later render where one of the listed values is different from what it was last render. This is the shape you'll write most often.

useEffect is not the same as useState

They're both hooks, both follow the two rules from [chapter 08](#) (top level only, never conditional), but they do fundamentally different jobs:

	useState	useEffect
Runs	During render	After the render commits
Purpose	Hold a value across renders	Do something to the outside world
Can it update state?	It <i>is</i> state	Yes, but carefully (see below)
Return value	[value, setter]	Nothing, or a cleanup function

Cleanup: undoing what the effect did

Some effects set something up that needs explicitly tearing down — a timer, a subscription, an event listener on `window`. If you don't undo it, you get duplicates piling up every time the effect re-runs.

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("tick");
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, []);
```

The returned function runs:

- **Before the effect runs again**, if its dependencies changed.
- **When the component is removed from the page** ("unmounted").

Think of the effect and its cleanup as a matched pair: "start the timer" pairs with "stop the timer." "Add an event listener" pairs with "remove it." "Open a connection" pairs with "close it." If your effect *does* something ongoing, ask yourself straight away what undoing it looks like, and write the cleanup at the same time you write the effect — not as an afterthought once you notice something's leaking.

Here's what goes wrong without it:

```
useEffect(() => {
  window.addEventListener("resize", handleResize);
  // ❌ no cleanup: every re-run adds ANOTHER listener, forever
}, [handleResize]);
```

Each time this effect re-runs, one more identical listener gets attached, and none of the old ones ever go away. `handleResize` ends up firing two, three, ten times per actual resize, and the only fix is a cleanup function that removes exactly the listener that was added.

Why the dependency array matters: two ways to get it wrong

Leave out a value you use, and the effect sees a stale one:

```
useEffect(() => {
  console.log(`You have ${count} items`);
}, []); // ❌ reads `count`, but never re-runs when it changes
```

This runs once, logs whatever `count` was on the very first render, and then never updates — a stale [closure](#), the exact same idea from chapter 12, just now living inside an effect instead of an event handler. React's linter (and the Vite starter includes one) will warn you about this: **every reactive value your effect's function reads must be in the array.**

Include an object or array that's rebuilt every render, and the effect never stops re-running:

```
useEffect(() => {
  console.log("options changed");
}, [{ limit: 10 }]); // ❌ a brand-new object, every single render
```

`{ limit: 10 }` is a new object every time the component renders — same content, different reference ([chapter 11](#)) — so React's "did the dependencies change?" check says yes, every time, forever. If you need an object or array as a dependency, either build it outside the component (if it never changes), store the *primitive values* that make it up as separate dependencies instead, or wrap it in `useMemo` ([chapter 27](#)).

Fetching data in an effect — and its trap

This is the most common reason to reach for `useEffect`, and it has a famous gotcha worth seeing once, deliberately:

```
function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((response) => response.json())
      .then((data) => setUser(data));
  }, [userId]);

  // ...
}
```

Looks right, and mostly works — until `userId` changes quickly. Say it goes from `"1"` to `"2"` before the request for `"1"` has come back. Now **two** requests are in flight, and whichever one happens to finish *last* wins, even if it's the stale one for `"1"`. The user briefly sees the wrong profile, or the screen flickers between the two. This is called a **race condition**.

The fix is a cleanup function that marks the *previous* request as no longer wanted:

```

useEffect(() => {
  let ignore = false;

  fetch(`/api/users/${userId}`)
    .then((response) => response.json())
    .then((data) => {
      if (!ignore) {
        setUser(data);
      }
    });

  return () => {
    ignore = true;
  };
}, [userId]);

```

When `userId` changes, React runs the cleanup for the *old* effect before running the new one. That sets `ignore = true` on the old request's closure, so when it eventually resolves, its `setUser` call is skipped. Only the most recent request is ever allowed to actually update state. [Chapter 18](#) builds on exactly this pattern, with loading and error states added.

`useEffect` calling `setState`: allowed, but think twice

Effects can update state — that's often the entire point, as in the fetch example above. But an effect that sets state **based only on other state or props**, with nothing external involved, is almost always a sign you didn't need an effect at all:

```

function Cart({ items }: { items: CartItem[] }) {
  const [total, setTotal] = useState(0);

  useEffect(() => {
    setTotal(items.reduce((sum, item) => sum + item.price, 0)); // ❌ don't do this
  }, [items]);

  // ...
}

```

This works, but it renders **twice** for every change to `items`: once with the old `total`, then the effect runs, calls `setTotal`, and triggers a second render to catch up. It's slower than it needs to be, and — much more importantly — it's solving a problem that [chapter 08](#) already solved:


```
function Cart({ items }: { items: CartItem[] }) {
  const total = items.reduce((sum, item) => sum + item.price, 0); // ✅ just calculate it
  // ...
}
```

No effect, no extra state, no extra render, and it genuinely can't go stale. **If you can calculate a value during render, calculate it during render.** This is worth its own heading, because it's the single most common way effects get misused.

When you do and don't need an effect

A rule of thumb that covers most cases: **if your code is reacting to a component rendering, rather than reacting to something outside React, you probably don't need an effect.**

Situation	Use an effect?
Deriving a value from props or state	No — calculate it during render (chapter 08)
Resetting state when a prop changes	Usually no — see below
Handling a specific user action, like a click	No — that's an event handler (chapter 07), which is exactly what it's built for
Fetching data based on a prop or piece of state	Yes
Subscribing to something outside React — <code>window</code> resize, a <code>WebSocket</code>	Yes
Syncing state to <code>localStorage</code>	Yes
Manually controlling a non-React widget (a map library, a chart)	Yes
Updating <code>document.title</code>	Yes

The "resetting state when a prop changes" row deserves a moment, because it's a common trap:

```
function ProfilePage({ userId }: { userId: string }) {
  const [comment, setComment] = useState("");

  useEffect(() => {
    setComment(""); // ❌ works, but there's a better tool
  }, [userId]);
  // ...
}
```

This "works," but it causes an extra render (component shows the *old* comment for an instant, then the effect fires and clears it) for something that has a cleaner fix: give the component a `key` tied to `userId` where it's used, so React treats a new user as an entirely new component instance and throws away the old state automatically ([chapter 06](#)'s key mechanism, used deliberately this time):

```
<ProfilePage key={userId} userId={userId} />
```

You don't need to master this trick yet — just recognise the shape "an effect that only resets state when a prop changes" as a smell worth a second look.

Effects and StrictMode

You'll notice your effects appear to run **twice** in development, with cleanup in between: effect → cleanup → effect. This is [chapter 12](#)'s StrictMode double-invocation again, extended to effects. It's deliberately checking that your cleanup function correctly undoes whatever your effect did — if it does, running "set up, tear down, set up" leaves you in exactly the same state as running it once, and you'll never notice. If it *doesn't* clean up properly, you'll see the symptom immediately (two intervals ticking, a duplicated listener), in development, instead of as a slow leak discovered in production. It only happens once, in dev; your built app runs the effect once.

Common mistakes

1. Leaving out a dependency your effect actually reads

```
useEffect(() => {  
  console.log(count);  
}, []); // ❌ stale forever after the first render
```

Every reactive value read inside the effect belongs in the array. Trust your editor's exhaustive-deps warning; it's almost always right.

2. A missing cleanup function for anything ongoing

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
}, []); // ❌ never cleared – piles up if this effect ever re-runs, and leaks on unmount
```

Timers, listeners, subscriptions: if you started it, return a function that stops it.

3. Fetching in an effect with no race-condition guard

Fast typing, fast prop changes, or a slow network can let an old response overwrite a newer one. Use the `ignore` flag pattern, or the built-in browser tool for cancelling requests outright, `AbortController` (covered in [chapter](#)

[18](#)).

4. Using an effect to calculate something you already have everything you need for

```
useEffect(() => {  
  setFullName(`${firstName} ${lastName}`);  
}, [firstName, lastName]); // ❌
```

Just write `const fullName = \`${firstName} ${lastName}\`;` during render. No effect needed at all.

5. An effect with an object or array literal in its dependency array

```
useEffect(() => { ... }, [{ id: userId }]); // ❌ a new object every render – runs every render
```

Depend on the primitive values themselves (`userId`), not a freshly built object that merely contains them.

6. Putting a `set` call directly in the render body instead of in an effect or handler

```
function Bad() {  
  const [count, setCount] = useState(0);  
  setCount(count + 1); // ❌ infinite loop – this isn't an effect, it just runs on every render  
  return <p>{count}</p>;  
}
```

This isn't really an "effects" mistake so much as a reminder of [chapter 08](#)'s rule: state only gets set from an event handler or from inside an effect's function — never from the plain body of a component.

7. Assuming the effect runs *during* the render, not after

Because effects run after the DOM is updated, anything that reads real layout information (an element's actual rendered width, say) works correctly inside an effect but would read stale or wrong values if you tried to read it during the render itself.

Quick recap

- `useEffect(fn, deps)` runs `fn` after React commits the render, whenever a value in `deps` has changed since the last render. An empty array means "once, after the first render." No array at all means "every render."
- Effects are for synchronising with things **outside React**: the browser, timers, storage, servers, third-party widgets. If you're only working with values already inside your component, you probably want to calculate them during render instead.

- Every reactive value the effect's function reads belongs in the dependency array — leaving one out gives you a stale closure.
 - Return a **cleanup function** from an effect that starts anything ongoing (a timer, a listener, a subscription). It runs before the effect re-runs, and when the component is removed.
 - Fetching inside an effect needs a guard against race conditions — a stale response from an earlier render arriving after a newer one.
 - **"Can I just calculate this during render?" is the single most useful question to ask before reaching for `useEffect`.**
 - StrictMode runs effect → cleanup → effect once in development, specifically to catch cleanup that doesn't properly undo the effect.
-

Next: try the [exercises](#), then move on to [18 Fetching Data](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch17/ex1/`, and so on.
 - **Watch the Console for effect and cleanup messages, not just the page.** Add a `console.log` at the top of every effect and every cleanup function while you build these — seeing the order they fire in is most of what this chapter teaches.
 - An exercise is done when the behaviour is correct, the Console shows the right sequence of messages (no duplicates that shouldn't be there, no missing cleanup), VS Code shows no red squiggles, and `npx tsc -b` prints nothing.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Three dependency arrays

In `src/ch17/ex1/Ex1.tsx`, build a component with two independent counters, `countA` and `countB`, each with its own `+1` button.

1. Add `useEffect(() => console.log("no array"))` — no dependency array at all.
2. Add `useEffect(() => console.log("empty array"), [])`.
3. Add `useEffect(() => console.log("watching A"), [countA])`.

Click each button several times, alternating between them, and answer in a comment:

4. Which effect(s) fire when you click A's button? Which fire when you click B's?
5. Does "empty array" ever fire more than once, no matter what you click?
6. Does "no array" fire on a click of **either** button? Why does that make it a poor choice for most real effects?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Document title sync

In `src/ch17/ex2/Ex2.tsx`, build a simple counter, and use an effect to keep the browser tab's title in sync:

Count: 3.

1. Click the button a few times and watch the actual browser tab, not just the page.
2. Add a second, completely unrelated piece of state — a text input for a name, say — with its own effect-free `<p>{name}</p>` display. Confirm typing in it does **not** cause the title effect to log again (add a `console.log` inside the effect to check).
3. Deliberately leave `count` out of the dependency array and observe the tab title get stuck. Put it back.
4. In a comment, explain why this is a textbook case for an effect rather than something to calculate directly in the JSX — what's actually "outside React" here that JSX can't reach on its own?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Build a leaking timer, then fix it

In `src/ch17/ex3/Ex3.tsx`:

```
function Ex3() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(true);

  useEffect(() => {
    setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);
  }, [isRunning]);

  return (
    <div>
      <p>{seconds} seconds</p>
      <button onClick={() => setIsRunning(!isRunning)}>
        {isRunning ? "Pause" : "Resume"}
      </button>
    </div>
  );
}
```

1. Run it. Click **Pause**, then **Resume**, then **Pause** again, several times, fairly quickly. Watch the number. What goes wrong, and why — connect it explicitly to the missing piece from the notes.
2. Fix it with a proper cleanup function.
3. After fixing it, click Pause/Resume rapidly again, ten or more times. Confirm the seconds count now goes up at a completely steady, correct rate regardless of how much you clicked.
4. There's a second bug: **Pause doesn't actually pause anything** — the timer never stops, it just keeps setting `seconds`, even while paused. Fix this too, using `isRunning` inside the effect (you may need to add it to the dependency array if it's not already effectively covered — think about what "effectively covered" even means here).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The race condition, caught in the act

This exercise makes a race condition happen on purpose, with a fake, slow "API," so you can watch the wrong data win.

In `src/ch17/ex4/`, start with `fakeApi.ts`:

```
export function fetchUser(id: string): Promise<{ id: string; name: string }> {
  // id "1" is deliberately slow, id "2" is fast – so requesting 1 then 2 quickly
  // means 2's response can arrive BEFORE 1's.
  const delay = id === "1" ? 2000 : 200;
  const name = id === "1" ? "Slow Sam" : "Fast Fiona";

  return new Promise((resolve) => {
    setTimeout(() => resolve({ id, name }), delay);
  });
}
```

Build `Ex4.tsx`: two buttons, **Load user 1** and **Load user 2**, and a component showing whichever user was last requested, using `fetchUser` inside a `useEffect` keyed on a `userId` piece of state.

1. Build the **broken** version first — no `ignore` flag, nothing guarding against a stale response.
2. Click **Load user 1**, then *immediately* click **Load user 2** (within the same second). Watch what the page ends up showing after both requests have had time to finish. Is it correct?
3. Fix it with the `ignore` flag pattern from the notes.
4. Repeat the same rapid double-click test. Confirm it now always ends up showing Fiona (the last one you actually asked for), no matter how the delays land.
5. Log a message from inside the cleanup function so you can see, in the Console, exactly when the stale request for Sam gets its response ignored.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): An effect that shouldn't be one

This one is about **removing** an effect, which is a skill in its own right.

In `src/ch17/ex5/Ex5.tsx`, start from this deliberately over-engineered component:

```

type Item = { id: string; name: string; price: number; quantity: number };

function ShoppingCart({ items }: { items: Item[] }) {
  const [itemCount, setItemCount] = useState(0);
  const [total, setTotal] = useState(0);
  const [isEmpty, setIsEmpty] = useState(true);
  const [mostExpensive, setMostExpensive] = useState<Item | null>(null);

  useEffect(() => {
    setItemCount(items.reduce((sum, item) => sum + item.quantity, 0));
  }, [items]);

  useEffect(() => {
    setTotal(items.reduce((sum, item) => sum + item.price * item.quantity, 0));
  }, [items]);

  useEffect(() => {
    setIsEmpty(items.length === 0);
  }, [items]);

  useEffect(() => {
    if (items.length === 0) {
      setMostExpensive(null);
      return;
    }
    const sorted = [...items].sort((a, b) => b.price - a.price);
    setMostExpensive(sorted[0]);
  }, [items]);

  return (
    <div>
      {isEmpty ? (
        <p>Your cart is empty.</p>
      ) : (
        <>
          <p>{itemCount} items — ${total.toFixed(2)}</p>
          <p>Most expensive: {mostExpensive?.name}</p>
        </>
      )}
    </div>
  );
}

```

1. Add a `console.log("ShoppingCart rendered")` at the top of the component, above every hook. Add a few items via a parent component's state and watch how many times `ShoppingCart` renders for **one** change to `items`. Write the number down, and explain in a comment why it's more than one.
2. Rewrite the whole component with **zero** `useEffect` calls and **zero** extra `useState` calls beyond what's genuinely needed — which, once you're done, should be none at all beyond the `items` prop itself. Every

value shown should be calculated directly during render.

3. Confirm the render count from question 1, repeated on your new version, is now exactly one per change to `items`.
4. In a closing comment, state the general rule this exercise was built to teach, in one or two sentences, using the words "render" and "effect."

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

18 Fetching Data

What is it?

Fetching data means calling a server for information and showing it on the page. In React, that's `fetch` (from JavaScript chapter 33) wrapped in an effect, with three pieces of state tracking what's happening: **loading**, **error**, and **data**.

```
const [recipes, setRecipes] = useState<Recipe[] | null>(null);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState<string | null>(null);
```

Nothing here is a new tool. This chapter is about combining `useState`, `useEffect`, and `fetch` into the one shape you'll write in almost every real app you build.

Why does it matter?

Every app you've built in this course used data that was already sitting in a file, ready the instant the page loaded. Real apps don't work that way. Data lives on a server, on the other side of a network request that takes time, and can fail. Three things become unavoidable the moment data comes from outside your app:

- **It isn't there yet.** For a moment — sometimes a long moment on a slow connection — there's nothing to show. The page needs to say so, not just look broken.
- **It might never arrive.** The network drops, the server is down, the search matches nothing. The page needs to say that too, clearly, not silently show stale or empty content.
- **TypeScript can't see the server.** `response.json()` returns `any` by default. The type safety you've relied on since [chapter 04](#) has to be re-established by hand, at the boundary where the data enters your app.

Get the shape of this right once, and you'll reuse it for the rest of your React career — every search box, every profile page, every dashboard. This is also the chapter where the [Recipe Finder project](#) starts to become possible.

Real-world example

Think about **ordering food for delivery**.

Ordering delivery	Fetching data
You place the order	The <code>fetch</code> call starts
"Your order is being prepared"	The loading state

Ordering delivery	Fetching data
The food arrives	<code>data</code> is set, loading ends
"Sorry, the restaurant is closed"	The error state
You cancel the order because you changed your mind	Cleanup cancels a request no longer needed
You can't eat before the food exists	You can't show data before it's arrived

A delivery app that shows nothing while you wait, with no "on its way" message, feels broken even when it isn't. The waiting and the possibility of failure are part of the experience, not an inconvenience to hide.

How it works

The three states, together

```
type Recipe = {
  id: string;
  name: string;
  thumbnail: string;
};

function RecipeSearch({ query }: { query: string }) {
  const [recipes, setRecipes] = useState<Recipe[] | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (query === "") {
      setRecipes(null);
      return;
    }

    let ignore = false;
    setIsLoading(true);
    setError(null);

    fetch(`https://www.themealdb.com/api/json/v1/1/search.php?s=${encodeURIComponent(query)}`)
      .then((response) => {
        if (!response.ok) {
          throw new Error(`Request failed: ${response.status}`);
        }
        return response.json();
      })
      .then((data) => {
        if (!ignore) {
          setRecipes(data.meals ?? []);
        }
      })
      .catch((err: unknown) => {
        if (!ignore) {
          setError(err instanceof Error ? err.message : "Something went wrong");
        }
      })
      .finally(() => {
        if (!ignore) {
          setIsLoading(false);
        }
      });
  });
}
```

```

    return () => {
      ignore = true;
    };
  }, [query]);

  if (isLoading) return <p>Searching...</p>;
  if (error) return <p>Something went wrong: {error}</p>;
  if (recipes === null) return <p>Type something to search.</p>;
  if (recipes.length === 0) return <p>No recipes found.</p>;

  return (
    <ul>
      {recipes.map((recipe) => (
        <li key={recipe.id}>{recipe.name}</li>
      ))}
    </ul>
  );
}

```

That's a lot in one place, so walk through it slowly. It's the [chapter 05](#) "four states" pattern, one effect, and the race-condition guard from [chapter 17](#), all working together.

Loading, error, data: why three, not one

It's tempting to reach for just one state — the data, `null` until it arrives — and skip the rest. Don't. Each of the three answers a question the others can't:

- **isLoading**: is a request in flight *right now*? Without it, there's no way to show a spinner or "Searching..." message.
- **error**: did the *last* request fail? Without it, a failed request just looks identical to "no results yet" — silently wrong, not obviously broken.
- **recipes**: what's the actual data, once it exists? `null` specifically means "nothing loaded yet," which is different from an empty array, which means "loaded, and there's genuinely nothing to show" ([chapter 05](#)'s empty-state lesson, back again).

Three separate booleans-and-values, not one combined value, because a request can be simultaneously "not loading" and "has neither data nor an error" (before the first search), and your JSX needs to tell all of these apart.

fetch doesn't reject on a 404 or a 500

This trips people up the first time. `fetch`'s promise only rejects on a genuine network failure — no connection, DNS failure, that kind of thing. A `404 Not Found` or a `500 Internal Server Error` still counts as "the request succeeded," as far as `fetch` is concerned; the server *did* answer, it just answered with bad news. You have to check `response.ok` yourself and throw if it's `false`:

```
if (!response.ok) {
  throw new Error(`Request failed: ${response.status}`);
}
```

Skip this, and a page that's genuinely down for maintenance will look like it "loaded successfully" with whatever garbage error page the server sent back as if it were real data.

Typing what comes back

`response.json()` returns `Promise<any>`, which means everything downstream of it is silently unchecked unless you do something about it. TypeScript chapter 13 covered this in depth; here's the version you'll use constantly in React.

Describe the shape you expect, based on the API's real response — check its documentation, or just log one response and look:

```
type MealDbResponse = {
  meals: null | Array<{
    idMeal: string;
    strMeal: string;
    strMealThumb: string;
  }>;
};
```

Assert it at the one place data enters your app, and convert into your own, cleaner shape immediately:

```
.then((data: MealDbResponse) => {
  const found: Recipe[] = (data.meals ?? []).map((meal) => ({
    id: meal.idMeal,
    name: meal.strMeal,
    thumbnail: meal.strMealThumb,
  }));
  setRecipes(found);
})
```

Two things worth noticing:

- `data: MealDbResponse` **is a promise, not a proof**. TypeScript trusts you here; it can't actually check the server sent what you said it would. If the API changes a field name, your code will still compile and then break at runtime. This is exactly what [Zod](#), in Level 4, is for — checking, not just asserting.
- **Converting to your own `Recipe` type immediately** means the ugly, third-party field names (`idMeal`, `strMeal`) only ever exist in this one function. The rest of your app works with a clean shape you control. This is worth doing even when it feels like extra typing — it's the boundary between "their API" and "your

app," and it's much easier to change one function than every component that used raw API field names directly.

`.then()` chains vs `async / await`

Everything above works exactly the same written with `async / await` (JavaScript chapter 32), which most real React code prefers for its readability:

```
useEffect(() => {
  let ignore = false;

  async function loadRecipes() {
    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch(`https://www.themealdb.com/api/json/v1/1/search.php?s=${query}`);
      if (!response.ok) {
        throw new Error(`Request failed: ${response.status}`);
      }
      const data: MealDbResponse = await response.json();
      if (!ignore) {
        setRecipes((data.meals ?? []).map(toRecipe));
      }
    } catch (err) {
      if (!ignore) {
        setError(err instanceof Error ? err.message : "Something went wrong");
      }
    } finally {
      if (!ignore) {
        setIsLoading(false);
      }
    }
  }

  loadRecipes();

  return () => {
    ignore = true;
  };
}, [query]);
```

The effect's function itself can't be `async` directly — `useEffect(async () => { ... }, [])` is a type error, because an effect's function is meant to optionally *return* a cleanup function, and an `async` function always

returns a `Promise` instead. The fix is what you see above: define an `async` helper function inside the effect, and call it.

Both versions do exactly the same thing. `async / await` tends to read more like a normal sequence of steps, especially once there's a `try / catch / finally` involved; `.then()` chains can get harder to follow as they grow. Use whichever your team prefers — you'll meet both in real code, so it's worth being able to read either.

Cancelling a request outright: `AbortController`

The `ignore` flag from [chapter 17](#) stops a stale response from being *used*, but the request itself still finishes in the background — wasted work, especially for something big or slow. `AbortController` cancels the request itself:

```
useEffect(() => {
  const controller = new AbortController();

  async function loadRecipes() {
    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch(`.../search.php?s=${query}`, {
        signal: controller.signal,
      });
      const data: MealDbResponse = await response.json();
      setRecipes((data.meals ?? []).map(toRecipe));
    } catch (err) {
      if (err instanceof DOMException && err.name === "AbortError") {
        return; // the request was cancelled on purpose – not a real error
      }
      setError(err instanceof Error ? err.message : "Something went wrong");
    } finally {
      setIsLoading(false);
    }
  }

  loadRecipes();

  return () => {
    controller.abort();
  };
}, [query]);
```

Notice there's no `ignore` flag needed any more — once a request is aborted, it rejects with an `AbortError`, which the `catch` block deliberately recognises and ignores rather than showing as a real error to the user. `AbortController` is the more "correct" tool when the underlying request is genuinely wasteful to let finish (a

large search, an expensive server computation). The `ignore` flag is simpler and completely sufficient when the request itself is cheap. Both are legitimate; know both, and reach for whichever fits.

Refetching when something else changes

The dependency array is doing real work here: add another value your fetch depends on, and it refetches automatically whenever that changes too.

```
useEffect(() => {  
  // ...fetch using both `query` and `category`...  
}, [query, category]);
```

Change either `query` or `category`, and React reruns the effect: cleanup for the old request, then a fresh one with the new values. You don't write any code that says "and also refetch when the category changes" — it falls out of the dependency array automatically, which is exactly the value effects add over doing this by hand in an event handler.

Debouncing a search-as-you-type box

Firing a request on every keystroke is wasteful and can itself cause races. A small delay, "wait until typing pauses," is called **debouncing** (JavaScript chapter 34), and it composes naturally with the cleanup you already have:

```
useEffect(() => {  
  if (query === "") return;  
  
  const timeoutId = setTimeout(() => {  
    // ...the actual fetch goes here...  
  }, 400);  
  
  return () => {  
    clearTimeout(timeoutId);  
  };  
}, [query]);
```

Every keystroke schedules a fetch 400ms in the future, and cleanup cancels the *previous* one before it fires — so only the fetch after the last keystroke in a burst ever actually runs. It's the identical cleanup mechanism from the timer example in [chapter 17](#), just applied to a fetch instead of an interval.

An honest preview of what's coming

This whole chapter — `isLoading`, `error`, the `ignore` flag, `AbortController`, `debouncing` — is real, correct, useful code, and it's worth knowing properly, the way you're taught long division before a calculator. But writing all of it by hand, for every single piece of data your app needs, gets old fast, and it's easy to get subtly wrong under pressure. [TanStack Query](#), in Level 4, does everything in this chapter for you — loading states, error handling, race conditions, caching, refetching — in a few lines. Learning the manual version first means you'll actually understand what that library is doing, instead of it feeling like magic.

Common mistakes

1. Only tracking the data, with no loading or error state

The page looks broken while waiting, and looks broken (or silently wrong) when a request fails, with no way to tell the two apart from "hasn't searched yet." Track all three.

2. Not checking `response.ok`

```
const data = await response.json(); // "succeeds" even on a 404 or 500
```

`fetch` only rejects on network failure. Check `response.ok` and `throw` yourself for HTTP error statuses.

3. Calling `setState` after the component might have moved on

Without an `ignore` flag or `AbortController`, a slow, now-irrelevant response can still call `setState` and overwrite something newer. This is the race condition from [chapter 17](#) — it applies just as much here.

4. Trusting `response.json()` 's type completely

```
const data = await response.json();
console.log(data.recipes[0].name); // any, any, any – no checking, no warning if it's wrong
```

Give the response a real type the moment it arrives, and convert to your own shape immediately, so the untyped part of your code is as small as possible.

5. Making the effect's function itself `async`

```
useEffect(async () => { ... }, []);
// ❌ Type 'Promise<void>' is not assignable to type 'void | Destructor'.
```

Define an `async` function inside the effect and call it; don't make the effect's own callback `async`.

6. Forgetting `finally` for `isLoading`

```
try {
  setIsLoading(true);
  // ...
  setIsLoading(false); // ✗ never runs if the fetch throws
} catch { ... }
```

Set `isLoading` back to `false` in a `finally` block, so it happens whether the request succeeded or failed.

7. Refetching on every keystroke with no debounce

Works, but it's wasteful and can flood a slow server or hit a rate limit, especially combined with a race condition that isn't properly guarded. A short debounce delay costs very little code and meaningfully improves both.

Quick recap

- Fetching data in React is `fetch` (or `async / await`) inside a `useEffect`, tracked with **three** pieces of state: `isLoading`, `error`, and the data itself.
- `fetch` only rejects on a network failure — check `response.ok` yourself and `throw` for HTTP error statuses.
- Type the response at the boundary where it enters your app, and convert it into your own clean shape immediately. TypeScript trusts an assertion; it doesn't verify one.
- Guard against race conditions with the `ignore` flag from [chapter 17](#), or cancel the request outright with `AbortController`.
- Add every value your fetch depends on to the dependency array, and refetching on a change falls out automatically.
- Debouncing delays a fetch until typing pauses, reusing the same cleanup mechanism as a timer.
- This manual pattern is worth knowing well — and it's also exactly what [TanStack Query](#) exists to do for you later.

Next: try the [exercises](#), then move on to [19 Refs](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch18/ex1/`, and so on.
- Several of these use a real, free API: [TheMealDB](#), which needs no account and no API key for the endpoints used here (they use the shared test key `1`, built into the example URLs).

- Keep the **Network** tab open in DevTools (F12) alongside the Console. It's the best way to actually see requests firing, being cancelled, and coming back.
 - Turn on **Network throttling** (in the Network tab, usually a dropdown near the top, set to "Slow 3G" or similar) for exercises 3 and 4 — race conditions are hard to trigger on a fast connection and obvious on a slow one.
 - An exercise is done when loading, error, and success states all behave correctly, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Your first fetch, all three states

In `src/ch18/ex1/Ex1.tsx`, build a component that fetches a random meal from

`https://www.themealdb.com/api/json/v1/1/random.php` when a **Surprise me!** button is clicked, and shows its name and a thumbnail image.

1. Track `isLoading`, `error`, and `meal` as three separate pieces of state.
2. Type the response shape yourself, based on what the API actually sends back (log the raw JSON once and look, or check the API's docs).
3. Show `Loading...` while waiting, the meal once it arrives, and a clear error message if the request fails.
4. **Test the error path for real**, not just in your head: temporarily change the URL to something broken (a typo in the domain) and confirm your error message shows up correctly. Put the URL back afterwards.
5. Click the button five times quickly in a row. Does the page ever flash the *previous* meal while loading the next one? If it does, is that a bug, or is it fine? Write your answer as a comment, with your reasoning — there's a defensible case either way, and the point is thinking it through, not landing on one specific answer.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Search as you type, with debounce

In `src/ch18/ex2/Ex2.tsx`, build a search box that queries

`https://www.themealdb.com/api/json/v1/1/search.php?s=<your text>` and shows a list of matching meal names as you type.

1. First, build it **without** debouncing — a fetch on every keystroke. Open the Network tab and type a full word slowly. Count how many requests fired for one word.
2. Add a 400ms debounce, using the pattern from the notes. Type the same word again and count requests — it should now be close to one, not one per letter.

3. Handle the "no matches" case: TheMealDB returns `{ meals: null }`, not an empty array, when nothing matches — make sure your code treats that as an empty list, not a crash.
4. Clearing the box back to empty should show neither a loading state nor an error — just a neutral "type something to search" message.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Catch your own race condition

Build this with **network throttling turned on** (Slow 3G), so the race condition is easy to trigger reliably.

In `src/ch18/ex3/Ex3.tsx`, build a component with three buttons, one per category: **Beef**, **Chicken**, **Seafood**. Clicking one fetches `https://www.themealdb.com/api/json/v1/1/filter.php?c=<category>` and shows the resulting list of meal names.

1. Build it **without** any race-condition guard first.
2. With throttling on, click **Beef**, then click **Seafood** almost immediately after (within a second). Watch what ends up on screen once both requests have finished. Is it the category you *last* clicked, or something else? Try it a few times — race conditions aren't always reproducible on the first try.
3. Fix it with the `ignore` flag pattern.
4. Repeat the rapid double-click test several times. Confirm the page always ends up showing the last category you clicked, no matter how the timing lands.
5. As a second fix, rebuild it using `AbortController` instead of the `ignore` flag. Confirm it also works. In a comment, note one thing that's different about how the two approaches behave (hint: check the Network tab's status column for the earlier, superseded request in each version).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A tiny reusable fetch pattern, without the hook

Before custom hooks ([chapter 20](#)) give you a cleaner way to package this up, get comfortable with the repetition by writing it out fully, twice, by hand.

In `src/ch18/ex4/`, build **two** separate components that each independently fetch and display something from TheMealDB:

- `CategoryList`: fetches `https://www.themealdb.com/api/json/v1/1/categories.php` once, on mount, and shows the list of category names.
- `AreaList`: fetches `https://www.themealdb.com/api/json/v1/1/list.php?a=list` once, on mount, and shows the list of cuisine areas (Italian, Mexican, and so on).

Requirements:

1. Both components independently track their own `isLoading`, `error`, and `data` — no props passed between them, no shared state.
2. Both correctly type their own response shape (the two APIs return meaningfully different JSON shapes — look at each one).
3. Render both inside `Ex4.tsx`, side by side. Confirm in the Network tab that they fire **two separate, independent** requests, and that one being slow (throttle again if you like) doesn't hold up the other's loading state.
4. Once both are working, count the lines of near-identical `isLoading` / `error` / `useEffect` boilerplate across the two files. Write the number in a comment. You'll come back to this exact pair of components in [chapter 20](#) and cut that number down to almost nothing.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Fetch that depends on another fetch

A genuinely common real-world shape: you can't make the second request until the first one tells you what to ask for.

In `src/ch18/ex5/Ex5.tsx`, build a two-step picker:

1. A `<select>` populated from `https://www.themealdb.com/api/json/v1/1/list.php?c=list` (the list of categories), fetched once on mount.
2. Once a category is chosen, a **second** fetch to `https://www.themealdb.com/api/json/v1/1/filter.php?c=<chosen category>`, showing that category's meals as a list of names with thumbnails.
3. Changing the selected category must correctly cancel or ignore any in-flight request for the *previous* category (reuse your race-condition fix from Exercise 3).
4. The two fetches need **two separate** loading and error states — the category dropdown shouldn't show "Loading..." while a meal list is being fetched, and vice versa. Get this wrong on purpose first (share one `isLoading` between both) and see how confusing the UI becomes before you split them apart properly.
5. If someone picks a category, then picks "-- choose a category --" again (add that as the default option), the meal list should clear back to nothing — not keep showing the last category's results.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

19 Refs

What is it?

A **ref** is a component's memory for a value that doesn't need to redraw the page when it changes.

```
import { useRef } from "react";

const countRef = useRef(0);

countRef.current = countRef.current + 1; // changes it – no re-render
console.log(countRef.current);           // reads it
```

Refs also have a second, very common job: **reaching a real DOM element** directly, for the handful of things JSX can't express — focusing an input, measuring an element's size, playing a video.

```
const inputRef = useRef<HTMLInputElement>(null);

<input ref={inputRef} />;

inputRef.current?.focus();
```

Why does it matter?

[Chapter 08](#) gave you `useState`: a value that survives re-renders, and tells React to redraw when it changes. That second part — "tells React to redraw" — is exactly right for anything shown on the page. It's the wrong tool for anything that **isn't**.

You met this gap by accident back in [chapter 12](#)'s exercises: a render counter built with `useState` causes an infinite loop, because *counting renders* would itself trigger another render, forever. A ref sidesteps that completely — it remembers a value across renders, exactly like state, but changing it is invisible to React, so it never causes a re-render of its own.

The other half of this chapter — reaching real DOM elements — fills a gap you've felt since [chapter 09](#)'s stretch goal: making a newly created input receive focus automatically. `autoFocus` covered the simple case. Refs are the general tool underneath it, and the only way to do the many things JSX simply has no prop for.

Real-world example

Think about a **car's odometer** versus its **speedometer**.

Speedometer	Odometer
Shows the current speed, right now, on the dashboard	Quietly counts total miles, in the background
The driver looks at it constantly — it's meant to be seen	Almost nobody watches it tick over in real time
It's meant to be seen	It just needs to be <i>right</i> when you eventually check it
That's <code>useState</code> : redraw the dashboard when it changes	That's <code>useRef</code> : keep a number correct without redrawing anything

A dashboard that flashed and redrew itself every time a tenth of a mile ticked over on the odometer would be exhausting to look at. Some values genuinely don't need an audience for every change — they just need to be remembered correctly for later.

How it works

`useRef`

```
const ref = useRef(initialValue);
```

`useRef` gives back **one object**, always the same object across every render of this component, with exactly one property: `.current`. Read it, write it, however you like:

```
function ClickLogger() {
  const clickCount = useRef(0);

  function handleClick() {
    clickCount.current = clickCount.current + 1;
    console.log(`Clicked ${clickCount.current} times`);
  }

  return <button onClick={handleClick}>Click me (check the Console)</button>;
}
```

Click it ten times: the Console counts up correctly, `1` through `10`. But the button's own text never mentions the count, because nothing here ever told React to re-render. That's the whole trade a ref makes: it remembers reliably, but changing it is invisible to React.

Refs vs. state, side by side

	useState	useRef
Survives re-renders?	Yes	Yes
Changing it causes a re-render?	Yes	No
Read/write it during render?	Read yes, change no (chapter 08)	Reading or writing during render is unsafe — see below
Read/write it in a handler or effect?	Yes	Yes
Shows up in React DevTools	Yes, under hooks	No

The middle row is the one to remember. State is for **anything the user should see change**. A ref is for **anything your code needs to remember, but the page doesn't need to reflect**.

What belongs in a ref

A short, genuinely useful list:

- **A timer or interval id**, so you can clear it later — `setInterval` 's return value has no reason to trigger a re-render just by existing.
- **The previous value of a prop or state**, to compare against the current one.
- **A count of something that doesn't drive the UI** — how many times an effect has run, for debugging.
- **A flag for "is this the first render?"**, used to skip an effect's logic the first time.
- **A real DOM element**, covered in the rest of this chapter.
- **A mutable value inside an event handler that needs to survive between calls**, without needing to show anywhere.

If you're ever unsure, ask the question from [chapter 08](#), reframed: **does the page need to visibly change when this value changes?** Yes → state. No → a ref is very likely the better fit.

Don't read or write a ref during render

```
function Bad() {  
  const renderCount = useRef(0);  
  renderCount.current = renderCount.current + 1; // ⚠️ works, but it's fragile  
  return <p>Rendered {renderCount.current} times</p>;  
}
```

This *happens* to work in simple cases, and you actually built exactly this in [chapter 12's exercises](#) as a debugging tool. But it quietly breaks React's assumption that rendering is pure ([chapter 03](#), [chapter 12](#)): a value changing during render, invisibly, with no corresponding re-render to reflect it, is exactly the kind of side effect React's `<StrictMode>` double-render is designed to catch problems from. It's fine for an internal debugging counter you're not relying on for correctness. For anything that actually matters, set a ref's value inside an **event handler** or an **effect**, never in the plain body of the component.

Reaching a real DOM element

This is the other major use of refs, and it's what most people mean when they first hear the word "ref" in React. Pass a ref to an element's `ref` attribute, and after that element is actually on the page, `ref.current` points at the real DOM node:

```
import { useRef } from "react";

function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);

  function handleFocusClick() {
    inputRef.current?.focus();
  }

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocusClick}>Focus the box</button>
    </div>
  );
}
```

Two typing details worth being deliberate about:

- `useRef<HTMLInputElement>(null)` — the type argument says what kind of element this ref will eventually hold, and the starting value is `null`, because before the component's first render finishes, there's no element yet to point at.
- `inputRef.current?.focus()` — the `?.` matters. TypeScript correctly knows `.current` might still be `null` (if this ran before the element mounted, or after it's gone), so it makes you handle that case, same as any other optional value (TypeScript chapter 06).

React sets `ref.current` to the real element right after that render commits, and sets it back to `null` when the element is removed from the page. You never assign to `.current` yourself for a DOM ref — React manages it for you; you only ever *read* it.

What you'd genuinely need this for

Everything in this section is something JSX has no prop for, which is exactly why a ref, not a prop, is the right tool:

```
// Focus an input
inputRef.current?.focus();

// Select all the text in it
inputRef.current?.select();

// Scroll an element into view
sectionRef.current?.scrollIntoView({ behavior: "smooth" });

// Read an element's actual rendered size
const width = boxRef.current?.getBoundingClientRect().width;

// Play or pause a video
videoRef.current?.play();
```

None of these are things you *describe* the way you describe a heading's text or a button's colour. They're actions you *do*, to an element that already exists on the page — which is precisely the gap between React's declarative model ([chapter 01](#)) and the handful of imperative things the browser occasionally requires.

The "autofocus a new field" problem, properly

[Chapter 09's stretch goals](#) used `autoFocus` to put the cursor in a newly created task's edit box. That attribute is a real, simple, and completely valid tool — but it only fires once, when the element is first created, and it can't be triggered again later, say by a button click on an element that already exists. A ref can:

```
function EditableField() {
  const inputRef = useRef<HTMLInputElement>(null);
  const [isEditing, setIsEditing] = useState(false);

  function startEditing() {
    setIsEditing(true);
  }

  useEffect(() => {
    if (isEditing) {
      inputRef.current?.focus();
    }
  }, [isEditing]);

  return isEditing ? (
    <input ref={inputRef} />
  ) : (
    <button onClick={startEditing}>Edit</button>
  );
}
```

Notice the `focus` call lives in an **effect**, not directly inside `startEditing`. That's deliberate: when `startEditing` runs, `isEditing` is `false` and the `<input>` doesn't exist in the DOM yet — this render hasn't happened. The effect runs *after* the render that actually puts the `<input>` on the page, which is the first moment `inputRef.current` is genuinely something you can call `.focus()` on.

Refs don't reset when their component re-renders

Unlike a plain variable inside the component (which gets recreated every render, [chapter 12](#)), the ref object itself is the **same object** across every render of a given component instance — only `.current` changes, and only when you or React changes it. This is what makes it suitable for things like "remember the previous value":

```
function PriceTicker({ price }: { price: number }) {
  const previousPrice = useRef(price);

  const direction = price > previousPrice.current ? "up" : price < previousPrice.current ?
  "down" : "same";

  useEffect(() => {
    previousPrice.current = price;
  });

  return <p>${price} ({direction})</p>;
}
```

The effect (with no dependency array, so it runs after every render) updates the stored "previous" value *after* the current render has already used it for comparison — so during any given render, `previousPrice.current` genuinely holds last render's value, not this one's.

Refs on your own components (React 19)

Passing `ref` to your **own** component, not a built-in HTML tag, needs one extra step, because your component has to decide what that ref should actually point at.

Since **React 19**, this is straightforward: a function component can simply accept `ref` as an ordinary prop, alongside its other props, no special wrapper needed:

```
type FancyInputProps = {
  placeholder: string;
  ref?: React.Ref<HTMLInputElement>;
};

function FancyInput({ placeholder, ref }: FancyInputProps) {
  return <input ref={ref} placeholder={placeholder} className="fancy" />;
}
```

```
const inputRef = useRef<HTMLInputElement>(null);

<FancyInput ref={inputRef} placeholder="Search..." />;
```

`FancyInput` receives the ref like any other prop, and hands it straight to the real `<input>` inside. Whoever uses `<FancyInput ref={inputRef} />` ends up with `inputRef.current` pointing at the actual `<input>` element, right through the wrapper.

If you read older tutorials or work in an older codebase, you'll see this done with `forwardRef`:

```
const FancyInput = forwardRef<HTMLInputElement, { placeholder: string }>({
  placeholder, ref }) {
  return <input ref={ref} placeholder={placeholder} className="fancy" />;
};
```

That was necessary before React 19, when `ref` couldn't be received as a normal prop on a function component at all. It still works in React 19 for backwards compatibility, but new code doesn't need it — plain props are simpler to read and simpler to type. Recognise `forwardRef` on sight; write the plain-props version yourself.

Refs and TypeScript's built-in element types

Every real HTML element has a matching TypeScript type, and it's worth getting the specific one right, the same way [chapter 07](#) taught you to pick the right event type:

Element	Ref type
<code><input></code>	<code>HTMLInputElement</code>
<code><textarea></code>	<code>HTMLTextAreaElement</code>
<code><button></code>	<code>HTMLButtonElement</code>
<code><div></code> , <code><section></code> , most containers	<code>HTMLDivElement</code> , <code>HTMLElement</code>
<code><video></code>	<code>HTMLVideoElement</code>
<code><a></code>	<code>HTMLAnchorElement</code>

Get it wrong and TypeScript tells you plainly:

```
const divRef = useRef<HTMLDivElement>(null);

```

Common mistakes

1. Using a ref for something the page needs to display

```
const countRef = useRef(0);
countRef.current = countRef.current + 1;
return <p>{countRef.current}</p>; // never updates on screen after the first render
```

If it needs to be visible and reactive, it's state, not a ref. This is the single most common ref mistake — reaching for it out of habit once you've learned it exists, for something `useState` was always the right tool for.

2. Reading `.current` before the element exists

```
const inputRef = useRef<HTMLInputElement>(null);
inputRef.current.focus(); // ✖ Object is possibly 'null'.
```

Use `?.`, or check `if (inputRef.current) { ... }` first. TypeScript is right to insist — on the very first render, before anything has mounted, `.current` really is `null`.

3. Calling a DOM method directly inside the render body

```
function Bad() {  
  const ref = useRef<HTMLInputElement>(null);  
  ref.current?.focus(); // ✗ runs during render, before the element necessarily exists yet  
  return <input ref={ref} />;  
}
```

DOM refs are only safe to *use* after the render has committed — in an event handler, or in an effect.

4. Calling `useRef()` with no argument

```
const ref = useRef<string>();  
// ✗ Expected 1 arguments, but got 0.
```

In **React 19**, `useRef` requires a starting value. Plenty of older tutorials write `useRef<T>()` with empty brackets, because React 18's types allowed it. If what you want is "empty to begin with," say so explicitly: `useRef<string | undefined>(undefined)`.

5. Expecting a ref change to be visible in DevTools

Refs deliberately don't show up in the React DevTools **Components** tab the way state does, because React itself doesn't track them as part of a component's reactive data. If you need to see a value while debugging, `console.log` it, or temporarily use state instead.

6. Passing `ref` to a component that doesn't accept it

```
function FancyInput({ placeholder }: { placeholder: string }) { // no `ref` in props  
  return <input placeholder={placeholder} />;  
}  
  
<FancyInput ref={inputRef} placeholder="Search..." />;  
// ✗ ref does nothing – inputRef.current stays null
```

Add `ref?: React.Ref<HTMLInputElement>` to the component's props type, and pass it down to a real element inside, as shown above.

7. Using the wrong element type for the ref

Mismatching, say, `HTMLDivElement` against an actual `<input>` either produces a type error at the point you attach it, or — worse — compiles fine but gives you `.current` typed as the wrong element, so autocomplete offers methods that don't exist on the real node.

Quick recap

- `useRef(initialValue)` gives back a stable object with one property, `.current`, that survives across renders. **Changing it never causes a re-render.**
 - Use a ref for anything your code needs to remember that the page doesn't need to visibly reflect: timers, previous values, DOM elements. Use state for anything that should redraw the page.
 - Attach a ref to a real element with `ref={myRef}`; after that element mounts, `myRef.current` points at it, typed to the specific element (`HTMLInputElement`, and so on).
 - DOM refs are for the handful of imperative things JSX has no prop for — focus, selection, scrolling, measuring, media playback.
 - Don't read or write a ref in the plain body of a component during render; do it in an event handler or an effect.
 - Since **React 19**, a function component can accept `ref` as an ordinary prop and hand it to an inner element directly — `forwardRef` is no longer required for this, though you'll still see it in older code.
-

Next: try the [exercises](#), then move on to [20 Custom Hooks](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch19/ex1/`, and so on.
 - For every exercise, keep the **Components** tab open and confirm your ref genuinely does **not** appear under the component's hooks the way state does — that's the clearest way to feel the state/ref distinction from the inside.
 - An exercise is done when the behaviour is correct, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): State or ref?

In `src/ch19/ex1/Ex1.tsx`, build a single component with **five** separate pieces of tracked information, and for each one, decide — and justify in a comment — whether it should be `useState` or `useRef`:

1. Whether a modal dialog is currently open (shown on screen).
2. How many times the component has rendered (for a debug message only you'll ever look at, printed to the Console, never shown on the page).

3. The text currently typed into a search box (shown on screen, live).
4. The id returned by `setInterval`, so a later button can call `clearInterval` on it.
5. Whether the user has scrolled past 100px on their **very first** visit only, used once to decide whether to show a "welcome" banner, and never checked again after that first decision (so re-checking it on every scroll event would be pointless work).

Build all five for real — wire up a modal, a real search box, a real timer with a start/stop button, and so on — using whichever tool you picked for each. If any of your choices turn out to be wrong once you see it running (something that should update the screen doesn't, or something flickers that shouldn't), fix it and write down what tipped you off.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Focus management

In `src/ch19/ex2/Ex2.tsx`, build a small login form: an email field, a password field, a **Log in** button, and a validation message area.

1. When the page first loads, the email field should already have focus, with no click needed.
2. Clicking **Log in** with an empty email field should show `Email is required` **and** move focus straight back into the email field.
3. Add a **Clear** button that empties both fields and refocuses the email field.
4. Do all of this using refs and `.focus()` — not `autoFocus` for every case, since requirement 2 needs to happen in response to a click, which `autoFocus` can't do at all.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): A stopwatch, built entirely on what you know

In `src/ch19/ex3/Ex3.tsx`, build a stopwatch showing elapsed time as `MM:SS`, with **Start**, **Stop**, and **Reset** buttons.

Requirements:

1. The displayed time is **state** (it needs to redraw every second).
2. The interval's id is a **ref** (clearing it doesn't need to redraw anything).
3. **Stop** genuinely stops the interval — clicking **Start** again afterwards must not create a second, overlapping interval (test this the way you tested [chapter 17, exercise 3](#): click Start/Stop rapidly several times and confirm the time still counts up at exactly one second per second).
4. **Reset** sets the time back to zero and stops the timer if it was running.

5. Clean up the interval properly if the component itself is ever removed from the page while running — add a **Hide stopwatch** button elsewhere in your test page that unmounts it (conditionally render it), and confirm via a `console.log` in the cleanup that it really does get cleared rather than silently ticking away forever in the background.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): Read the DOM directly — a character-counting textarea

In `src/ch19/ex4/Ex4.tsx`, build a textarea for a "bio" field with a live-updating message underneath: `142 / 200` characters, turning red past 200.

Then add a **genuinely ref-only** feature that state alone can't give you as cleanly: an **auto-growing** textarea that expands its own height to fit its content, with no scrollbar, up to a maximum of 10 lines (after which a scrollbar appears).

Requirements:

1. The character count is derived from **state**, the standard controlled-input way from [chapter 09](#) — not read from the ref.
2. The auto-grow behaviour reads the textarea's **actual rendered scroll height** via a ref (`textareaRef.current.scrollHeight`) inside an effect that runs whenever the text changes, and sets the element's height directly via `style.height` on the real DOM node.
3. Confirm in a comment: why does the auto-grow measurement genuinely need a ref and a real DOM read, rather than something you could calculate purely from the string's length in state? (Think about what a very long single word versus many short words would do to the number of *lines*, versus the number of *characters*.)

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A generic "previous value" ref, and a real use for it

Build `usePrevious.ts` in `src/ch19/ex5/` — a small helper (you'll formalise this into a proper custom hook with the rules from [chapter 20](#); for now just get the mechanism working) that, given any value, returns what that value was on the **previous** render:

```
function usePrevious<T>(value: T): T | undefined {  
  // your implementation, using useRef and useEffect  
}
```

Then build `Ex5.tsx`: a component with a number input (controlled state, as usual) that shows:

```
Now: 42  
Before: 37  
Change: +5
```

Where "Before" comes from `usePrevious`, and "Change" is the difference — showing nothing (not `NaN`, not `0`) on the very first render, before there's a previous value to compare against.

Requirements:

1. `usePrevious` must be genuinely generic — prove it by also using it on a **string** piece of state elsewhere on the same page (a name field, say, showing "You changed your name from Maya to Priya" only at the instant it actually changes, and nothing the rest of the time).
2. Use TypeScript's generics syntax properly (TypeScript chapter 08) — no `any` anywhere in `usePrevious.ts`.
3. In a comment inside `usePrevious.ts`, explain precisely why the effect that updates the stored ref must have **no dependency array argument at all** (not `[]`, not `[value]`) for this to correctly reflect "last render's value" rather than something else. Walk through what would go wrong with each of the other two options.

Stuck? Hint 1 is in the [Hints section](#) at the end.

20 Custom Hooks

What is it?

A **custom hook** is a function you write yourself, whose name starts with `use`, that calls other hooks inside it. It packages up a piece of reusable *logic* — not JSX, not a component, just behaviour — so you can use it from any component.

```
function useToggle(initialValue: boolean) {
  const [value, setValue] = useState(initialValue);
  const toggle = () => setValue((prev) => !prev);
  return [value, toggle] as const;
}
```

```
function Sidebar() {
  const [isOpen, toggleOpen] = useToggle(false);
  return <button onClick={toggleOpen}>{isOpen ? "Close" : "Open"}</button>;
}
```

Nothing here is a new React feature. `useToggle` is a plain function that happens to call `useState` inside it. That's the entire trick, and this chapter is about using it well.

Why does it matter?

Go back to [chapter 18's exercise 4](#): two components, `CategoryList` and `AreaList`, each with its own `isLoading`, `error`, `data`, and a `useEffect` doing almost exactly the same fetch-and-track dance, differing only in the URL and the shape of what comes back. If you did that exercise, you counted the duplicated lines yourself.

Components can't solve this the way you'd hope. You can't "call" one component's `useState` from inside another — each component's hooks are private to it. But the *logic* — "track loading, error and data; fetch on mount; clean up if the component goes away" — has nothing to do with JSX at all. It's just behaviour, and behaviour is exactly what a plain function is for. A custom hook is the tool for sharing **stateful logic** between components, the same way a plain function shares logic between other plain functions (JavaScript chapter 09) — it just happens to be allowed to use `useState`, `useEffect`, and the rest along the way.

Real-world example

Think about a **recipe versus a finished dish**.

A recipe	A custom hook
"Preheat the oven, mix the batter, bake 20 minutes"	The logic: <code>useState</code> , <code>useEffect</code> , wired together
Any kitchen can follow it	Any component can call it
Different kitchens, same recipe, different cakes	Different components, same hook, completely separate state
The recipe itself doesn't bake anything	The hook itself doesn't render anything
You still need an oven (an actual kitchen) to use it	You still need a component to call it in

The crucial line is the third one, and it's worth sitting with before you write your first custom hook: the recipe is shared, but each kitchen bakes its **own** cake. A custom hook shares the *logic*, never the *state itself* — every component that calls `useToggle(false)` gets its own independent `isOpen` , exactly like two components calling `useState` directly never share a value.

How it works

Extracting your first custom hook

Start from ordinary, repeated code — this is genuinely how you'll discover most custom hooks in real projects, by noticing you've written the same `useState / useEffect` pairing twice:

```
function ProfileForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const isValid = name !== "" && email.includes("@");
  // ...
}

function SignupForm() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const isValid = username !== "" && password.length >= 8;
  // ...
}
```

Different fields, different rules — but the *shape* repeats: a value, a setter, a validity check. Here's a narrower, genuinely reusable slice of that: a single controlled field with its own validity.

```
function useField(initialValue: string, isValid: (value: string) => boolean) {
  const [value, setValue] = useState(initialValue);

  return {
    value,
    onChange: (event: React.ChangeEvent<HTMLInputElement>) => setValue(event.target.value),
    isValid: isValid(value),
  };
}
```

```
function ProfileForm() {
  const name = useField("", (v) => v !== "");
  const email = useField("", (v) => v.includes("@"));

  return (
    <form>
      <input value={name.value} onChange={name.onChange} />
      {!name.isValid && <p>Name is required.</p>}

      <input value={email.value} onChange={email.onChange} />
      {!email.isValid && <p>That doesn't look like an email.</p>}
    </form>
  );
}
```

Every part of this should look familiar — it's `useState`, a change handler, and a derived boolean, exactly as you've written dozens of times since [chapter 09](#). The only new idea is that it's now wrapped in a function you can call twice in one component, and each call gets its own, completely independent `value` and `isValid`.

The naming rule is not a suggestion

A function must be named starting with `use` for two real reasons, not just convention:

1. **React's linter uses the name to know which functions are allowed to call hooks.** `useField` calling `useState` is fine and checked properly; a function called `getField` doing the exact same thing would trip the rules-of-hooks linting, because as far as the linter's concerned, `getField` is just an ordinary function that has no business calling `useState`.
2. **Readers of your code need to know, at a glance, that calling this thing has React's rules attached to it** — the same two rules from [chapter 08](#): top level only, never inside a condition or a loop.

So the two rules of hooks apply to your **own** hooks exactly as they apply to `useState` and `useEffect`:

```
function Bad({ isOpen }: { isOpen: boolean }) {
  if (isOpen) {
    const toggled = useToggle(false); // ✗ same violation as chapter 08, just one level
    removed
  }
}
```

If `useField` calls `useState` and `useEffect` inside it, then calling `useField` is exactly as bound by the rules as calling those hooks directly — because that's genuinely what's happening under the hood.

A custom hook shares logic, never state

This is the single most important idea in the whole chapter, worth its own worked proof:

```
function useCounter(initial: number) {
  const [count, setCount] = useState(initial);
  return { count, increment: () => setCount((c) => c + 1) };
}

function App() {
  const counterA = useCounter(0);
  const counterB = useCounter(100);

  return (
    <div>
      <button onClick={counterA.increment}>A: {counterA.count}</button>
      <button onClick={counterB.increment}>B: {counterB.count}</button>
    </div>
  );
}
```

Click A's button ten times, and B still reads 100. `useCounter` isn't a single shared counter that both buttons plug into — it's a *template* for making a counter, and every call builds a fresh, private one, exactly the way two `<Counter />` components never mix up their numbers back in [chapter 08](#). If you ever want components to share the *same* live value rather than the same *kind* of value, that's not a custom hook's job — that's lifting state up ([chapter 13](#)), or Context ([chapter 22](#)).

`useLocalStorage` : fixing the to-do app's milestone 8, properly

Remember the honest note at the end of [chapter 10](#): saving to `localStorage` by hand, in every handler, was "the long way round," waiting for the right tool. Here it is.

```
function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    const saved = localStorage.getItem(key);
    if (saved === null) return initialValue;
    try {
      return JSON.parse(saved) as T;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

```
function App() {
  const [tasks, setTasks] = useLocalStorage<Task[]>("react-todo-tasks", []);
  // use setTasks exactly like a normal useState setter – saving happens automatically
}
```

Walk through what moved where, compared to the to-do app's manual version:

- **The lazy initializer** (`useState<T>(() => { ... })`) is exactly the `loadTasks` pattern from chapter 10, now living inside the hook instead of the component.
- **The `useEffect`** replaces the hand-written `updateTasks` helper. Now, *any* call to `setValue` — from anywhere, for any reason — automatically gets saved, because the effect reacts to `value` changing, not to a specific function being called. That closes exactly the gap [chapter 17](#) opened with: "what if a change to `tasks` arrives some way other than through a handler you remembered to route through `updateTasks`?"
- **The generic `<T>`** (TypeScript chapter 08) means this one hook works for a to-do list, a theme preference, a shopping cart — anything JSON can represent — with full type checking at every call site.
- **as `const`** on the returned tuple keeps TypeScript treating it as a fixed-length pair (`[T, Dispatch<SetStateAction<T>>]`) rather than a general array, the same reason `useState` itself returns a tuple.

This single hook is genuinely something you'll reuse in nearly every React project you ever build.

useFetch : fixing chapter 18's duplication

Back to the exercise this chapter promised to resolve. The `isLoading / error / data` shape from [chapter 18](#), extracted once:


```

function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let ignore = false;
    setIsLoading(true);
    setError(null);

    fetch(url)
      .then((response) => {
        if (!response.ok) throw new Error(`Request failed: ${response.status}`);
        return response.json();
      })
      .then((json: T) => {
        if (!ignore) setData(json);
      })
      .catch((err: unknown) => {
        if (!ignore) setError(err instanceof Error ? err.message : "Something went wrong");
      })
      .finally(() => {
        if (!ignore) setIsLoading(false);
      });

    return () => {
      ignore = true;
    };
  }, [url]);

  return { data, isLoading, error };
}

```

```
function CategoryList() {
  const { data, isLoading, error } = useFetch<CategoriesResponse>(
    "https://www.themealdb.com/api/json/v1/1/categories.php"
  );

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Something went wrong: {error}</p>;
  if (!data) return null;

  return (
    <ul>
      {data.categories.map((c) => (
        <li key={c.idCategory}>{c.strCategory}</li>
      ))}
    </ul>
  );
}
```

`AreaList` becomes just as short, calling `useFetch<AreasResponse>(...)` with its own URL and its own response type. The entire fetch-track-cancel dance is written **once**. Every component that needs data from somewhere just describes *what* it needs (a URL, a type) and gets back the three familiar pieces, ready to render.

Returning an object vs. a tuple

You've now seen both shapes. Which to pick isn't arbitrary:

```
// tuple – like useState
const [value, setValue] = useLocalStorage("theme", "light");

// object – like useFetch
const { data, isLoading, error } = useFetch(url);
```

Use a tuple (as `const` on an array) when there are exactly two values and one obvious order — a value and its setter, mirroring `useState` itself. Callers can name both however they like: `const [isDark, setIsDark] = useLocalStorage(...)`.

Use an object when there are three or more values, or the order isn't obvious. With `{ data, isLoading, error }`, the caller can destructure only what they need, in any order, and the names are fixed and self-explanatory rather than positional. Nobody has to remember "is loading the second thing or the third thing back from this hook?"

Composing custom hooks from other custom hooks

Hooks can call other hooks, including your own, and this is where the reuse compounds. Combine `useFetch` and `useLocalStorage` to cache a slow API response across visits:

```
function useCachedFetch<T>(key: string, url: string) {
  const [cached, setCached] = useLocalStorage<T | null>(key, null);
  const { data, isLoading, error } = useFetch<T>(url);

  useEffect(() => {
    if (data) setCached(data);
  }, [data]);

  return { data: data ?? cached, isLoading: isLoading && cached === null, error };
}
```

Neither `useFetch` nor `useLocalStorage` needed to change at all to make this possible — that's the real payoff of extracting logic into small, focused hooks instead of one giant one that tries to do everything at once.

What a custom hook is *not*

- **It's not a component.** It returns plain values — numbers, objects, functions — never JSX. If you find yourself wanting a hook to "return a `<div>`," what you actually want is a component that happens to *use* a custom hook internally.
- **It's not a global store.** Calling the same custom hook from five components gives you five independent instances of its state, not one shared one, exactly as proven above. Sharing genuinely live state across components is [chapter 13](#) or [chapter 22](#)'s job.
- **It's not required to call `useState` or `useEffect` at all.** `usePrevious` from [chapter 19's exercises](#) is a completely legitimate custom hook built only from `useRef` and `useEffect`. Any function that calls one or more hooks, and follows the naming rule, qualifies.

Common mistakes

1. Not naming it starting with `use`

```
function toggleValue(initial: boolean) { // ❌ the linter can't tell this calls hooks
  const [value, setValue] = useState(initial);
  ...
}
```

React's linting for the rules of hooks specifically looks for the `use` prefix to know which functions to check. Skip it, and mistakes inside your own hook go uncaught.

2. Expecting two calls to share state

```
const a = useCounter(0);
const b = useCounter(0);
// clicking a's button changing b's count would be a bug, not a feature
```

Every call creates its own, independent state. If you need genuinely shared state, that's lifting state up or Context, not a custom hook.

3. Calling a custom hook conditionally

```
if (userIsLoggedIn) {
  const profile = useFetch(profileUrl); // ❌ same violation as any other hook
}
```

The rules of hooks apply to your own hooks exactly as they apply to React's. Put the check *inside* the hook, or call the hook unconditionally and branch on its result instead.

4. Building one giant hook that does everything

A `useApp()` hook handling fetching, form state, theme, and routing all at once is hard to read, hard to test, and hard to reuse in part. Prefer several small, focused hooks — `useFetch`, `useLocalStorage`, `useField` — composed together, the way `useCachedFetch` combined two above.

5. Forgetting the custom hook still needs to follow every rule its inner hooks have

If `useFetch` uses `useEffect` with a dependency array, every rule about that dependency array from [chapter 17](#) — exhaustiveness, avoiding fresh object literals — still fully applies, just one layer down. Wrapping a hook in your own function doesn't exempt it from anything.

Quick recap

- A custom hook is a plain function, named starting with `use`, that calls other hooks inside it — a way to share **stateful logic**, never state itself, between components.
- Every call to a custom hook creates its own independent state, exactly like calling `useState` directly in two different components.
- Your own hooks follow the same two rules as React's built-in ones: top level only, never conditional, never inside a loop.
- Extract a custom hook when you notice the same combination of hooks repeated across components — `useLocalStorage` and `useFetch` are two you'll reuse in nearly every project from here on.
- Return a **tuple** (as `const`) for a value-and-setter pair mirroring `useState`; return an **object** when there are three or more values or the order isn't obvious.

- Custom hooks can call other custom hooks, and this composition is where the real reuse pays off.

Next: try the [exercises](#), then build the [21 Project: Recipe Finder](#), which leans on almost everything from this level.

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch20/ex1/`, and so on. Put each hook in its own file (`useToggle.ts`, `useLocalStorage.ts`, ...), separate from the components that use it.
- For every hook you write, **use it from at least two components at once**, and prove in a comment or a quick test that their state stays independent. That proof is the real exercise — a hook that "seems to work" from one call site can still be sharing state by accident.
- An exercise is done when the behaviour is correct, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): `useToggle`

Write `useToggle.ts` in `src/ch20/ex1/`:

```
function useToggle(initialValue: boolean): [boolean, () => void] {  
  // ...  
}
```

1. Use it to build a component with **three independent** toggles on one page: a light switch, a "show password" eye icon, and a collapsible FAQ answer. Each must use its own call to `useToggle`.
2. Toggle the light switch several times. Confirm the other two are completely unaffected.
3. Add a `set` function too, so callers can force a specific value rather than only flipping it — change the return type to `[boolean, () => void, (value: boolean) => void]`, and use the new `set` function to build a **Reset all** button that forces the light switch back to its default without touching the other two.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): `useLocalStorage`

Write `useLocalStorage.ts` in `src/ch20/ex2/`, following the notes closely but typing and testing it yourself rather than copying it verbatim.

1. Use it to build **two** independent pieces of persisted state on one page: a `"visited"` boolean (shown as "Welcome back!" vs "Hello, first-timer!") and a `"favouriteColour"` string (a text input, shown live as the page's border colour).
2. Refresh the page. Both should come back exactly as you left them.
3. Open DevTools → Application → Local Storage, and confirm there are genuinely **two separate keys**, each holding correctly-formed JSON — not one shared blob.
4. Deliberately corrupt one of the two stored values by hand in DevTools (edit it to `not valid json`), then refresh. Confirm your hook falls back to the initial value for *that* key without crashing the whole page or affecting the *other* key.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Finish the job from chapter 18

Go back to [chapter 18, exercise 4](#) — the `CategoryList` and `AreaList` components you counted the duplicated lines in. If you didn't do that exercise, quickly rebuild a simplified version of both first (each independently fetching and listing something from TheMealDB), so you have the "before" to compare against.

In `src/ch20/ex3/`:

1. Write `useFetch.ts`, generic, following the notes.
2. Rewrite both `CategoryList` and `AreaList` to use it. Each should shrink to a handful of lines: a call to `useFetch<...>(url)`, and JSX for the three states.
3. Type each component's call to `useFetch` with its own specific response shape — `useFetch<CategoriesResponse>(...)` and `useFetch<AreasResponse>(...)` — and confirm TypeScript correctly infers the shape of `data` differently at each call site.
4. Add a **third**, brand-new component, `IngredientList`, fetching `https://www.themealdb.com/api/json/v1/1/list.php?i=list`, using `useFetch` from the very first line you write it — no duplicated boilerplate at all this time.
5. In a closing comment, compare the total line count of all three components combined against your chapter 18 count for just the first two. Say what you notice.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): `useDebounce`

Write `useDebounce.ts` in `src/ch20/ex4/` — a hook that takes a fast-changing value and a delay, and returns a version of that value that only updates once changes have paused for that long:

```
function useDebounce<T>(value: T, delayMs: number): T {  
  // ...  
}
```

1. Use it to rebuild the debounced search box from [chapter 18, exercise 2](#) — but this time, the component's own code should have **no `setTimeout` or `clearTimeout` anywhere in it**. It calls `useDebounce(query, 400)` and fetches using the debounced value, full stop.
2. Prove it's genuinely generic: use the same hook on a **number** slider elsewhere on the page, showing both the live value (`Dragging: 73`) and the debounced one (`Settled: 73` , updating only once you stop dragging for 400ms).
3. Type the whole thing properly — no `any` .

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): `useOnlineStatus` and `useWindowWidth` , composed into a real feature

These two hooks subscribe to something genuinely outside React — the browser's own `online / offline` events, and the window's resize event — which means real listeners, real cleanup, and no fetch involved at all.

In `src/ch20/ex5/` :

`useOnlineStatus.ts` : returns a `boolean` , `true` when the browser is online. Uses `navigator.online` for the initial value, and listens for the browser's `online` and `offline` events on `window` to keep it updated, with proper cleanup.

`useWindowWidth.ts` : returns the current `window.innerWidth` as a `number` , updated live as the window is resized, listening for the `resize` event on `window` , with proper cleanup.

Then build `Ex5.tsx` , a small dashboard combining both:

1. A banner reading `You're offline – some features may not work.` shown only when `useOnlineStatus()` is `false` . Test it for real: open DevTools → Network tab → set throttling to **Offline**, and confirm the banner appears without a page refresh. Switch back to **No throttling** and confirm it disappears, live.
2. A layout that switches from a single column to a two-column grid once `useWindowWidth()` crosses 640px, using inline styles driven directly by the hook's return value (not Tailwind or CSS Modules — the point here is the hook, not the styling tool). Resize your actual browser window and watch it happen without a refresh.

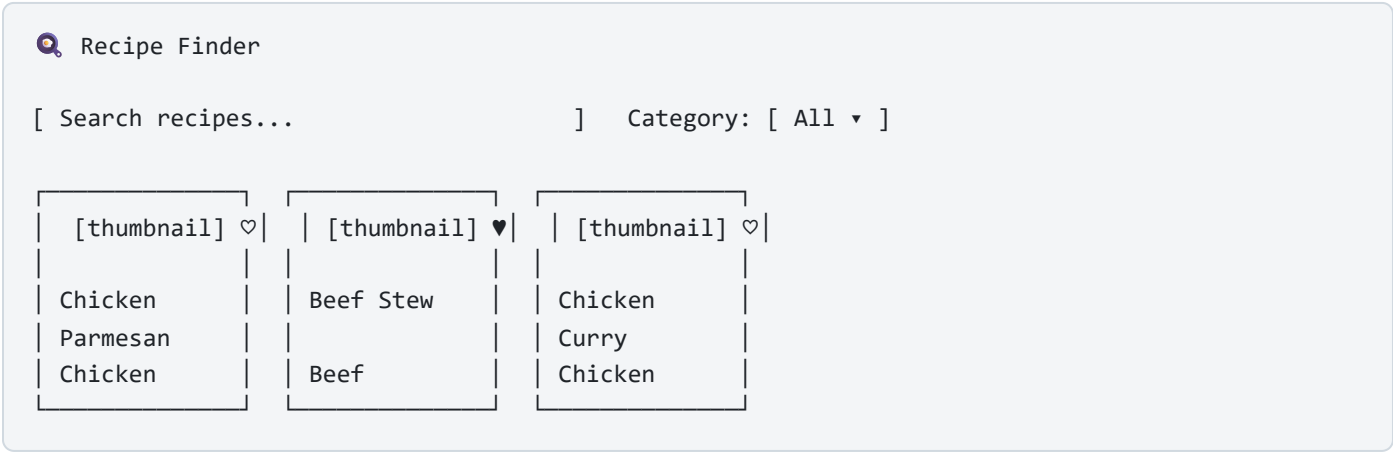
- Both hooks must properly remove their listeners — prove it by rendering `Ex5` conditionally from a parent (a **Hide dashboard** button that unmounts it), resizing the window or toggling offline mode *after* hiding it, and confirming (via a `console.log` in each cleanup) that nothing fires for an unmounted component.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

21 Project: Recipe Finder

What you'll build

An app that searches a live recipe API, shows the results as a grid of photo cards, opens full instructions when you click one, and remembers your favourites between visits.



By the end, your app will:

- search a live recipe API as you type, without flooding it with requests
- show clear loading, error, and empty states
- show full recipe details — photo, ingredients, instructions — when you click a card
- let you favourite recipes, saved so they survive a refresh
- show all your favourites, even ones from searches you've long since cleared
- filter results by category

This is the Level 2 project. It uses almost everything from this level:

Chapter	Where you'll use it
11 Updating Objects and Arrays	Adding and removing favourites without mutating state
13 Lifting State Up	The search box, the grid, and the detail view sharing state
14 Thinking in React	Planning the component tree and state before writing code
15 or 16 Styling	The finished stylesheet, or your own if you'd rather
17 Effects	Fetching, and syncing favourites to <code>localStorage</code>
18 Fetching Data	Loading and error states, for two different kinds of request
19 Refs	Focusing the search box when the app loads

Chapter	Where you'll use it
20 Custom Hooks	<code>useLocalStorage</code> and <code>useDebounce</code> , reused; two new hooks of your own

The API is [TheMealDB](#), a free recipe database. It needs no account and no real API key — everything here uses the shared test key `1`, which exists for exactly this kind of practice project.

Getting started

This project gets its own app, same as the to-do app did.

1. Open a terminal in this `React` folder and run `npm create vite@latest`, answering as in [chapter 01](#), naming the project `recipe-finder`.
2. Copy this chapter's `starter/src/` files over `recipe-finder/src/`.
3. Delete `recipe-finder/src/App.css` and `recipe-finder/src/assets/`.
4. `cd recipe-finder`, then `npm run dev`.

You should see the title and nothing else. Keep a second terminal on `npx tsc -b`, and keep DevTools open — you'll live in the **Network** tab this chapter, watching requests fire, get cancelled, and come back.

The CSS is finished for you. The class names it expects are in `starter/README.md`.

The big idea: one direction of travel

The to-do app in [chapter 10](#) owned all its data. Nothing arrived from outside; nothing could fail. This app is the opposite — almost everything on screen came from a server that's slow, occasionally down, and completely outside your control.

That changes the shape of the app. Data now flows in one direction through three layers, and keeping them separate is what makes eight milestones manageable instead of tangled:

```

TheMealDB's JSON    →  api.ts      →  hooks      →  components
strMeal, idMeal, ...  Recipe[]    + loading    JSX
(ugly, theirs)       (clean, yours) + error

```

Layer	Its one job	Never does
<code>api.ts</code>	Build URLs, call <code>fetch</code> , convert their shapes into yours	Know about React
Hooks	Track loading, error, and data over time	Know about JSX
Components	Turn state into JSX	Know that <code>strMeal</code> exists

That third column is the useful one. If you ever find yourself writing `strMeal` inside a component, a layer boundary has leaked, and the fix belongs one layer down — not in the component.

Milestone 1: Types, and a search box that focuses itself

Goal: the shape of your data, and a form that does nothing yet.

Make `src/types.ts`:

```
export type Recipe = {
  id: string;
  name: string;
  thumbnail: string;
  category: string;
};

export type RecipeDetails = Recipe & {
  instructions: string;
  ingredients: string[];
};
```

`RecipeDetails` is a `Recipe` **and** more, joined with `&` (TypeScript chapter 04). Anywhere that accepts a `Recipe` will happily accept a `RecipeDetails`, which is exactly what you want — the detail view can reuse the card's data without any conversion.

Now `src/SearchForm.tsx` — controlled, no fetching, [chapter 09](#)'s pattern exactly:

```
type SearchFormProps = {
  query: string;
  onQueryChange: (query: string) => void;
};
```

Inside it, an `<input className="search-input">` with a real `<label className="visually-hidden">`, and a submit button.

Then the one bit of [chapter 19](#) this project needs: **focus the box when the app loads**, so you can start typing immediately without reaching for the mouse.

```
const inputRef = useRef<HTMLInputElement>(null);

useEffect(() => {
  inputRef.current?.focus();
}, []);
```

In `App.tsx`, hold `query` in state, render `<SearchForm />`, and temporarily show `<p>Searching for: {query}</p>` so you can see it working.

Check it: the cursor is already in the box when the page loads. Typing updates the paragraph live. The Network tab shows no requests at all.

Milestone 2: Search on submit

Goal: press Enter, get real recipes — or a clear error.

Make `src/api.ts`. **This file is the only place in your app that knows what TheMealDB's JSON looks like:**

```
import type { Recipe } from './types.ts';

const BASE = "https://www.themealdb.com/api/json/v1/1";

type MealDbMeal = {
  idMeal: string;
  strMeal: string;
  strMealThumb: string;
  strCategory: string;
};

type MealDbSearchResponse = {
  meals: MealDbMeal[] | null;
};

function toRecipe(meal: MealDbMeal): Recipe {
  return {
    id: meal.idMeal,
    name: meal.strMeal,
    thumbnail: meal.strMealThumb,
    category: meal.strCategory,
  };
}

export async function searchRecipes(query: string): Promise<Recipe[]> {
  const response = await fetch(`${BASE}/search.php?s=${encodeURIComponent(query)}`);
  if (!response.ok) {
    throw new Error(`Search failed: ${response.status}`);
  }
  const data: MealDbSearchResponse = await response.json();
  return (data.meals ?? []).map(toRecipe);
}
```

Three details worth pausing on:

- `encodeURIComponent` turns a query like `chicken & rice` into something safe to put in a URL. Skip it and any search containing `&`, `?` or `#` silently breaks.
- `response.ok`, from [chapter 18](#) — `fetch` does *not* reject on a 404 or a 500, so you check and throw yourself.
- `data.meals ?? []` — TheMealDB returns `{ "meals": null }`, not an empty array, when nothing matches. `??` turns that into an empty array right here, so nothing downstream ever has to think about it.

Now in `App.tsx`, add a second piece of state, `submittedQuery`, set by the form's `onSubmit`. It's deliberately separate from the live `query` the input shows — you don't want to fetch on every keystroke yet, and separating "what's typed" from "what's been asked for" is what makes that possible.

Then the three-state pattern from [chapter 18](#), written out longhand for now:

```
const [recipes, setRecipes] = useState<Recipe[] | null>(null);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  if (submittedQuery === "") {
    setRecipes(null);
    return;
  }

  let ignore = false;
  setIsLoading(true);
  setError(null);

  searchRecipes(submittedQuery)
    .then((found) => {
      if (!ignore) setRecipes(found);
    })
    .catch((err: unknown) => {
      if (!ignore) setError(err instanceof Error ? err.message : "Something went wrong");
    })
    .finally(() => {
      if (!ignore) setIsLoading(false);
    });

  return () => {
    ignore = true;
  };
}, [submittedQuery]);
```

Render the four states as plain text for now — loading, error, "type something to search" (`recipes === null`), and a bare `<ul>` of names.

Check it: search `chicken` and see names appear. Search `zzzznotafood` and see an empty-results message, **not** an error — an empty result is a successful request. Temporarily break the domain in `BASE` to confirm the error path shows your message, then fix it.

Milestone 3: The results grid

Goal: photo cards, not a bare list.

Make `src/RecipeCard.tsx`. Read this structure carefully, because it's more deliberate than it first looks:

```
type RecipeCardProps = {
  recipe: Recipe;
  onSelect: (id: string) => void;
};

function RecipeCard({ recipe, onSelect }: RecipeCardProps) {
  return (
    <article className="recipe-card">
      <button className="recipe-card-main" onClick={() => onSelect(recipe.id)}>
        <img className="recipe-thumbnail" src={recipe.thumbnail} alt="" />
        <h3 className="recipe-name">{recipe.name}</h3>
        <p className="recipe-category">{recipe.category}</p>
      </button>
    </article>
  );
}
```

Why an `<article>` wrapping a `<button>`, rather than just making the whole card one big `<button>`?

Because in milestone 7 you're going to add a **favourite** button in the card's top corner. **HTML does not allow a button inside another button** — browsers handle it unpredictably, and the inner one may not be clickable at all. Building the card as a container with buttons *inside* it, as siblings, means that second button drops in later with no restructuring and no conflict between the two click targets.

That's worth naming as a general habit: when two clickable things overlap, **arrange them so they don't nest** rather than reaching for `stopPropagation` to untangle them afterwards. Structure beats patching.

The `alt=""` is also deliberate: the recipe's name is right there in the text, so the image adds nothing for a screen reader and an empty `alt` correctly marks it as decorative. An `alt={recipe.name}` here would just make the name get read out twice.

Now `src/RecipeGrid.tsx`, taking `recipes: readonly Recipe[]`, mapping to cards inside a `<div className="results-grid">` (mind the `key`, [chapter 06](#)), and showing `No recipes found.` when the array is empty.

Check it: a real photo grid. Resize the browser — the CSS handles the responsive columns. Tab through with the keyboard: each card should be reachable and openable with Enter, for free, because it's a real `<button>`.

Milestone 4: Recipe details

Goal: click a card, read the actual recipe.

Add to `api.ts`. The details endpoint returns ingredients in a genuinely awkward shape — twenty numbered fields, `strIngredient1` through `strIngredient20`, most of them empty strings or `null`:

```

type MealDbFullMeal = MealDbMeal & {
  strInstructions: string;
  [key: string]: string | null;
};

function extractIngredients(meal: MealDbFullMeal): string[] {
  const ingredients: string[] = [];

  for (let i = 1; i <= 20; i++) {
    const ingredient = meal[`strIngredient${i}`];
    const measure = meal[`strMeasure${i}`];

    if (typeof ingredient === "string" && ingredient.trim() !== "") {
      const amount = typeof measure === "string" ? measure.trim() : "";
      ingredients.push(amount === "" ? ingredient.trim() : `${amount} ${ingredient.trim()}`);
    }
  }

  return ingredients;
}

export async function getRecipeDetails(id: string): Promise<RecipeDetails> {
  const response = await fetch(`${BASE}/lookup.php?i=${encodeURIComponent(id)}`);
  if (!response.ok) {
    throw new Error(`Lookup failed: ${response.status}`);
  }
  const data: { meals: MealDbFullMeal[] | null } = await response.json();
  const meal = data.meals?.[0];
  if (!meal) {
    throw new Error("Recipe not found");
  }
  return {
    ...toRecipe(meal),
    instructions: meal.strInstructions,
    ingredients: extractIngredients(meal),
  };
}

```

The `[key: string]: string | null` is an **index signature** (TypeScript chapter 10): "this object also has other string keys I'm not listing individually." It's what lets you look up `strIngredient7` by a built-up name without TypeScript objecting. This is genuinely the right tool here — the alternative is writing out forty field names by hand.

Notice `...toRecipe(meal)` reusing the converter you already wrote, so the card's four fields are never described twice. And notice the `if (!meal) throw` — `data.meals` can be `null` for an id that doesn't exist, and `?.[0]` can give `undefined`. Handling it here means the component never has to.

In `App.tsx`, add `selectedId: string | null`. When it's set, fetch the details — **a second, completely separate three-state block**, with its own `detailsLoading` and `detailsError`. Feel the duplication; milestone 5 is about to fix it.

Build `src/RecipeModal.tsx` showing the photo, name, category, an ingredients `<ul>`, and the instructions, with a close button that clears `selectedId`.

Check it: click a card, read a recipe, close it. Click a different card — the previous recipe's details must not flash on screen before the new ones load. If they do, your loading state isn't being reset when `selectedId` changes.

Milestone 5: Two focused hooks

Goal: stop writing the same twelve lines twice.

You now have two near-identical loading/error/data blocks. [Chapter 20](#) built a generic `useFetch<T>(url)` for exactly this — but look closely at what you actually need here, because it doesn't quite fit:

- Chapter 20's version takes a **URL** and gives back **raw JSON**. Your `api.ts` functions already return clean, converted `Recipe[]` — so a URL-based hook would force the conversion back up into your components, undoing milestone 2's whole point.
- Your search needs to **not fetch at all** when the query is empty. A hook that fires on every render with whatever URL it's given has no way to express "skip this one."

So instead of bending one generic hook into a shape it doesn't want, write **two small, focused hooks** — which is exactly the advice chapter 20 closed with. Make `src/useRecipeSearch.ts`:

```

import { useEffect, useState } from "react";
import { searchRecipes } from "../api.ts";
import type { Recipe } from "../types.ts";

export function useRecipeSearch(query: string) {
  const [recipes, setRecipes] = useState<Recipe[] | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (query === "") {
      setRecipes(null);
      setError(null);
      return;
    }

    let ignore = false;
    setIsLoading(true);
    setError(null);

    searchRecipes(query)
      .then((found) => {
        if (!ignore) setRecipes(found);
      })
      .catch((err: unknown) => {
        if (!ignore) setError(err instanceof Error ? err.message : "Something went wrong");
      })
      .finally(() => {
        if (!ignore) setIsLoading(false);
      });

    return () => {
      ignore = true;
    };
  }, [query]);

  return { recipes, isLoading, error };
}

```

And `src/useRecipeDetails.ts`, the same shape, taking `id: string | null` and skipping the fetch when it's `null`.

Now `App.tsx` gets dramatically shorter:

```

const { recipes, isLoading, error } = useRecipeSearch(submittedQuery);
const { details, isLoading: detailsLoading } = useRecipeDetails(selectedId);

```

"But that's still duplicated!" — and you're right, the two hooks share a lot. That's a real trade-off, worth being honest about rather than pretending away:

- Two focused hooks are **easy to read** and each says exactly what it does. Their duplication is about a dozen lines.
- One generic hook removes that duplication, but needs a way to express "skip," a way to accept a function instead of a URL, and a dependency list passed in as an argument that your linter can't check for you.

At this size, two clear hooks beat one clever one. If you had *eight* of these, the calculation would flip. Knowing which side of that line you're on is the actual skill — and it's a big part of why [TanStack Query](#) exists in Level 4, having made this decision once, very carefully, for everybody.

Check it: identical behaviour, far less code in `App.tsx`. Both race-condition guards still work — test by clicking two cards in quick succession with the Network tab throttled to Slow 3G.

Milestone 6: Live search, debounced

Goal: results that update as you type, without a request per keystroke.

Bring in `useDebounce` from [chapter 20's exercises](#) — or write it now if you skipped it:

```
export function useDebounce<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}
```

Now delete `submittedQuery` entirely and feed the live `query` through the debouncer:

```
const [query, setQuery] = useState("");
const debouncedQuery = useDebounce(query, 400);
const { recipes, isLoading, error } = useRecipeSearch(debouncedQuery);
```

That's the whole change. `useRecipeSearch` doesn't know or care that its input is now debounced — it just sees a string that changes less often. Two hooks you wrote separately, composing cleanly, with no modification to either.

Keep the form's `onSubmit` working (calling `event.preventDefault()` and nothing else is fine — pressing Enter simply doesn't need to do anything special now), so Enter doesn't reload the page.

Check it: type `pasta` at a normal speed with the Network tab open. You should see **one** request, not five. Type slowly, pausing mid-word, and you'll see two — that's correct, and it's the debounce working exactly as intended.

Milestone 7: Favourites

Goal: a heart that survives a refresh.

Bring in `useLocalStorage` from [chapter 20](#). In `App.tsx`:

```
const [favouriteIds, setFavouriteIds] = useLocalStorage<string[]>("recipe-finder-favourites", []);

function toggleFavourite(id: string) {
  setFavouriteIds((prev) =>
    prev.includes(id) ? prev.filter((existing) => existing !== id) : [...prev, id]
  );
}
```

That's [chapter 11](#)'s `toggle`, doing real work: `filter` to remove, spread to add, never touching the original array.

Add the favourite button to `RecipeCard` — as a **sibling** of the main button, which is exactly what milestone 3's structure was for:

```
<article className="recipe-card">
  <button className="recipe-card-main" onClick={() => onSelect(recipe.id)}>
    ...
  </button>
  <button
    className={isFavourite ? "favorite-button active" : "favorite-button"}
    onClick={() => onToggleFavourite(recipe.id)}
    aria-label={isFavourite ? `Remove ${recipe.name} from favourites` : `Add ${recipe.name} to favourites`}
  >
    {isFavourite ? "♥" : "♡"}
  </button>
</article>
```

No `stopPropagation` anywhere — the two buttons are siblings, so a click on one was never going to trigger the other. That's the payoff for getting the structure right four milestones ago.

The `aria-label` matters here: a screen reader announcing just "♥" tells someone nothing about which recipe, or what clicking it would do.

Now make favourites actually visible. Add to `api.ts`:

```
export async function getRecipesByIds(ids: readonly string[]): Promise<RecipeDetails[]> {
  return Promise.all(ids.map((id) => getRecipeDetails(id)));
}
```

`Promise.all` (JavaScript chapter 31) fires every request **at once** and waits for all of them, rather than one after another — ten favourites take as long as the slowest single request, not ten times as long. Be aware it's all-or-nothing: if any one request fails, the whole thing rejects, which your existing `catch` will handle as a single error message.

Add a **Favourites** toggle button near the search box that switches the grid between search results and your saved recipes, using a third small hook (`useFavouriteRecipes` , wrapping `getRecipesByIds`) or by extending `useRecipeSearch` 's pattern once more.

Check it: favourite three recipes, refresh the page — the hearts are still filled. Clear the search box entirely, switch to Favourites, and see all three, including ones from searches you've since cleared. Check DevTools → Application → Local Storage to see the raw array of ids.

Milestone 8: Category filter

Goal: narrow what's on screen without a second search.

Add `getCategories` to `api.ts` , hitting `list.php?c=list` (its response shape is `{ meals: [{ strCategory: string }] }` — yes, `meals` , even though they're categories; APIs are like this sometimes).

Build `src/CategoryFilter.tsx` , a `<select className="category-select">` with an "All categories" default, populated by fetching once on mount.

Then combine it with the search. TheMealDB can't filter by name *and* category in one request — so don't try. Filter the results you already have:

```
const visibleRecipes = (recipes ?? []).filter(
  (recipe) => category === "all" || recipe.category === category
);
```

A **derived value** ([chapter 08](#)), computed during render, never stored. Changing the category filters instantly with no network request at all, because the data's already sitting in memory.

Check it: search `chicken` , then narrow to `Seafood` — an empty grid, which is correct. Switch to `Chicken` and they're back, instantly, with no request in the Network tab.

Now compare

You've now built two full apps: a to-do list that owned all its data, and this, where almost nothing is yours. It's worth ten minutes putting `App.tsx` from each side by side.

What's genuinely harder here:

- **Nothing is instant.** Every piece of data has a "not yet," a "went wrong," and a "here it is." The to-do app had exactly one state per thing; this has three, and forgetting any one of them produces a page that looks broken.
- **Requests can arrive out of order.** The `ignore` flag exists solely because the network doesn't respect the order you asked in. Nothing in the to-do app could do that.
- **You don't control the data's shape.** `strMeal`, twenty numbered ingredient fields, `null` instead of an empty array — none of that is a design *you'd* have chosen, and `api.ts` exists to stop it spreading.
- **TypeScript can't check the boundary.** `const data: MealDbSearchResponse = await response.json()` is a promise you're making, not one TypeScript verified. If TheMealDB renamed a field tomorrow, your code would compile perfectly and break at runtime. ([Zod](#), in Level 4, is the fix.)

What's easier than you'd expect:

- **Composing hooks.** `useDebounce` fed into `useRecipeSearch` in milestone 6 with zero changes to either. Small pieces with clear inputs and outputs snap together.
- **Adding a feature late.** The category filter was one derived value and one `<select>`. The favourite button dropped into a structure built four milestones earlier.
- **localStorage becoming invisible.** In the to-do app you routed every change through a manual `updateTasks` helper and hoped you hadn't missed a call site. Here, `useLocalStorage` means saving just *happens*, for any change, from anywhere.

The thing worth taking forward: the three-layer split. `api.ts` knowing about JSON, hooks knowing about time, components knowing about pixels. That separation is why milestone 8 was ten minutes' work instead of an afternoon — and it's a pattern that scales far past this app.

Common mistakes

1. Letting TheMealDB's field names escape `api.ts`

```
<h3>{recipe.strMeal}</h3> // ❌ RecipeCard should never know this name exists
```

If a component needs `strMeal`, then `toRecipe` didn't finish its job. Fix the converter, not the component.

2. Nesting the favourite button inside the card button

```
<button className="recipe-card">
  <button className="favorite-button">♥</button> // ❌ invalid HTML
</button>
```

A `<button>` cannot contain another `<button>`. Browsers do unpredictable things with it and the inner one may not be clickable at all. Use the `<article>` + two sibling buttons structure from milestone 3.

3. One loading state shared between search and details

Opening a recipe makes the whole page say "Loading...", including the results that are sitting there fully loaded. Keep the two hooks, and their three states each, entirely separate.

4. Treating an empty result as an error

`{ "meals": null }` means "nothing matched," which is a perfectly successful request. `No recipes found.` is the right response; `Something went wrong` is not.

5. Forgetting `encodeURIComponent`

Search for `chicken & rice` without it and the `&` silently truncates your query at the URL level. You'll get results — just not for what you typed, which is far more confusing than an outright error.

6. Skipping the race-condition guard because "it works on my machine"

It works because your connection is fast. Throttle to Slow 3G in the Network tab, click two cards quickly, and watch the wrong recipe win. Every fetching effect needs the `ignore` flag or an `AbortController`.

7. Caching whole recipes in `localStorage`

Only the favourite `ids` belong there. Cache full recipe objects and you've created a second copy of the truth that can silently drift out of date, with no way to tell.

Quick recap

- Data from a server always has **three** states, not one: loading, error, and the data itself. Every fetch in this project tracks all three.
- Convert someone else's JSON into **your own types** at the boundary, in one file. Nothing downstream should know what their field names were.
- Guard every fetching effect against **out-of-order responses** with the `ignore` flag — the network doesn't respect the order you asked in.
- **Two small, clear hooks beat one clever generic one** at this size. Knowing where that line sits is the real skill.
- Hooks **compose**: `useDebounce` fed straight into `useRecipeSearch` with no changes to either.

- Structure overlapping click targets so they **don't nest** rather than untangling them with `stopPropagation` afterwards.
 - Anything you can derive — the category-filtered list — gets computed during render, never stored.
-

Next: try the [stretch goals](#). That's the end of Level 2. [Level 3](#) opens with Context, for when passing `onToggleFavourite` down through three layers of props finally stops being worth it.

Exercises

These are **stretch goals**: extra features for your finished app. Do them in any order, and do as many as you like.

How to do these:

- Finish all 8 milestones first. Then commit your work, or copy the `recipe-finder` folder, so you've always got a working version to go back to.
 - `index.css` already has styles for these goals. Look for "Styles for the stretch goals" near the bottom.
 - Keep the Network tab open — several of these are about *when* requests fire, not just what ends up on screen.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Recent searches

Add a row of small buttons under the search box showing your last few searches. Clicking one runs that search again.

1. Keep them in a **third** `useLocalStorage` entry, so they survive a refresh.
2. Cap the list at 5, most recent first.
3. No duplicates — searching something you've searched before moves it to the front rather than adding a second copy.
4. Only record a search once it's actually produced results, not on every keystroke. (Think about where in your app you can tell the difference.)

What you should see: search for three different things, refresh the page, and they're all still listed and clickable.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Loading skeletons

Replace the plain "Loading..." text with **skeleton cards**: grey, gently pulsing rectangles in the shape of a real card.

1. Build a `SkeletonCard.tsx` using the `skeleton` class the stylesheet already provides (the pulse animation is built in).
2. Show six of them in the grid while a search is loading.
3. Keep the grid's layout identical, so there's no jump when real cards replace the skeletons.

What you should see: search for something with the Network tab throttled to Slow 3G. Instead of text, then a sudden full grid, the grid's shape appears immediately and fills in.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Recipe of the day, cached for the day

Add a **Recipe of the Day** section above the search results, showing one random recipe (`random.php`) — fetched **once per day**, not once per visit.

1. Cache the recipe **and** the date it was fetched, together, in one `localStorage` entry.
2. On load, compare the cached date with today's. Same day → use the cache, fetch nothing at all. Different day, or nothing cached → fetch a fresh one and store it with today's date.
3. Add a **Shuffle** button that always fetches a new one regardless of the date, so the daily cache is a default rather than a restriction.

What you should see: refresh five times and the recipe stays the same, with no request in the Network tab after the first. Click Shuffle and it changes immediately.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Full keyboard support

Make the app completely usable without a mouse.

1. Tab through the grid. Every card should be reachable, and Enter or Space should open it. (If you built milestone 3 with a real `<button>`, this already works — verify it rather than assuming.)
2. Pressing **Escape** while the modal is open closes it.
3. When the modal opens, focus moves **into** it — onto the close button — so a keyboard user isn't stranded behind an overlay they can't reach.

4. When the modal closes, focus returns to **the card that opened it**, not to the top of the page. Tabbing should carry on from where you were.

Requirement 4 is the interesting one, and it's a genuine, non-toy use of [chapter 19](#).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Favourites that work offline

Right now the favourites view fetches every favourite's details fresh, so with no connection it shows nothing but an error. Fix that properly.

1. Whenever a recipe's full details are successfully fetched — from a search, the detail modal, or Recipe of the Day — cache those details in `localStorage`, keyed by id, **if that recipe is currently favourited**.
2. Un-favouriting a recipe removes its cached details too, so unfavourited data doesn't pile up forever.
3. The favourites view reads the cache **first**, and only fetches ids it has nothing cached for. Anything it does fetch gets cached on the way through.
4. Add a **Surprise me** button, shown only when you have at least one favourite, that opens a random favourite's details — reusing your existing detail view, not a new code path.

Prove it works: favourite three recipes and open each one once. Then set the Network tab to **Offline** and switch to the favourites view. All three should still be there, fully readable, with no errors.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

When you're done

That's the end of Level 2. Before moving on, it's worth writing down:

- One thing that was much harder here than in the to-do app, and why.
- One place where a hook you wrote in an earlier chapter dropped in with no changes at all.
- One thing you still don't feel sure about.

Bring that last one to Claude and ask.

Level 3: Real Apps

Goal: build multi-page apps with shared data, types, and tests.

22 Context

What is it?

Context lets a component hand a value to everything inside it, however deep, without passing it through every layer as props.

```
const ThemeContext = createContext<Theme>("light");

// high up
<ThemeContext value="dark">
  <Page />
</ThemeContext>

// anywhere inside, however deep
const theme = useContext(ThemeContext);
```

Three pieces, every time: **create** the context, **provide** a value somewhere up the tree, **read** it somewhere below.

Why does it matter?

Props flow down one layer at a time. That's been fine so far, because your trees have been shallow. Here's what happens when they aren't:

```
function App() {
  const [theme, setTheme] = useState<Theme>("light");
  return <Page theme={theme} onThemeChange={setTheme} />;
}

function Page({ theme, onThemeChange }: PageProps) {
  return <Sidebar theme={theme} onThemeChange={onThemeChange} />; // doesn't use them
}

function Sidebar({ theme, onThemeChange }: SidebarProps) {
  return <SettingsPanel theme={theme} onThemeChange={onThemeChange} />; // doesn't use them
  either
}

function SettingsPanel({ theme, onThemeChange }: SettingsPanelProps) {
  return <ThemeToggle theme={theme} onThemeChange={onThemeChange} />; // nor this one
}

function ThemeToggle({ theme, onThemeChange }: ThemeToggleProps) {
  return <button onClick={() => onThemeChange(theme === "light" ? "dark" : "light")}>...
</button>;
}
```

`Page`, `Sidebar` and `SettingsPanel` don't care about the theme at all. They're couriers, carrying a parcel they'll never open, and they pay for it:

- **Four props types** mention `theme` and `onThemeChange`, three of them pointlessly.
- **Adding a fifth thing** the toggle needs means editing all four components again.
- **Reusing `Sidebar` elsewhere** now requires supplying a theme, even in a context that has no theming.

This is called **prop drilling**, and you've already felt a mild version of it — in [chapter 21](#), `onToggleFavourite` went from `App` through `RecipeGrid` to `RecipeCard`, and `RecipeGrid` did nothing with it but pass it along. One layer is fine. Four is not.

Context skips the couriers entirely.

Real-world example

Think about how a building supplies **water** versus **parcels**.

Parcels	Water
Handed person to person, floor by floor	Piped to the whole building at once

Parcels	Water
Everyone in the chain must handle it, even if it's not theirs	Only the taps that are turned on actually use it
That's props	That's context
Add a new parcel, and everyone in the chain changes what they carry	Add a tap anywhere, and it just works
You can see exactly where anything came from	Harder to trace where a value was set

That last row is the real trade-off, and it's why context isn't simply "better than props." A parcel's route is visible in the code; a value from context arrives from somewhere unstated. Use pipes for the things genuinely wanted everywhere — water, electricity, the theme, the logged-in user. Keep handing parcels directly for everything else.

How it works

Step 1: create the context

```
// ThemeContext.ts
import { createContext } from "react";

export type Theme = "light" | "dark";

export const ThemeContext = createContext<Theme>("light");
```

`createContext` takes a **default value**, used only when a component reads the context with no provider above it. The type argument says what kind of value this context carries.

Put each context in its own file. It'll be imported by both the provider and every reader, and a separate file keeps those imports from tangling.

Step 2: provide a value

Wrap the part of the tree that should see the value:

```
function App() {
  const [theme, setTheme] = useState<Theme>("light");

  return (
    <ThemeContext value={theme}>
      <Page />
    </ThemeContext>
  );
}
```

Everything inside those tags — `Page`, and everything `Page` renders, all the way down — can now read `theme`. Nothing outside them can.

React 19 note: rendering the context itself as the wrapper, `<ThemeContext value={...}>`, is new in React 19. Before that you had to write `<ThemeContext.Provider value={...}>`, which you'll see in most existing code and older tutorials. Both work in React 19; the shorter one is the one to write from now on.

Step 3: read it

```
import { useContext } from "react";

function ThemeToggle() {
  const theme = useContext(ThemeContext);

  return <button className={theme}>Current theme: {theme}</button>;
}
```

`useContext` is a hook, so the [rules from chapter 08](#) apply — top level, never conditional. It finds the **nearest** provider above this component in the tree and returns its value. If there isn't one, it returns the default from `createContext`.

And `ThemeToggle` can now be moved anywhere in the tree below the provider, and it'll keep working with no prop changes anywhere.

Passing a setter too

A value on its own is read-only. To let components *change* it, put the setter in the context as well — which means the context value becomes an object:

```
// ThemeContext.ts
export type ThemeContextValue = {
  theme: Theme;
  setTheme: (theme: Theme) => void;
};

export const ThemeContext = createContext<ThemeContextValue | null>(null);
```

Notice the type is now `ThemeContextValue | null`, with `null` as the default. That's deliberate, and the next section explains why.

The null default, and the custom hook that goes with it

What default value should `createContext` get for something like a theme *and* its setter? There isn't a sensible one — a fake setter that does nothing would silently swallow every update, which is far worse than an error.

So the honest default is `null`, paired with a small custom hook that turns "no provider" into a clear, immediate crash:

```
// ThemeContext.ts
export function useTheme(): ThemeContextValue {
  const context = useContext(ThemeContext);

  if (context === null) {
    throw new Error("useTheme must be used inside a ThemeProvider");
  }

  return context;
}
```

This small function earns its place three times over:

- **TypeScript.** `useContext(ThemeContext)` is typed `ThemeContextValue | null`, so every component reading it would need its own `if (context === null)` check. `useTheme` does that once, and returns a plain, non-null `ThemeContextValue`.
- **The error message.** Without it, forgetting the provider gives you `Cannot read properties of null (reading 'theme')` somewhere deep in a component. With it, you get a sentence telling you exactly what's wrong.
- **It's the public front door.** Components import `useTheme`, not `ThemeContext` — so you can change how the context works internally without touching them.

This custom-hook-per-context pattern is standard in real React codebases. Write it every time; it's six lines.

Bundling it up: a provider component

The last piece is wrapping the state and the provider together, so `App` doesn't have to know how theming works:

```
// ThemeProvider.tsx
import { useState, type ReactNode } from "react";
import { ThemeContext, type Theme } from "../ThemeContext.ts";

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>("light");

  return (
    <ThemeContext value={{ theme, setTheme }}>
      {children}
    </ThemeContext>
  );
}
```

```
function App() {
  return (
    <ThemeProvider>
      <Page />
    </ThemeProvider>
  );
}
```

That's the whole pattern, and it's worth recognising the shape, because you'll meet it constantly in libraries: the state lives inside the provider, `children` ([chapter 04](#)) lets it wrap anything, and consumers use a custom hook. Everything a feature needs is in two files, and `App` just switches it on.

Context doesn't replace state — it moves it

A common misreading: "context is for global state." It isn't. **Context is a delivery mechanism, not a storage mechanism.** The state is still ordinary `useState`, living in an ordinary component. Context only changes *who can see it* without being handed it.

That means everything you know still applies:

- Changing the value means calling the setter, which re-renders the provider.
- All the copying rules from [chapter 11](#) apply to state held in a provider.
- If the value is derivable, derive it — don't put it in the context.

When context is the wrong tool

Reach for it less often than you'd think. Try these first:

1. Lift state up ([chapter 13](#)). If the components that need the value are one or two layers apart, props are simpler and easier to trace. Prop drilling only becomes a real problem at three or more layers of pure pass-through.

2. Pass JSX down instead. Often the "drilling" disappears entirely if the parent builds the element itself and passes it as `children`:

```
// Instead of Layout drilling `user` down to Header...
<Layout>
  <Header user={user} />
</Layout>
```

`Layout` now knows nothing about `user` — it just renders whatever it's given. This solves a surprising share of prop-drilling problems, costs nothing, and keeps the data flow visible. Try it before you reach for context.

Good candidates for context, on the other hand, are values that are genuinely ambient — wanted almost anywhere, rarely changing:

- the current theme
- the logged-in user
- the chosen language
- a shopping cart, in an app where almost every page can add to it

Every consumer re-renders when the value changes

The one performance fact worth knowing now: when a provider's `value` changes, **every component reading that context re-renders**, however deep, whether or not it uses the part that changed.

That makes this a genuine trap:

```
<ThemeContext value={{ theme, setTheme }}> // ⚠ a brand-new object every render
```

That object literal is rebuilt on every render of the provider — a new reference each time ([chapter 11](#)) — so React concludes the value changed, and re-renders every consumer, even when `theme` is identical to last time.

For a theme that changes rarely, in a small app, this genuinely doesn't matter and you should not worry about it. When it does matter, the fix is `useMemo`, which is [chapter 27](#):

```
const value = useMemo(() => ({ theme, setTheme }), [theme]);

return <ThemeContext value={value}>{children}</ThemeContext>;
```

Know the trap exists; don't optimise for it until something is actually slow.

Several contexts, not one big one

It's tempting to put everything in one `AppContext`. Don't — because of the rule above, a component that only reads the theme would re-render every time the cart changed.

```
<ThemeProvider>
  <AuthProvider>
    <CartProvider>
      <App />
    </CartProvider>
  </AuthProvider>
</ThemeProvider>
```

Nesting providers like this is normal and completely fine. Split by **what changes together**: things that update at the same time, for the same reason, belong in one context.

Reading context conditionally with `use`

React 19 added `use`, which reads a context and — unlike `useContext` — *is* allowed inside a condition:

```
import { use } from "react";

function Panel({ showTheme }: { showTheme: boolean }) {
  if (showTheme) {
    const theme = use(ThemeContext); // legal, unlike useContext
    return <p>{theme}</p>;
  }
  return null;
}
```

Worth knowing it exists, because you'll see it. `useContext` remains completely standard and is what this course uses — reach for `use` only when you genuinely need the conditional read.

Common mistakes

1. Reading a context with no provider above it

```
function App() {  
  return <ThemeToggle />; // ❌ no ThemeProvider anywhere  
}
```

You silently get the default from `createContext`, which is usually wrong and sometimes `null`, producing a confusing crash deep inside a component. The `useTheme` guard hook turns this into a clear error message instead.

2. Skipping the custom hook and the `null` default

```
const ThemeContext = createContext<ThemeContextValue>({ theme: "light", setTheme: () => {} });
```

That fake `setTheme` does nothing at all. A component used outside a provider now *appears* to work, silently ignoring every click. `null` plus a guard hook fails loudly, which is what you want.

3. Putting everything in one context

One `AppContext` holding theme, user, cart and settings means every consumer re-renders whenever any of them changes. Split by what changes together.

4. Using context where lifting state up would do

Two components one layer apart do not need context. It's more machinery, and it hides where the value came from. Props are the default; context is the exception.

5. Creating the value object inline without thinking about it

```
<ThemeContext value={{ theme, setTheme }}>
```

A new object every render, so every consumer re-renders every time. Fine in a small app, worth `useMemo` in a big one — but know that it's happening.

6. Treating context as a state manager

Context doesn't store anything. It delivers a value from a `useState` (or a `useReducer`, [chapter 23](#)) that still lives in an ordinary component.

Quick recap

- Context delivers a value to everything below a provider, skipping the components in between — the fix for **prop drilling** through layers that don't care.
- Three steps: `createContext` (in its own file), wrap with `<MyContext value={...}>`, read with `useContext`.

- Default the context to `null` and export a **custom hook that throws** when there's no provider. It handles the TypeScript narrowing and gives a real error message.
 - Bundle the state and the provider into one `Provider` component taking `children`. That's the shape every library uses.
 - Context is **delivery, not storage**. The state is still ordinary `useState` in an ordinary component.
 - Try **lifting state up** and **passing JSX as `children`** first. Context is for genuinely ambient values: theme, current user, language.
 - Every consumer re-renders when the value changes, so split contexts by what changes together — and know that an inline `value={{ ... }}` object is a new reference every render.
-

Next: try the [exercises](#), then move on to [23 useReducer](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch22/ex1/`, and so on.
 - Put each context in its own file, with its own guard hook, from the very first exercise. It's six lines and it's the habit worth building.
 - Use the **Components** tab to check the provider is where you think it is — context bugs are nearly always "the provider isn't above the consumer."
 - An exercise is done when the behaviour is correct, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console is empty.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Feel the drilling, then fix it

Build this **deliberately badly** first, in `src/ch22/ex1/`, so the fix means something.

Five components, each in its own file, nested five deep: `Ex1` → `Page` → `Sidebar` → `SettingsPanel` → `FontSizeControl`. Only the last one uses anything.

1. Put `fontSize` state (a number, 12–32) in `Ex1`, and drill it — plus a setter — all the way down through every layer as props. `FontSizeControl` renders `+` and `-` buttons. `Ex1` renders a paragraph whose `style` uses the size, so you can see it working.
2. Count how many of the five components mention `fontSize` in their props type. Write the number in a comment.

3. Now add a **second** thing the control needs — a `fontFamily` string — and drill that too. How many files did you have to edit?
4. Rewrite it with context: a `FontContext`, a `FontProvider`, and a `useFont` guard hook. `Page`, `Sidebar` and `SettingsPanel` should end up with **no props at all**.
5. Add a *third* setting (`fontWeight`). How many files this time?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): The guard hook earns its keep

In `src/ch22/ex2/`, build a simple `UserContext` carrying `{ name: string; isLoggedIn: boolean }`, with a provider and a `useUser` hook.

Then break it on purpose and write down exactly what happens each time:

1. Render a component that calls `useUser` **outside** the provider, with the context defaulted to `null` and **no** guard hook — just `useContext(UserContext)` directly. What does TypeScript say when you try to read `user.name`? What happens at runtime if you force it through with `!`?
2. Now add the guard hook that throws. Render the same component outside the provider again. What's the error, and where in DevTools does it point you?
3. Finally, try it with a **fake default** instead — `createContext<UserValue>({ name: "", isLoggedIn: false })` and no guard. Render outside the provider. What happens now, and why is this the most dangerous of the three?

Write a one-paragraph comparison at the bottom of the file.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Theme, properly

In `src/ch22/ex3/`, build a complete theming setup, the way you would in a real app.

1. `ThemeContext.ts`: a `Theme = "light" | "dark"`, a context typed `ThemeContextValue | null`, and a `useTheme` guard hook.
2. `ThemeProvider.tsx`: holds the state, persists it to `localStorage` with your `useLocalStorage` hook from [chapter 20](#), and provides `{ theme, toggleTheme }`.
3. A page at least three components deep, where **two** widely separated components read the theme: a `ThemeToggle` button in a header, and a `Card` deep inside the content that styles itself from the theme.
4. The provider applies the theme by putting a class on a wrapping `<div>`, with CSS variables doing the actual colour work ([chapter 15](#)).

5. Refresh the page — the theme persists, and neither the header nor the card needed a single prop.

Then answer in a comment: `ThemeProvider` uses `useLocalStorage`, which is a custom hook, inside a component that provides a context. Is that a problem? Why or why not?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): Two contexts, and proving why

This exercise makes the "split by what changes together" rule visible rather than theoretical.

In `src/ch22/ex4/`, build a small app with a theme **and** a click counter that updates frequently.

Part 1: one context. Put both in a single `AppContext` providing `{ theme, toggleTheme, count, increment }`. Build two consumers: `ThemeLabel` (reads only `theme`) and `CountLabel` (reads only `count`). Put a `console.log("ThemeLabel rendered")` in the first.

Click the increment button ten times. How many times did `ThemeLabel` render? Was the theme involved at all?

Part 2: two contexts. Split them into `ThemeContext` and `CounterContext`, with separate providers, nested. Click increment ten times again. How many times does `ThemeLabel` render now?

Part 3: Write down the render counts from both parts, and explain in two or three sentences what rule you just demonstrated.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A toast notification system

Toasts — those small messages that slide in saying "Saved!" or "Something went wrong" — are the textbook case for context: **any** component, anywhere, needs to be able to trigger one, and none of them should have to receive a prop to do it.

In `src/ch22/ex5/`, build:

`ToastContext.ts` — a context providing one function:

```
type ToastContextValue = {
  showToast: (message: string, kind: "success" | "error") => void;
};
```

Plus a `useToast` guard hook.

`ToastProvider.tsx` — holds an array of active toasts in state, provides `showToast`, and renders the toasts themselves in a fixed-position stack, on top of `children`.

A test page with at least three completely unrelated components scattered at different depths — a form, a delete button, a settings panel — each calling `useToast()` and triggering different messages. None of them may take a toast-related prop.

Requirements:

1. Each toast has a unique id (`crypto.randomUUID()`), a message, and a kind.
2. Toasts stack — trigger three quickly and see three, in order.
3. Each toast disappears on its own after 4 seconds, and can also be dismissed early with an `×`.
4. Each toast's timer is cleaned up properly if it's dismissed early, or if the provider unmounts — no timers left running for toasts that are already gone.
5. The toast components themselves never mutate the array — adding and removing both follow [chapter 11](#).

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

23 useReducer

What is it?

`useReducer` is a second way to hold state. Instead of calling setters directly, components **describe what happened**, and one function decides what the new state should be.

```
const [state, dispatch] = useReducer(reducer, initialState);

dispatch({ type: "added", text: "Buy milk" });
```

The function that decides is called a **reducer**. It takes the current state and a description of what happened, and returns the new state:

```
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "added":
      return { ...state, tasks: [...state.tasks, newTask(action.text)] };
    // ...
  }
}
```

Everything you can do with `useReducer` you can do with `useState`. It's not more powerful — it's better *organised*, for a specific kind of problem.

Why does it matter?

`useState` is perfect while your state is small and its updates are simple. It starts to creak when a feature has many related pieces of state updated in many different ways, from many different places.

Picture the to-do app from [chapter 10](#), grown up a bit:


```
function TaskApp() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [filter, setFilter] = useState<Filter>("all");
  const [editingId, setEditingId] = useState<string | null>(null);

  function handleAdd(text: string) {
    setTasks([...tasks, { id: crypto.randomUUID(), text, done: false }]);
    setEditingId(null);
  }

  function handleDelete(id: string) {
    setTasks(tasks.filter((task) => task.id !== id));
    if (editingId === id) setEditingId(null); // easy to forget
  }

  function handleClearCompleted() {
    setTasks(tasks.filter((task) => !task.done));
    if (editingId !== null && !tasks.some((t) => t.id === editingId && !t.done)) {
      setEditingId(null); // even easier to forget
    }
  }
  // ...six more handlers
}
```

Look at what's going wrong. The **rules** — "deleting a task you're editing must stop the edit" — are scattered across handlers, restated slightly differently each time. Add a tenth handler and you have to remember every rule again. Miss one, and you get a bug that only shows up in a specific sequence of clicks.

`useReducer` gathers all of those rules into **one function**, where they sit side by side and can be read in one pass. The component goes back to doing what components are good at: describing what the user did, and rendering.

Real-world example

Think about the difference between **everyone editing a shared document** and **submitting request forms to one clerk**.

Everyone edits directly	Everyone submits a form
Each handler calls setters itself	Each handler calls <code>dispatch</code>
The rules live wherever someone remembered them	The rules live in one place: the clerk
Two people can make contradictory edits	One clerk, one decision, one at a time
To learn the rules, read every handler	To learn the rules, read the clerk

Everyone edits directly	Everyone submits a form
That's <code>useState</code>	That's <code>useReducer</code>
No paper trail	Every form is a record of exactly what was asked

That last row matters more than it sounds. Because every change arrives as a plain object saying what happened, you can log them, replay them, or write down the sequence that produced a bug — which is exactly what the DevTools for libraries like Redux ([chapter 35](#)) do.

How it works

The three pieces

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- **state** — the current state. Read it in your JSX exactly like a `useState` value.
- **dispatch** — call it with an **action** to say what happened. Like a setter, it triggers a re-render.
- **reducer** — the function that turns `(state, action)` into the next state.

Compare it directly with what you know:

	<code>useState</code>	<code>useReducer</code>
Getting the value	<code>const [count, setCount]</code>	<code>const [state, dispatch]</code>
Changing it	<code>setCount(count + 1)</code>	<code>dispatch({ type: "incremented" })</code>
Where the logic lives	In each handler	In the reducer
Good for	A few independent values	Several related values with shared rules

A complete, small example

```
type CounterState = {
  count: number;
  step: number;
};

type CounterAction =
  | { type: "incremented" }
  | { type: "decremented" }
  | { type: "step_changed"; step: number }
  | { type: "reset" };

const initialState: CounterState = { count: 0, step: 1 };

function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "incremented":
      return { ...state, count: state.count + state.step };
    case "decremented":
      return { ...state, count: Math.max(0, state.count - state.step) };
    case "step_changed":
      return { ...state, step: action.step };
    case "reset":
      return initialState;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(counterReducer, initialState);

  return (
    <div>
      <p>Count: {state.count} (step {state.step})</p>
      <button onClick={() => dispatch({ type: "incremented" })}>+</button>
      <button onClick={() => dispatch({ type: "decremented" })}>-</button>
      <button onClick={() => dispatch({ type: "step_changed", step: 5 })}>Step 5</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
}
```

Read `Counter` on its own: every handler is one line, and each says only *what happened*. The rule that the count can't go below zero lives in the reducer, once — not repeated in every place that might decrement.

Typing actions: the discriminated union

This is where TypeScript pays off enormously, and it's the pattern to learn from this chapter:

```

type CounterAction =
  | { type: "incremented" }
  | { type: "decremented" }
  | { type: "step_changed"; step: number }
  | { type: "reset" };

```

That's a **discriminated union** (TypeScript chapter 06): several object shapes, told apart by one shared field — here, `type`. It buys you three things:

1. TypeScript narrows inside each `case`.

```

case "step_changed":
  return { ...state, step: action.step }; // ✅ TypeScript knows action has .step here

case "reset":
  return { ...state, step: action.step };
  // ❌ Property 'step' does not exist on type '{ type: "reset"; }'.

```

Inside `case "step_changed"`, `action` is narrowed to exactly that one shape. You can't read a field that `action` doesn't carry.

2. Typos in action names are caught immediately.

```

dispatch({ type: "incremated" });
// ❌ Type '"incremated"' is not assignable to type '"incremented" | "decremented" | ...'.

```

3. Missing data is caught too.

```

dispatch({ type: "step_changed" });
// ❌ Property 'step' is missing in type '{ type: "step_changed"; }'.

```

Compare that with a pile of loose `useState` setters, where nothing connects "the kind of change" to "the data that change needs." This is one of the clearest wins TypeScript offers in a React app.

Exhaustiveness: catching a case you forgot

Add a new action type and forget to handle it, and by default the reducer just falls through and returns nothing — leaving `state` as `undefined`. You can make TypeScript catch that instead:

```
function counterReducer(state: CounterState, action: CounterAction): CounterState {
  switch (action.type) {
    case "incremented":
      return { ...state, count: state.count + state.step };
    // ... other cases ...
    default: {
      const exhaustive: never = action;
      throw new Error(`Unknown action: ${JSON.stringify(exhaustive)}`);
    }
  }
}
```

If every case is handled, `action` in the `default` branch has type `never` — "this can't happen" — and assigning it to a `never` variable is fine. The moment you add a fifth action type and don't handle it, `action` in `default` is that fifth shape, which **isn't** assignable to `never`, and TypeScript stops you:

 Type '{ type: "doubled"; }' is not assignable to type 'never'.

A compile-time error, pointing at exactly the case you forgot. This little trick is worth using in every reducer you write.

Reducers must be pure

A reducer follows the same rules as a component's render ([chapter 12](#)):

- **Return a new state object — never mutate the old one.** All of [chapter 11](#) applies, unchanged.
- **No side effects.** No `fetch`, no `localStorage`, no `Math.random()`, no `Date.now()`, no logging to a server. Given the same state and action, it must always return the same result.

```
case "added":
  state.tasks.push(newTask);           //  mutation
  return state;                       //  and the same reference, so React may not even re-
render

case "added":
  return { ...state, tasks: [...state.tasks, newTask] }; // 
```

Purity is why `StrictMode` can call your reducer twice in development without breaking anything, and it's what makes actions replayable.

So where do side effects go? Exactly where they went before: in event handlers, or in effects. Generate the id *before* dispatching, and pass it in the action:

```
function handleAdd(text: string) {  
  dispatch({ type: "added", id: crypto.randomUUID(), text });  
}
```

The reducer stays pure; the randomness happens outside it.

Naming actions

Two conventions you'll meet. Name actions after **what happened**, not what should change:

```
dispatch({ type: "added", text });           // ✅ describes the event  
dispatch({ type: "set_tasks", tasks });      // ⚠️ describes the mechanism
```

The first reads like a log of user activity, which is the whole point — a reducer full of `set_x` actions is just `useState` with extra steps. Past-tense names (`added` , `deleted` , `filter_changed`) keep you honest about this.

Converting `useState` to `useReducer`

You'll usually start with `useState` and move over when it starts hurting. The steps:

1. **Gather the related state into one object.** That object's type is your `State` .
2. **List every way it can change.** Each one becomes a member of your `Action` union, carrying whatever data it needs.
3. **Move each handler's body into the matching case** , changing `setX(...)` into `return { ...state, x: ... }` .
4. **Replace each handler body with a single `dispatch` .**

The to-do app's reducer, done this way, ends up looking like this:

```

type TaskAction =
  | { type: "added"; id: string; text: string }
  | { type: "toggled"; id: string }
  | { type: "deleted"; id: string }
  | { type: "cleared_completed" }
  | { type: "filter_changed"; filter: Filter };

function taskReducer(state: TaskState, action: TaskAction): TaskState {
  switch (action.type) {
    case "added":
      return {
        ...state,
        tasks: [...state.tasks, { id: action.id, text: action.text, done: false }],
      };
    case "toggled":
      return {
        ...state,
        tasks: state.tasks.map((task) =>
          task.id === action.id ? { ...task, done: !task.done } : task
        ),
      };
    case "deleted":
      return {
        ...state,
        tasks: state.tasks.filter((task) => task.id !== action.id),
        editingId: state.editingId === action.id ? null : state.editingId,
      };
    // ...
  }
}

```

Look at the `deleted` case: the rule "deleting the task you're editing also stops editing" is stated **once**, right next to the deletion it belongs to. In the `useState` version it was a separate `if` in a separate handler, easy to forget and easy to get subtly wrong.

Lazy initial state

`useReducer` takes an optional third argument: a function that computes the initial state, called only on the first render — the same idea as lazy initial state in [chapter 08](#):

```
function init(savedJson: string | null): TaskState {
  if (savedJson === null) return { tasks: [], filter: "all", editingId: null };
  // ...parse it
}

const [state, dispatch] = useReducer(taskReducer, localStorage.getItem(KEY), init);
```

The second argument becomes `init` 's argument. You won't need this often, but it's there.

`useReducer` and context, together

These two combine into one of the most useful patterns in React: state that any component can read *and* change, without a single prop.

```
const TaskStateContext = createContext<TaskState | null>(null);
const TaskDispatchContext = createContext<React.Dispatch<TaskAction> | null>(null);

export function TaskProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(taskReducer, initialState);

  return (
    <TaskStateContext value={state}>
      <TaskDispatchContext value={dispatch}>
        {children}
      </TaskDispatchContext>
    </TaskStateContext>
  );
}
```

Two contexts, not one, and it's deliberate. `dispatch` is completely stable — React guarantees it's the same function for the life of the component — so a component that only dispatches (a delete button, say) reads a value that *never changes*, and therefore never re-renders when the state does. Bundling them into one object would throw that away ([chapter 22](#)).

With a guard hook for each, any component anywhere can do:

```
const { tasks } = useTaskState();
const dispatch = useTaskDispatch();
```

No props, no drilling, and every rule about how tasks change still living in one reducer. This is roughly what Redux ([chapter 35](#)) offers, built from two React features you already know.

When to use which

Stay with `useState` when:

- the state is one or two independent values
- updates are simple and don't depend on each other
- there aren't many places that change it

Reach for `useReducer` when:

- several pieces of state change together, under shared rules
- the next state often depends on the previous one in non-trivial ways
- the same logic is repeated across handlers
- you want to be able to see all the ways your state can change, in one place

You'll know. When a fourth handler starts with "and also reset the editing id," that's the signal.

Common mistakes

1. Mutating state in the reducer

```
case "toggled":
  const task = state.tasks.find((t) => t.id === action.id);
  task.done = !task.done;    // ✗
  return state;              // ✗ same reference, React may not re-render
```

Every rule from [chapter 11](#) applies. Build a new object, always.

2. Side effects inside the reducer

```
case "added":
  localStorage.setItem("tasks", ...);    // ✗
  return { ...state, tasks: [...state.tasks, { id: crypto.randomUUID(), ... }] }; // ✗
```

Reducers must be pure. Do the storage write in an effect, and generate the id in the handler, passing it in the action.

3. Forgetting to `return` in a case

```
case "reset":
  initialState;    // ✗ computed and thrown away; falls through
```

Your state silently becomes `undefined` (or falls into the next case). The `never` exhaustiveness check catches the *missing case* problem; this one is caught by the reducer's declared return type, so don't skip the `: State`

annotation.

4. Actions named after setters

```
dispatch({ type: "set_tasks", tasks: newTasks });
```

If every action is `set_something`, you've rebuilt `useState` with more ceremony and none of the benefit. Name actions after what happened.

5. Reaching for `useReducer` too early

A single boolean does not need a reducer. The ceremony only pays for itself once there are genuinely shared rules to centralise.

6. Putting `state` and `dispatch` in one context

It works, but it means every dispatch-only component re-renders on every state change. Two contexts costs a few extra lines and keeps `dispatch`'s stability useful.

Quick recap

- `useReducer(reducer, initialState)` gives you `[state, dispatch]`. Components **describe what happened**; one reducer decides what the state becomes.
- Type actions as a **discriminated union** on `type`. TypeScript then narrows inside each `case`, and catches misspelled action names and missing data at both ends.
- Add a `default` case assigning `action` to a `never` variable to make TypeScript catch any action type you forgot to handle.
- Reducers must be **pure**: new state objects, no mutation, no side effects. Generate ids and random values in the handler and pass them in the action.
- Name actions after **what happened** (`added`, `deleted`), not after what to set.
- `useReducer` + two contexts (one for state, one for `dispatch`) gives any component read and write access with no props — a small, hand-rolled version of what Redux does.
- Use `useState` until the rules for changing state start repeating themselves across handlers. That's the moment to switch.

Next: try the [exercises](#), then move on to [24 React Router](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch23/ex1/`, and so on. Put each reducer and its types in their own file, separate from the component.
 - **Type every action union properly, and add the `never` exhaustiveness check** to every reducer from exercise 1 onward. Half the value of this chapter is in those two habits.
 - An exercise is done when the behaviour is correct, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console is empty.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): A counter with rules

In `src/ch23/ex1/`, build a counter with `useReducer` whose state is `{ count: number; step: number }` and whose actions are `incremented`, `decremented`, `step_changed`, and `reset`.

Rules the reducer must enforce (all of them in the reducer, none in the component):

1. The count can never go below 0 or above 100.
2. Changing the step never changes the count.
3. `reset` returns both count and step to their starting values.

Then break things on purpose and write down the exact error each time:

4. `dispatch({ type: "incremeted" })` — a typo in the action name.
5. `dispatch({ type: "step_changed" })` — the required `step` missing.
6. Inside `case "reset"`, try to read `action.step`.
7. Add a fifth action type to the union, `doubled`, but **don't** add a `case` for it. What does the `never` check say, and which line does it point at?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Convert a `useState` component

Here's a working component in `src/ch23/ex2/Ex2.tsx`. Your job is to convert it, not to redesign it — the behaviour afterwards must be identical.

```

function Ex2() {
  const [isOpen, setIsOpen] = useState(false);
  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [errors, setErrors] = useState<string[]>([]);

  function handleOpen() {
    setIsOpen(true);
    setErrors([]);
  }

  function handleClose() {
    setIsOpen(false);
    setTitle("");
    setBody("");
    setErrors([]);
  }

  function handleSubmit() {
    const found: string[] = [];
    if (title.trim() === "") found.push("Title is required");
    if (body.trim().length < 10) found.push("Body must be at least 10 characters");

    if (found.length > 0) {
      setErrors(found);
      return;
    }

    console.log("Submitting", { title, body });
    setIsOpen(false);
    setTitle("");
    setBody("");
    setErrors([]);
  }

  // ...a modal with two inputs, a Submit button, and the errors listed
}

```

1. Count how many times `setErrors([])` appears, and how many times the "clear the form" sequence is repeated. Write the numbers in a comment.
2. Convert it to a single `useReducer`. The state is one object; the actions are `opened`, `closed`, `title_changed`, `body_changed`, and `submitted`.
3. Notice what happens to the repetition in step 1 — where does "clear the form" live now?
4. Add a new rule: **closing the modal with unsaved text should keep the text**, so reopening restores it. How many places did you have to change?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): A shopping cart reducer

In `src/ch23/ex3/`, build a cart with `useReducer`. State:

```
type CartState = {
  items: CartItem[];
  discountCode: string | null;
};

type CartItem = {
  id: string;
  name: string;
  price: number;
  quantity: number;
};
```

Actions: `added`, `removed`, `quantity_changed`, `discount_applied`, `discount_removed`, `cleared`.

Rules, all enforced in the reducer:

1. Adding a product already in the cart increases its quantity instead of adding a second line.
2. Quantity can never go below 1 — a `quantity_changed` that would take it to 0 removes the item entirely.
3. `discount_applied` only accepts the code `SAVE10`; anything else leaves the state completely unchanged (return the same state object).
4. `cleared` empties the items **and** removes any discount.

The component shows the items, a subtotal, the discount (10% off, when applied), and a total. None of those three numbers is state.

5. Write a comment listing which of your rules would have been scattered across multiple handlers in a `useState` version.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Undo and redo, the reducer way

You built undo in [chapter 11's exercises](#) by hand. A reducer makes it dramatically cleaner, because every change already flows through one function.

In `src/ch23/ex4/`, build a simple drawing-list app (add a shape, remove a shape, change a shape's colour — no actual canvas needed, a list of coloured boxes is fine) with this state:

```
type HistoryState<T> = {
  past: T[];
  present: T;
  future: T[];
};
```

Requirements:

1. The reducer handles your normal actions (`shape_added` , `shape_removed` , `colour_changed`) **plus** `undo` and `redo` .
2. Every normal action pushes the current `present` onto `past` and clears `future` .
3. `undo` moves `present` to `future` and pops the last item off `past` .
4. `redo` is the mirror image.
5. Undo and redo buttons are disabled when their stack is empty.
6. No mutation anywhere — `past` and `future` are rebuilt with `slice` and spreads.

Then answer in a comment: what made this so much easier to add than it was in chapter 11?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Reducer plus context — a real app shell

Combine this chapter with [chapter 22](#) to build state that any component can read and change, with zero props.

In `src/ch23/ex5/` , build a small task board:

`taskReducer.ts` — state { `tasks: Task[]`; `filter: Filter`; `editingId: string | null` }, with actions `added` , `toggled` , `deleted` , `edited` , `edit_started` , `edit_cancelled` , `filter_changed` , `cleared_completed` .

Enforce these rules **in the reducer**:

1. Deleting the task currently being edited also ends the edit.
2. `cleared_completed` ends the edit if the edited task was completed and got cleared.
3. `edit_started` on a task that doesn't exist is ignored (state unchanged).
4. An `edited` action with empty text deletes the task instead.

`TaskProvider.tsx` — provides state and dispatch through **two separate contexts**, with a guard hook for each.

The components — `AddTaskForm` , `FilterBar` , `TaskList` , `TaskRow` , `Footer` — nested at least three levels deep, and **not one of them may take a single prop related to tasks**. Every one gets what it needs from the hooks.

Finally:

5. Add a `console.log` to `FilterBar` (which only dispatches, and reads only `filter`). Add and toggle several tasks. Does `FilterBar` re-render every time? Explain what you see, and what would change if state and dispatch shared one context.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

24 React Router

What is it?

React Router maps URLs to components, so one React app can have many pages:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/books" element={<BookList />} />
  <Route path="/books/:id" element={<BookDetails />} />
</Routes>
```

Visit `/books/42` and React Router renders `<BookDetails />`, with `42` available to it. The browser never reloads — React just swaps what's on screen — but the URL, the back button, and bookmarks all work exactly as people expect.

This course uses **React Router v7**. The package is called `react-router`.

Why does it matter?

Every app you've built so far has been one screen. Real apps aren't. And the moment you have more than one screen, you could try this:

```
const [page, setPage] = useState<"home" | "books" | "cart">("home");

{page === "home" && <Home />}
{page === "books" && <BookList />}
```

It works — and it's broken in ways that matter:

- **The URL never changes.** Every page is `localhost:5173`. You can't send someone a link to a specific book.
- **The back button leaves your app.** Browse five pages, press back, and you're on whatever site you were on before. This is the one users notice and hate.
- **Refreshing loses your place.** You're always back at the start.
- **Nothing can be bookmarked**, and search engines see a single page.

A URL isn't decoration — it's the address of a piece of your app, and users expect it to behave like one. React Router's real job is keeping your components and the browser's address bar in agreement, in both directions.

Real-world example

Think about a **hotel**.

Hotel	React Router
Room numbers on the doors	URL paths: <code>/books/42</code>
"Room 214, third floor, turn left"	The route table: which path shows which component
Reception directing you	The <code><Routes></code> block
Walking there yourself	Clicking a <code><Link></code>
Every room reachable directly from outside	Any URL can be opened, shared, or bookmarked
All rooms share the same lobby and lift	Nested routes sharing a layout
"Room 999 doesn't exist"	The <code>*</code> catch-all route

The thing that makes a hotel usable isn't that rooms exist — it's that every one has an address you can write down and come back to.

How it works

Setting it up

```
npm install react-router
```

The package name changed. For years this was `react-router-dom`, and that's what nearly every existing tutorial, Stack Overflow answer and codebase says. In **v7**, it's just `react-router` — `react-router-dom` still exists as a compatibility shim, but new projects install and import from `react-router`. If you copy a snippet importing from `react-router-dom`, change the import; everything else usually works unchanged.

Wrap your app once, in `main.tsx`:

```
import { BrowserRouter } from "react-router";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

`BrowserRouter` is what watches the real URL and tells everything inside it what's going on. Without it, every other piece in this chapter throws.

The route table

```
import { Routes, Route } from "react-router";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/books" element={<BookList />} />
      <Route path="/about" element={<About />} />
    </Routes>
  );
}
```

`<Routes>` looks at the current URL, picks the **one** `<Route>` that matches best, and renders its `element`. Note that `element` takes actual JSX — `<Home />`, not `Home` — which means you can pass props to a page component right there if you need to.

Linking between pages

This is the part people get wrong first, so it's worth being blunt about:

```
import { Link } from "react-router";

<a href="/books">Books</a>           // ❌ full page reload – your app restarts from scratch
<Link to="/books">Books</Link>      // ✅ React swaps the component, instantly
```

A plain `<a>` tells the **browser** to go and fetch a new document. Everything your app was holding — state, context, fetched data — is destroyed and rebuilt. It looks like it works (you do end up on the right page), but there's a visible flash and you've thrown away the entire point of a single-page app.

`<Link>` renders a real `<a>` underneath, so right-click → open in new tab and all the other things people expect from links still work. It just intercepts the ordinary click.

Use a real `<a>` only for links leaving your app — an external site, a file download.

NavLink, for navigation that knows where you are

`NavLink` is `Link` plus "am I the current page?", which is exactly what a nav bar needs:

```
import { NavLink } from "react-router";

<NavLink
  to="/books"
  className={({ isActive }) => (isActive ? "nav-link active" : "nav-link")}
>
  Books
</NavLink>
```

`className` can be a **function** here, receiving `{ isActive }`. Same for `style`, and for `children` if you want the content itself to change.

URL parameters

A segment starting with `:` is a wildcard that captures whatever's there:

```
<Route path="/books/:id" element={<BookDetails />} />
```

`/books/42` and `/books/hobbit` both match, and the captured piece is available inside:

```
import { useParams } from "react-router";

function BookDetails() {
  const { id } = useParams();
  // id: string | undefined
}
```

Note the type. `useParams` gives you `string | undefined` for every parameter, because React Router can't know at compile time which route rendered this component. TypeScript makes you deal with that, and it's right to:

```
function BookDetails() {
  const { id } = useParams();

  if (id === undefined) {
    return <p>No book selected.</p>;
  }

  const { data: book, isLoading } = useBook(id); // id is a string from here down
  // ...
}
```

You can name the expected params to get slightly better types — `useParams<{ id: string }>()` — but they're *still* possibly `undefined`, so the check stays. [Chapter 36](#) covers a router that does make these genuinely type-safe.

Navigating in code

Sometimes navigation follows an action rather than a click — after submitting a form, say:

```
import { useNavigate } from "react-router";

function NewBookForm() {
  const navigate = useNavigate();

  async function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    const created = await saveBook(form);
    navigate(`/books/${created.id}`);
  }
}
```

`navigate(-1)` goes back one entry, like the browser's back button. And `navigate("/login", { replace: true })` **replaces** the current history entry instead of adding one — use that after a redirect, so pressing back doesn't bounce the user straight back to the page they were redirected away from.

Don't reach for `useNavigate` when a `<Link>` would do. A link is a link: it's keyboard accessible, right-clickable, and shows its destination in the status bar. `useNavigate` is for navigation that genuinely can't be expressed as one.

Nested routes and `<Outlet>`

Most apps have pieces that stay put across pages — a header, a nav bar, a footer. Nesting routes expresses that directly:

```

<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<Home />} />
    <Route path="books" element={<BookList />} />
    <Route path="books/:id" element={<BookDetails />} />
    <Route path="*" element={<NotFound />} />
  </Route>
</Routes>

```

```

import { Outlet } from "react-router";

function Layout() {
  return (
    <div className="app">
      <Header />
      <main>
        <Outlet />      {/* the matched child route renders here */}
      </main>
      <Footer />
    </div>
  );
}

```

`<Outlet />` is the hole the child route's element drops into. `Layout` renders on every page; only the part inside `<Outlet>` changes. That means the header genuinely isn't re-created when you navigate — any state it holds survives, which a plain `<a>` could never manage.

Three details in that route table:

- **Child paths are relative.** `path="books"`, not `path="/books"`, because it's nested inside `path="/"`. A leading slash here is a common source of "why does nothing match?"
- `index` marks the child that shows when the parent's path matches exactly — `/` here.
- `path="*" matches anything nothing else caught. That's your 404 page, and every app should have one.`

Query strings

For state that belongs in the URL but isn't a path — filters, search terms, page numbers:

```
import { useSearchParams } from "react-router";

function BookList() {
  const [searchParams, setSearchParams] = useSearchParams();

  const query = searchParams.get("q") ?? "";
  const category = searchParams.get("category") ?? "all";

  function handleQueryChange(next: string) {
    setSearchParams({ q: next, category });
  }
  // ...
}
```

This reads and writes like `useState`, but the value lives in the URL: `/books?q=dune&category=scifi`.

That's a genuine upgrade over `useState` for anything a user might want to **share or bookmark**. Think back to the [Recipe Finder](#) — the search term and category filter were plain state, so you could never send someone a link to "chicken recipes in the Seafood category." Moving them into the URL fixes that with about the same amount of code.

`searchParams.get()` returns `string | null`, so `?? ""` gives you a sensible default.

Route order doesn't matter (much)

Unlike older routing libraries, v7 ranks routes by **specificity**, not by the order you wrote them. `/books/new` wins over `/books/:id` for the URL `/books/new`, even if `:id` is listed first. That removes a whole category of confusing bugs — but it's still clearest to write more specific routes first, for human readers.

A note on the bigger React Router

React Router v7 can do considerably more than this chapter shows: **loaders** that fetch a route's data before it renders, **actions** that handle form submissions, and a full framework mode with server rendering and file-based routing. That's a large topic, and it's most of what the library's own documentation is about now.

What this chapter teaches is called **declarative mode** — routes as components, data fetched with the hooks you already know from [chapter 18](#). It's completely legitimate, widely used, and the right place to start. If you later meet a codebase using `createBrowserRouter` with `loader` functions, you'll recognise the route shapes; the difference is *where the fetching happens*.

Common mistakes

1. Using `<a href>` instead of `<Link to>`

```
<a href="/books">Books</a>    // ❌ full page reload, all state lost
```

The single most common React Router mistake. You'll see a flash and your app restarts. Reserve `<a>` for external links.

2. Forgetting `<BrowserRouter>`

```
useNavigate() may be used only in the context of a <Router> component.
```

Every router hook and component needs `BrowserRouter` somewhere above it. Wrap the app once, in `main.tsx`.

3. A leading slash on a nested route's path

```
<Route path="/" element={<Layout />}>
  <Route path="/books" element={<BookList />} />    // ⚠️ absolute, not relative
</Route>
```

Child paths are relative to the parent. Write `path="books"`.

4. Assuming `useParams` gives you a string

```
const { id } = useParams();
fetchBook(id);
// ❌ Argument of type 'string | undefined' is not assignable to parameter of type 'string'.
```

Always `string | undefined`. Check it, or narrow it, before using it.

5. Forgetting the catch-all route

With no `path="*"`, a typo'd URL renders **nothing at all** — a blank page with no error, which looks exactly like a crash. Always have a 404.

6. `useNavigate` where a link belongs

```
<button onClick={() => navigate("/books")}>Books</button>    // ⚠️
```

Users can't middle-click it, can't see where it goes, and screen readers announce it as a button rather than a link. Use `<Link>` for navigation; use `navigate()` for "go there after something happened."

7. Keeping shareable state out of the URL

A search term in `useState` can't be linked to. If you'd want someone to be able to send that view to a colleague, it belongs in `useSearchParams`.

Quick recap

- Install `react-router` (v7 — not `react-router-dom`, whatever older tutorials say), and wrap your app once in `<BrowserRouter>`.
 - `<Routes>` picks the best-matching `<Route>` and renders its `element`. `path="*" is your 404.`
 - `<Link to>`, **never** `<a href>`, for internal navigation — an `<a>` reloads the whole app. `NavLink` adds an `isActive` flag for nav bars.
 - `:id` in a path captures a segment; read it with `useParams`, which always gives `string | undefined`.
 - Nest routes to share a layout, and put `<Outlet />` where the child should render. Child paths are **relative**; `index` marks the default child.
 - `useSearchParams` puts filters and search terms in the URL, so views can be shared and bookmarked.
 - `useNavigate` is for navigating after an action. For anything a user clicks to go somewhere, use a real link.
-

Next: try the [exercises](#), then move on to [25 TypeScript Patterns for React](#).

Exercises

How to do these:

- These need React Router installed: `npm install react-router` in your practice app, and `<BrowserRouter>` wrapped around `<App />` in `main.tsx`.
 - Because routing is app-wide, these exercises don't fit the usual one-folder-per-exercise pattern as neatly. Give each exercise its own folder under `src/ch24/`, and point `App.tsx` at the exercise you're working on by rendering its own `<Routes>` block.
 - **Test every exercise with the browser's back and forward buttons, and by refreshing on a deep URL.** Those are the two things this chapter exists for, and the two things it's easiest to accidentally break.
 - An exercise is done when the URLs are right, back/forward work, refreshing a deep link works, VS Code shows no red squiggles, and `npm run tsc -b` prints nothing.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Three pages, and the `<a>` trap

In `src/ch24/ex1/`, build three pages — `Home`, `About`, `Contact` — and a nav bar linking to all three.

1. Wire them up with `<Routes>` and `<Route>`, at `/`, `/about`, and `/contact`.
2. Build the nav bar with plain `<a href="...">` links **first**. Click between pages and watch the browser tab's loading spinner. Open the Network tab and watch what gets requested on each click.

3. Add a `console.log("App rendered")` at the top of `App`, and a `useState` counter somewhere in the nav bar. Click the counter a few times, then navigate with your `<a>` links. What happened to the counter?
4. Now swap every `<a>` for `<Link>`. Repeat step 3. What's different — in the Network tab, and to the counter?
5. Add a `*` route rendering a `NotFound` page. Type a nonsense URL to confirm it works.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): A layout that survives navigation

In `src/ch24/ex2/`, restructure exercise 1's pages to share a layout.

1. Build a `Layout` component with a header (containing the nav), an `<Outlet />`, and a footer.
2. Nest all three page routes inside a parent route rendering `Layout`. Use `index` for the home page.
3. Convert the nav links to `NavLink`, styling the active one differently.
4. Put a `useState` counter **in the header**. Navigate between all three pages and confirm the count doesn't reset — proving the header genuinely isn't being re-created.
5. Deliberately give one child route a leading slash (`path="/about"` instead of `path="about"`). What happens? Fix it, and write down what the symptom was.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): URL parameters and a detail page

In `src/ch24/ex3/`, build a small book browser using this data in `books.ts`:

```
export type Book = {
  id: string;
  title: string;
  author: string;
  year: number;
  blurb: string;
};

export const books: Book[] = [
  { id: "1", title: "The Hobbit", author: "J. R. R. Tolkien", year: 1937, blurb: "..."},
  { id: "2", title: "Dune", author: "Frank Herbert", year: 1965, blurb: "..."},
  { id: "3", title: "Small Gods", author: "Terry Pratchett", year: 1992, blurb: "..."},
];
```

1. `/books` lists every book's title as a `<Link>` to its detail page.

2. `/books/:id` shows one book's full details, read via `useParams`.
3. Handle the `id` being `undefined` properly — no `!`, no `as string`.
4. Handle an `id` that **doesn't match any book** (`/books/999`): show a "Book not found" message with a link back to the list, not a crash and not a blank page.
5. Add a **Back to all books** link on the detail page, and separately a **Back** button using `navigate(-1)`. Try both after arriving from different places (from the list, and by pasting the URL directly). In a comment, describe how they behave differently and when you'd use each.
6. Refresh the page while on `/books/2`. Does it still work?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Filters in the URL

Take exercise 3's book list and move its filtering into the URL, so any view can be shared.

1. Add a search box filtering by title, and a `<select>` filtering by author.
2. Both live in `useSearchParams`, **not** `useState`. The URL should read `/books?q=dune&author=all`.
3. Typing in the search box updates the URL live.
4. Copy the URL with both filters applied, open a new tab, and paste it — the filtered view should come up exactly as you left it.
5. Press the browser's **back** button after changing a filter. What happens? Is that what you'd want? Write down your answer.
6. Add a **Clear filters** button that removes the query string entirely, leaving a bare `/books`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A full mini-app with nested layouts

In `src/ch24/ex5/`, build a small account section with two levels of nested layout — the shape almost every real dashboard uses.

Routes:

```
/           → home page
/books      → book list (from exercise 3)
/books/:id  → book details
/account    → account layout, with its own sub-nav
/account    → (index) profile summary
/account/orders → order list
/account/orders/:orderId → one order's details
/account/settings → settings form
*           → 404
```

Requirements:

1. **Two nested layouts:** the app-wide `Layout` (header, footer, `<Outlet />`), and inside it an `AccountLayout` (a sidebar of account links, plus its own `<Outlet />`).
2. The account sidebar uses `NavLink` and correctly highlights the active section — including keeping "Orders" highlighted while you're on a *specific* order's page.
3. A "fake login" gate: a `useState` boolean in `App` (or better, a context from [chapter 22](#)). When logged out, visiting any `/account/*` URL shows a "Please log in" page instead, with a **Log in** button. Logging in should then show the page you originally asked for, not dump you at `/account`.
4. Refreshing on `/account/orders/7` while logged in lands you exactly there.
5. Every page is reachable by typing its URL directly, and back/forward work throughout.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

25 TypeScript Patterns for React

What is it?

Four techniques for typing components that are genuinely reusable:

```
<List items={books} renderItem={(book) => <BookCard book={book} />} /> // generic components
<Button variant="primary" onClick={...} type="submit" /> // reusing HTML props
<Alert kind="error" message="..." /> // props that depend on
each other
```

Everything up to now has typed props one component at a time, which is the right default. This chapter is about the handful of components — buttons, inputs, lists, modals — that get used *everywhere*, where a bit more type effort pays for itself many times over.

Why does it matter?

You've written plenty of props types by now, and they've all had the same shape: a fixed list of fields with fixed types. That works perfectly until you try to write something genuinely general, and then it stops:

```
type ListProps = {
  items: ???; // items of what?
  renderItem: (item: ???) => ReactNode;
};
```

There's no answer to fill in. `Book[]` makes the list only work for books. `any[]` makes it work for everything and check nothing — and then `renderItem`'s parameter is `any` too, so every typo inside it goes unnoticed. Neither is acceptable for a component you'll use fifty times.

The same wall shows up in three other places:

- A `<Button>` **wrapper** that needs to accept `onClick`, `disabled`, `type`, `aria-label` and the other forty things a real `<button>` accepts — without you typing all forty out by hand.
- A **component where one prop changes what others mean**: an `<Alert>` that needs a `retry` function when it's an error, and mustn't have one when it's a success.
- **Reusing a type you already have** rather than restating its shape and letting the two drift apart.

These patterns are how shared component libraries are built. You don't need them for a one-off card — you need them the moment something is used everywhere.

Real-world example

Think about a **shelving system** versus a **bookcase**.

A bookcase	Adjustable shelving
Built for books, specific height	<code>type ListProps = { items: Book[] }</code>
Works beautifully for books	Works beautifully for one case
Useless for shoes or files	Useless for anything but books
Buy a second one for shoes	Copy the component and change the type
Adjustable shelving: one system, any contents	Generic component: one component, any item type
Still holds things securely — it's not a pile on the floor	Still fully type-checked — it's not <code>any</code>

The important half of that last row: adjustable doesn't mean flimsy. A generic component isn't "typed loosely" — it's typed *precisely*, just with the specific type filled in per use.

How it works

Generic components

A **generic** (TypeScript chapter 08) is a type you leave as a blank, filled in at each use. Components take them the same way functions do:

```

type ListProps<T> = {
  items: readonly T[];
  renderItem: (item: T) => ReactNode;
  keyOf: (item: T) => string;
};

function List<T>({ items, renderItem, keyOf }: ListProps<T>) {
  if (items.length === 0) {
    return <p>Nothing to show.</p>;
  }

  return (
    <ul>
      {items.map((item) => (
        <li key={keyOf(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}

```

`<T>` after the component name declares the blank. Now `T` means "whatever type the items are," consistently, across all three props.

Using it, you don't specify `T` at all:

```

<List
  items={books}
  keyOf={(book) => book.id}
  renderItem={(book) => <span>{book.title} ({book.year})</span>}
/>

```

TypeScript **infers** `T` as `Book` from `items`, and then knows `book` is a `Book` inside both callbacks — with full autocomplete, and an error if you write `book.title`. The same component works for `User[]`, `Recipe[]`, anything:

```

<List items={users} keyOf={(u) => u.email} renderItem={(u) => <b>{u.name}</b>} />

```

One component, fully checked, every time.

Constraining the blank. If your component needs the items to have something in particular, say so with `extends`:

```

type ListProps<T extends { id: string }> = {
  items: readonly T[];
  renderItem: (item: T) => ReactNode;
};

function List<T extends { id: string }>({ items, renderItem }: ListProps<T>) {
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id}>{renderItem(item)}</li> // ✅ id is guaranteed to exist
      ))}
    </ul>
  );
}

```

Now `T` can be any type *that has a string `id`*, which lets you drop the `keyOf` prop entirely. Pass something without an `id` and TypeScript stops you at the call site.

A .tsx gotcha: in a `.tsx` file, `const List = <T>(props: Props<T>) => ...` fails to parse — TypeScript reads `<T>` as a JSX tag. Use a `function` declaration (as above), or write `<T,>` with a trailing comma to disambiguate. This course uses `function` declarations everywhere, which sidesteps it.

Reusing HTML props

Writing a `<Button>` wrapper that forwards everything a real button accepts:

```

type ButtonProps = {
  variant: "primary" | "secondary";
  children: ReactNode;
  onClick?: () => void;
  disabled?: boolean;
  type?: "button" | "submit" | "reset";
  // ...and aria-label, autoFocus, form, name, title, onFocus, onBlur...
};

```

This is unwinnable by hand — the list is enormous, and you'll always be missing whichever one you need next. React already has these types. Borrow them:

```
import type { ComponentPropsWithoutRef } from "react";

type ButtonProps = ComponentPropsWithoutRef<"button"> & {
  variant: "primary" | "secondary";
};

function Button({ variant, className, ...rest }: ButtonProps) {
  return (
    <button className={`btn btn-${variant} ${className ?? ""}`} {...rest} />
  );
}
```

`ComponentPropsWithoutRef<"button">` is "every prop a real `<button>` accepts." & joins it with your own additions (TypeScript chapter 04). Now this all works, fully checked, for free:

```
<Button variant="primary" type="submit" disabled aria-label="Save the form">Save</Button>
<Button variant="primary" onClick={handleClick} autoFocus>Go</Button>
<Button variant="primary" hrefTarget="_blank">Nope</Button>
// ❌ Property 'hrefTarget' does not exist on type ...
```

Two things doing real work in that component:

- `...rest` collects every prop you didn't name and spreads it onto the real `<button>` (JavaScript chapter 15). This is the one place where spreading props ([chapter 04](#) warned about it) is genuinely the right call — the whole purpose of the component is to pass them through.
- **Pulling `className` out separately** so you can merge it with your own classes rather than letting the caller's version silently replace them. Forget this, and `<Button className="mt-4">` would wipe out `btn btn-primary`.

`ComponentPropsWithoutRef` vs `ComponentProps`. Use `WithoutRef` unless you're deliberately handling a `ref`. If you *do* want the wrapper to forward a `ref`, React 19 makes that a plain prop ([chapter 19](#)), so use `ComponentProps<"button">` and pass `ref` through with everything else.

It works for your own components too: `ComponentProps<typeof RecipeCard>` gives you exactly that component's props, so a wrapper can never drift out of sync with what it's wrapping.

Props that depend on each other

Some components have props that only make sense in certain combinations:


```
type AlertProps = {
  kind: "success" | "error";
  message: string;
  onRetry?: () => void;    // ⚠️ only meaningful for errors
};
```

Nothing stops `<Alert kind="success" onRetry={...} />`, which is meaningless, or `<Alert kind="error" />` with no way to retry, which may be a bug. Comments can't enforce it. A **discriminated union** can — the same tool as [chapter 23](#)'s actions, applied to props:

```
type AlertProps =
  | { kind: "success"; message: string }
  | { kind: "error"; message: string; onRetry: () => void };

function Alert(props: AlertProps) {
  if (props.kind === "error") {
    return (
      <div className="alert error">
        {props.message}
        <button onClick={props.onRetry}>Try again</button>
      </div>
    );
  }

  return <div className="alert success">{props.message}</div>;
}
```

Now the rules are enforced at every call site:

```
<Alert kind="error" message="Failed" onRetry={handleRetry} />    // ✓
<Alert kind="success" message="Saved" />                          // ✓
<Alert kind="error" message="Failed" />
// ✗ Property 'onRetry' is missing in type ... but required in type '{ kind: "error"; ... }'.
<Alert kind="success" message="Saved" onRetry={handleRetry} />
// ✗ Object literal may only specify known properties, and 'onRetry' does not exist in type '{
kind: "success"; message: string; }'.
```

Note the parameter isn't destructured here. `function Alert(props: AlertProps)` rather than `function Alert({ kind, message, onRetry }: AlertProps)` — because narrowing needs `props.kind` checked as a whole. Destructure up front and TypeScript loses track of the connection between `kind` and the rest. Destructure *inside* each branch if you want to, once the narrowing has happened.

`satisfies`, for checking without widening

A newer operator worth knowing (TypeScript chapter 02 covers literal types). Compare:

```
const variants: Record<string, string> = {
  primary: "btn-blue",
  secondary: "btn-grey",
};
variants.primry;    // no error – Record<string, string> allows any key
```

```
const variants = {
  primary: "btn-blue",
  secondary: "btn-grey",
} satisfies Record<string, string>;

variants.primry;
// ✗ Property 'primry' does not exist on type '{ primary: string; secondary: string; }'.
```

`satisfies` **checks** the value against a type without **replacing** the value's own, more specific type. You get the validation of the annotation and the precision of inference at once — very useful for the lookup objects from [chapter 05](#).

Deriving types instead of restating them

When two types must agree, derive one from the other so they can't drift:

```
type ButtonProps = {
  variant: "primary" | "secondary" | "outline";
  size: "small" | "large";
};

// ✓ derived – add a variant and this updates itself
const variantClasses: Record<ButtonProps["variant"], string> = {
  primary: "btn-blue",
  secondary: "btn-grey",
  outline: "btn-outline",
};

// ✗ restated – add a variant and this silently goes stale
const variantClasses: Record<"primary" | "secondary" | "outline", string> = { ... };
```

`ButtonProps["variant"]` is an **indexed access type**: "whatever type that property has." Add `"danger"` to `ButtonProps` and the derived version immediately errors with "Property 'danger' is missing," pointing straight at the thing you forgot.

The same idea, with `typeof`, works from a value:

```
const ROLES = ["admin", "editor", "viewer"] as const;
type Role = (typeof ROLES)[number];    // "admin" | "editor" | "viewer"
```

`as const` makes the array's contents literal types rather than `string[]`, and `[number]` means "the type of any element." Now one list drives both the runtime values (for a `<select>`, say) and the type — they cannot disagree.

Typing children precisely

`children: ReactNode` is right nearly always. Two variations worth knowing:

```
type Props = {
  children: ReactNode;           // anything renderable – the default
};

type Props = {
  children: (item: Item) => ReactNode;  // a function – "render props"
};
```

The second is a component that hands data *back* to whatever's inside it:

```
<DataLoader url="/api/books">
  {(books) => <List items={books} renderItem={(b) => b.title} />}
</DataLoader>
```

You'll meet this in older libraries especially. Custom hooks ([chapter 20](#)) have replaced it for most purposes — `const books = useBooks(url)` is plainly easier to read — but recognise it when you see it.

A worked example: a typed, reusable `Select`

Putting three of these together:

```

import type { ComponentPropsWithoutRef, ReactNode } from "react";

type SelectProps<T> = Omit<ComponentPropsWithoutRef<"select">, "onChange" | "value"> & {
  label: string;
  options: readonly T[];
  value: T;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getValue: (option: T) => string;
};

function Select<T>({
  label,
  options,
  value,
  onChange,
  getLabel,
  getValue,
  ...rest
}: SelectProps<T>) {
  return (
    <label>
      {label}
      <select
        value={getValue(value)}
        onChange={(event) => {
          const found = options.find((o) => getValue(o) === event.target.value);
          if (found) onChange(found);
        }}
        {...rest}
      >
        {options.map((option) => (
          <option key={getValue(option)} value={getValue(option)}>
            {getLabel(option)}
          </option>
        ))}
      </select>
    </label>
  );
}

```

`Omit<..., "onChange" | "value">` (TypeScript chapter 09) removes the native versions of those two props, because this component replaces them with its own, better-typed versions — the native `onChange` hands you an event with a string; this one hands you the actual `T` you passed in.

```

<Select
  label="Author"
  options={authors}
  value={selectedAuthor}
  onChange={setSelectedAuthor}    // receives an Author, not a string
  getLabel={(a) => a.name}
  getValue={(a) => a.id}
  disabled={isLoading}           // a real <select> prop, still available
/>

```

Don't do this everywhere

A closing caution, because this chapter is easy to over-apply. **Most components should have a plain, boring props type.** A `<RecipeCard>` used in one place gains nothing from being generic, and a lot of clever typing makes code harder to read and much harder to fix when it goes wrong.

The signal for reaching into this chapter is **reuse across unrelated things**: you're writing your third nearly-identical list, or you're building a component every screen in the app will use. Until then, `type Props = { ... }` is the right answer.

Common mistakes

1. `any[]` instead of a generic

```
type ListProps = { items: any[]; renderItem: (item: any) => ReactNode };
```

Compiles, checks nothing, and silently makes every callback parameter unchecked too. A generic is barely more typing and keeps everything checked.

2. Arrow function generics in a `.tsx` file

```
const List = <T>(props: ListProps<T>) => { ... };
// ❌ parsed as JSX
```

Use a `function` declaration, or `<T,>` with a trailing comma.

3. Destructuring props before narrowing a discriminated union

```
function Alert({ kind, message, onRetry }: AlertProps) {
  if (kind === "error") {
    onRetry();    // ✗ TypeScript no longer knows onRetry exists here
  }
}
```

Take the whole `props` object, narrow on `props.kind`, then destructure inside the branch.

4. Forgetting to merge `className` in a wrapper

```
function Button({ variant, ...rest }: ButtonProps) {
  return <button className={`btn btn-${variant}`} {...rest} />;
}
```

`...rest` includes `className` if the caller passed one — and because it comes **after** your `className`, it silently overwrites yours entirely. Pull it out and merge it deliberately.

5. Restating a union instead of deriving it

Two copies of `"primary" | "secondary" | "outline"` will drift apart. `ButtonProps["variant"]` can't.

6. Reaching for `ComponentProps` when you just need a few props

If your component only ever needs `onClick` and `disabled`, listing them is clearer than inheriting forty. Use the HTML-props trick for genuine pass-through wrappers, not everywhere.

7. Making everything generic

A generic with only ever one concrete type isn't buying anything — it's just harder to read. Wait for the second real use case.

Quick recap

- A **generic component** (`function List<T>(props: ListProps<T>)`) works with any item type while staying fully checked. TypeScript infers `T` from the props at each call site.
- Constrain it with `extends` when your component needs the items to have something — `T extends { id: string }`.
- `ComponentPropsWithoutRef<"button">` gives you every prop a real element accepts. Join it with `&`, spread the `...rest`, and remember to merge `className` rather than let it be overwritten.
- **Discriminated union props** make impossible prop combinations un-writable. Don't destructure the parameter, or narrowing won't work.
- `satisfies` checks a value against a type without widening it — ideal for lookup objects.

- **Derive types rather than restating them:** `ButtonProps["variant"]`, or `(typeof ROLES)[number]` with `as const`.
- In a `.tsx` file, write generic components as `function` declarations — arrow-function generics collide with JSX syntax.
- Save all of this for components used **across unrelated things**. A plain props type is the right default.

Next: try the [exercises](#), then move on to [26 Error Boundaries and Suspense](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch25/ex1/`, and so on.
- **The test for every exercise here is the error message, not the working case.** For each one, deliberately write the wrong call and check that TypeScript stops you with a *useful* message. A type that doesn't catch mistakes isn't doing its job.
- No `any`, no `as`, and no `@ts-ignore` anywhere in this chapter. If you're reaching for one, the type needs rethinking.
- An exercise is done when the good calls compile, the bad calls are caught, `npx tsc -b` prints nothing, and the page works.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A generic `List`

In `src/ch25/ex1/`, build the generic `List` component from the notes, then prove it's genuinely generic.

1. `List<T>` takes `items: readonly T[]`, `renderItem: (item: T) => ReactNode`, and `keyOf: (item: T) => string`. It shows `Nothing to show.` for an empty array.
2. Use it **three** times on one page with three unrelated types: a list of books (objects), a list of plain `string` tags, and a list of `number`s.
3. In each `renderItem`, hover over the parameter and confirm TypeScript knows exactly what it is — `Book`, `string`, `number` — with no annotations from you.
4. Break it on purpose and write down each error:
 - misspell a field inside `renderItem` (`book.titel`)
 - pass `keyOf` a function returning a `number` instead of a `string`
 - pass `items={books}` but write `renderItem={({user: User}) => ...}`

5. Now add a constrained version, `IdList<T> extends { id: string }>`, that drops `keyOf` entirely. Use it with your books, then try to use it with the string array. What's the error, and is it a good one?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): A `Button` that accepts everything

In `src/ch25/ex2/`, build the `Button` wrapper from the notes.

1. `ButtonProps` is `ComponentPropsWithoutRef<"button">` joined with your own `variant: "primary" | "secondary" | "danger"` and `size?: "small" | "large"`.
2. The component merges its own classes with any `className` the caller passes, and spreads the rest.
3. Render at least six buttons using a mix of your props and native ones: `type="submit"`, `disabled`, `onClick`, `aria-label`, `autoFocus`, `title`, and a custom `className`.
4. **Prove the `className` merge works:** render `<Button variant="primary" className="mt-4">` and check in the Elements tab that *both* `btn-primary` and `mt-4` are on the element. Then deliberately remove your `className` handling (leaving it in `...rest`) and confirm your own classes vanish.
5. Try `<Button variant="primary" hrefTarget="_blank">`. Write down the error.
6. Add a `<Card>` component, then type a `<FeaturedCard>` wrapper using `ComponentProps<typeof Card>` so it can never drift out of sync with `Card`'s props. Add a prop to `Card` and confirm `FeaturedCard` picks it up with no edit.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Props that can't be wrong

In `src/ch25/ex3/`, build three components whose prop combinations are enforced by the types.

1. **Alert** — as in the notes: `kind: "success"` needs only a message; `kind: "error"` also requires `onRetry`.
2. **Avatar** — either an image or initials, never both, never neither:

```
<Avatar imageUrl="/maya.jpg" name="Maya" /> // ✓
<Avatar initials="MS" name="Maya" /> // ✓
<Avatar name="Maya" /> // ✗
<Avatar imageUrl="/maya.jpg" initials="MS" name="Maya" /> // ✗
```

3. **Field** — a form field where the `type` determines what else is allowed:


```

<Field type="text" label="Name" value={name} onChange={setName} /> // ✓
<Field type="select" label="Size" value={size} onChange={setSize}
  options=[["S", "M", "L"]] /> // ✓
<Field type="text" label="Name" value={name} onChange={setName}
  options=[["S"]] /> // ✗ options
is select-only
<Field type="select" label="Size" value={size} onChange={setSize} /> // ✗ options
is required

```

For each, write out the good and bad calls above in your test page, confirm the bad ones are caught, and record the error messages in a comment.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Derive, don't restate

In `src/ch25/ex4/`, build a small status-badge system that makes drift impossible.

1. Define the statuses **once**, as a `const` array with `as const`:

```
export const STATUSES = ["draft", "review", "published", "archived"] as const;
```

2. Derive the type from it: `type Status = (typeof STATUSES)[number]`.
3. Build a `StatusBadge` component taking `status: Status`, using a lookup object for label and colour, typed with `Record<Status, StatusStyle>` **and** written with `satisfies` so the object's own keys stay precise.
4. Build a `<StatusFilter>` `<select>` whose options are generated by mapping over `STATUSES` — so the runtime list and the type come from the same place.
5. **Now add a fifth status, "deleted", to the array only.** Save, and write down every error TypeScript gives you and which file each points at. Fix them all.
6. Do the same experiment on a "restated" version where `Status` is written out by hand as a union and the array is written out separately. Add a sixth status to just one of them. How many errors do you get, and how long would that bug have survived?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): A typed data table

Build the component every app eventually needs, in `src/ch25/ex5/`: a generic, fully typed `<DataTable>`.

```
<DataTable
  items={books}
  getRowId={(book) => book.id}
  columns=[
    { header: "Title", render: (book) => book.title, sortBy: (book) => book.title },
    { header: "Year", render: (book) => book.year, sortBy: (book) => book.year },
    { header: "Author", render: (book) => book.author },
  ]
/>
```

Requirements:

1. `DataTable<T>` takes `items: readonly T[]`, `getRowId: (item: T) => string`, and `columns`.
2. A column is `{ header: string; render: (item: T) => ReactNode; sortBy?: (item: T) => string | number }`. Give it its own exported `Column<T>` type.
3. Inside `render` and `sortBy`, the parameter must be correctly typed as `T` with **no annotation from the caller**.
4. Columns with a `sortBy` get a clickable header that sorts by it; clicking again reverses. Columns without one are not clickable at all.
5. Sorting never mutates `items` ([chapter 11](#)).
6. Show `No rows.` for an empty array.
7. Use it **twice** on one page with two unrelated types, to prove it's really generic.

Then, the type-level tests — write each of these and record what happens:

8. A column whose `render` reads a field that doesn't exist on `T`.
9. `sortBy` returning a `boolean`.
10. `items={books}` with a column written as `render: (user: User) => user.name`.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

26 Error Boundaries and Suspense

What is it?

Two features for the two things that go wrong around a component: it **crashed**, or it **isn't ready yet**.

```
<ErrorBoundary fallback={<p>Something went wrong.</p>}>
  <Suspense fallback={<Spinner />}>
    <Dashboard />
  </Suspense>
</ErrorBoundary>
```

An **error boundary** catches a crash anywhere below it and shows something sensible instead of a blank page. **Suspense** shows a fallback while something below it is still loading.

Both work the same way conceptually: you wrap a part of your tree and say what to show when that part can't show itself.

Why does it matter?

Try this in any app you've built: make a component throw.

```
function RecipeCard({ recipe }: RecipeCardProps) {
  return <h3>{recipe.name.toUpperCase()}</h3>; // recipe.name is undefined for one item
}
```

The whole page goes **blank**. Not that card — everything. The header, the nav, the search box you spent a chapter building: gone, replaced by nothing, with an error in the Console that your users will never see.

That's React working as designed, and the reasoning is sound: a component that threw mid-render has left React unable to know what the UI should look like, and showing a half-updated, possibly wrong interface is worse than showing none. A banking app displaying a stale balance is more dangerous than one showing an error.

But "blank page" is not an acceptable final answer. What you want is for the *broken part* to fail and everything else to carry on — one card showing "Couldn't load this recipe" while the other eleven work fine. That's an error boundary.

Suspense solves the mirror-image problem. You've been writing `if (isLoading) return <Spinner />` in every single data-fetching component since [chapter 18](#). Suspense lets a parent declare the loading UI **once**, for a whole section, and it's the foundation for code-splitting — not shipping your entire app's JavaScript to someone who only opened the home page.

Real-world example

Think about **electrical circuits in a house**.

Your house	React
One fuse for the whole house	No error boundary: any crash kills everything
A toaster shorts out, the whole house goes dark	One component throws, the page goes blank
Separate circuits per room	An error boundary around each section
The kitchen trips; the lights upstairs stay on	The broken widget shows a message; the rest works
A fuse you can reset	An error boundary with a "Try again" button
Where you put the fuses is a design decision	Where you put boundaries is a design decision

Nobody wires a house with one fuse. The interesting question isn't *whether* to have boundaries, it's how finely to divide them — which is exactly the judgement call this chapter is about.

How it works

Error boundaries are class components

Here's the one genuinely awkward thing in this chapter: **there is no hook for this**. Error boundaries must be class components, because they rely on two lifecycle methods that have no hook equivalent.

This is the only place in modern React where you need a class. You don't need to learn class components properly (JavaScript chapter 27 covers the syntax) — you need to be able to copy this one, and understand what it does:

```
import { Component, type ReactNode, type ErrorInfo } from "react";

type ErrorBoundaryProps = {
  children: ReactNode;
  fallback: ReactNode;
};

type ErrorBoundaryState = {
  hasError: boolean;
};

export class ErrorBoundary extends Component<ErrorBoundaryProps, ErrorBoundaryState> {
  state: ErrorBoundaryState = { hasError: false };

  static getDerivedStateFromError(): ErrorBoundaryState {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error("Caught by boundary:", error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback;
    }
    return this.props.children;
  }
}
```

The three parts:

- **getDerivedStateFromError** runs when a child throws. Whatever it returns becomes the new state — here, flipping **hasError** to **true**. It must be pure; it's only for deciding what to show.
- **componentDidCatch** runs afterwards and *is* allowed side effects. This is where you'd report the error to a logging service in a real app.
- **render** shows the fallback if something broke, and the children otherwise.

Use it like any other component:

```
<ErrorBoundary fallback={<p>Couldn't load the recipes.</p>}>
  <RecipeGrid recipes={recipes} />
</ErrorBoundary>
```

Now a throw inside **RecipeGrid** shows that one message, and the header, nav and search box carry on working.

What boundaries do and don't catch

This trips everyone up, so be precise. An error boundary catches errors thrown **while rendering** a component below it — during render, in a lifecycle method, or in a constructor.

It does **not** catch:

Not caught	Why, and what to do instead
Errors in event handlers	The handler runs long after render. Use <code>try / catch</code> inside it.
Errors in <code>setTimeout</code> or promises	Same reason — outside React's render cycle. <code>try / catch</code> , or <code>.catch()</code> .
Errors in the boundary itself	It can't catch itself. That's why boundaries stay tiny.
Errors during server rendering	Different mechanism (chapter 39).

The event handler exclusion is the one that catches people out:

```
<ErrorBoundary fallback={<p>Oops</p>}>
  <button onClick={() => { throw new Error("boom"); }}>Click</button>
</ErrorBoundary>
```

Clicking that does **not** show the fallback — the error just goes to the Console. Your fetching code from [chapter 18](#) already handles its own errors with `.catch()` and an `error` state, which is exactly right and is *not* something boundaries replace.

Resetting a boundary

A boundary that's tripped stays tripped. Give people a way out:

```

type ErrorBoundaryProps = {
  children: ReactNode;
  fallback: (reset: () => void) => ReactNode;
};

export class ErrorBoundary extends Component<ErrorBoundaryProps, ErrorBoundaryState> {
  state: ErrorBoundaryState = { hasError: false };

  static getDerivedStateFromError(): ErrorBoundaryState {
    return { hasError: true };
  }

  reset = () => {
    this.setState({ hasError: false });
  };

  render() {
    if (this.state.hasError) {
      return this.props.fallback(this.reset);
    }
    return this.props.children;
  }
}

```

```

<ErrorBoundary
  fallback={(reset) => (
    <div>
      <p>Couldn't load the recipes.</p>
      <button onClick={reset}>Try again</button>
    </div>
  )}
>
  <RecipeGrid recipes={recipes} />
</ErrorBoundary>

```

That `fallback` taking a function is the **render prop** pattern from [chapter 25](#) — the boundary hands `reset` back out to whatever's being rendered.

A useful trick: giving a boundary a `key` tied to something that changes resets it automatically. `<ErrorBoundary key={recipeId}>` starts fresh whenever the recipe changes, which is usually what you want — the same key mechanism as [chapter 06](#), used deliberately.

Use the library

Writing that class yourself once is worth doing, so you know what's in it. After that, use [react-error-boundary](#):

```
npm install react-error-boundary
```

```
import { ErrorBoundary } from "react-error-boundary";

function Fallback({ error, resetErrorBoundary }: FallbackProps) {
  return (
    <div role="alert">
      <p>Something went wrong: {error.message}</p>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}

<ErrorBoundary FallbackComponent={Fallback} onReset={() => refetch()}>
  <RecipeGrid />
</ErrorBoundary>
```

It handles resetting, `onError` reporting, resetting on prop changes, and a `useErrorBoundary` hook for pushing async errors into a boundary deliberately. It's small, it's the community standard, and it means you never write that class again.

Where to put boundaries

This is a design decision, and both extremes are wrong.

One boundary at the root catches everything, but the fallback is the whole app — one broken widget still takes the page down, just with a nicer message.

A boundary around every component means fallbacks everywhere, and a page that degrades into a patchwork of error messages.

A sensible default is **one per meaningful section of the page**:


```

<Layout>
  <ErrorBoundary fallback={<p>Navigation unavailable.</p>}>
    <Sidebar />
  </ErrorBoundary>

  <ErrorBoundary fallback={<p>Couldn't load your recipes.</p>}>
    <RecipeGrid />
  </ErrorBoundary>

  <ErrorBoundary fallback={<p>Recommendations unavailable.</p>}>
    <Recommendations />
  </ErrorBoundary>
</Layout>

```

Ask: **"if this broke, what should the user still be able to do?"** Anything that must keep working belongs outside the boundary. With routing ([chapter 24](#)), one boundary per route is a good starting point — a crash in one page leaves the nav working so people can go somewhere else.

And keep a root-level boundary as a last resort, so the worst case is a polite message rather than a blank screen.

Suspense

`<Suspense>` shows a fallback while something inside it isn't ready:

```

import { Suspense } from "react";

<Suspense fallback={<p>Loading...</p>}>
  <Dashboard />
</Suspense>

```

Note the parallel with error boundaries — wrap a section, declare what to show when it can't show itself. That's not a coincidence; they're designed as a pair.

The catch is what "isn't ready" means. Suspense only works with things that know how to talk to it, and your `useEffect`-based fetching from [chapter 18](#) **is not one of them**. Wrapping it in `<Suspense>` does nothing at all. Today, the things that work with Suspense are:

- `React.lazy`, for code-splitting — covered next, and available to you right now.
- **Frameworks and data libraries that opt in** — Next.js ([chapter 39](#)), TanStack Query's suspense mode ([chapter 31](#)).
- **The `use hook`** with a promise, in React 19 — genuinely powerful, but the ergonomics around caching the promise are still settling, so this course doesn't build on it.

So for now: use `Suspense` for lazy loading, and keep your own `isLoading` state for your own fetches. That's not a workaround, it's the current correct answer.

Code splitting with `React.lazy`

This is where `Suspense` earns its place in a normal app today.

By default, Vite bundles your whole app into one JavaScript file. Someone who opens the home page downloads the admin dashboard, the settings page, and every chart library, before seeing anything. `lazy` splits a component into its own file, fetched only when it's actually rendered:

```
import { lazy, Suspense } from "react";

const AdminDashboard = lazy(() => import("./AdminDashboard.tsx"));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/admin" element={<AdminDashboard />} />
      </Routes>
    </Suspense>
  );
}
```

`import()` with brackets is a **dynamic import** (JavaScript chapter 29) — it fetches a module at runtime and returns a promise. `lazy` wraps that so React can render it like any component, suspending while it downloads.

Two requirements: the lazily-loaded component must be a **default export**, and there must be a `<Suspense>` somewhere above it.

Routes are the natural place to split. Each page becomes its own chunk, downloaded when someone navigates there. Run `npm run build` before and after and compare the output — you'll see one large file become several smaller ones.

Don't split *everything*, though. Each split is an extra network request, and splitting a 2KB component makes things slower, not faster. Split at route level and around genuinely heavy things — a chart library, a rich text editor, a map.

Boundaries and Suspense together

A lazily-loaded component can fail to download — a flaky connection, a deploy that removed the old chunk. That's a render-time error, so a boundary catches it:

```

<ErrorBoundary fallback={<p>Couldn't load this page. Check your connection.</p>}>
  <Suspense fallback={<PageSkeleton />}>
    <AdminDashboard />
  </Suspense>
</ErrorBoundary>

```

Error boundary outside, Suspense inside. That order matters: the boundary needs to be able to catch failures from the Suspense boundary itself.

Common mistakes

1. Expecting a boundary to catch an event handler error

```

<button onClick={() => { throw new Error("boom"); }} /> // boundary never fires

```

Boundaries catch render errors only. Use `try / catch` in handlers, and keep your `error` state for fetches.

2. Writing an error boundary as a function component

There's no hook for this. It has to be a class, or you use `react-error-boundary`. This is the one exception in modern React.

3. One boundary at the root and nothing else

Better than nothing, but one broken widget still replaces the entire app. Add boundaries per section, or per route.

4. A fallback that can itself throw

```

<ErrorBoundary fallback={<p>{error.details.message}</p>}> // ⚠️

```

If the fallback throws, the boundary can't catch it — it propagates to the *next* boundary up, or blanks the page. Keep fallbacks dead simple.

5. Wrapping `useEffect`-based fetching in `<Suspense>` and expecting it to work

```

<Suspense fallback={<Spinner />}>
  <RecipeGrid />      {/* fetches with useEffect – Suspense does nothing here */}
</Suspense>

```

No error, no fallback, no effect whatsoever. Suspense only works with things that opt into it.

6. A lazy component that isn't a default export

```
const Admin = lazy(() => import("./Admin.tsx")); // ❌ if Admin.tsx uses a named export
```

Either make it the default export, or map it: `lazy(() => import("./Admin.tsx").then((m) => ({ default: m.Admin })))`.

7. Splitting everything

Every `lazy` is an extra request. Split routes and genuinely heavy dependencies, not small components.

8. Testing your boundary only in development

Vite's dev server shows a full-screen error overlay on top of your boundary's fallback — which makes it look like the boundary isn't working. Dismiss the overlay (press Escape or click its close button) to see what users would actually get, or test with a production build.

Quick recap

- An **error boundary** catches a crash in the components below it and shows a fallback, instead of React blanking the whole page.
- It must be a **class component** — the one remaining place modern React needs one. Write it once to understand it, then use `react-error-boundary`.
- Boundaries catch **render** errors only. Not event handlers, not timeouts, not promises — those still need `try / catch` and your own `error` state.
- Place boundaries **per section or per route**, asking "if this broke, what should still work?" Keep a root one as a last resort.
- Give people a way to recover: a `reset` function, or a `key` that changes.
- `<Suspense>` shows a fallback while something below it isn't ready, but only for things that opt in — today, mainly `React.lazy`.
- `lazy` + **dynamic** `import()` splits a component into its own downloaded chunk. Split at route level and around heavy dependencies, not everywhere.
- Put the **error boundary outside** and **Suspense inside**, so failed downloads get caught.

Next: try the [exercises](#), then move on to [27 Performance](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch26/ex1/`, and so on.

- **Vite's dev overlay will get in your way.** When a component throws, Vite covers the screen with a red error panel *on top of* your boundary's fallback. Press Escape or click its × to dismiss it and see what a user would actually see. If in doubt, run `npm run build && npm run preview` to test without it.
- An exercise is done when the broken part fails gracefully, the rest of the page still works, VS Code shows no red squiggles, and `npx tsc -b` prints nothing.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Watch it go blank, then fix it

In `src/ch26/ex1/`, build a page with three clearly separate sections: a `Header` with a working `useState` counter, a `BrokenWidget`, and a `Footer`.

```
function BrokenWidget() {
  const user = { name: undefined } as { name: string | undefined };
  return <h3>{user.name!.toUpperCase()}</h3>; // throws
}
```

1. Render all three with **no** error boundary. Dismiss the Vite overlay. What's left on the page? Increment the header counter first, then trigger the crash — what happened to the count?
2. Write the `ErrorBoundary` class from the notes yourself, by hand. Don't copy-paste it; type it out, and make sure you can say what each of the three parts does.
3. Wrap **only** `BrokenWidget` in it. Reload. Confirm the header (with a working counter) and footer are completely unaffected.
4. Now move the boundary to wrap all three. Trigger the crash again. What's the difference, and which version is better here?
5. Add a `componentDidCatch` that logs the error and the component stack. Read the stack in the Console — can you tell which component threw?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): What boundaries don't catch

In `src/ch26/ex2/`, build a page wrapped in your `ErrorBoundary` containing four buttons, each throwing an error a different way:

1. **During render** — a component that throws when a `shouldCrash` state is `true`.
2. **In an event handler** — `onClick={() => { throw new Error("handler"); }}`.
3. **In a `setTimeout`** — `onClick={() => setTimeout(() => { throw new Error("timeout"); }, 100)}`.

4. In a **rejected promise** — `onClick={() => { void Promise.reject(new Error("async")); }}`.

Click each one and record, in a comment: did the boundary catch it? What did the user see? What appeared in the Console?

5. Then fix numbers 2, 3 and 4 so the user gets a proper message rather than nothing — using `try / catch` and an `error` piece of state, the way [chapter 18](#) already taught you.

6. Write one sentence explaining the actual rule for what boundaries catch, in your own words.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Sectioned boundaries with recovery

In `src/ch26/ex3/`, build a dashboard with four independent panels: `SalesChart`, `RecentOrders`, `Inventory`, and `Notifications`.

1. Give each panel a **Break me** button that sets a state flag causing that panel to throw on its next render.
2. Wrap each panel in its own boundary with a fallback naming that panel: `Couldn't load Recent Orders`.
3. Break two panels. Confirm the other two still work completely normally, including their own state.
4. Extend your `ErrorBoundary` so `fallback` is a function receiving a `reset` callback, and add a **Try again** button to each fallback.
5. Make **Try again** genuinely work: after resetting, the panel should render normally again — which means the panel's "should I crash" flag has to be reset too. Think carefully about where that flag lives and how the boundary can clear it.
6. Add a root-level boundary as well, wrapping the whole dashboard, with a different message. Deliberately throw inside one of your *fallbacks* and confirm the root boundary catches it.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Code splitting, measured

This exercise is about the build output as much as the page.

In `src/ch26/ex4/`, build a small three-page app with React Router ([chapter 24](#)): `Home` (tiny), `Reports` (imports a genuinely heavy dependency — install `chart.js` or similar, or simulate it with a file containing a large exported constant array), and `Settings` (tiny).

1. Build it with normal, static imports first. Run `npm run build` and write down the size of the main JS chunk from the output.

2. Convert `Reports` to `lazy(() => import("./Reports.tsx"))`, with a `<Suspense fallback={...}>` above your `<Routes>`.
3. Run `npm run build` again. Write down the new main chunk size, and note the new separate chunk. How much smaller is the initial download?
4. Run `npm run preview`, open the Network tab, and navigate to `/reports`. Watch the extra chunk being fetched at that moment. Throttle to Slow 3G to actually see the Suspense fallback.
5. Convert `Reports` to use a **named** export instead of a default one and see what breaks. Fix it without changing the export back.
6. Wrap the lazy route in an error boundary too, and confirm the nesting order is boundary-outside, Suspense-inside.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Retrofit the Recipe Finder

Go back to your [chapter 21](#) Recipe Finder and make it genuinely robust.

1. Install `react-error-boundary` and use it rather than your hand-written class.
2. Add a boundary around the **results grid**, so a bad recipe card can't take down the search box. Prove it: make `RecipeCard` throw when the recipe's name contains a particular letter, search for something that matches, and confirm you can still type a new search.
3. Add a separate boundary around the **detail modal**, with a fallback offering a **Close** button — so a broken recipe doesn't trap the user behind a modal they can't dismiss.
4. Use `onReset` to actually refetch when the user clicks **Try again**, rather than just clearing the error state and rendering the same broken thing.
5. Code-split the modal: it's only needed once someone clicks a card, so `lazy` it with a `<Suspense>` fallback. Confirm in the Network tab that its chunk isn't downloaded until the first card is clicked.
6. Add a root boundary with a "Reload the app" button as a genuine last resort.

Then answer, in a comment: your fetch error handling from chapter 21 (the `error` state and the message it renders) still exists alongside all of this. Should it? What does each one cover that the other doesn't?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

27 Performance

What is it?

Three tools for telling React to skip work it would otherwise redo:

```
const MemoedCard = memo(Card);           // skip re-rendering a
component
const sorted = useMemo(() => sort(items), [items]); // skip recalculating a value
const handleClick = useCallback(() => select(id), [id]); // keep a function the same
between renders
```

And, far more importantly, the judgement about **when to use any of them**, which is: much less often than you'd think, and never without measuring first.

Why does it matter?

[Chapter 12](#) established that when a component re-renders, everything inside it re-renders too — by default, regardless of whether its props changed. That's usually completely fine. Re-running a function that returns some JSX is cheap, and React only touches the DOM where things actually differ.

It stops being fine in specific, identifiable situations:

- A list of 500 rows re-rendering on every keystroke in a search box.
- An expensive calculation — sorting, filtering, parsing — redone on every render when its inputs haven't changed.
- A component tree deep enough that "cheap × many" stops being cheap.

The trouble is that these tools are easy to apply *everywhere*, and doing so makes things **worse**: `memo` adds a comparison on every render, `useMemo` and `useCallback` add bookkeeping and hold onto memory, and all three make code harder to read. Applied blindly, you pay the cost everywhere and get the benefit almost nowhere.

So this chapter is at least as much about restraint and measurement as it is about the three functions.

Real-world example

Think about **traffic management in a city**.

City traffic	React performance
Traffic flows fine most of the day	Most re-renders are too cheap to notice

City traffic	React performance
One junction jams at rush hour	One component is slow, under specific conditions
Counting cars to find the real bottleneck	The React DevTools Profiler
Building a flyover at the bad junction	<code>memo</code> / <code>useMemo</code> where it's needed
Building flyovers at every junction	<code>memo</code> everywhere
Cost a fortune, made the city uglier, fixed nothing	Slower, noisier code, no measurable gain
"The jam is actually a broken traffic light"	The real problem is usually a design mistake, not a missing <code>memo</code>

That last row is the one to carry with you. Most React performance problems aren't solved by memoisation — they're solved by fixing something structurally wrong, like state sitting far higher in the tree than it needs to.

How it works

Measure first, always

Before reaching for any of these, find out what's actually slow.

Install the React DevTools Profiler (part of the same extension from [chapter 01](#)). Open the **Profiler** tab, press record, interact with the slow part of your app, and stop. You get a flame graph of every component that rendered, how long each took, and — most usefully — **why** each one rendered.

Turn on **"Record why each component rendered"** in the Profiler's settings. That single option answers the question you actually have: was it props, state, a parent, or a context?

Two things to know while reading the numbers:

- **Development builds are much slower than production builds.** Never conclude "React is slow" from a dev build. For real numbers, `npm run build && npm run preview`.
- **StrictMode double-renders in development** ([chapter 12](#)), which inflates counts. Compare *relative* numbers, not absolutes.

If nothing shows up as slow, **stop**. You're done. An app that feels fast is fast.

`memo` : skip re-rendering a component

`memo` wraps a component so React skips re-rendering it when its props haven't changed:

```
import { memo } from "react";

type RecipeCardProps = {
  recipe: Recipe;
  onSelect: (id: string) => void;
};

function RecipeCard({ recipe, onSelect }: RecipeCardProps) {
  return <article>{/* ... */</article>;
}

export default memo(RecipeCard);
```

Now when the parent re-renders, React compares this render's props with the last render's. If they're all the same, it reuses the previous result and doesn't call `RecipeCard` at all.

"The same" means `Object.is` — the reference comparison from [chapter 11](#). That's the detail that makes `memo` fail silently in practice:

```
function RecipeGrid({ recipes }: RecipeGridProps) {
  return (
    <div>
      {recipes.map((recipe) => (
        <RecipeCard
          key={recipe.id}
          recipe={recipe}
          onSelect={(id) => console.log(id)} // ⚠️ a brand-new function every render
        />
      ))}
    </div>
  );
}
```

That inline arrow function is a **new function object on every render**, so `onSelect` is never equal to last time, so `memo` never skips anything. You've added a comparison to every render and gained precisely nothing.

This is the single most common way `memo` gets silently wasted — which is exactly what `useCallback` exists to fix.

`useCallback` : keep a function stable

```
import { useCallback } from "react";

const handleSelect = useCallback((id: string) => {
  setSelectedId(id);
}, []);
```

`useCallback` returns **the same function object** across renders, as long as its dependencies haven't changed. The dependency array works exactly like `useEffect`'s ([chapter 17](#)): list every reactive value the function reads.

Now `memo` on the child actually works, because `onSelect` is genuinely the same reference each time.

`useCallback` **is almost always pointless on its own**. It only helps when the function is passed to a `memo`'d component, or used as a dependency of a `useEffect` or `useMemo`. Wrapping a handler passed to a plain `<button onClick={...}>` achieves nothing at all — a DOM element doesn't care whether its handler is a new function.

`useMemo` : skip recalculating a value

```
import { useMemo } from "react";

const sortedBooks = useMemo(
  () => [...books].sort((a, b) => a.title.localeCompare(b.title)),
  [books]
);
```

`useMemo` runs its function on the first render and remembers the result, only recomputing when a dependency changes. Two legitimate uses:

- 1. The calculation is genuinely expensive.** Sorting thousands of items, heavy filtering, parsing something big. Note the word *genuinely* — filtering a 20-item array is not expensive, and wrapping it costs more than it saves.
- 2. The result is an object or array used as a dependency elsewhere**, where a new reference would cause unwanted work:

```
const value = useMemo(() => ({ theme, setTheme }), [theme]);
return <ThemeContext value={value}>{children}</ThemeContext>;
```

That's the context fix promised back in [chapter 22](#). Without `useMemo`, the object literal is new every render, so every consumer re-renders even when `theme` is identical.

The same applies to an object or array in a `useEffect` dependency array — the infinite-loop trap from [chapter 17](#).

The three together

They're designed to work as a set, and individually they often do nothing:

```
function RecipeGrid({ recipes, query, onSelect }: RecipeGridProps) {
  const visible = useMemo(
    () => recipes.filter((r) => r.name.toLowerCase().includes(query.toLowerCase())),
    [recipes, query]
  );

  return (
    <div className="results-grid">
      {visible.map((recipe) => (
        <RecipeCard key={recipe.id} recipe={recipe} onSelect={onSelect} />
      ))}
    </div>
  );
}

export default memo(RecipeGrid);
```

For this to actually pay off, **the whole chain has to hold**: `RecipeCard` is `memo`'d, `onSelect` comes from a `useCallback` in the parent, and each `recipe` object keeps the same reference between renders (which it does, as long as you follow [chapter 11](#) and don't rebuild untouched objects). Break any link — one inline arrow, one `structuredClone` — and the whole chain stops working, silently, with no warning anywhere.

That fragility is the strongest argument for using these sparingly and verifying with the Profiler that they did something.

Fix the structure first

Before any of this, check whether the real problem is *where your state lives*. This is the fix that actually works, and it needs no memoisation at all.

```
// ❌ every keystroke re-renders the entire page, including the 500-row table
function Page() {
  const [query, setQuery] = useState("");

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <HugeTable rows={rows} />
      <Sidebar />
    </div>
  );
}
```

```
// ✅ typing only re-renders SearchBox
function SearchBox() {
  const [query, setQuery] = useState("");
  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}

function Page() {
  return (
    <div>
      <SearchBox />
      <HugeTable rows={rows} />
      <Sidebar />
    </div>
  );
}
```

Moving state **down** to the smallest component that needs it is the mirror image of lifting state up ([chapter 13](#)), and it's often the entire fix. No `memo`, no `useCallback`, less code than before.

The other structural fix is **passing JSX as children**, the same trick from [chapter 22](#):

```
function Wrapper({ children }: { children: ReactNode }) {
  const [count, setCount] = useState(0);
  return (
    <div>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      {children}
    </div>
  );
}

<Wrapper>
  <ExpensiveTree />    {/* created by the PARENT, so it doesn't re-render when count changes */}
</Wrapper>
```

`ExpensiveTree` is created outside `Wrapper`, so clicking the button doesn't re-create it. Same JSX element object, so React skips it entirely — no `memo` anywhere.

Lists: the case that really is slow

Rendering hundreds or thousands of rows is the one performance problem you'll reliably hit. `memo` on the row component helps, but past a few thousand rows nothing helps except **not rendering most of them**.

Virtualisation (or "windowing") renders only the rows currently visible, plus a few either side, and swaps them as you scroll. Libraries like [TanStack Virtual](#) do this. A 10,000-row table renders maybe 30 elements.

You don't need this today, but recognise the situation: if you're about to render a very long list and reaching for `memo`, virtualisation is probably the real answer.

The React Compiler

Everything above is manual work that follows mechanical rules — which is the kind of thing a machine should do. That's the **React Compiler**, an official build-time tool that analyses your components and inserts the memoisation automatically.

With it enabled, you write plain, simple React — no `memo`, no `useMemo`, no `useCallback` — and the compiler adds the equivalent where it's actually useful, without the fragility of a human forgetting one link in the chain.

Vite offered it as an option when you created your app in [chapter 01](#) (the "TypeScript + React Compiler" variant), and the advice there was to pick plain TypeScript for learning. That was deliberate: understanding *what* it does for you is far more valuable than having it silently do it.

It relies on your components following the rules — pure renders, no mutation, correct dependencies. Everything this course has insisted on since [chapter 03](#) is exactly what makes a component compilable.

The realistic position today: know these three tools, because you'll read code full of them and you'll need them in projects that don't use the compiler. Expect to write fewer of them by hand over time.

A sane default

1. **Write plain, clear React.** No memoisation.
2. **If something feels slow, profile it** — production build, Profiler tab, "why did this render" on.
3. **Look for a structural fix first:** state pushed down, JSX passed as `children`, a smaller list.
4. **Only then** apply `memo` / `useMemo` / `useCallback`, to the specific thing you measured.
5. **Profile again** and confirm it actually helped. If it didn't, take it out.

Step 5 gets skipped constantly, and it's how codebases end up covered in memoisation that does nothing.

Common mistakes

1. Optimising before measuring

By far the most common. You cannot guess which part of a React app is slow — the Profiler exists precisely because intuition is unreliable here.

2. `memo` with an inline function or object prop

```
<MemoedCard recipe={recipe} onSelect={() => select(id)} />    // new function every render
<MemoedCard config={{ compact: true }} />                    // new object every render
```

`memo` compares by reference and always sees a difference, so it never skips. You've added cost for nothing. Use `useCallback`, or hoist the object out.

3. `useCallback` on a function that isn't passed to a `memo`'d component

```
const handleClick = useCallback(() => setOpen(true), []);
return <button onClick={handleClick}>Open</button>;    // pointless
```

A DOM element doesn't care. This is pure overhead.

4. `useMemo` on something trivial

```
const total = useMemo(() => price * quantity, [price, quantity]);    // ⚠️
```

The multiplication is faster than the memoisation bookkeeping. Reserve `useMemo` for genuinely expensive work, or for stabilising a reference.

5. Wrong dependencies

```
const filtered = useMemo(() => items.filter((i) => i.name.includes(query)), [items]);  
// ❌ `query` is missing – the result never updates when you type
```

Stale results, with no error. The dependency rules from [chapter 17](#) apply identically. Trust the linter.

6. Concluding React is slow from a dev build

Development builds carry extra checks and StrictMode double-renders. Measure `npm run preview`.

7. Reaching for `memo` when the fix is moving state

If typing in a search box re-renders your whole page, the answer is usually to move that state into the search box, not to `memo` everything else.

Quick recap

- **Measure before optimising.** The React DevTools Profiler, with "why did this render" enabled, on a production build.
- `memo` skips re-rendering a component when its props are unchanged — compared by **reference**, which is why one inline arrow or object literal silently defeats it.
- `useCallback` keeps a function reference stable. It's only useful when feeding a `memo`'d component or a dependency array.
- `useMemo` skips recalculating a value. Use it for genuinely expensive work, or to stabilise an object/array used as a context value or a dependency.
- They work as a **chain** — break one link and the whole optimisation silently stops working.
- **Fix the structure first.** Moving state *down*, or passing JSX as `children`, often removes the problem entirely with less code.
- Very long lists need **virtualisation**, not memoisation.
- The **React Compiler** does this work automatically for components that follow the rules. Understand the manual version; expect to write less of it over time.

Next: try the [exercises](#), then move on to [28 Testing](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch27/ex1/`, and so on.

- **Every exercise here requires numbers, not opinions.** Record actual measurements from the React DevTools Profiler before and after each change. "It feels faster" is not an answer in this chapter.
 - Turn on **"Record why each component rendered"** in the Profiler's settings (the gear icon) before you start. It's the single most useful setting in the tool.
 - Remember StrictMode double-renders in development. Compare before-and-after numbers with each other, not against an absolute.
 - An exercise is done when you have measurements showing whether your change helped, VS Code shows no red squiggles, and `npx tsc -b` prints nothing.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Learn to read the Profiler

In `src/ch27/ex1/`, build a page with a search input and a list of 200 generated items (`Array.from({ length: 200 }, (_, i) => ({ id: String(i), name: `Item ${i}` })))`, rendered by an `ItemRow` component. Include a `Sidebar` component that takes no props and renders some static text.

1. Open the Profiler, record, type five characters in the search box, and stop.
2. Answer from the flame graph: how many components rendered per keystroke? Did `Sidebar` render? Did every `ItemRow`?
3. Click on `Sidebar` in the profiler and read **why** it rendered. Write down the exact wording React gives you.
4. Now do the same on a production build (`npm run build && npm run preview`, then profile there). How do the timings compare with the dev build?
5. Finally: was any of this actually slow enough for a user to notice? Write an honest yes or no with the millisecond figure that justifies it.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Make `memo` fail

In `src/ch27/ex2/`, take exercise 1's list and try to optimise it — badly, on purpose, first.

1. Wrap `ItemRow` in `memo`. Profile again. Did the number of rendering rows go down?
2. It didn't — because the parent passes `onSelect={ (id) => console.log(id) }` inline. Confirm this by reading "why did this render" on an `ItemRow`.
3. Fix it with `useCallback` in the parent. Profile again and record the new number of rendering rows per keystroke.
4. Now break it again in a different way: pass `style={{ padding: 4 }}` inline to each row. Profile. What happened, and why?

5. Fix that too, by hoisting the object outside the component. Confirm with a final profile.
6. Write a sentence describing the rule that connects steps 2, 4 and 5.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): The structural fix beats memoisation

This exercise makes the notes' central claim concrete.

In `src/ch27/ex3/`, build the "slow" version deliberately:

```
function Ex3() {
  const [query, setQuery] = useState("");

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <p>Searching for: {query}</p>
      <HugeList />      {/* 1000 rows, each doing a small amount of pointless work */}
    </div>
  );
}
```

Give each row a deliberately slow render (a loop summing to 100,000, say) so the problem is real and measurable.

1. Profile typing five characters. Record the total render time per keystroke.
2. **Fix it with memoisation:** `memo(HugeList)`. Profile again, record the number.
3. **Now throw that away** and fix it structurally instead: move `query` state into its own `SearchBox` component so the input owns it, and `HugeList` is a sibling that never re-renders. No `memo` anywhere. Profile, record the number.
4. **Third approach:** revert to the original structure, but pass `<HugeList />` as `children` to a component that holds the state. No `memo` anywhere. Profile, record.
5. Compare all four numbers in a table in a comment. Which approaches worked? Which was the least code? Which would be easiest for a colleague to accidentally break six months from now?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): When `useMemo` earns its place, and when it doesn't

In `src/ch27/ex4/`, build a page with a large dataset — 5,000 records with a name, a category, and a number — plus a search box, a category filter, and a "sort by" selector.

1. Compute the filtered-and-sorted list **without** `useMemo`. Profile typing in the search box, and record the time.
2. Add an unrelated piece of state — a counter button that has nothing to do with the list. Click it while profiling. Does the expensive sort run again? Should it?
3. Wrap the computation in `useMemo` with the right dependencies. Profile the counter button again. What changed?
4. Now profile typing in the **search box** again. Did `useMemo` help there? Why or why not?
5. Deliberately leave one dependency out of the array (drop `sortBy`). What breaks, and does anything warn you?
6. Finally, add a `useMemo` around something trivial — `const doubled = useMemo(() => count * 2, [count])`. Profile. Is it faster, slower, or unmeasurable? Write down your conclusion about where the line sits.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Profile and fix a real app

Take your [chapter 21](#) Recipe Finder, or your [chapter 24](#) exercise 5 app, and do a genuine optimisation pass — including being willing to conclude that nothing needs changing.

1. **Build for production** and profile three real interactions: typing in the search box, opening a recipe, and toggling a favourite.
2. For each, record: total render time, how many components rendered, and the top three slowest.
3. **Write down which, if any, are actually a problem**, with a threshold you've decided in advance (a common rule of thumb: anything under 16ms per interaction is imperceptible, since that's one frame at 60fps).
4. For anything over your threshold, find the cause using "why did this render" — and try a **structural** fix first.
5. Only if structure can't fix it, apply the minimum memoisation that does, and profile again to confirm it helped.
6. **Deliberately over-optimize**: now add `memo` to every component, `useCallback` to every handler, and `useMemo` to every derived value in the app. Profile the same three interactions. Record the numbers.
7. Write a short conclusion comparing step 5 and step 6. Include your view on readability, not just speed.

Then a last one, for honesty:

8. If your measurements in step 3 showed nothing over the threshold, say so plainly and revert everything. That's a completely valid — and common — outcome, and being able to reach it deliberately is the whole point of the chapter.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

28 Testing

What is it?

Automated checks that your components actually work, written as code that runs your components and clicks around in them:

```
test("shows a greeting with the name", () => {
  render(<Greeting name="Maya" />);
  expect(screen.getByRole("heading")).toHaveTextContent("Hello, Maya!");
});
```

Two tools do the work. **Vitest** runs the tests (the same family as Vite, so it needs almost no setup). **React Testing Library** renders components and finds things on the page the way a user would.

Why does it matter?

You've been testing all along — by clicking. Change something, alt-tab to the browser, click through the flow, check nothing broke. That works fine for a small app, and it stops working for a specific reason: **the amount of clicking grows with the size of the app, and you only have so much patience.**

By the time an app has a dozen features, checking them all by hand after every change takes ten minutes, so you stop doing it. You check the thing you changed, assume the rest is fine, and find out three weeks later that it wasn't.

Automated tests make that check cost nothing. Save the file, and a hundred interactions run in two seconds.

What that actually buys you, in order of how much it matters:

- **Changing code without fear.** This is the big one. Tests let you refactor, rename, and restructure aggressively, because the moment you break something, you know. Without them, working code gradually becomes code nobody dares touch.
- **Bugs caught at the moment you write them**, when the context is still in your head, rather than days later.
- **A description of what the component is supposed to do**, that can't go out of date, because it fails when it's wrong.

Real-world example

Think about a **smoke test in a factory** versus **inspecting the assembly line**.

Inspecting how it's built	Testing that it works
"Is bolt 47 torqued correctly?"	"Does the door open and close?"
Breaks when the design changes, even if the product is fine	Keeps passing through any redesign that still opens
Checks <code>useState</code> was called, or props were passed	Checks the user sees the right thing after clicking
Has to be rewritten constantly	Rewritten only when behaviour genuinely changes
Testing implementation	Testing behaviour

The whole philosophy of React Testing Library is in that table: test the door, not the bolts. A test that knows which hook you used will break the moment you switch from `useState` to `useReducer` — even though nothing the user experiences has changed at all. That's a test making your life worse.

How it works

Setting up

In your project:

```
npm install -D vitest @testing-library/react @testing-library/user-event @testing-library/jest-dom jsdom
```

What each one does:

Package	Job
<code>vitest</code>	Runs the tests
<code>@testing-library/react</code>	Renders components, finds elements
<code>@testing-library/user-event</code>	Simulates realistic clicking and typing
<code>@testing-library/jest-dom</code>	Extra assertions like <code>toBeInTheDocument()</code>
<code>jsdom</code>	A fake browser so DOM code can run in Node

Add to `vite.config.ts`:

```

/// <reference types="vitest/config" />
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: "jsdom",
    setupFiles: "./src/test-setup.ts",
  },
});

```

Create `src/test-setup.ts`:

```
import "@testing-library/jest-dom/vitest";
```

And add a script to `package.json`:

```

"scripts": {
  "test": "vitest"
}

```

`npm test` now runs your tests and watches for changes. `globals: true` means `test` and `expect` are available without importing them, matching how most examples are written.

Your first test

Put test files next to the component, named `Something.test.tsx`:

```

// Greeting.tsx
type GreetingProps = { name: string };

export function Greeting({ name }: GreetingProps) {
  return <h1>Hello, {name}!</h1>;
}

```

```
// Greeting.test.tsx
import { render, screen } from "@testing-library/react";
import { Greeting } from "../Greeting.tsx";

test("greet the person by name", () => {
  render(<Greeting name="Maya" />);

  expect(screen.getByRole("heading")).toHaveTextContent("Hello, Maya!");
});
```

Three steps, and nearly every test has them:

1. **Arrange** — `render` the component with the props you want to test.
2. **Act** — click, type, or do nothing if you're only checking what's shown.
3. **Assert** — `expect(...)` something to be true.

Finding things: queries

`screen` is how you find elements. There are several ways, and **which one you pick matters** — it's the difference between a test that describes behaviour and one that describes markup.

Use them in this order of preference:

```
screen.getByRole("button", { name: "Add task" }); // ① best
screen.getByLabelText("Email");                 // ② form fields
screen.getByText("No tasks yet");                 // ③ plain text
screen.getByTestId("recipe-card");                // ④ last resort
```

`getByRole` first, nearly always. It finds elements the way assistive technology does — by what they *are* (`button`, `heading`, `textbox`, `checkbox`, `link`) and what they're *called*. A test written this way checks accessibility for free: if `getByRole("button", { name: "Delete" })` can't find your button, a screen reader user probably can't either.

This is a genuinely useful feedback loop. Tests that are hard to write with `getByRole` are usually pointing at a real accessibility problem — a `<div onClick>` instead of a button, an input with no label, an icon button with no `aria-label`. All things [chapter 38](#) covers, surfacing here first.

`getByTestId` last, and only when nothing else works. It requires adding `data-testid` attributes that exist purely for tests and mean nothing to users.

`getBy`, `queryBy`, `findBy`

Three prefixes, three different jobs, and mixing them up is the most common beginner mistake:

Prefix	If found	If not found	Use for
<code>getBy...</code>	returns it	throws	things that should be there now
<code>queryBy...</code>	returns it	returns <code>null</code>	asserting something is absent
<code>findBy...</code>	returns a promise	rejects after a timeout	things that appear later (async)

```
expect(screen.getByRole("button")).toBeInTheDocument();           // must exist
expect(screen.queryByText("Error")).not.toBeInTheDocument();       // must NOT exist
expect(await screen.findByText("Loaded!")).toBeInTheDocument();    // will exist soon
```

Using `getBy` to check something is absent doesn't work — it throws before your assertion runs, and the error is about a missing element rather than a failing expectation. That's what `queryBy` is for.

Each also has an `...AllBy` variant returning an array: `getAllByRole("listitem")`.

Simulating a user

Use `user-event`, not `fireEvent`. It simulates what a real user does — a click involves hovering, pressing, and releasing; typing fires a keydown, keypress, input and keyup per character — which catches bugs `fireEvent` walks straight past.

```
import userEvent from "@testing-library/user-event";

test("adds a task", async () => {
  const user = userEvent.setup();
  render(<TaskApp />);

  await user.type(screen.getByLabelText("New task"), "Buy milk");
  await user.click(screen.getByRole("button", { name: "Add" }));

  expect(screen.getByText("Buy milk")).toBeInTheDocument();
});
```

Every `user.*` call is async and must be awaited. Forgetting the `await` is the single most common cause of a test that fails confusingly, because the assertion runs before the click has finished.

The common actions:

```
await user.click(element);
await user.type(input, "text");
await user.clear(input);
await user.selectOptions(select, "value");
await user.keyboard("{Enter}");
await user.tab();
```

Testing behaviour, not implementation

Here's the same component tested two ways.

Bad — tests how it's built:

```
test("applies the is-open class", async () => {
  const user = userEvent.setup();
  const { container } = render(<Accordion title="Details">Hidden content</Accordion>);

  await user.click(screen.getByRole("button"));

  expect(container.querySelector(".accordion-panel")).toHaveClass("is-open");
});
```

That test knows about a CSS class name. Rename the class, or switch from a class to conditional rendering, and it fails — even though the component works perfectly and nothing a user sees has changed.

Good — tests what a user experiences:

```
test("shows the panel when the toggle is clicked", async () => {
  const user = userEvent.setup();
  render(<Accordion title="Details">Hidden content</Accordion>);

  expect(screen.queryByText("Hidden content")).not.toBeInTheDocument();

  await user.click(screen.getByRole("button", { name: "Details" }));

  expect(screen.getByText("Hidden content")).toBeInTheDocument();
});
```

The second test passes whether the component uses `useState`, `useReducer`, or `context`. Rewrite the internals completely and it keeps passing — which is exactly what you want, because a passing test after a refactor is what tells you the refactor was safe.

A useful check: if your test mentions a hook name, a state variable name, or a CSS class, it's probably testing implementation. Tests should read like a description of using the app.

Testing components with props and callbacks

For a callback prop, use a **mock function** and check it was called correctly:

```
import { vi } from "vitest";

test("calls onDelete with the task id", async () => {
  const user = userEvent.setup();
  const onDelete = vi.fn();

  render(<TaskRow task={{ id: "1", text: "Buy milk", done: false }} onDelete={onDelete} />);

  await user.click(screen.getByRole("button", { name: /delete/i }));

  expect(onDelete).toHaveBeenCalled("1");
  expect(onDelete).toHaveBeenCalledTimes(1);
});
```

`vi.fn()` creates a function that records how it was called. This is how you test a component in isolation — `TaskRow` doesn't need a real parent, just something that records what it asked for.

Note `{ name: /delete/i }` — a regular expression (JavaScript chapter 37), matching case-insensitively. Handy when the accessible name is `Delete Buy milk` and you only care about part of it.

Testing asynchronous things

For components that fetch, `findBy` waits for an element to appear:

```
test("shows recipes after loading", async () => {
  render(<RecipeSearch query="chicken" />);

  expect(screen.getByText("Loading...")).toBeInTheDocument();

  expect(await screen.findByText("Chicken Parmesan")).toBeInTheDocument();
  expect(screen.queryByText("Loading...")).not.toBeInTheDocument();
});
```

`findByText` retries for a second or so before giving up, so you don't need arbitrary waits.

But a test shouldn't call a real API — it'd be slow, need a network, and fail when someone else's server has a bad day. **Mock it:**

```
import { beforeEach, afterEach, vi } from "vitest";

beforeEach(() => {
  vi.stubGlobal("fetch", vi.fn(async () => ({
    ok: true,
    json: async () => ({ meals: [{ idMeal: "1", strMeal: "Chicken Parmesan", strMealThumb: "",
strCategory: "Chicken" }] })),
  })));
});

afterEach(() => {
  vi.unstubAllGlobals();
});
```

For anything beyond a couple of endpoints, **MSW** (Mock Service Worker) is the better tool — it intercepts requests at the network level, so your code calls `fetch` exactly as it normally would and you define fake responses once, reusable across every test. Worth knowing the name for when hand-stubbing `fetch` starts to hurt.

Testing components that need a provider

A component using context ([chapter 22](#)) or routing ([chapter 24](#)) needs its providers in the test too:

```
function renderWithProviders(ui: ReactNode) {
  return render(
    <MemoryRouter>
      <ThemeProvider>{ui}</ThemeProvider>
    </MemoryRouter>
  );
}

test("shows the cart count", () => {
  renderWithProviders(<CartBadge />);
  expect(screen.getByText("0")).toBeInTheDocument();
});
```

`MemoryRouter` is React Router's router for tests — it keeps the URL in memory rather than touching the real address bar, and you can start it anywhere with `initialEntries=[["/books/42"]]`.

Writing one `renderWithProviders` helper and using it everywhere saves a lot of repetition.

What to test, and what not to

You can't test everything, and trying makes tests a burden rather than a help. In rough priority order:

Worth testing:

- Anything with real logic: validation, calculations, filtering, reducers.
- User flows that matter: adding an item, submitting a form, the empty state.
- Bugs you've fixed — a test for each stops it coming back.
- Edge cases: empty lists, missing data, a failed request.

Usually not worth testing:

- That a component renders some static text.
- Third-party libraries — they have their own tests.
- Exact styling. (`toHaveClass` is brittle; is the class the *behaviour*?)
- Implementation details, per the whole section above.

A reducer is the single highest-value thing to test in a React app. It's a pure function — state in, state out — so it needs no rendering, no mocking, and no DOM at all. If you wrote one in [chapter 23](#), that's where to start:

```
test("deleting the task being edited also ends the edit", () => {
  const before: TaskState = {
    tasks: [{ id: "1", text: "Buy milk", done: false }],
    filter: "all",
    editingId: "1",
  };

  const after = taskReducer(before, { type: "deleted", id: "1" });

  expect(after.tasks).toHaveLength(0);
  expect(after.editingId).toBeNull();
});
```

Three lines, no setup, and it pins down a rule that would take real clicking to verify by hand.

Testing a custom hook

A custom hook ([chapter 20](#)) can't be called outside a component, so testing one directly needs `renderHook`:

```
import { renderHook, act } from "@testing-library/react";

test("useToggle flips the value", () => {
  const { result } = renderHook(() => useToggle(false));

  expect(result.current[0]).toBe(false);

  act(() => {
    result.current[1](); // call toggle
  });

  expect(result.current[0]).toBe(true);
});
```

`act` wraps anything that causes a state update, so React finishes re-rendering before you assert. (You rarely need it in component tests — `user-event` handles it for you.)

Use this for genuinely reusable hooks with real logic — `useLocalStorage`, `useDebounce`. For a hook used by exactly one component, testing that component's behaviour usually covers it better, and survives refactoring.

Coverage is a hint, not a target

`vitest --coverage` reports how much of your code the tests ran. It's useful for spotting whole files nobody's tested. It is **not** a score to maximise — 100% coverage with tests that assert nothing meaningful is worse than 40% coverage of the parts that actually matter. Use it to find gaps, not to set targets.

Common mistakes

1. Forgetting `await` on a `user` action

```
user.click(button); // ✗ no await
expect(screen.getByText("Saved")).toBeInTheDocument(); // runs too early
```

Every `user.*` call is a promise. This produces confusing failures where the element "isn't there" but clearly is.

2. `getBy` when checking something is absent

```
expect(screen.getByText("Error")).not.toBeInTheDocument(); // ✗ throws before asserting
expect(screen.queryByText("Error")).not.toBeInTheDocument(); // ✓
```

3. Testing implementation

If a test mentions a state variable, a hook, or a class name, it'll break on a refactor that changed nothing users can see. Test what's on screen.

4. `getByTestId` as the default

It works, and it skips the accessibility feedback that makes `getByRole` valuable. If `getByRole` can't find your button, that's information worth having, not an obstacle.

5. Using `fireEvent` instead of `user-event`

`fireEvent.click` dispatches one bare event. Real clicks involve more, and `user-event` catches bugs the bare version misses — like a button that's actually disabled.

6. Calling real APIs in tests

Slow, flaky, and dependent on someone else's uptime. Stub `fetch`, or use MSW.

7. Chasing a coverage number

High coverage with weak assertions is a false sense of safety. Test what matters.

8. Forgetting providers

```
useTheme must be used inside a ThemeProvider
```

A component needs its context and router in tests too. Write a `renderWithProviders` helper once.

Quick recap

- **Vitest** runs tests; **React Testing Library** renders components and finds elements the way a user would. Setup is a few packages and a `test` block in `vite.config.ts`.
- Every test is **arrange** (`render`), **act** (`user.click`), **assert** (`expect`).
- Prefer `getByRole` with an accessible name. It matches how assistive technology finds things, so your tests check accessibility for free.
- `getBy` throws if missing, `queryBy` returns `null` (use it to assert absence), `findBy` waits for something async.
- Use `user-event`, and `await` every action.
- **Test behaviour, not implementation.** A good test survives a rewrite of the component's internals; if it mentions a hook or a state variable, it's testing the wrong thing.
- Use `vi.fn()` for callback props, and stub `fetch` (or use MSW) rather than calling real APIs.
- **Reducers are the highest-value, lowest-effort thing to test** — pure functions, no rendering needed.
- Coverage finds untested files. It isn't a score.

Next: try the [exercises](#), then build the [29 Project: Online Bookstore](#), which brings this whole level together.

Exercises

How to do these:

- These need the test setup from the notes installed in your practice app. Run `npm test` in a terminal and leave it running — it re-runs on every save, which is the whole point.
- One folder per exercise: `src/ch28/ex1/`, with test files sitting next to the components they test (`Counter.tsx` and `Counter.test.tsx`).
- **Watch a test fail before you make it pass.** Write the assertion, see red, then write the code. A test you've never seen fail might not be testing anything.
- An exercise is done when the tests pass, they still pass after you rename internal variables, `npx tsc -b` prints nothing, and no test calls a real API.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Your first tests, and the query ladder

In `src/ch28/ex1/`, build a `Greeting` component taking `name: string` and `isVip?: boolean`. It shows an `<h1>` greeting, and a "VIP member" badge only when `isVip` is true.

Write these tests:

1. It shows the name in a heading.
2. It shows the VIP badge when `isVip` is `true`.
3. It does **not** show the badge when `isVip` is omitted. (Which query prefix do you need here?)

Then, an exercise in query choice — find the heading four different ways:

4. `getByRole("heading", { name: /maya/i })`
5. `getByText("Hello, Maya!")`
6. `getByTestId("greeting")` (you'll need to add a `data-testid`)
7. Now **change the `<h1>` to an `<h2>`**. Which of your three queries still pass? Which broke? What does that tell you about which to prefer?
8. Change the greeting text from `Hello,` to `Hi,`. Which break now?

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Testing a reducer

The highest value per line of effort in this whole chapter. Take your cart reducer from [chapter 23, exercise 3](#) (or rebuild a simple version) and test it thoroughly in `src/ch28/ex2/`.

No `render`, no DOM, no `user-event` — a reducer is a pure function, so these tests are just calling a function and checking what comes back.

Write a test for each rule:

1. Adding a new product puts it in the cart with quantity 1.
2. Adding a product that's already there increases its quantity instead of adding a second line.
3. `quantity_changed` to 0 removes the item entirely.
4. `discount_applied` with `SAVE10` applies the discount.
5. `discount_applied` with anything else leaves the state **completely unchanged** — assert the returned state is the *same object* (`toBe`, not `toEqual`).
6. `cleared` empties the items and removes any discount.
7. The reducer never mutates its input: capture the state before, run an action, and assert the original object is unchanged.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Testing a form

In `src/ch28/ex3/`, build a sign-up form (email, password, a terms checkbox, a submit button) with validation, then test it properly.

Tests:

1. The submit button is disabled initially.
2. Filling everything in correctly enables the button.
3. An invalid email shows an error message **after the field is blurred**, not while typing.
4. Submitting calls an `onSubmit` prop exactly once, with the right values.
5. Submitting clears the form.
6. Pressing **Enter** in the email field submits, just like clicking the button.
7. The password is never included in anything the component logs (add a `console.log` on submit, spy on it with `vi.spyOn(console, "log")`, and assert the password isn't in what was logged).

Then, the important part:

8. **Refactor the component's internals** — switch from four `useState` calls to one `useReducer`. Do **not** change a single test. They should all still pass. If any test breaks, it was testing implementation; rewrite it to test behaviour instead.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Testing async and mocked fetch

In `src/ch28/ex4/`, build a small component that fetches a list of users from an API and shows loading, error, and success states — the three-state pattern from [chapter 18](#).

Tests, each with `fetch` stubbed differently:

1. Shows "Loading..." immediately on mount.
2. Shows the list of users once the fetch resolves (`findBy`).
3. The loading message is gone afterwards (`queryBy`).
4. Shows an error message when the fetch rejects.
5. Shows an error message when the response has `ok: false` with a 500 status — a different path from a rejection, and one that's easy to get wrong.
6. Shows "No users found" when the API returns an empty array — which is a **success**, not an error.
7. Clicking a **Retry** button after a failure fires a new request (assert the fetch mock's call count).

Requirements:

8. No test may call the real network. Confirm by disconnecting your wifi and running the suite.
9. Reset your mocks between tests so one test's stub can't leak into another. Prove it by deliberately removing the reset and watching a test fail.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Test a real feature end to end

Take your [chapter 21](#) Recipe Finder — or the task board from [chapter 23, exercise 5](#) — and write a test suite for a genuine, whole feature, not just isolated components.

Pick **favourites** (Recipe Finder) or **the whole task flow** (task board), and cover:

1. A `renderWithProviders` helper setting up every context and router the component tree needs. Write it once, use it in every test.
2. The full happy path as a single test that reads like a user story: search → see results → favourite one → switch to the favourites view → see it there.
3. Persistence: favouriting something writes to `localStorage`, and a fresh render picks it up. (Stub `localStorage`, or clear it between tests.)
4. Un-favouriting removes it from both the view and storage.

5. The empty state: with no favourites, the favourites view says so rather than showing an empty box.
6. An error path: the API fails, the error message shows, and the rest of the app (the search box) still works.
7. **An accessibility check that falls out for free:** every one of your queries uses `getByRole` with an accessible name. If any element can't be found that way — an icon-only button, an unlabelled input — **fix the component**, not the test.

Then:

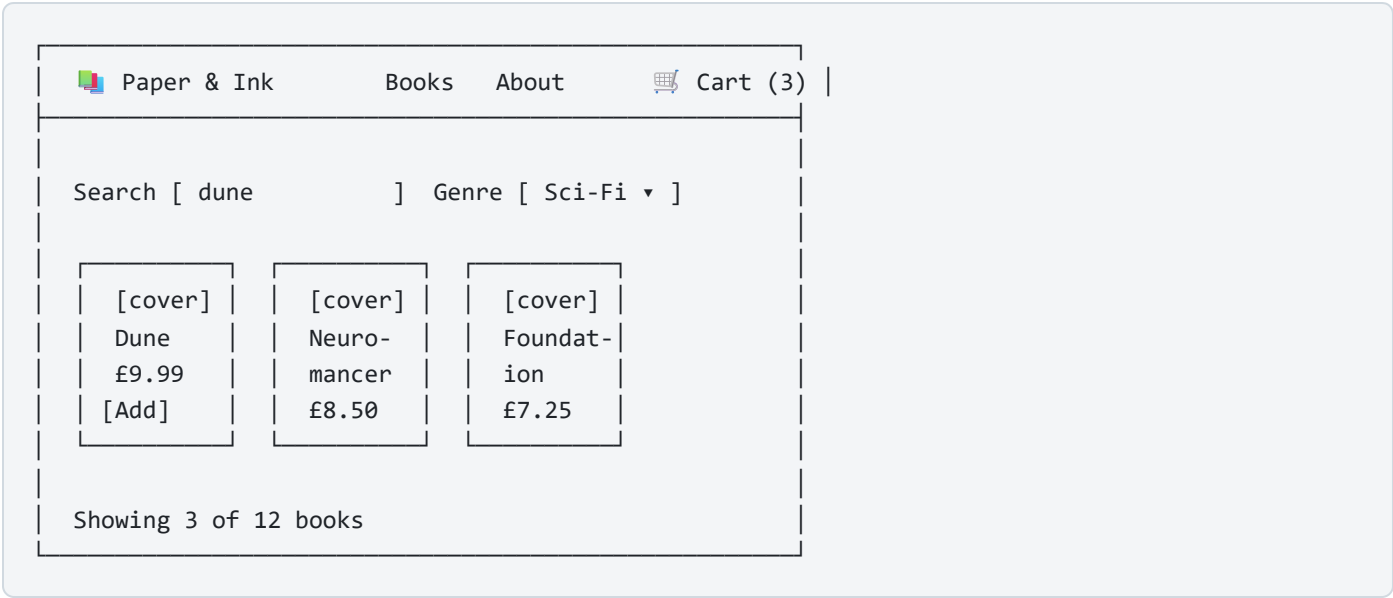
8. Run `npx vitest --coverage`. Note which files are untested. Pick the **one** with the most real logic in it and add tests for that. Then stop — don't chase the number.
9. Write a short comment answering: which of these tests would have caught a bug you actually hit while building the project originally?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

29 Project: Online Bookstore

What you'll build

A multi-page shop: browse books, filter them, open one, add it to a cart shared across every page, and check out — with tests covering the parts that matter.



By the end, your app will have:

- real URLs for every page, with working back/forward and shareable links
- filters that live in the URL, so a filtered view can be sent to someone
- a cart any page can read and change, with no prop drilling
- all the cart's rules in one reducer, and a test suite proving them
- error boundaries so one broken book can't blank the site
- a handful of tests covering the flows that actually matter

This is the Level 3 project, and it uses the whole level:

Chapter	Where you'll use it
22 Context	The cart, reachable from every page
23 useReducer	Every cart rule, in one place
24 React Router	Pages, layouts, book URLs, filters in the query string
25 TypeScript Patterns	A reusable <code>Button</code> , and a generic <code>List</code>
26 Error Boundaries	Per-route boundaries, and a lazily-loaded checkout

Chapter	Where you'll use it
27 Performance	Profiling before optimising — and probably not optimising
28 Testing	Reducer tests, and two or three real user flows

No API this time. The book data is a local file in the starter, on purpose — [chapter 21](#) already covered fetching thoroughly, and this project is about *structure*: routing, shared state, and tests. Adding a real API back in is a stretch goal.

Getting started

1. In a terminal in this `React` folder: `npm create vite@latest`, answering as usual, naming it `bookstore`.
2. Copy this chapter's `starter/src/` over `bookstore/src/`.
3. Delete `bookstore/src/App.css` and `bookstore/src/assets/`.
4. `cd bookstore`, then install what this project needs:

```
npm install react-router
npm install -D vitest @testing-library/react @testing-library/user-event @testing-library/jest-dom jsdom
```

5. Set up Vitest exactly as in [chapter 28](#) — the `test` block in `vite.config.ts`, and `src/test-setup.ts`.
6. `npm run dev` in one terminal, `npm test` in another, and `npx tsc -b` when you want a full check.

The starter gives you `books.ts` (12 books with covers, prices and genres), a finished `index.css`, and a stubbed `App.tsx`.

The big idea: three kinds of state, three different homes

The to-do app had one kind of state. The Recipe Finder had two. This app has **three**, and the whole project gets easier once you can tell them apart:

Kind	Example	Where it lives
Local	Is this dropdown open? What's half-typed?	<code>useState</code> , in the component
URL	Search text, genre filter, which book	<code>useSearchParams</code> , <code>useParams</code>
Shared app state	The cart	Context + reducer

The mistake that makes this project painful is putting something in the wrong one — most often, putting the search filters in `useState` (so they can't be shared or bookmarked) or the cart in a prop (so it has to be drilled through every page).

A quick test for each: **Would someone want to send this in a link?** → URL. **Does a component three pages away need it?** → shared. **Neither?** → local.

Milestone 1: Pages and layout

Goal: four real URLs, sharing a header.

Wrap `<App />` in `<BrowserRouter>` in `main.tsx`, then build the route table:

```
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<HomePage />} />
    <Route path="books" element={<BooksPage />} />
    <Route path="books/:id" element={<BookPage />} />
    <Route path="cart" element={<CartPage />} />
    <Route path="*" element={<NotFoundPage />} />
  </Route>
</Routes>
```

`Layout` holds the header (shop name, nav links, a cart indicator) and an `<Outlet />`. Use `NavLink` for the nav so the current page is highlighted.

Make each page a stub with a heading for now.

Check it: all five URLs work when typed directly. Back and forward work. A nonsense URL shows your 404. Put a `useState` counter in the header temporarily and confirm it *doesn't* reset when you navigate — proving the layout isn't being rebuilt.

Milestone 2: The book list, with filters in the URL

Goal: a browsable, filterable, **shareable** list.

`BooksPage` renders a grid of `BookCard`s from `books.ts`. Each card shows the cover, title, author, price, and an **Add to cart** button (not wired up yet). The whole card links to `/books/:id`.

Then the filters — a search box and a genre `<select>` — held in `useSearchParams`, not `useState`:

```
const [searchParams, setSearchParams] = useSearchParams();
const query = searchParams.get("q") ?? "";
const genre = searchParams.get("genre") ?? "all";
```

Filtering is a derived value, computed during render:

```
const visible = books.filter((book) => {
  const matchesQuery =
    book.title.toLowerCase().includes(query.toLowerCase()) ||
    book.author.toLowerCase().includes(query.toLowerCase());
  const matchesGenre = genre === "all" || book.genre === genre;
  return matchesQuery && matchesGenre;
});
```

Show `Showing 3 of 12 books`, and a proper empty state when nothing matches — with the filters still visible so people can undo what they did.

One decision worth making deliberately. Every keystroke in the search box adds a history entry, so **back** steps through your typing one letter at a time. Use `setSearchParams(next, { replace: true })` for the search box to avoid that, and think about whether the genre dropdown should behave the same way. There's no single right answer; make the call and write down why.

Check it: filter to something specific, copy the URL, open it in a new tab. You should land on exactly that view.

Milestone 3: The book page

Goal: one book, at its own URL.

`BookPage` reads the id with `useParams` and finds the book. Two cases to handle properly, and neither is optional:

```
const { id } = useParams();
const book = books.find((b) => b.id === id);

if (!book) {
  return <NotFoundPage message="We couldn't find that book." />;
}
```

`useParams` gives `string | undefined`, and `find` gives `Book | undefined` — so no `!`, no `as string`. Show the cover, title, author, genre, price, description, an **Add to cart** button, and a link back to the list.

Check it: `/books/3` works when typed directly and after a refresh. `/books/999` shows your not-found message rather than crashing. The back link works from both a fresh load and after arriving from the list.

Milestone 4: The cart reducer

Goal: every cart rule, in one pure function — and no UI yet.

This is the heart of the project, and it's deliberately built before anything renders it. Make

`src/cart/cartReducer.ts`:

```
export type CartItem = {
  bookId: string;
  title: string;
  price: number;
  quantity: number;
};

export type CartState = {
  items: CartItem[];
  promoCode: string | null;
};

export type CartAction =
  | { type: "added"; book: Book }
  | { type: "removed"; bookId: string }
  | { type: "quantity_changed"; bookId: string; quantity: number }
  | { type: "promo_applied"; code: string }
  | { type: "cleared" };
```

The rules, all of them living in the reducer:

1. `added` for a book already in the cart increases its quantity rather than adding a second line.
2. **Quantity is capped at 10** per title.
3. `quantity_changed` to 0 or less removes the item entirely.
4. `promo_applied` accepts only `BOOKS10` (10% off). Anything else returns the **exact same state object** — no change at all.
5. `cleared` empties the items *and* clears the promo code.

Add the `never` exhaustiveness check from [chapter 23](#) to the `default` case.

Now write the tests, before any UI exists. `src/cart/cartReducer.test.ts`, one test per rule above, plus one proving the reducer never mutates its input. This is the single highest-value testing you'll do all project — pure function, no rendering, no mocking.

Check it: `npm test` passes, with at least six tests, and you haven't rendered a single thing yet.

Milestone 5: The cart, everywhere

Goal: any page can read and change the cart, with no props.

Wire the reducer up through context, using **two contexts** ([chapter 23](#)):

```
const CartStateContext = createContext<CartState | null>(null);
const CartDispatchContext = createContext<React.Dispatch<CartAction> | null>(null);

export function CartProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(cartReducer, initialCartState);

  return (
    <CartStateContext value={state}>
      <CartDispatchContext value={dispatch}><{children}</CartDispatchContext>
    </CartStateContext>
  );
}
```

Plus a guard hook for each — `useCart()` and `useCartDispatch()` — that throw with a clear message when used outside the provider.

Wrap the app in `<CartProvider>` in `main.tsx`, inside `<BrowserRouter>`. Then:

- The header's cart indicator uses `useCart()` to show the total item count.
- Every **Add to cart** button uses `useCartDispatch()`.
- Not one cart-related prop is passed anywhere.

Check it: add books from the list page, from a book's own page, and watch the header count update from both. Navigate between pages — the cart survives, because the provider is above the routes.

Milestone 6: The cart page and checkout

Goal: somewhere to see and change what you've added.

`CartPage` shows each item with its title, price, a quantity control (`-` / `+`), a remove button, and a line total. Below: subtotal, the promo discount if applied, and the total. A promo code box. A **Clear cart** button. A **Checkout** button.

Every one of those numbers is **derived** — computed during render from the cart state, never stored ([chapter 08](#)).

An empty cart shows `Your cart is empty` and a link back to the books, not an empty table with a £0.00 total.

Then add a checkout page at `/checkout`, and **lazily load it** ([chapter 26](#)):

```
const CheckoutPage = lazy(() => import("./pages/CheckoutPage.tsx"));
```

with a `<Suspense fallback={...}>` above your routes. Checkout is a form (name, email, address) that validates, shows an order summary, and on submit clears the cart and shows a confirmation.

Check it: run `npm run build` and confirm checkout is a separate chunk. In `npm run preview`, watch the Network tab — that chunk shouldn't download until you visit `/checkout`.

Milestone 7: Reusable components, properly typed

Goal: two components used everywhere, typed with [chapter 25](#)'s patterns.

Button — accepts every native `<button>` prop plus your own `variant` and `size`:

```
type ButtonProps = ComponentPropsWithoutRef<"button"> & {
  variant: "primary" | "secondary" | "ghost";
  size?: "small" | "medium";
};
```

Merge `className` rather than letting the caller's overwrite yours. Replace every raw `<button>` in the app with it.

BookGrid — a generic list component:

```
type BookGridProps<T> = {
  items: readonly T[];
  getId: (item: T) => string;
  renderItem: (item: T) => ReactNode;
  emptyMessage: string;
};
```

Use it for the books page **and** the cart page, proving it's genuinely general.

Check it: `npx tsc -b` is clean. Try `<Button variant="danger">` and confirm TypeScript rejects it. Try passing a `className` and confirm in the Elements tab that both your classes and theirs are applied.

Milestone 8: Error boundaries and tests

Goal: it degrades gracefully, and the important flows are covered.

Boundaries ([chapter 26](#)) — install `react-error-boundary` and add:

- One around the **book grid**, so a bad book can't take out the header and nav.
- One around the **cart page**.

- One at the **root** as a last resort.
- One around the lazy checkout route, **outside** its `<Suspense>`.

Prove it: temporarily make one book's price `undefined` and call `.toFixed(2)` on it. The grid should show a fallback while the header, nav and cart count keep working.

Tests ([chapter 28](#)) — you already have the reducer covered. Add a `renderWithProviders` helper (wrapping `MemoryRouter` and `CartProvider`), then these flows:

1. The books page shows all 12 books.
2. Typing in the search box filters the list.
3. Clicking **Add to cart** increases the header's count.
4. Adding the same book twice gives one cart line with quantity 2.
5. Removing the last item shows the empty-cart message.
6. A book URL that doesn't exist shows the not-found message. (Start `MemoryRouter` at `/books/999`.)

Use `getByRole` with accessible names throughout. **If a query can't find something, fix the component** — an icon-only button needs an `aria-label`.

Check it: `npm test` is green, and every test still passes if you rename internal variables in your components.

Milestone 9: Profile it, and probably change nothing

Goal: practise measuring, and practise restraint.

Build for production (`npm run build && npm run preview`) and profile three interactions with the React DevTools Profiler ([chapter 27](#)), with "record why each component rendered" on:

1. Typing in the search box.
2. Adding a book to the cart.
3. Navigating between pages.

Write down, in a comment in `App.tsx`: the render time for each, and whether any exceeded 16ms.

If something did, find out why, and **try a structural fix first** — moving state down, or passing JSX as `children`. Only then reach for `memo`.

If nothing did — which is the likely outcome at twelve books — **write that down and change nothing**. Being able to finish a performance pass by deliberately doing nothing is a genuine skill, and it's the honest result here.

Now compare

Three projects in, it's worth looking at what changed.

	To-do (ch10)	Recipe Finder (ch21)	Bookstore (ch29)
Screens	1	1 + a modal	6 real URLs
State lives	One component	A few components	Local / URL / shared, deliberately split
State changes via	Setters in handlers	Setters in hooks	Actions through one reducer
Sharing state	Props	Props	Context
When it breaks	Blank page	Blank page	One section fails; the rest works
Checking it works	Clicking	Clicking	<code>npm test</code> , in two seconds

The thing that changed most is confidence. In the to-do app, "did I break anything?" was answered by clicking around. Here it's answered by a test suite — which is what makes it possible to restructure a component at milestone 7 without being afraid.

The thing that changed least is the fundamentals. Milestone 4's reducer follows exactly the copying rules from [chapter 11](#). The derived totals in milestone 6 are the same "don't store what you can calculate" from [chapter 08](#). Context is still just delivery for ordinary `useState`. Level 3 didn't replace anything you learned — it gave you places to put it.

Common mistakes

1. Filters in `useState` instead of the URL

It works, and it quietly removes the ability to share or bookmark a view. If someone would want to send it in a link, it belongs in `useSearchParams`.

2. Cart totals stored in state

```
const [total, setTotal] = useState(0); // ❌ goes stale the first time you forget to update it
```

Derive them. Every time.

3. One context holding state and dispatch together

Every dispatch-only component then re-renders on every cart change. Two contexts, as in [chapter 23](#).

4. Cart rules leaking into components

```
onClick={() => {  
  if (item.quantity < 10) dispatch({ type: "quantity_changed", ... }); // ✖  
}}
```

The cap belongs in the reducer. If a component knows a business rule, it'll disagree with the next component that also thinks it knows it.

5. `<a href>` in the nav

Full page reload, cart wiped. `<Link>` and `NavLink`, always.

6. `!` or as string on `useParams`

Both `useParams` and `find` can genuinely come back empty — a URL someone typed wrong, a book that was removed. Handle both; that's what the 404 case is for.

7. Tests that break when you refactor

If milestone 7's `Button` refactor breaks a test, that test was checking markup rather than behaviour. Rewrite it around `getByRole` and what the user sees.

8. Optimising at milestone 9 without measuring

Twelve books is not a performance problem. Measure, then act — and be willing to act by doing nothing.

Quick recap

- Sort state into **local**, **URL**, and **shared** before writing it. Most of this project's difficulty is misfiled state.
- Filters and ids belong in the **URL**, so views can be shared, bookmarked, and restored on refresh.
- Put every rule about how the cart changes in **one reducer** — and **test the reducer first**, before any UI exists. It's the cheapest, highest-value testing in a React app.
- Deliver shared state with **two contexts** (state and dispatch) and guard hooks. Components get what they need with no props.
- **Derive every total**. Nothing computed is ever stored.
- **Error boundaries per section**, so one broken thing can't blank the site.
- Write tests around **what a user does**, using `getByRole`. They should survive a refactor — and if they don't, they were testing the wrong thing.
- **Measure before optimising**, and be willing to finish by changing nothing.

Next: try the [stretch goals](#). That's the end of Level 3 — [Level 4](#) is the React toolbox: the libraries most teams reach for, and what problem each one actually solves.

Exercises

These are **stretch goals**: extra features for your finished bookstore. Do them in any order, and do as many as you like.

How to do these:

- Finish all 9 milestones first, then commit or copy the `bookstore` folder so you've got a working version to return to.
 - `index.css` already has styles for these goals — look for "Styles for the stretch goals" near the bottom.
 - **Write the test first for any of these that touches a cart rule.** You have a reducer test file already; adding to it before you change the reducer is the fastest way to work now.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Persist the cart

A cart that vanishes on refresh is a cart nobody trusts.

1. Save the cart state to `localStorage` whenever it changes, and load it on startup — using your `useLocalStorage` hook from [chapter 20](#), or an effect in `CartProvider`.
2. Load it as the reducer's **initial state**, using `useReducer`'s third argument (the lazy initialiser from [chapter 23](#)).
3. Handle corrupt stored data: edit the saved value to `not json` in DevTools, refresh, and confirm the app starts with an empty cart rather than a white screen.
4. Add a test proving a saved cart is restored on a fresh render.

What you should see: add three books, refresh, and the header still says 3.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Sorting, in the URL

Add a **Sort by** dropdown to the books page: *Title A–Z, Price: low to high, Price: high to low*.

1. The sort choice lives in the URL alongside the existing filters — `/books?q=dune&genre=scifi&sort=price-asc`.
2. Sorting never mutates the `books` array ([chapter 11](#)).
3. The sort survives a refresh and can be shared in a link.
4. Add a **Clear all filters** button that strips the whole query string back to a bare `/books`.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): Stock levels and validation

Give each book a `stock` number in `books.ts`, and make the app respect it.

1. A book with `stock: 0` shows **Out of stock** and a disabled Add button.
2. The cart can never hold more of a book than its stock. Enforce this **in the reducer**, not in the component — the cap replaces (or combines with) the existing limit of 10.
3. The cart page shows a warning on any line at its stock limit: `Only 2 left in stock`.
4. Checkout is blocked while any line exceeds stock, with a clear message saying which book.
5. **Write the reducer tests first**, for: adding at the limit does nothing, `quantity_changed` above stock clamps to stock, and adding an out-of-stock book is ignored entirely.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 4 (Medium): Saved for later

Add a wishlist — a second collection, alongside the cart, with its own rules.

1. A ♥ on every book card and book page toggles "saved for later".
2. A `/saved` route listing saved books, with **Move to cart** on each.
3. Saved items persist across refreshes, separately from the cart.
4. The header shows both counts: `🛒 3 ♥ 5`.
5. **Move to cart** adds to the cart *and* removes from saved, as one user action.
6. On the cart page, each line gets a **Save for later** button doing the reverse.

Then a design question, answered in a comment: did you extend the existing cart reducer with wishlist actions, or build a second reducer and context? Give your reasoning — both are defensible, and being able to argue for your choice matters more than which you picked.

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Make it a real app

Four bigger additions, each mapping to something a real shop needs. Take them in any order.

A. A real API. Replace `books.ts` with a fetch, using the three-state pattern and a custom hook from [chapter 18](#). You can serve the JSON yourself — put `books.json` in `public/` and fetch `/books.json` — which behaves exactly like a real API including loading states, but needs no server. Add loading skeletons and a proper error state with a retry.

B. A protected account area. An `AuthContext` with a fake login, an `/account` section with nested routes ([chapter 24](#)), and a `RequireAuth` layout route that redirects logged-out visitors to `/login` — **remembering where they were going** and sending them there after they log in.

C. Order history. Completing checkout saves the order (id, date, items, total) to a persisted list. `/account/orders` lists them; `/account/orders/:orderId` shows one. Test the full flow: add books → checkout → the order appears in history with the right total.

D. A proper test suite. Get to the point where you'd genuinely trust it:

- Every reducer rule covered, including the new ones from A–C.
- One test per user flow: browse → filter → add → cart → checkout → order history.
- The error path: the API fails, the error shows, the nav still works.
- The auth path: a logged-out user hitting `/account/orders` lands on login, and ends up at `/account/orders` after logging in.
- Run `npx vitest --coverage` and write down which files remain untested and whether that's acceptable.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

When you're done

That's the end of Level 3, and the end of "plain React". Everything from here on is libraries built on top of what you now know.

Before moving on, write down:

- One thing in this project that would have been genuinely painful without routing, context, or the reducer.
- One test that caught a real bug while you were building it.
- One part of the app you'd restructure if you started again, and why.

That last one is the most useful. Bring it to Claude and talk it through.

Level 4: The React Toolbox

Goal: learn the libraries most React teams use, and when each one is worth adding.

30 Axios

Welcome to Level 4: the React Toolbox. Each chapter in this level is a library that most React teams use. Each one solves a problem you've already felt by hand in Levels 2 and 3. So every chapter shows the plain React way first (or links back to it), then the library, then a short "When you don't need it" part. Libraries aren't magic, and none of them is compulsory. The skill is knowing when one is worth adding.

What is it?

Axios is a library for making HTTP requests: asking a server for data, or sending it some. It does the same job as `fetch` (JavaScript chapter 33), with fewer chores.

```
const response = await axios.get("https://www.themealdb.com/api/json/v1/1/random.php");
console.log(response.data); // the JSON, already turned into an object
```

This chapter uses **Axios 1.x**. Install it inside your `playground` folder:

```
npm install axios
```

Why does it matter?

Look back at [chapter 18](#). Every single request needed the same chores, and forgetting any one of them caused a bug. Here they are, next to what Axios does instead:

Chore	With <code>fetch</code>	With Axios
Fail on a 404 or a 500	Check <code>response.ok</code> and throw, every time	Throws for you

Chore	With <code>fetch</code>	With <code>Axios</code>
Read the JSON	A second <code>await response.json()</code>	Already in <code>response.data</code>
Send JSON	<code>JSON.stringify</code> plus a <code>Content-Type</code> header	Pass the object
Query strings	Build them, with <code>encodeURIComponent</code>	<code>params: { s: query }</code>
The server's address	Typed in full, in every file	Set <code>baseUrl</code> once
Give up on a stuck server	Remember <code>AbortSignal.timeout(5000)</code> on each call	<code>timeout: 5000</code> , once
A login token on every request	Add the header to every call, and never forget one	An interceptor, once

Being honest: `fetch` is built into every browser, costs nothing to install, and is perfectly fine. Plenty of good apps never use `Axios`. The biggest win isn't any single feature. It's that you set `Axios` up **once**, in one file, and every request in your app gets the same address, timeout, headers, and error behaviour. You'll also meet `Axios` in lots of real codebases, so it's worth being able to read it.

Real-world example

Think about sending parcels.

Posting it yourself at the post office (<code>fetch</code>)	A courier account (<code>Axios</code>)
You write the full address on every parcel	The account remembers your address (<code>baseUrl</code>)
You wrap and label everything yourself	They pack it for you (objects become JSON)
The receipt says "delivered" even when the parcel came back	They tell you plainly when a delivery failed (throws on 404 / 500)
You unwrap every parcel that arrives	It arrives unwrapped (<code>response.data</code>)
You wait at the counter as long as it takes	They give up after a set time and tell you (<code>timeout</code>)
You show your ID at the counter, every time	Your account card goes on every parcel automatically (interceptors)

Both get the parcel there. One of them needs a lot less remembering.

How it works

Your first request: `fetch` and `Axios` side by side

Here's chapter 18's "Surprise me!" request, both ways:

```
type MealDbResponse = {
  meals: null | Array<{ idMeal: string; strMeal: string; strMealThumb: string }>;
};

// With fetch (chapter 18)
async function getRandomMealWithFetch() {
  const response = await fetch("https://www.themealdb.com/api/json/v1/1/random.php");
  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }
  const data: MealDbResponse = await response.json();
  return data.meals?.[0];
}

// With Axios
import axios from "axios";

async function getRandomMealWithAxios() {
  const response = await axios.get<MealDbResponse>(
    "https://www.themealdb.com/api/json/v1/1/random.php");
  return response.data.meals?.[0];
}
```

(In a real file, the `import` goes at the top.) Four lines shorter, and nothing was forgotten: `Axios` checked the status and read the JSON for you.

What's inside a response

`Axios` hands you a **response object**. The data is one part of it:

```
const response = await axios.get<MealDbResponse>(
  "https://www.themealdb.com/api/json/v1/1/random.php");

console.log(response.status); // the status code: 200
console.log(response.headers["content-type"]); // a header (names are lower case): something
  like application/json
console.log(response.data.meals?.[0].strMeal); // the body, already an object: e.g. Chicken
  Handi (it's random!)
```

Most of the time you only want `response.data`.

Typing the response: a promise, not a proof

The `<MealDbResponse>` in `axios.get<MealDbResponse>(...)` is a **generic** (TypeScript chapter 08). It tells TypeScript what `response.data` will be. Leave it out and `response.data` is `any`, which switches off checking.

But be clear about what it does. It's the same as `const data: MealDbResponse = await response.json()` in chapter 18. **It's a promise you're making, not a proof.** Axios doesn't look at the data and check it. If the server sends something different, your code still compiles and breaks later. [Chapter 32](#) shows how to check for real, with Zod.

Query strings with `params`

In [chapter 21](#) you built search URLs by hand, and had to remember `encodeURIComponent`. Axios builds the query string from an object:

```
const response = await axios.get<MealDbResponse>
  ("https://www.themealdb.com/api/json/v1/1/search.php", {
    params: { s: "chicken & rice" },
  });
// Axios asks for something like: .../search.php?s=chicken+%26+rice
```

The `&` got encoded for you (`%26`), so it can't break the URL. Open the **Network** tab in DevTools and click the request to see the real address. A `params` value that is `undefined` is left out completely, which is handy for optional filters.

A practice API of your own: json-server

TheMealDB only lets you *read*. To practise adding, changing, and deleting, you need a server that saves things. **json-server** is a tiny fake REST API that runs on your own computer. (A **REST API** is a server where each kind of thing has an address, like `/books`, and you use GET, POST, PATCH, and DELETE on it.) It reads a JSON file and turns it into a working API.

1. Install it inside `playground`. The `-D` makes it a **dev dependency**: a tool you use while building, not part of the app you ship.

```
npm install -D json-server
```

2. Make `playground/db.json`. It goes at the top of the project, next to `package.json`, **not** inside `src/`:

```
{
  "books": [
    { "id": "1", "title": "The Hobbit", "author": "J.R.R. Tolkien", "pages": 310, "status": "finished" },
    { "id": "2", "title": "Atomic Habits", "author": "James Clear", "pages": 320, "status": "reading" },
    { "id": "3", "title": "Dune", "author": "Frank Herbert", "pages": 412, "status": "want" },
    { "id": "4", "title": "Sapiens", "author": "Yuval Noah Harari", "pages": 443, "status": "want" },
    { "id": "5", "title": "The Martian", "author": "Andy Weir", "pages": 369, "status": "finished" }
  ]
}
```

This is your **reading list**: books you want to read, are reading, or have finished.

3. Add a script to the "scripts" part of `package.json`. Your file has a few more scripts than this; leave them alone and add the `api` line (mind the commas):

```
"scripts": {
  "dev": "vite",
  "api": "json-server --port 3001 db.json"
}
```

4. Run it in a second terminal. Keep `npm run dev` running in the first one. Open another terminal inside `playground` and run:

```
npm run api
```

It prints a short message with its address and a list of endpoints. Now visit `http://localhost:3001/books` in your browser. You'll see your five books.

Why port 3001? Vite uses 5173 and json-server's default is 3000. In [chapter 39](#), Next.js also wants 3000, so we keep the practice API out of its way.

The **endpoints** (the addresses you can call) are:

Request	What it does
GET <code>/books</code>	All the books
GET <code>/books/3</code>	The book with id <code>"3"</code>
GET <code>/books?status=want</code>	Only the books you want to read
POST <code>/books</code>	Add a book. json-server makes up the <code>id</code>

Request	What it does
PATCH /books/3	Change some fields of book 3, keep the rest
DELETE /books/3	Remove book 3

A few things worth knowing:

- **Changes are real.** json-server writes them into `db.json`. Add a book and you'll see it appear in the file. To start fresh, paste the original five books back in.
- **Ids are strings**, like `"3"`. New books get a short random id.
- **It allows requests from other pages.** Your app runs on port 5173 and the API on 3001, which counts as a different origin. json-server allows it, so there's no CORS error (JavaScript chapter 33).
- **It notices hand edits.** In version 1, if you edit `db.json` while the server runs, it reloads the file. If you save broken JSON (a missing comma), the terminal shows an error until you fix it.
- **Version 1 has been labelled "beta" for a long time.** It works well, but details can change. Older tutorials use version 0.17, which had different options (like `--watch`, `_limit`, and number ids). This course avoids the sorting and paging options that changed between versions. When you need sorting, you'll do it in your own code.

One shared instance: `src/api/client.ts`

So far you've typed full addresses. That's the "write the address on every parcel" problem. `axios.create` makes an **instance**: your own copy of Axios with settings built in. Make `src/api/client.ts`:

```
// src/api/client.ts
import axios from "axios";

export const api = axios.create({
  baseURL: "http://localhost:3001",
  timeout: 5000,
});

export const mealDb = axios.create({
  baseURL: "https://www.themealdb.com/api/json/v1/1",
});
```

- `baseURL` is put in front of every address. `api.get("/books")` asks for `http://localhost:3001/books`.
- `timeout: 5000` means "give up after 5 seconds" (5000 milliseconds). More on that below.
- Two instances, because you talk to two servers. Each has its own settings.

The instances are made **once**, when the file is first imported (JavaScript chapter 29). Every file that imports `api` gets the same one. When you deploy the app ([chapter 40](#)), the address changes. Because it lives in one place,

that's a one-line change.

When a request fails

Axios **throws** for any status outside the 200s. You don't check `response.ok`; you `catch`. Try asking for a book that doesn't exist:

```
import axios from "axios";
import { api } from "../client.ts";

async function tryMissingBook() {
  try {
    const response = await api.get("/books/999");
    console.log(response.data);
  } catch (err) {
    if (axios.isAxiosError(err)) {
      if (err.response) {
        // The server answered, with bad news
        console.log(err.response.status); // 404
        console.log(err.message);        // Request failed with status code 404
      } else {
        // No answer at all: server not running, offline, or too slow
        console.log(err.code, err.message);
      }
    } else {
      console.log("Something else went wrong:", err);
    }
  }
}
```

Step by step:

- `err` is **unknown** in a `catch` (TypeScript chapter 13). Anything can be thrown, so TypeScript won't let you read `err.response` yet.
- `axios.isAxiosError(err)` is a type guard. Inside the `if`, TypeScript knows `err` is an Axios error.
- `err.response` exists when the server *did* answer. `err.response.status` is the code, and `err.response.data` is whatever body the server sent back.
- **No** `err.response` means no answer arrived. `err.code` tells you why. `"ERR_NETWORK"` (message `Network Error`) usually means the server isn't running or you're offline. A timeout usually shows `"ECONNABORTED"`.

Axios errors are real `Error` objects, so `err.message` always works. But messages like `Request failed with status code 404` are written for developers, not for the people using your app. You'll make friendlier ones in the exercises.

Reading and writing: `src/api/books.ts`

[Chapter 21](#) taught you to keep one file that knows about the API, so components never do. Here's that file for the reading list. Make `src/api/books.ts`:


```

// src/api/books.ts
import { api } from "../client.ts";

export type BookStatus = "want" | "reading" | "finished";

export type Book = {
  id: string;
  title: string;
  author: string;
  pages: number;
  status: BookStatus;
};

// A book before the server has given it an id
export type NewBook = Omit<Book, "id">;

// GET /books: every book on the list
export async function getBooks(): Promise<Book[]> {
  const response = await api.get<Book[]>("/books");
  return response.data;
}

// GET /books/:id: one book
export async function getBook(id: string): Promise<Book> {
  const response = await api.get<Book>(`/books/${id}`);
  return response.data;
}

// POST /books: add a book. The server sends it back with its new id.
export async function addBook(book: NewBook): Promise<Book> {
  const response = await api.post<Book>("/books", book);
  return response.data;
}

// PATCH /books/:id: change some fields, keep the rest
export async function updateBook(id: string, changes: Partial<NewBook>): Promise<Book> {
  const response = await api.patch<Book>(`/books/${id}`, changes);
  return response.data;
}

// DELETE /books/:id: remove a book
export async function deleteBook(id: string): Promise<void> {
  await api.delete(`/books/${id}`);
}

```

`Omit<Book, "id">` means "a `Book` without its `id`", and `Partial<NewBook>` means "any of those fields, all optional". Both are utility types from TypeScript chapter 09. So `updateBook("3", { status: "reading" })` is fine,

and `updateBook("3", { status: "done" })` is a type error.

Notice `addBook`. You passed a plain object. Axios turned it into JSON and set the `Content-Type` header for you. Compare that with the POST in JavaScript chapter 33: no `method`, no `headers`, no `JSON.stringify`, no `response.ok`.

Every file after this chapter uses these five functions. Components will never see a URL.

Using it in a component

Here's the reading list, with chapter 18's three-state pattern:

```
// src/ch30/ReadingList.tsx
import { useEffect, useState } from "react";
import { getBooks } from "../api/books.ts";
import type { Book } from "../api/books.ts";

function ReadingList() {
  const [books, setBooks] = useState<Book[] | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let ignore = false;

    getBooks()
      .then((found) => {
        if (!ignore) setBooks(found);
      })
      .catch((err: unknown) => {
        if (!ignore) setError(err instanceof Error ? err.message : "Something went wrong");
      })
      .finally(() => {
        if (!ignore) setIsLoading(false);
      });

    return () => {
      ignore = true;
    };
  }, []);

  if (isLoading) return <p>Loading your books...</p>;
  if (error) return <p>Something went wrong: {error}</p>;
  if (books === null || books.length === 0) return <p>Your reading list is empty.</p>;

  return (
    <ul>
      {books.map((book) => (
        <li key={book.id}>
          {book.title} by {book.author} ({book.status})
        </li>
      ))}
    </ul>
  );
}

export default ReadingList;
```

Stop `npm run api` and refresh: you'll see `Something went wrong: Network Error`. Start it again and the books come back.

Be honest about what just happened. The request got shorter. The component didn't. The loading state, the error state, and the `ignore` flag are all still here. And when you add a book, this list won't know. You'd have to reload it yourself. Axios makes *requests* nicer. It doesn't manage *data over time*. That's [chapter 31](#)'s job.

Cancelling a request

Axios takes the same `signal` as `fetch`, from chapter 18's `AbortController`:

```
useEffect(() => {
  const controller = new AbortController();

  mealDb
    .get<MealDbResponse>("/search.php", {
      params: { s: query },
      signal: controller.signal,
    })
    .then((response) => setMeals(response.data.meals ?? []))
    .catch((err: unknown) => {
      if (axios.isCancel(err)) return; // we cancelled it on purpose: not a real error
      setError(err instanceof Error ? err.message : "Something went wrong");
    });

  return () => {
    controller.abort();
  };
}, [query]);
```

When `query` changes, the cleanup cancels the old request. Axios then throws a **cancel error**.

`axios.isCancel(err)` recognises it (so does `err.code === "ERR_CANCELED"`). In chapter 18 you checked `err.name === "AbortError"` instead. Same idea, different name.

In the Network tab, a cancelled request shows `(canceled)` (Chrome spells it the American way). In development you'll also see one on first load. That's Strict Mode running your effect twice on purpose ([chapter 17](#)).

Interceptors: a checkpoint every request passes through

An **interceptor** is a function that runs for every request made through an instance. Think of the post room in a big office. Every outgoing letter gets stamped there, and every incoming letter gets sorted there, so nobody at a desk has to do it.

There are two kinds:

- A **request interceptor** runs before a request leaves. It can change it, for example by adding a header.
- A **response interceptor** runs when the answer comes back, before your code sees it.

Adding a login token to every request. Real APIs often need a **token**: a secret string that proves who you are, like a festival wristband. You send it in an `Authorization` header. Add this to `client.ts`, below the two instances:

```
// src/api/client.ts (continued)
api.interceptors.request.use((config) => {
  // A real app gets this from logging in. Ours is pretend.
  const token = "pretend-token-123";
  config.headers.Authorization = `Bearer ${token}`;
  return config; // hand the request back, or it never leaves
});
```

Now every call through `api` carries the header. Click a `/books` request in the Network tab and look under **Request Headers**. You'll also see an extra `OPTIONS` request: the CORS preflight from JavaScript chapter 33, because of the custom header. json-server says yes to it.

Notice it's on `api`, **not** `mealDb`. Interceptors belong to one instance. That matters: you never want to send your token to somebody else's server.

Logging every failed request. A response interceptor takes two functions: one for success, one for failure.

```
api.interceptors.response.use(
  (response) => response, // success: pass it straight through
  (error: unknown) => {
    if (import.meta.env.DEV && axios.isAxiosError(error)) {
      console.warn(
        `[api] ${error.config?.method?.toUpperCase()} ${error.config?.url} failed:`,
        error.response?.status ?? error.code
      );
    }
    return Promise.reject(error); // keep it an error, so your catch still runs
  }
);
```

`import.meta.env.DEV` is `true` only while you run `npm run dev`, so the logging never reaches real users.

The last line matters most. If a failure interceptor *returns* something instead of rejecting, the error disappears, and the caller thinks the request worked (JavaScript chapter 31). Always `return Promise.reject(error)` unless you truly mean to recover.

You *could* also swap the error for a friendlier one here. But then `axios.isAxiosError` and `err.response?.status` stop working for everyone downstream. It's usually better to keep the real error and turn it into a friendly message where you show it. Exercise 4 does exactly that.

Timeouts

Without a timeout, Axios waits as long as the browser does, which can be a very long time. The `timeout: 5000` in `client.ts` means: no answer in 5 seconds, give up and throw. The error has no `response`, and its message is something like `timeout of 5000ms exceeded`.

You can change it for one request, like a slow report:

```
const response = await api.get("/books", { timeout: 20000 }); // this one may take 20 seconds
```

(With `fetch`, modern browsers let you write `signal: AbortSignal.timeout(5000)`. It works, but you have to remember it on every call.)

When you don't need Axios

- **A small app with a few GET requests.** `fetch` plus one small helper, like `fetchJson` from TypeScript chapter 13, is perfectly good.
- **Every kilobyte counts.** Axios adds a little to your download. `fetch` adds nothing.
- **Your framework builds on `fetch`.** Next.js ([chapter 39](#)) adds its own features to `fetch`, so plain `fetch` is the normal choice there.

There are also small libraries built on top of `fetch` (like `ky`) that give you some of the same comforts.

Whichever you pick, the real lesson is the same: **one file that knows your API**, with small typed functions like `getBooks`, and components that never see a URL.

Common mistakes

1. Forgetting `.data`

```
const response = await api.get<Book[]>("/books");
setBooks(response); // ✗ a type error mentioning AxiosResponse<Book[], ...>
setBooks(response.data); // ✓
```

The response is the whole parcel: status, headers, and data. Your books are in `.data`.

2. Turning the body into JSON yourself

```
api.post("/books", JSON.stringify(book)); // ✗ Axios expects the object
api.post("/books", book); // ✓ Axios converts it and sets the header
```

That's a `fetch` habit. With Axios, it's extra work at best. At worst, the server gets a plain string with the wrong label and saves nothing useful.

3. Reading `err.response` without checking first

```
catch (err) {  
  console.log(err.response.status);  
  // ❌ 'err' is of type 'unknown'.  
}
```

Narrow it with `axios.isAxiosError(err)`, then use `err.response?.status`, because `response` is missing when no answer came back.

4. Checking for errors before checking for cancels

A cancelled request is *also* an Axios error, with no `response`. If you check `isAxiosError` first, a harmless cancel looks exactly like "the server is down". Check `axios.isCancel(err)` first, and return quietly.

5. Creating an instance inside a component

```
function ReadingList() {  
  const api = axios.create({ baseUrl: "http://localhost:3001" }); // ❌ a new one every render  
}
```

Make instances once, in `client.ts`, and import them. Adding interceptors inside a component is worse: they pile up, one more every render, and you'll see each log two, three, ten times.

6. Trusting the generic as a guarantee

`api.get<Book[]>` doesn't check anything. If the server sends `{ "items": [...] }` instead of an array, TypeScript can't tell you. [Chapter 32](#) fixes this.

7. Forgetting to start the practice API

```
GET http://localhost:3001/books net::ERR_CONNECTION_REFUSED
```

The Console shows that line, and your error is `Network Error` with `err.code` of `"ERR_NETWORK"`. Nothing is wrong with your code. Open a second terminal in `playground` and run `npm run api`.

Quick recap

- **Axios** does what `fetch` does, minus the chores: it throws on 404 / 500, reads the JSON into `response.data`, sends objects as JSON, and builds query strings from `params`.
- The big win is **one shared instance** in `src/api/client.ts`, with a `baseUrl` and a `timeout`.
- **json-server** turns `db.json` into a real practice API on port 3001. Run `npm run api` in a second terminal.

- Keep every request in small, typed functions like `getBooks` and `addBook` in `src/api/books.ts`. Components never see a URL.
 - In a `catch`, check `axios.isCancel(err)` first, then `axios.isAxiosError(err)`. `err.response` exists only when the server answered.
 - **Interceptors** are checkpoints every request passes through: add a header, log failures. Always hand the request back, and always reject errors you don't handle.
 - `axios.get<Book[]>` is a promise, not a proof. And Axios doesn't manage loading, caching, or refetching. That's the next chapter.
-

Next: try the [exercises](#), then move on to [31 TanStack Query](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch30/ex1/`, and so on.
 - Install Axios first (`npm install axios`) and make `src/api/client.ts` as in the notes.
 - Exercises 2 to 4 use the practice API. Set up json-server as in the notes, then keep it running with `npm run api` in a second terminal, next to `npm run dev`.
 - Keep the DevTools **Network** tab open. Click any request to see its address, headers, and response.
 - An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Surprise me, the Axios way

Your "Surprise me!" meal picker from chapter 18 works. Time to see how it looks with Axios.

In `src/ch30/ex1/Ex1.tsx`:

1. Rebuild [chapter 18's exercise 1](#): a **Surprise me!** button that loads a random meal from `/random.php` and shows its name and thumbnail.
2. Use the `mealDb` instance from `client.ts`, so the address in your component is just `"/random.php"`.
3. Give the request a generic, so `response.data` is typed. Keep loading, error, and meal as three pieces of state.
4. Log `response.status` and `response.headers["content-type"]` once, and write what you saw in a comment.

5. **Break it on purpose:** put a typo in `mea1Db`'s `baseUrl` domain. Click the button and write down the exact error message your page shows, and the value of `err.code` (log it). Then fix the typo.
6. Count the lines inside your click handler, and in the chapter 18 version. Write both numbers in a comment.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Your reading list, from your own API

You've got a real API on your own computer. Let's use it.

1. With `npm run api` running, visit `http://localhost:3001/books` in the browser. Then try `http://localhost:3001/books/2` and `http://localhost:3001/books?status=want`. Write in a comment what each one returned.
2. Make `src/api/books.ts` as in the notes (type it, don't paste).
3. In `src/ch30/ex2/Ex2.tsx`, show every book with its title, author, and page count, using `getBooks`. Include loading, error, and empty states.
4. Below the list, add a **Look up book 999** button that calls `getBook("999")`. When it fails with a 404, show `That book isn't on your list.` For any other failure, show a general message.
5. Stop `npm run api` and click the button again. Your page should say something different from the 404 case, because no answer arrived at all. Start the API again afterwards.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Add, finish, delete

A reading list you can't change isn't much use. Time for POST, PATCH, and DELETE.

In `src/ch30/ex3/Ex3.tsx`, starting from your exercise 2 list:

1. An **Add a book** button that calls `addBook` with hard-coded values (for example `Project Hail Mary` by Andy Weir, 476 pages, status `"want"`). No form yet: that's [chapter 33](#).
2. A **Mark finished** button on each book that isn't finished yet, using `updateBook(book.id, { status: "finished" })`.
3. A **Delete** button on each book, using `deleteBook`.
4. After **every** change, reload the list from the server, so the page shows what's really saved.
5. While a change is being saved, disable that button, so a double click can't add two books.
6. Open `db.json` in VS Code while you click. Watch your changes appear in the file. Look at the `id` json-server made for the new book.

7. When it all works, write a comment answering: how many places in your code had to remember to reload the list? What would happen if you forgot one?

What you should see: every click changes the list within a moment, and a page refresh shows the same list, because it's really saved.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Interceptors and friendly errors

Your API calls work. Now make them easier to debug, and kinder to the people using your app.

1. **A logger.** In `client.ts`, add a request interceptor to `api` that logs the method and URL of every request, like `[api] GET /books`. Only log while `import.meta.env.DEV` is true.
2. **A fake login.** Add the pretend `Authorization` header from the notes. Check in the Network tab that `/books` requests carry it, and that `mealDb` requests from exercise 1 **don't**.
3. **Break it on purpose:** delete the `return config` line from your request interceptor. Write down exactly what TypeScript says. Then put it back.
4. **Friendly messages.** Make `src/api/errors.ts` with a function `getErrorMessage(err: unknown): string` that turns any error into a sentence a normal person understands:
 - a 404: `We couldn't find that.`
 - a 500 or above: `The server had a problem. Please try again.`
 - a timeout: `The server took too long to answer.`
 - no answer at all: `Couldn't reach the server. Check your connection.`
 - anything else: a general message.
5. Use `getErrorMessage` in your exercise 2 and 3 components instead of `err.message`.
6. **Force a timeout.** Add a temporary test button that calls `api.get("/books", { timeout: 10 })` (just 10 milliseconds), and set the Network tab's throttling to **Slow 3G**. Click it and confirm your timeout message appears. Turn throttling off and remove the button afterwards.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Recipe search, the Axios way

Rebuild the heart of the [Recipe Finder](#) search, with everything this chapter taught. Combines chapters 18, 20, and 21.

1. Make `src/api/meals.ts` with a `Recipe` type (`id`, `name`, `thumbnail`) and a function:

```
export async function searchRecipes(query: string, signal?: AbortSignal):  
    Promise<Recipe[]>
```

It uses `mealDb` with `params` (no hand-built query string), passes the `signal` on to `Axios`, handles `{ meals: null }`, and converts TheMealDB's field names into your `Recipe` type.

2. Write a `useRecipeSearch(query)` hook in `src/ch30/ex5/useRecipeSearch.ts`, like chapter 21's, but cancel stale requests with an `AbortController` instead of an `ignore` flag. An empty query does no request.
3. In `Ex5.tsx`, a search box feeding the query through `useDebounce` (400ms, from [chapter 20](#)), then into your hook. Show loading, error, empty, and results states.
4. A cancelled request must **never** show an error message. Throttle to Slow 3G, type `chicken`, pause, then quickly change it to `beef`. The Network tab should show a `(canceled)` request, and your page should just show beef results.
5. Search for `chicken & rice` and check the real request address in the Network tab. Write in a comment what `Axios` turned it into.
6. Compare your hook with chapter 21's `useRecipeSearch`. What got shorter? What didn't? Write two sentences.

Keep `src/api/meals.ts`. Chapter 31 uses `searchRecipes` again.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

31 TanStack Query

What is it?

TanStack Query is a library that fetches data from a server, keeps a copy close by, and keeps that copy fresh. You tell it *what* data you want and *how* to get it. It handles the rest: loading, errors, cancelling, caching, and refetching.

```
const { data, isPending, isError } = useQuery({
  queryKey: ["books"], // the name of this data
  queryFn: getBooks,   // how to get it (from chapter 30)
});
```

This chapter uses **TanStack Query v5**. It used to be called **React Query**, and you'll still hear that name a lot. It was renamed when it grew versions for other frameworks (the React one is `@tanstack/react-query`). It changes faster than React does, and v5 renamed several things from v4. This chapter points out the renames as they come up.

Install it inside `playground`, along with its **devtools**, a panel that shows you what it's doing:

```
npm install @tanstack/react-query @tanstack/react-query-devtools
```

Why does it matter?

[Chapter 18](#) made a promise: a library would do everything in that chapter for you. Here's what you wrote by hand in chapters 18 and 21:

- Three pieces of state for every request: loading, error, and data.
- The `ignore` flag or an `AbortController`, so an old answer can't overwrite a new one.
- A dependency array, so the request runs again when the search changes.
- One hand-written hook per kind of data (`useRecipeSearch`, `useRecipeDetails`).

And here's what you **never had**, because it's hard to write by hand:

- **Caching.** A **cache** is a saved copy kept close by, so you don't have to ask again. Go to another page and back, and the data is already there. No spinner.
- **No duplicate requests.** Two components want the reading list. One request, shared.
- **Background refreshing.** Old data stays on screen while fresh data loads quietly.
- **Retries.** A request that fails once gets tried again.

- **Fresh data after a change.** In chapter 30's exercise 3 you reloaded the list by hand after every change. Here it's one line.

A new kind of state: server state. Your reading list lives in `db.json`, on the server. What your component holds is only a **copy**. Someone else (another tab, a colleague, you editing the file) can change the original, and your copy won't know. That's **server state: a copy of data that really lives somewhere else, and can go out of date**.

That's very different from a cart or a theme setting. Your app *owns* those. Nobody else can change them behind your back. Server state needs its own tools: a cache, rules for when a copy is too old, and a way to say "this copy is out of date now". `useState` has none of those. TanStack Query is built around them.

[Chapter 29](#) gave state three homes. Level 4 adds a fourth:

Kind	Example	Best home
Local UI state	Is this menu open? Half-typed text	<code>useState</code>
URL state	Filters, search text, which item	The router's search params and path params
Shared client state	Cart, theme, "compact view" setting	Context, or Zustand / Redux Toolkit (34 , 35)
Server state	The list of books on the server	TanStack Query

Real-world example

Think of your **fridge**. It's a cache of the supermarket.

Your fridge	TanStack Query
The supermarket has the real food	The server has the real data
Your fridge holds a copy of some of it	The cache holds a copy of some data
Milk bought today counts as fresh for a while	<code>staleTime</code> : how long data counts as fresh
Bread from a few days ago is still fine, but you pick up a fresh loaf next time you're out	Stale data is still shown, and refetched in the background
Something nobody has touched for ages gets thrown out	<code>gcTime</code> : unused data is thrown away after a while
A note on the fridge: "used the milk, buy fresh"	<code>invalidateQueries</code> : "this copy is out of date now"
Two flatmates want milk. One trip, not two	Two components, same data. One request

Your fridge	TanStack Query
The shopping list says "milk", not "the carton on the left"	The query key names the data you want

How it works

Setting up: one `QueryClient` for the whole app

The `QueryClient` is the cache, plus the rules for it. You make exactly one, and hand it to your whole app with a provider, like [chapter 22](#)'s Context. In `main.tsx`:

```
// src/main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
import "./index.css";
import App from "./App.tsx";

// Made once, outside any component, so it lives as long as the app does
const queryClient = new QueryClient();

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  </StrictMode>
);
```

`ReactQueryDevtools` adds a small floating button in the corner of the page. Click it to open the panel. It's only included while you develop, not in the app you ship.

Your first query: chapter 18's search, again

In chapter 30's exercise 5 you wrote `searchRecipes` in `src/api/meals.ts`. If you skipped it, here's a version:

```
// src/api/meals.ts
import { mealDb } from "../client.ts";

export type Recipe = { id: string; name: string; thumbnail: string };

type MealDbSearchResponse = {
  meals: Array<{ idMeal: string; strMeal: string; strMealThumb: string }> | null;
};

export async function searchRecipes(query: string, signal?: AbortSignal): Promise<Recipe[]> {
  const response = await mealDb.get<MealDbSearchResponse>("/search.php", {
    params: { s: query },
    signal,
  });
  return (response.data.meals ?? []).map((meal) => ({
    id: meal.idMeal,
    name: meal.strMeal,
    thumbnail: meal.strMealThumb,
  }));
}
```

Now chapter 18's `RecipeSearch`, with TanStack Query:

```
// src/ch31/RecipeSearch.tsx
import { useQuery } from "@tanstack/react-query";
import { searchRecipes } from "../api/meals.ts";

function RecipeSearch({ query }: { query: string }) {
  const { data, isPending, isError, error } = useQuery({
    queryKey: ["recipes", query],
    queryFn: ({ signal }) => searchRecipes(query, signal),
    enabled: query !== "",
  });

  if (query === "") return <p>Type something to search.</p>;
  if (isPending) return <p>Searching...</p>;
  if (isError) return <p>Something went wrong: {error.message}</p>;
  if (data.length === 0) return <p>No recipes found.</p>;

  return (
    <ul>
      {data.map((recipe) => (
        <li key={recipe.id}>{recipe.name}</li>
      ))}
    </ul>
  );
}

export default RecipeSearch;
```

Put it next to chapter 18's version. No `useState`. No `useEffect`. No `ignore` flag, no `finally`, no `setIsLoading(false)`. The three options do all of it:

- **queryKey** names the data: "recipes, for this search". It works like a dependency array. When `query` changes, the key changes, and TanStack Query fetches again.
- **queryFn** is how to get the data. It must return a promise, and **throw** if something goes wrong. Axios already throws on a 404 or a 500, so there's nothing extra to write.
- **enabled: query !== ""** means "don't run this at all for an empty search".

Try it: search `chicken`, then `beef`, then `chicken` again. The third search shows its results **instantly**, because the answer was still in the cache. (A quiet request still checks them in the background. More on that below.)

Pending, error, success (and fetching)

`useQuery` gives you a handful of values:

Value	What it means
<code>isPending</code>	There's no data yet
<code>isError</code> and <code>error</code>	The last try failed (after retries). <code>error</code> is an <code>Error</code>
<code>data</code>	The data. <code>undefined</code> until it arrives
<code>isFetching</code>	A request is running right now, including quiet background refreshes
<code>isLoading</code>	<code>isPending</code> and <code>isFetching</code> : the very first load is on its way

TypeScript follows your checks. After `if (isPending) return` and `if (isError) return`, it knows `data` is a `Recipe[]`, so you never need a `!`.

`isFetching` is for refreshes. Data can be on screen *and* updating at the same time: `{isFetching && <small>Updating...</small>}`.

Watch out for this one. In v4, "no data yet" was called `isLoading`. In v5 it's `isPending`. v5 still has `isLoading`, but it means something narrower: pending *and* actually fetching. The difference shows up with `enabled: false`. A disabled query has no data, so `isPending` is `true`, but it isn't fetching, so it stays that way **forever**. That's why `RecipeSearch` checks `query === ""` *before* `isPending`. Swap those two lines and an empty search box says "Searching..." until the end of time.

A tip for search boxes. While a new search loads, the key is new, so there's no data for it yet, and the list vanishes. To keep the old results on screen until the new ones arrive, add `placeholderData: keepPreviousData` (imported from `@tanstack/react-query`). In v4 this was `keepPreviousData: true`.

Query keys: the name of your data

A query key is an array that names a piece of data. Same key, same data, shared by everyone who asks.

```
["books"]           // the whole reading list
["books", "3"]      // one book
["books", { status: "want" }] // a filtered list
["recipes", "chicken"] // a search
```

The rules:

- **Put every value your `queryFn` uses into the key.** If the function uses `query`, the key needs `query`. Leave it out, and a new search shows the *old* search's cached results, because as far as the cache knows, you asked for the same thing.
- **Go from general to specific.** `["books"]` first, then an id or a filter. This lets you refer to a whole family at once: `["books"]` matches every key that *starts with* `"books"`. You'll use that for invalidating.

- **Keys are compared by what's in them.** A fresh `{ status: "want" }` object on every render is fine here. (Remember from [chapter 17](#) that an object in a `useEffect` dependency array reruns the effect every render? Keys don't have that problem.)

Canceling stale requests with `signal`

TanStack Query calls your `queryFn` with a **context object**, a bag of useful things. One of them is `signal`, an `AbortSignal` just like chapter 18's. That's why `RecipeSearch` writes `( { signal } ) => searchRecipes(query, signal)`: it passes the signal on to Axios.

Now, if you type a new search before the old one answers, or the component disappears, TanStack Query cancels the old request. You'll see `(canceled)` in the Network tab. In development, Strict Mode's double mount ([chapter 17](#)) can sometimes show one extra cancelled request on the first load. That's expected.

Even without `signal`, you can't get chapter 17's race condition. Each answer is stored under its *own* key, and the screen shows the key you're asking for now. That's the `ignore` flag's job, done for you. Passing `signal` just stops the wasted work.

Seeing the cache: two components, one request

Here are two components that both want the reading list:

```
import { useQuery } from "@tanstack/react-query";
import { getBooks } from "../api/books.ts";

function BookCount() {
  const { data } = useQuery({ queryKey: ["books"], queryFn: getBooks });
  return <p>{data ? `${data.length} books on your list` : "Counting..."}</p>;
}

function BookTitles() {
  const { data } = useQuery({ queryKey: ["books"], queryFn: getBooks });
  return <ul>{data?.map((book) => <li key={book.id}>{book.title}</li>)}</ul>;
}
```

Render both, with `npm run api` running. The Network tab shows **one** `GET /books`, not two. Open the devtools panel: there's one query, `["books"]`, with a small number showing how many components are using it. Click it to see its data and its state: fresh, fetching, stale, or inactive.

Now hide both components (a toggle button works) and show them again. The data appears **instantly** from the cache, and a quiet refetch checks it's still right.

(Repeating the key and the function in two places is asking for a typo. You'll fix that below with `queryOptions`.)

How long is data fresh? `staleTime` and `gcTime`

Option	Default	What it means
<code>staleTime</code>	<code>0</code>	How long data counts as fresh. Stale data is still shown, but gets refetched when a chance comes up
<code>gcTime</code>	5 minutes	How long data that no component is using stays in the cache before it's thrown away

(`gc` stands for **garbage collection**: clearing out things nobody uses. In v4 this option was called `cacheTime`.)

With `staleTime: 0`, data is stale the moment it arrives. That sounds wasteful, but it's the safe choice: you always show *something* straight away, and check with the server soon after. Set it per query, or for the whole app:

```
// One query: TheMealDB's categories hardly ever change
// (getCategories is a small function like chapter 21's)
useQuery({ queryKey: ["categories"], queryFn: getCategories, staleTime: Infinity });

// Every query in the app
const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 30 * 1000 }, // 30 seconds
  },
});
```

Pick it by asking: **how often does this data really change, and how bad is it to show an old copy for a bit?**

Defaults that surprise beginners

- **Refetch on window focus.** Switch to another window and back, and stale queries refetch. You'll see new requests appear in the Network tab "for no reason". It's there so that someone coming back to a tab after lunch sees fresh data. Turn it off with `refetchOnWindowFocus: false`, or make data stay fresh longer with `staleTime`.
- **Refetch on mount and on reconnect.** A component that mounts, or a laptop that gets its Wi-Fi back, refetches stale queries too.
- **Three retries.** A failed query is tried again 3 times, waiting a bit longer each time (about 1, 2, then 4 seconds). So an error message takes around **7 seconds** to appear. Change it with `retry: 1` or `retry: false`. `retry` can also be a function that gets the error, so you can skip retrying a 404, which will never succeed.

Changing data: `useMutation`

A **mutation** is a request that *changes* something on the server: POST, PATCH, or DELETE. Queries read, and run by themselves. Mutations write, and only run when you say so.

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { addBook } from "../api/books.ts";

function AddBookButton() {
  const queryClient = useQueryClient();

  const addMutation = useMutation({
    mutationFn: addBook,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["books"] }),
  });

  return (
    <button
      disabled={addMutation.isPending}
      onClick={() =>
        addMutation.mutate({ title: "Project Hail Mary", author: "Andy Weir", pages: 476,
status: "want" })
      }
    >
      {addMutation.isPending ? "Adding..." : "Add a book"}
    </button>
  );
}
```

- `useQueryClient()` gives you the same `QueryClient` you made in `main.tsx`.
- `mutate(variables)` runs `mutationFn` with those variables. It doesn't return anything. Errors end up in `addMutation.isError` and `addMutation.error`.
- `mutateAsync(variables)` does the same, but returns a promise you can `await`, to get the saved book back. Then catching errors is your job.
- `isPending` is `true` while the mutation runs (in v4 this was `isLoading` too).
- `invalidateQueries({ queryKey: ["books"] })` is the note on the fridge. It marks every query whose key starts with `"books"` as out of date, and refetches the ones on screen. Every component showing books updates. Nobody had to remember to reload anything.

Returning the promise from `onSuccess`, like above, keeps `isPending` `true` until the fresh list has arrived. So the button doesn't say "Add a book" again before the new book is on screen.

One argument only. `mutationFn` receives a single value. `updateBook` needs two, so wrap them in an object:

```
mutationFn: ({ id, changes }: { id: string; changes: Partial<NewBook> }) => updateBook(id,
    changes),
// Later: statusMutation.mutate({ id: book.id, changes: { status: "finished" } });
```

In [chapter 33](#), a real "Add a book" form will submit through a mutation like this one.

Showing the change straight away: optimistic updates

Chat apps show your message the moment you press Send, with a grey tick, before the server has said yes. That's an **optimistic update**: show the change now, and fix things up if it fails. It makes an app feel instant. It's worth it when the server nearly always says yes. You'll use it in the [task board project](#) for moving tasks between columns.

The simple way: show the variables while it's pending. The mutation remembers what you sent in `variables`:

```
const addMutation = useMutation({
  mutationFn: addBook,
  onSettled: () => queryClient.invalidateQueries({ queryKey: ["books"] }),
});

// inside the list's JSX:
{addMutation.isPending && <li style={{ opacity: 0.5 }}>{addMutation.variables.title}</li>}
```

`onSettled` runs after success or failure. The grey row shows until the real list comes back. This only changes one spot on screen, which is often all you need.

The cache way: change the cached data directly. When the change must show up everywhere, edit the cache itself. Here's an optimistic delete:

```

const deleteMutation = useMutation({
  mutationFn: deleteBook,
  onMutate: async (id) => {
    await queryClient.cancelQueries({ queryKey: ["books"] }); // 1. stop refetches
    // overwriting us
    const previous = queryClient.getQueryData<Book[]>(["books"]); // 2. save a snapshot
    queryClient.setQueryData<Book[]>(["books"], (old) => // 3. change the cache now
      old?.filter((book) => book.id !== id)
    );
    return { previous }; // 4. hand the snapshot on
  },
  onError: (_error, _id, snapshot) => {
    queryClient.setQueryData(["books"], snapshot?.previous); // 5. it failed: put it back
  },
  onSettled: () => queryClient.invalidateQueries({ queryKey: ["books"] }), // 6. check with the
  // server
});

```

`onMutate` runs **before** the request is sent. The comments tell the story, but two steps need a word more. **Cancel first** (step 1), because a refetch already on its way would land its old answer on top of your change. **Hand the snapshot on** (step 4), because whatever `onMutate` returns is passed to `onError`, which uses it to put things back.

Try it: stop `npm run api`, then delete a book. It vanishes, then comes back when the request fails. (The list's own refetch fails too while the server is off, so start it again afterwards.)

About the names. Recent v5 docs call that third `onError` argument `onMutateResult`, and add a fourth argument with the client in it. Older v5 docs and tutorials call the third one `context`. It's the same value either way, in the same position, so the code above works with both.

Reusing queries: `queryOptions` and custom hooks

Writing `{ queryKey: ["books"], queryFn: getBooks }` in every component invites typos. The `queryOptions` helper lets you write each query once:

```
// src/api/bookQueries.ts
import { queryOptions, useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { addBook, getBook, getBooks } from "../books.ts";

export const booksQuery = queryOptions({
  queryKey: ["books"],
  queryFn: getBooks,
});

export function bookQuery(id: string) {
  return queryOptions({
    queryKey: ["books", id],
    queryFn: () => getBook(id),
  });
}

export function useBooks() {
  return useQuery(booksQuery);
}

export function useBook(id: string) {
  return useQuery(bookQuery(id));
}

export function useAddBook() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: addBook,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: booksQuery.queryKey }),
  });
}
```

At runtime, `queryOptions` just hands back what you gave it. The win is in the types: its key remembers what data it holds. `queryClient.getQueryData(booksQuery.queryKey)` is a `Book[] | undefined` without you writing `<Book[]>`.

The hooks are [chapter 20](#)'s idea: hide the details behind a clear name. A component just says `const { data, isPending } = useBooks();`. Compare that with chapter 21's hand-written `useRecipeSearch`, around thirty lines. This one is three, and does more.

Suspense mode: `useSuspenseQuery`

[Chapter 26](#) said `<Suspense>` only works with things that opt in, and TanStack Query is one of them:

```
import { Suspense } from "react";
import { ErrorBoundary } from "react-error-boundary";
import { useSuspenseQuery } from "@tanstack/react-query";
import { booksQuery } from "../api/bookQueries.ts";

function BookList() {
  const { data } = useSuspenseQuery(booksQuery); // data is always there
  return <ul>{data.map((book) => <li key={book.id}>{book.title}</li>)}</ul>;
}

function ReadingListPage() {
  return (
    <ErrorBoundary fallback=<p>Couldn't load your books.</p>>
      <Suspense fallback=<p>Loading your books...</p>>
        <BookList />
      </Suspense>
    </ErrorBoundary>
  );
}
```

`BookList` has no loading or error checks at all. While the data loads, the nearest `<Suspense>` shows its fallback. If it fails, the error goes to the nearest error boundary. Same order as chapter 26: **boundary outside, Suspense inside**.

Two things to know. Suspense queries have no `enabled` option, since they always need their data. And if your fallback has a "Try again" button, wrap it in TanStack Query's `QueryErrorResetBoundary` so the query really retries (see the docs when you need it). Plain `useQuery` is still the everyday choice. Reach for suspense when you want a parent to decide the loading screen for a whole section.

Testing components that use queries

A component using `useQuery` needs a `QueryClientProvider` in tests too, just like context needed its provider in [chapter 28](#). Two rules:

- **A fresh `QueryClient` for every test.** Otherwise one test's cached data leaks into the next.
- `retry: false`. Otherwise a failure test waits through three retries and times out.


```
// src/test-utils.tsx
import type { ReactNode } from "react";
import { render } from "@testing-library/react";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

export function renderWithProviders(ui: ReactNode) {
  const queryClient = new QueryClient({
    defaultOptions: { queries: { retry: false } },
  });
  return render(<QueryClientProvider client={queryClient}>{ui}</QueryClientProvider>);
}
```

(If your components also use the router, put chapter 28's `MemoryRouter` in there too.)

Chapter 28 stubbed `fetch`. Axios doesn't use `fetch` by default, so stub your own API file instead. `vi.mock` swaps a whole module for fake functions:

```
// src/ch31/BookList.test.tsx
import { expect, test, vi } from "vitest";
import { screen } from "@testing-library/react";
import { getBooks } from "../api/books.ts";
import { renderWithProviders } from "../test-utils.tsx";
import { BookList } from "../BookList.tsx"; // a list that calls useBooks()

vi.mock("../api/books.ts"); // every function in that file becomes a vi.fn()

test("shows the books from the API", async () => {
  vi.mocked(getBooks).mockResolvedValue([
    { id: "1", title: "Dune", author: "Frank Herbert", pages: 412, status: "want" },
  ]);

  renderWithProviders(<BookList />);

  expect(await screen.findByText("Dune")).toBeInTheDocument();
});
```

`vi.mocked(getBooks)` is the same function, typed as a mock, so TypeScript checks your fake data is a real `Book[]`. The test never touches the network, and json-server doesn't need to be running.

When you don't need it

- **One request, on one page, for data that never changes.** Chapter 18's pattern, or chapter 20's `useFetch`, is fine.
- **Data that doesn't come from a server.** A cart, a theme, an open menu: that's client state. TanStack Query isn't a general store. Use `useState`, `Context`, or [Zustand](#).

- **A framework that fetches on the server.** In Next.js ([chapter 39](#)), many pages get their data before they reach the browser, and need no client-side fetching library at all.

Other libraries share the same idea. You may meet **SWR**, which is similar. Redux has its own version, RTK Query, in [chapter 35](#).

Common mistakes

1. A value missing from the query key

```
useQuery({ queryKey: ["recipes"], queryFn: () => searchRecipes(query) }); // ❌ every search  
    shares one cache entry  
useQuery({ queryKey: ["recipes", query], queryFn: () => searchRecipes(query) }); // ✅
```

If the function uses it, the key needs it. The ESLint plugin `@tanstack/eslint-plugin-query` has a rule that catches this for you.

2. Making the `QueryClient` inside a component

```
function App() {  
  const queryClient = new QueryClient(); // ❌ a brand-new, empty cache on every render  
}
```

Make it once, outside any component, as in `main.tsx` above.

3. Forgetting the provider

```
Error: No QueryClient set, use QueryClientProvider to set one
```

Some component called `useQuery` with no `QueryClientProvider` above it. In the app, check `main.tsx`. In tests, use `renderWithProviders`.

4. Copying data into `useState`

```
const { data } = useBooks();  
const [books, setBooks] = useState(data); // ❌ a second copy that never refreshes
```

Your `useState` copy is frozen at the first value, often `undefined`. Use `data` directly, and work out anything else from it during render ([chapter 08](#)).

5. Forgetting to invalidate after a mutation

The server changed, but the screen didn't. Then you switch windows, a focus refetch runs, and it "fixes itself", which is very confusing. Invalidate the right key in `onSuccess` or `onSettled`.

6. Expecting `mutate` to return the saved data

```
const saved = addMutation.mutate(newBook); // ❌ saved is undefined
```

Use `onSuccess`, or `await addMutation.mutateAsync(newBook)` inside a `try / catch`.

7. A `queryFn` that doesn't throw on HTTP errors

With plain `fetch` and no `response.ok` check, a 404 page counts as success, and gets cached as your data. Axios already throws. If you use `fetch`, check `response.ok` and throw, like chapter 18.

8. Checking `isPending` for a disabled query

With `enabled: false`, `isPending` stays `true` forever, so the page says "Loading..." for good. Handle the "not asked yet" case first, as `RecipeSearch` does.

9. Following a v4 tutorial

Lots of tutorials online are for v4 or older. The usual differences:

Old (v4 and earlier)	v5
<code>react-query</code> package	<code>@tanstack/react-query</code>
<code>useQuery(["books"], getBooks)</code>	<code>useQuery({ queryKey: ["books"], queryFn: getBooks })</code>
<code>isLoading</code> for "no data yet"	<code>isPending</code>
<code>cacheTime</code>	<code>gcTime</code>
<code>keepPreviousData: true</code>	<code>placeholderData: keepPreviousData</code>
<code>onSuccess</code> / <code>onError</code> on <code>useQuery</code>	Removed from queries (mutations still have them)

Quick recap

- **Server state** is a copy of data that really lives somewhere else, and can go out of date. It needs caching and refetching, which is what TanStack Query does.
- Make **one `QueryClient`** outside your components, and wrap the app in `QueryClientProvider`. Use the devtools to see the cache.
- `useQuery({ queryKey, queryFn })` replaces chapter 18's three states, the effect, and the `ignore` flag. Pass `signal` to Axios to cancel stale requests.

- **The query key names the data.** Put every value the `queryFn` uses into it. Keys go from general to specific: `["books"]`, then `["books", id]`.
 - Defaults: `staleTime` 0, `gcTime` 5 minutes, refetch on window focus, 3 retries. Change them when they don't fit.
 - `useMutation` changes data. **Invalidate** the matching key afterwards, and every screen updates.
 - **Optimistic updates** show a change before the server confirms it: with `variables` for one spot, or with `onMutate` and a rollback for the whole cache.
 - Wrap queries in `queryOptions` and custom hooks like `useBooks()`. In tests, use a fresh `QueryClient` with `retry: false`.
-

Next: try the [exercises](#), then move on to [32 Zod](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch31/ex1/`, and so on.
 - Install TanStack Query and its devtools, and set up `QueryClientProvider` and `ReactQueryDevtools` in `main.tsx`, as in the notes.
 - Reuse your `src/api/` files from chapter 30: `client.ts`, `books.ts`, and `meals.ts`.
 - Exercises 2, 3, and 5 use the practice API. Keep it running with `npm run api` in a second terminal.
 - Keep two panels open: the DevTools **Network** tab, and the TanStack Query devtools (the floating button in the corner of your page).
 - An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): The recipe search, one more time

You've written this search three times now: by hand in chapter 18, with a hook in chapter 21, and with Axios in chapter 30. This is the last time, promise.

In `src/ch31/ex1/Ex1.tsx`:

1. A search box whose text goes through `useDebounce` (400ms) and then into `useQuery`, using `searchRecipes` from `src/api/meals.ts`.

2. Show four states: "Type something to search", "Searching...", an error, and the results (or "No recipes found").
3. Don't run a request for an empty search. Pass `signal` on to Axios.
4. Search `chicken`, then `beef`, then `chicken` again. Write down what happened on the third search, and what the Network tab showed.
5. **Break it on purpose (1):** take `query` out of the query key, so it's just `["recipes"]`. Search for two different things. Write down what you see, and why. Then put it back.
6. **Break it on purpose (2):** move your `isPending` check above the empty-search check. Clear the box. Write down what the page says, and why. Then put it back.
7. Count the lines in your component and compare them with your chapter 18 version. Write both numbers in a comment.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Cache experiments

Time to watch the cache work, with the reading list. No new features, just experiments. Write what you see in comments as you go.

In `src/ch31/ex2/Ex2.tsx`:

1. Build `BookCount` and `BookTitles` from the notes, both using the key `["books"]`. Render both.
2. **One request:** check the Network tab. How many `GET /books` requests were there? In the devtools panel, how many components does it say are using `["books"]`?
3. **Instant return:** add a **Show / hide** button around both components. Hide them, and look at the query's state in the devtools. Show them again. Did you see "Counting..."? Did a request run?
4. **Refetch on focus:** click into another window, then back into the browser. Watch the Network tab.
5. **Fresh for a while:** give both queries `staleTime: 30 * 1000`. Repeat step 4 straight away, then again after 30 seconds. Watch the query's label change in the devtools.
6. **Thrown away:** give both queries `gcTime: 5000`. Hide the components, wait 10 seconds, and watch the devtools. Then show them again. What's different from step 3?
7. **Server state, for real:** with the page open, edit `db.json` by hand and change a book's title. Save. Look at your page (nothing changes yet). Now click into another window and back. What happened, and which default made it happen?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The reading list, with mutations

In chapter 30's exercise 3, you reloaded the list by hand after every change. Let's do it properly.

In `src/ch31/ex3/Ex3.tsx`:

1. Show every book with `useQuery` and the key `["books"]`.
2. An **Add a book** button (hard-coded values are fine) using `useMutation` with `addBook`.
3. A `<select>` on each book to change its status between `want`, `reading`, and `finished`, using `updateBook`.
4. A **Delete** button on each book, using `deleteBook`.
5. Every mutation invalidates `["books"]`. Disable each control while its mutation is pending.
6. A separate `ReadingSummary` component, placed somewhere else on the page, that shows `2 want to read`
`• 1 reading • 2 finished`. It gets **no props**: it calls `useQuery` with the same key. It must update by itself after every change.
7. Add three filter buttons: **Want, Reading, Finished**. Add a function `getBooksByStatus(status: BookStatus)` to `books.ts` that uses `params: { status }`, and give the filtered list the key `["books", { status }]`. Change a book's status and check that the filtered list updates too, even though you only invalidated `["books"]`.
8. In a comment, compare this with chapter 30's exercise 3. How many "reload the list" calls did you need this time?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): One query waits for another

Back to [chapter 18's exercise 5](#): choose a category, then see its meals. That needed two effects and two sets of state. Let's see how it looks now.

In `src/ch31/ex4/Ex4.tsx`:

1. Add two functions to `src/api/meals.ts`: `getCategories()` (from `/list.php`, with `params: { c: "list" }`) and `getMealsByCategory(category: string, signal?: AbortSignal)` (from `/filter.php`, with `params: { c: category }`).
2. A `<select>` of categories from a query with the key `["categories"]`. Categories hardly ever change, so give it `staleTime: Infinity`.
3. A second query for the chosen category's meals. It must not run until a category is chosen.
4. A default `-- choose a category --` option. Choosing it again clears the meal list.
5. Choose **Beef**, then **Chicken**, then **Beef** again. The third choice should be instant.
6. While a *background* refetch runs, show a small `Updating...` next to the list, without hiding it.

- Count your lines against your chapter 18 version, and write both numbers in a comment. Did you need to do anything special to keep the two loading states separate?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Instant status changes, custom hooks, and a test

Make the reading list feel instant, then prove it with a test. Combines this chapter with chapters 20 and 28.

- In `src/api/bookQueries.ts`, write `booksQuery` with `queryOptions`, a `useBooks()` hook, and a `useUpdateBook()` hook that returns a mutation for changing a book.
- Make `useUpdateBook` **optimistic**, the cache way: cancel `["books"]` queries, save a snapshot, change the book in the cache, roll back on error, and invalidate when settled.
- Rebuild exercise 3's status `<select>` with your hooks. Throttle to **Slow 3G**: the status should change the moment you choose it, long before the request finishes.
- Force a failure:** stop `npm run api`, then change a status. It should change, then snap back. Show a message like `Couldn't save that change, so it was undone`.
- You'll use this exact pattern in the [task board project](#) for moving tasks between columns. Write a comment explaining, in your own words, why step 2's "cancel" part is needed.
- A test.** Add `QueryClientProvider` to your `renderWithProviders`, with a fresh `QueryClient` and `retry: false`. With `vi.mock("../api/books.ts")`, write a test that the list shows the books that `getBooks` returns.
- Bonus test:** make `updateBook` reject, change a book's status with `user-event`, and check that the old status comes back.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

32 Zod

What is it?

Zod is a library for checking data while your app runs. You describe what good data looks like as a **schema**: a description of the data's shape, written as real code. Zod then checks real data against it. As a bonus, it gives you the matching TypeScript type for free.

```
import { z } from "zod";

const BookSchema = z.object({
  title: z.string(),
  pages: z.number(),
});

BookSchema.parse({ title: "Dune", pages: 412 }); // ✓ gives the data back
BookSchema.parse({ title: "Dune", pages: "412" }); // ✗ throws an error: pages should be a number
```

This chapter uses **Zod 4**. Install it in your `playground` app:

```
npm install zod
```

A warning about older tutorials. Zod 4 came out in 2025, and lots of what you'll find online (blog posts, Stack Overflow answers, AI answers) is still written for Zod 3. Most code is the same in both, but a few names changed:

Zod 3 (older tutorials)	Zod 4 (this chapter)
<code>z.string().email()</code>	<code>z.email()</code>
<code>error.flatten()</code>	<code>z.flattenError(error)</code>
<code>{ message: "Too short" }</code>	<code>{ error: "Too short" }</code>
<code>.passthrough()</code>	<code>z.looseObject({...})</code>

The old names mostly still work in Zod 4, but they're **deprecated** (on their way out), so VS Code shows them ~~crossed out~~. If an example online looks slightly off, check the [Zod docs](#) before you copy it.

Why does it matter?

TypeScript only exists while you're writing code. When Vite builds your app, every type is **erased**: deleted, leaving plain JavaScript for the browser to run. So a type can't check anything that arrives while the app is running, like a server's reply, a value saved in `localStorage`, or text someone typed.

You've made this kind of promise three times already:

```
return parsed as Task[]; // chapter 10: Loading saved tasks
const data: MealDbResponse = await response.json(); // chapters 18 and 21: TheMealDB
const response = await api.get<Book[]>("/books"); // chapter 30: the reading list
```

Each line says "trust me, it's this shape", and TypeScript believes you. Each time, the chapter said the same thing: this is **a promise, not a proof**, and Zod is how you check for real. Time to keep that promise.

Here's what goes wrong without a check. Say the reading list's server changes, and `title` is renamed to `name`. Your code still compiles. Then one of these happens:

- A component calls `book.title.toUpperCase()`, and the page crashes with `TypeError: Cannot read properties of undefined (reading 'toUpperCase')`. The error points at the component, far away from the real cause.
- Nothing crashes, and every title is quietly blank.
- Or `pages` arrives as the text `"412"`, and your "total pages read" comes out as `"310412320"`, because `+` joins strings together.

The bug is in the data, but it shows up somewhere else, later. Those are the hardest bugs to track down.

With Zod, bad data is stopped at the door, with a message saying exactly what was wrong and where:

```
✗ Invalid input: expected string, received undefined
  → at [2].title
```

That reads as "the book at index 2 has no title". One clear, early failure instead of a confusing one later.

Real-world example

Think about a **party with a bouncer at the door**.

At the party	In your app
The guest list you wrote down	Your TypeScript type. It says who should come in, but it can't stop anyone.

At the party	In your app
The bouncer checking each person's ID	Zod's <code>parse</code> , checking each piece of data as it arrives
"Sorry, you're not on the list, and here's why"	A <code>ZodError</code> , with the exact field and the reason
One bouncer, at the front door	Checking at the boundary , where data enters your app, not in every room
The host and the bouncer share one list	One schema gives you both the check <i>and</i> the type

Once a guest is inside, nobody checks their ID again. That's the deal: check carefully at the door, then trust what's inside.

How it works

Your first schema

A schema is built from small pieces snapped together:

```
import { z } from "zod";

const BookSchema = z.object({
  id: z.string(),
  title: z.string(),
  author: z.string(),
  pages: z.number(),
});
```

`z.object({...})` means "an object with these keys". `z.string()`, `z.number()` and `z.boolean()` each describe one value. Everything comes from the one `z` import.

`parse` and `safeParse`

There are two ways to check data. `parse` gives the data back if it's good, and **throws** an error if it isn't:

```
const book = BookSchema.parse({ id: "1", title: "The Hobbit", author: "J.R.R. Tolkien", pages: 310 });
console.log(book.title); // prints: The Hobbit

BookSchema.parse({ id: "2", title: "Atomic Habits", author: "James Clear", pages: "320" });
// ❌ throws a ZodError
```

`safeParse` never throws. It gives back a result object that says whether it worked:

```
const result = BookSchema.safeParse({ id: "2", title: "Atomic Habits", author: "James Clear",
  pages: "320" });

if (result.success) {
  console.log(result.data.title); // the checked data
} else {
  console.log(result.error.issues); // what was wrong
}
```

```
[
  {
    expected: 'number',
    code: 'invalid_type',
    path: [ 'pages' ],
    message: 'Invalid input: expected number, received string'
  }
]
```

`result` is a union: either `{ success: true, data }` or `{ success: false, error }`. Checking `result.success` narrows it, just like the discriminated unions in TypeScript chapter 06. Try to read `result.data.title` before the `if`, and TypeScript stops you with `'result.data' is possibly 'undefined'`. It's the same idea as the `Result<T>` type from TypeScript chapter 13, ready-made.

Which one should you use?

- `parse` when bad data means something is genuinely broken, like a server sending the wrong shape. Throwing is right, because it *is* an error.
- `safeParse` when bad data is normal and expected, like what someone typed into a form, or an old value in `localStorage`. You want to handle it, not crash.

One source of truth: `z.infer`

You don't write the `Book` type by hand any more. Zod works it out from the schema:

```
type Book = z.infer<typeof BookSchema>;
// { id: string; title: string; author: string; pages: number }
```

Read it as "the type that `BookSchema` checks for". `typeof BookSchema` gets the type of the schema value (TypeScript chapter 10), and `z.infer` pulls the data's type out of it.

This matters more than it looks. If you wrote a type *and* a schema, one day someone would add a field to one and forget the other. With `z.infer`, there's only one thing to change. The schema is the **single source of truth**: the one place the answer lives.

`parse` returns that type too. `BookSchema.parse(data)` hands you a `Book`, with no `as` anywhere.

Arrays, fixed choices, and missing values

A book in the reading list also has a status. `z.enum` describes a value that must be one of a fixed list of strings:

```
const BookSchema = z.object({
  id: z.string(),
  title: z.string(),
  author: z.string(),
  pages: z.number(),
  status: z.enum(["want", "reading", "finished"]), // "want" | "reading" | "finished"
});
```

The inferred type for `status` is the union `"want" | "reading" | "finished"`, exactly like chapter 30's handwritten `BookStatus`.

A few more pieces you'll use all the time, shown on a reader's profile:

```
const ReaderSchema = z.object({
  name: z.string(),
  favouriteGenres: z.array(z.string()), // string[]
  nickname: z.string().optional(), // may be missing
  currentBookId: z.string().nullable(), // must be there, but may be null
  newsletter: z.boolean().default(false), // missing? use false
});
```

- `z.array(X)`: an array where every item matches `X`.
- `.optional()`: the key can be left out. The type is `string | undefined`.
- `.nullable()`: the key must be there, but its value can be `null`. TheMealDB's `{ meals: null }` is exactly this.
- `.default(false)`: if the value is missing, Zod fills in `false`. So the data you get back always has it.

One subtle point about `.default()`: the data *going in* can skip that key, but the data *coming out* always has it. `z.infer` gives you the coming-out type. If you ever need the going-in type, it's `z.input<typeof ReaderSchema>`. You'll meet this again in [chapter 33](#).

Rules for strings and numbers

A type says "it's a string". Real data needs more, like "a string that isn't empty". Zod adds rules by chaining them on:

```
const Title = z.string().trim().min(1, "Please enter a title").max(200, "Keep the title under 200 characters");

const Email = z.email("Please enter a valid email");

const Pages = z
  .number({ error: "Pages must be a number" })
  .int("Pages must be a whole number")
  .positive("Pages must be more than 0");
```

- `.trim()` removes spaces from both ends **before** the later rules run. So `" "` becomes `""`, and `.min(1)` catches it. The trimmed text is also what you get back: `Title.parse(" Dune ")` gives `"Dune"`.
- `.min()` and `.max()` on a string count characters.
- `z.email()` checks the text looks like an email address.
- `.int()` means a whole number. `.positive()` means more than 0. Numbers have `.min()` and `.max()` too.
- `{ error: "..."} on z.number() itself is the message for "this isn't a number at all".`

The text at the end of each rule is your own error message. It's a shortcut for `{ error: "..."} . Leave it out and Zod uses its default, like Too small: expected number to be >0 . That's fine for a developer, but not something to show your users.`

Reading the errors

Every failed check gives you a `ZodError` . Its `issues` array lists **every** problem, not just the first. Each issue has a `path` (where) and a `message` (what):

```
const result = BookSchema.safeParse({
  id: "1",
  name: "Dune", // oops: should be "title"
  author: "Frank Herbert",
  pages: 412,
  status: "someday",
});

if (!result.success) {
  for (const issue of result.error.issues) {
    console.log(issue.path.join("."), "→", issue.message);
  }
}
```

```
title → Invalid input: expected string, received undefined
status → Invalid option: expected one of "want"|"reading"|"finished"
```

For a person to read, `z.prettifyError(result.error)` gives a tidy summary, like the one at the top of this chapter.

For forms, you usually want "the errors for each field". `z.flattenError` groups the messages by field name:

```
const fieldErrors = z.flattenError(result.error).fieldErrors;
// { title: ["Invalid input: expected string, received undefined"], status: ["Invalid option: ..."] }

fieldErrors.title?.[0]; // the first message for title, or undefined if title is fine
```

Each field gets an array, because one field can break several rules at once. There's also `formErrors`, for problems that don't belong to one field.

Changing the shape: `.transform()`

In [chapter 18](#), you gave TheMealDB's reply a type, then converted `idMeal`, `strMeal` and `strMealThumb` into your own clean `Recipe`. In [chapter 30](#) you did the same again, in `searchRecipes` in `src/api/meals.ts`: a `MealDbSearchResponse` type (a promise) and a `.map(...)` by hand.

Zod can do both jobs in one step. `.transform()` runs after the checks pass, and turns the data into whatever you return. Replace the hand-written type and mapping in `src/api/meals.ts`:

```
// src/api/meals.ts
import { z } from "zod";
import { mealDb } from "../client.ts";

export const MealSchema = z
  .object({
    idMeal: z.string(),
    strMeal: z.string(),
    strMealThumb: z.string(),
  })
  .transform((meal) => ({
    id: meal.idMeal,
    name: meal.strMeal,
    thumbnail: meal.strMealThumb,
  }));

export const MealListSchema = z
  .object({ meals: z.array(MealSchema).nullable() })
  .transform((data) => data.meals ?? []); // TheMealDB sends null for "no results"

export type Recipe = z.infer<typeof MealSchema>;
// { id: string; name: string; thumbnail: string }

export async function searchRecipes(query: string, signal?: AbortSignal): Promise<Recipe[]> {
  const response = await mealDb.get<unknown>("/search.php", {
    params: { s: query },
    signal,
  });
  return MealListSchema.parse(response.data);
}
```

This is chapter 18's boundary rule, now enforced. The ugly field names live in one schema, get checked, and never leak into your components. `z.infer` gives the type **after** the transform, so `Recipe` is exactly the clean shape chapter 30 wrote by hand, and nothing that uses `searchRecipes` needs to change.

Notice `mealDb.get<unknown>`. That's you being honest: you don't know what's in the reply yet. Chapter 30's `<Book[]>` was a promise. `<unknown>` plus a schema is a proof.

Text that should be a number: `z.coerce`

Some data always arrives as text, even when it means a number. URL search params are the classic case: `?page=3` gives you `"3"`. **Coercing** means converting the value first, then checking it:

```
const PageNumber = z.coerce.number().int().positive();

PageNumber.parse("3"); // 3, a real number
PageNumber.parse("abc"); // ❌ Invalid input: expected number, received NaN
```

`z.coerce.number()` runs JavaScript's `Number(...)` on the input first. That brings along `Number`'s surprises:

```
z.coerce.number().parse(""); // 0, because Number("") is 0
z.coerce.boolean().parse("false"); // true, because any non-empty string is true
```

An empty box silently becomes `0`. It's the same trap as `Number(event.target.value)` in [chapter 09](#). So only coerce where the input really is text, and add a rule (like `.positive()`) that catches the `0`. For `"true"` and `"false"` text, use `z.stringbool()`, which understands them properly.

Rules across fields: `.refine()`

Some rules need two fields at once. "Confirm password must match password" can't be checked by looking at one field alone. `.refine()` adds your own check to the whole object:

```
const SignUpSchema = z
  .object({
    email: z.email("Please enter a valid email"),
    password: z.string().min(8, "Password must be at least 8 characters"),
    confirmPassword: z.string(),
  })
  .refine((data) => data.password === data.confirmPassword, {
    error: "Passwords don't match",
    path: ["confirmPassword"],
  });
```

Your function returns `true` for good data and `false` for bad. `path` says which field the error belongs to, so it can appear under the confirm box instead of floating at the top of the form:

```
const result = SignUpSchema.safeParse({
  email: "maya@example.com",
  password: "12345678",
  confirmPassword: "1234",
});

if (!result.success) {
  console.log(z.flattenError(result.error).fieldErrors);
  // { confirmPassword: ["Passwords don't match"] }
}
```


A small Zod 4 detail: the refine still runs when other fields break rules like `.min()` , so people see all their mistakes at once. It's skipped if a field has the wrong *type* altogether, like a number where a string should be. Form fields are strings, so this rarely comes up.

Extra keys are dropped

```
const book = BookSchema.parse({ id: "1", title: "Dune", author: "Frank Herbert", pages: 412,
  status: "want", rating: 5 });
console.log(book); // no rating in here: it's gone
```

By default, `z.object` **strips** keys it doesn't know about. That's usually what you want: your app only gets what it asked for. If you'd rather:

- **fail** on unknown keys, use `z.strictObject({...})` . The error says `Unrecognized key: "rating"` .
- **keep** them, use `z.looseObject({...})` .

If a field seems to vanish after parsing, this is why. Add it to the schema.

Where to validate: at the boundaries

You don't sprinkle Zod everywhere. You check where data **enters** your app from somewhere you don't control. Those places are called **boundaries**:

Boundary	Example	Use
API responses	<code>getBooks()</code>	<code>parse</code> , inside the API function
Saved data	<code>localStorage</code> , loaded at startup	<code>safeParse</code> , with a fallback
Forms	What someone typed	<code>safeParse</code> , or React Hook Form (chapter 33)
URL params	<code>?page=3</code> , <code>/books/:id</code>	<code>safeParse</code> , often with <code>z.coerce</code>

Inside your app, between your own components and functions, TypeScript already checks everything. Once data is through the door, trust it. Checking it again in every component is like the bouncer following each guest from room to room.

The reading list, checked for real

Now apply this to chapter 30's `src/api/books.ts` . The hand-written types go, and schemas take their place:

```
// src/api/books.ts
import { z } from "zod";
import { api } from "../client.ts";

export const BookStatusSchema = z.enum(["want", "reading", "finished"]);

export const BookSchema = z.object({
  id: z.string(),
  title: z.string().trim().min(1, "Please enter a title"),
  author: z.string().trim().min(1, "Please enter an author"),
  pages: z
    .number({ error: "Pages must be a number" })
    .int("Pages must be a whole number")
    .positive("Pages must be more than 0"),
  status: BookStatusSchema,
});

export const BookListSchema = z.array(BookSchema);

// Everything except id: what you send when you add a book
export const NewBookSchema = BookSchema.omit({ id: true });

export type BookStatus = z.infer<typeof BookStatusSchema>;
export type Book = z.infer<typeof BookSchema>;
export type NewBook = z.infer<typeof NewBookSchema>;
```

Three things to notice:

- **The type names haven't changed.** `Book`, `NewBook` and `BookStatus` are the same names chapter 30 used, so no other file needs to change.
- `.omit({ id: true })` is Zod's version of TypeScript's `Omit<Book, "id">` (TypeScript chapter 09). `NewBookSchema` gets all the same rules for free: title and author required, pages a positive whole number, status one of the three. [Chapter 33](#) uses it to check the "Add a book" form.
- **The rules are written once**, on `BookSchema`, and both the API and the form use them.

Then the API functions check what comes back:

```
export async function getBooks(): Promise<Book[]> {
  const response = await api.get<unknown>("/books");
  return BookListSchema.parse(response.data);
}

export async function getBook(id: string): Promise<Book> {
  const response = await api.get<unknown>(`/books/${id}`);
  return BookSchema.parse(response.data);
}
```

Do the same in `addBook` and `updateBook`. The server replies with the saved book, so `BookSchema.parse(response.data)` works there too. `deleteBook` gets nothing back, so there's nothing to check.

`api.get<Book[]>` has become `api.get<unknown>` plus a real check. You've swapped a promise for a proof, and the API can no longer lie to your components.

What TanStack Query shows when the data is wrong

Your [chapter 31](#) code doesn't change at all. `useBooks()` from `src/api/bookQueries.ts` still gives you a `Book[]`. The difference is what happens when the data is bad.

Try it. With the API running, open `db.json`, change one book to `"pages": "lots"`, and save. (json-server 1 notices hand edits and reloads the file, as chapter 30 said.) Then reload your book list.

`getBooks` calls `parse`, and `parse` throws a `ZodError`. TanStack Query treats that like any other failed request:

1. It **retries**, 3 times by default, waiting a little longer each time. So for about 7 seconds you see your loading state.
2. Then the query goes into its **error state**, and `error` is the `ZodError`.

If your component shows `error.message`, you'll see something like this on the page:

```
[
  {
    "expected": "number",
    "code": "invalid_type",
    "path": [2, "pages"],
    "message": "Invalid input: expected number, received string"
  }
]
```

Ugly, but honest: it points straight at the `pages` of the book at index 2. Before Zod, the list would have rendered happily, and your "total pages" would have quietly turned into nonsense. A loud, early failure at the boundary beats a quiet, late one deep inside a component.

For real users, show a friendly message and keep the details in the Console. `instanceof` narrows the error, as in TypeScript chapter 13:

```

if (error) {
  if (error instanceof z.ZodError) {
    console.error(z.prettifyError(error)); // the details, for you
    return <p>The server sent book data we didn't expect.</p>;
  }
  return <p>Something went wrong: {error.message}</p>;
}

```

If you need this in several components, move it into a small `getErrorMessage(error)` function. And since retrying won't fix bad data, you can tell TanStack Query not to bother. Chapter 31 said `retry` can be a function that gets the error. Add this to `booksQuery` in `bookQueries.ts`:

```

retry: (failureCount, error) => !(error instanceof z.ZodError) && failureCount < 3,

```

Put `db.json` back when you're done.

Loading from `localStorage` safely

Chapter 10's `loadTasks` checked `Array.isArray`, then promised `as Task[]`. Here's the Zod way, shown with a small settings object for the reading list:

```

const SettingsSchema = z.object({
  view: z.enum(["grid", "list"]),
  showFinished: z.boolean(),
});

type Settings = z.infer<typeof SettingsSchema>;

const defaultSettings: Settings = { view: "grid", showFinished: true };

function loadSettings(): Settings {
  const saved = localStorage.getItem("reading-list-settings");
  if (saved === null) return defaultSettings;

  try {
    const result = SettingsSchema.safeParse(JSON.parse(saved));
    return result.success ? result.data : defaultSettings;
  } catch {
    return defaultSettings; // not even valid JSON
  }
}

```

There are two kinds of bad data here, and two guards. `try / catch` handles text that isn't JSON at all. `safeParse` handles JSON in the wrong shape, like something an older version of your app saved. Either way, the app starts

with sensible settings instead of a white screen. This is `safeParse` 's natural home: bad saved data is expected, not an emergency. (JavaScript chapter 23 has more on `localStorage`.)

When you don't need it

- **Data that's already inside your app.** Props, state and function arguments are checked by TypeScript. Don't check them again.
- **Data you wrote yourself**, like a list of books in a `.ts` file. TypeScript can already see it.
- **Tiny one-off checks.** `if (query.trim() === "")` doesn't need a library.

Reach for Zod at the boundaries, and wherever the rules are big enough that writing the checks by hand would be long and easy to get wrong. (The type guards you wrote by hand in TypeScript chapter 13 are exactly what Zod writes for you.)

Common mistakes

1. Writing the type and the schema separately

```
type Book = { id: string; title: string; pages: number }; // ✗ a second copy
const BookSchema = z.object({ id: z.string(), title: z.string(), pages: z.number(), status:
  z.string() });
```

They've already drifted apart: the type has no `status`. Delete the hand-written type and use `z.infer<typeof BookSchema>`.

2. Using `parse` where bad data is normal

`parse` on a form's values, or on `localStorage`, throws on everyday bad input. In an event handler, that's an uncaught error. During render, it's a white screen. When bad data is expected, use `safeParse` and handle both cases.

3. Validating deep inside components

```
function BookCard({ book }: { book: Book }) {
  const checked = BookSchema.parse(book); // ✗ already checked in getBooks
}
```

Check once, at the boundary (the API function). Everything after that gets data that's already been checked, and TypeScript knows its type.

4. Forgetting `typeof` in `z.infer`

```
type Book = z.infer<BookSchema>;  
// ✗ 'BookSchema' refers to a value, but is being used as a type here. Did you mean 'typeof  
BookSchema'?
```

A schema is a value, not a type. `z.infer<typeof BookSchema>` is always the shape.

5. Being surprised by `z.coerce`

`z.coerce.number()` turns `""` into `0`, and `z.coerce.boolean()` turns `"false"` into `true`. Only coerce text you really need to convert, and add a rule that catches the surprise values.

6. Expecting unknown keys to survive

A key that isn't in the schema is stripped from the result. If `rating` "disappears" after parsing, add it to the schema (or use `z.looseObject`).

7. Checking your own internal data

Running Zod on props, state, or data your own code just built adds work and catches nothing. TypeScript already checks it. Zod is for data from outside.

8. Copying Zod 3 code from the internet

`z.string().email()`, `error.flatten()` and `{ message: "..."}` still mostly work, but they're deprecated. If VS Code crosses something out, look up the Zod 4 name (the table at the top of this chapter covers the common ones).

Quick recap

- TypeScript types are erased when your code runs, so `as Task[]`, `data: MealDbResponse` and `api.get<Book[]>` are promises, not proofs. Zod checks for real.
- A **schema** describes the shape. `parse` returns the data or throws. `safeParse` returns `{ success, data }` or `{ success, error }`, and TypeScript narrows it.
- `z.infer<typeof Schema>` makes the schema your **single source of truth**: one place for both the type and the rules.
- Add rules with `.trim()`, `.min()`, `.max()`, `z.email()`, `.int()`, `.positive()`, and your own messages. Read failures with `error.issues`, `z.prettifyError`, or `z.flattenError(error).fieldErrors` for forms.
- `.transform()` checks and reshapes in one step. `.refine()` handles rules across fields, with `path` saying where the error belongs.
- `z.coerce` converts text first, including its surprises (`""` becomes `0`). Unknown keys are stripped by default.
- Validate at the **boundaries**: API responses (inside the API functions), `localStorage`, forms, and URL params. When `getBooks` finds bad data, TanStack Query shows its error state, early and clearly.

Next: try the [exercises](#), then move on to [33 React Hook Form](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch32/ex1/`, and so on.
- Install Zod if you haven't yet: `npm install zod`.
- Exercise 4 uses the practice API. Keep it running with `npm run api` in a second terminal, next to `npm run dev`. Exercise 3 uses TheMealDB, so you need to be online.
- Keep the Console open. Zod's messages are the whole point of this chapter, so read them properly.
- An exercise is done when it works in the browser, VS Code shows no red squiggles, `npm run tsc -b` prints nothing, and the Console has no unexpected errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Check the to-do list

Chapter 10's to-do app saved tasks shaped like `{ id: string; text: string; done: boolean }`, and loaded them back with `as Task[]`. Write the schema that checks them for real.

1. In `src/ch32/ex1/task.ts`, write `TaskSchema`: `id` is a string, `text` is a trimmed string with at least 1 character (message: A task needs some text), and `done` is a boolean.
2. Export the type with `z.infer`. Hover over it in VS Code and check it matches chapter 10's `Task`.
3. Also export `TaskListSchema`: an array of tasks.
4. In `Ex1.tsx`, make an array of test values and run each through `TaskListSchema.safeParse` when a **Check** button is clicked. For each one, log `ok` and the data, or each issue as `path → message`. The test values:
 - a good list of two tasks
 - a list where one task has `done: "yes"`
 - a list where one task has `text: " "`
 - a list where one task has no `id`
 - a list where one task has an extra `priority: "high"`
5. **Before** you click, write down in a comment what you expect for each one. Then click and compare. Which one surprised you?
6. Add a second button that calls `TaskListSchema.parse` (not `safeParse`) on the `done: "yes"` list. What does the Console say? Then wrap it in `try / catch` and log `z.prettifyError(error)` instead.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 2 (Easy): A `useLocalStorage` that can't be fooled

Your `useLocalStorage` hook from [chapter 20](#) does `JSON.parse(saved)` as `T`. That's another promise. Make it check.

1. Copy the hook into `src/ch32/ex2/useLocalStorage.ts`.
2. Give it a schema as a second argument, so it's called like `useLocalStorage("ch32-tasks", TaskListSchema, [])`. Reuse the schema from Exercise 1.
3. Inside, replace the `as T` with `safeParse`. If the saved value fails, use `initialValue`, and log a warning with `z.prettifyError` so you can see why.
4. In `Ex2.tsx`, build a tiny task list that uses it: add a task, tick a task. Refresh, and everything should still be there.
5. Hover over the `tasks` variable. It should be `Task[]`, worked out from the schema, with no `<Task[]>` written anywhere.
6. Now break the saved data. Open DevTools → **Application** → **Local Storage**, and edit the value to each of these in turn, refreshing after each one:
 - `not json at all`
 - `{"hello": 1}`
 - `[{"id": 1, "text": "Buy milk", "done": false}]`
 - `[]`

The first three must start with an empty list and a warning in the Console, never a white screen. The last one is fine: it's a valid, empty list.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): TheMealDB, checked and reshaped

In [chapter 31's exercise 4](#), you added `getCategories()` and `getMealsByCategory()` to `src/api/meals.ts`, each with a hand-written type. Swap those promises for proofs, then add a new function.

1. `getMealsByCategory`: `filter.php` replies in the same shape as `search.php`. Check its reply with the notes' `MealListSchema`. Does it need any changes? (Try it and see.)
2. `getCategories`: `list.php?c=list` replies with `{ meals: [{ strCategory: "Beef" }, ...] }`. Yes, the key is called `meals` even though they're categories. Write a schema with a `.transform` so the function returns a plain `string[]` of names, like `["Beef", "Breakfast", ...]`.

3. **A new one, getCategoryCards**: `categories.php` replies with `{ categories: [...] }`, where each item has `idCategory`, `strCategory`, `strCategoryThumb` and `strCategoryDescription`. Transform each into your own `Category` type: `{ id, name, thumbnail, description }`, and export the type with `z.infer`.
4. In `Ex3.tsx`, show the category cards (picture and name) with `useQuery` and the key `["categoryCards"]`. Clicking a card shows that category's meals below, using `getMealsByCategory`.
5. No `strSomething` names anywhere outside `meals.ts`. Search your `src/ch32/ex3/` folder to check.
6. **Break it on purpose**. In your category schema, rename `strCategory` to `strCategoryName`. Reload. Write down what the page shows, roughly how long it took to show it, and the exact message `z.prettifyError` gives. Then undo it.
7. **Break it another way**. Change `idCategory: z.string()` to `z.number()`. What's the message now? This time the server is right and *your* schema is wrong. Zod catches wrong assumptions on your side too.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The reading list can't lie any more

Apply the notes to your real reading-list code, then try to fool it.

1. Replace the hand-written types in `src/api/books.ts` with the schemas from the notes. Keep the exported type names (`Book`, `NewBook`, `BookStatus`), so nothing else has to change. Run `npm run tsc`. It should print nothing, which proves every other file still compiles.
2. Make `getBooks`, `getBook`, `addBook` and `updateBook` all parse the server's reply. Use `<unknown>` instead of `<Book[]>` or `<Book>` on every request.
3. Your chapter 31 reading list should work exactly as before. Check the Network tab: the same requests, the same results.
4. Now the experiments. For each one, edit `db.json` and save, reload the page, and write down what the page shows and what the Console says. Put the book back the way it was after each one.
 1. One book has `"pages": "lots"`.
 2. One book has no `"title"` at all.
 3. One book has `"status": "abandoned"`.
 4. One book has an extra `"rating": 5`. Does anything break? Where did `rating` go?
5. Change the list's error display, so users see a friendly sentence and the Console gets the `z.prettifyError` details.
6. Add the `retry` function from the notes to `booksQuery`. Repeat experiment 1 and time how long the error takes to appear, before and after.
7. Answer in a comment: before this chapter, what would experiment 1 have done to your page? Where would the bug have shown up, and how would you have found the cause?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A sign-up form, checked by Zod

Combine [chapter 09](#)'s controlled inputs with a Zod schema. Keep it in plain `useState`: no form library yet. That's the point.

The form has **Name**, **Email**, **Password**, **Confirm password**, and an **I agree to the terms** checkbox.

1. In `src/ch32/ex5/signUpSchema.ts`, write `SignUpSchema`:

- `name`: trimmed, at least 2 characters
- `email`: a valid email address
- `password`: at least 8 characters, **and** at least one digit (look up `.regex()` in the Zod docs)
- `confirm password`: must match the password, with the error on the confirm field
- `agree`: must be `true` (look up `z.literal`)

Give every rule a friendly message, and export `type SignUp = z.infer<typeof SignUpSchema>`.

2. Build the form in `Ex5.tsx`, chapter 09 style: controlled inputs, values in `useState`, and a real `<label>` for every field.
3. On submit, `safeParse` the values. If it fails, show the first message for each field under that field, using `z.flattenError`. If it passes, log `result.data` and show `Welcome, Maya!` in place of the form.
4. No red before the first submit. After a failed submit, the errors update as the person types, so each one disappears the moment it's fixed.
5. Link each error to its input: give the error `<p>` an `id`, point the input's `aria-describedby` at it, and set `aria-invalid` on the input while it has an error. ([Chapter 38](#) explains why. For now, just wire it up.)
6. Test it by making every mistake at once. Each field should show its own error, including `Passwords don't match`.
7. Finally, count the lines in `Ex5.tsx` (not the schema), and write the number in a comment. Also count your `useState` calls and your `onChange` handlers. You'll rebuild this exact form in [chapter 33](#) and compare.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

33 React Hook Form

What is it?

React Hook Form is a library that runs your forms for you: it keeps track of the values, checks them, collects the errors, and handles submitting. You write much less code than in chapter 09, and your form re-renders far less often.

```
const { register, handleSubmit } = useForm<NewBook>();

<form onSubmit={handleSubmit(saveBook)}>
  <input {...register("title")} />
  <button type="submit">Add book</button>
</form>
```

This chapter uses **React Hook Form 7**, with `@hookform/resolvers` 5 to connect it to Zod. You'll often see React Hook Form shortened to **RHF**. Install both inside `playground`:

```
npm install react-hook-form @hookform/resolvers
```

Why does it matter?

[Chapter 09](#) taught you to build forms by hand, and that's worth knowing. But look at what three fields with rules needed:

```
// Chapter 09 style: three fields, and it's already long
const [title, setTitle] = useState("");
const [author, setAuthor] = useState("");
const [pages, setPages] = useState("");
const [touched, setTouched] = useState(new Set<string>());
const [hasSubmitted, setHasSubmitted] = useState(false);
const [isSubmitting, setIsSubmitting] = useState(false);

const titleError = title.trim() === "" ? "Please enter a title" : null;
const authorError = author.trim() === "" ? "Please enter an author" : null;
const pagesError = Number.isInteger(Number(pages)) && Number(pages) > 0 ? null : "Pages must be a whole number";

// ...plus three inputs, each with value, onChange, onBlur, and an error message
```

Chapter 09 admitted three problems with this, and promised a fix:

- **It doesn't scale.** Every new field adds a `useState`, an `onChange`, a "touched" check, and an error. A 20-field form becomes hundreds of lines of plumbing.
- **Every keystroke re-renders the whole form**, because each letter goes into state. That's fine for three fields. With 30 fields and a live preview, you'll see it in the Profiler ([chapter 27](#)).
- **TypeScript can't check your field names.** With one object and `name="emia1"`, a typo just silently doesn't work.

There's a fourth problem too. Those rules (`title.trim() === ""`) are a second copy of the rules you already wrote in [chapter 32](#)'s `NewBookSchema`. Two copies drift apart.

React Hook Form fixes all four. And the chapter 09 promise gets kept: with a typed `useForm<NewBook>()`, a misspelled field name is a type error, right in VS Code.

Real-world example

Think about filling in a form at a **clinic's reception desk**.

At the reception desk	React Hook Form
The receptionist doesn't watch you write each letter	Inputs keep their own values; typing doesn't re-render your component
They check the whole form when you hand it in	Validation runs on submit (the default)
They point at the exact boxes that need fixing	<code>errors.title.message</code> , shown next to that field, and focus moves to the first problem
They check against the clinic's official checklist	Your Zod schema, plugged in with a resolver
Once you're fixing a box, they tell you the moment it's right	After a failed submit, errors update as you type
"Sorry, our system is down, please try again"	A server error for the whole form, with <code>setError("root.serverError", ...)</code>

A receptionist who commented on every letter as you wrote it would be exhausting. One who checks at the end and points at exactly what to fix is helpful.

How it works

Your first form: `useForm`, `register`, `handleSubmit`

```
// src/ch33/FirstForm.tsx
import { useForm } from "react-hook-form";

type BookForm = {
  title: string;
  author: string;
};

function FirstForm() {
  const { register, handleSubmit } = useForm<BookForm>();

  function onSubmit(values: BookForm) {
    console.log(values); // { title: "Dune", author: "Frank Herbert" }
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <label htmlFor="title">Title</label>
      <input id="title" {...register("title")} />

      <label htmlFor="author">Author</label>
      <input id="author" {...register("author")} />

      <button type="submit">Add book</button>
    </form>
  );
}

export default FirstForm;
```

Three new pieces:

- `useForm<BookForm>()` creates the form. The type says which fields exist.
- `register("title")` connects one input to the form, under the name `title`.
- `handleSubmit(onSubmit)` wraps your function. On submit, it calls `preventDefault` for you, collects all the values, checks them, and calls `onSubmit` **only if everything is valid**.

No `useState`, no `onChange`, no `event.preventDefault()`. Type something, press Enter, and the Console shows both values in one object.

What `{...register("title")}` really gives the input

`register` returns an ordinary object. Log it to see:

```
console.log(register("title"));
// { name: "title", onChange: f, onBlur: f, ref: f }
```

The `{...}` spread puts all four onto the `<input>` at once:

- `name` becomes the input's `name` attribute.
- `onChange` tells RHF "this field changed". It does **not** put the value into React state.
- `onBlur` tells RHF "the person left this field", so it knows the field was touched.
- `ref` hands RHF the real DOM input, like `useRef` in [chapter 19](#).

That `ref` is the trick. These inputs are **uncontrolled**, as in [chapter 09](#)'s last section: the browser keeps what's typed, and RHF reads it straight from the element when it needs it. Nothing goes through `useState`, so **typing doesn't re-render your component**.

See it for yourself. Put `console.log("render")` at the top of `FirstForm` and type a whole title. Nothing is logged. Do the same in a chapter 09 form and you'll get a log for every letter. (RHF does re-render when something you show changes, like an error appearing or disappearing.)

Showing errors

`formState.errors` holds an entry for each field that failed, with its message. RHF has simple built-in rules you can pass to `register`:

```
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<BookForm>();

<input id="title" {...register("title", { required: "Please enter a title" })} />
{errors.title && <p className="field-error">{errors.title.message}</p>}
```

Other built-in rules include `minLength`, `maxLength`, `min`, `max`, and `pattern`, like `minLength: { value: 2, message: "At least 2 letters" }`. They're fine for a small form. But now your rules are scattered through the JSX, and chapter 32 already wrote them all down once, in a schema. You'll plug that in shortly.

By default, nothing is checked until the first submit. After that, each error disappears the moment you fix it.

Field names are checked now

Here's chapter 09's promise, kept. Give `useForm` the chapter 32 `NewBook` type and misspell a field:

```
const { register } = useForm<NewBook>();

<input {...register("emial")} />
// ✖ Argument of type '"emial"' is not assignable to parameter of type '"status" | "title" | "author" | "pages"'.

```

(The names may be listed in a different order for you.) `errors.emial` is flagged too: `Property 'emial' does not exist on type 'FieldErrors<...>'`. The field names come from the type, so a typo can't slip through.

The real way: one schema, plugged in with `zodResolver`

A **resolver** is a small adapter that lets RHF use a validation library for its checking. `zodResolver` plugs in a Zod schema:

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { NewBookSchema } from "../api/books.ts";
import type { NewBook } from "../api/books.ts";

const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<NewBook>({
  resolver: zodResolver(NewBookSchema),
});
```

On submit, RHF hands every value to `NewBookSchema`. Zod's issues come back as `errors`, one per field, with the messages you wrote in chapter 32. So `errors.title?.message` is `"Please enter a title"`, and `errors.pages?.message` might be `"Pages must be a whole number"`.

This is the big win. **One schema is now the type, the rules, and the messages**, shared by the API check in `books.ts` and the form. Change a rule once, and both follow.

A few things change with a resolver:

- **Drop the built-in rules.** With a resolver, RHF ignores `required`, `minLength` and the rest. Keep all the rules in the schema.
- `onSubmit` **gets the checked data.** It's Zod's output, so the title has already been trimmed.

- **The `<NewBook>` is optional.** The resolver tells `useForm` the types anyway. It's fine to keep it for clarity, as long as the schema has no `.default()` or `.transform()`. If it does, see mistake 10.

Starting values: `defaultValues`

```
useForm<NewBook>({
  resolver: zodResolver(NewBookSchema),
  defaultValues: { title: "", author: "", status: "want" },
});
```

`defaultValues` does three jobs. It's what each field starts with, what `reset()` goes back to, and what RHF compares against to know if anything changed. Give every field one.

The exception is `pages`. There's no good "empty" number (`0` would show up as a `0` in the box), so it's left out, and the box starts blank. That's a deliberate choice, not a slip.

Numbers: `valueAsNumber`

Try the form now: type `320` for pages and submit. You'll see `Pages must be a number`. The browser always gives text from an input, even `type="number"`, so Zod got `"320"`. That's chapter 09's "number inputs still give you a string", back again.

Tell RHF to convert it:

```
<input id="pages" type="number" {...register("pages", { valueAsNumber: true })} />
```

Now RHF converts the text to a number **before** checking it. An empty box becomes `NaN` (not a number), and Zod rejects it with `Pages must be a number`, which is exactly right.

Why not use `z.coerce.number()` in the schema instead? Because `NewBookSchema` is shared with the API. Coercing there would let a server reply of `"pages": "320"` slip through as if it were fine, and chapter 32 showed that `coerce` turns an empty box into `0`. Convert the text where it comes from: the input.

A `<select>` for the status

A `<select>` registers just like an input:


```
import { BookStatusSchema } from "../api/books.ts";
import type { BookStatus } from "../api/books.ts";

const statusLabels: Record<BookStatus, string> = {
  want: "Want to read",
  reading: "Reading",
  finished: "Finished",
};

<label htmlFor="status">Status</label>
<select id="status" {...register("status")}>
  {BookStatusSchema.options.map((status) => (
    <option key={status} value={status}>
      {statusLabels[status]}
    </option>
  ))}
</select>
```

`BookStatusSchema.options` is the array `["want", "reading", "finished"]`, straight from the schema. `Record<BookStatus, string>` makes TypeScript insist on a label for every status. And remember chapter 09's hint about a typo in an `<option value>`? TypeScript still can't see that one, but now Zod catches it the moment you submit.

When to show errors: `mode`

`useForm` takes a `mode` option that decides when fields are first checked:

mode	First check	How it feels
<code>"onSubmit"</code> (the default)	When you submit. After that, as you type	Quiet until you try, then helpful
<code>"onTouched"</code>	When you leave a field. After that, as you type	Chapter 09's "touched" pattern, for free
<code>"onBlur"</code>	Every time you leave a field	Updates only when you move on
<code>"onChange"</code>	Every keystroke	Complains while you're still typing, and re-renders the most

For short forms, the default is fine. For longer ones, `"onTouched"` is usually the friendliest: nobody sees red before they've had a go, and a finished field gets checked straight away. Avoid `"onChange"`. Nobody likes a form that tells them their email is wrong after one letter.

Submitting and saving: `isSubmitting` and `useAddBook`

Now connect the form to the server, through the `useAddBook()` hook from [chapter 31](#):

```
import { useAddBook } from "../api/bookQueries.ts";

function AddBookForm() {
  const addBook = useAddBook();

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isSubmitting },
  } = useForm<NewBook>({
    resolver: zodResolver(NewBookSchema),
    defaultValues: { title: "", author: "", status: "want" },
    mode: "onTouched",
  });

  async function onSubmit(values: NewBook) {
    await addBook.mutateAsync(values);
    reset(); // only reached if the save worked
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      { /* ...the fields... */ }
      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? "Saving..." : "Add book"}
      </button>
    </form>
  );
}
```

- `isSubmitting` is `true` from the moment you submit until `onSubmit`'s promise finishes. That's why `onSubmit` is `async` and **awaits** the save. `mutateAsync` returns a promise; `mutate` doesn't, so with `mutate` the button would flick back to "Add book" instantly.
- `useAddBook()` **invalidates** `["books"]`, so your book list updates by itself. Its `onSuccess` returns that promise, so "Saving..." stays until the new book is actually on screen.
- `reset()` **comes after the `await`**. If the save fails, `mutateAsync` throws, `reset()` never runs, and whatever the person typed is still there. Clearing their typing before the save worked would be cruel.
- `noValidate` switches off the browser's own error bubbles (for example, `type="number"` refusing `abc`). Your Zod messages are clearer, and it's confusing to get both.

(RHF's docs also show another way to reset: watch `formState.isSubmitSuccessful` in an effect. Both work. Resetting right after the `await` keeps "save, then clear" together in one place.)

Server errors: `setError("root.serverError", ...)`

The form can be perfect and the save can still fail: the server is down, or it says no. That error doesn't belong to any one field. RHF has a spot for whole-form errors, called `root`:

```
const { setError /* ...and the rest */ } = useForm<NewBook>({ /* ... */ });

async function onSubmit(values: NewBook) {
  try {
    await addBook.mutateAsync(values);
    reset();
  } catch {
    setError("root.serverError", {
      message: "Couldn't save the book. Is the practice API running?",
    });
  }
}
```

```
{errors.root?.serverError && (
  <p role="alert" className="field-error">
    {errors.root.serverError.message}
  </p>
)}
<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? "Saving..." : "Add book"}
</button>
```

Put it just above the button, where the person is looking. RHF clears `root` errors automatically on the next submit. `role="alert"` makes screen readers read it out as soon as it appears ([chapter 38](#) explains more).

The `try / catch` matters. If `onSubmit` throws, `handleSubmit` passes the error on, and you get `Uncaught (in promise)` in the Console with nothing on screen. (For friendlier messages, like "That book already exists", check `axios.isAxiosError(error)` and `error.response?.status`, as in [chapter 30](#).)

Try it: stop `npm run api`, fill in the form, and submit. The message appears, and everything you typed is still there.

Accessible errors

An error message only helps if everyone can find it. Here's one field, done properly:

```
import { useId } from "react";

const titleId = useId();
const titleErrorId = `${titleId}-error`;

<label htmlFor={titleId}>Title</label>
<input
  id={titleId}
  aria-invalid={errors.title ? "true" : "false"}
  aria-describedby={errors.title ? titleErrorId : undefined}
  {...register("title")}
/>
{errors.title && (
  <p id={titleErrorId} className="field-error">
    {errors.title.message}
  </p>
)}
```

- **htmlFor** and **id** connect the label to the input, as in chapter 09. Clicking the label focuses the box, and screen readers announce the label.
- **useId()** makes an id that's unique for each copy of the component. A hard-coded `id="title"` breaks if two forms are on one page (an Add form and an Edit form, say), because ids must be unique.
- **aria-invalid** tells assistive technology this field has a problem. (ARIA attributes are extra labels for screen readers and similar tools.)
- **aria-describedby** links the error text to the input, so a screen reader reads it with the field, something like "Title, invalid entry, Please enter a title". (The exact words depend on the screen reader.)
- **RHF moves focus to the first invalid field** when a submit fails. Keyboard and screen reader users land right on the problem, without hunting for it. It works because `register` gave RHF the input's `ref`.

That's a lot to repeat for every field, so it's a good job for a small component of your own (exercise 5 tries it). [Chapter 38](#) goes much further.

An edit form: load, fill in, save only when changed

Editing means the form starts with a book's current values. But the book has to load first. The simplest safe approach is: **wait for the data, then build the form**.

```

import { useBook, useUpdateBook } from "../api/bookQueries.ts";
import type { Book } from "../api/books.ts";

function EditBookPage({ id }: { id: string }) {
  const { data: book, isPending, isError } = useBook(id);

  if (isPending) return <p>Loading the book...</p>;
  if (isError) return <p>Couldn't load this book.</p>;

  return <EditBookForm key={book.id} book={book} />;
}

function EditBookForm({ book }: { book: Book }) {
  const updateBook = useUpdateBook();

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isDirty, isSubmitting },
  } = useForm<NewBook>({
    resolver: zodResolver(NewBookSchema),
    defaultValues: {
      title: book.title,
      author: book.author,
      pages: book.pages,
      status: book.status,
    },
  });

  async function onSubmit(values: NewBook) {
    await updateBook.mutateAsync({ id: book.id, changes: values });
    reset(values); // what you just saved is the new starting point
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      </* the same fields as the Add form */>
      <button type="submit" disabled={!isDirty || isSubmitting}>
        Save
      </button>
      <button type="button" disabled={!isDirty} onClick={() => reset()}>
        Discard changes
      </button>
    </form>
  );
}

```

`defaultValues` is read **once**, when the form is first created. So the page component deals with loading and errors, and only renders the form once the book exists. The form is born already filled in, with no effect needed.

- `useBook(id)` is chapter 31's hook for the `["books", id]` query. `useUpdateBook()` is the one you wrote in chapter 31's exercise 5. If yours takes different arguments, change the `mutateAsync` call to match.
- `isDirty` is `true` when any value differs from `defaultValues`. So Save stays disabled until something really changed. Change a field back to how it was, and Save disables again.
- `reset(values)` after a save makes the saved values the new defaults, so `isDirty` goes back to `false`. Plain `reset()` throws away unsaved changes.
- `key={book.id}` gives each book its own fresh form. If the page switches to another book, React builds a new form with the new book's values: the same `key` trick as [chapter 26](#)'s error boundaries.
- **Background refetches can't wipe what someone's typing.** When TanStack Query refetches on window focus, `book` changes, but the form already read its defaults.

RHF also has a `values` option that refills the form whenever the data changes. It's handy, but a background refetch would then overwrite half-typed changes, unless you also set `resetOptions: { keepDirtyValues: true }`. That's why this chapter prefers "wait, then build".

Watching values: `watch` and `useWatch`

Sometimes you want to show a value as it's typed, like a live preview:

```
const { register, watch } = useForm<NewBook>({ /* ... */ });

const title = watch("title");
const author = watch("author");

<p>Preview: {title || "Untitled"} by {author || "someone"}</p>
```

`watch` works, but it has a cost: the **whole form component** re-renders on every keystroke in a watched field. That gives back the re-render saving RHF gave you. For a small form, fine. For a big one, move the preview into its own small component with `useWatch`, so only that component re-renders:

```
import { useWatch } from "react-hook-form";
import type { Control } from "react-hook-form";

function TitlePreview({ control }: { control: Control<NewBook> }) {
  const title = useWatch({ control, name: "title" });
  return <p>Preview: {title || "Untitled"}</p>;
}

// in the form: const { control } = useForm<NewBook>(...); and <TitlePreview control={control} />
```

Inputs that aren't real inputs: Controller

`register` needs a real HTML input to attach its `ref` to and read from. Some components don't have one: a star rating made of buttons, a date picker from a library, a fancy dropdown. They're controlled components, with `value` and `onChange` props like chapter 09's. `Controller` is the adapter between the two. Say your review form's schema has `rating: z.number().int().min(1).max(5)`:

```
import { Controller } from "react-hook-form";

<Controller
  name="rating"
  control={control}
  render={({ field }) => (
    <StarPicker value={field.value} onChange={field.onChange} />
  )}
/>
```

`field` has `value`, `onChange`, `onBlur`, `name` and `ref`. You hand over whichever the component understands. Use `register` for normal inputs, and `Controller` only when you have to.

Lists of fields: useFieldArray

Some forms have a list of the same fields: ingredients in a recipe, guests at a booking. `useFieldArray` handles adding and removing rows:

```
import { useFieldArray, useForm } from "react-hook-form";

const RecipeSchema = z.object({
  name: z.string().trim().min(1, "Please enter a name"),
  ingredients: z
    .array(z.object({ name: z.string().trim().min(1, "Please enter an ingredient") }))
    .min(1, "Add at least one ingredient"),
});
type RecipeForm = z.infer<typeof RecipeSchema>;

const { register, control, handleSubmit } = useForm<RecipeForm>({
  resolver: zodResolver(RecipeSchema),
  defaultValues: { name: "", ingredients: [{ name: "" }] },
});
const { fields, append, remove } = useFieldArray({ control, name: "ingredients" });

{fields.map((field, index) => (
  <div key={field.id}>
    <input {...register(`ingredients.${index}.name`)} />
    <button type="button" onClick={() => remove(index)}>Remove</button>
  </div>
))}
<button type="button" onClick={() => append({ name: "" })}>Add ingredient</button>
```

- **Each row is an object**, `{ name: "" }`, not a plain string. `useFieldArray` needs objects.
- `key={field.id}`, not the index. RHF makes a stable id for each row, for exactly the reasons in [chapter 06](#).
- **The names are paths**, like `ingredients.0.name`, and they're type-checked too: `ingredients.${index}.nmae` is an error.
- `type="button"` on Add and Remove, or they'd submit the form (chapter 09's mistake 8).

Testing a form

Forms test well with [chapter 28](#)'s tools, with three things to know:

- **Checking is async**. After `await user.click(submitButton)`, find the error with `await screen.findByText(...)`, not `getByText`.
- `handleSubmit` **calls your function with two arguments**: the values, then the submit event. So if a test passes a `vi.fn()` in as `onSubmit`, `toHaveBeenCalledWith(values)` fails. Check the first argument only, or add `expect.anything()` for the second.
- **Don't let `vi.mock` replace your schemas**. Chapter 31's `vi.mock("../api/books.ts")` turns *everything* in that file into fakes, including `NewBookSchema`, and the form's checking falls over. Keep the real file and fake only what you need:


```
vi.mock("../api/books.ts", async (importOriginal) => {
  const real = await importOriginal<typeof import("../api/books.ts")>();
  return { ...real, addBook: vi.fn() }; // real schemas, fake addBook
});
```

When you don't need it

- **Small forms.** A search box, a login, a single "add task" field: chapter 09's `useState` is short, clear, and fine.
- **A form that drives the page as you type**, like a live filter. There, updating state on every keystroke is the whole point.
- **React 19's form actions in a framework.** In Next.js ([chapter 39](#)), `<form action={...}>` and `useActionState` can send a form to the server with very little code. Plenty of teams still use RHF there for rich checking in the browser.

And whichever you use, remember JavaScript chapter 51: **the browser checks for convenience, the server checks for safety**. Anyone can skip your form and send anything. A real server must check again.

Common mistakes

1. Forgetting the spread

```
<input {register("title")} /> // ❌ '...' expected.
<input ref={register} /> // ❌ an old (version 6) tutorial: a long type error
<input {...register("title")} /> // ✅
```

2. Numbers arriving as text

Without `valueAsNumber: true`, typing `320` gives `Pages must be a number`, because Zod receives `"320"`. Add it to every number input.

3. Mixing `value` and `onChange` with `register`

```
<input {...register("title")} value={title} onChange={(e) => setTitle(e.target.value)} /> // ❌
```

Your `onChange` replaces RHF's, so RHF never hears about the typing. If you need to react to changes, use `register("title", { onChange: ... })`, or `watch`.

4. `onSubmit={handleSubmit}` instead of `onSubmit={handleSubmit(onSubmit)}`

TypeScript rejects it with a long error that includes `Types of parameters 'onValid' and 'event' are incompatible`. `handleSubmit` is a function that *makes* your submit handler. Call it with yours.

5. Not awaiting the save

```
function onSubmit(values: NewBook) {  
  addBook.mutate(values); // ❌ returns nothing to wait for  
}
```

`isSubmitting` flicks off at once, and an impatient double-click can save the book twice. Make `onSubmit` `async` and `await addBook.mutateAsync(values)`.

6. No `defaultValues`

Then `reset()` has nothing to go back to, `isDirty` has nothing to compare with, and `Controller` inputs can warn about switching from uncontrolled to controlled (chapter 09's warning). Give every field a starting value.

7. `watch` everywhere

Each `watch` re-renders the whole form on every keystroke in that field. Watch only what you display, and use `useWatch` in a small child component for big forms.

8. Error text that isn't linked to its field

A red message under a box is invisible to a screen reader unless the input points at it with `aria-describedby`. Add `aria-invalid` too.

9. Resetting before the save worked

```
reset(); // ❌ the typing is gone...  
await addBook.mutateAsync(values); // ...and then the save fails
```

Always `await` the save first, then `reset()`.

10. `useForm<T>` with a schema that has `.default()` or `.transform()`

```
Type 'Resolver<{ ...; status?: ... | undefined; }, any, { ...; status: ...; }>' is not  
assignable to type 'Resolver<...>'.
```

With `.default()`, the data going in and the data coming out have different types (chapter 32), but `useForm<T>` says they're the same. Leave out `<T>` and let the resolver tell `useForm` both types.

Quick recap

- **React Hook Form** runs your forms: values, checking, errors, and submitting. `useForm`, then `{...register("name")}` on each input, then `onSubmit={handleSubmit(onSubmit)}`.

- `register` spreads `name`, `onChange`, `onBlur` and `ref`. Inputs stay **uncontrolled**, so typing doesn't re-render the form.
- With a typed `useForm<NewBook>()`, field names are type-checked: `register("emial")` is a type error.
- `resolver: zodResolver(NewBookSchema)` makes one schema the type, the rules, and the messages, shared with the API. Use `valueAsNumber` for number inputs.
- Give every field `defaultValues`. `mode: "onTouched"` is usually the friendliest time to show errors.
- Make `onSubmit` `async`, `await` the mutation, then `reset()`. `isSubmitting` covers the wait, and `setError("root.serverError", ...)` shows a save that failed.
- Link errors with `aria-invalid` and `aria-describedby`. RHF focuses the first invalid field for you.
- For editing, load first and build the form second. `isDirty` enables Save only when something changed. `watch` has a re-render cost, `Controller` wraps non-native inputs, and `useFieldArray` handles lists.

Next: try the [exercises](#), then move on to [34 Zustand](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch33/ex1/`, and so on.
- Install the libraries if you haven't yet: `npm install react-hook-form @hookform/resolvers`.
- Exercises 3 and 4 use the practice API and your chapter 31 hooks in `src/api/bookQueries.ts`. Keep the API running with `npm run api` in a second terminal, next to `npm run dev`.
- Exercise 5 uses Vitest and React Testing Library from [chapter 28](#). Run `npm test` in another terminal.
- Test every form by **pressing Enter** as well as clicking the button, just like in chapter 09.
- An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Chapter 09's contact form, rebuilt

A contact form for a bookshop: **Name**, **Email**, **Topic** (a select: General, Order question, Complaint) and **Message** (a textarea).

1. In `src/ch33/ex1/ContactOld.tsx`, build it chapter 09 style: one `useState` per field, and errors shown only after a field has been left (`onBlur`). The rules: name is required, email contains an `@`, and the message is 10 to 500 characters, with a live `123 / 500` counter under it.

2. In `ContactNew.tsx`, build the same form with React Hook Form and **built-in rules only** (no Zod yet). Use `mode: "onTouched"` so errors appear at the same moments as the old version.
3. Show each error under its field. Make the live counter with `watch("message")`.
4. On submit, log the values, then `reset()` the form.
5. Count the lines in both files, and write both numbers in a comment at the top of `ContactNew.tsx`.
6. In React DevTools, open the **Components** tab settings (the cog) and turn on **Highlight updates when components render**. Type a sentence into **Name** in each version. Which one flashes on every key? Now type into **Message** in the new version. Why does it flash there? Write both answers in a comment.
7. The old version disabled Send until everything was valid. The new one lets people press it, then shows exactly what to fix and moves focus to the first problem. Which do you prefer, and why? One or two sentences in a comment.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Your sign-up form, with its schema plugged in

In [chapter 32's exercise 5](#), you built a sign-up form with `useState` and Zod, and counted its lines. Rebuild it with React Hook Form, using **the same schema file, unchanged**.

1. In `src/ch33/ex2/Ex2.tsx`, import `SignUpSchema` from your chapter 32 file, and create the form with `resolver: zodResolver(SignUpSchema)`.
2. Register all five fields, including the checkbox. Show each error under its field, linked with `aria-invalid` and `aria-describedby`, as in the notes.
3. On a successful submit, show `Welcome, Maya!` (with their name) in place of the form.
4. **Break it on purpose.** Change one `register("confirmPassword")` to `register("confirmPasword")`. Write down the exact TypeScript error in a comment. Then try `errors.emial` and write that one down too. Put both back.
5. Make every mistake at once and submit. Then fix the fields one by one, and watch each error disappear as you type. Does `Passwords don't match` disappear when you fix the **password** box to match, or only when you type in the confirm box? Write down what you see.
6. Count the lines, the `useState` calls, and the `onChange` handlers. Write them next to your chapter 32 numbers.
7. **Break it another way.** In a copy of the schema, add `newsletter: z.boolean().default(false)`, and use it with `useForm<SignUp>`. Read the error (it's mistake 10 in the notes). Fix it by removing `<SignUp>`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): "Add a book", for real

Build the reading list's real "Add a book" form, then break it on purpose to see why each piece is there.

1. In `src/ch33/ex3/AddBookForm.tsx`, build the form with four fields: title, author, pages, and status. Use `zodResolver(NewBookSchema)`, `defaultValues`, `valueAsNumber` for pages, and a `<select>` built from `BookStatusSchema.options`.
2. Every field has a label connected with `useId`, plus `aria-invalid` and `aria-describedby` on its error.
3. Submit through `useAddBook()` from `src/api/bookQueries.ts`. While saving, the button says `Saving...` and is disabled. After a successful save, `reset()`.
4. Put your chapter 31 book list (using `useBooks()`) under the form. A new book must appear in it without a reload.
5. After a save, show `Added Dune` to your list. above the form (with the real title). It should disappear as soon as the person starts typing the next title.
6. **Server errors.** Stop `npm run api` and submit a valid book. A message appears above the button, and everything typed is still there. Start the API again and submit: the message goes and the book saves.
7. **Break it on purpose**, one at a time. Write down what happens each time, then undo it:
 1. Remove `valueAsNumber`.
 2. Change `await addBook.mutateAsync(values)` to `addBook.mutate(values)`. Turn on **Slow 3G** throttling in the Network tab and watch the button.
 3. Move `reset()` above the `await`, then submit with the API stopped.
8. Type `Dune` (with spaces) as the title and save. Look at `db.json`. What was saved, and why?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): "Edit book", filled in when the data arrives

1. In `src/ch33/ex4/Ex4.tsx`, show the books with `useBooks()`, each with an **Edit** button. Clicking one stores that book's id in state, and renders `<EditBookPage id={editingId} />` below the list.
2. `EditBookPage` loads the book with `useBook(id)`, shows loading and error states, and only then renders `EditBookForm`, with a `key`.
3. Don't copy and paste the four fields from exercise 3. Move them into a `BookFields` component that both forms use.
4. **Save** is enabled only when something changed. **Discard changes** puts the saved values back. After a successful save, show `Saved.`, and Save disables again.
5. Save through your `useUpdateBook()` from chapter 31's exercise 5.
6. Turn on **Slow 3G**. Click Edit on one book, then quickly on another. The form must end up showing the second book, never the first one's values.
7. **An experiment.** Start editing a book's title, but don't save. Now change that same book's **author** by hand in `db.json`, then click into another window and back (chapter 31's `refetch` on window focus). What does

the form show? Now try it with RHF's `values` option instead of the "wait, then build" pattern, and then with `resetOptions: { keepDirtyValues: true }` added. Write down what happens each time. Which would you want in a real app, and why? There's no single right answer.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A recipe form, with a list of ingredients and tests

Combine this chapter with Zod ([chapter 32](#)), refs as props ([chapter 19](#)), and testing ([chapter 28](#)).

1. In `src/ch33/ex5/recipeSchema.ts`, write `RecipeSchema`:
 - `name`: trimmed, required
 - `servings`: a whole number from 1 to 12
 - `category`: one of a `z.enum` of your choice
 - `ingredients`: a list of `{ name, amount }` (both text, like `"Flour"` and `"200g"`). Each name is required, and there must be **at least one** ingredient.
 - `method`: at least 20 characters
2. In `RecipeForm.tsx`, use `useFieldArray` for the ingredients, with **Add ingredient** and **Remove** buttons. Start with one empty row. Allow removing the last row, so the "at least one" rule can really fail.
3. Accessible errors everywhere: every field, every ingredient row, and the whole-list error. Give each ingredient input a label like `Ingredient 1 name` (it can be visually hidden).
4. Make a reusable `TextField` component (label, input, error text, the `aria` wiring, and `useId` inside it) and use it for every text input. `{...register("name")}` must work on it.
5. `RecipeForm` takes a prop `onSubmit: (recipe: Recipe) => void`, so it can be tested without a server.
6. In `RecipeForm.test.tsx`, write these tests:
 1. Submitting the empty form shows `Please enter a name`, and does **not** call `onSubmit`.
 2. Removing the only ingredient and submitting shows your "at least one ingredient" message.
 3. Filling in everything (with two ingredients) and submitting calls `onSubmit` once, with exactly the right values. `servings` must be a number, and the name must be trimmed.
 4. After a failed submit, the first invalid field has focus.
7. **Extra challenge**: add a **Move up** button to each row, using `useFieldArray`'s `move` or `swap`. Check that a row's error moves with it.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

34 Zustand

What is it?

Zustand is a small library for a shared **store**. A store is a box of state that lives outside your components. Any component can read from it or change it, and there's no provider to wrap around your app.

```
// create the store once, in its own file
export const useCartStore = create<CartStore>()((set) => ({ ... }));

// read it in any component, anywhere in the app
const count = useCartStore((state) => state.items.length);
```

(A fun fact: "Zustand" is the German word for "state".)

This chapter uses **Zustand 5**. Install it in your playground:

```
npm install zustand
```

Why does it matter?

You've already built shared state twice by hand: the theme in [chapter 22](#), and the bookstore cart in [chapter 29](#), with `useReducer` and two contexts. It works, and you understand every line. But you probably noticed three things:

- **There's a lot of setup.** The cart needed a reducer, an action union, two contexts, a provider, two guard hooks, and a wrapper in `main.tsx`. That's a lot of code before a single button works.
- **Every reader re-renders on every change.** A component that reads the state context re-renders when *anything* in it changes ([chapter 22](#)). The header's cart count re-renders when you apply a promo code, even though the count didn't change.
- **It only works inside React.** `useContext` is a hook, so only components can call it. An Axios interceptor from [chapter 30](#) can't read your context.

Zustand fixes all three:

- **One function call** makes the store. No provider, no guard hooks.
- **Selectors** let each component pick just the piece it needs. It only re-renders when that piece changes.
- **The store is also a plain object**, so code outside React can read it and change it.

This is Level 4's theme again: a library that solves a problem you've already felt by hand. It's not magic, and it's not always needed. The end of this chapter says when to skip it.

Real-world example

Think about a **family whiteboard** on the kitchen wall.

Kitchen whiteboard	Zustand store
It hangs in the kitchen, not in anyone's bedroom	The store lives outside your components
Anyone can walk up and read it, no need to ask	Any component can read it, with no provider
You only look at your own line: "Sam: dentist, Tuesday"	A selector picks just the piece you care about
You only care when <i>your</i> line changes	A component only re-renders when its piece changes
House rules at the top say how to change it	Actions in the store are the way to change it
Someone in the garden can shout "what's on the board?"	Code outside React can call <code>getState()</code>

Context is more like the house intercom. Every announcement goes to every room with a speaker switched on, even if it's not for them.

How it works

Your first store

A store is made with `create`. Put it in its own file:

```
// src/ch34/counterStore.ts
import { create } from "zustand";

type CounterState = {
  count: number;
  step: number;
  increment: () => void;
  reset: () => void;
};

export const useCounterStore = create<CounterState>((set) => ({
  count: 0,
  step: 1,
  increment: () => set((state) => ({ count: state.count + state.step })),
  reset: () => set({ count: 0 }),
}));
```


- `create` **gives you back a hook**. That's why its name starts with `use`, and the hook rules from [chapter 08](#) apply.
- **The function you pass in returns the starting state**. Zustand hands it `set`, which changes the state later.
- **State and actions live together**. `count` and `step` are data. `increment` and `reset` are **actions**: functions stored in the same object, whose job is to change the data.

Why the extra `()`?

`create<CounterState>()(...)` looks like a typo, but it's two calls in a row, on purpose. When you call a generic function, TypeScript makes you choose: give it *all* the type arguments, or let it work out *all* of them. You can't do half. Here you want half: you say what your state looks like, and TypeScript works out the types added by **middleware** (optional extras that wrap the store, like saving to `localStorage`, later in this chapter).

Two calls solve it. The first takes the type you write. The second takes your function, and TypeScript infers the rest. At runtime the first `()` does nothing; it only exists for TypeScript. Without middleware, forgetting it happens to work, and then it breaks the day you add some (see mistake 6). So always write it.

Reading the store with a selector

```
import { useCounterStore } from "../counterStore.ts";

function CountDisplay() {
  const count = useCounterStore((state) => state.count);
  return <p>Count: {count}</p>;
}

function IncrementButton() {
  const increment = useCounterStore((state) => state.increment);
  return <button onClick={increment}>+1</button>;
}
```

The function you pass to the hook is a **selector**: a small function that picks the piece of state this component needs.

Zustand runs your selector after every change to the store, and compares the result with last time using `Object.is` (the reference check from [chapter 11](#)). **Only if the result is different does the component re-render.**

- `CountDisplay` re-renders when `count` changes, but not when `step` changes.
- `IncrementButton` picks a function that never changes, so store changes never re-render it. That's what the separate dispatch context gave you in [chapter 23](#), for free.

And there's no provider. Put these components anywhere and they work.

Changing the state with `set`

`set` has two forms, like a `useState` setter:

```
reset: () => set({ count: 0 }), // "make it this"
increment: () => set((state) => ({ count: state.count + state.step })), // "based on what it is now"
```

Use the function form when the new value depends on the old one, just like `setCount((prev) => prev + 1)` in [chapter 12](#).

`set` merges. This is the big difference from `useState`. `set({ count: 0 })` changes `count` and keeps everything else: `step` and both actions are still there. A `useState` setter would replace the whole value.

But it only merges the top level. Anything nested, like an array of cart items, still follows every rule from [chapter 11](#):

```
addItem: (item) => set((state) => { state.items.push(item); return state; }), // ❌ same object
addItem: (item) => set((state) => ({ items: [...state.items, item] })), // ✅ a new array
```

The ❌ version hands back the same object, so the `Object.is` check says "nothing changed" and nobody re-renders. The cart has a new book in it, and the page doesn't show it.

The bookstore cart, as a Zustand store

Now let's rebuild [chapter 29](#)'s cart with the same five rules, so you can compare them side by side.

The examples use chapter 29's `Book` type. In the playground, copy `29-project-online-bookstore/starter/src/books.ts` into `src/shop/books.ts`. (Careful: the playground also has a `Book` type in `src/api/books.ts`, for chapter 30's reading list. It's a different shape, so import from the right file.)

```

// src/shop/cartStore.ts
import { create } from "zustand";
import type { Book } from "../books.ts";

export const MAX_QUANTITY = 10;
export const PROMO_CODE = "BOOKS10";

export type CartItem = {
  bookId: string;
  title: string;
  price: number;
  quantity: number;
};

export type CartState = {
  items: CartItem[];
  promoCode: string | null;
};

type CartActions = {
  add: (book: Book) => void;
  remove: (bookId: string) => void;
  changeQuantity: (bookId: string, quantity: number) => void;
  applyPromo: (code: string) => void;
  clear: () => void;
};

export type CartStore = CartState & CartActions;

export const initialCartState: CartState = { items: [], promoCode: null };

export const useCartStore = create<CartStore>()((set, get) => ({
  ...initialCartState,

  add: (book) => {
    const existing = get().items.find((item) => item.bookId === book.id);
    if (existing) {
      // Rule 1: the same book again means one more copy, not a second line
      get().changeQuantity(book.id, existing.quantity + 1);
      return;
    }
    set((state) => ({
      items: [...state.items, { bookId: book.id, title: book.title, price: book.price, quantity:
        1 }],
    }));
  },

  remove: (bookId) =>

```

```

    set((state) => ({ items: state.items.filter((item) => item.bookId !== bookId) })),

changeQuantity: (bookId, quantity) => {
  // Rule 3: 0 or less removes the line
  if (quantity <= 0) {
    get().remove(bookId);
    return;
  }
  // Rule 2: never more than 10 of one title
  set((state) => ({
    items: state.items.map((item) =>
      item.bookId === bookId ? { ...item, quantity: Math.min(quantity, MAX_QUANTITY) } : item
    ),
  }));
},

applyPromo: (code) => {
  // Rule 4: only BOOKS10 works. For anything else, don't call set at all.
  if (code !== PROMO_CODE) return;
  set({ promoCode: code });
},

// Rule 5: clearing empties the items AND the promo code
clear: () => set(initialCartState),
}));

```

A few things to notice:

- **Every rule lives in the store**, just as every rule lived in the reducer in chapter 29. A component calls `add(book)` and knows nothing about the cap of 10.
- **get is new**. Zustand hands your function a second tool, `get()`, which returns the current state. Here it lets `add` look at the cart and reuse `changeQuantity`, so the cap of 10 is written in exactly one place.
- **A wrong promo code never calls set**. The state stays the very same object, and nothing re-renders. That's chapter 29's "return the exact same state object" rule.
- **clear can pass initialCartState to set** because `set` merges. The data goes back to the start, and the actions stay.

Using it is one line per thing a component needs:

```

function AddToCartButton({ book }: { book: Book }) {
  const add = useCartStore((state) => state.add);
  return <button onClick={() => add(book)}>Add to cart</button>;
}

```

Compared with chapter 29, there's no action union, no `switch`, no contexts, no provider and no guard hooks. Be honest about what you gave up, too. The action union was a checked list of "everything that can happen to the

cart"; here the `CartActions` type does that job. And you've lost the log of every action. `devtools`, below, gives it back.

Totals: derived, never stored

The item count, subtotal, discount and total are all **derived**: worked out from the items, never stored ([chapter 08](#)). Write each one as a small selector function next to the store:

```
// src/shop/cartStore.ts (continued)
export const selectItemCount = (state: CartState) =>
  state.items.reduce((sum, item) => sum + item.quantity, 0);

export const selectSubtotal = (state: CartState) =>
  state.items.reduce((sum, item) => sum + item.price * item.quantity, 0);

export const selectDiscount = (state: CartState) =>
  state.promoCode === PROMO_CODE ? selectSubtotal(state) * 0.1 : 0;

export const selectTotal = (state: CartState) => selectSubtotal(state) - selectDiscount(state);
```

Pass one straight to the hook:

```
function CartIndicator() {
  const count = useCartStore(selectItemCount);
  return <span className="cart-indicator">🛒 Cart ({count})</span>;
}
```

Each selector returns a **number**, and numbers are compared by value. So `CartIndicator` only re-renders when the count really changes. Applying a promo code changes the total but not the count, so the header stays still.

Prove it: who re-renders?

Don't take this on trust. Add `console.log("CartIndicator rendered")` to `CartIndicator`, and make a second component that reads something else:

```
function PromoStatus() {
  const promoCode = useCartStore((state) => state.promoCode);
  console.log("PromoStatus rendered");
  return <p>{promoCode ? `Code ${promoCode} applied` : "No code applied"}</p>;
}
```

You do this	What logs
Add a book	<code>CartIndicator</code> rendered only
Apply <code>BOOKS10</code>	<code>PromoStatus</code> rendered only
Type a wrong code	Nothing at all
Press Clear cart	Both, because both values changed

In chapter 29, both components would read `useCart()`, so both would log every time. (In development, `StrictMode` may show each log twice. That's normal, see [chapter 12](#).) For a visual check, record with the React DevTools Profiler from [chapter 27](#). Components that didn't render show up greyed out.

Picking several values at once

This one catches everybody. You want two values, so you return them together:

```
// ❌ a brand-new object every time the selector runs
const { items, promoCode } = useCartStore((state) => ({
  items: state.items,
  promoCode: state.promoCode,
}));
```

In Zustand 5, this crashes the component:

```
Uncaught Error: Maximum update depth exceeded. This can happen when a component repeatedly
calls setState inside componentWillUpdate or componentDidUpdate. React limits the number of
nested updates to prevent infinite loops.
```

You may also see `The result of getSnapshot should be cached to avoid an infinite loop` in the Console just before it.

Here's why. The selector builds a **new object** every time it runs, the same trap as `value={{ theme, setTheme }}` in [chapter 22](#). Zustand compares with `Object.is`, sees a new object, and decides the state changed. So React renders again, the selector makes another new object, and around it goes until React stops it.

Two fixes:

```
// ✅ Fix 1: one call per value. Simple, and often the clearest.
const items = useCartStore((state) => state.items);
const promoCode = useCartStore((state) => state.promoCode);

// ✅ Fix 2: useShallow
import { useShallow } from "zustand/react/shallow";

const { items, promoCode } = useCartStore(
  useShallow((state) => ({ items: state.items, promoCode: state.promoCode })))
);
```

`useShallow` compares the new object with the last one **field by field**, one level deep. ("Shallow" means it doesn't look inside the fields.) If every field is the same, it hands back the old object, so there's no loop. It works for arrays too: `useShallow((state) => [state.items, state.promoCode])`.

The same trap hides in `filter` and `map`, which always return a new array. `useCartStore((state) => state.items.filter(...))` loops too. Select `state.items`, then filter in the component.

Version note: in Zustand 4, this mistake only caused extra re-renders, so older tutorials are full of it. Some also pass `shallow` as a second argument to the hook, which Zustand 5's `create` doesn't support. Use `useShallow` instead.

Saving to `localStorage`: `persist`

Remember [chapter 29's first stretch goal](#)? A lazy initialiser, an effect to save, and care with corrupt data. Zustand has this built in, as **middleware**: a wrapper that adds a feature to your store, like a phone case that adds a stand.

```
// src/shop/cartStore.ts
import { create } from "zustand";
import { persist } from "zustand/middleware";

export const useCartStore = create<CartStore>()(
  persist(
    (set, get) => ({
      // ...exactly the same state and actions as before
    }),
    {
      name: "bookstore-cart", // the localStorage key
      partialize: (state) => ({ items: state.items }), // what to save
    }
  )
);
```

- `name` is the `localStorage` key. Add a book, then open DevTools → **Application** → **Local Storage**. You'll see `bookstore-cart` holding something like `{"state":{"items":[...]},"version":0}`. (`version` is for the day you change the shape of your state. That's beyond this chapter.)
- `partialize` picks what to save. This shop decided a promo code shouldn't survive a refresh, so only `items` is saved. Leave it out and all your data is saved. Actions are never saved: functions can't be turned into JSON, and they come from your code each time anyway.

Refresh, and the cart is still there. The store loads the saved value when it's created.

Seeing every change: `devtools`

The `devtools` middleware connects your store to the **Redux DevTools** browser extension, which lists every change with the state before and after. You'll install it and use it properly in [chapter 35](#).

```
import { devtools, persist } from "zustand/middleware";

export const useCartStore = create<CartStore>()(
  devtools(
    persist(
      (set, get) => ({ /* ...the same store */ }),
      { name: "bookstore-cart", partialize: (state) => ({ items: state.items }) }
    ),
    { name: "Cart" } // the store's name in the DevTools
  )
);
```

The Zustand docs recommend `devtools` on the **outside**, as the last wrapper. Every `set` call now shows up in the extension. Unless you name it, it's called `anonymous` (newer Zustand 5 releases try to guess a name from your code). To choose the name yourself, pass a third argument to `set`. The `undefined` in the middle keeps the normal merging:

```
set((state) => ({ items: state.items.filter((item) => item.bookId !== bookId) }), undefined,
  "cart/remove");
```

Using the store outside React

The hook from `create` is also the store itself, with plain methods on it:


```

useCartStore.getState();           // the current state, right now
useCartStore.getState().clear();   // call an action
useCartStore.setState({ promoCode: null }); // change the state directly (it merges, like set)

const unsubscribe = useCartStore.subscribe((state) => {
  console.log("The cart now has", state.items.length, "lines");
});

```

None of these are hooks, so you can call them anywhere: a plain function, a timer, a test, or an Axios interceptor. Here's the classic example. On a real API with logins, every request needs a token. Say you have a tiny `useAuthStore` holding `token`, `login` and `logout`. (The practice API has no logins, so this just shows the shape.)

```

// src/api/client.ts, from chapter 30, with one addition
import { useAuthStore } from "../stores/authStore.ts";

api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

```

Context can't do this, because only components can call `useContext`. The same goes for a response interceptor that logs the user out when the server says "not allowed", or a TanStack Query `onError` callback ([chapter 31](#)). Both can call `useAuthStore.getState().logout()` from outside any component.

One warning: `getState()` doesn't **subscribe**. A component that shows `useCartStore.getState().items.length` in its JSX won't re-render when the cart changes. In JSX, always use the hook with a selector. Inside an event handler, `getState()` is fine, because it reads the latest value at the moment of the click.

Testing the store

The store is plain functions, so tests can call the actions directly, with no rendering and no providers ([chapter 28](#)):

```
// src/shop/cartStore.test.ts
import { beforeEach } from "vitest";
import { useCartStore, initialCartState, selectItemCount } from "../cartStore.ts";
import { books } from "../books.ts";

const dune = books[0];

beforeEach(() => {
  localStorage.clear(); // persist saved the last test's cart
  useCartStore.setState(initialCartState); // reset the data, keep the actions
});

test("adding the same book twice gives one line with quantity 2", () => {
  const { add } = useCartStore.getState();
  add(dune);
  add(dune);

  const state = useCartStore.getState();
  expect(state.items).toHaveLength(1);
  expect(selectItemCount(state)).toBe(2);
});

test("a wrong promo code changes nothing at all", () => {
  const before = useCartStore.getState();
  before.applyPromo("FREEBOOKS");
  expect(useCartStore.getState()).toBe(before); // the very same object
});
```

Why the `beforeEach`? There's only one store, created when the file is first imported, and it lives for the whole test run. Without a reset, the second test starts with the first test's books still in the cart. Tests that depend on each other's leftovers pass or fail depending on their order, which is a horrible bug to chase.

Component tests work the same way: nothing to add to `renderWithProviders`, but reset the store before each test. (For big projects, the Zustand docs show a way to reset every store automatically.)

Where state belongs

Chapter 29 taught three homes for state. Level 4 adds a fourth:

Kind	Example	Best home
Local UI state	Is this menu open? Half-typed text	<code>useState</code> (or React Hook Form for form fields)
URL state	Filters, search text, which item	The router's search params and path params
Shared client state	Cart, theme, "compact view" setting	Context, or Zustand / Redux Toolkit

Kind	Example	Best home
Server state	The list of books on the server	TanStack Query

Zustand is for **shared client state**: data that belongs to this browser, like the cart. It is **not** a place to keep a copy of server data. Fetching books in an effect and calling `useLibraryStore.setState({ books })` takes you straight back to [chapter 18](#): loading flags, errors, stale data and refetching, all by hand. Server state is "a copy of data that really lives somewhere else, and can go out of date" ([chapter 31](#)). Keep it in TanStack Query, and let Zustand hold your app's own choices.

Which one should I use?

Situation	Reach for
One component needs it, or a parent and a child	<code>useState</code> , lifted up if needed (chapter 13)
A few values many components read, that rarely change (theme, current user, language)	Context (chapter 22)
Shared state that changes often, read by many components (a cart, a music player)	Zustand
You need to read or change it outside React	Zustand

When you don't need it

Context is built into React and costs nothing to install. For a theme or the logged-in user, which change rarely, the extra re-renders don't matter, and Context is perfectly fine. A small app with a cart read in three places is fine with chapter 29's context and reducer, too.

And a store is never the answer for state only one component uses. That's `useState`, and it always will be. Reach for Zustand when shared state changes often, when many components read different parts of it, or when code outside React needs it.

Common mistakes

1. Calling the hook with no selector

`const { items } = useCartStore();` subscribes to the **whole** store. It works, but the component re-renders on every change to the store, even ones it doesn't care about. That throws away the main reason to use Zustand. Always pass a selector.

2. A selector that returns a new object or array

`useCartStore((state) => ({ items: state.items, promoCode: state.promoCode })))` crashes with `Maximum update depth exceeded` in Zustand 5. Use one hook call per value, or wrap the selector in `useShallow`.

3. Mutating inside `set`

`state.items.push(item)` inside `set` changes the old array, and nobody re-renders. `set` merges the top level, but nested arrays and objects still need copying ([chapter 11](#)).

4. Storing derived totals

A `total` field in the store means every action must remember to update it, and the first one that forgets makes it wrong. Work it out with a selector like `selectTotal`, every time.

5. Putting server data in the store

Copying fetched data into Zustand means rebuilding loading, errors, caching and refetching by hand. Server data belongs to TanStack Query ([chapter 31](#)).

6. Forgetting the `()` in `create<T>()(...)`

`create<ThemeState>((set) => ...)` happens to work without middleware, which is why it's so confusing when you add `persist` later. TypeScript then shows a long error that starts like this:

```
Argument of type 'StateCreator<ThemeState, [], [{"zustand/persist", ThemeState}]]>' is not assignable to parameter of type 'StateCreator<ThemeState, [], []>'.
```

If you see `StateCreator` and a middleware name in an error, look for the missing `()` first.

7. One giant store, or a store for one component

One `useAppStore` holding the cart, the theme, the user and a form's half-typed text is hard to read and hard to test. Make one small store per job (`useCartStore`, `useThemeStore`). And if only one component uses a value, it's `useState`, not a store.

8. Using `getState()` in JSX

`<span>{useCartStore.getState().items.length}</span>` reads once and never updates, because `getState()` doesn't subscribe. Use the hook in JSX. Save `getState()` for event handlers, tests, and code outside React.

Quick recap

- A Zustand **store** is shared state that lives outside your components. `create<T>()(...)` makes it and gives you back a hook. No provider needed.

- Keep **actions inside the store**, next to the data, so every rule lives in one place, just like a reducer.
 - Always read with a **selector**. A component only re-renders when the piece it picked changes.
 - A selector that returns a new object or array causes an infinite loop in Zustand 5. Use one call per value, or `useShallow`.
 - `set` **merges** the top level, but nested arrays and objects still need copying ([chapter 11](#)).
 - **Derive** totals with selector functions. Never store them.
 - `persist` saves to `localStorage`, `devtools` shows every change, and `getState()` / `setState()` work outside React.
 - Zustand is for **shared client state**. Server data belongs to TanStack Query, and one-component state belongs to `useState`.
-

Next: try the [exercises](#), then move on to [35 Redux Toolkit](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch34/ex1/`, and so on. Put each store in its own file, like `src/ch34/ex1/themeStore.ts`.
 - Install Zustand first if you haven't: `npm install zustand`.
 - Exercises 3 and 4 use chapter 29's books. Copy `29-project-online-bookstore/starter/src/books.ts` into `src/shop/books.ts` if you didn't while reading the notes.
 - Keep the Console open. Several exercises ask you to watch which components log a render.
 - An exercise is done when it works in the browser, VS Code shows no red squiggles, `npm run tsc -b` prints nothing, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): A theme that remembers

You built a theme with context in [chapter 22](#). Now build it with a store, and make it survive a refresh.

1. In `themeStore.ts`, make `useThemeStore` with `theme: "light" | "dark"` and a `toggle` action. Use the `create<T>()(...)` form.
2. A `Header` component with a button that says **Switch to dark** or **Switch to light**, depending on the current theme.

3. A `Page` component that puts the theme on its wrapper as a class (`className={theme}`) and shows `Current theme: dark` . Add a little CSS so the two themes look different.
4. `Header` and `Page` are siblings inside `Ex1` . No props, no provider.
5. Wrap the store in `persist` , with the key `ch34-theme` . Toggle, refresh, and check the theme stayed. Find the saved value in DevTools → **Application** → **Local Storage**.
6. **Break it on purpose:** delete the first `()` from `create<ThemeState>()(...)` . Write down the first line of the error. Put it back.
7. **Break it again:** in DevTools, change the saved value to `not json` and refresh. Does the app crash? What theme do you get? Write down what happened.

Finally, in a comment, compare line counts with your chapter 22 `ThemeContext.ts` and `ThemeProvider.tsx` .

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 2 (Easy): Who re-renders?

The whole point of selectors is fewer re-renders. Prove it to yourself.

1. Make `useCounterStore` with `count` , `step` , `increment` (adds `step` to `count`) and `stepChanged(step: number)` .
2. Build three components, each with a `console.log` naming itself:
 - `CountDisplay` shows the count.
 - `StepDisplay` shows the step.
 - `Controls` has a `+` button and a `<select>` for the step (1, 5, 10). It only selects the two actions.
3. Click `+` three times, then change the step. Fill in a table in a comment: for each action, which components logged?
4. **Break it:** change `CountDisplay` to `const { count } = useCounterStore();` (no selector). Repeat step 3. What changed in your table, and why?
5. **Break it differently:** change `CountDisplay` to select `{ count: state.count, step: state.step }` in one object. Write down the exact error you get. Then fix it with `useShallow` .
6. Record the same clicks with the React DevTools Profiler ([chapter 27](#)) and check it agrees with your table.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The bookstore cart, with Zustand

Rebuild [chapter 29](#)'s cart as a store, with exactly the same rules. Work in `src/shop/` in the playground, or in a **copy** of your bookstore app (keep the context version to compare). If you typed the store in while reading the notes, close the notes and write it again from memory first, then compare.

1. `cartStore.ts` with `items`, `promoCode`, and the actions `add`, `remove`, `changeQuantity`, `applyPromo` and `clear`.
2. All five rules live in the store, and nowhere else:
 - adding a book already in the cart adds one to its quantity
 - quantity is capped at 10
 - changing the quantity to 0 or less removes the line
 - only `BOOKS10` is accepted, and a wrong code changes nothing at all
 - clearing empties the items **and** the promo code
3. Selector functions for the item count, subtotal, discount (10% with `BOOKS10`) and total. None of them is stored.
4. A small UI: the first four books with **Add to cart** buttons, a `CartIndicator`, a cart panel with `- / + /` remove on each line, a promo code box, and **Clear cart**. The only prop allowed is the `book` passed to each add button.
5. The promo box shows `Code applied` or `That code isn't valid`.
6. Save the items (but not the promo code) with `persist`.
7. In a comment, compare line counts: chapter 29's `cartReducer.ts` plus your provider file, against `cartStore.ts`. Then write one thing you lost by leaving the reducer behind.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Test every rule

The store is plain functions, so it's as easy to test as chapter 29's reducer. Write `src/shop/cartStore.test.ts` for your store from exercise 3.

1. Reset the store (and `localStorage`) in a `beforeEach`, as in the notes.
2. One test per rule, at least:
 - adding the same book twice gives one line with quantity 2
 - adding a book 11 times stops at 10, and `changeQuantity(id, 15)` also gives 10
 - `changeQuantity` to 0 removes the line, and so does `-1`
 - `BOOKS10` sets the promo code, and a wrong code leaves `getState()` as the **very same object** (`toBe`, not `toEqual`)
 - clearing empties the items and the promo code
3. Tests for the selectors: add two copies of Dune (£9.99), apply `BOOKS10`, and check the subtotal, discount and total.
4. A test proving actions don't mutate: keep a reference to `getState().items`, add a book, and check the old array still has its old length.
5. **Break it on purpose:** comment out the `beforeEach` and run the tests again. Which ones fail? Why those? Put it back.

6. One component test: render your `CartIndicator` and an add button, click the button with `user-event`, and check the count changed ([chapter 28](#)).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Toasts you can call from anywhere

A **toast** is a small message that pops up in a corner ("Book saved", "Couldn't reach the server") and disappears by itself. Any part of the app should be able to show one, even code that isn't a component.

In `src/ch34/ex5/`, build a toast system on top of chapter 30's practice API:

1. `toastStore.ts` with `toasts: Toast[]` (each has an `id`, a `message` and a `kind` of "success" | "error"), plus `show(message, kind)` and `dismiss(id)`.
2. A `Toaster` component that shows every toast in a corner, each with a `×` button. Give the list `role="status"` so screen readers announce new toasts ([chapter 38](#) explains why).
3. **Show one from outside React.** Add an Axios response interceptor to chapter 30's `api` that shows `Couldn't reach the server` when a request fails. Test it by stopping `npm run api` and loading your reading list. (If you did [chapter 31](#), also show a success toast from a mutation's `onSuccess`, like after deleting a book.)
4. Each toast **disappears after 4 seconds**. Pressing `×` removes it straight away. Work out what its timer then does 4 seconds later, and make sure that's harmless.
5. At most 3 toasts on screen. A fourth pushes out the oldest. This is a rule, so it lives in the store.
6. In a comment, answer: which of the four homes for state does a list of toasts belong in, and why not `useState` in `Toaster`?
7. **Stretch:** test the timer with Vitest's fake timers: `vi.useFakeTimers()`, call `show`, then `vi.advanceTimersByTime(4000)`, and check the toast is gone. Call `vi.useRealTimers()` afterwards.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

35 Redux Toolkit

What is it?

Redux is a well-known library for app-wide state. All the shared state lives in one **store**. The only way to change it is to **dispatch** an **action**: a plain object saying what happened. **Reducers** then decide what the new state should be.

If that sounds familiar, it should. It's the `useReducer` idea from [chapter 23](#), scaled up to the whole app.

Redux Toolkit (RTK for short) is the official, modern way to write Redux. It cuts out most of the repetitive code Redux used to need.

```
const dispatch = useAppDispatch();
dispatch(added(book)); // "a book was added to the cart"
```

This chapter uses **Redux Toolkit 2** with **React Redux 9**. Install both in your playground:

```
npm install @reduxjs/toolkit react-redux
```

`@reduxjs/toolkit` holds the store and the tools for writing reducers. `react-redux` connects the store to your components.

Why does it matter?

Here's an honest question. [Chapter 34](#)'s Zustand is smaller and simpler. So why learn Redux too?

- **It's everywhere.** A huge number of existing React apps use Redux, and it shows up in a lot of job listings. There's a good chance you'll work on one.
- **Its strict structure helps big teams.** Every change is an action. Every rule is in a reducer. Every feature is laid out the same way. A new person on the team knows exactly where to look.
- **Its DevTools are brilliant.** [Chapter 23](#) promised that because every change is a plain object, you can log them, replay them and step back through them. Redux is where that promise pays off.
- **You already understand it.** Actions, reducers and dispatch are [chapter 23](#) and the [chapter 29](#) bookstore. Redux Toolkit just removes the boring parts.

And just as honestly: for a new, small app, Redux is often more than you need. Zustand, or plain context, may be plenty. The table near the end of this chapter helps you choose.

Real-world example

Think about **a bank**.

A bank	Redux
You can't walk into the vault and change your balance	Components can't change the store directly
You fill in a slip: "deposit £50"	You dispatch an action: <code>{ type: "account/deposited", payload: 50 }</code>
The teller applies the bank's rules	A reducer works out the new state
The ledger holds the one true balance	The store holds the one true state
Savings and loans each keep their own pages of the ledger	Each slice owns its own part of the state
The audit log records every slip, in order	The Redux DevTools record every action
An auditor can replay the day, slip by slip	"Time travel": step back through the actions

The bank is slower than keeping cash in a jar. But when something goes wrong, you can see exactly what happened and when. That's the trade Redux makes.

How it works

A very short history

Redux came out in 2015. Writing it by hand meant action type constants, action creator functions, big `switch` reducers full of spreads, and several files per feature. People called all that repeated setup code "boilerplate", and complained about it a lot. Redux Toolkit arrived in 2019 as the official fix, and it's now the way the Redux team recommends writing all Redux code.

So if a tutorial shows `createStore`, `const ADD_TODO = "ADD_TODO"`, giant `switch` statements, or `connect(mapStateToProps)`, it's the **old style**. The ideas are the same. The code isn't.

The words you'll meet

Word	What it means	Where you've seen it
Store	The one object holding all the app's shared state	<code>useReducer</code> 's state, for the whole app
Slice	One feature's part of the state, with its reducers and actions (for example, <code>cart</code>)	One reducer file
Action	A plain object saying what happened: <code>{ type: "cart/added", payload: book }</code>	<code>{ type: "added", book }</code> in chapter 29
Payload	The data an action carries, like the contents of a parcel	The extra fields on your actions
Reducer	A function that takes the state and an action, and returns the new state	<code>cartReducer</code>
Dispatch	Sending an action to the store	<code>dispatch</code> from <code>useReducer</code>
Selector	A function that reads a piece of the state	Zustand's selectors (chapter 34)

Your first slice: the cart

A **slice** holds one feature's state and every rule for changing it. Here's chapter 29's cart, with the same five rules. It uses chapter 29's `Book` type from `src/shop/books.ts` , as in [chapter 34](#).

```

// src/shop/cartSlice.ts
import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
import type { Book } from "../books.ts";

export const MAX_QUANTITY = 10;
export const PROMO_CODE = "BOOKS10";

export type CartItem = {
  bookId: string;
  title: string;
  price: number;
  quantity: number;
};

export type CartState = {
  items: CartItem[];
  promoCode: string | null;
};

export const initialState: CartState = { items: [], promoCode: null };

const cartSlice = createSlice({
  name: "cart",
  initialState: initialState,
  reducers: {
    added(state, action: PayloadAction<Book>) {
      const book = action.payload;
      const existing = state.items.find((item) => item.bookId === book.id);
      if (existing) {
        // Rules 1 and 2: one more copy, but never more than 10
        existing.quantity = Math.min(existing.quantity + 1, MAX_QUANTITY);
      } else {
        state.items.push({ bookId: book.id, title: book.title, price: book.price, quantity: 1 });
      }
    },
    removed(state, action: PayloadAction<string>) {
      state.items = state.items.filter((item) => item.bookId !== action.payload);
    },
    quantityChanged(state, action: PayloadAction<{ bookId: string; quantity: number }>) {
      const { bookId, quantity } = action.payload;
      if (quantity <= 0) {
        // Rule 3: 0 or less removes the line
        state.items = state.items.filter((item) => item.bookId !== bookId);
        return;
      }
      const item = state.items.find((i) => i.bookId === bookId);
      if (item) {
        item.quantity = Math.min(quantity, MAX_QUANTITY);
      }
    }
  }
});

```

```

    }
  },
  promoApplied(state, action: PayloadAction<string>) {
    // Rule 4: only BOOKS10. For anything else, change nothing.
    if (action.payload !== PROMO_CODE) return;
    state.promoCode = action.payload;
  },
  cleared() {
    // Rule 5: back to the start, items AND promo code
    return initialCartState;
  },
},
});

export const { added, removed, quantityChanged, promoApplied, cleared } = cartSlice.actions;
export default cartSlice.reducer;

```

Let's take it piece by piece.

- `name: "cart"` becomes the first part of every action type: `cart/added`, `cart/removed`, and so on.
- `initialState` gives the starting state. RTK works out the state's type from it, so give it a type.
- **Each function in `reducers`** handles one action. RTK also makes an **action creator** with the same name: a function that builds the action object for you.
- `PayloadAction<Book>` says what the payload is. `added` carries a `Book`, `removed` carries a `bookId` string.
- **The names are chapter 29's**, in camelCase, because each one becomes a function you call: `quantity_changed` became `quantityChanged`.

Here's what an action creator gives you:

```

added(dune);
// { type: "cart/added", payload: { id: "1", title: "Dune", price: 9.99, ... } }

```

No typing out `{ type: "..."} by hand, and no typos in action names.`

Wait, isn't that mutation?

Look again at `existing.quantity = ...` and `state.items.push(...)`. In [chapter 11](#), [chapter 23](#) and [chapter 34](#), that was a bug. Here it's fine, and the reason is a library built into Redux Toolkit called **Immer**. (Chapter 11 mentioned it by name.)

RTK doesn't hand your reducer the real state. It hands it a **draft**: a stand-in object that quietly records every change you make. When your function finishes, Immer builds a brand-new state from those changes. Anything you changed gets a new object. Anything you didn't touch is reused as-is. That's exactly the copying chapter 11 taught, done for you.

It's like editing a document with "track changes" on. You scribble on the draft, and at the end a clean new version is produced. The original was never touched.

Here's the cap rule both ways:

```
// Chapter 29: copy by hand
return {
  ...state,
  items: state.items.map((item) =>
    item.bookId === action.bookId ? { ...item, quantity: Math.min(action.quantity, 10) } : item
  ),
};

// Redux Toolkit: "change" the draft, and Immer makes the copy
const item = state.items.find((i) => i.bookId === bookId);
if (item) {
  item.quantity = Math.min(quantity, MAX_QUANTITY);
}
```

Three things to keep straight:

- **This only works inside RTK reducers** (the functions in `createSlice`). In components, in `useState`, in `useReducer` and in Zustand, chapter 11's rules still apply. Outside a reducer, the store's state is **frozen**, so "changing" it throws an error (see mistake 2).
- **In a reducer, either change the draft or return a new state. Never both.** `cleared` returns a new state and changes nothing. The others change the draft and return nothing.
- **Changing nothing keeps the same object.** A wrong promo code hits `return;` before touching the draft, so Immer hands back the very same state object. That's chapter 29's rule 4, for free.

The store

The store brings your slices together:

```
// src/shop/store.ts
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "./cartSlice.ts";

export const store = configureStore({
  reducer: {
    cart: cartReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

The key `cart` decides where the slice's state lives: `state.cart.items`. `RootState` is the type of the whole state, worked out by TypeScript from the store itself. `AppDispatch` is the type of its `dispatch`.

`configureStore` also switches on some helpful extras for you: the Redux DevTools connection, and development-only checks that warn you about common mistakes.

Typed hooks

React Redux gives you `useSelector` (read from the store) and `useDispatch` (get `dispatch`). Make typed versions once, and use them everywhere instead of the plain ones:

```
// src/shop/hooks.ts
import { useDispatch, useSelector } from "react-redux";
import type { AppDispatch, RootState } from "../store.ts";

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

`.withTypes` needs React Redux 9.1 or later. In older code you'll see `TypedUseSelectorHook<RootState>` instead, which does the same job.

The provider

Unlike Zustand, Redux does need a provider: one, for the whole store. Wrap your app in `main.tsx`, next to any providers you already have:

```
// src/main.tsx
import { Provider } from "react-redux";
import { store } from "../shop/store.ts";

// ...inside render(), around <App />:
<Provider store={store}>
  <App />
</Provider>
```

Using it in components

```
import { useAppDispatch, useAppSelector } from "../hooks.ts";
import { added } from "../cartSlice.ts";
import { selectItemCount } from "../cartSelectors.ts"; // written in the next section
import type { Book } from "../books.ts";

function AddToCartButton({ book }: { book: Book }) {
  const dispatch = useAppDispatch();
  return <button onClick={() => dispatch(added(book))}>Add to cart</button>;
}

function CartIndicator() {
  const count = useAppSelector(selectItemCount);
  return <span className="cart-indicator">🛒 Cart ({count})</span>;
}
```

Like Zustand, `useAppSelector` only re-renders a component when the value it selected changes. So `CartIndicator` stays still when a promo code is applied. And `dispatch` never changes, so a component that only dispatches never re-renders because of the store.

Selectors, and `createSelector` for totals

Selectors are plain functions that take the whole state:

```
// src/shop/cartSelectors.ts
import { createSelector } from "@reduxjs/toolkit";
import type { RootState } from "../store.ts";
import { PROMO_CODE } from "../cartSlice.ts";

export const selectCartItems = (state: RootState) => state.cart.items;
export const selectPromoCode = (state: RootState) => state.cart.promoCode;

export const selectItemCount = (state: RootState) =>
  state.cart.items.reduce((sum, item) => sum + item.quantity, 0);

export const selectTotals = createSelector(
  [selectCartItems, selectPromoCode],
  (items, promoCode) => {
    const subtotal = items.reduce((sum, item) => sum + item.price * item.quantity, 0);
    const discount = promoCode === PROMO_CODE ? subtotal * 0.1 : 0;
    return { subtotal, discount, total: subtotal - discount };
  }
);
```

`selectItemCount` returns a number, which is compared by value, so it's fine as it is.

`selectTotals` returns an **object**, and a new object every time would mean a re-render after every action. `createSelector` fixes that. It's **memoised**: it remembers its last inputs and its last result, like `useMemo` in [chapter 27](#). The first array lists the input selectors. If `items` and `promoCode` are the same as last time, it skips the maths and hands back the very same object.

```
const { subtotal, discount, total } = useAppSelector(selectTotals);
```

The totals are still derived, never stored ([chapter 08](#)). `createSelector` just avoids working them out, and re-rendering, when nothing changed.

The Redux DevTools

Install the **Redux DevTools** extension for Chrome or Edge from the browser's extension store. Reload your app, open DevTools, and find the **Redux** tab. `configureStore` connects to it automatically.

Add a few books, change a quantity, and apply `BOOKS10`. You'll see:

- **A list of every action**, in order: `cart/added`, `cart/added`, `cart/quantityChanged`, `cart/promoApplied`.
- **Tabs for the selected action**: **Action** shows its payload, **State** shows the whole state afterwards, and **Diff** shows just what changed.
- **Time travel**. **Jump** on an earlier action shows your app exactly as it was at that moment. **Skip** shows what the app would look like if that action had never happened. A slider replays the whole session.

(The buttons move around a little between versions of the extension, but they're all there.)

This is chapter 23's promise kept. It only works because reducers are pure and actions are plain objects: replay the same actions, and you always get the same state.

One honest detail: by default the connection is on in production builds too. Many teams turn it off there by adding `devTools: import.meta.env.DEV` next to `reducer` in `configureStore`.

A second slice: the wishlist

Real apps have several slices. Here's a wishlist, like [chapter 29's stretch goal 4](#):

```
// src/shop/wishlistSlice.ts
import { createSlice, type PayloadAction } from "@reduxjs/toolkit";
import { added } from "../cartSlice.ts";

type WishlistState = { bookIds: string[] };

const initialState: WishlistState = { bookIds: [] };

const wishlistSlice = createSlice({
  name: "wishlist",
  initialState,
  reducers: {
    toggled(state, action: PayloadAction<string>) {
      const id = action.payload;
      if (state.bookIds.includes(id)) {
        state.bookIds = state.bookIds.filter((savedId) => savedId !== id);
      } else {
        state.bookIds.push(id);
      }
    },
  },
  // react to actions that belong to OTHER slices
  extraReducers: (builder) => {
    builder.addCase(added, (state, action) => {
      // once a book is in the cart, it doesn't need to be on the wishlist
      state.bookIds = state.bookIds.filter((id) => id !== action.payload.id);
    });
  },
});

export const { toggled } = wishlistSlice.actions;
export default wishlistSlice.reducer;
```

Add it to the store, and `RootState` updates itself to `{ cart: CartState; wishlist: WishlistState }`:

```
reducer: {
  cart: cartReducer,
  wishlist: wishlistReducer,
},
```

`extraReducers` is how one action changes two slices. When `cart/added` is dispatched, the cart adds the book and the wishlist drops it. That's one user action, one dispatch, and both slices stay in step. In chapter 29 you had to choose between one big reducer or two dispatches. Redux gives you a third way.

(RTK 1 tutorials sometimes write `extraReducers` as an object. RTK 2 removed that; use the `builder` form shown here.)

Async code and server data

RTK has `createAsyncThunk` for async work. You write an async function, and RTK dispatches `pending`, `fulfilled` and `rejected` actions around it, which your slice handles to set loading flags. It works, but look closely: it's [chapter 18](#)'s loading, error and data by hand, just inside Redux.

For **server data**, use a tool built for it. Redux Toolkit includes **RTK Query**, Redux's own answer to TanStack Query ([chapter 31](#)): caching, loading states and refetching, written for you.

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
import type { Book } from "../api/books.ts"; // chapter 30's reading-list Book

export const booksApi = createApi({
  baseQuery: fetchBaseQuery({ baseUrl: "http://localhost:3001" }),
  endpoints: (build) => ({
    getBooks: build.query<Book[], void>({ query: () => "/books" }),
  }),
});

export const { useGetBooksQuery } = booksApi; // const { data, isLoading, error } =
      useGetBooksQuery();
```

It also needs its reducer and middleware added to `configureStore`; the RTK Query docs show the two lines. Watch the names: RTK Query says `isLoading` where TanStack Query v5 says `isLoading`.

Using TanStack Query alongside Redux is also completely normal. Either way, the rule from chapter 34 stands: **don't hand-write loading flags in slices, and don't copy server data into them**. Slices are for client state, like the cart.

Testing

A slice's reducer is a pure function, which makes it the easiest thing in the app to test ([chapter 28](#)). Call it with a state and an action, and check what comes back:

```
// src/shop/cartSlice.test.ts
import cartReducer, { added, promoApplied, initialCartState } from "../cartSlice.ts";
import { books } from "../books.ts";

const dune = books[0];

test("adding the same book twice gives one line with quantity 2", () => {
  let state = cartReducer(initialCartState, added(dune));
  state = cartReducer(state, added(dune));

  expect(state.items).toHaveLength(1);
  expect(state.items[0].quantity).toBe(2);
});

test("a wrong promo code changes nothing at all", () => {
  const state = cartReducer(initialCartState, added(dune));
  expect(cartReducer(state, promoApplied("FREEBOOKS"))).toBe(state);
});
```

No store, no rendering and nothing to reset. Each test builds its own state.

To test selectors, or several slices together, make a fresh store per test. Change `store.ts` to export a function that builds one:

```
export function makeStore() {
  return configureStore({ reducer: { cart: cartReducer, wishlist: wishlistReducer } });
}

export const store = makeStore();
```

A test can then call `makeStore()`, dispatch a few actions, and check `selectTotals(store.getState())`. For component tests, wrap them in `<Provider store={makeStore()}>` inside your `renderWithProviders` helper.

Choosing: context, Zustand or Redux Toolkit?

	useReducer + Context (ch 29)	Zustand (ch 34)	Redux Toolkit
Install	Nothing, it's built in	One small package	Two packages
Setup	Reducer, two contexts, provider, guard hooks	One <code>create</code> call	Slice, store, typed hooks, one provider

	useReducer + Context (ch 29)	Zustand (ch 34)	Redux Toolkit
Changing state	<code>dispatch({ type: "added", book })</code>	<code>add(book)</code>	<code>dispatch(added(book))</code>
Copying rules	By hand (ch 11)	By hand (ch 11)	Immer does it inside reducers
Re-renders	Every reader of the state context	Only when the selected piece changes	Only when the selected piece changes
DevTools	React DevTools only	Redux DevTools, with <code>devtools</code> middleware	Redux DevTools, built in
Outside React	No	<code>getState()</code> , <code>setState()</code>	<code>store.getState()</code> , <code>store.dispatch()</code>
Server data	TanStack Query	TanStack Query	RTK Query, or TanStack Query
Choose it when	A few shared values, small app	Most new apps that need shared client state	Big teams, existing Redux apps, you want strict structure and the DevTools

When you don't need it

If you're starting a small app, Redux Toolkit is usually more than you need. Try the simplest thing first: `useState` , then context, then Zustand. Reach for Redux when the team is large, when the app already uses it, or when seeing every action in the DevTools is worth the extra setup. And even in a Redux app, state that only one component cares about stays in `useState` .

Common mistakes

1. Following an old-style Redux tutorial

`createStore` , action type constants, hand-written `switch` reducers and `connect` still run, but they're not how Redux is written today. (Your editor even shows `createStore` crossed out, because it's marked as deprecated.) Look for `createSlice` and `configureStore` .

2. "Mutating" state outside an RTK reducer

```
const items = useAppSelector(selectCartItems);
items[0].quantity = 5; // ❌ in a component
```

Immer only works inside `createSlice` reducers. Everywhere else, the state is frozen, and Chrome throws:

```
TypeError: Cannot assign to read only property 'quantity' of object '#<Object>'
```

Change state by dispatching an action. And in `useState`, `useReducer` and Zustand, chapter 11's copying rules still apply.

3. Changing the draft and returning a new value

```
cleared(state) {  
  state.promoCode = null;  
  return { items: [], promoCode: null }; // ❌ both at once  
}
```

[Immer] An immer producer returned a new value *and* modified its draft. Either return a new value *or* modify the draft.

Pick one. A sneaky version is a one-line arrow function: `added: (state, action) => state.items.push(...)` returns whatever `push` returns (the new length), so it counts as "returning a value". TypeScript usually flags that one, because a reducer mustn't return a number. Use curly braces.

4. Plain `useSelector` and `useDispatch`

```
const items = useSelector((state) => state.cart.items);  
// ❌ 'state' is of type 'unknown'.
```

Plain `useSelector` doesn't know your state's shape. Use `useAppSelector` and `useAppDispatch` from `hooks.ts`, everywhere.

5. A selector that returns a new object every time

`useAppSelector((state) => ({ items: state.cart.items, promoCode: state.cart.promoCode }))` builds a new object every time. This doesn't crash like it does in Zustand 5, but the component re-renders after every action, and React Redux warns you in development:

```
Selector unknown returned a different result when called with the same parameters. This can lead to unnecessary rerenders.
```

(`unknown` is the selector's name; an inline arrow function doesn't have one.) Select each value separately, or use `createSelector`.

6. Putting values that aren't plain data into the state

Dates, class instances, Maps and functions don't survive being turned into JSON and back, and the DevTools can't show or replay them properly. RTK warns in development:

```
A non-serializable value was detected in an action, in the path: `payload`. Value: ...
```

The path tells you where the bad value is. Store plain data instead. For a date, store the ISO string: `new Date().toISOString()`.

7. Putting everything in Redux

Half-typed form text, whether a menu is open, the hover state of a card: that's local UI state. It belongs in `useState` (or React Hook Form). A store full of those is noisy, slow to work with, and fills the DevTools with junk.

8. Copying server data into a slice

A `books` slice filled by a fetch, with hand-written `isLoading` and `error` fields, is chapter 18 all over again. Use RTK Query or TanStack Query.

Quick recap

- **Redux** keeps shared state in one **store**. Components **dispatch actions**, and **reducers** decide the new state. It's chapter 23's `useReducer` idea, app-wide.
- **Redux Toolkit** is the modern way to write it: `createSlice` for each feature, `configureStore` for the store. Anything with `createStore` and `switch` statements is the old style.
- Inside `createSlice` reducers, **Immer** lets you write code that looks like mutation and produces a proper copy. Only there. Everywhere else, chapter 11's rules still apply.
- Make **typed hooks** once with `.withTypes`, and wrap the app in one `<Provider store={store}>`.
- Use **plain selectors** for single values, and `createSelector` for derived objects like totals, so they're memoised and never stored.
- The **Redux DevTools** show every action, the state after it, what changed, and let you time-travel.
- `extraReducers` lets one action update several slices.
- Server data belongs in **RTK Query** or **TanStack Query**, not in hand-written slices. And for a small new app, Zustand or context may be all you need.

Next: try the [exercises](#), then move on to [36 TanStack Router](#).

Exercises

How to do these:

- Work in your practice app, one folder per exercise: `src/ch35/ex1/`, and so on. Give each exercise its own `slice`, `store` and `hooks.ts` files.
- Install the libraries first if you haven't: `npm install @reduxjs/toolkit react-redux`.
- Install the **Redux DevTools** extension for Chrome or Edge, and keep its **Redux** tab open while you work.
- To keep exercises separate, wrap each `ExN` component's content in its own `<Provider store={store}>`, instead of changing `main.tsx`.
- Exercises 3 to 5 use chapter 29's books in `src/shop/books.ts`, as in [chapter 34](#).
- An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A counter, in slow motion

Learn the parts of Redux on something tiny, and watch every step in the DevTools.

1. `counterSlice.ts` with state `{ count: number; step: number }` and four reducers: `incremented`, `decremented` (never below 0), `stepChanged` (carries the new step) and `reset`.
2. `store.ts` with `configureStore`, plus the `RootState` and `AppDispatch` types. `hooks.ts` with `useAppSelector` and `useAppDispatch`.
3. `Ex1` shows the count and step, with `+`, `-`, a step `<select>` (1, 5, 10) and **Reset**.
4. Click around, then in the Redux tab find: the full list of actions, the payload of a `counter/stepChanged` action, and the **Diff** for a `counter/incremented`.
5. **Time travel:** click **Jump** on an action from a few steps back. What happens to the page? Now try **Skip** on one `counter/incremented`. Write down, in a comment, what each button did.
6. **Break it on purpose:** write `incremented: (state) => state.count++` as a one-liner. What does TypeScript say, and why? Put it back.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): From `useReducer` to a slice

Take the task reducer you wrote in [chapter 23](#) (exercise 5, or the `taskReducer` in the notes) and turn it into a slice. The app's behaviour must stay exactly the same.

1. Copy the old reducer file into `src/ch35/ex2/` for comparison. Then write `tasksSlice.ts` with the same actions, in camelCase: `added`, `toggled`, `deleted`, `clearedCompleted`, `filterChanged`, plus any editing actions you had.

2. Keep every rule, like "deleting the task being edited also ends the edit". Rules still live in the reducers, not in components.
3. Keep making the id **before** dispatching, and pass it in the payload, like in chapter 23. A reducer is still a pure function, even with Immer.
4. Swap the component's `useReducer` for `useAppSelector` and `useAppDispatch`. Nothing else in the component should need to change much.
5. In a comment, compare the two files: which cases got shorter? Did any get longer? What happened to the `never` exhaustiveness check, and why don't you need it any more?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The bookstore cart, the Redux way

Build chapter 29's cart one more time, as a slice, so you can compare all three versions. Write the slice from memory first, without the notes open. Then check it against them.

1. `cartSlice.ts` with `added`, `removed`, `quantityChanged`, `promoApplied` and `cleared`, and the same five rules as [chapter 34's exercise 3](#): add again means +1, a cap of 10, 0 or less removes, only `BOOKS10`, and clearing clears the promo too.
2. `store.ts`, `hooks.ts`, and a `<Provider>` around `Ex3`.
3. The same UI as chapter 34's exercise 3: four books, a `CartIndicator`, a cart panel with `- / + / remove`, a promo box, and **Clear cart**. Copy your components and swap only the hooks.
4. The promo box shows `Code applied` or `That code isn't valid`, without the component knowing what the right code is.
5. In the DevTools, apply a wrong code. Find its `cart/promoApplied` action and look at the **Diff** tab. What does it show, and why?
6. In a comment, fill in a small table comparing chapter 29, 34 and 35: number of files, lines of code, how a component adds a book, and what you can see in the DevTools.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Memoised totals, and a test for every rule

Add `createSelector` totals to your exercise 3 cart, then test the slice properly ([chapter 28](#)).

1. `cartSelectors.ts` with `selectItemCount`, plus `selectTotals` built with `createSelector`, returning `{ subtotal, discount, total }`. Use it on the cart panel.
2. `cartSlice.test.ts`, calling `cartReducer(state, action)` directly. At least one test per rule:
 - adding the same book twice gives one line with quantity 2

- adding a book 11 times stops at 10, and `quantityChanged` to 15 also gives 10
 - `quantityChanged` to 0 or `-1` removes the line
 - `BOOKS10` sets the promo code, and a wrong code returns the **very same** state object (`toBe`)
 - `cleared` empties the items and the promo code
3. Add `makeStore()` to `store.ts`, and write selector tests: dispatch two copies of Dune (£9.99) and `BOOKS10` on a fresh store, then check all three totals.
 4. **Prove the memoising:** call `selectTotals` twice on the same state and check you get the very same object. Then add a book and check you get a different one.
 5. **Break it on purpose:** replace `selectTotals` with a plain arrow function that builds the same object. Run the app and find the warning in the Console. Write down its first sentence, then put `createSelector` back.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Two slices, one action, and a verdict

Add [chapter 29's stretch goal 4](#), the wishlist, to your exercise 3 cart. Then decide which library you'd use for the bookstore.

1. A `wishlist` slice with `toggled(bookId)`. A ♥ button on each book toggles it.
2. A "Saved for later" list with a **Move to cart** button on each book, and a **Save for later** button on each cart line.
3. **Each of those buttons dispatches exactly one action**, and that one action updates **both** slices. Decide which slice owns each action, and where you need `extraReducers`.
4. The notes' wishlist drops a book whenever it's added to the cart, from anywhere. Keep that rule or remove it? Argue for your choice in a comment.
5. The header shows both counts: 🛒 3 ♥ 5.
6. **Save both slices to localStorage** with `store.subscribe`, and load them on startup with `configureStore`'s `preloadedState`. Only write a slice when it actually changed. Corrupt saved data must give an empty cart and wishlist, not a white screen.
7. One test on a fresh `makeStore()`: dispatch **Move to cart** and check both slices changed.
8. **The verdict.** In a comment at the top of `Ex5.tsx`, write a short paragraph: would you build the real bookstore with Zustand or Redux Toolkit, and why? Mention at least setup, testing, the DevTools, and how "move to cart" would work in each. (In Zustand, would the wishlist be its own store, or part of the cart store?)

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

36 TanStack Router

What is it?

TanStack Router is a router, like React Router from [chapter 24](#): it maps URLs to pages. The difference is that it's built around TypeScript. TypeScript knows every route in your app, so a broken link, a missing param or a bad search param gets a red squiggle in VS Code before the code ever runs.

```
<Link to="/books/$bookId" params={{ bookId: "3" }}>Foundation</Link>
```

Misspell `/books`, or forget `params`, and that line won't compile.

This chapter uses **TanStack Router v1** (the package is `@tanstack/react-router`) with **file-based routing** through its Vite plugin, `@tanstack/router-plugin`. File-based routing means each page is a file, and your folders *are* the route table. It's made by the same team as TanStack Query ([chapter 31](#)), and the two are designed to work together.

This library moves fast. Version 1 has had well over a hundred minor releases, and some names have changed along the way. This chapter was checked against 1.170. If something doesn't match what you see, trust the [official docs](#).

Why does it matter?

Chapter 24 left three gaps where TypeScript couldn't help you:

```
const { id } = useParams();           // string | undefined, on every page, every time
<Link to="/boks/3">Dune</Link>        // a typo: compiles fine, 404s in the browser
const page = searchParams.get("page"); // string | null. "2"? "banana"? Check it yourself.
```

All three have the same cause. React Router's declarative mode can't tell TypeScript which routes exist, or which route rendered this component. So `to` accepts any string, every param might be missing, and search params are untyped text.

TanStack Router closes those gaps by generating a map of your routes that TypeScript can read. `<Link to>` only accepts URLs that exist. A page's params are `string`, never `undefined`. And search params are checked by a Zod schema ([chapter 32](#)), so `page` comes out as a real `number`.

To be fair to React Router: it's used far more widely, it's excellent, and its v7 "framework mode" can generate route types too. TanStack Router simply starts from TypeScript instead of adding it later. Neither choice is wrong, and knowing both means you can work in either kind of codebase.

Real-world example

Think about **addressing an envelope** versus **using a sat-nav**.

Paper envelope	Sat-nav
You can write any address, real or not	You can only choose addresses that exist
A typo is found when the letter comes back	A typo is flagged while you type
"Flat 3" or "Flat three"? The postie guesses	The house number has to be a number
You keep your own list of streets	The map updates itself when new roads open
It can't tell you if anyone's home	Neither can the sat-nav: a real address can be an empty house

That last row matters. TanStack Router can prove `/books/3` is a real *kind* of page. It can't prove book 3 exists. You still handle that, and this chapter shows where.

How it works

A new practice app

File-based routing takes over the whole app: the files in `src/routes/` decide what's on screen, and there's no `App.tsx` any more. So this chapter uses a **new** Vite app.

1. In a terminal in the `React` folder, run `npm create vite@latest` and answer as in [chapter 01](#), naming the project `router-practice`.
2. `cd router-practice`, then install the router, its devtools, and the plugin:

```
npm install @tanstack/react-router @tanstack/react-router-devtools
npm install -D @tanstack/router-plugin
```

3. Delete `src/App.tsx`, `src/App.css` and `src/assets/`. Copy `books.ts` and `index.css` from `React/29-project-online-bookstore/starter/src/` into `src/`, so you have the bookstore's 12 books and all its styles.

Then add the plugin to `vite.config.ts`. A **plugin** is an add-on that gives Vite an extra job. This one watches `src/routes/` and writes the route map for you:

```
// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import { tanstackRouter } from "@tanstack/router-plugin/vite";

export default defineConfig({
  plugins: [
    // The router plugin must come BEFORE react()
    tanstackRouter({ target: "react", autoCodeSplitting: true }),
    react(),
  ],
});
```

The docs are firm that the router plugin goes before `react()`. Older tutorials import it as `TanStackRouterVite`: same plugin, old name. (TanStack also has a command that sets all this up for you, `npx @tanstack/cli create --router-only` at the time of writing. Doing it by hand once shows you what each piece is for.)

Routes are files

Make a `src/routes/` folder. Each file in it is a page, and where it sits decides its URL:

File	URL
<code>src/routes/__root.tsx</code>	none: it wraps every page (the layout)
<code>src/routes/index.tsx</code>	<code>/</code>
<code>src/routes/about.tsx</code>	<code>/about</code>
<code>src/routes/books/index.tsx</code>	<code>/books</code>
<code>src/routes/books/\$bookId.tsx</code>	<code>/books/1</code> , <code>/books/42</code> , ...

`index` means "the page for this folder itself". `$` marks a **path param**, a part of the URL that changes. It's chapter 24's `:id` with a different symbol.

Every route file exports a `Route`:

```
// src/routes/about.tsx
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/about")({
  component: AboutPage,
});

function AboutPage() {
  return <h1 className="page-title">About Paper & Ink</h1>;
}
```

The string in `createFileRoute("/about")` must match where the file lives, but you don't have to type it. Create an **empty** file in `src/routes/` while `npm run dev` is running, and the plugin fills in a starter route. Move a file, and it fixes the string. (It writes single quotes and no semicolons; change them if you like.) One surprise: an index route's string ends in a slash, so `books/index.tsx` gets `createFileRoute("/books/")`.

The route map: `routeTree.gen.ts`

While `npm run dev` runs, the plugin writes `src/routeTree.gen.ts` (**gen** is short for generated). It lists every route and their types. Somewhere in the middle is a line like this:

```
to: '/' | '/about' | '/books/$bookId' | '/books'
```

That's a union type (TypeScript chapter 06) of every URL in your app. It's the sat-nav's map. **Never edit this file:** it's rebuilt every time you add, rename or delete a route. Do commit it to Git, though. The docs say it's part of your app's source code.

The root route: a layout for every page

`__root.tsx` plays the part of chapter 24's `Layout` route. It renders on every page, and `<Outlet />` is where the current page goes:

```
// src/routes/__root.tsx
import { createRootRoute, Link, Outlet } from "@tanstack/react-router";
import { TanStackRouterDevtools } from "@tanstack/react-router-devtools";

export const Route = createRootRoute({
  component: RootLayout,
  notFoundComponent: () => <p className="status-message">Page not found.</p>,
});

function RootLayout() {
  return (
    <div className="app">
      <header className="site-header">
        <nav className="main-nav">
          <Link to="/" className="nav-link" activeOptions={{ exact: true }}>Home</Link>
          <Link to="/books" className="nav-link">Books</Link>
          <Link to="/about" className="nav-link">About</Link>
        </nav>
      </header>
      <main className="page">
        <Outlet />
      </main>
      <TanStackRouterDevtools />
    </div>
  );
}
```

`notFoundComponent` replaces chapter 24's `path="*" route`: it shows for any URL that matches no route.

`TanStackRouterDevtools` is a debugging panel, covered at the end.

Starting the router: `main.tsx`

Replace `src/main.tsx` with this:

```
// src/main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { createRouter, RouterProvider } from "@tanstack/react-router";
import { routeTree } from "./routeTree.gen.ts";
import "./index.css";

const router = createRouter({ routeTree });

// Tell TypeScript about YOUR routes
declare module "@tanstack/react-router" {
  interface Register {
    router: typeof router;
  }
}

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <RouterProvider router={router} />
  </StrictMode>,
);
```

`createRouter` builds the router from the generated map, and `<RouterProvider>` takes the place of chapter 24's `<BrowserRouter>` and `<App />`.

The `declare module` block is the strange-looking part. The library's `Link` was written once, for every app in the world, so on its own it can't know *your* routes. This block adds your router's type to an empty `Register` interface inside the library, and TypeScript combines the two. From then on, every `Link`, `useNavigate` and `useSearch` knows your routes. TypeScript chapter 04 called this **declaration merging** and said libraries use it. Here's one doing it, and it's why this must be an `interface`, not a `type`.

Now run `npm run dev` and visit `/`, `/about` and `/nonsense`. The header stays put; only the part inside `<Outlet />` changes.

Links that TypeScript checks

`Link` works like chapter 24's, except for params: you write the route's *pattern*, and pass the params separately.

```
<Link to="/about">About</Link>
<Link to="/books/$bookId" params={{ bookId: "3" }}>Foundation</Link>
```

Now make mistakes on purpose. A typo in `to`:


```
Type '"/bokks"' is not assignable to type '". | ".. | "/" | "/about" | "/books/$bookId" | "/books"'.
Did you mean '"/books"'?
```

Forgetting `params`:

```
Property 'params' is missing in type '{ children: string; to: "/books/$bookId"; }' but required in type ...
```

Even `to="/books/3"` is an error, because `/books/3` isn't on the list. The pattern-plus- `params` style feels wordy at first. It pays off when you rename a route: every link to the old name turns red, and you fix them all before a user finds one.

Active links. Chapter 24 needed a separate `NavLink`. Here, every `Link` knows when it points at the current page, and adds three things to its `<a>` when it does:

Added when active	Useful for
<code>class="active"</code> , joined onto your own <code>className</code>	Chapter 29's CSS: <code>.nav-link.active</code> just works
<code>data-status="active"</code>	Styling with an attribute selector instead
<code>aria-current="page"</code>	Screen readers announce "current page" (chapter 38)

Want a different class? Use `activeProps={{ className: "current" }}`. A link also counts as active on every page *below* it, like `NavLink` without `end`. That's why Home has `activeOptions={{ exact: true }}`; without it, Home is highlighted everywhere.

Path params: a string, never undefined

Here's the book page, using the local data for now:

```
// src/routes/books/$bookId.tsx
import { createFileRoute } from "@tanstack/react-router";
import { books } from "../../books.ts";

export const Route = createFileRoute("/books/$bookId")({
  component: BookPage,
});

function BookPage() {
  const { bookId } = Route.useParams(); // bookId: string
  const book = books.find((b) => b.id === bookId);

  if (!book) {
    return <p className="status-message">We couldn't find that book.</p>;
  }
  return <h1 className="book-detail-title">{book.title}</h1>;
}
```

`Route.useParams()` belongs to *this* route, so it knows this page always has a `bookId`. No `undefined`, no check, no `!`, unlike every page in chapter 24.

But notice what didn't go away: `find` still returns `Book | undefined`. A URL can have the right *shape* and still point at nothing, like `/books/999`. The param is a `string`; that doesn't mean the book exists. Loaders, below, give that case a proper home.

Search params, checked by Zod

In [chapter 29](#) you put the book filters in the URL so any view could be shared. But you read every value by hand, and `?page=banana` would have gone straight through. Here a Zod schema does the checking. Install it with `npm install zod`, then:

```
// src/routes/books/index.tsx
import { createFileRoute, Link, useNavigate } from "@tanstack/react-router";
import { z } from "zod";
import { books, GENRES } from "../../books.ts";

const bookSearchSchema = z.object({
  q: z.string().default("").catch(""),
  genre: z.enum(["all", ...GENRES]).default("all").catch("all"),
  sort: z.enum(["title", "price-asc", "price-desc"]).default("title").catch("title"),
  page: z.number().int().min(1).default(1).catch(1),
});

export const Route = createFileRoute("/books/")({
  validateSearch: bookSearchSchema,
  component: BooksPage,
});
```

`validateSearch` runs the schema on every visit, before the page renders. Each field ends with two helpers:

- `.default(x)` means "if it's missing, use `x`". It also tells the router the param is optional, so `<Link to="/books">` doesn't have to pass any.
- `.catch(x)` means "if it's there but it's rubbish, use `x`" instead of failing.

URL	What <code>Route.useSearch()</code> gives you
<code>/books</code>	<code>{ q: "", genre: "all", sort: "title", page: 1 }</code>
<code>/books?genre=Crime&page=2</code>	<code>genre: "Crime"</code> and <code>page: 2</code> , a real number
<code>/books?page=banana</code>	<code>page: 1</code>
<code>/books?genre=poetry</code>	<code>genre: "all"</code>

`page: 2` arrives as a number with no `Number()` from you, because TanStack Router reads search values like JSON. Leave out each helper to see why it's there. Without `.catch()`, `?page=banana` fails validation and the route's error screen shows instead of the page. Without `.default()`, every `<Link to="/books">` in your app says Property 'search' is missing.

(Version note: Zod 4 schemas go straight in, because Zod 4 follows a shared standard called **Standard Schema** that the router understands. With Zod 3, you'd wrap the schema in `zodValidator()` from `@tanstack/zod-adapter`. You'll see both in tutorials.)

Reading them is one line, and every value is typed:

```
function BooksPage() {
  const { q, genre, page } = Route.useSearch(); // string, "all" | Genre, number

  const matching = books.filter((book) => {
    const matchesQuery = book.title.toLowerCase().includes(q.toLowerCase());
    const matchesGenre = genre === "all" || book.genre === genre;
    return matchesQuery && matchesGenre;
  });
  // ...show 4 of `matching` per page...
}
```

Changing them with a link: `search` can be a function that gets the current values (`prev`) and returns new ones.

```
<Link from={Route.fullPath} search={(prev) => ({ ...prev, page: prev.page + 1 })}>
  Next page
</Link>
<Link to="/books" search={{ genre: "Crime" }}>Crime novels</Link>
```

`from={Route.fullPath}` means "starting from this page", which is how TypeScript knows `prev.page` is a number. (The docs also show `to="."`, but then `prev` is loosely typed and `prev.page` might be `undefined`.)

Changing them from code, like a search box, uses `useNavigate` with the same `from`:

```
const navigate = useNavigate({ from: Route.fullPath });

<input
  className="search-input"
  value={q}
  onChange={(event) =>
    navigate({
      search: (prev) => ({ ...prev, q: event.target.value, page: 1 }),
      replace: true, // no history entry per keystroke (chapter 29's decision)
    })
  }
/>
```

One thing you'll notice in the address bar: visit `/books` and it becomes `/books?`

`q=&genre=all&sort=title&page=1`. The router writes the checked values back into the URL. That's harmless, and the exercises show how to tidy it.

	Chapters 24 and 29: <code>useSearchParams</code>	TanStack Router: <code>validateSearch</code>
Type of <code>page</code>	<code>string null</code>	<code>number</code>
<code>?page=banana</code>	Your code's problem	Becomes <code>1</code>

	Chapters 24 and 29: <code>useSearchParams</code>	TanStack Router: <code>validateSearch</code>
Default values	<code>?? ""</code> every time you read	Once, in the schema
A link to <code>?genre=Poetry</code>	Compiles	Type error

Loaders: fetch before the page shows

Real data lives on a server, and [chapter 18](#) fetched it in an effect. That has a built-in delay: the component renders first with nothing to show, *then* the effect runs, and only *then* does the request start. If a child component fetches too, its request can't even start until the parent's data has arrived. Requests that wait on each other like that are called a **waterfall**.

A **loader** is a function on the route that fetches the page's data. The router knows where you're going the moment you click, so it starts the loader right then, before rendering anything. If several routes on the page have loaders, they run at the same time.

To feel the difference, make a fake slow API in `src/api.ts`:

```
// src/api.ts
import { books, type Book } from "./books.ts";

// Pretend to be a slow server: wait, then answer.
function wait(ms: number): Promise<void> {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

export async function fetchBooks(): Promise<Book[]> {
  await wait(1500);
  return books;
}

// Like a real API, "no such book" is an answer (null), not a crash.
export async function fetchBook(id: string): Promise<Book | null> {
  await wait(1500);
  return books.find((book) => book.id === id) ?? null;
}
```

Now the book page gets a loader, plus a component for each of the other outcomes:

```
// src/routes/books/$bookId.tsx
import { createFileRoute, Link, notFound, useRouter } from "@tanstack/react-router";
import type { ErrorComponentProps } from "@tanstack/react-router";
import { fetchBook } from "../../api.ts";

export const Route = createFileRoute("/books/$bookId")({
  loader: async ({ params }) => {
    const book = await fetchBook(params.bookId);
    if (!book) {
      throw notFound(); // show notFoundComponent instead
    }
    return book;
  },
  component: BookPage,
  pendingComponent: () => <p className="status-message">Loading book...</p>,
  errorComponent: BookError,
  notFoundComponent: () => <Link to="/books">No such book. Back to all books</Link>,
});

function BookPage() {
  const book = Route.useLoaderData(); // Book: not null, not undefined, not loading
  return <h1 className="book-detail-title">{book.title}</h1>;
}

function BookError({ error }: ErrorComponentProps) {
  const router = useRouter();
  return (
    <div role="alert" className="status-message error">
      <p>Couldn't load this book: {error instanceof Error ? error.message : "unknown error"}</p>
      <button onClick={() => router.invalidate()}>Try again</button>
    </div>
  );
}
```

Look at `BookPage`. No `useState`, no `useEffect`, no `isLoading`, no `ignore` flag. By the time it renders the data is there, and `Route.useLoaderData()` knows its type from what the loader returned. Every other outcome has its own component:

What happened	What shows
Still loading after 1 second	<code>pendingComponent</code>
The loader threw an error	<code>errorComponent</code> (<code>router.invalidate()</code> runs the loader again)
The loader threw <code>notFound()</code>	<code>notFoundComponent</code>
It worked	<code>component</code>

That's chapter 18's loading, error and data states, plus chapter 26's error boundary, declared once per route. (In 1.170, `error` is typed `unknown`, hence the `instanceof Error` check. Older versions typed it as `Error`; the check works for both.)

About that "1 second": it's on purpose. While a loader runs, the router keeps the **old page on screen**, the way a normal website does. Only if the wait passes 1 second does `pendingComponent` appear, and then it stays for at least half a second so it never just flickers. (The route options `pendingMs` and `pendingMinMs` change those numbers.) On the very first page load there's no old page to keep, so the screen is blank for that first second. So be honest about what loaders do: they don't make the wait disappear. They start it earlier, and give it a proper place.

Want to start even earlier? Add `defaultPreload: "intent"` to `createRouter`, and the router runs a page's loader as soon as you **hover** over a link to it. By the time you click, the data is often already there.

Loaders and TanStack Query together

The router keeps loader results for a few minutes, but it's a basic cache: nothing is shared between routes, and there are no mutations. You already know a proper one, TanStack Query from [chapter 31](#). The two fit neatly: **the router starts the fetch early, and the query cache keeps it.**

After `npm install @tanstack/react-query`, give the root route a **context**: a typed box of things every loader can reach, a bit like chapter 22's Context but for loaders. In `__root.tsx`:

```
import type { QueryClient } from "@tanstack/react-query";
import { createRootRouteWithContext } from "@tanstack/react-router";

type RouterContext = { queryClient: QueryClient };

// The double call, ()({ ... }), is on purpose
export const Route = createRootRouteWithContext<RouterContext>()({
  component: RootLayout,
});
```

In `main.tsx`, create the client, hand it to the router, and wrap `<RouterProvider>` in chapter 31's `<QueryClientProvider client={queryClient}>`:

```
const queryClient = new QueryClient();

const router = createRouter({
  routeTree,
  context: { queryClient }, // leave this out: "Property 'context' is missing"
  defaultPreload: "intent",
  defaultPreloadStaleTime: 0, // the docs' advice: let TanStack Query decide what's fresh
});
```

Then describe the query once with `queryOptions`, and use it in both the loader and the component:

```
// src/queries.ts
export const booksQueryOptions = queryOptions({
  queryKey: ["books"],
  queryFn: fetchBooks,
});

// src/routes/books/index.tsx
export const Route = createFileRoute("/books/")(
  {
    validateSearch: bookSearchSchema,
    loader: ({ context }) => context.queryClient.ensureQueryData(booksQueryOptions),
    component: BooksPage,
  }
);

function BooksPage() {
  const { data: books } = useSuspenseQuery(booksQueryOptions); // Book[]
  // ...everything else as before...
}
```

`ensureQueryData` means "give me what's in the cache, and only fetch if there's nothing there". So the loader fills the cache before render, and the component reads the same cache. Because the data is sure to be there, `useSuspenseQuery` (the Suspense-ready hook [chapter 26](#) promised) types `data` as `Book[]`, never `undefined`. Plain `useQuery` works too, with `data` typed `Book[] | undefined`. Come back to `/books` later and the list appears at once from the cache, with all of chapter 31's refetching rules still working. [Chapter 37](#) builds a whole app this way.

Code splitting, for free

In [chapter 26](#) you split pages into separate downloads by hand, with `lazy()` and `<Suspense>`. The `autoCodeSplitting: true` option does it for every route file. Run `npm run build` and you'll see a small extra JavaScript file per page. There's one rule: **don't export your page components** from a route file, only `Route`. An exported component gets pulled into the main bundle and isn't split.

Devtools

`<TanStackRouterDevtools />` adds a small floating button to a corner of the page while you develop. Open it to see which routes matched, their params, search params and loader data, and what's cached. When a page shows the wrong thing, look there first. It isn't shown in production builds.

When you don't need it

- **An app with a couple of pages.** A form and a thank-you page don't need a route map. React Router, or no router at all, is fine.
- **A team already happy on React Router.** Switching routers touches every page. That costs more than a few typed links are worth, and React Router's framework mode has type generation of its own.
- **Next.js.** The framework in [chapter 39](#) comes with its own file-based router.

Common mistakes

1. Editing `routeTree.gen.ts`

Your change works until you next add a route. Then the plugin rewrites the file and your change is gone. Change the files in `src/routes/` instead.

2. The plugin missing, or listed after `react()`

Without the plugin, the route map is never written, and you get `Cannot find module './routeTree.gen.ts'`. (You'll also see that if you run `npx tsc -b` in a fresh app before `npm run dev` has ever run.) The docs say `tanstackRouter()` must come before `react()`, so keep it first in the list.

3. Forgetting the `Register` block

This one is sneaky, because nothing breaks. The app runs, and there are no errors, because links aren't being checked at all. Delete the block and `<Link to="/bokks">` compiles happily. If a misspelled link isn't going red, check `main.tsx`.

4. Using `:id`, React Router's way

```
<Link to="/books/:bookId" params={{ bookId: "3" }}> // ❌  
// Type '"/books/:bookId"' is not assignable to type '"/books/$bookId"'?
```

TanStack Router uses `$`, in file names and in `to`.

5. Ordinary components inside `src/routes/`

The plugin treats every file there as a route. Put a `BookCard.tsx` in it, and the terminal warns that it `does not export a Route`. Keep components in `src/components/`. If you really want one next to its route, start its name

with a dash, like `-BookCard.tsx`, and the plugin skips it.

6. Reading search params without `validateSearch`

```
const { page } = Route.useSearch();  
// ❌ Property 'page' does not exist on type '{}'.  

```

Without a schema, the router has no idea what's in the URL, so neither does TypeScript.

7. Thinking "a string, not undefined" means "it exists"

`bookId` is always a string. Book `"999"` still doesn't exist. Check in the loader and `throw notFound()`.

8. Mixing up `react-router` and `@tanstack/react-router`

Both export a `Link`, an `Outlet` and a `useNavigate`. In a project with both installed (say, halfway through moving the bookstore across), VS Code's auto-import can pick the wrong one. React Router's `Link` accepts any string, so the tell-tale sign is a typo that doesn't go red. Check the import line.

Quick recap

- **TanStack Router** is a router built around TypeScript, so links, params and search params are checked before your code runs. This chapter uses v1 with file-based routing.
- **Routes are files** in `src/routes/`: `__root.tsx` is the layout, `index` is a folder's own page, and `$bookId` is a path param. The Vite plugin, listed before `react()`, writes `routeTree.gen.ts`. Never edit it.
- The **Register** block in `main.tsx` connects your routes to TypeScript. Without it, nothing is checked.
- `Route.useParams()` gives a `string`, never `undefined`, but the item can still be missing. That's what `throw notFound()` is for.
- **validateSearch** with a Zod schema turns the query string into typed values. `.default()` makes a param optional, and `.catch()` swaps rubbish for a fallback.
- **Loaders** fetch before render, with `pendingComponent`, `errorComponent` and `notFoundComponent` for the other outcomes. With TanStack Query, the loader calls `ensureQueryData` and the component reads the same cache.

Next: try the [exercises](#), then build the [37 Project: Task Board](#), which brings the whole of Level 4 together.

Exercises

How to do these:

- These exercises use the new `router-practice` app from the notes, not the playground. File-based routing takes over the whole app, so there's no `src/ch36/` folder this time. Exercises 1 to 4 build **one app, step by step**. Before you start each one, commit (or copy the folder) so you can always go back.
 - Keep `npm run dev` running while you work. The plugin only rewrites `routeTree.gen.ts` while it's running.
 - **Test every exercise with the back and forward buttons, and by refreshing on a deep URL** like `/books/3?page=2`. Those are the things a router exists for.
 - An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc -b` prints nothing, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Three pages and a shared header

A small bookshop wants a website: a home page, an about page and a contact page, with the same header on all three.

1. Set up `router-practice` as in the notes: the installs, `vite.config.ts` (plugin before `react()`), `main.tsx` with the `Register` block, and `__root.tsx` with a header, `<Outlet />` and a `notFoundComponent`.
2. Add `index.tsx` and `about.tsx` by writing them yourself.
3. With `npm run dev` running, create `src/routes/contact.tsx` as an **empty** file. Open it again a moment later. What's in it now? Tidy it up into a real contact page.
4. Open `src/routeTree.gen.ts` and find the `to:` line. Write down what it looked like before and after step 3.
5. Put a nav in the header with a link to each page, using `className="nav-link"`. Check in the Elements tab that the current page's link gets `active`, `data-status` and `aria-current`. Make sure Home isn't highlighted on the other pages.
6. Break it on purpose: change a link to `to="/contatc"`. Write the exact TypeScript error in a comment. Fix it.
7. Now delete the `Register` block from `main.tsx` and make the same typo again. What does VS Code say this time? Put the block back, and write one sentence about why that block matters.

What you should see: three pages, one header that never flickers, and a friendly message at `/nonsense`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): A page for every book

The shop's catalogue is the 12 books in `src/books.ts` (copied from chapter 29's starter).

1. Add `src/routes/books/index.tsx`: a list of every book's title, each one a `<Link>` to its own page.

2. Add `src/routes/books/$bookId.tsx`: the cover, title, author, genre, and the price as £9.99. Read the id with `Route.useParams()`.
3. Hover over `bookId` in VS Code and write down its type. Compare it with chapter 24's `useParams`.
4. `/books/999` must show "We couldn't find that book" with a link back to the list. Not a crash, and not a blank page.
5. Add **Books** to the header nav. Check that it stays highlighted while you're on a single book's page.
6. Make three mistakes on purpose and write down each error in a comment: a link with `to="/books/3"`, a link with `params={{ id: "3" }}`, and a link with `to="/books/:bookId"`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Filters that live in the URL

Now make the book list searchable, like [chapter 29's milestone 2](#), but with checked search params.

1. Add `validateSearch` to `books/index.tsx` with four params: `q` (default `""`), `genre` (`"all"` or one of `GENRES`, default `"all"`), `sort` (`"title"`, `"price-asc"` or `"price-desc"`, default `"title"`) and `page` (a whole number, at least 1, default 1).
2. Add a search box, a genre `<select>` and a sort `<select>`. All three read from `Route.useSearch()` and write with `useNavigate`. Changing any filter goes back to page 1.
3. Show 4 books per page, with **Previous** and **Next** links. Hide Previous on page 1, and Next on the last page.
4. Sort without mutating the array ([chapter 11](#)).
5. Add a **Clear filters** link that goes back to a plain `/books`.
6. Visit each of these by typing it into the address bar, and write down what the page shows: `?page=banana`, `?genre=poetry`, `?page=0`, `?sort=cheapest`, and `?q=1984`. One of them may surprise you.
7. Remove `.catch(1)` from `page` and visit `?page=banana` again. Then put it back, remove `.default(1)` instead, and read the errors in VS Code. Put it back again.
8. The URL fills up with every default (`?q=&genre=all&sort=title&page=1`). Tidy it so that plain `/books` stays plain.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Loaders and a slow server

Real catalogues live on a server. Time to fake one and load the data properly.

1. Create `src/api.ts` from the notes, with `fetchBooks()` and `fetchBook(id)`. Add a `console.log` inside each, so you can see when they run.

2. Give **both** book routes a loader. The components read with `Route.useLoaderData()`. Neither should import `books` directly any more.
3. The book loader throws `notFound()` for a missing book, and the route shows its own `notFoundComponent`.
4. Add a `pendingComponent` to both routes. Set the fake delay to 500ms, then 2000ms. When does the pending message appear, and what's on screen before it? Write down your answers.
5. Add an `errorComponent` with a **Try again** button. Make the API fail on purpose (for example, throw an `Error` whenever a flag you control is `true`). Check that the error shows, the header still works, and Try again recovers once you switch the flag off.
6. Turn on `defaultPreload: "intent"` in `createRouter`. Hover over a book link without clicking, and watch the Console.
7. Go list → book → back to the list. Does the list appear instantly? Does `fetchBooks` run again?
8. In a comment, compare this with chapter 18's fetch-in-an-effect. Write down three differences you noticed: in the code you wrote, and in how it feels to use.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Pick one, A or B

Both combine this chapter with earlier ones. Do one, or both if you're enjoying it.

A. Move the bookstore across. Copy your finished chapter 29 `bookstore` folder to `bookstore-ts`, run `npm uninstall react-router`, and install TanStack Router and the plugin. Then rebuild its routing:

1. `__root.tsx` holds the layout: shop name, nav, and the cart indicator from `useCart()`. `<CartProvider>` still wraps everything in `main.tsx`.
2. The book list keeps its filters in the URL, now with `validateSearch`.
3. The book page uses a loader and `notFound()`. The cart and checkout pages work as before, and a nonsense URL shows your 404.
4. Remove your hand-written `lazy()` for checkout. Run `npm run build` and confirm checkout is still its own chunk.
5. Get `npm tsc -b` clean. Keep a list of every broken link or wrong param TypeScript found for you along the way.
6. Get your chapter 28 tests passing again. `MemoryRouter` is gone, so `renderWithProviders` needs a new way to start at a given URL.

B. Add TanStack Query. In `router-practice`:

1. Pass a `QueryClient` through the router context with `createRootRouteWithContext`.
2. Both loaders call `ensureQueryData`. Use `booksQueryOptions` for the list and a `bookQueryOptions(id)` function for one book, with the `["books"]` and `["books", id]` keys from [chapter 31](#). The components read with `useSuspenseQuery`.

3. A missing book still shows the not-found page.
4. Add the TanStack Query devtools next to the router's. Go list → book → back, and use the devtools to prove the list came from the cache.
5. Set `staleTime: 30_000` on the list query and repeat step 4. What changed in the Console and in the devtools? Compare with what you saw in exercise 4, step 7.

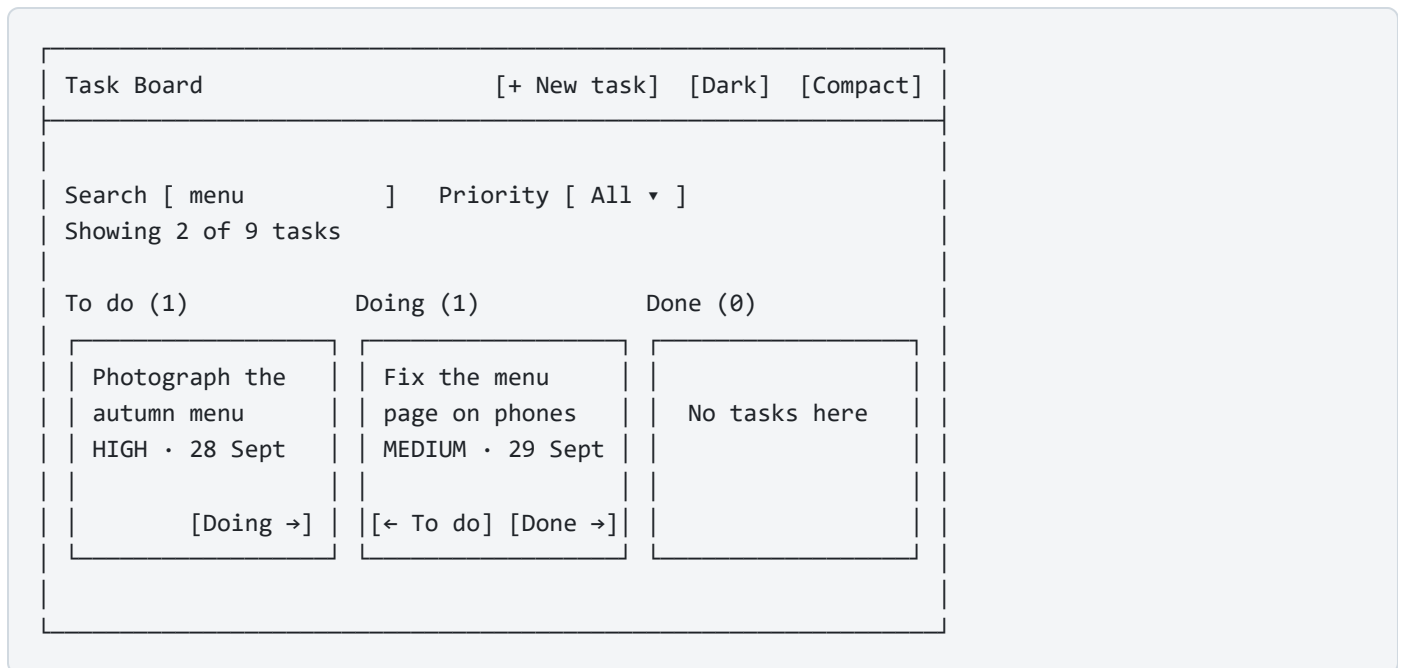
Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

37 Project: Task Board

What you'll build

A task board for a small team, with three columns of cards: **To do**, **Doing** and **Done**. You can add tasks, edit them, delete them, and move them from column to column. The tasks are saved on a server, so they're still there tomorrow.

This kind of board is called a **Kanban board** (*kanban* is Japanese for "signboard"). Picture a wall of sticky notes in columns, where you move each note along as the work gets done.



By the end, your app will have:

- a board that loads tasks from a real (practice) API, with proper loading and error states
- search and priority filters in the URL, so a filtered board can be shared in a link
- a page for every task at its own URL, like `/tasks/3`
- one form for both creating and editing a task, with clear error messages
- moves between columns that feel instant, and undo themselves if the server says no
- a light and dark theme and a compact view, remembered between visits
- tests for the parts most likely to break

This is the Level 4 project. It uses the whole toolbox, and gives each library **one job**:

Chapter	Its one job in this app
30 Axios	One <code>api</code> instance in <code>src/api/client.ts</code> , with an interceptor that turns errors into friendly messages
31 TanStack Query	Every read and write of tasks, and the instant moves between columns
32 Zod	The shape of a task, checked on every response. Also the shape of the URL's search params
33 React Hook Form	One <code>TaskForm</code> , used for both creating and editing
34 Zustand	Your preferences: the theme and compact cards, saved between visits
36 TanStack Router	The pages, typed links, filters in the URL, and loading a task before its page appears
28 Testing	Tests for the schemas, the preferences store, and the form
35 Redux Toolkit	Not used, on purpose (see below)
38 Accessibility (next level)	Moving cards with real buttons, not drag and drop, so it works with a keyboard

Why no Redux Toolkit? Real apps pick one library for shared client state, not two. This app's shared client state is tiny, just two settings, and Zustand handles that in about fifteen lines. Redux Toolkit shines in apps with lots of shared state and lots of people working on them. Swapping one for the other is a stretch goal, so you can compare them on the same app.

Why buttons, not drag and drop? Dragging needs a mouse or a finger, and someone using only a keyboard can't drag. So every card gets real buttons, like **Doing** →, that work with Tab and Enter. Drag and drop is a stretch goal, added *on top of* the buttons.

The API is your practice API from [chapter 30](#): json-server on port 3001. This app gets its own `db.json`, with tasks for a small team launching a bakery's new website.

Versions: React 19, Axios 1, TanStack Query v5, Zod 4, React Hook Form 7 (with `@hookform/resolvers` 5), Zustand 5, TanStack Router v1, and json-server 1 (still labelled beta). These libraries change fast. If something doesn't match what you see, check the chapter that taught it, then the library's docs.

Getting started

This project gets its own app, like `router-practice` in [chapter 36](#). File-based routes take over the whole app, so it can't live in the playground.

1. In a terminal **in this React folder**, run `npm create vite@latest`, answering as in [chapter 01](#), and name it `taskboard`.
2. Copy this chapter's `starter/src/` over `taskboard/src/`, and `starter/db.json` into `taskboard/`, **next to** `package.json` (not inside `src/`).
3. Delete `taskboard/src/App.tsx`, `taskboard/src/App.css` and `taskboard/src/assets/`. The route files replace `App.tsx`.
4. `cd taskboard`, then install everything:

```
npm install axios @tanstack/react-query @tanstack/react-query-devtools zod react-hook-form
@hookform/resolvers zustand @tanstack/react-router
npm install -D @tanstack/router-plugin json-server vitest @testing-library/react @testing-
library/user-event @testing-library/jest-dom jsdom
```

5. Add two scripts to `package.json`: `"api": "json-server --port 3001 db.json"` and `"test": "vitest"`.
6. Set up `vite.config.ts`, `src/test-setup.ts` and `src/main.tsx`. The starter README has all three. (Copy the config carefully: its two plugins must go in a certain order.)

Then open three terminals, all inside `taskboard`:

1. `npm run dev`: the app, at `http://localhost:5173`
2. `npm run api`: the practice API, at `http://localhost:3001/tasks`
3. `npm test`: the tests, re-running every time you save

Stop the playground's API first. Only one program can use port 3001 at a time. If the playground's json-server is still running, `npm run api` fails with an error mentioning `EADDRINUSE` ("address already in use"). Use `npx tsc -b` whenever you want a full type check.

The starter README also has a map of where every file goes. Two rules keep it tidy:

- **Only `src/api/` talks to the server.** No component calls `api.get` or knows the URL `/tasks`. Same idea as `api.ts` in the [Recipe Finder](#).
- **Only route files go in `src/routes/`.** The router plugin turns *every* file in there into a page, so components, stores and tests live somewhere else.

The big idea: four homes for state

The bookstore had three kinds of state. This app has four, and each one has a home:

Kind	In this app	Its home
Local	Half-typed form fields	React Hook Form
	"Is the delete confirmation showing?"	<code>useState</code>
URL	Search text, priority filter, which task is open	TanStack Router
Shared client	Theme, compact cards	Zustand
Server	The tasks themselves	TanStack Query

A quick test for each, asked in this order:

- **Is it a copy of something on the server?** → TanStack Query.
- **Would someone want to send it in a link?** → the URL.
- **Do components far apart need it?** → Zustand.
- **None of those?** → local, in the component.

The most common mistake is putting server data somewhere else: fetching the tasks, then copying them into Zustand or `useState` "so the whole app can use them". Now there are two copies. TanStack Query refetches and updates its copy. Yours goes stale, and nobody notices until the board shows a task that was deleted an hour ago. Any component can call `useQuery(tasksQueryOptions)` and get the same cached tasks. That is the shared copy.

Milestone 1: Setup and routes

Goal: three pages with real URLs, sharing one header.

Your `main.tsx` (from the starter README) creates the router and does the `Register` step from [chapter 36](#), so TypeScript can check every link in the app against your real routes. The `routeTree` it imports lives in `routeTree.gen.ts`, which the router plugin writes while `npm run dev` is running. **Never edit that file.**

The starter gives you `__root.tsx` and `index.tsx`. Add two more route files:

File	URL	For now, it shows
<code>src/routes/index.tsx</code>	<code>/</code>	The board stub (from the starter)
<code>src/routes/tasks/new.tsx</code>	<code>/tasks/new</code>	A heading: "New task"
<code>src/routes/tasks/\$taskId.tsx</code>	<code>/tasks/3</code>	A heading with the task's id

In current versions, if you create an empty file in `src/routes/` while the dev server is running, the plugin fills in a starter route for you. On the task page, `Route.useParams()` gives you `taskId` as a `string`. Compare that with

[chapter 24](#), where `useParams` was always `string | undefined`. This router knows the page only exists when there's an id.

Then fill in the root layout in `__root.tsx`: the app name linking to `/`, a **New task** link, a theme toggle button that does nothing yet (milestone 8 wires it up), and `<Outlet />`. Add a `NotFoundComponent` to the root route, with a link back to the board.

Check it: type each URL straight into the address bar: `/`, `/tasks/new`, `/tasks/3`. Each shows its page under the header, and `/nonsense` shows your not-found page. Notice that `/tasks/new` shows the new-task page, not a task with the id "new": a fixed path beats a `$param`. Then make a typo on purpose, `<Link to="/taks/new">`, and write down the exact error `npx tsc -b` gives you.

Milestone 2: The API layer

Goal: every request in one folder, and every response checked. No UI yet.

Start with `src/api/client.ts`: the `api` instance from [chapter 30](#), with `baseUrl: "http://localhost:3001"`. Add a small `ApiError` class that extends `Error` and has one extra field, `status: number | null` (`null` means the server never answered). Then the interceptor turns every failure into one:

```
api.interceptors.response.use(
  (response) => response,
  (error: unknown) => {
    if (axios.isAxiosError(error)) {
      const status = error.response?.status ?? null;
      return Promise.reject(new ApiError(friendlyMessage(status), status));
    }
    return Promise.reject(error);
  },
);
```

Now every error has a message a person can read, and still keeps its status code, so later code can tell "not found" apart from "server down". `friendlyMessage` is yours to write: `null` might say "Can't reach the server. Is the API running (npm run api)?", `404` "That task doesn't exist.", and anything else "Please try again".

Next, `src/api/schemas.ts`. **These schemas are the single source of truth.** The types, the API checks and the form's error messages all come from here:

```

export const STATUSES = ["todo", "doing", "done"] as const;
export const PRIORITIES = ["low", "medium", "high"] as const;

export const TaskSchema = z.object({
  id: z.string(),
  title: z.string().trim().min(1, "Give the task a title").max(80, "Keep the title to 80 characters or fewer"),
  description: z.string().max(500, "Keep the description to 500 characters or fewer"),
  status: z.enum(STATUSES),
  priority: z.enum(PRIORITIES),
  dueDate: z.iso.date().nullable(), // "2026-10-05", or null for "no due date"
  createdAt: z.iso.datetime(), // "2026-09-14T09:20:00.000Z"
});

export const TaskListSchema = z.array(TaskSchema);
export const NewTaskSchema = TaskSchema.omit({ id: true, createdAt: true });

export type Task = z.infer<typeof TaskSchema>;
export type NewTask = z.infer<typeof NewTaskSchema>;
export type TaskStatus = Task["status"];

```

A few things to notice. `description` can be an empty string: that's what "no description" looks like here, so nobody has to wonder whether it's `""` or `undefined`. `NewTaskSchema` leaves out what the app fills in itself (json-server makes the `id`, and `createTask` adds `createdAt`). And the error messages live in the schema, so the form in milestone 5 gets them for free.

Then `src/api/tasks.ts`, with five functions. Each sends a request and **parses** what comes back ([chapter 32](#)):

```

export async function getTasks(): Promise<Task[]> {
  const response = await api.get("/tasks");
  return TaskListSchema.parse(response.data);
}

```

Write the rest the same way: `getTask(id)`, `createTask(newTask)` (POST, adding `createdAt: new Date().toISOString()`), `updateTask(id, changes)` (PATCH, with `changes: Partial<NewTask>`), and `deleteTask(id)` (DELETE, returning nothing).

Now test the schemas, before any UI exists. It's the bookstore's reducer-first move again: plain functions, no rendering, no mocking. In `src/api/schemas.test.ts`, start from one valid task object (copy one from `db.json`) and change one thing per test:

```
test("rejects a priority the app doesn't know", () => {
  const result = TaskSchema.safeParse({ ...validTask, priority: "urgent" });

  expect(result.success).toBe(false);
  expect(result.error?.issues[0].path).toEqual(["priority"]);
});
```

Aim for five: a valid task passes; `priority: "urgent"` fails at `["priority"]`; `dueDate: null` passes but `"next Tuesday"` fails; a title of only spaces fails with your message; and `NewTaskSchema` accepts a task with no `id` or `createdAt`.

Check it: `npm test` passes. Then try the real API. In `db.json`, change one task's `"priority"` to `"urgent"` and save. (json-server usually notices. If not, stop it and run `npm run api` again.) With the app open in the browser, type this in the DevTools Console:

```
const { getTasks } = await import("/src/api/tasks.ts");
await getTasks();
```

This works because Vite serves your source files as modules while you develop. You should get a Zod error naming `priority` and the three allowed values. The API sent bad data, and your code caught it. Put `"medium"` back. (If the Console trick feels fiddly, do this check after milestone 3, when the board can show the error.)

Milestone 3: Reading the board

Goal: three columns of real tasks, from one request.

In `main.tsx`, create one `QueryClient`. Wrap `<RouterProvider>` in `<QueryClientProvider>`, with `<ReactQueryDevtools />` inside it too ([chapter 31](#)). Then make `src/api/queries.ts`, so every component asks for tasks in the same way:

```
export const tasksQueryOptions = queryOptions({ queryKey: ["tasks"], queryFn: getTasks });

export function taskQueryOptions(taskId: string) {
  return queryOptions({ queryKey: ["tasks", taskId], queryFn: () => getTask(taskId) });
}
```

(Watch the names: `tasks` for the list, `task` for one.) Both keys start with `"tasks"` on purpose. Query keys match by their beginning, so `invalidateQueries({ queryKey: ["tasks"] })` refreshes the list **and** every single task. You'll rely on that in milestones 5 and 6.

On the board page, call `useQuery(tasksQueryOptions)`, handle `isPending` and `isError` first, then split the data during render:

```

<div className="board">
  {STATUSES.map((status) => (
    <Column key={status} status={status} tasks={tasks.filter((task) => task.status === status)}
  />
  ))}
</div>

```

One query, split by status, never stored. Not three queries: that's three requests, and a moved task would have to leave one cache and join another. Not three `useState` arrays: that's a copy of server data. Just one list, and three `filter` calls during render. It's "don't store what you can calculate" from [chapter 08](#), again.

For the error state, show the `ApiError`'s friendly message and a **Try again** button that calls `refetch()`. If the error isn't an `ApiError`, it's probably Zod saying the data is wrong. Show a general "The server sent something this app doesn't understand", and `console.error` the details for yourself.

Build `Column` (a `<section>` with a heading like **To do (4)**, and a `<ul>` of cards) and `TaskCard` (the title, a priority badge and the due date). A lookup object keeps the column names in one place:

```

export const STATUS_LABELS = { todo: "To do", doing: "Doing", done: "Done" } satisfies
  Record<TaskStatus, string>;

```

`satisfies` ([chapter 25](#)) makes TypeScript complain if you ever add a status and forget its label. Before you build the card, read the starter README's notes on its structure (a link and buttons, side by side) and on formatting dates.

Check it: three columns with 4, 3 and 2 tasks. The Query devtools show **one** query, `["tasks"]`. Now stop json-server (Ctrl+C in terminal 2) and refresh the page. "Loading..." stays for several seconds before your error appears, because TanStack Query retries three times first. Start the API again and click **Try again**.

Milestone 4: Filters in the URL

Goal: a search box and a priority filter that live in the URL.

In the [bookstore](#), filters came from `useSearchParams` as plain strings. TanStack Router checks them with a Zod schema instead ([chapter 36](#)). In `src/routes/index.tsx`:

```
const BoardSearchSchema = z.object({
  q: z.string().default(""),
  priority: z.enum(["all", "low", "medium", "high"]).default("all"),
});

export const Route = createFileRoute("/")({
  validateSearch: BoardSearchSchema,
  component: BoardPage,
});
```

Now `Route.useSearch()` gives you `{ q: string; priority: "all" | "low" | "medium" | "high" }`. Not `string | null`: the real type. Filter during render, exactly as the bookstore's milestone 2 did (match `q` against the title and description, and `priority` unless it's `"all"`). Then split `visible`, not `tasks`, into the columns.

To change a filter, navigate to the same page with new search params:

```
const navigate = useNavigate({ from: Route.fullPath });

function handleSearchChange(value: string) {
  navigate({ search: (prev) => ({ ...prev, q: value }), replace: true });
}
```

`replace: true` stops every keystroke from adding a history entry, the same decision you made in the bookstore. Make it again for the priority dropdown, and write down why.

Keep `FilterBar` a plain component that takes `q`, `priority` and two `on...Change` props. The route reads and writes the URL; the component just shows the controls. One snag: a `<select>`'s value is a plain `string`. Instead of reaching for `as`, check it with a Zod enum. The DOM is a boundary too.

Then the small things that make a board feel finished: **"Showing 4 of 9 tasks"** under the filters (derived, of course); a **"No tasks here"** message in any empty column, instead of a blank gap; and filters that **stay visible** when nothing matches, so people can undo what they did.

Check it: choose **Medium**. You should see "Showing 4 of 9 tasks". Copy the URL into a new tab: same board, same filter. Then edit the URL by hand to `?priority=urgent`. The schema rejects it, and the router shows an error. Now add `.catch("all")` after `.default("all")` and try again. Which would you rather give someone who follows an old, broken link?

Milestone 5: Creating tasks

Goal: one form component, and a page that uses it to add tasks.

Make `src/components/TaskForm.tsx`. The most important decision in this milestone: **the form doesn't know about the API**. It checks what was typed, then hands the values to whoever is using it:

```
type TaskFormProps = {
  defaultValues: NewTask;
  submitLabel: string;
  onSubmit: (values: NewTask) => Promise<unknown>;
  disableUntilChanged?: boolean;
};
```

That's what makes it reusable (milestone 6 uses it for editing) and testable (milestone 9 renders it with a fake `onSubmit`: no server, no router). So there's no `useMutation` inside it, and no `Link` either. Put the Cancel link in the page, next to the form.

Inside, `useForm<NewTask>` takes `resolver: zodResolver(NewTaskSchema)` and the `defaultValues`. The submit handler is where server errors get caught:

```
async function submit(values: NewTask) {
  try {
    await onSubmit(values);
    reset(values); // what was just saved becomes the new "unchanged"
  } catch (error) {
    setError("root", {
      message: error instanceof Error ? error.message : "Couldn't save the task.",
    });
  }
}
```

There are five fields: title (`<input>`), description (`<textarea>`), status and priority (two `<select>` s), and due date (`<input type="date">`). Each gets a real `<label>`, plus `aria-invalid` and `aria-describedby` pointing at its error message, as in [chapter 33](#). Show `errors.root?.message` above the buttons in a `<p role="alert">`.

The date needs converting. An empty date box gives you `""`, but the schema wants `null`. `setValueAs` converts it on the way in, the way `valueAsNumber` did for numbers:

```
{...register("dueDate", { setValueAs: (value: string) => (value === "" ? null : value) })}
```

The submit button is `disabled={isSubmitting || (disableUntilChanged && !isDirty)}`, and says "Saving..." while `isSubmitting` is true.

Now the page, `src/routes/tasks/new.tsx`. It owns the mutation, and passes it to the form:


```
const createMutation = useMutation({
  mutationFn: createTask,
  onSuccess: async () => {
    await queryClient.invalidateQueries({ queryKey: ["tasks"] });
    await navigate({ to: "/" });
  },
});

<TaskForm defaultValues={emptyTask} submitLabel="Create task" onSubmit={values =>
  createMutation.mutateAsync(values)} />
```

`emptyTask` is a blank `NewTask` (empty title and description, `"todo"`, `"medium"`, and a `null` due date). Keep it in `schemas.ts`, so the page and the tests can share it.

`mutateAsync` returns a promise, so the form's `isSubmitting` stays true until the server answers. That's your "Saving..." state for free, with no need to check `createMutation.isPending` as well. One source of truth.

Check it: create a task with the status **Doing**. You land on the board, and it's in the Doing column, with no page refresh. Click **Create task** with an empty title: the message appears, and the Network tab shows no request, because the check happened first. Stop `json-server` and submit a valid task: your "Can't reach the server" message appears, and everything you typed is still there. Finally, change `register("title")` to `register("titel")` and read the type error. [Chapter 09](#) said TypeScript couldn't catch that. Now it can.

Milestone 6: The task page: view, edit, delete

Goal: every task at its own URL, loaded before the page appears, and edited with the same form.

The task page's loader needs the `QueryClient`, but loaders run outside React, so they can't call hooks. **Router context** passes it in ([chapter 36](#)). In `__root.tsx`, swap `createRootRoute` for this:

```
type RouterContext = { queryClient: QueryClient };

export const Route = createRootRouteWithContext<RouterContext>()({
  component: RootLayout,
  notFoundComponent: NotFoundPage,
});
```

(Yes, two sets of brackets: the first call takes the type, the second takes the options.) Then hand it over in `main.tsx`: `createRouter({ routeTree, context: { queryClient } })`. Now the loader, in `src/routes/tasks/$taskId.tsx`:

```
export const Route = createFileRoute("/tasks/$taskId")({
  loader: async ({ context, params }) => {
    try {
      await context.queryClient.ensureQueryData(taskQueryOptions(params.taskId));
    } catch (error) {
      if (error instanceof ApiError && error.status === 404) {
        throw notFound();
      }
      throw error;
    }
  },
  notFoundComponent: () => <p className="status-message">We couldn't find that task.</p>,
  component: TaskPage,
});
```

`ensureQueryData` means "use the cached task if there is one, and fetch it if not". The page only renders once the task is there, so inside `TaskPage`, `useSuspenseQuery(taskQueryOptions(taskId))` gives you `data` that's never `undefined`. And milestone 2's `status` pays off here: a 404 becomes a proper not-found page, while "server down" stays an error.

Show the task's details (status, priority, due date, when it was created), then the same `TaskForm`, this time for editing: `submitLabel="Save changes"`, `disableUntilChanged`, and `defaultValues={NewTaskSchema.parse(task)}`. That last one is a neat trick. Zod objects drop keys they don't know, so it turns a `Task` into a `NewTask` by removing `id` and `createdAt`.

The update must refresh **both** caches, the board's list and this task:

```
const updateMutation = useMutation({
  mutationFn: (changes: NewTask) => updateTask(taskId, changes),
  onSuccess: (updated) => {
    queryClient.setQueryData(taskQueryOptions(taskId).queryKey, updated);
    return queryClient.invalidateQueries({ queryKey: ["tasks"] });
  },
});
```

Delete, with a confirm step. "Is the confirmation showing?" is local state: a plain `useState(false)`. The first click on **Delete task** shows "Delete this task for good?" with **Yes, delete** and **Cancel**. Only **Yes, delete** runs the mutation. When it succeeds, go to `/` first, *then* tidy the cache: `removeQueries` for this task's key, and `invalidateQueries` for the list. If you remove the task from the cache while its page is still showing, the page tries to fetch it again and gets a 404.

Check it: open a task from the board, then refresh. It appears straight away, with no loading flash. `/tasks/999` shows your not-found message. Edit a title: **Save changes** stays disabled until you change something, and goes

back to disabled after saving. Back on the board, the new title is there. Delete a task: it's gone from the board, and pressing Back to its old URL shows not found.

Milestone 7: Moving tasks, optimistically

Goal: moves that feel instant, and undo themselves if they fail.

Each card gets a button for each neighbouring column. Two lookup objects say where a task can go:

```
const PREVIOUS: Record<TaskStatus, TaskStatus | null> = { todo: null, doing: "todo", done: "doing" };
const NEXT: Record<TaskStatus, TaskStatus | null> = { todo: "doing", doing: "done", done: null };
```

The buttons are real `<button>`s, siblings of the card's link. The visible text is short, and a visually hidden part gives each one a full name. A screen reader hears "Move Photograph the autumn menu to Doing", not five identical "Doing" buttons:

```
<button className="btn btn-secondary btn-small move-forward" onClick={() => onMove(task.id, next)}>
  <span className="visually-hidden">Move {task.title} to </span>
  {STATUS_LABELS[next]} <span aria-hidden="true">></span>
</button>
```

Without help, a click waits for the server before the card moves. It's only a fraction of a second, but it feels sluggish, and the move will almost always work. So move the card **first**, and put it back if the server says no. That's an **optimistic update** ([chapter 31](#)). Add `useMoveTask` to `src/api/queries.ts`:

```

export function useMoveTask() {
  const queryClient = useQueryClient();
  const { queryKey } = tasksQueryOptions;

  return useMutation({
    mutationFn: ({ id, status }: { id: string; status: TaskStatus }) => updateTask(id, { status
      }),

    onMutate: async ({ id, status }) => {
      await queryClient.cancelQueries({ queryKey }); // 1. stop a refetch overwriting us
      const previous = queryClient.getQueryData(queryKey); // 2. remember how it was
      queryClient.setQueryData(queryKey, (old) => // 3. move the card now
        old?.map((task) => (task.id === id ? { ...task, status } : task)),
      );
      return { previous };
    },
    onError: (_error, _variables, snapshot) => {
      queryClient.setQueryData(queryKey, snapshot?.previous); // 4. put it back
    },
    onSettled: () => queryClient.invalidateQueries({ queryKey: ["tasks"] }), // 5. ask the
      server
    });
}

```

Step 5 runs whether the move worked or not. It's the safety net: whatever you guessed, the server's answer wins in the end. And because `queryKey` came from `tasksQueryOptions`, `getQueryData` and `setQueryData` already know the data is `Task[]`.

Call it in the board, not in the card. This one is subtle. When a task moves, React removes its card from the old column and creates a brand-new card in the new one, because a component's state belongs to its place in the tree ([chapter 12](#)). If the card owned the mutation, its error would vanish with the old card. So the board page calls `useMoveTask()` once, passes an `onMove(id, status)` prop down through `Column` to each `TaskCard`, and shows `moveTask.error.message` in a `<p role="alert">` above the columns when `moveTask.isError`.

Check it: moves feel instant. Throttle the Network tab to Slow 3G and move a card: it moves at once, and the PATCH finishes later. Now stop json-server and move a card. It jumps to the new column, jumps back, and your message appears. (Without throttling, this can happen so fast you barely see the jump.) Start the API again, move a card, and refresh: it stayed moved.

An honest note: if you click lots of moves very fast while the server is failing, the rollbacks can trip over each other. The refetch in `onSettled` puts everything right in the end, and that's enough for a board this size.

Milestone 8: Preferences with Zustand

Goal: a light or dark theme and compact cards, remembered after a refresh.

Why Zustand? Run the four-homes test. The theme isn't on the server, so it's not TanStack Query. Nobody sends their theme in a link, so it's not the URL. And components far apart need it: the header's toggles and every card. That's shared client state. Make `src/stores/preferences.ts`, with `theme` (`"light" | "dark"`), `compact` (a boolean), and an action to toggle each:

```
export const usePreferences = create<PreferencesState>(){
  persist(
    (set) => ({
      theme: "light",
      compact: false,
      toggleTheme: () => set((state) => ({ theme: state.theme === "light" ? "dark" : "light" })),
      toggleCompact: () => set((state) => ({ compact: !state.compact })),
    }),
    { name: "taskboard-preferences" },
  ),
};
```

`persist` saves the store in `localStorage` under that name, and loads it again on startup ([chapter 34](#)). You write no saving code at all, just like `useLocalStorage` in [chapter 20](#).

The stylesheet switches colours when `<html>` has `data-theme="dark"`. Changing `<html>` is a job outside React, so it's an effect ([chapter 17](#)): `document.documentElement.dataset.theme = theme`, re-run when `theme` changes. Put the effect in a small `PreferenceToggles` component in the header, next to the two toggle buttons. Keeping it there, not in the root layout, means only the toggles re-render when a preference changes. It's [chapter 27](#)'s "move state down", for a store. Give each toggle `aria-pressed`, so a screen reader hears "pressed" or "not pressed".

Then each `TaskCard` reads just what it needs, and adds `compact` to its class name:

```
const compact = usePreferences((state) => state.compact);
```

The selector matters. A card that selected the whole store would also re-render every time the theme changed. (You may see the light theme flash for a moment on refresh, because effects run just after the first paint. A proper fix needs a tiny script in `index.html`. It isn't worth it here.)

Check it: switch to dark and refresh: still dark. Find the saved JSON in DevTools → Application → Local Storage. Then check your selectors: turn on "Highlight updates when components render" in React DevTools (or put a `console.log` in `TaskCard`), and toggle the theme. The cards shouldn't re-render. Toggle compact: now they do.

Milestone 9: Tests

Goal: tests for the three things most likely to break: the data rules, the store and the form.

The **schemas** are already tested (milestone 2).

The **preferences store** is plain TypeScript, so test it by calling it directly, with no rendering. Reset it before every test, or one test's dark theme leaks into the next ([chapter 34](#)):

```
beforeEach(() => {
  localStorage.clear();
  usePreferences.setState(usePreferences.getInitialState());
});
```

Then three small tests: it starts light and not compact; `usePreferences.getState().toggleTheme()` switches to dark and back; `toggleCompact()` flips `compact`.

TaskForm, on its own. This is where milestone 5's decision pays off. The form doesn't use the API or the router, so the test needs no providers at all. Just pass `vi.fn()` as `onSubmit` ([chapter 28](#)):

```
test("sends what the user typed", async () => {
  const user = userEvent.setup();
  const handleSubmit = vi.fn();
  render(<TaskForm defaultValues={emptyTask} submitLabel="Create task" onSubmit={handleSubmit} />);

  await user.type(screen.getByLabelText("Title"), "Book the venue");
  await user.selectOptions(screen.getByLabelText("Priority"), "high");
  await user.click(screen.getByRole("button", { name: "Create task" }));

  await waitFor(() =>
    expect(handleSubmit).toHaveBeenCalledWith({
      title: "Book the venue", description: "", status: "todo", priority: "high", dueDate: null,
    }),
  );
});
```

`waitFor` (from `@testing-library/react`) keeps retrying the check for a moment. You need it because React Hook Form checks the form asynchronously, so `onSubmit` runs a moment after the click. Write two more:

1. **An empty title shows the error, and never calls `onSubmit`.** Click **Create task** straight away, `await screen.findByText("Give the task a title")`, then `expect(handleSubmit).not.toHaveBeenCalled()`.
2. **A server error shows in the form.** Make `onSubmit` fail with `vi.fn().mockRejectedValue(new Error("Can't reach the server."))`, submit a valid task, and find the message with `findByRole("alert")`.

Use `getByRole` and `getByLabelText` throughout. If one can't find a field, the field is probably missing its label. Fix the form, not the test.

An honest note. You haven't tested whole pages, like "the board shows 9 tasks". That needs the router, a `QueryClient` and a fake API, all set up inside the test. It's very doable, but it's a big step up in setup, so it's a stretch goal.

Check it: `npm test` is green, with about eleven tests. Rename a few variables inside `TaskForm`: every test still passes, because the tests check what the user sees, not how you wrote it.

Now compare

Put the to-do app, the bookstore and the task board side by side:

	To-do (ch10)	Bookstore (ch29)	Task board (ch37)
Where the data lives	<code>localStorage</code>	A file in <code>src/</code>	A server (your practice API)
Getting it	Read once at startup	Imported	TanStack Query: cached, retried, refreshed
Saving changes	An <code>updateTasks</code> helper	Reducer actions, in memory only	Mutations; moves are optimistic
Shared state	None	Context + reducer	Zustand, for two settings
Forms	<code>useState</code> per field, checked by hand	Checkout, checked by hand	React Hook Form + one Zod schema, for create <i>and</i> edit
Pages	One	React Router; <code>useParams</code> gives <code>string undefined</code>	TanStack Router; params and search are typed
Bad data	<code>as Task[]</code> , and hope	Can't happen: it's your own file	Zod rejects it, with a clear message
Checking it works	Clicking	Tests	Tests, plus TypeScript catching broken links
Who wrote the plumbing	You, all of it	You, nearly all of it	The libraries. You wrote the decisions

The libraries didn't replace the fundamentals. Look at what each one actually replaced:

- TanStack Query: the loading, error and data state and the `ignore` flags you wrote by hand in [chapter 18](#) and the Recipe Finder.
- Zod: `as Task[]`, which was only a promise.
- React Hook Form: a `useState` per field, and the hand-written checks from [chapter 09](#).

- Zustand: a context and a provider, like the bookstore's cart.
- TanStack Router: `string | undefined` params and `searchParams.get()` strings.

Every one of them replaced **boilerplate you had already written yourself**. That's why none of them felt like magic. And the fundamentals are still everywhere. The board filters during render: that's [chapter 08](#). The optimistic update copies the list with `map` and a spread: [chapter 11](#). A moved card loses its state: [chapter 12](#). You know what's underneath, so when a library surprises you, you can work out why.

A question to think about: if this app were half the size, which library would you remove first? Our answer is Zustand. Two settings could live in a context, or in `useLocalStorage` from chapter 20, with about the same amount of code. Next might be the router, if the app had only one page. TanStack Query and Zod would be the last to go: as long as there's a server, there's loading, errors, caching, and data you can't trust. Do you agree? Being able to argue for *removing* a library is just as useful as knowing how to add one.

Common mistakes

1. Copying server data into Zustand or `useState`

```
const { data } = useQuery(tasksQueryOptions);
const [tasks, setTasks] = useState(data ?? []); // ❌ a second copy that never updates
```

The copy goes stale the moment TanStack Query refetches. Use `data` directly, wherever you need it.

2. One query per column

A `["tasks", "todo"]` query, then `"doing"`, then `"done"`: three requests, three caches, and a moved task has to leave one and join another. Fetch once, and split with `filter` during render.

3. Filters in Zustand instead of the URL

It works, and it quietly takes away sharing, bookmarking and refreshing a filtered board. If someone would send it in a link, it goes in the URL.

4. Forgetting the single-task cache after an edit

Naming the keys `["tasks"]` and `["task", id]` looks harmless. But now `invalidateQueries({ queryKey: ["tasks"] })` misses the task page, and it keeps showing the old title. Start every task key with `"tasks"`, so one invalidation covers them all.

5. Parsing in components instead of in `tasks.ts`

If every component that fetches has to remember `TaskListSchema.parse(...)`, one of them will forget. Parse once, in the function that made the request. Everything outside `src/api/` can then trust the data.

6. A `TaskForm` that calls the API itself

Once it contains `useMutation` or a `Link`, you can't reuse it for editing without `if (isEditing)` branches, and you can't test it without a server and a router. Take `onSubmit` as a prop, and let the page decide what submitting means.

7. An optimistic update with no rollback

A `setQueryData` in `onMutate` with no `onError`: when the server says no, the board keeps showing a move that never happened. Take a snapshot in `onMutate`, restore it in `onError`, and refetch in `onSettled`.

8. Drag and drop as the only way to move a task

Someone using a keyboard or a screen reader can't move anything. Buttons first. Drag and drop is a nice extra on top.

Quick recap

- Sort state into **four homes** before you write it: local (React Hook Form, `useState`), URL (the router), shared client (Zustand) and server (TanStack Query).
- **Server data belongs to TanStack Query.** Never copy it into Zustand or `useState`.
- Keep the **API layer in one folder**: one Axios instance, Zod schemas as the single source of truth, and every response parsed where it's fetched.
- **Fetch once, derive the rest.** Columns, filtered lists and counts are all calculated during render.
- **Filters and ids live in the URL**, checked by a schema, so they're typed and shareable.
- **One form for create and edit**, which only calls `onSubmit`. That's what makes it reusable and easy to test.
- **Optimistic updates** make moves feel instant: snapshot, change, roll back on error, refetch when settled.
- Each library replaced **code you had already written by hand**. Knowing what's underneath lets you choose, or remove, a library with confidence.

Next: try the [stretch goals](#). That's the end of Level 4. [Level 5](#) starts with accessibility: making sure everyone can use what you build, including people who never touch a mouse. Later, [chapter 40](#) covers putting apps online, including what to do about the practice API, because json-server only runs on your own computer.

Exercises

These are **stretch goals**: extra features for your finished task board. Do them in any order, and do as many as you like.

How to do these:

- Finish all 9 milestones first. Then commit or copy the `taskboard` folder (including `db.json`), so you've always got a working version to go back to.
 - `index.css` already has styles for these goals. Look for "Styles for the stretch goals" near the bottom, and the class names in `starter/README.md`.
 - Keep all three terminals running: `npm run dev`, `npm run api` and `npm test`.
 - **Write the test first for anything with rules in it**, like dates or sorting. Pure functions are the cheapest things to test.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Due-date badges

A date like "29 Sept" makes you do the maths. A badge saying **Overdue** doesn't.

1. Write a pure function in `src/dates.ts` that takes a task's `dueDate`, its `status` and today's date (as a "YYYY-MM-DD" string), and returns the badge to show, or `null` for none.
2. The badges: **Overdue** (due before today), **Due today**, **Due tomorrow**, and **Due in N days** for up to a week ahead. Anything further away, or with no due date, gets no badge.
3. A task in **Done** never gets a badge, however late it was.
4. **Write the tests first**, in `src/dates.test.ts`: one for each badge, one for "more than a week away", one for `null`, and one for a finished task.
5. Show the badge on each card with the `due-badge` classes (`overdue`, `due-today`, `due-soon`).

What you should see (if today were 30 September 2026, as in the seed data): "Photograph the autumn menu" and "Fix the menu page on phones" say **Overdue**, "Set up the online order form" says **Due today**, "Write the About page" says **Due tomorrow**, and "Test the site on an old laptop" says **Due in 6 days**. The two Done tasks have no badge.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Sorting, in the URL

Add a **Sort by** dropdown to the filter bar: *Newest first*, *Priority (high first)* and *Due date (soonest first)*.

1. The choice lives in the URL next to the filters, like `/?priority=all&sort=due`. Add it to your board's search schema, with a default.
2. Sorting happens inside each column, during render. Never sort the cached array itself ([chapter 11](#)).
3. Tasks with no due date go **last** when sorting by due date.
4. Test your sort functions, including a tie (two tasks with the same priority) and a `null` due date.

5. Add a **Clear filters** link that takes you back to the board with every search param at its default.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Undo delete

Deleting something by accident is horrible. Replace the confirm step with an **Undo** button, like most email apps do.

1. Make a toast store in Zustand: a list of toasts, each with an id, a message and an optional action (a label and a function). Add `showToast` and `dismissToast` actions. This store is **not** persisted: toasts shouldn't come back after a refresh.
2. Show the toasts in a `toast-region` in the root layout. Give it `role="status"`, so screen readers announce new ones.
3. Deleting a task goes straight back to the board and shows "Task deleted" with an **Undo** button, for about five seconds.
4. Clicking **Undo** brings the task back.

There are two honest ways to do step 4, and both have a catch:

- **Wait before deleting.** Hide the task from the board straight away (an optimistic update), but only send the DELETE after five seconds, if nobody pressed Undo. The catch: what if they close the tab during those five seconds?
- **Delete now, re-create on Undo.** Send the DELETE at once, and POST the task again if they press Undo. The catch: find out whether json-server keeps an `id` you send in a POST, or makes a new one. If the id changes, what happens to a link someone saved to `/tasks/3`?

Pick one, build it, and write a comment explaining your choice.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Swap Zustand for Redux Toolkit

This app picked Zustand. Try the other road, so your opinion is based on experience rather than on what you've read.

1. Do this on a copy of the project (or a new git branch).
2. Replace `src/stores/preferences.ts` with a Redux Toolkit slice ([chapter 35](#)): the same two settings, and a reducer for each toggle.
3. Set up the store, the `<Provider>`, and the typed hooks, as chapter 35 showed you.

4. Keep the preferences saved between visits. Redux Toolkit has no built-in `persist`, so write it yourself: load from `localStorage` when you create the store, and save when the state changes.
5. Update the store tests, then open Redux DevTools and watch your toggles appear as actions.
6. Write down: how many files changed, how many lines each version took, which one was easier to test, and which one you'd pick **for this app**. Then, at what size of app would you change your mind?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Make it bigger

Three larger additions. Pick one, or take them in any order.

A. Drag and drop, on top of the buttons. Use a library such as `@dnd-kit` to let people drag cards between columns. The move buttons stay: dragging is an extra, never the only way. A drop calls the same `onMove` as the buttons, so it gets the optimistic update for free. Use the `dragging` and `drop-target` classes to show what's happening. (Drag-and-drop libraries change their APIs often, and `@dnd-kit` has both an older and a newer set of packages. Check its docs for the one it currently recommends.)

B. More than one board. Add a `boards` collection to `db.json`, and give every task a `boardId`. Move the board to `/boards/$boardId`, with its tasks at `/boards/$boardId/tasks/$taskId`, using a layout route for the board's header and a `board-nav` of links to the other boards. Make `/` send people to the first board. Think carefully about your query keys: what should `invalidateQueries` refresh when a task on board 2 changes?

C. Tests for whole pages. Test the real routed pages, not just their parts: the board shows 9 tasks; a URL with `?priority=high` shows 3; `/tasks/999` shows not found; and a move rolls back when the server fails. Use TanStack Router's memory history to start at any URL, a fresh `QueryClient` per test, and **MSW** (Mock Service Worker, named in [chapter 28](#)) to fake json-server.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

When you're done

That's the end of Level 4, and the end of the toolbox. You've now used the libraries most React teams use, and you know what each one replaced.

Before moving on, write down:

- One library you'd happily use again, and one you're not sure earned its place in this app.
- One bug a library caught for you while you were building (a Zod error, a typed link, a type error in a form).
- One thing you'd do differently if you started this project again.

Bring the last one to Claude and talk it through.

Level 5: Mastery

Goal: build apps that everyone can use, and ship them for the world to see.

38 Accessibility

Welcome to Level 5: Mastery. In Levels 1 to 4 you learned to build real apps, with the tools real teams use. Level 5 is the last two steps: building apps that **everyone** can use, and shipping them for the world to see. This chapter is the first step. [Chapter 39](#) meets a React framework, [chapter 40](#) puts your apps online, and [chapter 41](#) is a project of your own.

What is it?

Accessibility means building apps that everyone can use, whatever their body or situation. It's often written **a11y**: an "a", then 11 letters, then a "y". "Everyone" includes people who:

- are blind and use a **screen reader**: a program that reads the page aloud (or sends it to a braille display) and lets them move around it with the keyboard.
- can't use a mouse, and use a keyboard, a switch, or their voice instead.
- have low vision and zoom right in, or are colour blind.
- have shaky hands, or cognitive differences like dyslexia or ADHD.

The rulebook most teams use is **WCAG**, the Web Content Accessibility Guidelines. The current version is **WCAG 2.2**. Each rule has a level: **A** (the basics), **AA** (the usual target) and **AAA** (the strictest). Most teams, and many laws, aim for **AA**.

There's nothing to install. This chapter uses React 19, plain HTML, a little CSS, and tools already in your browser. It's about using HTML and React *well*.

An honest note: accessibility is a big field, and one chapter can't cover all of it. This one covers what you'll use every week. And ten minutes with a real screen reader will teach you more than any checklist, including this one.

Why does it matter?

It's real people. The World Health Organization estimates that about **1 in 6** people worldwide live with a significant disability. If your app doesn't work for them, it doesn't work.

It's everyone, some of the time. Disability isn't always permanent. (This idea comes from Microsoft's inclusive design guides.)

Permanent	Temporary	Situational
One arm	A broken arm	Holding a baby
Blind	Blurry eyes after an eye test	Bright sunlight on your phone
Deaf	An ear infection	A noisy train, no headphones

It's often the law. Many countries require it, and their laws usually point at WCAG AA. For example, the European Accessibility Act has covered many online shops and banking services in the EU since June 2025. The details vary by country.

It makes things better for everyone. The little ramps where a pavement meets the road are called **curb cuts**. They were built for wheelchairs, and they help people with prams, suitcases and bikes too. Software is full of this **curb-cut effect**: captions help people in noisy cafés, and clear labels help everyone.

It's cheap now and expensive later. Most accessibility comes free with the right HTML element. A `<button>` costs nothing extra on day one. Replacing 200 `<div onClick>`s a year later costs weeks.

Real-world example

Think about a **public library that's been built well**.

A building everyone can use	Your app
A ramp beside the steps	A real <code><button></code> : works with a mouse, a keyboard and a screen reader
Doors that open by themselves, for anyone	Native elements (<code><button></code> , <code><label></code> , <code><dialog></code>) that do the hard work
A sign on every door	A label on every input, a name on every button
Braille on the lift buttons	<code>alt</code> text on images, <code>aria-label</code> on icon buttons
The lift says "Third floor"	Live regions that announce "Added to cart"

A building everyone can use	Your app
You step out of the lift facing the right way	Focus management: after an action, focus lands somewhere sensible

Nobody thinks of the ramp as "special". It's just part of a good building.

How it works

Semantic HTML first

Semantic HTML means using the element that says what a thing *is*. It's the biggest accessibility win there is, and it's free.

```
// ❌ Looks like a button. Isn't one.
<div className="btn" onClick={handleAdd}>Add to cart</div>

// ✅ Is one.
<button type="button" className="btn" onClick={handleAdd}>Add to cart</button>
```

A real <button>	A <div onClick>
You can reach it with Tab	Tab skips straight past it
Enter and Space press it	Only a mouse click works
A screen reader says "Add to cart, button"	It says "Add to cart", like any text
<code>disabled</code> works	There's no such thing

To fake all that, a div needs `role="button"`, `tabIndex={0}`, and an `onKeyDown` that handles Enter and Space. On every single one. People usually reach for a div because buttons come with browser styling, so reset the styling instead: `background: none; border: none; padding: 0; font: inherit; color: inherit;`.

Link or button? If it **goes somewhere** (the URL changes), it's a `<Link>` ([chapter 24](#)). If it **does something** on this page, it's a `<button>`. Screen reader users hear which one it is, and expect it to act that way.

This is also the **first rule of ARIA**. **ARIA** (Accessible Rich Internet Applications) is a set of attributes, `role` and the `aria-*` ones, that tell assistive technology what something is or what state it's in. The rule: **if a native HTML element can do the job, use it instead of ARIA**. ARIA only changes what's *announced*. It adds no behaviour. `role="button"` doesn't make a div work with Enter.

Headings, lists and landmarks

A sighted person scans a page. A screen reader user jumps around it using its structure. WebAIM's well-known survey of screen reader users keeps finding that jumping from heading to heading is the most popular way to find things. So:

- **One `<h1>` per page**, saying what the page is.
- **Heading levels in order**, like a book's chapters and sections. Don't jump from `<h2>` to `<h4>` because the `<h4>` "looked the right size". Pick the level for its meaning, and set the size in CSS.
- **Lists are `<ul>` or `<ol>`**. A screen reader says "list, 12 items" before you start.
- **Landmarks** mark the big regions, so people can jump straight to them:

```
<header className="site-header">
  <Link className="shop-name" to="/">Paper & Ink</Link>
  <nav aria-label="Main">{/* NavLinks */}</nav>
</header>
<main id="main-content">{/* the page */}</main>
<footer>© Paper & Ink</footer>
```

Many sites also add a **skip link** as the very first thing on the page: `<a href="#main-content">Skip to main content</a>`, hidden until it gets focus. Keyboard users can then jump past the nav instead of tabbing through it on every page.

Names for everything

Every input and every button needs a **name**: the words a screen reader says for it. Tests find things by the same name, which is why `getByRole("button", { name: "Delete" })` worked in [chapter 28](#).

Inputs get a `<label>`, as you've done since [chapter 09](#). But in a **reusable** component, a hard-coded `id="email"` breaks as soon as there are two on one page, because ids must be unique. React's `useId()` makes a unique one for each use:

```
import { useId } from "react";

type TextFieldProps = { label: string; value: string; onChange: (value: string) => void };

function TextField({ label, value, onChange }: TextFieldProps) {
  const id = useId(); // a different id every place TextField is used

  return (
    <div className="form-field">
      <label htmlFor={id}>{label}</label>
      <input id={id} value={value} onChange={(event) => onChange(event.target.value)} />
    </div>
  );
}
```

A placeholder is not a label. Here's the full picture [chapter 10](#) promised. A placeholder vanishes the moment someone types, so they have to remember what the box was for. It's usually pale grey and hard to read. And screen readers don't reliably announce it.

When there's no room for a visible label, keep a real one and hide it visually, with the `visually-hidden` class from your starter CSS:

```
/* On screen: gone. For screen readers: still there. */
.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  margin: -1px;
  padding: 0;
  overflow: hidden;
  clip-path: inset(50%); /* squashed to one pixel, then clipped to nothing */
  white-space: nowrap;
  border: 0;
}
```

Why not `display: none`? Because that hides it from screen readers too (there's a table on this below).

Icon-only buttons need `aria-label`. A screen reader reads `×` as something like "times" or "multiplication x", depending on the screen reader. Neither says what the button does, or to what:

```
// ❌ "times, button". Five of these in a cart all sound the same.
<button type="button" onClick={() => onRemove(item.bookId)}>X</button>

// ✅ "Remove Dune from cart, button"
<button type="button" aria-label={`Remove ${item.title} from cart`} onClick={() =>
onRemove(item.bookId)}>X</button>
```

Put the item's name in the label. Screen reader users can list every button on a page, and "Remove, Remove, Remove" is no help. You did this for chapter 10's [x](#) and [chapter 21](#)'s [♥](#).

Emoji are read aloud too: 📖 Paper & Ink becomes "books Paper & Ink". Hide decoration with `<span aria-hidden="true">📖</span>`. And when a button *has* visible words, its name should contain them. `<button aria-label="Close">Cancel</button>` confuses voice-control users, who say "click Cancel" and find nothing with that name.

Images need alt text that says what matters about the image, in its place on the page:

```
 { /* tells you
something new */}
<img className="book-cover" src={book.cover} alt="" /> { /* repeats the text next to it:
decorative */}
<Link to="/"></Link> { /* the only thing in a link
*/}

<img src={book.cover} /> { /* ❌ no alt: some screen readers read the file name instead */}
<img src={book.cover} alt="image" /> { /* ❌ says nothing: it already says "image" */}
```

`alt=""` isn't lazy. It's a deliberate "skip this". The bookstore's starter README explains why the covers use it: the title is right there, so a cover `alt` would read the title twice.

Keyboard access

Many people use only a keyboard. These are the keys they expect to work:

Key	What it does
Tab / Shift + Tab	Move to the next / previous interactive thing
Enter	Follow a link, press a button, submit a form
Space	Press a button, tick a checkbox
Arrow keys	Move within a group: radio buttons, a <code><select></code> , a menu
Escape	Close or cancel: a dialog, a menu

Real elements handle all of these. Your job is mostly not to break them.

Keep focus visible. The **focus ring** is the outline around whatever the keyboard is on. Without it, a keyboard user is like a mouse user with an invisible pointer.

```
button:focus { outline: none; } /* ❌ the ring is gone, and nothing replaces it */

/* ✅ A clear ring. :focus-visible shows it for keyboard users, not after mouse clicks. */
a:focus-visible,
button:focus-visible {
  outline: 3px solid var(--accent);
  outline-offset: 2px;
}
```

The browser decides when `:focus-visible` applies: roughly, yes when you Tab to something, and usually no when you click a button. Your starter stylesheets from chapters 10, 21 and 29 already have rules like this.

Keep the Tab order logical. Tab follows the order of the HTML, not the order on screen. If CSS moves things around (flexbox `order`, `row-reverse`), focus jumps about. Keep the HTML in reading order.

`tabIndex` controls whether something can take focus:

Value	Meaning	Use it for
0	"Put this in the normal Tab order"	Rarely: a custom widget with no native element
-1	"Not in the Tab order, but code can focus it"	Headings and regions you focus from code
1 or more	"Jump the queue"	Never. It scrambles the order for the whole page

Focus management in React

In React, things appear and disappear all the time. Each time, ask: **where is focus now, and where should it be?**

After deleting something. A keyboard user presses a row's delete button. The row disappears, and so does the button they were on. Focus falls back to `<body>`, the top of the page. They have to Tab all the way back, and a screen reader may say nothing at all. So move focus yourself, with a ref from [chapter 19](#):

```
function CartPage() {
  const { items } = useCart();
  const dispatch = useCartDispatch();
  const headingRef = useRef<HTMLHeadingElement>(null);

  function handleRemove(bookId: string) {
    dispatch({ type: "removed", bookId });
    headingRef.current?.focus(); // the heading is already on the page, so this is safe
  }

  return (
    <section className="page">
      <h1 ref={headingRef} tabIndex={-1} className="page-title">Your cart</h1>
      {items.length === 0 ? <p>Your cart is empty.</p> : <ul className="cart-list">{/* rows */}</ul>}
    </section>
  );
}
```

In chapter 19, focusing had to wait for an effect, because the input didn't exist yet. Here the heading already exists, so the handler can focus it straight away. `tabIndex={-1}` is what lets a heading take focus. Focusing the next row's button would be even kinder, but takes more code. (You'll also *announce* "Removed Dune" in a moment.)

After a route change. On a normal website, clicking a link loads a new page, and the screen reader announces it and starts at the top. A **single-page app** (one that swaps content without loading a new page, which is what React Router does) gives no such signal. Focus stays on the link you clicked, and a screen reader user may not know anything changed. A common fix: give every page's `<h1>` `tabIndex={-1}`, and let the layout focus it.

```
function Layout() {
  const { pathname } = useLocation();
  const mainRef = useRef<HTMLElement>(null);
  const previousPath = useRef(pathname);

  useEffect(() => {
    if (pathname === previousPath.current) return; // first load: leave focus alone
    previousPath.current = pathname;
    mainRef.current?.querySelector("h1")?.focus();
  }, [pathname]);

  return <main id="main-content" ref={mainRef}><Outlet /></main>; // plus your header and footer
}
```

That's chapter 19's "remember the previous value" ref, doing real work. Comparing paths (rather than using a "first render?" flag) also stops Strict Mode's double effects from stealing focus on the first load. Watching only `pathname` matters too: typing in the bookstore's search box changes `?q=`, and you don't want focus yanked out

of the box. If a page is lazy-loaded, its heading may not exist yet when this runs; then let the page focus its own heading when it mounts. A focus ring on a heading can look odd, and since a heading can't be clicked, hiding it is fine: `h1[tabindex="-1"]:focus { outline: none; }`.

Give each page its own title too. In React 19 you can render `<title>Your cart | Paper & Ink</title>` inside any component, and React moves it into the `<head>` for you. It's the first thing a screen reader reads on a new tab.

Dialogs, the easy way: `<dialog>`

[Chapter 21's exercise 4](#) asked you to make the recipe modal keyboard-friendly by hand: an Escape listener, focus in, focus back. Even with all that, Tab could wander onto the page *behind* the overlay, and a screen reader could still read it.

HTML now has an element that does all of this: `<dialog>`. Open it with `showModal()` and the browser:

- moves focus into the dialog,
- makes the rest of the page **inert** (it can't be clicked, tabbed to or read), so focus stays inside,
- closes it when someone presses **Escape**,
- in current browsers, puts focus back where it was (usually the opening button) when it closes,
- gives you a `::backdrop` to style as the dark background.

`showModal()` is a method on the real element, so you need a ref:

```
// src/components/Modal.tsx
import { useEffect, useId, useRef, type ReactNode } from "react";

type ModalProps = { title: string; onClose: () => void; children: ReactNode };

export function Modal({ title, onClose, children }: ModalProps) {
  const dialogRef = useRef<HTMLDialogElement>(null);
  const titleId = useId();

  // Open as a modal as soon as this component is on the page.
  useEffect(() => {
    const dialog = dialogRef.current;
    if (dialog && !dialog.open) {
      dialog.showModal();
    }
  }, []);

  return (
    <dialog ref={dialogRef} className="modal" aria-labelledby={titleId} onClose={onClose}>
      <button type="button" className="modal-close" aria-label="Close" onClick={() =>
dialogRef.current?.close()}>
        ×
      </button>
      <h2 id={titleId} className="modal-title">{title}</h2>
      {children}
    </dialog>
  );
}
```

Using it is chapter 21's conditional rendering: `{details && <Modal title={details.name} onClose={() => setSelectedId(null)}>...</Modal>}`.

How closing works: Escape (the browser) and the Close button (`close()`) both close the dialog. That fires its `close` event, which calls `onClose`, which removes the component. One path, every time. So always close *through the dialog*. If the parent just stops rendering an open dialog, the browser never closes it properly, and can't put focus back.

- **Don't add a cleanup that calls `close()`** . In development, Strict Mode runs effects twice ([chapter 17](#)), and that extra `close()` would fire `onClose` and shut the modal the moment it opened. The `!dialog.open` check makes the second run harmless.
- **Clicking the backdrop doesn't close it** by default. That's fine: Escape and a Close button are what matter. Newer browsers support a `closedby="any"` attribute for backdrop clicks; check support before relying on it.
- `aria-labelledby` names the dialog, so a screen reader says something like "Chicken Parmesan, dialog", and a test can use `getByRole("dialog", { name: "Chicken Parmesan" })`.

Announcing changes: live regions

A sighted user presses **Add to cart** and sees "Cart (3)". A screen reader user presses it and hears nothing. Focus didn't move, so the screen reader had no reason to speak.

A **live region** is an element the screen reader watches. When its text changes, it reads the new text out:

```
// Always on the page. Only the text inside changes.
<p role="status" className="visually-hidden">{announcement}</p>
```

- `role="status"` is **polite**: it waits until the screen reader finishes what it's saying. Use it for "Added Dune to your cart", "Saved", "3 results found". (`aria-live="polite"` is the plain-attribute version.)
- `role="alert"` is **assertive**: it interrupts. Keep it for urgent things, like "Payment failed".

Four rules make them work:

1. **The region must be on the page *before* the text changes.** `{message} && <p role="status">{message}</p>}` appears and fills at the same moment, and is often missed. Render the `<p>` always, and change its text. (`role="alert"` is usually announced even when it appears with its text, but always-there is safest.)
2. **The same text twice is no change**, so it isn't announced. Add something that differs: "Added Dune. 2 in your cart."
3. **It can be visible.** The bookstore's count helps everyone: `<p className="result-count" role="status">Showing {visible.length} of {books.length} books</p>`.
4. **Don't overdo it.** If a count changes on every keystroke and gets chatty, announce a debounced value (your `useDebounce` from [chapter 20](#)).

Hiding things: who can still find them?

[Chapter 05](#) warned that screen readers "may still announce" things you hide with CSS. Now you can be exact. It depends *how* you hide it:

How you hide it	Seen on screen?	Read by a screen reader?	Still in the HTML?
Not rendering it: <code>{show} && <p>...</p>}</code>	No	No	No
<code>display: none</code> , or the <code>hidden</code> attribute	No	No	Yes
<code>visibility: hidden</code>	No (leaves a gap)	No	Yes
<code>opacity: 0</code>	No, but it still takes space and can still be clicked and tabbed to	Yes	Yes

How you hide it	Seen on screen?	Read by a screen reader?	Still in the HTML?
Off screen, or visually-hidden	No	Yes	Yes
<code>aria-hidden="true"</code>	Yes	No	Yes

So `display: none` hides things from everyone, and `opacity: 0` hides them from almost no one. Chapter 05's advice stands: not rendering is the default. Use `visually-hidden` for "screen readers only", and `aria-hidden="true"` for "eyes only" (decoration, like that ). Never put `aria-hidden="true"` on a button or anything else that takes focus. Keyboard users can still Tab to it, and the screen reader then says nothing.

Accessible forms

Forms are where people most often get stuck. Put together what you have so far:

```
const emailId = useId();
const errorId = useId();

<div className="form-field">
  <label htmlFor={emailId}>Email</label>
  <input
    id={emailId}
    type="email"
    required
    value={email}
    onChange={(event) => setEmail(event.target.value)}
    aria-invalid={emailError !== null}
    aria-describedby={emailError ? errorId : undefined}
  />
  {emailError && (
    <p id={errorId} className="field-error">
      <span aria-hidden="true">⚠ </span>
      {emailError}
    </p>
  )}
</div>
```

Tab into this box and a screen reader says something like "Email, edit, required, invalid entry, Enter an email like name@example.com".

- **required** is announced. Show it visually too, with the word "required" or an explained asterisk. (It also turns on the browser's own pop-up checks. To show only your own messages, put `noValidate` on the `<form>`; the field is still announced as required.)

- `aria-invalid` says "this one is wrong". `aria-describedby` points at the error's id, so it's read after the label. It can list several ids, separated by spaces: a hint *and* an error.
- **The error is in words**, not just a red border. The `△` is decoration, so it's hidden.
- **On submit, move focus to the first field with an error.** [React Hook Form](#) does this for you by default. By hand, it's a ref and `.focus()`.
- **Don't disable the submit button until the form is valid.** A disabled button can't be focused, and doesn't say *why* it won't work.

ARIA states worth knowing

ARIA is at its best describing **state** that HTML has no attribute for:

Attribute	Put it on	A screen reader adds
<code>aria-expanded</code>	A button that shows or hides something	"collapsed" / "expanded"
<code>aria-pressed</code>	A toggle button (on/off)	"pressed" / "not pressed"
<code>aria-current="page"</code>	The nav link for the page you're on	"current page"

```
<button type="button" aria-expanded={isOpen} onClick={() => setIsOpen(!isOpen)}>Delivery and
returns</button>
<button type="button" aria-pressed={inStockOnly} onClick={() => setInStockOnly(!inStockOnly)}>In
stock only</button>
```

For a simple show/hide, HTML has one built in: `<details>` and `<summary>`, which the hints in these exercises use. The first rule of ARIA again.

A toggle shows its state in one of two ways. Either keep the name fixed and use `aria-pressed`, or change the name ("Add to favourites" / "Remove from favourites", as in chapter 21) with no `aria-pressed`. Not both: "Remove from favourites, pressed" is confusing.

`aria-current="page"` is added for you by React Router's `NavLink` when the link is active. Your `active` class from chapter 24 tells eyes; this tells ears.

`aria-label` or `aria-labelledby`? Both set a name. `aria-label` is a name you type, which nobody sees. `aria-labelledby` says "my name is the text of that element", by id, like the dialog above. Prefer visible text inside the element, then `aria-labelledby`, then `aria-label`.

Colour, size and motion

Contrast. WCAG AA asks for a **contrast ratio** of at least **4.5:1** for normal text, and 3:1 for large text (about 24px, or 19px bold) and for the edges of inputs and buttons. To check, inspect an element in DevTools and click the

colour square next to `color` in the Styles pane: the picker shows the ratio, with a tick or a cross. Try it on your own projects. The pale "muted" grey in the to-do app's stylesheet (`#7b8794`), on white, comes out at about 3.7:1: fine for large text, too faint for small text.

Never colour alone. Roughly 1 in 12 men can't easily tell red from green. Colour can *add* meaning, but mustn't be the only sign of it. A red border says nothing to them; `<p className="promo-message error">That code isn't valid. Try BOOKS10.</p>` says it to everyone. Underline links inside paragraphs for the same reason.

Size. WCAG 2.2 asks for click targets of at least **24 by 24 pixels** (or enough space around smaller ones). That tiny `×` matters to anyone with shaky hands, and to everyone on a bumpy bus.

Motion. Some people get dizzy or sick from animation, and turn on "reduce motion" in their operating system (on Windows: Settings, Accessibility, Visual effects, then turn off "Animation effects"). CSS can read that setting:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

That calms the skeleton-card pulse in your recipe finder and bookstore CSS. In JavaScript, `window.matchMedia("(prefers-reduced-motion: reduce)").matches` tells you the same, for example before chapter 19's `smoothScrollIntoView`. To fake the setting, open DevTools' **Rendering** panel (three-dots menu, More tools) and find "Emulate CSS media feature prefers-reduced-motion".

Testing accessibility

No single check finds everything. Use four kinds.

- 1. Unplug your mouse.** Put it out of reach and use every flow with the keyboard alone. Can I reach everything with Tab? Can I always see where focus is? Can I use it with Enter, Space, arrows and Escape? Can I get *out* of it again? After each action, does focus land somewhere sensible?
- 2. Use a screen reader.** On Windows, install **NVDA** (free, from nvaccess.org), or use the built-in **Narrator** (**Ctrl + Windows + Enter** turns it on and off). On a Mac, **VoiceOver** is built in (**Cmd + F5**). Some NVDA keys to start with (the "NVDA key" is **Insert**, or **Caps Lock** if you chose the laptop layout):

Key	What it does
Ctrl	Stop talking

Key	What it does
Insert + Down Arrow	Read everything from here
H / Shift + H	Next / previous heading
D	Next landmark
Insert + F7	A list of all the links, headings and landmarks
Insert + Q	Quit NVDA

It's normal to feel lost the first time: it talks fast, and it's a new way of moving around. NVDA's **Speech Viewer** (NVDA menu, then Tools) shows everything it says as text, which helps while you learn. Everyday screen reader users are experts, so your clumsy first try isn't their experience. The question is: *could* an expert do everything, and would it make sense?

3. Let tools find the obvious things.

- **DevTools' Accessibility pane.** In Chrome or Edge, inspect an element and open the **Accessibility** tab beside Styles. It shows the element's **role** and **name**, which is exactly what a screen reader gets.
- **Lighthouse.** In DevTools, open the **Lighthouse** tab, tick only **Accessibility**, and run it. You get a score and a list of problems.
- **axe DevTools**, a free browser extension from Deque. It adds a DevTools tab that scans the page and explains each problem.
- **Your linter.** Your projects use Oxlint ([chapter 01](#)), which has a built-in `jsx-a11y` plugin that checks your JSX. Add it to `plugins` in `.oxlintrc.json`, keeping the ones already there (setting `plugins` replaces the default list). For example, if yours lists `react`, `typescript` and `oxc`, it becomes:

```
{ "plugins": ["react", "typescript", "oxc", "jsx-a11y"] }
```

Then run `npm run lint`. Most of its rules are in Oxlint's "correctness" group, which is on by default, and they catch things like an `<img>` with no `alt` or a `<div onClick>`. Check it's working: leave out an `alt` on purpose, and you should get a complaint. (If you don't, add `"categories": { "correctness": "warn" }` to the file.) If a project uses ESLint instead, the same rules come from the `eslint-plugin-jsx-a11y` package: import it in `eslint.config.js` and add `jsxA11y.flatConfigs.recommended` to the config. At the time of writing, that package officially supports ESLint only up to version 9, so npm may complain on a newer ESLint.

An honest warning about tools. Automated checks catch only some problems; commonly quoted figures range from about a third to a half. A tool can see that an image has `alt` text. It can't tell whether the text is any

good, whether focus lands somewhere sensible, or whether the page makes sense read aloud. **A Lighthouse score of 100 doesn't mean your app is accessible.** It means the tools found nothing more to say.

4. Write tests the way a screen reader reads. [Chapter 28](#) promised this: **if a test can't find it by role and name, neither can a screen reader.** So `getByRole` tests check accessibility as they go:

```
test("the remove button says what it removes", async () => {
  const user = userEvent.setup();
  const onRemove = vi.fn();
  render(<CartRow item={{ bookId: "3", title: "Dune", price: 9.99, quantity: 1 }} onRemove=
{onRemove} />);

  await user.click(screen.getByRole("button", { name: "Remove Dune from cart" }));

  expect(onRemove).toHaveBeenCalled("3");
});
```

jest-dom has more: `expect(input).toHaveAccessibleDescription(/enter an email/i)`, `expect(screen.getByRole("status")).toHaveTextContent("Showing 3 of 12 books")`, and `expect(heading).toHaveFocus()` after a delete. `await user.tab()` presses Tab, so you can test where focus goes.

Common mistakes

- 1. `<div onClick>` instead of a button.** No Tab, no Enter, no Space, no "button". Use `<button>` and reset its styling in CSS.
- 2. A placeholder instead of a label.** It vanishes as you type, and screen readers don't reliably read it. Every input gets a `<label>`, even a `visually-hidden` one.
- 3. Icon buttons with no name.** `<button>X</button>` is "times, button". Add an `aria-label` that says what it does, and to what.
- 4. `outline: none` with nothing in its place.** Keyboard users lose track of where they are. Style a clear ring with `:focus-visible`.
- 5. Missing or useless `alt` text.** No `alt` can mean the file name gets read out, and `alt="image"` says nothing. Describe what matters, or use `alt=""` for decoration.
- 6. Picking heading levels for their size.** An `<h4>` straight after an `<h1>` breaks the outline people navigate by. Choose the level for its meaning; set the size in CSS.
- 7. Colour-only messages.** A red border says nothing to a colour-blind user or a screen reader user. Say it in words.

- 8. ARIA sprinkled everywhere.** Wrong ARIA is worse than none, because it makes the screen reader say things that aren't true. Native elements first, ARIA only for what HTML can't say.
- 9. Losing focus.** Something is deleted, a dialog closes, or the route changes, and focus drops to `<body>`. Decide where it should go, and move it there with a ref.
- 10. Announcing nothing.** "Added to cart", "Saved" and "No results" appear on screen and stay silent for a screen reader. Use a live region that's always on the page.
- 11. Drag and drop with no alternative.** Dragging needs a mouse and a steady hand, and WCAG 2.2 asks for a single-click alternative. That's why [chapter 37's task board](#) moves cards with buttons: it works with a keyboard.
- 12. Treating a clean score as "done".** Lighthouse and axe catch some problems, not most. Finish with the keyboard and a screen reader.

Quick recap

- **Accessibility (a11y)** means everyone can use your app, including people using screen readers, keyboards, zoom, or shaky hands. **WCAG 2.2 AA** is the usual target.
- **The right HTML element does most of the work.** `<button>` to do, `<Link>` to go, headings in order, landmarks. Use ARIA only when HTML has no answer.
- **Everything needs a name:** a `<label>` for every input (`useId()` in reusable components), an `aria-label` for icon buttons, and fitting `alt` text (or `alt=""`).
- **Everything must work with a keyboard**, with a visible focus ring and a logical Tab order.
- **Manage focus when things change:** after a delete, a dialog, or a route change. `<dialog>` with `showModal()` does most of the dialog work for you.
- **Announce changes** with a live region that's on the page before its text changes.
- **Don't rely on colour alone**, keep text contrast at 4.5:1, and respect `prefers-reduced-motion` .
- **Test four ways:** keyboard, screen reader, automated tools, and `getByRole` tests. Tools find some problems. People find the rest.

Next: try the [exercises](#), then move on to [39 Next.js and Server Components](#).

Exercises

How to do these:

- Most of these improve apps you've already built: the to-do app ([chapter 10](#)), the recipe finder ([chapter 21](#)), the bookstore ([chapter 29](#)) and the task board ([chapter 37](#)). Exercise 2 uses your `playground` , in `src/ch38/ex2/` .

- **Before you change a project, commit it** (or copy the folder), so you can compare before and after.
 - Install **NVDA** (free, from nvaccess.org), or use **Narrator** (**Ctrl + Windows + Enter**). Headphones help. Keep NVDA's Speech Viewer open while you're learning.
 - Turn on the `jsx-a11y` plugin in each project's `.oxlintrc.json`, as in the notes, and run `npm run lint`.
 - An exercise is done when it works in the browser **with the keyboard alone**, VS Code shows no red squiggles, `npx tsc -b` prints nothing, `npm run lint` shows no accessibility warnings you can't explain, and the Console has no unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Unplug your mouse

Your chapter 10 to-do app looks finished. Time to find out whether it is.

1. Put your mouse out of reach. Using only the keyboard, add three tasks, tick one off, delete one, try each filter, and use **Clear completed**. (Include dark mode and editing, if you did those stretch goals.)
2. Write down every problem in a comment at the top of `App.tsx`: what you did, what happened, and what should have happened. Aim for at least three. Pay special attention to anything that **disappears right after you press it**.
3. Run **Lighthouse** (Accessibility only) and add anything new it finds to your list.
4. Turn on NVDA or Narrator, and Tab to the filter buttons. Without looking at the screen, can you tell which filter is selected? If not, add that to the list.
5. Fix every problem on your list.
6. Do the whole keyboard walk again, from the start. Then write one sentence: which problem would you never have found with a mouse?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Fix the broken book card

Someone wrote this card in a hurry. It looks fine. It has **at least six** accessibility problems.

In `src/ch38/ex2/`, create `BookCard.tsx` with this code, and render it from `Ex2.tsx` under an `<h1>Books</h1>`:

```

import { useState } from "react";

type BookCardProps = {
  title: string;
  price: number;
  cover: string;
  isSaved: boolean;
  onAdd: () => void;
  onToggleSave: () => void;
};

export function BookCard({ title, price, cover, isSaved, onAdd, onToggleSave }: BookCardProps) {
  const [email, setEmail] = useState("");
  const [hasError, setHasError] = useState(false);

  return (
    <div className="card">
      <img src={cover} />
      <h4>{title}</h4>
      <p>£{price.toFixed(2)}</p>
      <div className="btn" onClick={onAdd}>Add to cart</div>
      <button onClick={onToggleSave}>{isSaved ? "♥" : "♡"}</button>

      <form
        onSubmit={(event) => {
          event.preventDefault();
          setHasError(!email.includes("@"));
        }}
      >
        <input
          placeholder="Email me when the price drops"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
          style={{ borderColor: hasError ? "red" : "#ccc" }}
        />
        <button>Notify me</button>
      </form>
    </div>
  );
}

```

1. **Before changing anything**, list every problem you can find, as comments. For each one, write who it hurts: a keyboard user, a screen reader user, a colour-blind user, or someone else.
2. Run `npm run lint` with `jsx-a11y` on. Which of your problems did it catch? Which did it miss? Write that down too.
3. In `BookCard.test.tsx`, write tests that **fail right now**, all using `getByRole` with a name: the Add to cart button, the save button (with a name that includes the book's title), the email box (found by its label), and

the title as a heading with `level: 2`. Add one more: after an invalid submit, the email box `toHaveAccessibleDescription` with your error text.

4. Fix the component until every test passes. Don't change the tests to make them pass.
5. Tab through the card, then listen to it with a screen reader. Does it make sense without seeing it?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): A real dialog for the recipe finder

Replace the recipe finder's hand-built modal with the native `<dialog>`.

1. Build the typed `Modal` from the notes in `src/components/Modal.tsx`: a `<dialog>`, `showModal()` through a `ref`, and a name from `aria-labelledby` with `useId()`.
2. Use it for the recipe details. Delete the `modal-overlay` `div`; the dialog's `::backdrop` replaces it. (While the details load, the dialog still needs a title. Try it with the Network tab throttled.)
3. Check each of these with the keyboard only:
 - Opening a card puts focus on the dialog's close button.
 - Tab never reaches the cards behind the dialog.
 - **Escape** closes it.
 - After it closes, focus is back on the card you opened, and Tab carries on to the next card.
4. Check it with a screen reader. It should announce the recipe's name and "dialog". Press **H** in NVDA while the dialog is open: can you reach any heading behind it?
5. If you did [chapter 21's exercise 4](#) by hand, delete the code that `<dialog>` now does for you. Write in a comment how many lines went.
6. Write one test: clicking a card opens a dialog named after the recipe (`findByRole("dialog", { name: ... })`), and clicking the close button removes it.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Speak up, bookstore

Make the bookstore tell screen reader users what sighted users can see.

1. When any **Add to cart** button is pressed, on any page, politely announce `Added Dune to your cart`. The announcing region lives in `Layout`, is always rendered, and is `visually-hidden`.
2. Add the same book twice. The second press must be announced too.
3. Make `Showing 3 of 12 books` a polite status. Type in the search box with a screen reader on. If it's too chatty, announce from a debounced value.

4. Removing a book on the cart page announces `Removed Dune from your cart`, and focus lands somewhere sensible.
5. Make the checkout form's errors accessible: a `<label>` on every field, `aria-invalid`, `aria-describedby` pointing at each error, errors in words (not only colour), and focus on the first bad field when a submit fails.
6. The promo code message (`Code applied` or `That code isn't valid`) is announced when it appears.
7. Test it all with NVDA or Narrator. Write down exactly what it said for steps 1, 3 and 5.
8. Add tests: after adding a book, the `status` role has the right text; after an invalid checkout submit, the email field `toHaveAccessibleDescription` with its error.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A full audit

Take your **task board** from [chapter 37](#), or the bookstore, and audit it the way a professional would.

1. **Keyboard walk:** every flow, no mouse.
2. **Screen reader walk:** jump through the headings with **H**, the landmarks with **D**, and open the elements list with **Insert + F7**. Does the outline make sense? Does every button's name say what it does?
3. **Tools:** run Lighthouse and axe DevTools on every page, and `npm run lint` with `jsx-a11y` on.
4. Record every problem in a table in a new `a11y-audit.md` in the project: the problem, who it affects, and **which check found it** (keyboard, screen reader, Lighthouse, axe, lint, or a test).
5. Fix the **five most harmful** problems. Rank by harm: "can't do the task at all" beats "confusing", which beats "annoying".
6. Move focus to each page's `<h1>` on route change, and give every page its own `<title>`.
7. On the task board, move a card to another column using only the keyboard. Where does focus go afterwards? It should stay with the card (on its button in the new column, for example), and the move should be announced, like `Moved "Write tests" to Done`.
8. Finish with a short paragraph: how many problems did the tools find, and how many only a person found? Does that match the warning in the notes?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

When you're done

That's the first chapter of Level 5. From now on, "works with a keyboard and a screen reader" is part of "done", just like "the tests pass". When [chapter 40](#) puts your apps online, run Lighthouse and axe again on the live sites. And make accessibility part of the plan for your [final project](#) from day one, when it's cheap.

39 Next.js and Server Components

What is it?

Next.js is a React **framework**. What's the difference between a library and a framework?

- A **library** is a tool you call. React, Zod and Axios are libraries. Your code is in charge.
- A **framework** is a structure you fill in. It decides where your files go, and *it* calls *your* code.

A library is a drill you pick up when you need a hole. A framework is the frame of a house: the rooms are already laid out, and you fill them in.

Next.js is React plus what bigger apps end up needing: routing, rendering on the server, loading data, and a build setup for putting the app online. It's also the best-known home of **Server Components**: React components that run on the server (or once, when you build the app), and never in the browser. Their code isn't sent to the browser at all. Only what they render is.

```
// A Server Component: it can be async, and await data directly
export default async function CategoriesPage() {
  const response = await fetch("https://www.themeadb.com/api/json/v1/1/categories.php");
  const data = await response.json();
  return <h1>{data.categories.length} categories</h1>;
}
```

No `useState` and no `useEffect`. By the time this component returns JSX, the data is already there.

This chapter uses **Next.js 16** (16.3 when this was written) with the **App Router**, and React 19. The App Router is the modern way to build Next.js pages, in an `app/` folder. (The older Pages Router uses a `pages/` folder instead. Many tutorials online still teach it: see mistake 10.) **Next.js changes fast**, faster than anything else in this course. If something here doesn't match what you see, trust the docs at nextjs.org/docs, which show the version on every page. You'll create an app in a moment with `npx create-next-app@latest next-practice`.

Why does it matter?

Every app in this course so far has been a **single-page app** (SPA). Vite sends the browser a nearly empty HTML page plus a lot of JavaScript, and the browser builds everything. Open your playground and press `Ctrl + U` (View Source):

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
```

That's all the browser gets at first, and it causes four problems:

- **The first load is blank.** Nothing shows until the JavaScript has downloaded and run. On a cheap phone with a slow connection, that's seconds of white screen. And anything that reads the page without running JavaScript sees nothing, including many link previews in chat apps. (Google does run JavaScript, but less reliably than it reads HTML.)
- **Data comes last.** The [chapter 18](#) order is a **waterfall**: a chain where each step waits for the one before.
- **Secrets can't live in the browser.** Everything in front-end code is public (JavaScript chapter 51). A paid API key in React code is a key anyone can copy.
- **Anything private needs a second project.** Saving to a database or using a secret key means building and running a separate backend.

Frameworks fix this by moving some of the work to the server:

Vite app (chapter 18):

1. Almost empty page -> blank screen
2. JavaScript runs -> still blank
3. Effect starts fetch -> "Loading..."
4. Data arrives -> the recipes

Next.js with a Server Component:

1. The server fetches the data and renders HTML
2. HTML arrives with the recipes in it
3. JavaScript for the buttons loads, and the buttons start working

The **React team now recommends** starting new apps with a framework (react.dev lists Next.js first). But a Vite SPA is still a great choice for many apps: a dashboard behind a login, an internal tool, a game, or learning. And nothing you've learned is wasted. Components, props, state, hooks, context and TypeScript work exactly the same inside Next.js.

Real-world example

Think about eating at a **restaurant**.

At the restaurant	In a React app
SPA: you're handed a box of ingredients and a recipe book, and you cook at your table	The browser downloads all the JavaScript and builds the page itself
Server rendering: the kitchen cooks, and a finished plate arrives	The server sends ready-made HTML

At the restaurant	In a React app
The kitchen's pantry and suppliers	Databases, files and APIs that only the server can reach
The secret sauce recipe never leaves the kitchen	Secrets and server code never reach the browser
The waiter at your table: refills, the bill, requests	Client Components: the interactive bits, like buttons and forms
Ordering dessert: the waiter takes it to the kitchen and brings it back	A Server Function: the browser asks, the server does the work

The big change is that there are now **two places** your code can run: the kitchen and the table. Most of this chapter is about which code belongs where.

How it works

Creating the app

Open a terminal in this `React` folder (next to `playground`), and run `npx create-next-app@latest next-practice`. You need Node.js 20.9 or newer. When it asks about the recommended defaults, choose **No**, **customize settings**, then answer:

Question	Your answer
TypeScript?	Yes
Which linter?	ESLint
React Compiler?	No , as in chapter 01 . (Chapter 27 explains what it does.)
Tailwind CSS?	Your choice. Yes if you liked chapter 16 . The examples barely style anything.
Code inside a <code>src/</code> directory?	Yes , like your Vite apps
App Router?	Yes
Customize the import alias?	No (keeps <code>@/*</code>)
Include AGENTS.md?	Your choice. It's a note telling AI assistants, like Claude, to use the docs for your installed version.

(The questions change between versions. If yours look different, pick TypeScript, ESLint, the App Router and `src/`, and the default for anything else.)

Then `cd next-practice` and `npm run dev`, and open `http://localhost:3000`. That's Next.js's port, and it's why the practice API from [chapter 30](#) uses 3001: the two never clash.

The files that matter:

- `src/app/` holds your pages. **Folders in here become URLs.**
- `src/app/layout.tsx` is the **root layout**: the `<html>` and `<body>` around every page.
- `src/app/page.tsx` is the home page, `/`.
- `next.config.ts` holds settings (empty for now). `.next/` is build output: never edit it.

Notice what's missing: no `index.html`, no `main.tsx`, no `createRoot`. Next.js owns those. You write pages, and the framework decides when to call them. (It even has its own **bundler**, Turbopack, doing the job Vite does in your other apps. It's the default, so there's nothing to set up.)

Two changes from the rest of the course. Next.js's TypeScript setup doesn't allow `.ts` or `.tsx` at the end of an import, so here you write `import { getMeal } from "@lib/mealsdb";`. `@/` is a shortcut for `src/`, so that's `src/lib/mealsdb.ts`. And type-check with `npx tsc --noEmit` instead of `npx tsc -b`. (`npm run build` checks types too.)

Folders are routes

In [chapter 24](#) you wrote a route table in code. In Next.js, **the folders are the route table**. A folder inside `app/` is one piece of the URL, and a `page.tsx` inside it makes that URL a real page. A page is just a component, exported as the **default export**.

File	URL
<code>src/app/page.tsx</code>	<code>/</code>
<code>src/app/about/page.tsx</code>	<code>/about</code>
<code>src/app/recipes/[id]/page.tsx</code>	<code>/recipes/52772</code> , <code>/recipes/anything</code>
<code>src/app/recipes/RecipeCard.tsx</code>	none: only files called <code>page.tsx</code> become pages

That last row matters. Other files in a folder are just ordinary files, so you can keep a page's components right next to it.

A **layout** wraps pages. Replace the generated root layout with this:

```
// src/app/layout.tsx
import type { Metadata } from "next";
import type { ReactNode } from "react";
import Link from "next/link";
import "./globals.css";

export const metadata: Metadata = { title: "Recipe Box", description: "Recipes from TheMealDB" };

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <body>
        <nav>
          <Link href="/">Home</Link> | <Link href="/about">About</Link>
        </nav>
        <main>{children}</main>
      </body>
    </html>
  );
}
```

- **children is the current page.** It does the job of `<Outlet />` in chapter 24.
- **The root layout is required,** and it must have `<html>` and `<body>`.
- **metadata** sets the `<title>` and description: the text search engines and link previews show.
- **Layouts stay put.** Moving between pages doesn't re-mount the layout, so state inside it survives, just like chapter 24's layout routes. Any folder can have its own `layout.tsx` too.
- **Link takes href, not to.** It swaps the page without a full reload. In a production build, it also **prefetches** (loads the next page in the background) when it scrolls into view, so clicks feel instant.

Dynamic routes and params

Square brackets make a **dynamic segment**, like `:id` in chapter 24. Its value arrives as `params`:

```
// src/app/recipes/[id]/page.tsx
type RecipePageProps = {
  params: Promise<{ id: string }>;
};

export default async function RecipePage({ params }: RecipePageProps) {
  const { id } = await params;
  return <h1>Recipe {id}</h1>;
}
```

- `params` is a **Promise**, so you `await` it. This changed in Next.js 15, and it's why the page is `async`. Older tutorials write `params.id` directly.
- `id` is a **plain string**, not `string | undefined` as with `useParams` in chapter 24. This file only renders for `/recipes/[id]`, so the value is always there.

Here's chapter 24, translated:

React Router (chapter 24)	Next.js App Router
<code><Route path="/books/:id" ...></code>	A folder: <code>app/books/[id]/page.tsx</code>
A layout with <code><Outlet /></code>	<code>layout.tsx</code> with <code>{children}</code>
<code><Link to="/books"></code>	<code><Link href="/books"></code> , from <code>next/link</code>
<code>useParams()</code> gives <code>string undefined</code>	<code>await params</code> gives <code>string</code>
<code>path="*" for a 404</code>	<code>not-found.tsx</code> (below)
<code>useNavigate()</code> , <code>useSearchParams()</code>	<code>useRouter()</code> , <code>useSearchParams()</code> , from <code>next/navigation</code>
<code>NavLink</code>	Nothing built in: compare <code>usePathname()</code> with the link's <code>href</code>

The hooks from `next/navigation` only work in Client Components, coming up soon. Don't import from `next/router`: that's the old Pages Router.

Server Components are the default

Every component in the `app` folder is a **Server Component** unless you say otherwise. It runs on the server, it can be `async`, and it can `await` data right in the component. First, a helper that keeps the API's details in one place (the "boundary" idea from [chapter 18](#)):

```
// src/lib/mealsdb.ts
const BASE = "https://www.themealdb.com/api/json/v1/1";

export type Category = { id: string; name: string };
type CategoriesResponse = { categories: Array<{ idCategory: string; strCategory: string }> };

export async function getCategories(): Promise<Category[]> {
  const response = await fetch(`${BASE}/categories.php`);
  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }
  const data: CategoriesResponse = await response.json();
  return data.categories.map((c) => ({ id: c.idCategory, name: c.strCategory }));
}
```


Then the page:

```
// src/app/categories/page.tsx
import { getCategories } from "@lib/mealsdb";

export default async function CategoriesPage() {
  const categories = await getCategories();
  console.log("Rendering categories. typeof window is:", typeof window);

  return (
    <ul>
      {categories.map((category) => (
        <li key={category.id}>{category.name}</li>
      ))}
    </ul>
  );
}
```

Put that next to chapter 18 and look at what's gone:

Chapter 18 (in the browser)	Server Component
Three <code>useState</code> s: data, loading, error	None. The data is just a variable.
A <code>useEffect</code> to start the fetch	None. You <code>await</code> it.
An <code>ignore</code> flag or <code>AbortController</code>	None. It runs once per request, so there's nothing to race.
The data is <code>null</code> at first, so every render checks	The data is there before any JSX runs
Loading and error UI inside every component	<code>loading.tsx</code> and <code>error.tsx</code> , once per route (below)

One thing hasn't changed: `data: CategoriesResponse` is still a promise, not a proof. [Zod](#) works on the server just as it does in the browser.

Proving where it ran

Visit `/categories` , and check three things:

- 1. **The terminal** running `npm run dev` prints `typeof window is: undefined` . There's no `window` on the server, because there's no browser there. (In development, the browser's Console shows a copy too, labelled **Server**. Next.js replays server logs there to help you debug. The label tells you where it really ran.)
- 2. **The Network tab**. Reload, and filter for `categories.php` . Nothing. The server made that request, not your browser.

3. **View Source** (Ctrl + U). The category names are right there in the HTML, instead of an empty `<div id="root">`.

What Server Components can't do

A Server Component runs on the server, sends its result, and is done. So it can't use anything that needs a browser, or needs to run again later: no **state or effects** (`useState`, `useEffect`, `useRef`, or custom hooks built on them), no **event handlers** (`onClick`, `onChange`), no **browser APIs** (`window`, `localStorage`), and no **context** ([chapter 22](#)).

Add `useState` to the categories page, and Next.js stops you with an error something like this (the exact words change between versions):

You're importing a component that needs `useState`. This React Hook only works in a Client Component. To fix, mark the file (or its parent) with the `"use client"` directive.

An `onClick` in a Server Component gives something like `Event handlers cannot be passed to Client Component props`. Both point the same way: interactive code needs a Client Component.

Client Components with `"use client"`

Put `"use client"`; at the very top of a file, above the imports, and its components become **Client Components**. They work like every component you've written so far:

```
// src/app/recipes/[id]/FavouriteButton.tsx
"use client";

import { useState } from "react";

export function FavouriteButton({ recipeName }: { recipeName: string }) {
  const [isFavourite, setIsFavourite] = useState(false);

  return (
    <button type="button" aria-pressed={isFavourite} onClick={() =>
      setIsFavourite(!isFavourite)}>
      <span aria-hidden="true">{isFavourite ? "♥" : "♡"}</span> Favourite {recipeName}
    </button>
  );
}
```

A Server Component page renders it like any other component: `<FavouriteButton recipeName={meal.name} />`. You'll see the whole recipe page in a moment. Three things to understand about `"use client"`:

It marks a boundary, not one component. The file with `"use client"`, and every file *it* imports, becomes part of the JavaScript sent to the browser. So you don't add it to every file, only to the "front door" of each interactive part.

"Client" doesn't mean "only in the browser". This confuses almost everyone. On the first visit, Next.js *also* runs Client Components on the server, to make HTML, so the button is in the page straight away. Then the browser downloads their JavaScript, and React **hydrates** them: it attaches state and event handlers to the HTML that's already there, like switching the electricity on in a house that's already built. So a Client Component runs in both places. That's why you can't read `localStorage` during render, even in a Client Component: on the server, it doesn't exist. Read it in an effect or an event handler, which only run in the browser.

Keep them small. Think of a page as server-rendered, with small **islands** of interactivity. Only the islands' code goes to the browser. Put `"use client"` on the page itself, and the whole page (and everything it imports) ships to the browser, and it can't be `async` any more.

Passing things across the boundary

Props from a Server Component to a Client Component travel over the network, so they must be **serialisable**: possible to turn into text and back, like `JSON.stringify` in JavaScript chapter 23.

Fine as props	Not fine as props
Strings, numbers, booleans, <code>null</code>	Functions, like <code>onToggle={() => ...}</code>
Plain objects and arrays of those	Class instances
JSX, including Server Components	Server-only things, like an open database connection

(React can also send a few extras, like `Date` and promises, and Server Functions are the one kind of function you *can* pass.) Pass an ordinary function, and you'll get an error something like `Functions cannot be passed directly to Client Components`.

Server Components can go *inside* Client Components, as `children`. Say `Collapsible` is a client island with an open/closed state, and `CategoryList` is an `async` Server Component:

```
// In a Server Component page:
<Collapsible title="Categories">
  <CategoryList />
</Collapsible>
```

The server renders `<CategoryList />` first, and hands the *finished result* to `Collapsible` as `children`, so it stays a Server Component. What doesn't work is *importing* `CategoryList` inside `Collapsible.tsx`, because

everything a client file imports becomes client code. Context providers work the same way: a provider is a Client Component, usually a small `Providers` wrapper around `{children}` in the root layout.

The mental model



	Runs on the server	Runs in the browser	Its code is sent to the browser
Server Component	✓	✗	✗
Client Component	✓ (for the first HTML)	✓	✓
Server Function	✓	✗ (the browser only calls it)	✗

Loading states with `loading.tsx`

A page that awaits slow data makes the visitor wait. Add a `loading.tsx` next to the page:

```
// src/app/recipes/[id]/loading.tsx
export default function Loading() {
  return <p>Loading recipe...</p>;
}
```

Now, when someone clicks through to a recipe, the layout stays on screen and this shows straight away, until the page is ready.

That's chapter 26's promise kept: Next.js wraps your page in `<Suspense fallback={<Loading />}>` for you. In [chapter 26](#), `Suspense` did nothing for `useEffect` fetching, because it only works with things that opt in. `async` Server Components opt in. So you can also put `<Suspense>` around one slow part of a page, like `<Suspense fallback={<p>Loading...</p>}><SimilarRecipes /></Suspense>`. The rest of the page shows at once, and the slow part arrives when it's ready. That's called **streaming**: the server sends the page in pieces as they're finished.

Errors with `error.tsx`

Chapter 26 said errors during server rendering use a "different mechanism". This is it. An `error.tsx` file is an error boundary for its folder:

```
// src/app/recipes/[id]/error.tsx
"use client"; // error boundaries must be Client Components

type RecipeErrorProps = {
  error: Error & { digest?: string };
  retry: () => void;
};

export default function RecipeError({ error, retry }: RecipeErrorProps) {
  return (
    <div role="alert">
      <p>Couldn't load this recipe. {error.message}</p>
      <button type="button" onClick={() => retry()}>Try again</button>
    </div>
  );
}
```

- **It must be a Client Component.** Error boundaries are built on state (remember chapter 26's class).
- **It catches errors in that folder's page and below**, while the layout above keeps working: chapter 26's "one fuse per room". (It doesn't cover the `layout.tsx` in the same folder. The root layout has a special `global-error.tsx`.)
- `retry()` fetches and renders the page again. It became official in Next.js 16.3. Older versions and most tutorials use `reset()`, which re-renders without fetching again. Both exist today.
- **In production, the message is hidden.** An error thrown in a Server Component reaches the browser as a generic message plus a `digest`: an id you can match with your server's logs. The real message might contain private details, so it stays on the server.

"Not found" with `notFound()`

An unknown recipe id isn't a crash. It's a missing page, and it deserves a proper 404. TheMealDB answers an unknown id with `{ meals: null }`, so say `getMeal(id)` in `src/lib/mealsdb.ts` returns `Meal | null` (you'll write it in exercise 3):

```
// src/app/recipes/[id]/page.tsx
import { notFound } from "next/navigation";
import { getMeal } from "@/lib/mealsdb";
import { FavouriteButton } from "../FavouriteButton";

export default async function RecipePage({ params }: RecipePageProps) {
  const { id } = await params;
  const meal = await getMeal(id); // Meal | null

  if (!meal) {
    notFound(); // stops here, and shows not-found.tsx instead
  }

  return (
    <article>
      <h1>{meal.name}</h1>
      <FavouriteButton recipeName={meal.name} />
      <p>{meal.instructions}</p>
    </article>
  );
}
```

`notFound()` comes from `next/navigation`, and it never returns (its return type is `never`, as in [chapter 23](#)). So TypeScript knows `meal` isn't `null` after the `if`, with no `!`. Next to the page, a `not-found.tsx` is a plain component: a heading and a `Link` back to the categories. And a `not-found.tsx` at the top of `app/` also catches every URL that matches no page, like chapter 24's `path=""`.

One route folder can now hold all of this:

```
src/app/recipes/[id]/
├─ page.tsx           the page (a Server Component)
├─ loading.tsx        shown while page.tsx is waiting
├─ error.tsx          shown if something throws ("use client")
├─ not-found.tsx      shown when you call notFound()
└─ FavouriteButton.tsx an ordinary component, no special meaning
```

Changing data: Server Functions

Reading data is half the story. What about saving? In an SPA, saving needs an API on a server somewhere, like `json-server` in [chapter 30](#), plus a request to reach it. Next.js has a shortcut: write an `async` function marked `"use server"`, and call it from a form.

That's a **Server Function**. Put `"use server";` at the top of a file, and every function it exports runs on the server. When the browser calls one, Next.js sends a `POST` request behind the scenes, and you never write a URL.

When a Server Function handles a form or changes data, the docs also call it a **Server Action**. Same thing, different job title.

Here's a small guestbook, starting with the storage. It reads and writes a JSON file with `node:fs/promises` from JavaScript chapter 49. This code could never run in a browser, and that's the point:

```
// src/lib/guestbook.ts
import "server-only";
import { readFile, writeFile } from "node:fs/promises";
import path from "node:path";

export type Entry = { id: string; name: string; message: string };
// process.cwd() is the project folder, so this is next-practice/data/guestbook.json
const FILE = path.join(process.cwd(), "data", "guestbook.json");

export async function getEntries(): Promise<Entry[]> {
  return JSON.parse(await readFile(FILE, "utf8")) as Entry[];
}

export async function addEntry(entry: Entry): Promise<void> {
  const entries = await getEntries();
  await writeFile(FILE, JSON.stringify([...entries, entry], null, 2));
}
```

Create `data/guestbook.json` (next to `package.json`) containing `[]` first. `import "server-only"` is a guard: if a Client Component ever imports this file, the build fails with a clear error. Next.js understands it without installing anything. (If your editor complains anyway, `npm install server-only` quiets it.)

Next, the Server Function. It checks its input with Zod ([chapter 32](#)), because **the browser can't be trusted**. The form's `required` attributes are nice, but anyone can send a POST with whatever they like. That's rule 1 of JavaScript chapter 51: never trust user input.

```
// src/app/guestbook/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { z } from "zod";
import { addEntry } from "@lib/guestbook";

const EntrySchema = z.object({
  name: z.string().trim().min(1, "Please tell us your name"),
  message: z.string().trim().min(1, "Please write a message").max(200, "Keep it under 200 characters"),
});

export type SignState = {
  errors?: { name?: string[]; message?: string[] };
  success?: boolean;
};

export async function signGuestbook(prevState: SignState, formData: FormData):
  Promise<SignState> {
  const result = EntrySchema.safeParse({
    name: formData.get("name"),
    message: formData.get("message"),
  });

  if (!result.success) {
    return { errors: z.flattenError(result.error).fieldErrors };
  }

  await addEntry({ id: crypto.randomUUID(), ...result.data });
  revalidatePath("/guestbook");
  return { success: true };
}
```

- `formData.get("name")` reads the input with `name="name"`, the `FormData` from [chapter 09](#). Its type is `FormDataEntryValue | null`, which is exactly why Zod checks it.
- **Problems you expect, like a blank field, are returned, not thrown.** `error.tsx` is for real crashes. `z.flattenError` turns Zod's error into one list of messages per field, like `{ name: ["Please tell us your name"] }`.
- `revalidatePath("/guestbook")` tells Next.js "the data on this page changed, so render it fresh". The fresh page comes back in the same response. Leave it out, and the file changes but the list on screen doesn't.

The form uses a hook, so it's a Client Component:


```
// src/app/guestbook/SignForm.tsx
"use client";

import { useActionState } from "react";
import { signGuestbook, type SignState } from "../actions";
import { SubmitButton } from "../SubmitButton";

const initialState: SignState = {};

export function SignForm() {
  const [state, formAction] = useActionState(signGuestbook, initialState);

  return (
    <form action={formAction}>
      <label htmlFor="name">Name</label>
      <input id="name" name="name" required />
      {state.errors?.name && <p>{state.errors.name[0]}</p>}
      <label htmlFor="message">Message</label>
      <textarea id="message" name="message" required />
      {state.errors?.message && <p>{state.errors.message[0]}</p>}
      <SubmitButton />
      {state.success && <p aria-live="polite">Thanks for signing!</p>}
    </form>
  );
}
```

This is [chapter 09](#)'s promise, kept:

- `action={formAction}` **replaces** `onSubmit`. In React 19, a form's `action` can be a function. React collects the `FormData` and calls it for you: no `preventDefault`, and no `useState` for each field.
- `useActionState(fn, initialState)`, from `react`, gives you `[state, formAction, isPending]`. `state` is whatever your function returned last time, here the errors. That's why the Server Function takes `prevState` first and the `FormData` second.

The pending button uses `useFormStatus`, from `react-dom`. It reports on the `<form>` it's *inside*, so it has to be its own component, rendered within the form. (`isPending` from `useActionState` does the same job, if you prefer.)

```
// src/app/guestbook/SubmitButton.tsx
"use client";

import { useFormStatus } from "react-dom";

export function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}>
      {pending ? "Signing..." : "Sign the guestbook"}
    </button>
  );
}
```

Finally, the page stays a Server Component. It reads the file directly, and drops the form in as an island:

```
// src/app/guestbook/page.tsx
import { getEntries } from "@/lib/guestbook";
import { SignForm } from "../SignForm";

export default async function GuestbookPage() {
  const entries = await getEntries();
  return (
    <>
      <h1>Guestbook</h1>
      <SignForm />
      <ul>
        {entries.map((entry) => <li key={entry.id}>{entry.name}: {entry.message}</li>)}
      </ul>
    </>
  );
}
```

No API endpoint, no fetch, no loading state, and no JSON parsing in the browser. The form posts, the server saves and re-renders, and the page updates.

A Server Function is a public door. Next.js turns it into a real endpoint that anyone can call with a POST request, not only your form. Hiding the form from logged-out visitors doesn't protect it. Inside every Server Function: validate the input, check the person is allowed to do this (once your app has logins), and return only what the page needs.

Secrets and environment variables

Server code finally gives secrets a safe home. Put them in a `.env.local` file in the project folder (next to `package.json`, not in `src/`):

```
WEATHER_API_KEY=abc123-very-secret
NEXT_PUBLIC_SITE_NAME=Recipe Box
```

Next.js loads these into `process.env` for you. There are two kinds:

- **Normal variables**, like `WEATHER_API_KEY`, exist only on the server. Read `process.env.WEATHER_API_KEY` in a Server Component or Server Function, and it never reaches the browser. In a Client Component, it's simply empty.
- **NEXT_PUBLIC_ variables** are copied into the browser's JavaScript when you build, so anyone can read them. They're for things like a site name. **Never put a secret in one.** The prefix literally means "make this public".

`create-next-app` already lists `.env*` files in `.gitignore`, so they stay out of git. For any file that must never reach the browser, add `import "server-only";` at the top.

Caching, honestly

Next.js works hard not to do the same work twice, and **how it does that has changed a lot between versions**. The short version for Next.js 16, as its docs describe it:

- **fetch results aren't cached by default.** (In Next.js 13 and 14 they were, so older tutorials say the opposite.)
- **But a page may be prerendered.** If a page has no dynamic `[segment]` and reads nothing from the request (like cookies or `searchParams`), `npm run build` may render it once, ahead of time. Everyone then gets that saved copy until something like `revalidatePath` refreshes it.
- **In development, pages are always rendered fresh**, so stale data usually shows up only after you build.
- **There's also a newer, opt-in model called Cache Components** (`cacheComponents: true` in `next.config.ts`, with a `"use cache"` directive), where you mark exactly what to cache. This chapter doesn't use it.

If your data ever looks stale, read the caching page of the docs for your version before anything else.

How the Level 4 libraries fit in

Most of Level 4 still works in a Next.js app. What changes is *where* each one runs.

Library	In a Next.js app
Zod	Everywhere. In Server Functions, it's your real line of defence.
React Hook Form	In Client Components, for big forms. Its submit handler can call a Server Function.
Zustand , Redux Toolkit	Shared client state, in Client Components only
TanStack Query	In Client Components, for data that changes while you look at it: polling, infinite scroll, live updates
Axios	Works on both sides, though most Next.js code uses <code>fetch</code>
React Router , TanStack Router	Not used. The <code>app</code> folder is the router.
Vitest and Testing Library	Still great for Client Components and plain functions. Vitest can't render <code>async</code> Server Components yet, so those are usually tested end to end , with a tool that drives a real browser.

The biggest shift is server state. Much of what you'd fetch with TanStack Query in a Vite app, like "load the books for this page", becomes an `async` Server Component with no library at all.

Other frameworks

Next.js isn't the only React framework. **React Router** has a **framework mode** (it used to be a separate framework called Remix), which [chapter 24](#) mentioned. It adds server rendering, **loaders** (functions that fetch a page's data on the server before it renders) and **actions** (functions that handle form posts). **TanStack Start** does similar things on top of [TanStack Router](#), and **Expo** is a framework for phone apps. They name things differently, but this chapter's big ideas carry over: render on the server, keep secrets and data access on the server, and send the browser only the interactive parts.

When to choose what

A Vite SPA fits when...	Next.js fits when...
The app lives behind a login (a dashboard, an admin tool)	The content is public, and should show up in search engines and link previews
It's a tool or game that mostly works in the browser	A fast first load matters, even on slow phones
It should work offline	You need server-only code: secrets, a database, files

A Vite SPA fits when...	Next.js fits when...
You want the simplest hosting: static files, anywhere	You'd rather not build a separate backend for small jobs
You're learning, or trying out an idea	You want routing, loading and error screens decided for you

Next.js has costs too. It needs a host that can run a server ([chapter 40](#) looks at hosting), there's more to learn, and you always have to think about which side your code runs on. Neither choice is "the grown-up one". Pick the one that fits the app.

Common mistakes

1. Hooks or `onClick` in a Server Component

A `useState` or an `onClick` in a page without `"use client"` is an error. Move the interactive part into a small `"use client"` component, and render that from the page.

2. `"use client"` on every file

It works, and it quietly turns your app back into an SPA with extra steps: everything ships to the browser, and nothing fetches on the server. Add it only where you need state, effects or event handlers, as far down the tree as you can.

3. Thinking Client Components only run in the browser

`localStorage.getItem("name")` in a Client Component's render fails on the server with `localStorage is not defined`. Client Components are also rendered on the server for the first HTML. Read browser-only things in `useEffect` or an event handler.

4. Passing a function from a Server Component to a Client Component

`<FavouriteButton onToggle={() => console.log("hi")} />` in a Server Component fails, because props have to cross the network. Pass data, and keep the function inside the Client Component.

5. Importing server-only code into a Client Component

A Client Component that imports your file-reading code tries to send it to the browser. At best it breaks, because `fs` doesn't exist there. At worst, private logic ends up in public JavaScript. Put `import "server-only";` at the top of those files, so the build catches it.

6. Forgetting to `await params`

`<h1>{params.id}</h1>` gives `Property 'id' does not exist on type 'Promise<{ id: string; }>'`. Since Next.js 15, `params` is a Promise. Make the page `async`, and `await params`.

7. A secret in a `NEXT_PUBLIC_` variable

`NEXT_PUBLIC_PAYMENT_SECRET` is a secret published to every visitor. Secrets get no prefix, and are only read on the server.

8. Trusting what the form sends

Reading `formData.get("name")` and saving it straight away, with no checks. A Server Function is a public endpoint. Validate with Zod on the server every time, even if the browser checked already.

9. "My change doesn't show up" or "my data is stale"

Two usual causes. After a Server Function changes data, you forgot `revalidatePath`, so the page on screen isn't re-rendered. Or it works in `npm run dev` but not after `npm run build`, because the page was prerendered. Check the caching docs for your version.

10. Following an old tutorial

A `pages/` folder, `getServerSideProps`, `getStaticProps`, or `import { useRouter } from "next/router"` all mean the older **Pages Router**. It still works, but in the `app` folder, hooks come from `next/navigation`, and data loads in `async` Server Components. Check the tutorial's version too: before 15 it won't `await params`, and before 13 there are no Server Components at all.

Quick recap

- **Next.js is a React framework:** you fill in its structure, and it calls your code. Folders in `app/` are routes, `page.tsx` makes a page, and `layout.tsx` wraps pages and stays mounted.
- **Components are Server Components by default.** They run on the server, can be `async` and `await` data directly, and send HTML instead of code. No effects, no loading state juggling.
- **"use client" marks a boundary** for small interactive islands. Client Components are still rendered on the server first, then hydrated in the browser. Props across the boundary must be serialisable.
- `loading.tsx`, `error.tsx` and `not-found.tsx` give each route its Suspense fallback, error boundary and 404. And `params` is a Promise: `await` it.
- **Server Functions** (`"use server"`) handle changes: a form's `action`, `useActionState` for the result, `useFormStatus` for a pending button, and `revalidatePath` to refresh the page. They're public endpoints, so validate with Zod every time.
- **Secrets stay on the server** in `process.env`. Only `NEXT_PUBLIC_` variables reach the browser.
- **Caching has changed between versions.** When data looks stale, check the docs for your version.
- **A Vite SPA is still a great choice** for many apps, and everything from chapters 01 to 38 works in both.

Next: try the [exercises](#), then move on to [40 Deploying](#).

Exercises

How to do these:

- Work in `React/next-practice`, the Next.js app you made in the notes. These exercises build one small recipe site, so each one adds pages to the same app. In Next.js, folders are routes, so there's no `src/ch39/ex1/` this time. You'll make folders like `src/app/categories/` instead.
 - Keep `npm run dev` running, and keep its terminal where you can see it. Server logs and server errors show up there now, as well as in the browser's Console.
 - Next.js shows errors in an overlay on top of the page while you develop. Read it, then close or minimise it to see what a visitor would see.
 - An exercise is done when it works in the browser, VS Code shows no red squiggles, `npx tsc --noEmit` prints nothing, and neither the terminal nor the Console shows unexpected errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Pages, a layout, and a dynamic route

Give the recipe site its skeleton.

1. Replace `src/app/page.tsx` with a short home page: a heading and one sentence. Add an `/about` page.
2. Replace the root layout with a header holding the site name and a nav: **Home**, **About**, and a link to one recipe, `/recipes/52772`. Use `Link` from `next/link`.
3. Make `src/app/recipes/[id]/page.tsx` show `Recipe id: 52772` (or whatever id is in the URL). Type `params` yourself, and `await` it.
4. Prove the layout doesn't re-mount. Make a small `VisitCounter` component (a button that counts its clicks) and put it in the header. Click it a few times, then move between the pages. The number should survive.
5. Change one nav `Link` into a plain `<a href>`. Click it, and watch the counter. Write down what happened and why, then change it back.
6. Break it on purpose: remove the `await`, and use `params.id` directly. Write down the exact error VS Code shows. Then put it back, and visit `/recipes` with no id. What do you get, and why?
7. If you have time: make the nav highlight the current page, like chapter 24's `NavLink`.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Fetch on the server, and prove it

Show TheMealDB's recipe categories, with no `useEffect` in sight.

1. In `src/lib/mealsdb.ts`, write `getCategories()` for `https://www.themealdb.com/api/json/v1/1/categories.php` (or extend the one from the notes). Return your own `Category` type with an id, name, thumbnail and description. Check `response.ok`.
2. Make `/categories` a Server Component that awaits it, and shows each category's name and thumbnail. Add it to the nav. (A plain `<img>` is fine. ESLint may suggest Next.js's own `<Image>` component instead. That's a warning, not an error.)
3. Add a `console.log` that includes `typeof window`. Find every place its output appears. Write down how you can tell which one is the real one.
4. In the Network tab, reload and filter for `categories.php`. Then filter for `themealdb`. Why does the second filter find requests when the first finds none? Answer in a comment.
5. View Source (Ctrl + U) and search for "Seafood".
6. Find your `CategoryList` from chapter 18's exercise 4 in the playground. Count its lines, count your new page's lines, and write both numbers in a comment.
7. Break it: put a typo in the URL. What do you see in the browser, and what's in the terminal? (There's no `error.tsx` yet. That's exercise 4.)

Stuck? Hint 1 is in the [Hints section](#) at the end.

Exercise 3 (Medium): A recipe page with a small client island

Turn `/recipes/[id]` into a real recipe page, with just one interactive part.

1. Add `getMeal(id)` to `src/lib/mealsdb.ts`, using `lookup.php?i=<id>`. Return your own `Meal` type (id, name, category, area, instructions, thumbnail), or `null` when TheMealDB answers `{ meals: null }`.
2. Make the page a Server Component that shows the recipe. For now, if `getMeal` returns `null`, show a short message. (Exercise 4 improves this.)
3. Add a `FavouriteButton` Client Component that toggles on and off, using `aria-pressed`. Pass it only the data it needs, as plain strings.
4. Make favourites survive a refresh with `localStorage`. You can reuse your `useLocalStorage` hook from [chapter 20](#). If the terminal says `localStorage is not defined`, work out *why* before you fix it. Watch the Console for hydration warnings, too.
5. View Source. Is the button's HTML already in the page, before any JavaScript runs? Write down what that tells you about Client Components.
6. Break it on purpose, one at a time, and write down each error: (a) move the `useState` into the page itself; (b) add `"use client"` to the page file, keeping its `async` and `await`; (c) add an `onToggle` prop to the button's props type, and pass `onToggle={() => console.log("hi")}` from the page.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Loading, errors, and not found

Make the recipe page behave well when things are slow, broken, or missing.

1. Add a `loading.tsx` to `recipes/[id]`. To actually see it, slow `getMeal` down with `await new Promise((resolve) => setTimeout(resolve, 2000));`. Click through from the home page with a `Link`. What stays on screen while it loads? Remove the delay afterwards.
2. Replace exercise 3's "not found" message with `notFound()`, and add a `not-found.tsx` that links back to `/categories`. Try `/recipes/999999`, and check the header and nav are still there.
3. Add a root `src/app/not-found.tsx` too, and try `/nonsense`. Which of your two not-found pages shows, and why?
4. Add an `error.tsx` with a **Try again** button. To test it, make the page throw an error when the id is `crash`, and visit `/recipes/crash`.
5. Compare development with production. Stop the dev server, run `npm run build`, then `npm run start`, and visit `/recipes/crash` again. Write down the error message you see in each, and why they're different.
6. Remove the `if (!meal)` check for a moment. What does TypeScript say about `meal.name`? Put the check back, and notice that after `notFound()` you need no `!`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A reading list with Server Functions

The reading list from Level 4, the Next.js way: no json-server, no Axios, no TanStack Query.

1. Create `data/reading-list.json` (next to `package.json`) with three books copied from the playground's `db.json`, keeping the same fields: `id`, `title`, `author`, `pages`, `status`.
2. In `src/lib/reading-list.ts`, starting with `import "server-only";`, write `getBooks()` and `saveBooks(books)` using `node:fs/promises`.
3. A `/reading-list` page (a Server Component) lists the books with their status. Add it to the nav.
4. An **Add a book** form with title, author, pages, and a status `<select>`. It calls an `addBook` Server Function in `src/app/reading-list/actions.ts`.
5. `addBook` validates with a Zod schema like chapter 32's `NewBookSchema`: title and author required, pages a positive whole number, and status one of `"want"`, `"reading"` or `"finished"`. Use `useActionState` to show each field's error under that field.
6. When validation fails, the visitor's typing mustn't disappear. Try it first: you'll probably find it does.
7. A `SubmitButton` using `useFormStatus` that says **Adding...** while the form is pending. Add a one-second delay inside `addBook` so you can see it.
8. Call `revalidatePath` after saving. Then comment it out, add a book, and write down what happens. Put it back.

9. Give each book a **Remove** button that calls a second Server Function, `removeBook` . It must check that the id it's given really exists.
10. Try it in production: `npm run build` , then `npm run start` . Add a book. Then comment out `revalidatePath` again, rebuild, add another book, and reload the page. Explain what you see, using the caching section of the notes.
11. Answer in a comment at the top of `page.tsx` :
 - Anyone can call `removeBook` , not only your form. What stops a stranger emptying your list? What would you need to add?
 - What would it take to build this same feature in the Vite playground? List the pieces.

Stuck? Hint 1, Hint 2, Hint 3, Hint 4 and Hint 5 are in the [Hints section](#) at the end.

40 Deploying

What is it?

Deploying means putting your app on a computer that's always switched on and connected to the internet, so anyone can open it with a link. That computer is called a **host**.

For a Vite app, deploying has two parts:

1. **Build** it into plain HTML, CSS and JavaScript files, with `npm run build`.
2. **Upload** those files to a host, which gives you a web address like `https://my-bookstore.netlify.app`.

A Next.js app from [chapter 39](#) is a little different. Some of its code runs on a server, so it needs a host that can run code, not just hand out files.

No new library this time. This chapter was written with **Vite 8** (the tool that made your apps in [chapter 01](#)) and **Next.js 16**. One honest warning up front: hosting companies change their names, their free plans and their dashboards often. So this chapter explains *what* you're doing and *why*. If a button has moved or a screen looks different, look for the same idea under a new name.

Why does it matter?

An app that only runs on `localhost` can only be used on your computer, by you. You can't send it to a friend, try it on your phone, or put a link in a job application.

Deploying also shows you bugs that `npm run dev` hides:

- **Refreshing a page gives a "404 Not Found"**. Clicking to `/books/3` works, but pressing refresh on it doesn't.
- **A setting is missing**. The app works on your computer and breaks online, because a setting only existed in a file on your laptop.
- **Capital letters matter**. Windows treats `Button.tsx` and `button.tsx` as the same file. The Linux computers that build your app online don't.
- **localhost stops working**. Your task board from [chapter 37](#) talks to `http://localhost:3001`. Online, that address points somewhere else entirely.

You'll meet, and fix, every one of these in this chapter.

Real-world example

Think about the difference between cooking for yourself and running a food stall.

Running a food stall	Deploying an app
Cooking in your own kitchen, just for you	<code>npm run dev</code> on <code>localhost</code>
Cooking the food in advance, ready to serve	<code>npm run build</code>
Packing it into boxes	The <code>dist</code> folder
Tasting one box before you open	<code>npm run preview</code>
Renting a pitch at the market	A host
A sign so people can find you	The URL
Changing the recipe means cooking and restocking	Changing the code means building and deploying again
The secret family recipe stays at home	Secrets never go in the build

Nobody at the market can taste food that's still in your kitchen. And they can read every label on every box you bring.

How it works

Build it

Open a terminal in your bookstore from [chapter 29](#) (`React/bookstore`) and run:

```
npm run build
```

You'll see something like this (your numbers and names will be different):

```
dist/index.html          0.46 kB | gzip: 0.29 kB
dist/assets/index-DqX3a7Lm.css  6.10 kB | gzip: 1.84 kB
dist/assets/CheckoutPage-C1bN8sQe.js  2.87 kB | gzip: 1.19 kB
dist/assets/index-B4hTz9Wk.js 248.33 kB | gzip: 78.52 kB
✓ built in 1.24s
```

Vite has made a new folder called `dist` (short for "distribution"):

```
dist/
├─ index.html           the one HTML page
├─ (anything from public/) like the tab icon, copied as it is
└─ assets/
    ├─ index-B4hTz9Wk.js    all your components, joined and shrunk
    ├─ index-DqX3a7Lm.css   all your CSS
    └─ CheckoutPage-C1bN8sQe.js the lazy checkout page from chapter 26
```

That `CheckoutPage` file is the **chunk** you split off with `lazy` in [chapter 26](#). A chunk is one piece of your app's code, downloaded only when it's needed.

This folder is your whole app. No `.tsx` files, no `node_modules`, no TypeScript. Just files a browser can read. The `dist` folder is what you upload, never your project folder.

Hashed file names

What's that jumble in `index-B4hTz9Wk.js`? It's a **hash**: a short fingerprint made from the file's contents. Change one letter of your code, build again, and the fingerprint changes, so the file gets a new name.

Why bother? Browsers **cache** files (keep a copy, so they don't download them again). That's great for speed, but it has a risk: a visitor's browser might keep using yesterday's code after you've fixed a bug.

Hashed names solve it. If the code changes, the name changes, and `index.html` points to the new name. The browser has never seen that name, so it downloads it fresh. If the code didn't change, the name stays the same, and the cached copy is still correct. It's like an edition number on a book: a new edition gets a new number, so nobody confuses it with the old one.

There's one catch, and you've met it before. Someone opens your site, you deploy a new version, and *then* they click through to checkout. Their old page asks for the old `CheckoutPage-C1bN8sQe.js`, which may be gone. That's the "deploy that removed the old chunk" from [chapter 26](#), and it's why an error boundary around lazy routes is worth having.

Test the real build with `npm run preview`

```
npm run preview
```

This serves your `dist` folder at `http://localhost:4173`, almost the same way a host would (one difference comes up below). It's your "taste one box" step. You used it in chapters 26 and 27 to see chunks load and to measure real speed. Before any deploy, click through the preview once.

Check before you ship

Open your app's `package.json` and find the `build` script. In the Vite React + TypeScript template, it looks like this:

```
"build": "tsc -b && vite build"
```

`&&` means "only run the second command if the first one worked". So a type error stops the build, and you can't publish an app TypeScript knows is broken ([chapter 01](#) promised you this). If your script is different, run `npx tsc -b` yourself first.

The build doesn't run your tests, so run those too:

```
npm test -- --run
```

`npm test` on its own starts Vitest in watch mode, which never finishes. The `--` means "pass what comes next to the script, not to npm", and `--run` tells Vitest to run every test once and stop.

Static hosting: any host that serves files will do

Your built bookstore is **static files**: files that are the same for every visitor. The host doesn't run any of your code. It just hands the files to browsers, and all the React work happens in the browser.

That means almost any host works. These four are popular, and all of them are fine for learning:

Host	Upload a folder, no Git?	Builds from GitHub?	The refresh fix (below)	Runs Next.js server code?
Netlify	Yes (Netlify Drop)	Yes	A <code>_redirects</code> file	Yes
Vercel	Mostly Git, or its command-line tool	Yes	A <code>vercel.json</code> file	Yes (Vercel makes Next.js)
Cloudflare	Yes	Yes	Automatic on Pages, one setting on Workers	With an adapter
GitHub Pages	No	Yes	No built-in fix	No

This isn't a ranking. Each one changes its plans and features often, so check their own pages before you pick. This chapter uses Netlify for the step-by-step parts, only because its drag-and-drop deploy needs no Git at all.

Your first deploy: drag and drop

Start with the bookstore, because it has no API. Everything it needs is inside `dist`.

1. Run `npm run build` in `React/bookstore`.
2. Go to app.netlify.com/drop. Signing up or logging in first is a good idea. (At the time of writing, a site dropped without an account gets a temporary password until you claim it.)
3. Drag the **dist folder** (not the `bookstore` folder!) onto the page.
4. Wait a few seconds. Netlify gives you a link ending in `.netlify.app`.

Open it. Your bookstore is on the internet. Send the link to your phone and try it there too.

To update it later: build again, then drag the new `dist` onto your site's **Deploys** page. (Netlify can also build a project folder for you if you're logged in. Dropping `dist` yourself is still the clearest way to see what a deploy really is.)

The refresh 404 problem

Now try this on your live bookstore:

1. Click through to a book, like `/books/3`. It works.
2. Press **refresh**. You get **Page not found**.

Here's why. When you clicked, no request went to the host at all. React Router ([chapter 24](#)) just changed the address bar and showed a different component. All of that lives inside `index.html` and its JavaScript.

When you refresh, the browser asks the host for `/books/3`. The host looks for a file called `books/3` in `dist`. There isn't one, so it says 404. The host has no idea your router exists.

The fix is a **rewrite**: a rule that tells the host "if you can't find a file for this address, send `index.html` instead". The page loads, React Router reads the address, and shows book 3. This is also called a **fallback**.

On Netlify, create a file called `_redirects` (no extension) in your `public/` folder:

```
/*      /index.html    200
```

Read it as: "for any address (`/*`), send `/index.html`, with status `200` (OK)". Vite copies everything in `public/` into `dist`, so the rule gets deployed with your app. Netlify only uses it when no real file matches, so your JavaScript and CSS files are still served as normal.

On Vercel, add a `vercel.json` file next to `package.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

On other hosts, look for a setting called something like "single-page app", "rewrites" or "fallback". Cloudflare Pages does it automatically if you don't have a `404.html` file. On Cloudflare Workers it's one setting (`not_found_handling`). Cloudflare is moving its sites from Pages to Workers, so these names may change.

GitHub Pages has no built-in fallback, and it needs two extra steps. Your site usually lives at `https://your-name.github.io/bookstore/`, not at the root. So you set Vite's `base` option to `"/bookstore/"` in `vite.config.ts`, and give React Router the same starting point with `<BrowserRouter basename={import.meta.env.BASE_URL}>`. For refreshes, the common workaround is a copy of `index.html` called `404.html`. It works, but it's more fiddly than the others. [Vite's deploy guide](#) walks through it.

Once the fix is live, a nonsense address like `/banana` also gets `index.html`. That's fine: your own `path="**"` route from chapter 29 shows your own 404 page.

Why doesn't `npm run dev` have this problem? Vite's dev server and `npm run preview` already do this fallback for you. That's why you only find the bug after deploying.

Environment variables

Your recipe finder ([chapter 21](#)) has TheMealDB's address typed into `api.ts`:

```
const BASE = "https://www.themealdb.com/api/json/v1/1";
```

Real apps often need different settings in different places: a test API on your laptop, the real one online. An **environment variable** is a setting that lives outside your code, so you can change it without touching the code. You used them with Node's `--env-file` in JavaScript chapter 50.

Vite reads them from `.env` files in your project folder (next to `package.json`, not in `src/`). Make a file called `.env.local`:

```
VITE_MEALDB_URL=https://www.themealdb.com/api/json/v1/1
```

Then read it in your code with `import.meta.env`:

```
const BASE = import.meta.env.VITE_MEALDB_URL;
```

The rules:

- **Only variables starting with `VITE_` reach your code.** Anything else in the file is ignored by your app. That prefix is on purpose: it stops you leaking a setting by accident. (Vite also fills in a few of its own, like `import.meta.env.DEV`, which you used in [chapter 30](#).)
- **The value is always a string.** `VITE_DEMO_MODE=false` gives you the string `"false"`, which is truthy! Compare it: `import.meta.env.VITE_DEMO_MODE === "true"`.
- **The value is baked in when you build.** Vite swaps `import.meta.env.VITE_MEALDB_URL` for the actual text while building. Change the setting later, and you must build (and deploy) again.
- **If a new value doesn't show up in `npm run dev`,** stop the dev server and start it again.

Vite reads several files. These three are enough:

File	Used by	Put in Git?
<code>.env.local</code>	Every command, on your computer only	No. Vite's own <code>.gitignore</code> skips <code>*.local</code> files
<code>.env.development</code>	<code>npm run dev</code>	Yes, if it holds nothing private
<code>.env.production</code>	<code>npm run build</code>	Yes, if it holds nothing private

Online, the host sets them for you. Most hosts have an "Environment variables" page in their settings. Add `VITE_MEALDB_URL` there with the real value. When the host builds your app, Vite finds the variable and bakes it in. Variables set this way win over the ones in `.env` files. (With drag and drop, *you* build it, so the values come from your own `.env` files.)

Typing your variables

By default, TypeScript types every `import.meta.env` value you haven't told it about as `any`. So a typo like `VITE_MEALBD_URL` slips straight through. Tell it which variables you have. Create `src/vite-env.d.ts`:

```
// src/vite-env.d.ts
interface ViteTypeOptions {
  strictImportMetaEnv: unknown; // makes any name not listed below an error
}

interface ImportMetaEnv {
  readonly VITE_MEALDB_URL: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

Now `import.meta.env.VITE_MEALBD_URL` is a type error, and `VITE_MEALDB_URL` is a `string`.

This is one of the places where you must use `interface`, not `type`. You're adding to interfaces Vite already defines. That's **declaration merging**, which TypeScript chapter 04 said you'd rarely need. This is one of those rare times. Don't put any `import` lines in this file, or the merging stops working. (Older Vite templates created this file for you, with a `/// <reference types="vite/client" />` line at the top. If yours has that line, keep it. The `strictImportMetaEnv` switch only exists in recent Vite versions.)

But notice: that `string` is a **promise, not a proof**, the same lesson as chapters 18 and 32. If you forget to set the variable on the host, it's `undefined` at runtime, and TypeScript can't know. So check it once, where you read it:

```
const BASE = import.meta.env.VITE_MEALDB_URL;

if (!BASE) {
  throw new Error("VITE_MEALDB_URL is missing. Add it to .env.local, or to your host's settings.");
}
```

A clear error in the Console beats a thousand requests to `undefined/search.php`. (Zod from [chapter 32](#) can check a whole group of variables at once, if you have several.)

Everything in the build is public

This is the most important rule in the chapter. **Every file in `dist` is downloaded to every visitor's browser, and they can read all of it.** Open your live site, press `F12`, and look in the **Sources** tab. Or open `dist/assets/index-*.js` in VS Code and search for `themealdb`. The URL is sitting right there, in plain text.

So a `VITE_` variable is not a hiding place. It's a *setting*, not a *secret*. Never put an API secret key, a password, or anything that costs money in one. This is the "don't hide a spare key under the doormat" rule from JavaScript chapter 51.

A secret needs a **server** to keep it: your browser asks your server, and your server adds the secret and talks to the other API. That's exactly what Server Components and Server Functions gave you in [chapter 39](#). (You met Next.js's version of this rule there too: only variables starting with `NEXT_PUBLIC_` reach the browser. The rest stay on the server.)

Git-connected deploys

Dragging `dist` works, but you have to remember to build and drag every time. The professional way is to connect the host to your code on GitHub:

1. You **push** (upload) your code to GitHub.
2. The host notices, downloads it, runs `npm run build` itself, and deploys `dist`.
3. Every push after that deploys again, automatically.

Git is a tool that saves snapshots of your project, called **commits**, so you can go back in time. **GitHub** is a website that stores a copy of your Git history online. A **repository** (or **repo**) is one project's folder plus its whole history. Git will get its own folder in this course, so this section only covers the few commands you need, with one line on what each does.

First, a trap: your app is already inside a repository

Your `docs` folder is already a Git repository. So every folder inside it, including `React/bookstore`, is already part of that one repository.

Check it yourself. Open a terminal in `React/bookstore` and run `git status`. It prints `On branch main`, because you're inside the `docs` repo.

If you now ran `git init` in `bookstore`, you'd create a repository inside a repository. Git gets confused: the outer one stops tracking the bookstore's files, and warns `adding embedded git repository`. **Don't run `git init` inside a folder that's already in a repo.**

You have two good options:

- **A. Copy the app to its own folder, outside `docs`, and make it its own repo.** Recommended for anything you'll show people. One project per repository means a clean README, a short link, and something you can put on your CV.
- **B. Push your whole `docs` repo to GitHub**, and tell the host which subfolder the app lives in (the "base directory", below). Less copying, but your whole learning folder goes to GitHub. You can make that repository private, and hosts can still build from it.

The minimum Git commands (option A)

1. Copy `React/bookstore` to a new folder outside `docs`, like `Projects/bookstore`. Leave `node_modules` and `dist` behind (they're huge, and easy to re-create).
2. Open a terminal in the new folder. Run `git status`. This time you *want* the error `fatal: not a git repository`. It means no repository owns this folder yet.
3. Run `npm install`, then `npm run build`, to check everything still works.
4. Open `.gitignore` (Vite made it). Check it lists `node_modules`, `dist` and `*.local`. Add a line with `.env`, to match your `docs` habit.

Then:

```
git init                # start a new repository in this folder
git add .               # pick every file, except those in .gitignore
git status              # look before you commit: no node_modules, dist or
.env!
git commit -m "First version of the bookstore" # save a snapshot, with a message
```

(The first commit may ask who you are. Run `git config --global user.name "Your Name"` and `git config --global user.email "you@example.com"` once, then commit again.)

Next, on github.com, create a **new repository** called `bookstore`. Leave it empty: no README, no `.gitignore`, no licence. You already have files. GitHub then shows you commands like these:

```
git branch -M main      # call your branch "main", GitHub's usual name
git remote add origin https://github.com/YOUR-NAME/bookstore.git # where GitHub's copy lives
git push -u origin main # upload your commits to GitHub
```

The first push may open a browser window to sign in to GitHub. After that, every change is three commands:

```
git add .
git commit -m "Fix the refresh 404"
git push
```

(For option B, run `git status` in `docs` first to check you have nothing half-finished, then do the GitHub part from the `docs` folder: create an empty repository, then `remote add` and `push`.)

Connect the host

On your host, choose something like **Add new project** → **Import from Git**, and pick your repository. Most hosts spot that it's a Vite app and fill in the settings for you. Check them anyway:

What it means	Netlify calls it	Vercel calls it	Your value
Which folder holds the app	Base directory	Root Directory	Empty for option A. <code>React/bookstore</code> for option B
How to build it	Build command	Build Command	<code>npm run build</code>
Which folder to upload	Publish directory	Output Directory	<code>dist</code> (inside the app's folder)

Then add your environment variables on the host's settings page, and deploy. From now on, `git push` is your deploy button. If the build fails, the host shows a **build log**: the same output you'd see in your own terminal.

Read it from the first red line, just like a TypeScript error.

Preview deploys

Here's a lovely extra. Say you want to try a dark theme without breaking the live site:

```
git switch -c try-dark-theme          # make a new branch and move onto it
```

A **branch** is a separate line of work: your commits go there, and `main` stays untouched. Make your change, commit, then `git push -u origin try-dark-theme`.

On GitHub, open a **pull request**: a request to merge a branch into `main`, where you can look at the changes first. Most hosts then build that branch too, and post a **preview URL** on the pull request: a separate copy of the site with your change in it, while the real site stays as it was. Happy with it? Merge the pull request, and the host deploys it for real.

Continuous integration: a robot that checks every push

Continuous integration (CI) is a robot that runs your checks every time you push. You don't have to remember to run the tests. It does, every time.

GitHub has one built in, called **GitHub Actions**. You describe the job in a **YAML** file, a settings format where indentation matters (like Python). Create `.github/workflows/ci.yml` in your app's repository:

```
name: CI

on:
  push:
    branches: [main]      # run on every push to main...
  pull_request:          # ...and on every pull request

jobs:
  check:
    runs-on: ubuntu-latest # a fresh Linux computer, borrowed from GitHub
    steps:
      - uses: actions/checkout@v7 # download your code onto it
      - uses: actions/setup-node@v7 # install Node.js
        with:
          node-version: 24 # a current LTS version
      - run: npm ci # install exactly what package-lock.json says
      - run: npx tsc -b # check the types
      - run: npm test -- --run # run every test once
      - run: npm run build # prove it builds
```

Commit it and push. On GitHub, open the **Actions** tab to watch it run. A green ✓ appears next to your commit when every step passes, and a red X when one fails. Click the X to see which step failed and why.

A few notes:

- `npm ci` is `npm install` for robots. It installs exactly the versions in `package-lock.json` (JavaScript chapter 50), and stops with an error if `package.json` and the lock file disagree.
- `npx tsc -b` **looks doubled up**, since the build runs it too. Keeping it as its own step makes a red X easier to read: you see "types failed", not "build failed".
- **The version numbers change.** `@v7` and Node `24` were current when this was written (autumn 2026). Check the latest on the [actions/checkout](#) and [actions/setup-node](#) pages, and the LTS version on [nodejs.org](#).
- **Option B?** The workflow file must be at the top of the repository (`docs/.github/workflows/ci.yml`), with a `working-directory: React/bookstore` setting. Ask Claude to help you adjust it.

Now CI tests every push, and the host deploys it. Together, they're a small version of what real teams use every day.

Deploying Next.js

Your chapter 39 app is not just static files. Server Components run on a server before the page is sent, and Server Functions run on a server when a form is submitted. So it needs a host that can **run code**.

Your options:

- **Vercel** makes Next.js, so it supports every feature first. Import your repository, and it recognises Next.js by itself.
- **Netlify, Cloudflare and others** support Next.js too, through adapters. Check their Next.js pages, because a brand-new Next.js feature can take a while to arrive.
- **Any host that runs Node.js.** Run `npm run build`, then `npm start`. That's the whole production server.

Try the production version locally first, the same "taste one box" idea: `npm run build`, then `npm start`, then open `http://localhost:3000`.

Set your environment variables on the host too. Server-only ones (with no `NEXT_PUBLIC_` prefix) are the right home for a secret key: they stay on the server and never reach the browser.

If your Next.js site uses no server features at all, it can be exported as plain static files instead. Add `output:` "export" to `next.config.ts`, and `npm run build` makes an `out` folder you can upload anywhere. Check the Next.js docs first for the list of features that can't work this way.

What about my API?

Your task board from [chapter 37](#) talks to json-server at `http://localhost:3001`. Deploy it, open it on your phone, and it can't load any tasks.

Here's exactly why. `localhost` means "this computer". Your deployed app runs in the *visitor's* browser. On your phone, `localhost:3001` means "port 3001 on this phone", and there's no json-server there. It might even seem to work on your own laptop, while your json-server happens to be running. That's the trap. Always test on another device.

You have three honest options:

1. **A "demo mode" that stores tasks in the browser.** Keep the same functions in your api layer (`src/api/tasks.ts`), but when a `VITE_DEMO_MODE` setting is `"true"`, they read and write `localStorage` instead of calling the server. Every visitor gets their own private task board.
2. **Host a real backend.** json-server was built for practice and prototypes, not for the real internet: anyone could delete all your tasks, and many hosts don't keep files your app writes. A proper API is what the upcoming Node.js and Express course is for. (When that day comes, change the `baseUrl` in `src/api/client.ts` to read a `VITE_API_URL` variable, and nothing else needs to know.)
3. **Use a hosted backend service.** Some companies give you a database with a ready-made API. They need an account, keys and security rules, so they're a topic of their own.

For a portfolio, choose option 1 for now. The api layer from chapter 37 makes it a small, contained change. The components and TanStack Query hooks call `getTasks()` exactly as before. Only the inside of the function changes:

```
// src/api/tasks.ts (the idea, shown for one function)
const DEMO_MODE = import.meta.env.VITE_DEMO_MODE === "true";

export async function getTasks(): Promise<Task[]> {
  if (DEMO_MODE) {
    return getDemoTasks(); // new: reads the tasks from localStorage
  }
  // ...exactly what your chapter 37 version already does
}
```

That's the payoff of keeping every server call in one file. The rest of the app never knew where tasks lived, so it doesn't care that they've moved. You'll build the whole thing in the exercises.

After deploying: a five-minute check

Every time you deploy something new, check these on the **live** URL:

1. **Open it on your phone.** Not a narrow browser window, your real phone.

2. **Refresh a deep link**, like `/books/3`. No 404.
3. **Open the Console** (`F12`). No red errors, and no failed requests in the Network tab.
4. **Run Lighthouse** on the live URL (DevTools → **Lighthouse** tab, ideally in a private window so extensions don't interfere). Check the **Performance** and **Accessibility** scores, as in [chapter 38](#). Live numbers are the real ones.
5. **Try it with the Network tab set to a slow connection**. Do your loading states still make sense?

Custom domains and HTTPS: every host above gives you HTTPS (the padlock) automatically, for free. If you buy your own domain name, like `sam-codes.dev`, the host's "Domains" settings page walks you through pointing it at your site.

Common mistakes

1. Uploading the project folder instead of `dist`

Dropping `bookstore/` uploads your `.tsx` files and `node_modules`, and the browser can't run any of it. You get a blank page, or thousands of files that take forever. Build first, then upload `dist`.

2. The refresh 404

Clicking around works, refreshing a deep link says "Page not found". The host is looking for a file that doesn't exist. Add the rewrite to `index.html` (`public/_redirects` on Netlify, `vercel.json` on Vercel).

3. A secret in a `VITE_` variable

`VITE_PAYMENT_SECRET=sk_live_12345` ❌ ends up in the JavaScript every visitor downloads

Everything in the build is public. Secrets belong on a server. If a secret was ever deployed or pushed, cancel it with the service and make a new one (JavaScript chapter 51).

4. Forgetting the variable on the host

It works locally (because of your `.env.local`) and breaks online. Add every `VITE_` variable in the host's settings. And remember they're baked in at build time: after changing one, deploy again.

5. Capital letters in file names

Windows doesn't care whether it's `components/Button.tsx` or `Components/Button.tsx`. The Linux computer that builds your app does:

```
error TS2307: Cannot find module './Components/Button.tsx' or its corresponding type declarations.
```


TypeScript usually warns you on Windows too (a name that "differs only in casing"). The sneaky case is renaming a file *only* by its capital letters, like `button.tsx` to `Button.tsx`: Git on Windows may not notice. Use `git mv button.tsx Button.tsx` so Git records the new name. JavaScript chapter 52 warned you about this one.

6. Pointing a deployed app at `localhost`

`localhost` is the visitor's own device. Use a real address for a real API, or a demo mode.

7. Committing `node_modules` or `.env`

Always run `git status` after `git add .` and before `git commit`. If you see `node_modules` in the list, stop, fix `.gitignore`, and run `git rm -r --cached node_modules` to take it back out.

8. `git init` inside a folder that's already in a repository

```
warning: adding embedded git repository: React/bookstore
```

Check with `git status` before you `git init`. If you've already done it, delete the hidden `.git` folder inside the app's folder only (never the one in `docs`!), and choose option A or B instead.

9. Wrong base or publish directory

The build log says it can't find `package.json` (wrong base directory), or the live site is blank or says "Page not found" at `/` (wrong publish directory). For a Vite app, the base is the folder containing `package.json`, and the publish directory is `dist` inside it.

Quick recap

- Deploying means building your app into plain files (`npm run build` → `dist`) and putting them on a host. Test the real build first with `npm run preview`.
- Hashed file names like `index-B4hTz9Wk.js` change whenever the code changes, so browsers never run an old cached copy.
- A single-page app needs a **rewrite to `index.html`**, or refreshing a deep link gives a 404.
- Only `VITE_` variables reach your code, as strings, baked in at build time. Set them on the host too.
Everything in the build is public, so never put a secret in one.
- The simplest deploy is dragging `dist` onto Netlify. The professional one is Git-connected: push to GitHub, and the host builds, deploys, and makes preview URLs for pull requests.
- Don't `git init` inside `docs`. Copy the app out and give it its own repository, or set the host's base directory.
- CI runs your type check, tests and build on every push, and shows a green ✓ or a red X.
- Next.js needs a host that runs code. json-server can't come with you, so give your task board a `localStorage` demo mode behind the same api functions.

Next: try the [exercises](#), then move on to the [41 Final Project](#).

Exercises

How to do these:

- This time you work in your finished project apps, not the playground: `React/bookstore` (chapter 29), `React/recipe-finder` (chapter 21), `React/taskboard` (chapter 37) and `React/next-practice` (chapter 39).
- Before each exercise, make sure the app works locally, `npx tsc -b` prints nothing, and (where the app has tests) `npm test -- --run` passes. Deploying a broken app just gives you a broken app with a URL.
- Remember the trap from the notes: your `docs` folder is already a Git repository. **Never run `git init` inside it.** Copy an app out first (option A), or push `docs` and set a base directory (option B).
- Hosting screens change often. If a button in these steps has moved, look for the same idea under a different name, or ask Claude where it went.
- An exercise is done when it works **on the live URL, on your phone**, and the Console there has no unexpected errors.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your work (share the live link, too).

Exercise 1 (Easy): Look inside the box

Before you put your bookstore online, find out exactly what you'd be uploading.

1. In `React/bookstore`, open `package.json` and read the `build` script. Write down what it runs, and in which order. (A comment at the top of `App.tsx` is a fine place for all your answers.)
2. Run `npm run build`. Open the new `dist` folder in VS Code. Open `dist/index.html`: which JavaScript and CSS files does it load? Is the checkout chunk one of them? Why not?
3. Find the checkout chunk in `dist/assets/` and write down its full file name.
4. Change one word in `CheckoutPage.tsx` (the heading, say) and build again. Which file names in `dist/assets/` changed, and which stayed the same? Did the main `index-*.js` change too? Search inside it for `CheckoutPage-` and work out why.
5. Run `npm run preview`, open the Network tab, and go to `/books`, then `/checkout`. Note the moment the checkout chunk downloads.
6. Search the main `index-*.js` file for the title of one of your books. Found it? Write one sentence about what this means for anything you'd want to keep secret.
7. Break it on purpose: double-click `dist/index.html` to open it straight from the disk. Write down what you see and the first error in the Console. Why does a built app still need to be *served*?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Your bookstore, live

Time to open the stall.

1. Build the bookstore and deploy `dist` with Netlify Drop (or any host that lets you upload a folder).
2. Open the live link on your phone. Browse, add two books to the cart, and open the cart page.
3. Reproduce the refresh 404: open a book page, like `/books/3`, and refresh. Write down exactly what the host shows. Then paste the same link into a brand-new tab. Same result?
4. Fix it with a `public/_redirects` file. Build again and check that `dist/_redirects` exists before you deploy.
5. Deploy the new `dist` to the **same** site (drag it onto that site's Deploys page), so the URL stays the same.
6. Check all three: refreshing `/books/3` works, `/books/999` shows your "couldn't find that book" message, and `/banana` shows your own 404 page.
7. Add a book to the cart and refresh. Is the cart still there? Is that a hosting problem? Explain in one or two sentences.

What you should see: a bookstore that behaves exactly like `npm run preview` did, on a real URL.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The recipe finder, deployed from GitHub

Your recipe finder from chapter 21 gets a proper, automatic deploy, with its API address in an environment variable.

1. Decide: option A (copy `recipe-finder` to its own folder outside `docs`) or option B. Write down which, and why.
2. Move TheMealDB's address out of `api.ts` into `VITE_MEALDB_URL` in `.env.local`. Type it in `src/vite-env.d.ts` with `strictImportMetaEnv` on, and throw a clear error if it's missing. Misspell the name once in `api.ts` and read the type error, then fix it.
3. Put the app on GitHub. Before your first commit, run `git status` and check that `node_modules`, `dist` and `.env.local` are **not** in the list.
4. Connect the repository to a host (Netlify, Vercel or Cloudflare). Check the build settings. **Deploy once without setting the variable.** What happens on the live site? Where does your error message show up?
5. Now add `VITE_MEALDB_URL` in the host's settings and deploy again. Search for a recipe on your phone.
6. Change something visible (the title's emoji, say), commit and push. Time how long it takes from `git push` to seeing the change live.

7. Make a preview deploy: create a branch, change a colour, push it, and open a pull request on GitHub. Find the preview link. Check that the live site hasn't changed. Then merge, and watch the live site update.
8. Does the recipe finder need a `_redirects` file? Answer in a sentence, with your reason.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A robot that checks your work

Add CI to the bookstore, and watch it catch a bug you "forgot" to test.

1. Make the bookstore its own repository on GitHub, the same way as Exercise 3 (option A is easiest here).
2. Add `.github/workflows/ci.yml` from the notes. Commit, push, and watch it run in the **Actions** tab until you get a green ✓.
3. Break a rule on purpose: in `cartReducer.ts`, change the quantity cap from 10 to 11. **Don't run the tests yourself.** Commit and push. Find the red X, open it, and write down which step failed, which test failed, and what the message said.
4. Fix it, push, and get back to green.
5. Now break the types instead: pass a number where a component expects a string. Push. Which step fails this time? Why is it helpful that `npm run lint` has its own step?
6. Make a branch with a small change and open a pull request. Find where the check's ✓ or X shows up on the pull request page.
7. Bonus: add a step that runs `npm run lint` (the linter Vite set up in chapter 01).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Ship the rest

Two apps that can't just be dragged onto a host, and a final check on both.

Part A: the Next.js app. Deploy `React/next-practice` from chapter 39.

1. Run the production version locally first: `npm run build`, then `npm start`.
2. Deploy it to a host that runs Next.js. Add any server-only environment variables in the host's settings.
3. On the live site, use something that runs on the server (submit a form that calls a Server Function). Then open the Sources tab and search for your secret's value. It must not be there.

Part B: the task board, in demo mode. Make `React/taskboard` work online without json-server.

1. Add a `VITE_DEMO_MODE` variable, typed in `vite-env.d.ts`. Remember it's a string.

2. Give **every** function in `src/api/tasks.ts` (`getTasks`, `getTask`, `createTask`, `updateTask`, `deleteTask`) a demo version that uses `localStorage`. Your routes, components and `queries.ts` must not change at all. Prove it: when you're done, `git diff` (or your memory!) should show changes only in `src/api/`, `vite-env.d.ts`, and one small banner.
3. On a visitor's first visit, start them with three or four example tasks, so they don't see an empty board.
4. Check saved data when you load it, with the `TaskListSchema` you already have ([chapter 32](#)). Edit it into nonsense in DevTools and refresh: the board should start fresh, not crash.
5. Show a small banner in demo mode: "Demo: your tasks are saved in this browser only."
6. Write tests for the demo functions (Vitest's `jsdom` environment has `localStorage`).
7. Deploy with `VITE_DEMO_MODE=true` set on the host. Locally, with the variable unset, it should still use json-server exactly as before.

Part C: check both. Run Lighthouse on both live URLs. For each, fix one thing it reports (accessibility or performance), deploy again, and write down the score before and after.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

41 Final Project

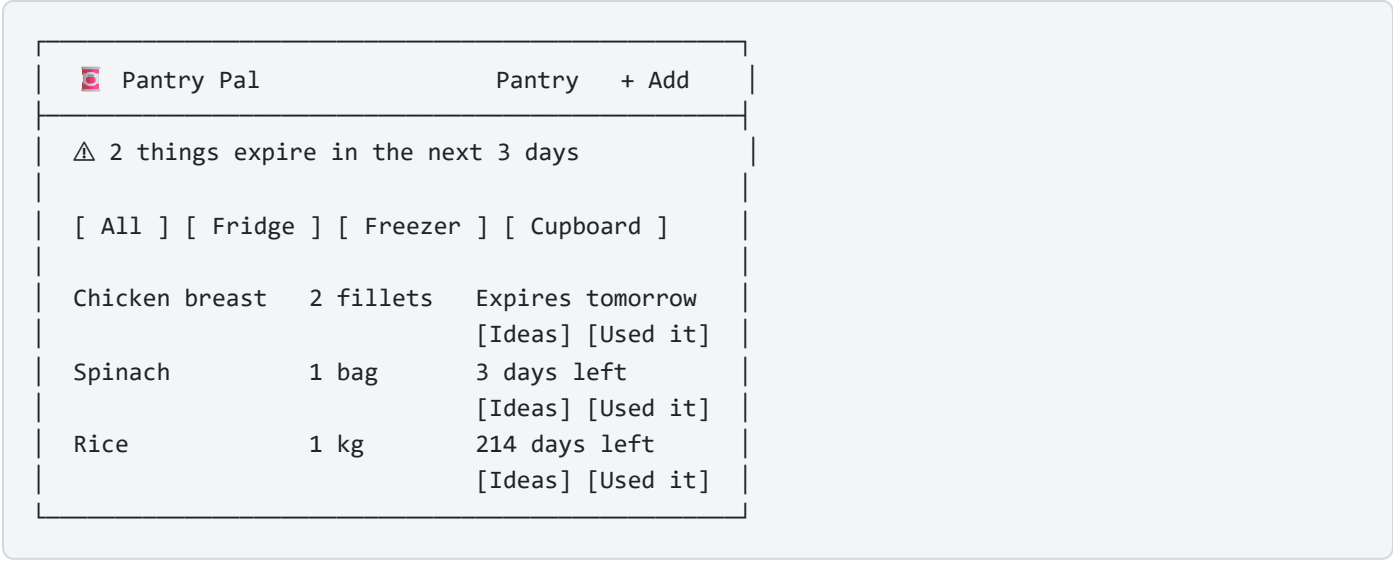
You've reached the last chapter of the React course. 🎉

This time there's no step-by-step recipe. You'll pick your own project and take it all the way from an idea to a live URL, the way professional developers do. That's a big jump from following milestones, so this chapter walks you through the process, one step at a time.

What you'll build

Something of your own! The [exercises](#) have a menu of project ideas, from a habit tracker to your own portfolio site. You can also bring your own idea.

To show you the process, this chapter follows one example project all the way through: **Pantry Pal**, an app that shows what food is in your kitchen, what expires soon, and what you could cook with it. It isn't on the menu, so there's nothing to copy. Watch *how* it gets decided and built, then do the same with your project.



Here are the steps:

- 1. Pick one small idea.
- 2. Split the must-haves from the nice-to-haves.
- 3. Sketch the screens, then break them into components.
- 4. Sketch the data, and give every piece of state a home.
- 5. Choose your tools, only the ones you need.
- 6. Plan your folders.
- 7. Build the smallest working version, in small steps.
- 8. Test the logic that matters.
- 9. Do an accessibility pass.

10. Deploy it, and write a README.
11. Ask for a code review.

If you did the JavaScript final project, some of these will feel familiar. That's on purpose. The process doesn't change much. What's new is how many more decisions React gives you, and steps 4 and 5 are where you make them.

Getting started

Don't create the app yet. Steps 1 to 6 happen on paper (or in a notes file). You'll create the app in step 7 (with `npm create vite@latest`, or with Next.js's own setup command if step 5 picks Next.js), once you know what you're building.

When you do, make the project **outside** your `docs` folder, next to it for example. Then it can be its own Git repository from the first day, and you avoid the repo-inside-a-repo trap from [chapter 40](#). (Keeping it in `docs/React/` works too. You'll just set a base directory when you deploy.)

The big idea: decide before you type

In every project so far, the big decisions were made for you: "the cart lives in context", "use TanStack Query for the tasks". In this project, **you** make them. That's the real skill this chapter practises.

Two questions do most of the work:

- **Where does each piece of state live?** You learned the four homes in Level 4: local, URL, shared client, and server. Most of the pain in a React app comes from state in the wrong home.
- **Which tools do I actually need?** Level 4's theme was that every library solves a problem you've already felt by hand. So here's the rule: **start with plain React, and add a library when you feel the pain it removes.** Not before, "because you can".

A project where these two questions were answered well feels easy to build. A project where they weren't feels like wading through mud, no matter how good the code is.

Step 1: Pick one small idea

A good project starts with a problem you can say in one sentence:

Pantry Pal: see which food in my kitchen I should use up first.

When you pick yours:

- **Pick something you'd actually use**, or that a friend would. You'll care more about finishing it, and you'll notice what's missing.
- **Keep it small.** A small app that's finished and online beats a big app that's half-built on your laptop.
- **Watch out for "and".** If your sentence needs "and" three times, it's probably three projects.

Too big	Just right
A meal-planning platform with shopping, nutrition and friends	A page that shows which food to use up first
A Trello clone with teams and live updates	A task board for one person (you built one in chapter 37!)
An app that scans receipts with the camera	Typing in what you bought
A social network for readers	A bookshelf of what you're reading

Check it: you can say your idea in one sentence, to someone who isn't a programmer, and they get it.

Step 2: Must-haves and nice-to-haves

Make two lists before you write any code:

- **Must-haves** make the app useful at all. Without any one of them, it doesn't do its job.
- **Nice-to-haves** would be great, *later*.

Together, the must-haves are your **MVP**, short for **minimum viable product**: the smallest version of the app that's genuinely useful. You build that first, and nothing else.

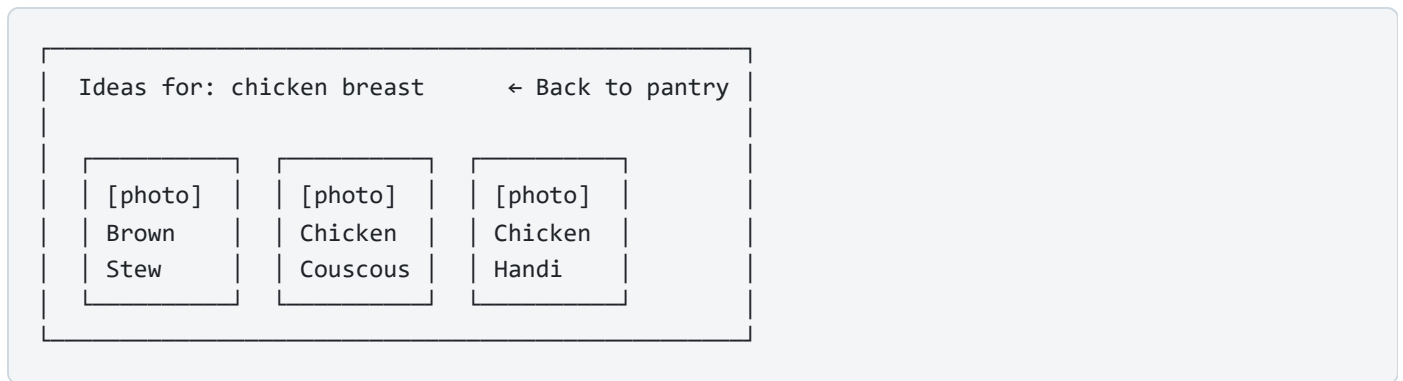
Must-have (the MVP)	Nice-to-have (later, maybe)
See my food, soonest to expire first	Edit an item
See how many days each item has left	A shopping list of things I've used up
Add an item: name, amount, where it's kept, expiry date	Scan a barcode to add an item
Mark an item "used it"	A reminder the day before something expires
Show only the fridge, freezer or cupboard	Share the pantry with my housemates
Get recipe ideas for an item	Dark mode
Everything's still there after a refresh	Stats on how much food I've saved

Look at "share the pantry with my housemates". Sharing means a server, accounts and a database. That's a project of its own (the Node.js course, later), which is exactly why it sits on the right.

Check it: you have five to eight must-haves, and every one of them passes the question "would the app be useless without this?"

Step 3: Sketch the screens, then break them into components

Grab a pen. Boxes and words are enough: what does the person see, and what can they press? Pantry Pal has three screens: the pantry list (sketched at the top of this chapter), an **Add item** form, and a page of recipe ideas:



Now do step 1 of [chapter 14](#): draw a box around every chunk you could name out loud in one breath. "That's the header." "That's one item row." "That's the little expiry label." Each box is a component:



Notice `ExpiryBadge`. It's tiny, but it has a real job ("tomorrow", "3 days left", "Expired!"), and it's used in every row. Small components with one job are the easiest ones to get right.

Check it: every box on your sketch has a component name, and every component has one job you can describe in a sentence.

Step 4: Sketch the data, and give every piece of state a home

What does the app need to *remember*? For Pantry Pal, it's one list of items:

```
export type StoredIn = "fridge" | "freezer" | "cupboard";

export type PantryItem = {
  id: string;
  name: string;           // "Chicken breast"
  amount: string;         // "2 fillets" (free text, on purpose)
  storedIn: StoredIn;
  expiresOn: string;      // "2026-10-01", an ISO date
};
```

The same habits as the JavaScript final project, now with types:

- **Store the facts, and work out the rest.** The item remembers *when* it expires, not the words "3 days left". Those words change every day, so the app works them out during render ([chapter 08](#)).
- **Dates as ISO strings**, like "2026-10-01". They survive `localStorage` and JSON, and nobody has to guess what `01/10` means.
- **A union type for anything with a fixed set of values.** `storedIn` can only ever be one of three words, and TypeScript will hold you to it. (Why not call the type `Location`? The browser already has a type with that name, for `window.location`, and a clash would be confusing.)

Next, list **how the items change**. You did this for the bookstore's cart in chapter 29. It becomes your reducer's action type:

```
export type PantryAction =
  | { type: "added"; item: PantryItem }
  | { type: "used_up"; id: string };
```

That's the whole MVP: two actions. (The new item's `id` is made in the event handler with `crypto.randomUUID()`, not in the reducer, so the reducer stays pure.)

Now the most important table in your plan. Sort **every** piece of state into its home, using the four homes from Level 4:

Piece of state	Kind	Home
What's half-typed in the Add form	Local UI	<code>useState</code> in <code>ItemForm</code>
Whether "Are you sure?" is showing	Local UI	<code>useState</code> in <code>ItemRow</code>

Piece of state	Kind	Home
Which place is showing (? where=fridge)	URL	<code>useSearchParams</code>
Which ingredient's ideas are showing	URL	The path: <code>/ideas/chicken%20breast</code>
The pantry items	Shared client	One reducer, shared with context, saved to <code>localStorage</code>
Recipe ideas from TheMealDB	Server	TanStack Query, key <code>["ideas", ingredient]</code>
"3 days left", the banner's count, the sort order	None!	Derived during render

That last row matters as much as the others. Everything in it is worked out from the items and today's date, so it can never go stale.

The quick tests from chapter 29, with one more added: **Does the data really live somewhere else?** → server state. **Would someone want to send it in a link?** → URL. **Does a component far away need it?** → shared. **None of those?** → local.

Check it: every piece of state in your app appears in your table exactly once, and your "derived" row isn't empty.

Step 5: Choose your tools, only the ones you need

Here's a table for making this decision on purpose:

I need...	Reach for...	But only if...
More than one page, with real URLs	React Router (24), or TanStack Router (36)	There's more than one screen worth linking to
Data from a server	TanStack Query (31), with <code>fetch</code> or <code>Axios</code> (30)	The data really lives somewhere else
To check data from outside my code	Zod (32)	It comes from an API, <code>localStorage</code> , or the URL
A big form	React Hook Form (33)	Lots of fields, or rules that connect fields
Client state that many parts share	Context + <code>useReducer</code> (22 , 23), or Zustand (34) / Redux Toolkit (35)	Far-apart components read and change it

I need...	Reach for...	But only if...
Server-only code, secrets, or pages search engines must find	Next.js (39)	You have a secret key, or search results matter

And here's how Pantry Pal answers it, including the tools it **deliberately leaves out**:

Tool	Pantry Pal?	Why
React Router	✓ Yes	Three screens, and the ideas page deserves a real URL
TanStack Query	✓ Yes	Recipe ideas come from TheMealDB. Opening "Ideas" for chicken twice should use the cache
Zod	✓ Yes	Checks TheMealDB's answers, <i>and</i> the saved pantry when it's loaded (users can edit <code>localStorage</code>)
Axios	✗ No	One endpoint, read-only. Plain <code>fetch</code> in one api file is enough
React Hook Form	✗ No	One form with four fields. A controlled form from chapter 09 is fine
Zustand or Redux Toolkit	✗ Not yet	Two pages share the items. The bookstore's reducer + context pattern handles that. If more pages needed them and the provider started to feel heavy, Zustand would be the next step
Next.js	✗ No	No secret keys, and nobody needs to find their own pantry on Google

Every "no" is a thing you don't have to install, learn the quirks of, or keep up to date. That's a real saving, not a missed opportunity.

Check it: for every library you picked, you can name the pain it removes *in your app*. For every one you didn't, you can say why not.

Step 6: Plan your folders

Decide which folder does which job, before there are forty files:

```

pantry-pal/
├── public/
│   └── _redirects           the refresh fix, from chapter 40
├── src/
│   ├── main.tsx           providers: router, query client, pantry
│   ├── App.tsx            the route table
│   ├── api/
│   │   └── recipes.ts     TheMealDB calls and their Zod schemas
│   ├── pantry/
│   │   ├── types.ts
│   │   ├── expiry.ts      daysUntilExpiry and friends: pure functions
│   │   ├── expiry.test.ts
│   │   ├── pantryReducer.ts
│   │   ├── pantryReducer.test.ts
│   │   └── PantryContext.tsx provider, usePantry(), usePantryDispatch()
│   ├── pages/
│   │   ├── PantryPage.tsx
│   │   ├── AddItemPage.tsx
│   │   ├── AddItemPage.test.tsx the one user flow that matters most
│   │   └── IdeasPage.tsx
│   └── components/
│       ├── ItemRow.tsx
│       ├── ExpiryBadge.tsx
│       └── RecipeCard.tsx
├── README.md
└── package.json

```

Three habits in that tree:

- **Tests sit next to the file they test.** When you open `expiry.ts`, its tests are right there.
- **`api/` is the only folder that knows `TheMealDB` exists**, the same three-layer split as the recipe finder in [chapter 21](#). If you use Zustand, a `src/stores/` folder is the usual home for stores.
- **`expiry.ts` never imports `React`.** It takes data in and hands answers back, so it's the easiest file in the project to test.

Check it: you can point at the folder where any feature on your must-have list will live.

Step 7: Build the MVP in small steps

Now create the app (`npm create vite@latest`, naming it `pantry-pal`), install only the tools step 5 chose, and make your first commit:

```

git init
git add .
git commit -m "Empty Vite app"

```

(Run `git status` first. Outside `docs`, it should say `not a git repository`.)

Then add **one** piece at a time, and run the app after each one:

1. Hard-coded items, shown on one page. No state at all: the static version from [chapter 14](#).
2. The expiry labels, from `daysUntilExpiry` (step 8 shows it).
3. The reducer and context. "Used it" removes an item.
4. Routes, and the Add page with its form.
5. Save and load with `localStorage`, checked with Zod on the way in.
6. The Ideas page, with TanStack Query.
7. The fridge / freezer / cupboard filter, in the URL.
8. Only now: the styling, and nice-to-haves.

After each working step, commit. `git add .`, then `git commit -m "Add the expiry labels"`. Each commit is a save point: if step 5 goes horribly wrong, you can get back to the end of step 4.

Tip: if a step feels too big, it is. Split it until each piece takes under an hour.

Check it: the app runs, `npx tsc -b` prints nothing, and you have a commit, after every numbered step.

Step 8: Test the logic that matters

You can't test everything, and you don't need to ([chapter 28](#)). Test what would really hurt if it broke. For Pantry Pal, the most important logic is working out how long food has left:

```
// src/pantry/expiry.ts
const DAY_MS = 24 * 60 * 60 * 1000;

// Days until the item expires. 0 means today. Negative means it already has.
export function daysUntilExpiry(expiresOn: string, today: string): number {
  const difference = new Date(expiresOn).getTime() - new Date(today).getTime();
  return Math.round(difference / DAY_MS);
}
```

```
// src/pantry/expiry.test.ts
import { daysUntilExpiry } from './expiry.ts';

test("chicken that expires tomorrow has 1 day left", () => {
  expect(daysUntilExpiry("2026-10-01", "2026-09-30")).toBe(1);
});

test("food that expires today has 0 days left", () => {
  expect(daysUntilExpiry("2026-09-30", "2026-09-30")).toBe(0);
});

test("food that expired two days ago gives -2", () => {
  expect(daysUntilExpiry("2026-09-28", "2026-09-30")).toBe(-2);
});
```

`npm test` runs them (with the `globals: true` setup from chapter 28). Notice that `today` is a **parameter**, not read from the clock inside the function. That keeps it pure, so the tests give the same answer every day, including the day you show it to someone. The page passes in today's date, and that's the only place the real clock is read. (Careful: `new Date().toISOString()` gives the date in UTC, which can be a day out just after midnight. JavaScript chapter 19 has the details.)

After that, in order of value:

1. **The reducer.** One test per rule, just like the bookstore's cart. It's the highest-value testing in a React app.
2. **The saved-data schema.** Hand it nonsense, and check the app starts with an empty pantry instead of crashing.
3. **One user flow:** type a new item into the form, press Add, and see it in the list. `renderWithProviders` with a `MemoryRouter`, `user-event`, and `getByRole` throughout.

Don't test TheMealDB, TanStack Query or React Router. They have their own tests.

Check it: `npm test -- --run` passes, and the logic you'd be most embarrassed to get wrong is covered.

Step 9: The accessibility pass

Don't save this for the very end, when it's hardest to change. Do a quick pass after the MVP works, using the checklist from [chapter 38](#):

- **Unplug the mouse.** Can you add an item, filter, open ideas and mark something used, with only the keyboard? Can you always see where the focus is?
- **Every input has a real label**, not just a placeholder.
- **Every button has a name that makes sense on its own.** Five buttons all called "Used it" are confusing to a screen reader user. "Used it: Chicken breast" isn't.
- **Colour is never the only signal.** A red badge also says "Expired!" in words.

- **Headings go in order**, and each page has a clear `<h1>`.
- **Run Lighthouse and axe DevTools**, and fix what they find.
- **Then spend ten minutes with a screen reader** (NVDA or Narrator), because a clean score doesn't mean the app makes sense read aloud.

Check it: you did the whole main flow with only the keyboard, and again with a screen reader, and the tools have no easy wins left.

Step 10: Deploy it, and write a README

Put it online the way [chapter 40](#) showed you: a GitHub repository, a Git-connected host, `_redirects` (or your host's version), and a CI workflow. Then check it on your phone.

"It's not ready yet" is not a reason to wait. Deploy the MVP. You can deploy again every day. An app that's online and a bit plain is worth far more than a perfect one nobody can see.

Then write a `README.md`. It's the first thing anyone sees on GitHub, so it's your project's front door:

- **The name and one sentence** (your step 1 sentence!).
- **A live link**, right at the top.
- **A screenshot.** Put the image in the repo, and show it with Markdown's image syntax. The text in the square brackets is the image's alt text:

```
![Pantry Pal's pantry page, with two items expiring soon](screenshot.png)
```

- **What it does:** your must-have list, in plain words.
- **Built with:** React 19, TypeScript, and the tools from step 5. Credit the APIs you use (for Pantry Pal, TheMealDB).
- **How to run it:** `npm install`, `npm run dev`, `npm test`.
- **What I learned:** two or three honest sentences. One thing that was harder than you expected is great.
- **What's next:** your nice-to-have list.

Check it: a friend can open your README, click the live link, and use the app without asking you anything.

Step 11: Ask for a code review

When a feature works, ask Claude to review it. A good request says what the code is, what you're unsure about, and what kind of help you want:

"Here's my Pantry Pal project: `pantryReducer.ts`, `PantryContext.tsx` and `PantryPage.tsx`. It works and the tests pass. Please review it for bugs, state that's in the wrong home, and accessibility problems. I'm not sure the fridge / freezer filter belongs in the URL. Don't rewrite it for me: point out the problems and explain why."

Go through the skills checklist below and the accessibility pass above before you ask. It's much more useful to get feedback on the things you *didn't* spot.

Check it: you asked for at least one review, and you understood (and fixed or argued with!) every point in it.

Your skills checklist

Show off the skills that fit your idea. Don't force them all in: a project that uses a library it doesn't need shows *less* skill, not more.

Skill	Chapters
Components, props, lists and events	03 to 07
State, forms, and updating objects and arrays	08 , 09 , 11
Planning components and where state lives	13 , 14
Styling, or Tailwind	15 , 16
Effects, fetching and custom hooks	17 , 18 , 20
Context and reducers	22 , 23
Routing with real URLs	24 or 36
TypeScript patterns (generic components, <code>satisfies</code>)	25
Error boundaries and lazy loading	26
Measuring before optimising	27
Tests that check behaviour	28
Server data with TanStack Query	30 , 31
Checking outside data, and bigger forms	32 , 33
A store for shared client state	34 or 35
Accessibility	38

Skill	Chapters
Server Components, when you need a server	39
Deploying and CI	40

Common mistakes

- 1. Starting too big.** "A recipe social network" never gets finished. "A page that shows which food to use up first" does. Finish small, then grow it.
- 2. Adding every Level 4 library "because you can".** Zustand, Redux Toolkit, React Hook Form and Axios, all in a two-page app with one form. Each one is more to learn, more to set up, and more to go wrong. Add a library when you feel the pain it removes.
- 3. Styling before it works.** Hours of CSS on buttons that don't do anything yet. Build the MVP first. Paint last.
- 4. State in the wrong home.** Filters in `useState` (they can't be shared), server data copied into a store (it goes stale), or a "days left" number stored instead of worked out (it's wrong tomorrow). Step 4's table catches all three.
- 5. Writing a lot before running anything.** Run the app after every small change. A bug from the last five minutes is easy to find. A bug from the last five hours isn't.
- 6. Leaving accessibility until the end.** By then the structure is set, and a `<div onClick>` in forty places is forty fixes. A quick keyboard walk after the MVP costs ten minutes.
- 7. Never deploying because it's "not ready yet".** It will never feel ready. Deploy the MVP, and let the live URL push you to finish.

Quick recap

- Pick **one small idea** you can say in a sentence, and build the **MVP** first.
- Sketch the screens, then **break them into components**, the chapter 14 way.
- Sort every piece of state into its **home** (local, URL, shared client, server) and derive everything else.
- **Choose tools on purpose.** Start with plain React, and add a library only when you feel the pain it removes.
- Build in **small steps**, run it after each one, and commit every working step.
- Test the logic that matters, do an **accessibility pass**, then **deploy early** with a good README.
- Ask for a code review, and learn from it.

Congratulations! 🎉

Look at how far you've come. In chapter 01, you replaced Vite's welcome page with `<h1>Hello, React!</h1>`. Now you can plan an app from a blank page, choose your tools and say why, build it with typed components, test the parts that matter, make it work for people using a keyboard or a screen reader, and put it on the internet for anyone to use.

Project	What it taught you
To-do app (10)	Components and state, the React way
Recipe Finder (21)	Data from someone else's server, and hooks that compose
Bookstore (29)	Pages, shared state, and tests that give you confidence
Task Board (37)	The whole toolbox, and why server state is different
Your project (41)	Making every decision yourself, and shipping it

That's the journey from "following a tutorial" to "building software". Be proud of it.

Where to go next:

- **Node.js frameworks, like Express:** build your own APIs for your React apps to talk to. Your task board could finally have a real backend, and your secrets a server to live on.
- **React Native:** use everything you've learned here to build apps for phones.

Each of these will get its own folder here. Until then, keep building. Pick another idea from the [exercises](#), rebuild an app you use every day, or go back to one of your projects and do its nice-to-haves. Every project you finish makes the next one easier.

Back to the roadmap

Exercises

This is a **project menu**. Pick one idea, and build it from start to finish.

How to do these:

- Pick **one** project. (Or bring your own idea: see the end of this page.)
- Make it its own app, **outside** your `docs` folder, so it can be its own Git repository from day one ([chapter 40](#)).
- Follow the 11 steps in [notes.md](#). Steps 1 to 6 happen on paper, before you create the app.
- The tools listed with each idea are **suggestions, not a shopping list**. Use step 5's rule: add a library when you feel the pain it removes.

- Every API on this menu was free and needed no key when this was written (autumn 2026). APIs change, though. Before you build on one, read its docs and try one request in your browser's address bar. (REST Countries used to be the classic free countries API, and it now needs a key. That's exactly why you check.)
 - A project is done when the MVP works, `npx tsc -b` prints nothing, `npm test -- --run` passes, it's live on a URL, and it has a README.
 - Share your code with Claude for a review after each working feature.
-

Exercise 1 (Easy): Habit Tracker

Tick off your daily habits and watch your streaks grow.

Must-haves:

- Add and remove habits (drink water, read, stretch...).
- A checkbox for each habit, for today.
- Each habit's current streak: days in a row.
- Everything saved in `localStorage`.

Nice-to-haves: a seven-day grid per habit, a "best streak" record, reorder habits, a dark theme with context.

Practices: state and forms (08, 09), updating arrays (11), `useLocalStorage` (20), a reducer (23), tests (28). No library needed, and that's the point.

If you built this in JavaScript chapter 52, rebuild it the React way, then compare the two, like you did with the to-do app in chapter 10.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Study Timer

A Pomodoro timer: 25 minutes of focus, then a 5-minute break.

Must-haves:

- The time left, shown as `mm:ss`.
- Start, pause and reset buttons.
- Switches between "Focus" and "Break" by itself.
- Counts how many focus sessions you've finished today.

Nice-to-haves: change the lengths (saved), a sound when time's up, the time left in the browser tab's title, press Space to pause.

Practices: effects with cleanup (17), refs (19), a custom hook like `useCountdown` (20), a reducer for the modes (23), and `aria-live` so a screen reader hears "Break time!" (38).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Book Shelf

Search for books and keep your own shelf: want to read, reading, finished.

Must-haves:

- A search box with debounced results: title, author, first published year, and a cover.
- Loading, error and "nothing found" states.
- Add a book to your shelf with a status. Change its status. Remove it.
- The shelf is saved, and the search text lives in the URL (`?q=dune`).

Uses the free Open Library API: `https://openlibrary.org/search.json?q=the+hobbit&limit=10`. Covers come from `https://covers.openlibrary.org/b/id/<cover_i>-M.jpg`.

Nice-to-haves: filter the shelf by status (in the URL), a star rating, pages-read progress, a "finished this year" count.

Practices: TanStack Query with the search text in the query key (31), Zod (32), `useDebounce` (20), URL state (24), context or Zustand for the shelf (22, 34).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Flag Quiz

Guess the country from its flag, ten flags a round.

Must-haves:

- Show a flag and four possible countries. One is right.
- Say whether the answer was right (in words, not just colour), then move on.
- A score, and a results screen after ten questions, with **Play again**.
- A best score, saved.

Uses the free Flagpedia files: the list of codes and names is at `https://flagcdn.com/en/codes.json`, and each flag is an image like `https://flagcdn.com/w320/fr.png`.

Nice-to-haves: a "learn" page listing every flag with a search box, a timed mode, a harder mode with eight choices.

Practices: a reducer as a small state machine (23), a discriminated union for the game's phases (25), TanStack Query to load the list once (31), tests (28), `aria-live` and focus management (38).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Medium): Expense Tracker, with charts

See where your money goes each month.

Must-haves:

- Add an expense: description, amount in £, category, and date, with validation (amount above zero, no dates in the future).
- The list and the total for one month, with the month in the URL (`?month=2026-09`).
- Delete an expense.
- Everything saved in `localStorage` .

Nice-to-haves: a chart of spending by category, a budget per category with a warning, edit an expense, export to CSV.

Practices: React Hook Form with a Zod schema (32, 33), a reducer or Zustand (23, 34), URL state (24), tests for the totals (28), `Intl.NumberFormat("en-GB", { style: "currency", currency: "GBP" })` for money.

For the chart, you can use a chart library such as Recharts. It's optional, so check it supports React 19 when you install it. Or start with plain CSS bars, where each bar's width is a percentage. You might find that's all you need.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 6 (Medium): Pokédex

Browse every Pokémon, page by page, and open any one for its details.

Must-haves:

- A list, 20 per page, with the page in the URL (`?page=3`). Next and Previous buttons.
- A detail page for each one (`/pokemon/pikachu`): picture, types, and stats.
- A type filter (fire, water...) in the URL, too.
- Loading and error states, and the browser's back button works everywhere.

Uses the free PokéAPI: `https://pokeapi.co/api/v2/pokemon?limit=20&offset=40` for a page, `https://pokeapi.co/api/v2/pokemon/pikachu` for one, and `https://pokeapi.co/api/v2/type/fire` for a type.

Nice-to-haves: pick a team of six (saved), compare two side by side, load the next page early when someone hovers over **Next**.

Practices: TanStack Router with validated search params, or React Router (36, 24), TanStack Query with the page in the query key (31), Zod (32), lazy-loaded routes (26), `loading="lazy"` on images (27).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 7 (Challenge): Week Meal Planner

Plan the week's dinners, and get the shopping list for free.

Must-haves:

- Search for recipes and add one to a day, Monday to Sunday.
- Move a meal to another day, or remove it.
- Open a planned meal to see its ingredients and instructions.
- A shopping list built from every planned meal's ingredients, where you can tick things off.
- The plan and the ticks are saved.

Uses TheMealDB, which you know well: `search.php?s=`, `lookup.php?i=` and `random.php`.

Nice-to-haves: drag and drop between days (with buttons as a keyboard alternative), a "surprise me" dinner, a printable shopping list (CSS `@media print`), drinks for the weekend from TheCocktailDB (`https://www.thecocktaildb.com/api/json/v1/1/search.php?s=mojito`).

Practices: TanStack Query, including `useQueries` for several meals at once (31), Zod (32), Zustand or Redux Toolkit for the plan (34, 35), routes (24), accessibility for anything draggable (38).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 8 (Challenge): Your Portfolio Site, in Next.js

A site about you, showing the projects you've built, live on the internet. It's the one project on this menu that will help you every time you apply for a job.

Must-haves:

- A home page: who you are and what you like building.

- A projects page with at least three projects: a screenshot, one sentence each, a live link and a code link.
- A page per project (/projects/bookstore), with what it does and what you learned.
- A way to contact you (a `mailto:` link is fine).
- Deployed, with good Lighthouse scores for performance and accessibility.

Nice-to-haves: a contact form that sends through a Server Function (checked with Zod on the server), a dark theme, a short blog post about one thing you learned.

Practices: Next.js App Router and Server Components (39), page titles and descriptions so search engines understand each page (the reason Next.js fits this one), `next/image`, accessibility (38), deploying (40).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bring your own idea

Have something you'd rather build? Great. That's usually the project you'll finish. Before you start, check it against the notes:

1. **Say it in one sentence**, with no more than one "and".
 2. **Write the must-have list**. Five to eight items. If it's twenty, cut it in half, then in half again.
 3. **Check your API**. Is it free? Does it need a key? (If it needs a *secret* key, you need a server: that means Next.js, or waiting for the Node.js course.) Try one request in your browser before you plan around it.
 4. **Fill in the four-homes table** from step 4. If you can't place something, ask Claude about it before you write any code.
-

When you're done

- **Share the link**. Send it to a friend and watch them use it without helping. Where do they get stuck?
- **Ask Claude for a review** of the whole project, using the request in step 11 as a starting point.
- **Write down three things**: one decision you're glad you made, one you'd make differently, and what you'd build next.

Then pick another idea. Every project you finish makes the next one easier.

Hints

Try each exercise on your own first. Come here only when you're stuck, and read one hint at a time.

01 Getting Started

Exercise 1 (Easy): Your first React app

[Back to the exercise](#)

Hint

Use VS Code's search inside the file (Ctrl + F) to find `Get started`. It's inside an `<h1>`. To start the server again, the terminal must be inside the `playground` folder.

Exercise 2 (Easy): The café sign

[Back to the exercise](#)

Hint

Copy the example exercise file from "How to do these" and change only the text between `<h1>` and `</h1>`. The biggest heading in HTML is `<h1>`.

Exercise 3 (Medium): Follow the trail

[Back to the exercise](#)

Hint 1

Look for `getElementById` in `main.tsx`. What does `getElementById` give back when no element has that id? (You saw this in JavaScript chapter 20 and TypeScript chapter 12.)

Hint 2

"Target container" means the box React was told to draw into. Did React actually get a box? And can TypeScript read `index.html`, or only your `.ts` and `.tsx` files?

Exercise 4 (Medium): Break it on purpose

[Back to the exercise](#)

Hint 1

For step 1, Vite covers the page with an error box that shows the file, the line, and what's wrong. That's because a missing tag is a *syntax* error: Vite can't translate the file at all. In VS Code, hover over the red squiggly line to read the message.

Hint 2

For step 2, reread "Checking your types" in the notes. For step 3, the page may just go blank, with no error box. When that happens, look in the Console (`F12`). Remember `ReferenceError` from JavaScript chapter 02?

Exercise 5 (Challenge): Plain DOM code vs React

[Back to the exercise](#)

Hint 1

For Part 1, you need `querySelector` to find the button, `addEventListener("click", ...)` to react to clicks, and `textContent` to change what the button says. TypeScript will insist that you deal with the button possibly being `null`. TypeScript chapter 12 shows a few ways.

Hint 2

For Part 2, the text inside the button can mix plain words and `{count}`, like `Cups of coffee today: {count}`. Don't forget to change the `import` line in `App.tsx` so it shows `Ex5`.

02 JSX

Exercise 1 (Easy): Fix the library card

[Back to the exercise](#)

Hint 1

Go through the three rules from the notes one at a time: one parent, close every tag, camelCase attributes. Two of the problems are the same kind of mistake, and two are attribute names.

Hint 2

Fix the errors from the top down, and save after each fix. While the *syntax* is broken, TypeScript can't read the file properly, so some errors only appear after you've fixed the ones above them.

Exercise 2 (Easy): Movie ticket

[Back to the exercise](#)

Hint 1

Put the variable names in curly braces: `{seat}`. To turn `12.5` into `12.50`, use `toFixed` from JavaScript chapter 05. And remember, the `$` is just text in JSX.

Hint 2

For the question, think about what `const` promises, and reread the part about literal types in TypeScript chapter 02 or 06.

Exercise 3 (Medium): Product card

[Back to the exercise](#)

Hint 1

For Part 1, a type alias lists each property and its type: `type Product = { name: string; ... };`. The annotation goes after the variable name: `const product: Product = { ... }.`

Hint 2

For Part 2, look at the table "What shows up on the page" in the notes, and the part about `ReactDOM`.

Hint 3

Attributes use braces with no quotes: `src={product.image}`. The style needs double braces, and the value `20` means pixels. For the last line, you need something that picks between two values and is an *expression*.

Exercise 4 (Medium): From HTML to JSX

[Back to the exercise](#)

Hint 1

Use the checklist "From HTML to JSX" at the end of the notes, and go through it line by line. The comment, the `style`, and the `maxLength` are the trickiest parts.

Hint 2

Paste the HTML into your function's `return` first, and let TypeScript show you what's wrong. Remember to read each error's last sentence first.

Exercise 5 (Challenge): Shop receipt

[Back to the exercise](#)

Hint 1

An array of a type is written `CartItem[]`. Then `reduce` can add up the quantities for **Items**, and add up `price * quantity` for **Subtotal**. Start both totals at `0`. Hover over your totals: TypeScript should say they're numbers.

Hint 2

Tax is the subtotal times `taxRate`. The `8` in `Tax (8%)` comes from `taxRate * 100`. Use `toFixed(2)` only when you *show* a money value, because it turns the number into a string. (Try `subtotal.toFixed(2) + tax` and see what TypeScript thinks of the result.)

Hint 3

For the most expensive item, `reduce` can keep whichever item has the higher price so far. You could also sort a copy of the array, but make sure it's a copy (JavaScript chapter 16), so you don't change `cart` itself.

03 Components

Exercise 1 (Easy): Fix the games shop

[Back to the exercise](#)

Hint 1

Start with what the *file* needs before anything else can be shown at all. Then read the "Common mistakes" list in the notes from the top: the first two are both in this file.

Hint 2

`<header />` gives no error, because `header` is a real HTML tag. That's the point of this exercise. And for question 1, look up hoisting in JavaScript chapter 14.

Exercise 2 (Easy): Split the bakery page

[Back to the exercise](#)

Hint 1

Each file follows the same shape: a function with a capital-letter name, a `return` with the JSX, and `export default` at the bottom.

Hint 2

`Header` returns two elements, so it needs one parent. A fragment (`<>` and `</>`) keeps the page identical, because it adds nothing. A `<div>` would work too, but it would change the HTML.

Hint 3

The `&` in "Flour & Salt" is fine in JSX, just like in HTML.

Exercise 3 (Medium): Build a film night poster

[Back to the exercise](#)

Hint 1

Do one component at a time. Write `PosterHeader.tsx`, import it into `Poster.tsx`, and check the page before you write the next one. Fixing one broken file is much easier than fixing five.

Hint 2

The `·` character is just text. Copy and paste it, or use a plain `-` instead.

Hint 3

In the Components tab, `App` is at the top. Your components are underneath it. Plain HTML tags like `<h1>` aren't shown by default, so your drawing will be shorter than the one in the notes.

Exercise 4 (Medium): Three bad habits

[Back to the exercise](#)

Hint 1

For the number, read "A component should be predictable" in the notes, and look for `StrictMode`.

Hint 2

For the Components tab question, read mistake 2 in the notes. React only knows a component exists if you use it as a *tag*.

Exercise 5 (Challenge): Bike shop dashboard

[Back to the exercise](#)

Hint 1

`filter` gives you a smaller array, and `.length` tells you how big it is. That handles "Finished" and "Still to do". For the money, filter first and then `reduce`, or do both in one `reduce`.

Hint 2

To show nothing from a component, `return null;`. Look at "Returning nothing on purpose" in the notes. Returning `<></>` would also look right on the page, but `null` says what you mean.

Hint 3

Both `JobStats` and `PendingAlert` import the same `repairs` array. That's fine for now, but notice how it feels: two components reaching into the same file to get their data. From chapter 04, the parent will hand data to them instead, which is the React way round.

Hint 4

For the weekday, `new Date().toLocaleDateString("en-GB", { weekday: "long" })` gives you the day's name.

04 Props

Exercise 1 (Easy): The name badge

[Back to the exercise](#)

Hint 1

The shape is in the notes under "Typing props" and "Destructuring props". The type goes above the function, and the annotation goes after the destructured object: `function Badge({ name, role }: BadgeProps)`.

Hint 2

For number 4, it's worth noticing *why* that's an error. TypeScript won't let you pass a prop the component never asked for, because it's almost always a typo.

Exercise 2 (Easy): The menu, finished at last

[Back to the exercise](#)

Hint 1

`toFixed(2)` from JavaScript chapter 05 turns `2` into `"2.00"`.

Hint 2

For the Vegan line, you only know one way to choose between two things so far: the ternary from chapter 02. Show the JSX when it's true, and `null` when it's false. ([Chapter 05](#) has neater ways, and this is exactly the problem it solves.)

Exercise 3 (Medium): A button you can reuse

[Back to the exercise](#)

Hint 1

`"small" | "large"` is a union of two literal types, from TypeScript chapter 06. It's better than `string` here because it means nothing but those two words can ever be passed.

Hint 2

Defaults go in the destructuring: `function Button({ label, colour = "steelblue", size = "small", disabled = false }: ButtonProps)`. That's a long line. Breaking it across several lines is fine and normal.

Hint 3

Work the font size out **above** the `return`, into a `const`, then use it in the style object. Keep the JSX for showing things.

Exercise 4 (Medium): A card that holds anything

[Back to the exercise](#)

Hint 1

The type is `ReactNode`, and it comes from React: `import type { ReactNode } from "react";`. See "The `children` prop" in the notes.

Hint 2

`children` is never written as an attribute. You put the content between `<Card>` and `</Card>`, and React fills the prop in for you.

Hint 3

CSS property names are camelCase in JSX, so it's `borderRadius`, `marginBottom`, and `border: "1px solid grey"`.

Exercise 5 (Challenge): Library checkout page

[Back to the exercise](#)

Hint 1

For Part 1, the prop type is a one-line wrapper around the type you already have:

```
type BookRowProps = {
  book: Book;
};
```

Import the type with `import type { Book } from "../data.ts";`.

Hint 2

For `LateNotice`, work out the fine and pick the colour **above** the `return`, then use a ternary for the text. Remember `0` is falsy in JavaScript, but be explicit here: `daysLate === 0` says what you mean.

Hint 3

For Part 2, `readonly Book[]` is the type. `filter(b => b.daysLate > 0).length` gives you the overdue count, and `reduce` adds up the fines. Work out the fine per book the same way in both components, or better, put that sum in one small function in `data.ts` and import it into both.

Hint 4

For Part 4, question 2: make a copy first (JavaScript chapter 16). `[...books].sort(...)` sorts the copy and leaves the parent's array alone. Modern JavaScript also has `toSorted()`, which returns a new sorted array and never touches the original.

05 Conditional Rendering

Exercise 1 (Easy): The stray zero

[Back to the exercise](#)

Hint 1

Read "The `0` trap" in the notes. Then look at the table of what React shows for each value, back in [chapter 02](#).

Hint 2

For question 3, think about what `&&` gives back when the left side is falsy, and then what React does with *that particular* falsy value. `0` and `""` are both falsy, but React treats them differently.

Exercise 2 (Easy): Train departure board

[Back to the exercise](#)

Hint 1

Three cases for the status line means the ternary is going to get ugly. Look at "Option 5: a variable that holds JSX" in the notes.

Hint 2

For the platform, you want a fallback **value**, not a different element. `??` does that in one character pair. Read the last section before "Common mistakes".

Exercise 3 (Medium): One card, four states

[Back to the exercise](#)

Hint 1

"In order" is the clue. Early returns handle "in order" beautifully: check the first case, return, check the next, return. No `else` needed anywhere.

Hint 2

For the question, read "TypeScript and narrowing" in the notes. Each early return removes one possibility from what's left below it.

Hint 3

Note that `temperature` can be `0` or below. Be careful with any condition you write about it, and don't put it on the left of an `&&`.

Exercise 4 (Medium): Order status badge

[Back to the exercise](#)

Hint 1

For Part 2, you need two things per status, so the object's values are little objects: `{ label: "...", colour: "..." }`. Give that shape a type alias of its own.

Hint 2

`Record<OrderStatus, Style>` is the type that says "one entry for every status". It's in the notes under "Option 6", and in TypeScript chapter 09.

Exercise 5 (Challenge): Library account page

[Back to the exercise](#)

Hint 1

Build it one rule at a time and check the page after each. Five rules at once is five bugs at once.

Hint 2

For rule 3, that's a whole different page, not a missing line. Early return.

Hint 3

Watch out in rule 4: `loans.length` and the overdue count are both numbers, and both could be `0`. Neither belongs on the left of an `&&` on its own.

Hint 4

For the Renew button, two conditions have to be true at once. You can `&&` them together inside the brackets: `{a && b && <button>...</button>}`. If that gets hard to read, work out a `const canRenew = ...` above the `return` and use that instead. The second version is usually nicer, because the name explains the rule.

06 Rendering Lists

Exercise 1 (Easy): Your first list

[Back to the exercise](#)

Hint 1

For question 6, read mistake 4 in the notes. This is why a name is only a safe key when it really is unique.

Hint 2

For the filtered list, `includes` tells you whether a string contains another one (JavaScript chapter 06). Chain it: `.filter(...).map(...)`, or make a `const` above the `return`.

Exercise 2 (Easy): The café menu, for real

[Back to the exercise](#)

Hint 1

For question 2, the empty check belongs inside `Menu`, so every menu gets it for free. An early return is the cleanest way.

Hint 2

For question 4, see "Careful with `sort`" in the notes. You need a copy: `[...drinks]` or `toSorted`.

Exercise 3 (Medium): Prove the key bug

[Back to the exercise](#)

Hint 1

If step 2 reloads the whole page and clears your tick, add the task while the page is still open and watch carefully. If it still resets, tick the box again after the save and then move a task from the front to the end instead.

Hint 2

The tick box isn't in your array anywhere. That's the point: the browser is keeping it, attached to a particular `<li>`. React decides which `<li>` is "the same one as before", and the key is how it decides.

Exercise 4 (Medium): Grouped shopping list

[Back to the exercise](#)

Hint 1

For the total, you need to reach inside each aisle. `flatMap` gives you one flat array of all the items, and then `reduce` adds the quantities up. Or `reduce` over the aisles and `reduce` again inside. Either is fine.

Hint 2

`(1 item)` versus `(2 items)` is a ternary on `items.length === 1`. Careful with the "must not appear at all" case: that's a third possibility, so consider working the whole label out above the `return`.

Hint 3

Each `map` needs its own keys, and they only have to be unique among their own siblings. The `id`s in this data are unique across the whole file anyway, which is common but not required.

Exercise 5 (Challenge): Leaderboard

[Back to the exercise](#)

Hint 1

Do the work above the `return`, in stages, with a named `const` for each: the finishers, the sorted list, the unfinished ones. Don't try to do it all in one chain.

Hint 2

For the shared positions: a runner's position is "how many runners are strictly faster than me, plus one". That rule gives you the skipping behaviour for free, and it's easier than tracking a counter.

Hint 3

For two-digit seconds, `String(seconds).padStart(2, "0")` (JavaScript chapter 06). `Math.floor(total / 60)` gives the minutes and `total % 60` the seconds.

Hint 4

For `ClubTally`, `reduce` into an object gives you `{ "Mill Lane": 3, Riverside: 1 }`. Then `Object.entries` turns that object into an array of `[name, count]` pairs, which you can `map` over. Each pair is a tuple (TypeScript chapter 03), so you can destructure it right in the callback. A `Map` works too (JavaScript chapter 35).

Hint 5

For the comma-separated names in rule 4, you don't need `map` at all. `join(", ")` on an array of names does it in one go.

07 Events

Exercise 1 (Easy): Four buttons, one bug

[Back to the exercise](#)

Hint 1

For question 4, think about what each expression *evaluates to*, and what `onClick` is willing to accept.

`handleHello()` gives back `undefined`; `greet("Tom")` also gives back `undefined` ... so look more carefully at what TypeScript says, and at what the two functions' return types actually are.

Hint 2

`onClick` accepts a function **or** `undefined` (because the prop is optional). That's the whole answer to question 4, and it's a good lesson: TypeScript can't save you from every version of this mistake. You have to recognise the shape.

Exercise 2 (Easy): The vending machine

[Back to the exercise](#)

Hint 1

The prop type is `onBuy: (name: string, price: number) => void;`. The `=> void` part means "returns nothing worth using".

Hint 2

Inside `ProductButton`, you need to pass arguments, so the handler has to be wrapped: `onClick={() => onBuy(name, price)}`.

Exercise 3 (Medium): Event detective

[Back to the exercise](#)

Hint 1

For Part 2, look up `stopPropagation` in the notes. The handler needs the event object now, so it needs a parameter.

Hint 2

For Part 3, the button's handler type is `React.MouseEvent<HTMLButtonElement>`. `currentTarget` is where the handler is; `target` is where the click actually landed.

Hint 3

For Part 4, use `onKeyDown` and check `event.key === "Enter"`. The type is `React.KeyboardEvent<HTMLInputElement>`.

Exercise 4 (Medium): A form that doesn't reload the page

[Back to the exercise](#)

Hint 1

For question 1, the Console clears itself when the page reloads — that's your clue. Turn on **Preserve log** in the Console's settings if you want to see the message before it disappears.

Hint 2

The handler's type is `React.FormEvent<HTMLFormElement>`, and the first line of its body is `event.preventDefault();`.

Hint 3

You can't read what was typed yet — that's [chapter 09](#). Logging a fixed message is enough here.

Exercise 5 (Challenge): A to-do list that talks back

[Back to the exercise](#)

Hint 1

Three function props on `TaskRow`, all the same shape:

```
type TaskRowProps = {
  task: Task;
  onToggle: (id: string) => void;
  onDelete: (id: string) => void;
  onEdit: (id: string) => void;
};
```

Hint 2

For requirement 1, that's `stopPropagation` on each button. You'll need the event, so the handlers become `(event) => { ... }` with two statements.

Hint 3

For handlers that log the task's **text** from just an id, the parent has the array: `tasks.find((task) => task.id === id)`. That returns `Task | undefined`, so TypeScript will make you handle the "not found" case (TypeScript chapter 06). That's not TypeScript being annoying — `find` really can come back empty.

Hint 4

The tick box will look stuck: you click it and it snaps straight back. That's correct for now, and it's worth sitting with for a second. `checked={task.done}` ties the box to data that never changes, so React puts it right back where the data says it should be. It's doing exactly what you asked. Chapters 08 and 09 give you the missing half.

If instead you see a warning about a read-only field, you used `onClick` rather than `onChange` on the tick box.

08 State

Exercise 1 (Easy): The counter, three ways

[Back to the exercise](#)

Hint 1

For question 3, read "State updates are not instant" in the notes. All three lines are working from the same `count`.

Hint 2

For question 6, you can either check inside the handler before setting, or clamp the value with `Math.max(0, ...)`. Both are fine. The second one is harder to get wrong later.

Exercise 2 (Easy): Light switch and secret message

[Back to the exercise](#)

Hint 1

To flip a boolean: `setIsOn(!isOn)`, or better, `setIsOn((prev) => !prev)`.

Hint 2

For the step counter, store the **index** in state (a number), not the text. The text is a derived value: `steps[index]`.

Hint 3

For the final question, look at mistake 9 in the notes, and think about which of your three values ever change at the same moment.

Exercise 3 (Medium): Shopping cart

[Back to the exercise](#)

Hint 1

The cart item type needs both its own id and the product's details:

```
type CartItem = {  
  id: string;           // unique to this line in the cart  
  product: Product;  
};
```

Then `useState<CartItem[]>([])`.

Hint 2

Adding: `setItems([...items, newItem])` or `setItems((prev) => [...prev, newItem])`. Removing: `setItems(items.filter((item) => item.id !== id))`. Both make a **new** array, which is the whole point.

Hint 3

The total is `reduce` over the items, adding `item.product.price`. Work it out above the `return`, in a `const`.

Exercise 4 (Medium): Quiz

[Back to the exercise](#)

Hint 1

Three pieces: which question you're on (a number), which option is selected for the current question (`number` | `null` — `null` means "not answered yet"), and the score so far (a number).

The score could arguably be derived if you stored every answer, and that's a good design too. Try the simple version first.

Hint 2

`selectedIndex === null` is your "has this been answered?" test, and it drives almost everything on screen: whether the buttons work, whether colours show, whether Next appears. Work out a `const isAnswered = selectedIndex !== null;` and use that.

Hint 3

For the colours, write a small function above the `return` that takes an option's index and returns its colour. That keeps the ternaries out of the JSX and stops them nesting.

Hint 4

For **Try again**, you're setting all three pieces back at once. Three setter calls in one handler is fine — React batches them into a single re-render.

Exercise 5 (Challenge): Seat booking

[Back to the exercise](#)

Hint 1

The two pieces of state are: the selected seat ids (a `string[]`), and whether the booking is confirmed (a `boolean`). The count, the total, the "is this seat selected?" test, and "can I select more?" are all derived.

Hint 2

To build the rows: `const rows = ["A", "B", "C", "D"];` and `const seatNumbers = [1, 2, 3, 4, 5, 6];`. A seat's id is `row + number`. Map the rows on the outside and the numbers on the inside, with a key on each.

Hint 3

The toggle is one `if / else`, and both branches make a **new** array:

```
if (selected.includes(id)) {
  setSelected(selected.filter((seat) => seat !== id));
} else {
  setSelected([...selected, id]);
}
```

Hint 4

For requirement 5, don't sort the state — sort a **copy** when you display it: `[...selected].sort().join(", ")`. Sorting the state itself would work by accident here, but it's the mutation habit you're trying to break, and `sort` on state is a real bug waiting for a less lucky day.

Hint 5

For requirement 6, the "booked seats stay taken" extra needs `takenSeats` to become state too — which is a third piece, and a legitimate one, because it genuinely can't be derived from anything. Getting that judgement right is the point of requirement 7: the rule isn't "as few as possible", it's "none that could be worked out".

09 Forms

Exercise 1 (Easy): Feel the loop

[Back to the exercise](#)

Hint 1

For question 3, read the four-step loop in the notes. The letter in the box isn't the one the browser put there.

Hint 2

For question 5: the greeting will work, and Clear will not. That's the difference between "React is reading the box" and "React is driving the box".

Exercise 2 (Easy): Sign-up form

[Back to the exercise](#)

Hint 1

The checkbox is `checked={agreed}` and `onChange={(e) => setAgreed(e.target.checked)}`. Not `value`.

Hint 2

Work out `const canSubmit = ...` above the `return` and use `disabled={!canSubmit}`. Three conditions in the JSX would be unreadable.

Hint 3

For the Clear button inside a `<form>`, see mistake 8 in the notes. The default `type` of a button inside a form is `submit`.

Exercise 3 (Medium): Pizza order

[Back to the exercise](#)

Hint 1

For toppings, `useState<string[]>([])`. Each checkbox's `checked` is `toppings.includes(name)`, and its handler toggles: filter it out if it's there, spread it in if it isn't. Same shape as the seat picker in [chapter 08](#).

Hint 2

For the prices, a lookup object keyed by the option name beats a chain of ternaries. That's back in [chapter 05](#), and `Record<Size, number>` types it so you can't forget one.

Hint 3

For requirement 3, read the warning about number inputs in the notes. Keep the state as a **string**, and convert where you calculate: `Number(quantity) || 0`, or check for `""` explicitly.

Hint 4

Six fields is past the point where separate `useState` calls feel good. Try it that way first, then look at the object-plus-one-handler version in the notes and decide which you'd rather come back to in a month. There isn't a wrong answer; noticing the trade-off is the exercise.

Exercise 4 (Medium): Searchable, filterable list

[Back to the exercise](#)

Hint 1

This is the big idea of the exercise: filtering is a **derived value**. Above the `return`, chain `filter` and then sort a copy. Every keystroke re-renders, so it's always right, and there's nothing to keep in sync.

Hint 2

`title.toLowerCase().includes(search.toLowerCase())` handles the case-insensitive match, and an empty search matches everything for free, because every string includes `""`.

Hint 3

`useState<Genre | "all">("all")` types the select. Then a wrong `<option value="fictoin">` is still not caught (option values are plain strings), but every *use* of the state is.

Hint 4

Reset is one handler that calls all four setters. React batches them into one re-render.

Exercise 5 (Challenge): Multi-step booking form

[Back to the exercise](#)

Hint 1

For requirement 5, one object per step is a natural fit: `who`, `when`, and `extras`. Each gets its own type and its own `useState`. That also makes "is this step valid?" three small, separate functions.

Hint 2

Write one `const isValid = ...` above the `return`, worked out from the current step. Then `disabled={!isValid}` on Next, and the button logic never changes.

Hint 3

For "touched", a `Set` of field names is tidier than a boolean each (JavaScript chapter 35). Remember to make a **new** Set when you add to it: `setTouched(new Set(touched).add("email"))`. A `string[]` works just as well if a Set feels like a stretch.

Hint 4

Requirement 3 comes free if you build it right, and that's worth noticing. Because all the answers live in state in the parent, moving between steps is only changing which step number is showing. Nothing is unmounted and re-created, so nothing can be lost. Compare that with how much work this would have been in JavaScript chapter 22.

Hint 5

Build it one step at a time, and get the whole flow working with step 1 alone before you add step 2. A three-step form built all at once is a three-step form you can't debug.

10 Project: To-Do App, the React Way

Exercise 1 (Easy): Dark mode

[Back to the exercise](#)

Hint 1

One boolean in state, and a derived class name: `const appClass = isDark ? "app dark" : "app";`

Hint 2

For saving, `localStorage` only stores strings. `String(isDark)` going in, and `saved === "true"` coming out, is the simplest honest version. Use the lazy initial state form again.

Exercise 2 (Easy): Tick everything at once

[Back to the exercise](#)

Hint 1

`tasks.every((task) => task.done)` tells you whether they're all done. One `map` handles both directions:

```
const allDone = tasks.every((task) => task.done);
updateTasks(tasks.map((task) => ({ ...task, done: !allDone })));
```

Exercise 3 (Medium): Honest empty states

[Back to the exercise](#)

Hint 1

An `emptyMessage: string` prop is the straightforward version. `emptyMessage: ReactNode` ([chapter 04](#)) lets `App` pass whole JSX, which is more flexible and about the same amount of code.

Hint 2

Working out the message in `App` is a chain of conditions that's easier as an `if / else if` into a variable than as nested ternaries ([chapter 05](#)).

Exercise 4 (Medium): Edit a task

[Back to the exercise](#)

Hint 1

The first real decision: does "which task is being edited" live in `TaskItem` or in `App`? Requirement 4 answers it. If each row kept its own flag, two rows could both be editing. Keep `editingId: string | null` in `App`.

Hint 2

The half-typed text is different. Only the row being edited cares about it, so a `useState` inside `TaskItem` is right — start it from `task.text`.

Hint 3

`onDoubleClick` on the `<span>`, and `onKeyDown` on the input checking `event.key === "Enter" or "Escape"`. `onBlur` handles "clicking away".

Hint 4

Getting the cursor into the box automatically needs `autoFocus` on the input. That works fine here. The general tool for "do something to a real element on the page" is a ref, in [chapter 19](#).

Hint 5

For requirement 5, `stopPropagation` ([chapter 07](#)) — or simply put the `onDoubleClick` only on the text `<span>`, not the whole `<li>`, which avoids the problem instead of fixing it. The second is better. Prefer not creating a conflict over handling one.

Exercise 5 (Challenge): Undo

[Back to the exercise](#)

Hint 1

The simplest design that satisfies all five rules: keep one piece of state holding the previous **whole task list**, plus a message.

```
type UndoState = {
  tasks: Task[];
  message: string;
};

const [undo, setUndo] = useState<UndoState | null>(null);
```

Before a destructive change, store the current array. To undo, put it straight back. Requirement 2 comes free, because you saved the whole thing.

Hint 2

That works because you've never changed an array in place. The old array is still intact and correct — you only ever built new ones alongside it. Keeping a snapshot of mutable data would have been far harder. This is the payoff for a rule that has felt like a chore all chapter.

Hint 3

For requirement 3, clear the undo state at the top of every other handler. Rather than remembering it in four places, call `setUndo(null)` inside `updateTasks`, and have the destructive handlers set it *after* they call `updateTasks`.

Hint 4

If you want the bar to fade away on its own after a few seconds, you need a timer that's cleaned up properly when the component goes away — and that's exactly what [chapter 17](#) is for. Leave it manual for now.

11 Updating Objects and Arrays in State

Exercise 1 (Easy): Fix the settings form

[Back to the exercise](#)

Hint 1

Two separate problems are stacked here: `settings.notifications.email = ...` mutates a nested object, and `setSettings(settings)` passes back the exact same reference it started with. Read "Why React needs a new reference" in the notes.

Hint 2

The fix needs two spreads, nested: a new `notifications` object, inside a new `settings` object.

Exercise 2 (Easy): Shopping cart quantities

[Back to the exercise](#)

Hint 1

`map`, with a spread on the matching item: `items.map((item) => (item.id === id ? { ...item, quantity: item.quantity + 1 } : item))`.

Hint 2

For the floor of 1, `Math.max(1, item.quantity - 1)` is tidier than an `if` inside the map.

Exercise 3 (Medium): Contact book with addresses

[Back to the exercise](#)

Hint 1

For requirement 3, the `map` callback needs an `if` inside it (or a ternary): change `country` only when `address.country === "UK"`, and return the contact completely unchanged otherwise — not even wrapped in a

```
fresh { ...contact }.
```

Hint 2

For the sort, `[...contacts].sort((a, b) => Number(b.favourite) - Number(a.favourite))` puts `true` (favourited) first. `Number(true)` is `1`.

Exercise 4 (Medium): Kanban board

[Back to the exercise](#)

Hint 1

Work out the destination column's id first (a small lookup: `{ todo: "doing", doing: "done" }`), then build the new `columns` array with one `map`: for the source column, filter the card out; for the destination column, add it; every other column, leave completely alone.

Hint 2

You need the *card itself* before you filter it out, or you'll have nothing to add to the destination column. Find it first with `.find()`, and keep in mind `find` can come back `undefined` — TypeScript chapter 06 will make you handle that.

Exercise 5 (Challenge): Undo, properly

[Back to the exercise](#)

Hint 1

Two extra pieces of state: `past: Column[][]` and `future: Column[][]`. Every "real" change does three things: push the current `columns` onto `past`, apply the change, clear `future`.

Hint 2

Undo: take the last entry off `past`, push the *current* `columns` onto `future`, and set `columns` to that entry. Redo is the mirror image. `slice(0, -1)` and `at(-1)` (or `[array.length - 1]`) are useful here, and both make copies rather than mutating.

Hint 3

For the final question: if an old update had ever done `card.text = newText` instead of building a new card, then the copy of `columns` sitting in `past` would contain that *same* card object — so mutating it "later" would silently rewrite history too. The whole exercise only works because every past snapshot is made of objects nothing will ever touch again.

12 How Rendering Works

Exercise 1 (Easy): Predict the log

[Back to the exercise](#)

Hint 1

Read "The snapshot: state is fixed for the whole render" in the notes. `count` inside `handleClick` was fixed the moment this render's `Ex1` function ran — nothing that happens afterwards, however long you wait, can change it.

Exercise 2 (Easy): Three clicks, one render

[Back to the exercise](#)

Hint 1

For question 3, read "Batching" in the notes. Three `setCount` calls inside one handler cause **one** re-render, not three — so your render log should print once per click, not three times, regardless of how many `setCount` calls are inside the handler.

Exercise 3 (Medium): The reluctant child

[Back to the exercise](#)

Hint 1

For question 3, React only touches the parts of the real DOM that actually changed between renders.

`Timestamp`'s output is identical every time, so even though its function runs, nothing about it needs updating on screen.

Hint 2

For question 4: think about which component's state changed, and which components are *nested inside* that one.

`AnotherCounter` is a sibling, not a child, of the button that changed — is it inside the component whose state changed, or outside it?

Exercise 4 (Medium): The stale closure, on purpose

[Back to the exercise](#)

Hint 1

A plain `let` inside the component body gets **reassigned** by every render and every `onChange`, so by the time the timeout fires 3 seconds later, it's reading whatever the *latest* value happens to be — not the value from when you clicked. That's the opposite of what a snapshot gives you.

Hint 2

State works differently because the handler that started the timeout was created during a specific render, and it closes over that render's `const` value of the state — which can never change, no matter what happens to state afterwards. That's exactly the mechanism from Exercise 1.

Exercise 5 (Challenge): Build a tiny render logger

[Back to the exercise](#)

Hint 1

You need something that survives across renders **without** itself triggering a re-render when it changes — which rules out `useState`. That's exactly what a ref is for, and you're about to meet them properly in [chapter 19](#). For now:

```
import { useRef } from "react";

function useRenderCount() {
  const count = useRef(0);
  count.current = count.current + 1;
  return count.current;
}
```

Hint 2

For question 3, remember `StrictMode` double-invokes your component's render in development. Watch what the count does on the very first paint versus what it does on a click afterwards — is the "extra" render a permanent, ongoing thing, or a one-off?

13 Lifting State Up

Exercise 1 (Easy): Two clocks, one truth

[Back to the exercise](#)

Hint 1

Once `brightness` lives in `Ex1`, both `BrightnessSlider` and `BrightnessPreview` (and now `BrightnessLabel`) just take it as a prop. Only `BrightnessSlider` also needs a callback prop, because it's the only one that changes it.

Exercise 2 (Easy): Search box and results, properly

[Back to the exercise](#)

Hint 1

The filtering itself is a derived value in `Ex2`, computed from `books` and `query` and passed down as the already-filtered array — `ResultsList` shouldn't need to know `query` exists at all. Compare that with `ResultCount`, which needs `query` itself (for the message) as well as the count.

Exercise 3 (Medium): Tabs with a shared "unsaved changes" flag

[Back to the exercise](#)

Hint 1

Three things live in the parent: which tab is active, `hasUnsavedChanges`, and — since `Save` needs to know what to save — the form values themselves (or at minimum, the parent's callback props are how each tab reports "something changed").

Hint 2

`window.confirm` returns a boolean, synchronously — no promise, no callback (JavaScript chapter 18 has a similar pattern with `try / catch` you can compare it to, though this one's much simpler). Use its return value directly to decide whether to switch.

Exercise 4 (Medium): Two sliders that must add up to 100

[Back to the exercise](#)

Hint 1

Store just one number, say `cardio`. `strength` is `100 - cardio`, always — never stored. There's genuinely only one independent piece of information here, which is exactly the situation from the temperature converter in the notes.

Hint 2

Both sliders can be built from the same underlying component if you like — one showing `cardio` and calling `setCardio(value)`, the other showing `100 - cardio` and calling `setCardio(100 - value)` — but two separate, slightly different JSX blocks are completely fine too.

Exercise 5 (Challenge): Master-detail with editing

[Back to the exercise](#)

Hint 1

`ContactApp` looks up the selected contact with `contacts.find(...)`, which can come back `undefined` — that's requirement 5, and it's the same "find can fail" situation from earlier chapters. Pass `null` down to `ContactDetails` when there's no match, and let it handle that case itself, the same way you handled an empty list in [chapter 05](#).

Hint 2

`ContactDetails`'s prop for reporting a change is one function: `onChange: (updated: Contact) => void`. It builds the updated contact with a spread and calls `onChange` — `ContactApp` is the one that knows how to fold that into the array with `map`, from [chapter 11](#).

Hint 3

For requirement 2's cursor placement: `autoFocus` works nicely for "the field that was just created should start focused," but be aware it's a slightly blunt tool — it fires on mount, not on "this specific contact became selected." If the name field seems to steal focus at odd moments once you've built the whole thing, that's a real limitation you're seeing, not a mistake — it's part of what makes refs, in the next chapter but two, worth learning properly.

14 Thinking in React

Exercise 1 (Easy): A rating widget

[Back to the exercise](#)

Hint 1

Two pieces of state: the committed `rating` (a number), and a separate `hoveredStar: number | null` for the preview. What's actually drawn — "is this star filled?" — is derived from whichever of the two is relevant: use the hover value if it's not `null`, otherwise the real rating.

Exercise 2 (Easy): A shopping list with a running total

[Back to the exercise](#)

Hint 1

The only real state is the list of `{ name, quantity }` items. The price per line and the total are both derived — the price by looking the name up in your lookup object, the total by summing across the list.

Exercise 3 (Medium): A filterable, sortable staff directory

[Back to the exercise](#)

Hint 1

For step 3, sort column and sort direction could be one piece of state (a string like `"name-asc"`) or two. Either is defensible — the point of the exercise is noticing there's a decision to make and writing down which you picked and why, not landing on one particular answer.

Hint 2

Chain the derived list the same way as in the notes: filter by department, then filter by search text, then sort a **copy** ([chapter 06](#)) — never the original `employees` array.

Exercise 4 (Medium): A multi-step design, given only the picture

[Back to the exercise](#)

Hint 1

Whether a card is flipped is state that resets every time you move to the next card — it's not something that survives across cards. Think about whether that argues for one `isFlipped` flag reset on each "next," or something derived from "which card index have I flipped."

Hint 2

The score is a case worth thinking about carefully. You could store one running number, incremented on "Got it right." You could instead store every answer given, as an array, and derive the score by filtering it. The second is more data but makes "review your wrong answers" trivial to add later. Pick one, and say in a comment what the other approach would have cost or bought you.

Exercise 5 (Challenge): Redesign something you already built

[Back to the exercise](#)

Hint 1

The account's list of loans has to become state now — it's no longer a fixed prop, since Renew and Return both change it. That state has to live somewhere all three things that touch it (the filter box, the renew buttons, the return buttons) can reach — likely higher up than where any individual loan row lives.

Hint 2

The filter text and the underlying loans array are two different pieces of state with two different owners to think about, even though they end up combined into one derived, displayed list — exactly like the search-and-filter example in the notes.

15 Styling

Exercise 1 (Easy): Make the collision happen, then fix it

[Back to the exercise](#)

Hint 1

Plain CSS files are just concatenated into one global stylesheet, in the order they're imported. Whichever `.title` rule appears later in that combined file wins, by ordinary CSS rules — it has nothing to do with which component "seems" more specific.

Exercise 2 (Easy): A styled button component, three ways

[Back to the exercise](#)

Hint 1

Inline styles can't express `:hover` at all — that's a CSS-only feature. If you want hover, it either needs a real CSS rule (which is one good argument for CSS Modules) or a `useState` tracking whether the mouse is over the button, using `onMouseEnter` / `onMouseLeave` from [chapter 07](#).

Exercise 3 (Medium): Find the bug in someone else's styles

[Back to the exercise](#)

Hint 1

One mistake is a typo in a module property name. One is a plain string used where a module lookup was needed. One is a plain string used where a module lookup was needed, in the *other* direction. Go line by line through the JSX.

Exercise 4 (Medium): Theming with CSS variables

[Back to the exercise](#)

Hint 1

If you find yourself writing `style={{ background: theme === "dark" ? "#111" : theme === "light" ? "#fff" : "#000" }}` in your `.tsx` file, you've reimplemented what the CSS variables were supposed to do for you. The component should only ever choose a *class name*, never a colour.

Exercise 5 (Challenge): Build a reusable `cx` and a small design-system button

[Back to the exercise](#)

Hint 1

Type the parameter as `Array<string | false | null | undefined>` (or the equivalent rest-parameter form, `...classes: (string | false | null | undefined)[]`) so TypeScript allows exactly the kinds of expressions you'll pass it: plain strings, and `condition && "class-name"` expressions, which can be `false` as well as a string.

Hint 2

`.filter(Boolean)` removes every falsy value from an array in one call, because `Boolean` returns `false` for `""`, `0`, `null`, `undefined`, and `false` itself, and `filter` keeps only the entries where the callback returns something truthy.

16 Tailwind CSS

Exercise 1 (Easy): Convert a card, class by class

[Back to the exercise](#)

Hint 1

`border`, `border-grey-300`, `rounded-xl`, `p-5`, `bg-white`, `shadow-sm`, `max-w-xs` gets you most of the card shell. Check Tailwind's colour and spacing scale pages for the closest matches to the exact hex values and pixel sizes above.

Exercise 2 (Easy): A responsive nav bar

[Back to the exercise](#)

Hint 1

`flex flex-col items-center sm:flex-row sm:items-center sm:justify-between` on the wrapping element is the whole layout. Everything else is spacing and colour.

Exercise 3 (Medium): A tab bar with conditional classes

[Back to the exercise](#)

Hint 1

This is nearly identical in shape to the `FilterButtons` component from the [to-do app project](#) — same "which one is selected" state, same idea, just with Tailwind classes instead of a CSS Modules `active` class.

Exercise 4 (Medium): Dark mode toggle

[Back to the exercise](#)

Hint 1

This is the same React logic as [chapter 15, exercise 4](#) — state, `localStorage`, a class toggled on a wrapper. The only thing that's different is that the CSS side of it is Tailwind's `dark:` prefixes instead of your own `:root` variables.

Exercise 5 (Challenge): A small design-system Button, the Tailwind way

[Back to the exercise](#)

Hint 1


```
const variantClasses: Record<ButtonProps["variant"], string> = {
  primary: "bg-blue-600 text-white hover:bg-blue-700",
  secondary: "bg-grey-200 text-grey-900 hover:bg-grey-300",
  outline: "border border-grey-300 text-grey-900 hover:bg-grey-50",
};
```

`Record<ButtonProps["variant"], string>` reaches into the props type to reuse the union, rather than retyping `"primary" | "secondary" | "outline"` a second time — the same `keyof` /indexed-access idea from [chapter 11](#).

Hint 2

Tailwind has `disabled:opacity-50 disabled:cursor-not-allowed` variants that key off the real HTML `disabled` attribute automatically — no manual `isDisabled ? ... : ...` branch needed for those specific styles, only for the `disabled` attribute itself.

17 Effects

Exercise 1 (Easy): Three dependency arrays

[Back to the exercise](#)

Hint 1

Remember every click that changes state causes the whole component to re-render, whichever button was clicked. "No array" and "watching A" both react to *every* render, just for different reasons — read the notes on what an empty array versus no array versus a filled array each actually promise.

Exercise 2 (Easy): Document title sync

[Back to the exercise](#)

Hint 1

`document.title = ...` is changing something in the browser that exists completely outside anything React renders — there's no JSX for a browser tab. That's the signal that this belongs in an effect rather than being calculated during render.

Exercise 3 (Medium): Build a leaking timer, then fix it

[Back to the exercise](#)

Hint 1

For question 1: every time `isRunning` changes, this effect runs *again*, starting a brand-new interval — and the old one is never stopped. Toggle it five times and you have five intervals all firing at once.

Hint 2

For question 4, the simplest fix is an early return inside the effect: if `isRunning` is `false`, don't start an interval at all, just return with nothing scheduled (and nothing to clean up).

Exercise 4 (Medium): The race condition, caught in the act

[Back to the exercise](#)

Hint 1

The effect depends on `userId`. Clicking the second button changes `userId`, which runs the cleanup for the *first* effect (setting its `ignore` to `true`) before starting the new one — even though the first request's `.then()` hasn't resolved yet. When it finally does resolve, its own `ignore` check stops it from calling `setUser`.

Exercise 5 (Challenge): An effect that shouldn't be one

[Back to the exercise](#)

Hint 1

Every one of these four `useEffect` calls is the "calculating something you already have everything you need for" mistake from the notes, four times over. None of them touch anything outside React — they only ever read `items`, which is already sitting right there as a prop.

Hint 2

For question 1's render count: the *first* render shows the default state (`itemCount: 0`, `total: 0`, etc.) while `items` already has real data in it — briefly showing wrong numbers — and then each of the four effects fires afterwards, each one calling its own `set`, potentially causing further renders on top of that. Watching it directly is far more convincing than being told the number.

18 Fetching Data

Exercise 1 (Easy): Your first fetch, all three states

[Back to the exercise](#)

Hint 1

The response from `random.php` has the shape `{ meals: [{ idMeal, strMeal, strMealThumb, ... }] }` — an array with exactly one item, not a single object directly. Don't skip actually looking at a real response; guessing the shape and getting it slightly wrong is a very true-to-life mistake.

Exercise 2 (Easy): Search as you type, with debounce

[Back to the exercise](#)

Hint 1

`data.meals ?? []` turns `null` into an empty array in one step — the same nullish-coalescing tool from [chapter 05](#), doing real work here.

Exercise 3 (Medium): Catch your own race condition

[Back to the exercise](#)

Hint 1

This is deliberately the same shape as the `UserProfile` example in [chapter 17](#) — a fetch inside an effect, keyed on a piece of state that can change again before the first request finishes.

Hint 2

For question 5: with `AbortController`, the Network tab shows the first (now-unwanted) request's status as "(cancelled)" almost immediately after the second one starts. With the `ignore` flag alone, that first request keeps running to completion in the background — it just gets ignored once it arrives. Same visible result on screen, different amount of wasted work underneath.

Exercise 4 (Medium): A tiny reusable fetch pattern, without the hook

[Back to the exercise](#)

Hint 1

Resist the urge to make these share a component or a helper function yet — the whole point of this exercise is to feel the repetition honestly, so that the fix in chapter 20 feels earned rather than arbitrary.

Exercise 5 (Challenge): Fetch that depends on another fetch

[Back to the exercise](#)

Hint 1

This needs **two separate `useEffect` calls**: one with an empty dependency array (or none needed at all, if you fetch categories once at the very top) for the category list, and a second depending on the selected category for the meal list. Keeping them separate is exactly what makes requirement 4 straightforward — trying to force one effect to do both jobs is what makes it hard.

Hint 2

For requirement 5, an early return at the top of the second effect — if the selected category is the empty/default value, clear the meal list and skip fetching entirely — handles it cleanly, the same shape as the empty-query guard in the chapter's main example.

19 Refs

Exercise 1 (Easy): State or ref?

[Back to the exercise](#)

Hint 1

Ask the question from the notes for each one: does the page need to visibly change when this value changes? Number 2 is the clearest "no." Number 3 is the clearest "yes." The others are worth talking through properly rather than guessing.

Exercise 2 (Easy): Focus management

[Back to the exercise](#)

Hint 1

For the very first focus (requirement 1), an effect with an empty dependency array, calling `.focus()` once after the first render, does the job. `autoFocus` on the `<input>` itself is also a completely legitimate choice for just this one case — try both and see which reads more clearly to you.

Exercise 3 (Medium): A stopwatch, built entirely on what you know

[Back to the exercise](#)

Hint 1

This combines a `useRef` (for the interval id) with a `useEffect` (whose cleanup clears it) with `useState` (for the displayed seconds) — three tools from three different chapters, working together. Start it running with an effect keyed on an `isRunning` boolean, similar to [chapter 17, exercise 3](#), but store the interval id in a ref this time instead of leaving it as a local variable inside the effect.

Exercise 4 (Medium): Read the DOM directly — a character-counting textarea

[Back to the exercise](#)

Hint 1

A common working pattern: in the effect, first reset `textareaRef.current.style.height = "auto"` (so the box can shrink back down if text was deleted), then immediately read `scrollHeight` and set `style.height` to that value in pixels. Doing the reset first matters — skipping it means the box can grow but never shrink.

Exercise 5 (Challenge): A generic "previous value" ref, and a real use for it

[Back to the exercise](#)

Hint 1

```
function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => {
    ref.current = value;
  });
  return ref.current;
}
```

Note the starting value. In **React 19**, `useRef` requires an argument — `useRef<T>()` with empty brackets is a type error (Expected 1 arguments, but got 0). Older tutorials write it that way because React 18's types allowed it. Say what you mean instead: the type is `T | undefined`, and it starts as `undefined`.

Trace through two renders by hand, on paper, before you decide the dependency array question — what does `ref.current` hold **during** each render, versus what gets written into it **after**?

20 Custom Hooks

Exercise 1 (Easy): `useToggle`

[Back to the exercise](#)

Hint 1

The toggle function should use the updater form, `set_value((prev) => !prev)`, not `set_value(!value)` — the same reasoning as [chapter 08](#): if `toggle` is ever called twice in one handler for some reason, the updater form is the one that behaves correctly.

Exercise 2 (Easy): `useLocalStorage`

[Back to the exercise](#)

Hint 1

The `try / catch` around `JSON.parse` inside the lazy initializer is what saves you in step 4 — same reasoning as the to-do app's milestone 8 in [chapter 10](#), just living inside the hook now instead of the component.

Exercise 3 (Medium): Finish the job from chapter 18

[Back to the exercise](#)

Hint 1

Each component's whole body should now look almost identical in shape to the `CategoryList` example in the notes — only the URL, the response type, and what's actually rendered in the "success" branch differ.

Exercise 4 (Medium): `useDebounce`

[Back to the exercise](#)

Hint 1

Inside the hook: a `useState` holding the debounced value, and a `useEffect` keyed on `[value, delayMs]` that sets a timeout to update it, with a cleanup that clears that timeout — nearly identical machinery to the timer exercise in [chapter 17](#), just wrapped up and made generic.

Exercise 5 (Challenge): `useOnlineStatus` and `useWindowWidth`, composed into a real feature

[Back to the exercise](#)

Hint 1

Both hooks share a shape: `useState` for the current value, and a `useEffect` with an empty dependency array that adds a listener on mount and removes the exact same listener function in its cleanup. The listener function needs to be defined once (not inline as a fresh arrow function each render) so that the function you remove in cleanup is really the same one you added.

Hint 2

For requirement 3, remember from [chapter 17](#) that cleanup runs when a component unmounts, no differently from when a dependency changes — hiding `Ex5` via conditional rendering is exactly the situation cleanup exists to handle.

21 Project: Recipe Finder

Exercise 1 (Easy): Recent searches

[Back to the exercise](#)

Hint 1

For requirements 2 and 3 together, in one expression:

```
setRecent((prev) => [term, ...prev.filter((t) => t !== term)].slice(0, 5));
```

Filter the term out of wherever it was, put it on the front, then trim. All new arrays, nothing mutated — [chapter 11](#).

Hint 2

For requirement 4, your search hook already knows when a request has succeeded and what it returned. An effect in `App.tsx` watching `recipes` is one way; returning the successful term from the hook is another. Try the one that means `App.tsx` doesn't need to know *when* a fetch finished.

Exercise 2 (Easy): Loading skeletons

[Back to the exercise](#)

Hint 1

`Array.from({ length: 6 })` gives you six items to map over. You'll need a `key` — this is one of the rare, genuinely legitimate uses of the array index for one, since the list is a fixed length, never reordered, and the items have no identity or state of their own ([chapter 06](#)).

Exercise 3 (Medium): Recipe of the day, cached for the day

[Back to the exercise](#)

Hint 1

`new Date().toISOString().slice(0, 10)` gives a `YYYY-MM-DD` string, which compares correctly with `===`.

Store `{ recipe, fetchedOn }` as **one** cached object rather than two separate `useLocalStorage` calls — two entries can drift out of sync with each other; one can't.

Hint 2

To test the "next day" path without waiting, edit the stored `fetchedOn` by hand in DevTools → Application → Local Storage, set it to yesterday, and refresh.

Exercise 4 (Medium): Full keyboard support

[Back to the exercise](#)

Hint 1

For requirement 4, you need to remember *which element* was clicked, across the whole time the modal is open. That's a value that must survive re-renders but should never cause one — the definition of a ref.

Store the clicked button's DOM node in a ref at the moment a card is selected, then call `.focus()` on it after the modal closes.

Hint 2

For requirement 2, a `keydown` listener on `window` inside an effect, checking `event.key === "Escape"` — with cleanup that removes the listener, [chapter 17](#). Only add the listener while the modal is actually open.

Exercise 5 (Challenge): Favourites that work offline

[Back to the exercise](#)

Hint 1

The cache is a second `useLocalStorage`, holding a lookup object rather than an array:

```
const [cache, setCache] = useLocalStorage<Record<string, RecipeDetails>>("recipe-finder-cache", {});
```

Keeping "which ids are favourited" and "what do I know about each one" as two separate entries is deliberate — it's much closer to how a real cache layer is built, and it means clearing one never corrupts the other.

Hint 2

For requirement 2, building a new object **without** one key is the object version of `filter`. Destructuring with rest does it in one line (JavaScript chapter 15):

```
setCache((prev) => {  
  const { [id]: _removed, ...rest } = prev;  
  return rest;  
});
```

Hint 3

For requirement 3, split the favourite ids into "already cached" and "needs fetching" before you do anything. If the second list is empty, don't call the network at all — that's what makes it work offline.

22 Context

Exercise 1 (Easy): Feel the drilling, then fix it

[Back to the exercise](#)

Hint 1

Question 5 is the whole point of the exercise. With context, adding a setting touches the context type, the provider, and the one component that uses it — three files, no matter how deep the tree gets. Drilling touched every layer.

Exercise 2 (Easy): The guard hook earns its keep

[Back to the exercise](#)

Hint 1

For question 3: nothing crashes. The component renders happily, showing a logged-out user, and you have no indication whatsoever that you forgot the provider. That's a bug that ships.

Exercise 3 (Medium): Theme, properly

[Back to the exercise](#)

Hint 1

For the last question — no, it's completely normal, and worth understanding why. A provider is just a component. It can use any hook any other component can. Context and custom hooks solve different problems (delivery vs. reusable logic) and combine freely.

Exercise 4 (Medium): Two contexts, and proving why

[Back to the exercise](#)

Hint 1

You can use your `useRenderCount` hook from [chapter 12's exercises](#) instead of `console.log` if you'd rather see the number on the page.

Hint 2

Remember that StrictMode doubles render counts in development. That's fine — compare Part 1's number against Part 2's, rather than treating either as an absolute.

Exercise 5 (Challenge): A toast notification system

[Back to the exercise](#)

Hint 1

The self-dismissing timer is the interesting part. Putting a `setTimeout` inside `showToast` works, but cleaning it up is awkward. A neater design: each rendered `<Toast>` owns its own effect with a `setTimeout` that calls `onDismiss(id)`, and a cleanup that clears it ([chapter 17](#)). Then requirement 4 comes free — unmounting the toast unmounts its timer.

Hint 2

`showToast` is recreated on every render of `ToastProvider`, which means the context value changes every render. In an app this small it doesn't matter. If you want to do it properly, `useCallback` is [chapter 27](#) — it's a reasonable time to skim ahead, but don't let it block you finishing the feature.

Hint 3

Use the updater form when adding a toast — `setToasts((prev) => [...prev, newToast])`. Three toasts triggered in quick succession from different components is exactly the situation where reading a stale `toasts` would lose one ([chapter 12](#)).

23 useReducer

Exercise 1 (Easy): A counter with rules

[Back to the exercise](#)

Hint 1

Question 7 is the one worth savouring. Without the `never` check, adding an action type to the union and forgetting to handle it produces no error at all — just silently broken behaviour at runtime. With it, you get a compile error before you've even saved.

Exercise 2 (Easy): Convert a `useState` component

[Back to the exercise](#)

Hint 1

Validation is a pure function of the state, so it can live in the reducer — `case "submitted"` computes the errors and either stores them or clears the form. But the `console.log` is a side effect, and reducers must be pure. Think about where that has to go instead, and what the reducer should return so the component knows it's safe to do it.

Hint 2

For the side-effect problem: one clean approach is for the component to check the state *after* dispatching — but remember the state you can read in the handler is the old snapshot ([chapter 12](#)). A simpler approach for this exercise: compute validity in the handler (calling a shared `validate(state)` function the reducer also uses), do the logging there, and dispatch only if it's valid.

Exercise 3 (Medium): A shopping cart reducer

[Back to the exercise](#)

Hint 1

Rule 2 is a nice example of a reducer doing real work: one action, two possible outcomes, decided in one place. The component just dispatches `quantity_changed` and doesn't need to know that a decrement from 1 means removal.

Hint 2

For rule 3, returning the *exact same* `state` object (not a copy) is the correct way to say "nothing changed" — React compares by reference, so it can skip re-rendering entirely ([chapter 11](#)).

Exercise 4 (Medium): Undo and redo, the reducer way

[Back to the exercise](#)

Hint 1

The answer to the last question is the real lesson: there's exactly **one** place where state changes, so "remember the previous state" is one line, applied to every action at once — rather than something you have to remember in each of six handlers.

A neat way to express that: handle `undo / redo` in a `switch`, and for everything else, compute the new `present` and wrap it with `{ past: [...state.past, state.present], present: next, future: [] }`.

Exercise 5 (Challenge): Reducer plus context — a real app shell

[Back to the exercise](#)

Hint 1

For question 5: `FilterBar` reads the state context (for `filter`), so it *will* re-render when tasks change — splitting the contexts helps components that dispatch *only*. If you want to see the difference properly, add a `DeleteAllButton` that uses only `useTaskDispatch()` and nothing else, and watch its render count stay flat.

Hint 2

Rule 4 is a good test of whether your action names are honest. `edited` with empty text deleting the task is a *rule*, and it belongs in the reducer — the component should just dispatch `edited` with whatever was typed and not know about the special case at all.

24 React Router

Exercise 1 (Easy): Three pages, and the `<a>` trap

[Back to the exercise](#)

Hint 1

Question 3 is the whole point. With `<a>`, the counter resets to 0 on every navigation, because the browser threw away your entire app and started it again from `index.html`. With `<Link>`, it keeps counting.

Exercise 2 (Easy): A layout that survives navigation

[Back to the exercise](#)

Hint 1

For question 5, the symptom is worth recognising: with the extra slash, the path becomes `//about` in effect, nothing matches, and you get your `NotFound` page (or a blank screen if you haven't added one). It looks like the route is "missing" rather than "misspelled."

Exercise 3 (Medium): URL parameters and a detail page

[Back to the exercise](#)

Hint 1

Question 5 is the interesting one. `navigate(-1)` retraces the user's actual history, which might be a completely different site if they arrived by pasting the URL. `<Link to="/books">` always goes to a known place. For "back to the list," the link is usually the safer choice.

Hint 2

Requirements 3 and 4 are two different missing cases: no `id` in the URL at all, and an `id` that matched nothing in your data. `books.find(...)` returns `Book | undefined`, so TypeScript will insist you handle the second — the same situation as every `find` since [chapter 04](#).

Exercise 4 (Medium): Filters in the URL

[Back to the exercise](#)

Hint 1

For question 5: every filter change adds a history entry, so back steps through your filter history one keystroke at a time. That's often annoying for a search box. `setSearchParams(next, { replace: true })` updates the URL *without* adding a history entry — try both and decide which you prefer for this case.

Hint 2

`searchParams.get("q")` returns `string | null`. `?? ""` gives you a usable default, and keeps the input controlled ([chapter 09](#)).

Exercise 5 (Challenge): A full mini-app with nested layouts

[Back to the exercise](#)

Hint 1

For requirement 2, `NavLink` marks itself active for exact matches by default at deeper levels — check whether `/account/orders` stays highlighted on `/account/orders/7` in your setup, and look at `NavLink`'s `end` prop, which controls exactly this.

Hint 2

For requirement 3's "send them where they were going," you need to remember the attempted URL. `useLocation()` gives you the current one; you can stash its `pathname` when showing the login page and `navigate(stashed)` after logging in. Getting this right is what separates a login flow people tolerate from one they find infuriating.

Hint 3

A clean way to express requirement 3 is a small `RequireAuth` component that either renders `<Outlet />` or the login page, used as a layout route wrapping the account routes — so the check is written once rather than in every account page.

25 TypeScript Patterns for React

Exercise 1 (Easy): A generic `List`

[Back to the exercise](#)

Hint 1

For question 5, the error appears at the *call site*, saying a `string` isn't assignable to `{ id: string }`. That's exactly right — the constraint is the component saying "I need items with ids," and the caller is the one who

broke the promise.

Exercise 2 (Easy): A `Button` that accepts everything

[Back to the exercise](#)

Hint 1

Question 4 is the one to actually run rather than reason about. The bug is invisible in the code and obvious in the Elements tab: `...rest` spread *after* your `className` attribute silently replaces it.

Exercise 3 (Medium): Props that can't be wrong

[Back to the exercise](#)

Hint 1

For `Avatar`, the union has two members that share `name` but differ otherwise. Excess property checking is what catches the "both" case — passing `initials` to the image variant isn't a known property of that member.

Hint 2

Remember not to destructure the parameter for any of these. `function Field(props: FieldProps)`, then narrow on `props.type`, then destructure inside the branch if you like.

Exercise 4 (Medium): Derive, don't restate

[Back to the exercise](#)

Hint 1

Question 6 is the point of the whole exercise. In the restated version, adding to the array only means the `<select>` gains an option that nothing else knows about — no error at all, and a badge that silently renders nothing when someone picks it.

Exercise 5 (Challenge): A typed data table

[Back to the exercise](#)

Hint 1

For requirement 4, tracking "which column is sorted" is tricky when columns are objects rather than strings. The simplest approach that stays type-safe: store the sorted column's `index` (`number | null`) plus a direction, and

look the column up when you need it.

Hint 2

For requirement 3 to work, `columns` needs to be typed as `readonly Column<T>[]` so that `T` flows from `items` into every column's callbacks. If TypeScript infers `unknown` for your callback parameters, the link between `items` and `columns` has been broken somewhere — check that both mention the *same* `T`.

Hint 3

For requirement 5, remember `sortBy` can return a `string` or a `number`, and those sort differently. `localeCompare` for strings, subtraction for numbers — a small `if (typeof a === "string")` inside the comparator handles both, and TypeScript narrows it properly.

26 Error Boundaries and Suspense

Exercise 1 (Easy): Watch it go blank, then fix it

[Back to the exercise](#)

Hint 1

For question 1, the answer is "nothing at all" — a completely white page. That's the behaviour this whole chapter exists to prevent, and it's worth seeing once with your own eyes rather than being told about.

Exercise 2 (Easy): What boundaries don't catch

[Back to the exercise](#)

Hint 1

Only number 1 trips the boundary. That's the whole lesson: boundaries are a render-time safety net, not a general error handler. Your fetching code's `.catch()` and `error` state aren't made redundant by boundaries — they're doing a different job.

Exercise 3 (Medium): Sectioned boundaries with recovery

[Back to the exercise](#)

Hint 1

Requirement 5 is the interesting one. If the crash flag lives *inside* the panel, resetting the boundary re-renders the same panel instance with the flag still `true`, so it throws again immediately. Two ways out: lift the flag above the boundary, or give the boundary a `key` that changes on reset so React builds a fresh panel.

Hint 2

For requirement 6, note what happens: the root boundary catches it, and the *entire dashboard* is replaced by the root fallback. That's why the notes say to keep fallbacks dead simple.

Exercise 4 (Medium): Code splitting, measured

[Back to the exercise](#)

Hint 1

For question 5, `lazy` expects a module whose `default` is the component. With a named export you map it yourself:

```
const Reports = lazy(() => import("./Reports.tsx").then((m) => ({ default: m.Reports })));
```

Exercise 5 (Challenge): Retrofit the Recipe Finder

[Back to the exercise](#)

Hint 1

For requirement 3, the fallback needs to be able to close the modal — which means it needs access to the function that clears `selectedId`. `FallbackComponent` receives `resetErrorBoundary`; you can wire `onReset` in the parent to clear `selectedId`, so one click both resets the boundary and closes the modal.

Hint 2

The closing question matters more than any of the code. The `error` state handles a request that *failed* — the network was down, the server said 500 — which is an expected, ordinary thing your app should handle gracefully and specifically. The boundary handles a *bug* — something threw that you didn't anticipate. Conflating them means either treating bugs as normal or treating a flaky connection as a catastrophe.

27 Performance

Exercise 1 (Easy): Learn to read the Profiler

[Back to the exercise](#)

Hint 1

Question 5 is the real exercise. At 200 rows, on a modern machine, the answer is almost certainly "no" — and the correct response to that is to change nothing. Learning to stop here is the skill most React developers skip.

Exercise 2 (Easy): Make `memo` fail

[Back to the exercise](#)

Hint 1

Both failures are the same rule from [chapter 11](#): an object or function literal creates a **new reference** every render, and `memo` compares by reference. Content being identical is irrelevant.

Exercise 3 (Medium): The structural fix beats memoisation

[Back to the exercise](#)

Hint 1

All three fixes should produce roughly the same render time. The interesting differences are in the *other* columns of your table: lines of code, and how easy it is to accidentally undo. The `memo` version breaks the moment someone adds an inline prop; the structural ones can't break that way.

Exercise 4 (Medium): When `useMemo` earns its place, and when it doesn't

[Back to the exercise](#)

Hint 1

Questions 2–4 together are the point. `useMemo` doesn't make the calculation faster — it makes it **not happen** when its inputs are unchanged. So it's a large win for the unrelated counter, and no win at all for typing in the search box, because typing genuinely does change the inputs.

Exercise 5 (Challenge): Profile and fix a real app

[Back to the exercise](#)

Hint 1

Step 6 usually produces numbers that are the same or very slightly *worse*, with considerably more code. That result is worth having in your own measurements rather than taking on trust — it's the strongest argument you'll ever have against reflexive memoisation in a code review.

Hint 2

If you want a bigger problem to find, temporarily seed the Recipe Finder with several hundred fake recipes rather than the handful an API search returns. Small apps often genuinely have no performance problem to fix, which makes step 8 the honest answer.

28 Testing

Exercise 1 (Easy): Your first tests, and the query ladder

[Back to the exercise](#)

Hint 1

Question 7 is the interesting one: `getByRole("heading")` still passes, because an `<h2>` is still a heading. That's a test correctly *not* caring about a change that doesn't affect users — which is exactly the property you want.

Exercise 2 (Easy): Testing a reducer

[Back to the exercise](#)

Hint 1

For requirement 5, `expect(after).toBe(before)` checks reference equality, and it will only pass if your reducer returns the exact same object rather than a copy. That's the behaviour [chapter 11](#) recommended, now pinned down by a test.

Hint 2

For requirement 7, the cleanest version: build the state, `JSON.stringify` it, run the action, then assert the stringified original still matches. Or deep-freeze it with `Object.freeze` and let a mutation throw.

Exercise 3 (Medium): Testing a form

[Back to the exercise](#)

Hint 1

Question 8 is the real exam. Any test that broke was reaching into *how* the component works. Tests that click, type, and check what's on screen sail through a refactor like this without noticing — which is exactly why they're worth writing.

Hint 2

For requirement 6, `user.keyboard("{Enter}")` after typing in the field, or `user.type(input, "text{Enter}")` in one go.

Exercise 4 (Medium): Testing async and mocked fetch

[Back to the exercise](#)

Hint 1

For question 9, `beforeEach / afterEach` with `vi.unstubAllGlobals()`, or `vi.restoreAllMocks()`. Leaky mocks produce the worst kind of test failure: one that only happens when tests run in a particular order.

Hint 2

For requirement 5, remember `fetch` doesn't reject on a 500 — your stub should resolve with `{ ok: false, status: 500 }`, and your component's `response.ok` check is what turns that into an error. If your test passes without that check in the component, the test isn't testing what you think.

Exercise 5 (Challenge): Test a real feature end to end

[Back to the exercise](#)

Hint 1

Requirement 7 is doing double duty on purpose. A heart button showing only ♥ has no accessible name at all, so `getByRole("button", { name: /favourite/i })` can't find it — and neither can a screen reader user. The fix is the `aria-label` the project notes asked for, and your test is what proves it's actually there.

Hint 2

For requirement 3, `localStorage` works in jsdom but persists across tests in the same file. `beforeEach(() => localStorage.clear())` keeps them independent — and that independence is what makes a failing test mean

something.

Hint 3

Question 9 is worth taking seriously rather than answering glibly. Think back to what actually broke while you were building chapter 21 — a stale result from a race condition, a favourite that didn't persist, a crash on an empty search. Writing the test that would have caught it is the most direct way to learn what tests are *for*.

29 Project: Online Bookstore

Exercise 1 (Easy): Persist the cart

[Back to the exercise](#)

Hint 1

Keep the reducer itself completely pure — it must not touch `localStorage`. Loading happens in the initialiser (before the reducer ever runs), and saving happens in an effect watching the state. The reducer stays a plain function of `(state, action)`.

Exercise 2 (Easy): Sorting, in the URL

[Back to the exercise](#)

Hint 1

A lookup object mapping each sort key to a comparator function keeps this tidy and avoids a chain of `if s` — [chapter 05](#)'s pattern, and a good place for `satisfies` from [chapter 25](#).

Exercise 3 (Medium): Stock levels and validation

[Back to the exercise](#)

Hint 1

Requirement 2 means the action needs to carry the stock figure, or the reducer needs access to the book data. Passing the whole `book` in the `added` action (as the milestone-4 type already does) makes this straightforward — the reducer can read `book.stock` without importing anything.

Exercise 4 (Medium): Saved for later

[Back to the exercise](#)

Hint 1

The deciding question for requirement 5: "move to cart" is **one** user action that changes **two** collections. If they're separate reducers, you're dispatching two actions that must both succeed to stay consistent. If they're one reducer, it's a single atomic action. That's a genuine argument for combining them — and a genuine argument against, if the two collections otherwise have nothing to do with each other.

Exercise 5 (Challenge): Make it a real app

[Back to the exercise](#)

Hint 1

For B, `useLocation()` gives you the URL the user was refused. Stash its `pathname` (in router state via `<Navigate to="/login" state={{ from: location }} />`, or in context), and `navigate(from, { replace: true })` after a successful login. `replace` matters — without it, pressing back lands them on the login page again.

Hint 2

For A, serving the JSON from `public/` is worth knowing as a technique in its own right: files in `public/` are served as-is at the root, so `public/books.json` is available at `/books.json`. It gives you real network behaviour — loading delays, the ability to simulate failure by renaming the file — with no backend at all.

Hint 3

For D, the auth flow test is the one most worth writing carefully, because it's the one most likely to have a real bug. Render at `/account/orders` with `MemoryRouter`'s `initialEntries`, assert you see the login page, log in through the form, and assert you end up on the orders page — not the account home.

30 Axios

Exercise 1 (Easy): Surprise me, the Axios way

[Back to the exercise](#)

Hint 1

The random endpoint still answers with an array holding one meal: `{ meals: [ ... ] }`. With Axios, that's `response.data.meals?.[0]`.

Hint 2

To read `err.code` in the `catch`, narrow first: `if (axios.isAxiosError(err)) console.log(err.code);`. A domain that doesn't exist means no answer ever came back, so there's no `err.response` at all.

Exercise 2 (Easy): Your reading list, from your own API

[Back to the exercise](#)

Hint 1

The 404 check is `axios.isAxiosError(err) && err.response?.status === 404`. The `?` matters: when the server isn't running, there is no `response`.

Hint 2

Keep the lookup's result and error in their own state, separate from the list's. The list shouldn't disappear just because one lookup failed. That's the same "two separate loading states" lesson as chapter 18's exercise 5.

Exercise 3 (Medium): Add, finish, delete

[Back to the exercise](#)

Hint 1

One way to reload: move the loading code out of the effect into a `loadBooks` function, call it from the effect, and call it again after each change. Another way: keep a `reloadCount` number in state, put it in the effect's dependency array, and add one to it after each change.

Hint 2

Each handler is `async`, and follows the same shape: set a "saving" state, `await` the change, reload, then clear the saving state in a `finally`. Notice how often you're writing that shape. Remember the number for chapter 31.

Exercise 4 (Medium): Interceptors and friendly errors

[Back to the exercise](#)

Hint 1

Check things in order, from most specific to least: is it an Axios error? Did the server answer (`err.response`)? What was the status? If there was no answer, was it a timeout (`err.code === "ECONNABORTED"`)? Everything else falls through to the general message.

Hint 2

A timeout's exact `err.code` can depend on your Axios settings. If `"ECONNABORTED"` doesn't match, log `err.code` and `err.message` during step 6 and see what you really get. Checking real values beats guessing.

Exercise 5 (Challenge): Recipe search, the Axios way

[Back to the exercise](#)

Hint 1

In the hook's `catch`, the order of checks matters: `axios.isCancel(err)` first (return quietly), then everything else. A cancel is also an Axios error, so checking `isAxiosError` first would show it as a failure.

Hint 2

The empty-query guard goes at the top of the effect, before the `AbortController` is made. Clear the results and return early, just like chapter 21's version.

Hint 3

For question 6, the honest answer is: the request itself got shorter (no `response.ok`, no `json()`, no `encodeURIComponent`), but the loading, error, and cancelling code around it barely changed. Keep that thought for the next chapter.

31 TanStack Query

Exercise 1 (Easy): The recipe search, one more time

[Back to the exercise](#)

Hint 1

The key and the function go together: `queryKey: ["recipes", debouncedQuery]` and `queryFn: ({ signal }) => searchRecipes(debouncedQuery, signal)`. Use the *debounced* value in both, not the live one.

Hint 2

For step 6: a query with `enabled: false` and no data is pending, but not fetching. It will never leave that state on its own. Reread the "Watch out for this one" part of the notes.

Exercise 2 (Easy): Cache experiments

[Back to the exercise](#)

Hint 1

The number next to a query in the devtools is how many components (called **observers**) are using it right now. When it drops to 0, the query becomes **inactive**, and the `gcTime` countdown starts.

Hint 2

If step 7 doesn't update, check your `staleTime`. From step 5, the data may still count as fresh, and fresh data isn't refetched on focus. Set `staleTime` back to the default and try again.

Exercise 3 (Medium): The reading list, with mutations

[Back to the exercise](#)

Hint 1

`mutationFn` takes one argument. For the status change, pass an object and unpack it: `({ id, status }: { id: string; status: BookStatus }) => updateBook(id, { status })`.

Hint 2

The counts in `ReadingSummary` are **derived**: `filter` the books during render and count them ([chapter 08](#)). Don't store them in state.

Hint 3

For step 7: invalidating `["books"]` matches every key that *starts with* "books". So `["books", { status: "want" }]` is marked out of date too. That's why keys go from general to specific.

Exercise 4 (Medium): One query waits for another

[Back to the exercise](#)

Hint 1

This is called a **dependent query**: one that waits for something else first. It's just `enabled`. Something like `enabled: category !== ""`, with `category` in the key too.

Hint 2

Step 6 is `isFetching`, not `isPending`. `isPending` means "no data yet". `isFetching` means "a request is running right now", even when there's data on screen.

Exercise 5 (Challenge): Instant status changes, custom hooks, and a test

[Back to the exercise](#)

Hint 1

The cache change in step 2 is a `map`: every book stays the same, except the one whose `id` matches, which gets `{ ...book, ...changes }`. That's [chapter 11](#)'s copying rule, applied to the cache.

Hint 2

In step 4, the refetch in `onSettled` fails too, because the server is still stopped. Once its retries run out, the `list` query is in an error state as well. If your component does `if (isError) return ...`, the whole list disappears. Try showing the list whenever `data` exists, and the error as a small message above it.

Hint 3

For step 7, `user.selectOptions(select, "finished")` from [chapter 28](#) chooses an option. Give each select a label (like `Status of Dune`), so you can find it with `getByRole("combobox", { name: ... })`. A `<select>` counts as a "combobox" for `getByRole`. To wait for the old value to come back, `findByDisplayValue` works on a select: it matches the text of the chosen option.

Hint 4

If the bonus test waits a long time and fails, check that `retry: false` is really in your test `QueryClient`. Mutations don't retry by default, but the refetch after the rollback is a query, and queries do.

32 Zod

Exercise 1 (Easy): Check the to-do list

[Back to the exercise](#)

Hint 1

For an item inside an array, the `path` includes its index. `[1, "done"]` joined with `."` reads as `1.done`: "the task at index 1, its `done` field".

Hint 2

In a `catch`, `error` is `unknown` (TypeScript chapter 13). Narrow it with `error instanceof z.ZodError` before you hand it to `z.prettifyError`.

Hint 3

The `priority` one doesn't fail at all. Look closely at the data it logs. The notes' section "Extra keys are dropped" explains what happened.

Exercise 2 (Easy): A `useLocalStorage` that can't be fooled

[Back to the exercise](#)

Hint 1

The type of a schema parameter, where `T` is the data it checks for, is `z.ZodType<T>`. Put it before `initialValue`, and TypeScript works out `T` from the schema you pass.

Hint 2

There are two different kinds of failure: text that isn't JSON (so `JSON.parse` throws), and JSON in the wrong shape (so `safeParse` fails). The notes' `loadSettings` handles both.

Hint 3

Something to think about, as a comment: after a failed load, the hook's effect saves `initialValue` straight back, so the bad data is gone for good. For a to-do list that's fine. When might quietly deleting someone's saved data be a problem?

Exercise 3 (Medium): TheMealDB, checked and reshaped

[Back to the exercise](#)

Hint 1

A `.transform` can go on the **outer** object, not just on each item. For `getCategories`, the outer transform can pull out `data.meals` and map each one to its `strCategory`. Then the function returns exactly the `string[]` its callers want.

Hint 2

For step 6, it takes several seconds because TanStack Query retries 3 times first. The notes show a `retry` function that skips retrying a `ZodError`. It works in any `useQuery`, not just `booksQuery`.

Hint 3

The descriptions are long. A transform can do more than rename: it can also shorten, join, or work things out. If you want a one-line summary on each card, the transform is a fine place to make it.

Exercise 4 (Medium): The reading list can't lie any more

[Back to the exercise](#)

Hint 1

If `npx tsc -b` complains about another file after step 1, you've probably renamed or forgotten an export. The other files import `Book`, `NewBook`, `BookStatus` and the five functions, so those names must all still exist.

Hint 2

Chapter 30 said json-server 1 reloads `db.json` when you save it by hand. If your edit seems to be ignored, check the terminal running `npm run api`: a missing comma makes the file invalid JSON, and json-server says so there.

Hint 3

For experiment 4, open the TanStack Query devtools panel and look at the cached data for `["books"]`. Compare it with `http://localhost:3001/books` in a browser tab.

Exercise 5 (Challenge): A sign-up form, checked by Zod

[Back to the exercise](#)

Hint 1

Your state can't be typed as `SignUp`. `SignUp` says `agree` is `true`, but the box starts unticked. The form's values *while someone is filling it in* aren't the same thing as *valid data*. Give the state its own type (with `agree: boolean`), and let `safeParse` turn it into a `SignUp`.

Hint 2

You don't need a piece of state for the errors. Keep one boolean, `hasSubmitted`. Then, during render, run `safeParse` on the current values when `hasSubmitted` is `true`. The errors are a **derived value** ([chapter 08](#)), so they're always up to date, and requirement 4 comes for free.

Hint 3

`z.flattenError(result.error).fieldErrors.confirmPassword?.[0]` is either the first message or `undefined`. That's exactly the shape you need for `{message && <p id="...">{message}</p>}`.

Hint 4

For the checkbox, remember chapter 09: `checked={values.agree}` and `event.target.checked`, not `value`.

33 React Hook Form

Exercise 1 (Easy): Chapter 09's contact form, rebuilt

[Back to the exercise](#)

Hint 1

Built-in rules go in `register`'s second argument, each with its own message: `{ required: "Please enter your name" }`, or `{ minLength: { value: 10, message: "..."} }`.

Hint 2

For the email, `pattern` takes a regular expression and a message (JavaScript chapter 37). Or `validate` takes a function that returns `true` when the value is fine, or an error message when it isn't.

Hint 3

For the counter, give `message` a starting value of `""` in `defaultValues`. Then `watch("message")` is always a string, and `.length` just works.

Exercise 2 (Easy): Your sign-up form, with its schema plugged in

[Back to the exercise](#)

Hint 1

If TypeScript complains about `agree: false` in `defaultValues`, that's chapter 32's hint about the checkbox, back again: the schema says `agree` must be `true`, and the form's types come from the schema. Two ways out: leave `agree` out of `defaultValues` (an unticked checkbox reads as `false` anyway), or write the rule as `z.boolean()` with a `.refine` that checks it's `true`, so the form is allowed to hold either value.

Hint 2

For step 5: after a failed submit, RHF re-checks **the field you're typing in**. The mismatch error lives on `confirmPassword`, so typing in the password box may not clear it. Look up the `deps` option of `register` in the React Hook Form docs. It tells RHF "when this field changes, re-check those fields too".

Exercise 3 (Medium): "Add a book", for real

[Back to the exercise](#)

Hint 1

`mutateAsync` gives back the saved book, id and all. So `const saved = await addBook.mutateAsync(values)` lets you use `saved.title` in your message.

Hint 2

For step 5, `register` accepts its own `onChange` option, which runs as well as RHF's (mistake 3 in the notes). Hide the message there.

Hint 3

For step 8, think about who handed `onSubmit` its values. The resolver passes on Zod's **output**, not what was typed. Which rule in `BookSchema` changes the text?

Exercise 4 (Medium): "Edit book", filled in when the data arrives

[Back to the exercise](#)

Hint 1

For `BookFields`, React Hook Form exports types for the two things it needs: `UseFormRegister<NewBook>` for `register`, and `FieldErrors<NewBook>` for `errors`. Pass both in as props.

Hint 2

For step 4: after saving, `reset(values)` makes what you saved the new starting point, so `isDirty` goes back to `false`. Plain `reset()` goes back to the *old* starting values.

Hint 3

For step 6, the `key` on `EditBookForm` is what makes this safe. Try taking it out, and switch between books while one is loading. Can you get the wrong values to show?

Exercise 5 (Challenge): A recipe form, with a list of ingredients and tests

[Back to the exercise](#)

Hint 1

For `TextField`, type the props as `ComponentProps<"input">` plus your own `label` and `error`. In React 19, `ref` is an ordinary prop ([chapter 19](#)), so when you spread the rest of the props onto the real `<input>`, `register`'s `ref` goes along with them.

Hint 2

The whole-list error doesn't belong to any one input. Log `errors.ingredients` after submitting with no rows. Depending on whether rows were registered before, React Hook Form puts a list-level message at `errors.ingredients.root.message` or at `errors.ingredients.message`. Showing `errors.ingredients?.root?.message ?? errors.ingredients?.message` covers both.

Hint 3

Checking is async, so find error messages with `await screen.findByText(...)`. And read the notes' "Testing a form" part before test 3: `handleSubmit` calls your function with **two** arguments.

Hint 4

For test 4, `expect(element).toHaveFocus()` comes with `jest-dom`. Find the element the way a user would: `getByLabelText("Recipe name")` or similar.

Hint 5

Build it in small steps: name and servings with one test first, then the ingredients list, then the rest. A big form built all at once is a big form you can't debug (chapter 09 said the same about multi-step forms).

34 Zustand

Exercise 1 (Easy): A theme that remembers

[Back to the exercise](#)

Hint 1

`toggle` depends on the old theme, so use the function form of `set`: `set: set((state) => ({ theme: ... }))`. And remember `set` merges, so you only return the field you're changing.

Exercise 2 (Easy): Who re-renders?

[Back to the exercise](#)

Hint 1

For step 3, expect `Controls` to log only once, when it first appears. It selects functions, and the functions in a store never change. That's the same win the separate dispatch context gave you in [chapter 23](#).

Hint 2

In development, `StrictMode` may log each render twice ([chapter 12](#)). Count the *pairs*, not the lines.

Exercise 3 (Medium): The bookstore cart, with Zustand

[Back to the exercise](#)

Hint 1

For requirement 5: the message is **local UI state**. Only the promo box cares about it, so it's a `useState` in that component, not a field in the store. After calling `applyPromo(code)`, the box can check whether it worked with `useCartStore.getState().promoCode === code`.

Hint 2

If you find yourself writing `if (item.quantity < 10)` in a component, stop. That's chapter 29's mistake 4, "rules leaking into components". The `+` button should call `changeQuantity(id, item.quantity + 1)` and let the store decide.

Exercise 4 (Medium): Test every rule

[Back to the exercise](#)

Hint 1

Money in floating point is rarely exact: `9.99 * 2 * 0.9` isn't exactly `17.982`. Use `expect(total).toBeCloseTo(17.98, 2)` instead of `toBe`.

Hint 2

For step 5: the failures depend on test order. A test that expects an empty cart fails if an earlier test left books behind. That's why the reset matters: each test should pass on its own, in any order.

Exercise 5 (Challenge): Toasts you can call from anywhere

[Back to the exercise](#)

Hint 1

Zustand actions are not reducers. A reducer must be pure ([chapter 23](#)), but an action is more like an event handler: it can make an id with `crypto.randomUUID()` or start a timer. Only the function you pass to `set` needs to be pure.

Hint 2

There are two sensible places for the 4-second timer. **In the store**, `show` starts a `setTimeout` that calls `dismiss`. **In a `ToastItem` component**, an effect starts the timer and its cleanup clears it ([chapter 17](#)). Try one, and write down why you picked it. Think about what happens in each version when a toast is dismissed early, and whether you can still show a toast when no `Toaster` is on the page.

Hint 3

For requirement 3, the interceptor's error handler must still reject, or the code that made the request will think it succeeded:

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    // show the toast here
    return Promise.reject(error);
  }
);
```

35 Redux Toolkit

Exercise 1 (Easy): A counter, in slow motion

[Back to the exercise](#)

Hint 1

`decremented` can "change" the draft directly, with `Math.max` keeping it at 0 or above. For `reset`, returning a fresh starting state is the simplest way. Either change the draft or return a new state, never both.

Hint 2

For step 6: what does `state.count++` give back? And what does an arrow function without curly braces return? Mistake 3 in the notes is about the same thing.

Exercise 2 (Easy): From `useReducer` to a slice

[Back to the exercise](#)

Hint 1

For the last question: in chapter 23, you typed out `{ type: "toggled", id }` yourself, so a union type was needed to catch typos. Now you call `toggled(id)`, a function RTK made for you. What happens if you misspell a function name when you import or call it?

Hint 2

Curious how RTK projects usually make ids? Look up the `prepare` callback in the `createSlice` docs. It's optional here; passing the id in the payload is perfectly fine.

Exercise 3 (Medium): The bookstore cart, the Redux way

[Back to the exercise](#)

Hint 1

For requirement 4, keep the code the user last tried in local `useState`. Then, during render, compare it with the promo code selected from the store. If they tried something and the store's code doesn't match it, it wasn't valid. That's a derived value, not more state.

Hint 2

For requirement 5: a wrong code returns before touching the draft, so Immer hands back the very same state object. Nothing changed, so there's nothing to show.

Exercise 4 (Medium): Memoised totals, and a test for every rule

[Back to the exercise](#)

Hint 1

Money in floating point is rarely exact, so use `toBeCloseTo(17.98, 2)` rather than `toBe` for totals.

Hint 2

A selector made with `createSelector` also has a `recomputations()` method, which tells you how many times it really did the maths. It's a neat way to prove step 4.

Exercise 5 (Challenge): Two slices, one action, and a verdict

[Back to the exercise](#)

Hint 1

`store.subscribe(listener)` calls your listener after **every** dispatched action. To write only when something changed, keep the last `state.cart` you saved and compare it with `!==`. Immer reuses unchanged objects, so a reference check is enough ([chapter 11](#)).

Hint 2

Write a `loadSavedState()` function that returns `{ cart?: CartState; wishlist?: WishlistState } | undefined`, with `JSON.parse` inside a `try / catch`. Type it from your slice state types, not from `RootState`: `RootState` is worked out *from* the store you're in the middle of building. For extra safety, check the parsed data with a Zod schema from [chapter 32](#).

Hint 3

For requirement 3, watch out for a trap. If the cart slice imports an action from the wishlist file, and the wishlist imports `added` from the cart file, the two files import each other. That's a **circular import**, and it can cause confusing bugs. One way out: make the shared actions in a third file with `createAction<Book>` (`"shop/movedToCart"`), and let each slice handle them in `extraReducers`.

RTK also has a tidier tool for side effects like saving, `createListenerMiddleware`. It's worth a look once `store.subscribe` works.

36 TanStack Router

Exercise 1 (Easy): Three pages and a shared header

[Back to the exercise](#)

Hint 1

For step 5, links are active on every page *below* them, and every page is below `/`. Look at the `activeOptions` prop on the Home link in the notes.

Hint 2

For step 7, the silence is the lesson. Without `Register`, TypeScript has no map of your routes, so `to` quietly accepts any string. Nothing is wrong, and nothing is checked.

Exercise 2 (Easy): A page for every book

[Back to the exercise](#)

Hint 1

The book list's link needs two props: the route's pattern in `to`, and the actual id in `params`. The pattern is the same for every book; only `params` changes.

Hint 2

Step 4 is the "string, not undefined" lesson. `bookId` is always a string, but `books.find(...)` can still come back `undefined`. TypeScript will make you handle it, the same as every `find` since [chapter 04](#).

Exercise 3 (Medium): Filters that live in the URL

[Back to the exercise](#)

Hint 1

For step 2, TypeScript won't let you pass `event.target.value` straight into `genre` or `sort`. Read the error: a `<select>` gives you *any* string, and not every string is a genre. You already have something that turns an unknown string into a checked value. Look at `bookSearchSchema.shape`.

Hint 2

For step 4, a lookup object from sort key to compare function keeps this tidy, like chapter 29's stretch goal 2. `satisfies` from [chapter 25](#) can check you've covered every key. `z.infer<typeof bookSearchSchema>["sort"]` gives you the union of sort keys.

Hint 3

For step 6, the surprise is `?q=1984`. TanStack Router reads search values like JSON, so `1984` arrives as a *number*, and `z.string()` rejects it. When the router writes a number-like string itself, it adds quotes (`?q=%221984%22`) so it survives the round trip. Only hand-typed URLs hit this. Look up `z.coerce` in [chapter 32](#) if you'd like to fix it.

Hint 4

For step 8, the router has a helper called `stripSearchParams`. It goes in the route's `search: { middlewares: [...]}` option, and you give it an object of your default values.

Exercise 4 (Medium): Loaders and a slow server

[Back to the exercise](#)

Hint 1

For step 4, remember the router's two timing rules from the notes: it waits before showing a pending screen, and once it shows one, it keeps it for a minimum time. Also try it both ways: clicking a link from another page, and refreshing on the book's URL.

Hint 2

For step 5, a loader that throws is a render-time failure as far as the route is concerned, so the route's `errorComponent` catches it. Nothing above the route breaks. `useRouter()` gives you the router, and the notes show which method runs the loaders again.

Hint 3

For step 7, the router has its own small cache. By default it shows what it has at once, then quietly reloads in the background. The Console tells you whether that background reload happened.

Exercise 5 (Challenge): Pick one, A or B

[Back to the exercise](#)

Hint 1

For A, step 1: the root layout reads the cart, so `CartProvider` has to sit *outside* `RouterProvider` in `main.tsx`, the same way it sat outside the routes in chapter 29.

Hint 2

For A, step 6: TanStack Router has its own `createMemoryHistory({ initialEntries: [...] })`. Your test helper can create a router with it and render `<RouterProvider>`. Remember that loaders are async, so use `findBy...` queries, not `getBy...`, for what the page shows.

Hint 3

For B, step 3: `fetchBook` answers `null` for a missing book, so the query caches `null` too. The loader can check what `ensureQueryData` returns and throw `notFound()`. The component still sees `Book | null` in its type, because TypeScript can't see that the loader already checked. Decide how you want to handle that, and write down why.

Hint 4

For B, step 4: the Query devtools come from `@tanstack/react-query-devtools`, as in chapter 31. Watch the `["books"]` query's status as you move between pages, and keep your `console.log` in `fetchBooks` so you can see whether a real fetch happened.

37 Project: Task Board

Exercise 1 (Easy): Due-date badges

[Back to the exercise](#)

Hint 1

Read the starter README's note on dates first. Comparing `"YYYY-MM-DD"` strings with `<` and `===` works, because ISO dates sort correctly as text. For "how many days away", turn both strings into dates at UTC midnight and divide the difference by the number of milliseconds in a day.

Hint 2

Taking `today` as an argument is what makes this testable. Your tests can say "pretend it's 30 September" without touching the clock. The component calls it with a real `todayIso()` helper; the tests call it with a fixed string.

Exercise 2 (Easy): Sorting, in the URL

[Back to the exercise](#)

Hint 1

A lookup object from each sort key to a compare function keeps this tidy, with `satisfies Record<SortKey, (a: Task, b: Task) => number>` ([chapter 25](#)). For priority, a small `{ high: 0, medium: 1, low: 2 }` lookup turns words into numbers you can subtract.

Hint 2

For **Clear filters**, a typed `<Link to="/">` with an empty `search` object is enough, because every search param has a default. Try giving it a sort key that doesn't exist, and read the type error.

Exercise 3 (Medium): Undo delete

[Back to the exercise](#)

Hint 1

Storing a function in a Zustand store is fine, as long as the store isn't persisted (functions can't be turned into JSON). Each toast can clear itself with a `setTimeout` started when it's shown. Remember to clear that timer if the toast is dismissed early.

Hint 2

For "wait before deleting", your chapter 31 tools already do the hiding: `setQueryData` to remove the task from the cached list, just like `useMoveTask` changes one. Undo is then "put the old list back", which is exactly a rollback.

Exercise 4 (Medium): Swap Zustand for Redux Toolkit

[Back to the exercise](#)

Hint 1

For step 4, `store.subscribe(() => ...)` runs after every action, so it's a natural place to save. Read the saved JSON inside a `try / catch`, like the to-do app's `loadTasks` in [chapter 10](#), and pass it to `configureStore` as `preloadedState`.

Hint 2

Nothing about the tasks should change in this exercise. If you find yourself moving tasks into the Redux store, stop: server data stays in TanStack Query, whichever library holds your client state.

Exercise 5 (Challenge): Make it bigger

[Back to the exercise](#)

Hint 1

For A, make each column a drop target and each card draggable, but give the card a separate small `drag-handle` button to drag by. Then the card's link and move buttons still work normally when you click them.

Hint 2

For B, json-server can filter for you: `GET /tasks?boardId=2`. Keys like `["boards", boardId, "tasks"]` keep each board's tasks separate, while still sharing a beginning with anything else about that board. And a loader can `throw redirect(...)` just as it can `throw notFound()`.

Hint 3

For C, build the router inside each test with `createRouter({ routeTree, history: createMemoryHistory({ initialEntries: ["/tasks/999"] }) , context: { queryClient } })`, and render it inside a `QueryClientProvider`. Turn off retries in the test's `QueryClient`, or your error tests will wait through three retries. MSW then answers `http://localhost:3001/tasks` with whatever each test needs.

38 Accessibility

Exercise 1 (Easy): Unplug your mouse

[Back to the exercise](#)

Hint 1

When a task's delete button disappears, focus drops to `<body>`. Something that's always on the page makes a safe landing spot, like the "New task" input or a heading with `tabIndex=-1`. A ref ([chapter 19](#)) gets you there. The same problem hides in at least two other places in this app. Look for other things that vanish when you use them.

Hint 2

The selected filter only *looks* different, because of a CSS class. Ears can't see classes. Which ARIA state from the notes means "this toggle is on"? And if Lighthouse flags contrast, the fix lives in `--text-muted` in `index.css` (check dark mode too, if you built it).

Exercise 2 (Easy): Fix the broken book card

[Back to the exercise](#)

Hint 1

For the image, decide first: is the cover decorative here, or does it tell you something the text next to it doesn't? The notes (and the bookstore's starter README) have an opinion. If you choose `alt=""`, notice that `queryByRole("img")` then finds nothing. An image with an empty `alt` is left out of what screen readers get.

Hint 2

For the heart, the notes give you two honest options: a fixed name like "Save Dune for later" with `aria-pressed`, or a name that changes. Pick one, not both. With the first option, a test can even check the state: `getByRole("button", { name: /save dune/i, pressed: true })`.

Exercise 3 (Medium): A real dialog for the recipe finder

[Back to the exercise](#)

Hint 1

The starter CSS was written for a `<div>`. On a `<dialog>` you'll probably need `border: none` (browsers give dialogs a default border), and `position: fixed` instead of `.modal`'s `position: relative`, or the dialog can end

up in the wrong place when the page is scrolled. Style the dark background with `.modal::backdrop`.

Hint 2

If focus doesn't come back to the card, check how the modal closes. Does *every* path go through `dialog.close()` (or the browser's own Escape), so the `close` event fires? If the parent simply stops rendering the dialog, the browser never closes it properly.

Hint 3

At the time of writing, jsdom (the pretend browser your tests run in) doesn't implement `showModal()` or `close()`, so the test crashes with an error saying `showModal` is not a function. A small stand-in in `src/test-setup.ts` gets you going:

```
HTMLDialogElement.prototype.showModal = function (this: HTMLDialogElement) {
  this.setAttribute("open", "");
};
HTMLDialogElement.prototype.close = function (this: HTMLDialogElement) {
  this.removeAttribute("open");
  this.dispatchEvent(new Event("close"));
};
```

It's only a stand-in: it doesn't keep focus inside or handle Escape. You check those by hand, in a real browser.

Exercise 4 (Medium): Speak up, bookstore

[Back to the exercise](#)

Hint 1

Lots of components need to announce things, but the region should live in one place. That's a job for context ([chapter 22](#)): an `AnnouncerProvider` holding the message, with an `announce(text)` function any component can call, and a guard hook like the ones you've written before.

Hint 2

For step 2: if the text doesn't change, nothing is announced. Including the new quantity ("Added Dune. 2 in your cart.") changes it.

Hint 3

For step 6: is a wrong promo code urgent enough to interrupt someone mid-sentence? Decide between `role="status"` and `role="alert"`, and remember the rule about the region being on the page before its text changes.

Exercise 5 (Challenge): A full audit

[Back to the exercise](#)

Hint 1

When a card moves column, React renders it in a different list, so it's a brand-new element and focus is lost. Remember the moved card's id, and once the new render is on the page, focus that card's button. An effect that runs when the id changes (like chapter 19's "focus after render" example) does it.

Hint 2

If your task board uses TanStack Router ([chapter 36](#)), it has its own `useLocation()` hook. Import it from `@tanstack/react-router`, and the notes' route-change pattern works the same way.

39 Next.js and Server Components

Exercise 1 (Easy): Pages, a layout, and a dynamic route

[Back to the exercise](#)

Hint 1

The counter needs `useState`, so its file starts with `"use client"`. The layout itself stays a Server Component: it just renders `<VisitCounter />`, like any other component.

Hint 2

For step 6: look at which folders have a `page.tsx`. A folder without one isn't a page, even if a folder inside it is.

Hint 3

For step 7: `usePathname()` from `next/navigation` gives you the current path, like `"/about"`. It's a hook, so the link component that uses it has to be a Client Component. Compare the path with the link's `href`.

Exercise 2 (Easy): Fetch on the server, and prove it

[Back to the exercise](#)

Hint 1

For step 4: look at the **Type** column for the requests that `themealdb` finds. Your HTML contains `<img>` tags pointing at TheMealDB, and who loads the pictures?

Exercise 3 (Medium): A recipe page with a small client island

[Back to the exercise](#)

Hint 1

`data.meals?.[0]` gives you either the first meal or `undefined`, in one step. That covers both "no meals" and "empty list".

Hint 2

For step 4: a hook that reads `localStorage` inside its `useState` starting value runs during render, and Client Components render on the server too. Start with a safe default that's the same on both sides, then read the real value in a `useEffect` ([chapter 17](#)), which only runs in the browser. Starting with the same value on both sides is also what stops the hydration warning.

Exercise 4 (Medium): Loading, errors, and not found

[Back to the exercise](#)

Hint 1

The test crash can be one line at the top of the page, right after you await `params`: `if (id === "crash") throw new Error("The kitchen is on fire");`

Hint 2

For step 3: a `not-found.tsx` inside a folder is only used when code in that folder calls `notFound()`. A URL that matches no page at all goes to the one at the top of `app/`.

Exercise 5 (Challenge): A reading list with Server Functions

[Back to the exercise](#)

Hint 1

`FormData` values are always strings. `pages` arrives as `"310"`, not `310`. Zod's `z.coerce.number()` turns the string into a number before the other checks run.

Hint 2

For step 6: in React 19, a form is reset after its action finishes, even when your function returned errors. One fix: return what they typed as part of your state, next to the errors, and use it as each field's `defaultValue`.

Hint 3

For step 9: a Server Function can take extra arguments. `removeBook.bind(null, book.id)` makes a version with the id already filled in, which you can pass to a small form's `action`. A hidden `<input name="id">` works too. Either way, the id comes from the browser, so look it up before you trust it.

Hint 4

For step 10: this page has no dynamic `[segment]` and reads nothing from the request. What might `npm run build` do with a page like that?

Hint 5

For the Vite question: think about where `reading-list.json` would live, what program would read and write it, how the browser would reach that program, and where you'd validate. [Chapter 30](#) is a big clue.

40 Deploying

Exercise 1 (Easy): Look inside the box

[Back to the exercise](#)

Hint 1

For question 4: the main file has to know the checkout chunk's name, so it can ask for it later. So when that name changes, the main file's contents change, and a different content means a different fingerprint. The CSS file didn't mention the checkout at all, so its name has no reason to change.

Hint 2

For question 7: look at the `src` on the `<script>` tag in `dist/index.html`. It starts with `/`, meaning "from the root of the website". Opened from the disk, there is no website, so the browser looks at the root of your drive instead.

Exercise 2 (Easy): Your bookstore, live

[Back to the exercise](#)

Hint 1

If the fix didn't work, check three things. The file is in the `public/` folder at the top of your project (next to `src/`, not inside it). Its name is exactly `_redirects`, with no extension: Windows sometimes hides a `.txt` on the end, so create it in VS Code rather than Notepad. And you deployed the `new dist`.

Hint 2

For question 7: where does the cart live? Think back to chapter 29, and to its first stretch goal. The host just hands out files. It never sees your cart at all.

Exercise 3 (Medium): The recipe finder, deployed from GitHub

[Back to the exercise](#)

Hint 1

For step 4: the variable is baked in at build time, so a missing one means the built code contains `undefined`. Your `throw` runs as soon as `api.ts` loads, so look in the Console of the live site. After adding the variable, the host must **build again**. Look for a "Redeploy" or "Trigger deploy" button, or just push a new commit.

Hint 2

For step 8: the refresh 404 only happens for addresses the router shows but no file exists for. How many different addresses does your recipe finder have? (If you added routes in the stretch goals, the answer changes.)

Exercise 4 (Medium): A robot that checks your work

[Back to the exercise](#)

Hint 1

If GitHub says the workflow file is invalid, it's almost always indentation. YAML uses spaces, never tabs, and each level must line up exactly. Compare yours with the notes, line by line.

Hint 2

If `npm ci` fails with a message about `package.json` and `package-lock.json` not being in sync, you installed or changed a package without committing the updated lock file. Run `npm install` locally, then commit `package-lock.json`.

Exercise 5 (Challenge): Ship the rest

[Back to the exercise](#)

Hint 1

For Part A: if `npm start` complains that it can't find a production build, you skipped `npm run build`. For a secret that shows up in the Sources tab: check whether its name starts with `NEXT_PUBLIC_`, and whether a Client

Component reads it.

Hint 2

For Part B: keep `tasks.ts` readable by putting the `localStorage` work in its own small file, with plain functions like "load all tasks" and "save all tasks". Each api function then picks one path or the other. That file is also the easy one to test.

Hint 3

New demo tasks need ids, and there's no json-server to make them. `crypto.randomUUID()` gives you a unique string, and it's built into every modern browser. And don't worry about TanStack Query: after a demo mutation, invalidating the query makes it call your `getTasks` again, which now just reads `localStorage`. Your optimistic `useMoveTask` keeps working too, because it only ever calls `updateTask`.

41 Final Project

Exercise 1 (Easy): Habit Tracker

[Back to the exercise](#)

Hint

Store the dates each habit was done as ISO strings, like `"2026-09-30"`. Then write `getStreak(doneDates, today)` as a pure function, and test it before it goes anywhere near a component. Try the awkward cases: nothing done yet, done today but not yesterday, and a streak that ended last week.

Exercise 2 (Easy): Study Timer

[Back to the exercise](#)

Hint 1

Don't count ticks. `setInterval` isn't exact, and a background tab slows it down. Save the moment the timer should end (`Date.now()` plus the length), and on every tick work out the time left from that. JavaScript chapter 30 explains why.

Hint 2

If the timer runs twice as fast in development, your effect isn't cleaning up its interval. `StrictMode` runs effects twice on purpose to catch exactly this ([chapter 17](#)).

Exercise 3 (Medium): Book Shelf

[Back to the exercise](#)

Hint 1

Log one response and look at it before you write a schema. The books are in `docs`, and plenty of them have no `author_name` or `cover_i` at all. Make those optional in Zod, and decide what the card shows when they're missing.

Hint 2

If the results flash empty on every keystroke, look up `placeholderData: keepPreviousData` in TanStack Query v5. It keeps showing the old results until the new ones arrive.

Exercise 4 (Medium): Flag Quiz

[Back to the exercise](#)

Hint 1

`codes.json` has more than countries in it: US states (`us-ca`), parts of the UK (`gb-eng`), and groups like `eu` and `un`. Decide what counts as a country for your quiz, and filter the list in one pure, tested function.

Hint 2

Randomness makes tests flaky. Shuffle the questions *outside* the reducer and pass them in with a "start" action, the same trick as passing `today` into `daysUntilExpiry`. Then your tests can hand the reducer a fixed list and know exactly what happens.

Exercise 5 (Medium): Expense Tracker, with charts

[Back to the exercise](#)

Hint

Store amounts in pence, as whole numbers (£12.50 is `1250`), just as the shopping cart in JavaScript chapter 12 stored cents. Adding up decimals gives surprises like `0.1 + 0.2 = 0.30000000000000004`. Every total and every chart bar is derived from the list, never stored.

Exercise 6 (Medium): Pokédex

[Back to the exercise](#)

Hint 1

The list endpoint only gives each Pokémon's name and a URL. The number at the end of that URL is its id. The picture is in the detail response: look for `official-artwork` inside `sprites`.

Hint 2

The type endpoint returns *every* Pokémon of that type in one go, with no paging. So when a type is chosen, page through that list yourself with `slice`. Your page number still lives in the URL either way.

Exercise 7 (Challenge): Week Meal Planner

[Back to the exercise](#)

Hint 1

Save only the meal **ids** in your plan, never whole recipes. That was a common mistake in [chapter 21](#): a second copy of the truth that quietly goes out of date. TanStack Query fetches and caches the details from the ids.

Hint 2

Don't try to add up the amounts. TheMealDB's measures are free text ("1 cup", "200g", "a pinch"). Group the list by ingredient name and show each measure next to it. It's honest, and it's what a real shopper needs.

Exercise 8 (Challenge): Your Portfolio Site, in Next.js

[Back to the exercise](#)

Hint 1

Keep your projects in one typed array, in a file like `projects.ts`, and use `satisfies` ([chapter 25](#)) so a typo in a field name is an error. Every page reads from that one array. Adding a project later means adding one object.

Hint 2

Each project page is a dynamic route, like `app/projects/[slug]/page.tsx`. In recent Next.js versions, `params` is a promise you `await`. Check chapter 39 and the current Next.js docs, because this is exactly the kind of detail that changes between versions.